

```
> init_module python_core
> loading libraries....
> import aprendizaje
> status: ready
> run estudiante_mode
> compiling knowledge...
> done.
```

```
> #info contingencia_diparlamentea...
> import np as np
> |logstudent_progress_tracked
```

Apuntes de PYTHON

> print("Hola, Mundo!")

para principiantes

> define_variables_data_types

Cuaderno de bitácora

> import NumPy

para nuevos programadores

```
> data_frame = pd.DataFrame(data)
> learning_rate = 0.001
> print(current_status)
> [info] module_structure_created
> [info] beginner_curriculum_ready
```

Nahuel Villafañe

```
> [log] student_progress_tracked
> for i in range(10): print(i)
```

```
> x = 5
> y = 10
> if x < y: print("Correct")
```

```
> print("Crisaef")
printi = if x 55
data_type = "Crisaef"
```

Sobre el autor

Hola, querido lector. Me presento: soy **Nahuel Villafañe**. Soy un estudiante avanzado de la carrera Licenciatura en Ciencias Fisicas en la Facultad de Ciencias Naturales y Exactas de Buenos Aires y programador. He publicado numerosos apuntes que he hecho con la inteligencia artificial y mis notas como solucionarios, resueltos y apuntes teoricos. Mi filosofia es ayudar a los demas y compartir el conocimiento mediante los apuntes que creo con la inteligencia artificial y mis notas a mano.

Este cuaderno nació de mi propia experiencia aprendiendo Python. Como toda persona que empieza desde cero, me encontré con miles de preguntas, conceptos que parecían chino, y una sensación abrumadora de "¿Por dónde empiezo? ¿Que es esto? ¿Como funciona esto?".

Por eso decidí escribir este cuaderno: para que vos no pases por lo mismo que yo. Acá está todo explicado en criollo, con dibujitos y con ejemplos para que se entienda mejor.

¿Por qué Python? Porque es el lenguaje más amigable para empezar. Porque lo usa todo el mundo. Porque con Python podés hacer casi cualquier cosa.

¿Para quién es este cuaderno? Para cualquiera que nunca haya programado. Para el que tiene miedo de empezar. Para el que ya intentó y se frustró. Para el que quiere aprender a su ritmo.

Un consejo antes de empezar: No leas este cuaderno como si fuera una novela. Programar no se aprende leyendo, se aprende **haciendo**. Hacé los ejercicios. Equivocate. Rayá el cuaderno. Volvé atrás. Releé. Ese es el verdadero camino.

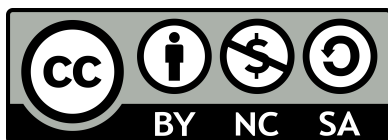
Quiero agradecer por el apoyo a mi profesor de Introduccion a Python: Antonio, le agradezco inmensamente por sus conocimientos, buena predisposicion, excelentes explicaciones y su gran ayuda. A mi padre, por haberme incentivado a estudiar esta carrera de programacion. A mis amigos que me han apoyado y dado aliento. Y a vos, querido lector, por leer mi libro que me costo dias y noche editarlo para que llegue a tu dispositivo y puedas leerlo. Gracias y espero disfrutes de este libro que costo, pero se pudo realizar.

Nahuel Villafañe

@clasesconnahu

github.com/Fisicoloco99

Manuel Villafañe - Python Paso



Licencia Creative Commons

CC BY-NC-SA 4.0

© 2026 Nahuel Villafañe
Todos los derechos reservados sobre esta obra

Esta obra está bajo una licencia
**Creat Commons Reconocimiento - No Comercial -
Compartir Igual 4.0 Internacional.**

Usted es libre de:

- **Compartir** — copiar y redistribuir el material en cualquier medio o formato
- **Adaptar** — remezclar, transformar y construir a partir del material

Bajo las siguientes condiciones:

- **Reconocimiento (BY)** — Debe reconocer adecuadamente la autoría, proporcionar un enlace a la licencia e indicar si se han realizado cambios.
- **No Comercial (NC)** — No puede utilizar el material para fines comerciales.
- **Compartir Igual (SA)** — Si remezcla, transforma o construye a partir del material, debe distribuir su contribución bajo la misma licencia del original.

La licencia completa está disponible en:

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Python paso a paso
Nahuel Villafañe

Índice

Manuel Villafañe - Python Paso

Fundamentos e Instalación

Instalación de Python y Entornos de Desarrollo

¿Qué es Python?

p. 30

IDEs y Editores de Texto

p. 31

Tabla comparativa de IDEs

p. 31

Instalación de IDLE

p. 38–39

Instalación de Visual Studio Code

p. 39–40

Instalación de otros IDEs (PyCharm, Thonny, Spyder, etc.)

p. 40–41

Conceptos Básicos: Programas, Compiladores e Intérpretes

¿Qué es un programa?

p. 47

Compilador vs Intérprete

p. 47–48

Ejecución de programas en diferentes IDEs

p. 50–52

Tabla resumen de ejecución

p. 52

Elementos del Lenguaje

Variables y Tipos de Datos

Definición de variable

p. 59

Reglas para nombrar variables

p. 59–60

Tipos de variables (int, float, str)

p. 60

Ejemplo práctico: suma de números

p. 62–63

Ejercicios propuestos

p. 64–65

Cadenas de Caracteres (Strings)

Definición y operaciones básicas

p. 69

Asignación y concatenación

p. 69–70

Búsqueda con find()

p. 70–71

Extracción de subcadenas (slicing)

p. 71

Comparación de cadenas

p. 72

Ejercicios resueltos

p. 72–76

Palabras Reservadas

Lista de palabras reservadas

p. 78

Errores comunes y cómo evitarlos

p. 78–79

Ejercicios resueltos

p. 80–83

Operadores Aritméticos

Suma, resta, multiplicación, división

p. 84–85

Exponente (**), división entera (//) y módulo (%)

p. 86

Tabla resumen de operadores

p. 87

Ejercicios resueltos

p. 88–92

Operadores Relacionales (de Comparación)

Los seis operadores (<, >, ==, !=, <=, >=)

p. 122–123

Comparación de números y texto

p. 123

Uso en condicionales

p. 123–124

Ejercicios resueltos

p. 125–129

Operadores Lógicos

and, or, not

p. 130–133

Combinación de condiciones

p. 133

Ejercicios resueltos

p. 134–138

Operadores de Asignación

Asignación simple (=)

p. 139

Asignación compuesta (+=, -=, *=, etc.)

p. 139–141

Ejercicios resueltos

p. 142–146

Los parámetros sep y end

El parámetro end

p. 147–148

El parámetro sep

p. 149

Combinando sep y end

p. 149–151

Ejercicios resueltos

p. 151–154

Manuel Villafañe - Python paso a paso

Estructuras de Control

Sentencias Condicionales

Condicional simple: if

p. 93–96

Operadores de comparación

p. 94

Condicional compuesta: if-else

p. 101–104

Condicional múltiple: if-elif-else

p. 110–114

Ejemplos con promedio de notas

p. 95, 103–104

Ejercicios resueltos

p. 97–100, 105–109, 115–121

Bucles o Ciclos

Bucle while

p. 155–159

Ejemplos: suma, validación, factorial

p. 158–159

Sentencias break y continue

p. 166–168

La función range()

p. 308–312

Bucle for

p. 319–322

Recorrer listas y cadenas con for

p. 320–321

Bucles anidados

p. 321–322, 329–334

Ejercicios resueltos

p. 160–165, 170–175, 314–317, 322–327, 335–345

Estructuras de Datos

Listas

Creación y acceso por índice

p. 347–348

Rebanadas (slicing)

p. 348–349

Modificar elementos

p. 349

Recorrer listas con for

p. 349–350

Métodos: `append()`, `insert()`, `extend()`

p. 350

Métodos: `pop()`, `remove()`, `clear()`

p. 351

Ordenamiento: `sort()`, `reverse()`, `sorted()`

p. 352

Búsqueda: `index()`, `count()`, `in`

p. 352–353

Listas paralelas y carrito de compras

p. 354–355

Ejercicios resueltos

p. 356–364

Diccionarios

Creación y acceso por clave

p. 381–383

Listas de diccionarios

p. 384

Recorrido: `.items()`, `.keys()`, `.values()`

p. 385–386, 403–406, 415–418, 515–518

Métodos: `get()`, `setdefault()`, `update()`

p. 471–474, 494–496, 505–508

Métodos: `pop()`, `popitem()`, `clear()`

p. 385, 483–485, 426–428

Método `fromkeys()`

p. 449–452

Ejercicios clave para dominar diccionarios

p. 529–535

Ejemplos prácticos: login, inventario, agenda

p. 387–389, 405–406, 541–542

Otras Estructuras

Tuplas (inmutabilidad, empaquetado, desempaquetado)

p. 741–745

Sets (conjuntos, operaciones de teoría de conjuntos)

p. 752–759

Funciones y Módulos

Funciones

Definición y utilidad

p. 366

Funciones puras vs impuras

p. 367–369

Parámetros y return

p. 368–369

Ejemplos: calculadora, cajero, lista de compras

p. 371–374

Ejercicios resueltos

p. 375–380

Módulos Importantes de la Biblioteca Estándar

Módulo random

p. 553–558

Módulo string

p. 575–577

Módulo os

p. 591–599, 600–615, 616–640

Módulo shutil

p. 815–824

Módulo datetime

p. 770–778

Módulo math

p. 779–788

Módulo re	p. 790–800
Módulo pathlib	p. 802–813
Módulo csv	p. 849–860
Módulo json	p. 861–873
Módulo copy	p. 930–937
Módulo sys	p. 932–933
Módulo collections	p. 825–836
Módulo itertools	p. 837–848
Entornos virtuales (venv)	p. 874–884
Manejo de Excepciones	
Try, except, else, finally	p. 673–677
Excepciones comunes	p. 641–646
Creación de excepciones personalizadas	p. 885–888

Ejercicios de manejo de errores

p. 647–655, 677–685

Manuel Villafañe - Python paso a paso

Métodos de Strings y Formateo

Métodos de Strings

`rstrip()`, `lstrip()`

p. 205–209

`title()`, `istitle()`

p. 216–220

`islower()`, `isupper()`, `lower()`, `upper()`

p. 228–232

`swapcase()`

p. 241–244

`capitalize()`

p. 255–258

Alineación: `center()`, `ljust()`, `rjust()`

p. 271–274

`count()`

p. 285–289

`startswith()`, `endswith()`

p. 296–299

Formateo de Cadenas

Método `format()`

p. 185–188

F-strings

p. 194–198

Parámetros `sep` y `end` de `print()`

p. 147–151

Proyectos, Compilación y Calidad de Código

Proyectos Integradores

Sistema de gestión de alumnos y notas

p. 706–709

Carrito de compras

p. 710–711

Cajero automático

p. 712–713

Calculadora con menú

p. 713–714

Juego del ahorcado (dos versiones)

p. 714–717, 729–734

Piedra, papel o tijera

p. 717–719

Generador de contraseñas

p. 719–721

PyInstaller: Creación de Ejecutables

¿Qué es PyInstaller?

p. 687

Guía paso a paso (Windows, Linux, Mac)

p. 688–691

Opciones: `--onefile`, `--windowed`, `--icon`

p. 691

Ejercicios con PyInstaller

p. 693–703

Depuración y Logging

Depurador pdb

p. 896–897

Módulo logging

p. 898–899

Ejercicios

p. 899–903

Buenas Prácticas y Estilo

PEP 8 (nombres, indentación, líneas)

p. 916–919

Docstrings

p. 920–921

Punto de entrada: `if __name__ == "__main__":`

p. 905–908

Argumentos de línea de comandos con `sys.argv`

p. 906–908

Módulo `copy` y más sobre `sys`

p. 930–933

Técnicas Avanzadas

Comprensión de listas, diccionarios y sets

p. 761–763

Función `len()`

p. 176–179

Recursividad

p. 939–942

Manuel Villafañe - Python paso a paso

Recursos y Cierre

Guía de Recursos para Seguir Aprendiendo

Documentación oficial y PEPs

p. 958–959

Libros recomendados

p. 961–962

Cursos online (gratis y de pago)

p. 962–963

Canales de YouTube

p. 963–964

Comunidades y foros

p. 964

Herramientas esenciales (VS Code, Git, pip, black)

p. 965

Librerías esenciales (numpy, pandas, requests, flask)

p. 959–961

Plataformas para practicar (Codewars, Exercism, LeetCode)

p. 966

Plan de estudio post-libro

p. 967

Tablas y Resúmenes Útiles

Métodos de diccionarios

p. 386, 496, 500

Operadores aritméticos y relacionales

p. 87, 122

Errores comunes

p. 641–646, 984

Comandos útiles para terminal y entornos virtuales

p. 875–876, 983

Sobre el autor y licencia

Nahuel Villafañe

p. 2

Licencia Creative Commons

p. 3

Instalación de Python

¿Qué es Python?

Arranquemos por el principio. Cuando yo di mis primeros pasos, me acuerdo que me decían “Python es interpretado, de tipado dinámico, multiparadigma...” y yo ponía cara de “¿qué carajo me estás diciendo?”. Así que dejame explicártelo en criollo, sin palabras raras.

Imaginate que querés enseñarle a un robot cómo preparar un mate. Le das una lista de pasos bien claros: calentá agua, poné la yerba, cebá. Programar es más o menos eso: darle instrucciones a la computadora. Python es uno de los idiomas que entiende, y tiene unas cuantas características que lo hacen muy piola:

- **Interpretado:** La computadora lee y ejecuta tu código línea por línea, ahí mismo. No necesita un “traductor” previo que lo convierta todo de golpe. Esto es genial porque si te mandás una macana, Python te frena en el acto y te dice “che, acá hay un error”. Es como un amigo que te corrige mientras hablás, en lugar de esperar a que termines todo el discurso.
- **De tipado dinámico:** No tenés que declarar si una variable es número, texto o lo que sea. Python se da cuenta solo cuando el programa corre. Más adelante lo vas a ver en acción, pero por ahora quedate con que escribís menos y te concentrás en lo importante.
- **Multiplataforma:** Funciona en Windows, Linux, Mac... como un auto que puede andar en cualquier calle del mundo sin modificarle nada. Lo mismo el código Python: lo escribís una vez y corre en cualquier sistema.
- **Multiparadigmático:** Soporta varios estilos de programación (funcional, orientado a objetos, imperativo). No te asustes con los nombres, de a poco los vas a ir conociendo.

¿Y dónde escribo mi código?

Cuando arranqué, lo primero que pensé fue: “Bueno, abro el Bloc de notas y listo”. Escribí `print("hola")`, guardé el archivo como `hola.txt` y... no pasó nada. Resulta que además del texto, necesitás un entorno que sepa ejecutar ese código. Ahí entran los **editores de texto** y los **IDEs**.

¿Qué es un IDE?

Un **IDE** (Entorno de Desarrollo Integrado) es como una navaja suiza: tiene el cuchillo (editor), la lupa (depurador para encontrar errores), el destornillador (terminal integrada) y mil chiches más, todo junto. Un **editor de texto** es solo el cuchillo: escribe, pero no te ayuda en nada más.

IDE = Editor + Depurador + Terminal + Extras
Ejemplo: PyCharm, VS Code (con extensiones), IDLE, Spyder, Thonny
Editor = Solo escribe texto
Ejemplo: Bloc de notas, Gedit, Sublime Text

Lista completa de IDEs y editores para Python

Acá te muestro una tabla con los más importantes. Después te explico cada uno con más detalle.

NOMBRE	TIPO	CARACTERISTICA
IDLE	IDE básico	Viene con Python, ideal para empezar
Visual Studio Code	Editor + IDE (con ext.)	El más popular, gratuito y súper potente
PyCharm	IDE completo	El favorito de profesionales, versión gratuita y de pago
Jupyter Notebook / JupyterLab	Entorno web	Perfecto para ciencia de datos y aprendizaje
Google Colab	Entorno web	Jupyter en la nube, gratis, no necesita instalación
Spyder	IDE científico	Ideal para ingenieros y científicos de datos
Thonny	IDE educativo	Diseñado para principiantes absolutos
Sublime Text	Editor rápido	Extremadamente rápido, muy customizable
Atom	Editor	Hecho por GitHub, fácil de usar (ya no tiene soporte oficial)

Vim / Neovim	Editor (terminal)	Para expertos, se usa desde la terminal, muy potente
Emacs	Editor (terminal)	El "eterno rival" de Vim, extremadamente personalizable
Eclipse + PyDev	IDE completo	Si ya usás Eclipse para Java, le agregás Python
NetBeans	IDE completo	Similar a Eclipse, soporta varios lenguajes
Komodo IDE	IDE completo	De pago, pero muy completo (versión gratis limitada)
Wing IDE	IDE completo	Especializado en Python, versión gratuita y de pago
Eric	IDE completo	Código abierto, hecho en Python para Python
Mu Editor	IDE educativo	Simple, para enseñar a chicos y principiantes
Replit	IDE en la nube	Programás desde el navegador, sin instalar nada
Codeanywhere	IDE en la nube	Similar a Replit, con terminal integrada
GitHub Codespaces	IDE en la nube	Entorno completo en la nube de GitHub
CodeSandbox	IDE en la nube	Principalmente para web, pero también soporta Python
Cursor	Editor IA	Basado en VS Code, con inteligencia artificial
Zed	Editor rápido	Nuevo editor, muy rápido, hecho en Rust
Lapce	Editor rápido	Alternativa open source a Zed, también en Rust
Bluefish	Editor	Ligero, para múltiples lenguajes
Geany	IDE ligero	Muy liviano, ideal para PCs viejas
Ninja IDE	IDE ligero	Simple y rápido, multiplataforma

Rodeo	IDE científico	Especializado en ciencia de datos (ya no se actualiza)
Spyder	IDE científico	Ya lo mencionamos, es el mejor para datos
Deepnote	IDE en la nube	Para ciencia de datos, colaborativo
Noteable	IDE en la nube	Similar a Deepnote, para notebooks
Hex	IDE en la nube	Para análisis de datos colaborativo
DataSpell	IDE científico	De JetBrains (los de PyCharm), especializado en datos
Bloc de notas (Windows)	Editor básico	Viene con Windows, solo para escribir código sin ayudas
Gedit	Editor	Editor por defecto de Linux (Gnome), simple y funcional
TextEdit	Editor	Editor por defecto de Mac (cuidado con el formato rico)
Notepad++	Editor	Muy popular en Windows, ligero y con resaltado de sintaxis

Explicación detallada de los más importantes

Ahora te cuento uno por uno los más relevantes, así sabés cuál elegir.

IDLE

IDLE es el entorno que viene **incluido con Python**. Cuando instalás Python, se instala solito. Es simple, sin lujos, pero cumple. Como la bicicleta con rueditas de aprendizaje: perfecta para dar las primeras pedaladas.

Ventajas:

- No necesitás instalar nada adicional.
- Es fácil de usar, tiene intérprete interactivo y editor de código.

Desventajas:

- Es básico, le faltan funciones avanzadas.
- Visualmente es bastante feúcho.

¿Para quién es? Para principiantes que recién empiezan.

```
+-----+
| IDLE (Integrated Development Environment) |
|                                           |
| Viene con Python. No se descarga aparte. |
| Perfecto para empezar.                  |
|                                           |
| +-----+                               |
| | Python 3.12.0 Shell                    |
| | >>> print("Hola")                     |
| | Hola                                  |
| | >>>                                   |
| +-----+                               |
+-----+
```

Visual Studio Code (VS Code)

VS Code es el editor más popular hoy. Es gratuito, de código abierto, y con extensiones se convierte en un IDE superpoderoso. Es como un auto al que le podés agregar el stereo, el GPS y los asientos de cuero. Para Python, la extensión que más vas a usar es la de Microsoft.

Ventajas:

- Gratuito, rápido, con terminal integrada.
- Miles de extensiones (Python, Git, Docker, etc.).
- Te sincroniza con GitHub.

Desventajas:

- Hay que instalar las extensiones (aunque es muy fácil).
- Consume más memoria que un editor simple.

¿Para quién es? Para todos: principiantes, avanzados, profesionales. Yo lo uso a diario y te lo recomiendo.

PyCharm

PyCharm es el IDE de JetBrains, una empresa que hace herramientas de mucha calidad. Es como un taller mecánico completo: todo integrado, sin necesidad de agregarle nada. Tiene versión Community (gratis) y Professional (paga).

Ventajas:

- Todo viene listo: autocompletado impecable, detección de errores al toque.
- Muy bueno para proyectos grandes.
- Integración con bases de datos y frameworks web.

Desventajas:

- La versión gratuita tiene menos funciones.

- Consume bastante memoria RAM; en computadoras viejas puede arrastrarse.
- Para alguien que recién empieza, puede ser abrumador.

¿Para quién es? Para programadores avanzados y proyectos profesionales.

Jupyter Notebook / JupyterLab

Jupyter es un entorno que se abre en el navegador. Te permite mezclar código, texto, imágenes y gráficos en un mismo documento (llamado **notebook**). Es como un cuaderno de laboratorio digital: ejecutás una celda, ves el resultado, escribís una anotación y seguís.

Ventajas:

- Ideal para ciencia de datos y aprendizaje.
- Ejecutás partes del código por separado, muy visual.

Desventajas:

- Para scripts muy largos o proyectos grandes se vuelve incómodo.

¿Para quién es? Científicos de datos, estudiantes, docentes.

Google Colab

Es Jupyter pero en la nube de Google. No instalás absolutamente nada, solo necesitás un navegador y una cuenta de Google. Viene con GPU gratis para machine learning. Una joyita.

Ventajas:

- Gratuito, sin instalación, accedés desde cualquier lado.
- Se guarda automáticamente en Google Drive.

Desventajas:

- Necesitás internet sí o sí.
- Tiene límites en el tiempo de ejecución.

¿Para quién es? Para cualquier persona que quiera probar Python al instante, o para proyectos de machine learning.

Spyder

Está pensado para científicos e ingenieros. Tiene un explorador de variables como el de MATLAB, y una terminal IPython integrada. Si venís de ese mundo, te va a resultar familiar.

¿Para quién es? Científicos, ingenieros, académicos.

Thonny

Thonny es un IDE hecho para principiantes absolutos. Te muestra paso a paso cómo se ejecuta el código, y tiene un depurador visual que es una maravilla. Si nunca tocaste código en tu vida, este es tu lugar.

¿Para quién es? Personas que nunca programaron antes.

Sublime Text

Es un editor de texto rapidísimo. Abre al toque y consume pocos recursos. Ideal si tenés una máquina modesta. La versión completa es de pago, pero podés evaluarlo todo el tiempo que quieras.

¿Para quién es? Programadores que valoran la velocidad y la simplicidad.

Vim / Neovim

Se usa desde la terminal. La curva de aprendizaje es un paredón, pero una vez que lo dominás, volás. Todo con el teclado, sin mouse. Es el favorito de muchos administradores de sistemas.

¿Para quién es? Expertos que quieren máxima eficiencia (y no le tienen miedo a la terminal).

Emacs

El eterno rival de Vim. También se maneja desde la terminal, pero es mucho más personalizable (casi podés hacer cualquier cosa dentro de Emacs). Tiene versión gráfica.

¿Para quién es? Expertos que quieren control total sobre su entorno.

Eclipse + PyDev

Si ya usás Eclipse para Java, podés agregar el plugin PyDev y programar en Python en el mismo lugar. Útil para no tener que cambiar de herramienta.

¿Para quién es? Programadores que ya usan Eclipse para otros lenguajes.

Editores simples (Bloc de notas, Gedit, TextEdit, Notepad++)

Son editores de texto pelados. No tienen autocompletado ni depurador, pero sirven para escribir código si no querés instalar nada más. Eso sí, asegurate de guardar los archivos con extensión `.py` y con codificación UTF-8.

¿Para quién es? Para una prueba rápida o si no querés instalar nada.

IDE en la nube (sin instalar nada)

Si no podés o no querés instalar software, estos servicios te permiten programar Python desde el navegador:

- **Replit:** Muy popular, entorno completo con terminal.
- **Google Colab:** Notebooks con GPU gratis.
- **GitHub Codespaces:** Un VS Code en la nube (requiere plan de pago para uso intensivo).
- **Codeanywhere, Deepnote, Noteable, Hex:** todos enfocados en ciencia de datos y colaboración.

```
+-----+
|  IDE EN LA NUBE: programás desde el navegador  |
|  |                                              |
|  Ventajas:                                     |
|  |                                              |
```

	- No instalás nada	
	- Accedés desde cualquier computadora	
	- Todo se guarda en la nube	
	Desventajas:	
	- Necesitás internet	
	- Puede ser lento	
	- Algunos tienen límites de uso	
+-----+		

Editores modernos nuevos

Últimamente aparecieron editores muy rápidos escritos en Rust:

- **Cursor:** Basado en VS Code, pero con inteligencia artificial integrada que te ayuda a escribir código.
- **Zed:** Súper rápido, minimalista.
- **Lapce:** Alternativa open source a Zed, también en Rust.

¿Cuál me conviene?

Eso depende de vos, pero acá te dejo una guía basada en mi experiencia (y en la de mucha gente):

+-----+	
SI SOS...	TE RECOMIENDO...
+-----+	
Principiante absoluto	IDLE (viene con Python) o
(nunca programaste)	Thonny
+-----+	
Principiante con ganas de	Visual Studio Code
aprender bien	
+-----+	
Estudiante de ciencia de	Jupyter Notebook / Google Colab
datos / machine learning	o Spyder
+-----+	
Programador profesional	PyCharm (versión Professional
	si podés pagar)
+-----+	
Tu PC es vieja / lenta	Sublime Text, Geany,
	Notepad++
+-----+	
Querés programar desde	Replit, Google Colab,
cualquier lugar sin instalar	GitHub Codespaces
+-----+	
Querés aprender a usar la	Vim o Emacs (pero preparate
terminal como un experto	para sufrir al principio)
+-----+	

Yo, personalmente, aprendí con **Visual Studio Code** y todavía lo uso a diario. Me parece el mejor equilibrio entre facilidad, potencia y popularidad. Pero lo importante es que elijas uno y te pongas a escribir código.

Reflexión: ¿Ya tenés una idea de cuál vas a usar? Anotá en un papel (o en un archivo de texto) cuál elegirías según lo que leíste y por qué. No hace falta que sea la decisión definitiva, podés cambiar más adelante.

Instalación de Python (y de paso IDLE)

Ahora sí, manos a la obra. Vamos a instalar Python. Como te conté, IDLE viene en el paquete, así que al instalar Python ya lo tenés.

Anécdota personal: Cuando instalé Python por primera vez en Windows, no marqué la bendita casilla de “Add Python to PATH”. Pasé tres horas probando por qué python no funcionaba en la terminal. Era desesperante. No te comas ese garrón; prestá atención a ese paso.

Método en Windows

1. **Visitar el sitio oficial:** Entrá a python.org. La página suele detectar que estás en Windows y te muestra un botón enorme de descarga. Bajate el archivo `.exe` de la última versión.
2. **Ejecutar el instalador:** Hacé doble clic en el `.exe`. ¡Ojo acá! Antes de hacer clic en “Install Now”, **marcá la casilla que dice “Add Python to PATH”**. Esa casilla es sagrada.

```
+-----+
| Python 3.12.0 (64-bit) Setup |
|                               |
| Install Python 3.12.0 (64-bit) |
|                               |
| [-] Install launcher for all users (recommended) |
| [X] Add Python to PATH <---- MARCALA |
|                               |
| [ Customize installation ] [ Install Now ] |
+-----+
```

3. **Verificar la instalación:** Cuando termine, abrí el menú de inicio y buscá “IDLE”. Debería aparecer. También podés abrir la terminal (cmd), escribir `idle` y presionar Enter.

```
+-----+
| Python 3.12.0 Shell |
+-----+
| Python 3.12.0 (tags/v3.12.0:... ) on win32 |
| Type "help", "copyright", "credits" or "license" |
| for more information. |
|                               |
| >>> _ |
+-----+
```

Método en macOS

1. Bajá el instalador .pkg desde python.org.
2. Seguí los pasos del asistente (siguiente, siguiente, siguiente).
3. Abrió IDLE desde la carpeta Aplicaciones o usando Spotlight.

Método en Linux (Ubuntu)

1. Abrió una terminal con `Ctrl+Alt+T`.
2. Actualizá la lista de paquetes: `sudo apt update`.
3. Instalá IDLE: `sudo apt install idle3`.

```
usuario@ubuntu:~$ sudo apt update
usuario@ubuntu:~$ sudo apt install idle3
Do you want to continue? [Y/n] y
```

Instalación de Visual Studio Code

A diferencia de IDLE, VS Code no viene con Python. Hay que descargarlo e instalarlo aparte. Pero es muy fácil, ya vas a ver.

Paso 1: Descargar

Andá a code.visualstudio.com y descargá el instalador que corresponda a tu sistema operativo.

Paso 2: Instalar

- **En Windows:** Ejecutá el .exe, aceptá la licencia, elegí la carpeta y, de nuevo, marcá la opción "Add to PATH" para que puedas abrirlo desde la terminal.

```
+-----+
| Visual Studio Code Setup |
| Select Additional Tasks  |
| [X] Create a desktop icon |
| [X] Add "Open with Code" action |
| [X] Add to PATH (requires restart) |
| [ Back ] [ Next ] [ Cancel ] |
+-----+
```

- **En macOS:** Arrastrá VS Code a la carpeta Aplicaciones. Después, abrí VS Code, presioná `Cmd+Shift+P` y ejecutá "Shell Command: Install 'code' command in PATH".
- **En Linux:** Descargá el archivo .deb y hacé doble clic, o ejecutá `sudo dpkg -i nombre.deb`.

Paso 3: Abrir VS Code

Buscá el icono y abrílo. Te va a aparecer una pantalla de bienvenida.

Paso 4: Instalar la extensión de Python

En la barra izquierda, hacé clic en el ícono de Extensiones (son cuatro cuadraditos). Buscá “Python” e instalá la que dice “Microsoft” (la que tiene más descargas).

Paso 5: Probar que funcione

Creá un archivo nuevo llamado `hola.py`, escribí `print("Hola")` y ejecutalo con el botón de Play (o con `Ctrl+F5`). Si ves “Hola” en la terminal, ¡está todo listo!

Paso 6: Festejar

```
+-----+
|               |
|  [OK] INSTALACION COMPLETA [OK]  |
|               |
|  Ya tenés Python y VS Code listos!  |
|               |
|  Ahora sí... ¡a programar!  |
|               |
+-----+
```

Instalación de otros IDEs (resumen)

No voy a hacer una guía paso a paso de todos porque sería interminable, pero te tiro los comandos y pasos clave para los más importantes:

PyCharm

1. Andá a jetbrains.com/pycharm.
2. Descargá la versión Community (gratis).
3. Ejecutá el instalador y seguí los pasos.

Thonny

1. Andá a thonny.org.
2. Descargá el instalador para tu sistema.
3. Ejecutalo y listo.

Spyder

1. La forma más fácil es instalar **Anaconda** (un paquete de herramientas para ciencia de datos).
2. Si ya tenés Python, podés usar: `pip install spyder`.

Jupyter Notebook

1. Con pip: `pip install notebook`.
2. Ejecutar: `jupyter notebook`.

Google Colab

1. No instalás nada. Entrá a `colab.research.google.com`.
2. Iniciá sesión con Google.
3. Creá un notebook nuevo y empezá a escribir código.

Sublime Text

1. Andá a `sublimetext.com`, descargalo e instalalo.
2. Para ejecutar Python, instalá el paquete “SublimeREPL” o usá la terminal externa.

Replit

1. Entrá a `replit.com`, creá una cuenta gratis.
2. Hacé clic en “Create Repl”, elegí Python y a programar.

Dale, repasemos rápido lo importante:

- Python es un lenguaje interpretado, de tipado dinámico y multiplataforma.
- Un IDE es un entorno que integra editor, depurador, terminal y más. Un editor solo escribe texto.
- IDLE viene con Python, es ideal para empezar.
- VS Code es mi recomendación personal para la mayoría.
- Existen IDEs en la nube que no requieren instalar nada.
- Al instalar en Windows, nunca te olvides de marcar “Add Python to PATH”.

Ejercicios del capítulo

Antes de avanzar al próximo tema, pongamos a prueba lo que aprendiste. Te propongo 20 ejercicios. Intentá resolverlos vos solo, sin mirar las soluciones. Equivocarse es parte del juego, pero si te trabás, volvé a leer la sección correspondiente.

1. Definí con tus propias palabras qué es un **IDE** y mencioná al menos tres ejemplos.
2. Explicá la diferencia entre un editor de texto y un IDE. Da un ejemplo de cada uno.
3. ¿Por qué IDLE es una buena opción para alguien que recién empieza a programar?
4. Escribí los pasos exactos para instalar Python (y por lo tanto IDLE) en Windows, sin olvidar la casilla importante.
5. ¿Qué comando usarías en una terminal de Ubuntu para instalar IDLE?
6. Imaginá que tu amigo nunca oyó hablar del PATH. Explicá con una analogía de la vida diaria qué significa “Agregar Python al PATH” y por qué es importante.
7. Nombra al menos cuatro ventajas de Visual Studio Code.
8. ¿Qué son las extensiones en VS Code y por qué necesitás instalar la extensión de Python?
9. Describí cómo se abre la terminal integrada en VS Code.
10. Mencioná tres IDEs o plataformas que permiten programar Python desde el navegador, sin instalar nada en la computadora.
11. ¿Cuál es la principal ventaja de usar un IDE en la nube? ¿Y la principal desventaja?
12. Tenés una computadora con Windows 7 y 1 GB de RAM. ¿Qué editor o IDE le recomendarías a un amigo para aprender Python? Justificá tu respuesta.
13. Explicá con un ejemplo concreto qué significa que Python sea **multiplataforma**.
14. ¿Cuál es la diferencia entre PyCharm Community y PyCharm Professional?
15. Después de instalar Python, abrí IDLE y escribí un pequeño script que muestre “Hola, mundo!”. Contame paso a paso qué hiciste y qué apareció en pantalla.
16. ¿Para qué tipo de usuario está especialmente pensado Jupyter Notebook? ¿Qué lo hace especial?
17. Si quisieras instalar Spyder mediante el administrador de paquetes pip, ¿qué comando deberías escribir en la terminal?
18. Buscá (o recordá) cómo se instala Thonny en macOS y escribí los pasos resumidos.
19. Cuando instalás Python en Windows y marcás la opción “Add Python to PATH”, ¿qué se modifica en el sistema? Explicálo de forma sencilla.
20. Estás en una biblioteca y la computadora pública no te permite instalar programas. ¿Cómo podrías programar en Python igual? Mencioná al menos dos alternativas.

Soluciones a los ejercicios

Ahora sí, vamos con las soluciones. Comparalas con lo que pensaste. Si hay diferencias, volvé a la sección correspondiente y repasá. Esa es la mejor forma de fijar el conocimiento.

Ejercicio 1: ¿Qué es un IDE?

Un **IDE** (Entorno de Desarrollo Integrado) es como un taller de mecánica completo: junta el editor de texto, el depurador (que te ayuda a encontrar errores), una terminal integrada y muchas herramientas más, todo en el mismo programa. Tres ejemplos: **PyCharm**, **Visual Studio Code** (con las extensiones adecuadas) y **Thonny**.

Ejercicio 2: Diferencia entre editor e IDE

Un editor de texto es como una hoja en blanco: solo escribís. Un IDE es el taller con todas las herramientas juntas. Ejemplo de editor: **Bloc de notas**. Ejemplo de IDE: **PyCharm**.

Ejercicio 3: ¿Por qué IDLE para principiantes?

Porque viene incluido con Python, no requiere instalar nada extra y su interfaz es muy simple. Es ideal para concentrarse en aprender a programar sin distracciones.

Ejercicio 4: Pasos para instalar Python en Windows

1. Entrá a python.org, descargá el archivo `.exe`.
2. Hacé doble clic en el instalador.
3. **Antes de continuar, marcá la casilla Add Python to PATH.**
4. Clic en **Install Now** y esperá.
5. Buscá **IDLE** en el menú de inicio o abrí `cmd` y escribí `idle`.

Ejercicio 5: Comando para instalar IDLE en Ubuntu

```
sudo apt update
sudo apt install idle3
```

Respondé y cuando te pregunte si querés continuar.

Ejercicio 6: Analogía del PATH

Imaginá que estás en un shopping enorme. Cada local es un programa. Si querés ir al local “Python”, pero no sabés en qué piso está, vas a dar vueltas sin encontrarlo. Agregar Python al PATH es como poner un cartel gigante en la entrada que dice “Python → Piso 3, Local 15”. Así, desde cualquier parte del shopping (cualquier carpeta de tu compu) podés ejecutar el programa sin escribir la dirección completa. Si no lo agregás, el sistema te dice “no encuentro ese programa” y tenés que escribir la ruta entera cada vez. ¡Un bajón!

Ejercicio 7: Ventajas de Visual Studio Code

- Es gratuito y de código abierto.
- Tiene miles de extensiones, lo vuelven superflexible.
- Incluye una terminal integrada.
- Es liviano y rápido.
- Se sincroniza con GitHub y otras herramientas.

Ejercicio 8: Extensiones y la de Python

Las extensiones son como accesorios que le agregás a un taladro: le dan funciones que no trae de fábrica. VS Code por sí solo es un editor de texto, pero al instalar la extensión de Python se convierte en un IDE para Python (autocompletado, detección de errores, ejecución de código, etc.).

Ejercicio 9: Abrir terminal integrada en VS Code

Hay dos maneras:

- Atajo de teclado: `Ctrl + `` (Windows/Linux) o `Cmd + `` (Mac).
- Menú: `Ver → Terminal`.

Ejercicio 10: IDEs en la nube

- **Google Colab**: notebooks en línea, con GPU gratis.
- **Replit**: entorno completo con terminal y editor.
- **GitHub Codespaces**: un VS Code en la nube.

Ejercicio 11: Ventaja y desventaja de los IDE en la nube

Ventaja principal: no necesitás instalar nada, accedés desde cualquier computadora con internet. Desventaja: dependés de la conexión; si internet es lento o se corta, no podés trabajar. Además, algunos servicios limitan el tiempo de uso.

Ejercicio 12: Recomendación para PC antigua

Con 1 GB de RAM, un IDE pesado como PyCharm dejaría la máquina inusable. Recomendaría **Sublime Text** (muy rápido y liviano), **Geany** (IDE ligero) o **Notepad++** con la terminal de Windows aparte.

Ejercicio 13: Significado de multiplataforma

Significa que el mismo código Python funciona sin cambios en Windows, macOS y Linux. Por ejemplo, un script que imprime “Hola” lo ejecutás en una PC con Windows y después en una Mac, y funciona exactamente igual. Es como un auto que puede circular en cualquier país sin modificarle nada.

Ejercicio 14: PyCharm Community vs Professional

PyCharm Community es la versión gratuita, con soporte para Python puro y desarrollo web básico. Es suficiente para aprender y para muchos proyectos reales. PyCharm Professional es de pago e incluye soporte para frameworks web avanzados, bases de datos y herramientas científicas. Es como el taller del mecánico especializado, con máquinas extra que el aficionado no necesita.

Ejercicio 15: “Hola, mundo!” en IDLE

1. Abrí IDLE desde el menú de inicio.
2. Apareció la ventana de la shell de Python con el prompt `>`.

Escribí `print("Hola, mundo!")` y presioné Enter.

En la línea siguiente apareció `Hola, mundo!` (sin las comillas). Eso es la salida.

Si creás un archivo nuevo (File >New File), pegás el mismo código y lo ejecutás con F5, el resultado es el mismo.

Ejercicio 16: Para quién es Jupyter Notebook

Está pensado para científicos de datos, investigadores y docentes. Lo especial es que mezcla celdas de código con texto, imágenes y gráficos en un solo documento. Podés ejecutar una celda, ver el resultado, escribir una explicación y seguir. Es como un cuaderno de laboratorio digital.

Ejercicio 17: Instalar Spyder con pip

Abrí la terminal y escribí:

```
pip install spyder
```

Después, para lanzarlo, escribí `spyder` y Enter.

Ejercicio 18: Instalar Thonny en macOS

1. Entrá a thonny.org.
2. Bajá el archivo `.dmg`.
3. Abrí el `.dmg` y arrastrá el ícono de Thonny a la carpeta Aplicaciones.
4. Listo, ya lo podés abrir desde Launchpad.

Ejercicio 19: ¿Qué modifica “Add Python to PATH”?

Agrega la carpeta donde se instaló Python (algo como `C:\Users\tuUsuario\AppData\Local\Programs`) a una variable del sistema llamada `PATH`. Esta variable es una lista de carpetas donde Windows busca los programas cuando escribís un comando. Así, podés escribir `python` en cualquier terminal y Windows sabe dónde encontrarlo sin que tengas que escribir la ruta completa.

Ejercicio 20: Programar sin instalar nada

Si la computadora pública no te deja instalar, usá entornos en la nube:

- **Google Colab:** solo necesitás un navegador y una cuenta de Google.
- **Replit:** creás una cuenta gratuita y tenés un entorno completo.

Ambos te permiten escribir, ejecutar y guardar tu código sin tocar el disco rígido de la máquina. ¡Magia pura!

La instalación es el primer paso. Elegir el editor correcto es el segundo. El tercero es nunca parar de programar.

¿Qué es un programa?

Definición de programa

Un programa es una serie de instrucciones previamente codificadas, las cuales permiten realizar una tarea específica en una computadora.

A estas series de instrucciones se la conoce como **código fuente**. El cual es escrito por personas conocidas como **programadores**. El código debe ser escrito en un lenguaje de programación que en este caso será Python, que posteriormente puede ser convertido a un archivo ejecutable.

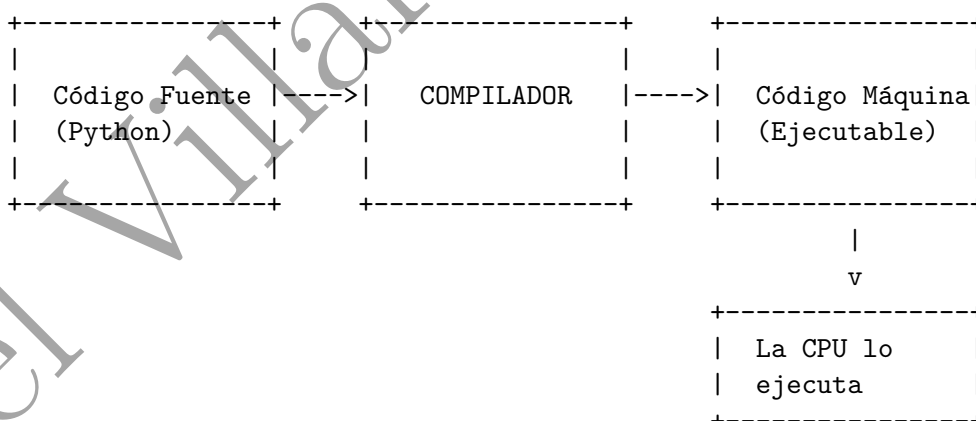
Para que el código fuente pueda ser convertido en un archivo ejecutable es necesaria la ayuda de un **compilador** y posteriormente ser ejecutado en una unidad central de procesamiento (CPU) con la ayuda de un **intérprete**.

¿Qué es un compilador?

Ahora te preguntarás querido lector, ¿qué es un compilador? Esa pregunta es válida, te lo respondo así:

Un **compilador** es un tipo de traductor que transforma un programa entero escrito en un lenguaje de programación (llamado **código fuente**) a lenguaje máquina, que posteriormente puede ser ejecutado. A este proceso de traducción se lo llama **compilación**.

Dibujo de cómo funciona un compilador:



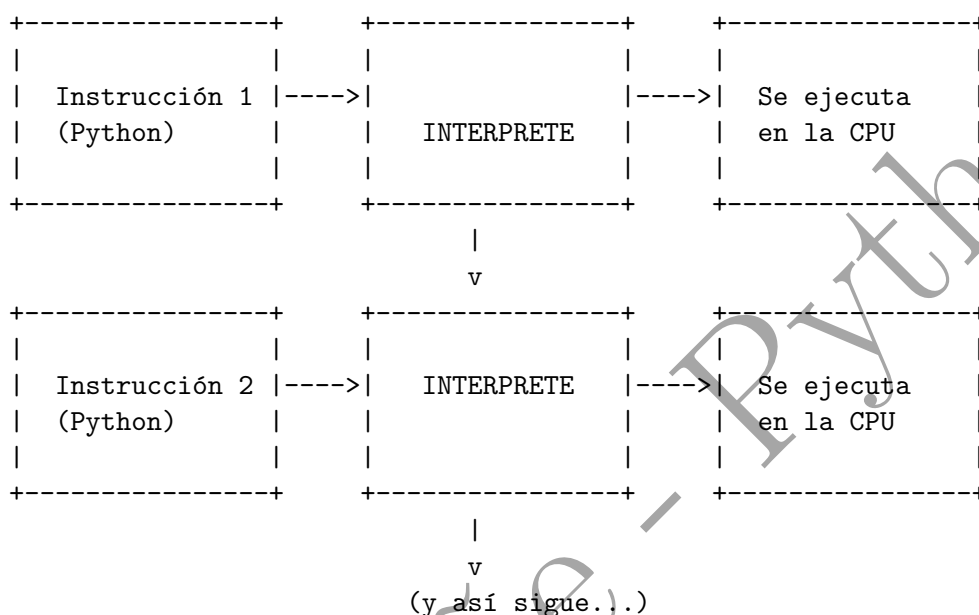
Ejemplo: Imaginá que escribís un programa entero en Python. El compilador lo lee todo de una vez, lo traduce a lenguaje que entiende la computadora (0s y 1s), y genera un archivo ejecutable. Después, cada vez que querés ejecutar ese programa, no necesitás volver a traducirlo.

¿Qué es un intérprete?

Otra pregunta que puede surgir es: todo muy lindo, pero ¿qué es un intérprete? Ahí querido lector lo responderé de la siguiente forma:

Los **intérpretes** se diferencian de los compiladores en que, mientras el compilador traduce el programa completo de una vez al código máquina del sistema, los intérpretes solo realizan la traducción a medida que sea necesaria, típicamente instrucción por instrucción, y normalmente no guardan el resultado de dicha traducción.

Dibujo de cómo funciona un intérprete:



Ejemplo: Python funciona con un intérprete. Cuando escribís una línea de código, Python la lee, la traduce y la ejecuta al instante. Luego pasa a la siguiente línea. Si hay un error en la línea 5, las líneas 1 a 4 ya se ejecutaron igual.

Diferencia clave entre compilador e intérprete

	COMPILADOR	INTERPRETE
Traducción	Todo el programa de una vez	Instrucción por instrucción
Resultado	Genera un ejecutable	No genera un archivo aparte
Velocidad de ejecución	Más rápido al ejecutar	Más lento porque traduce cada vez que corre
Ejemplos	C, C++, Rust	Python, JavaScript, Ruby

El intérprete de Python en acción (IDLE)

Cuando abrís IDLE, lo primero que ves es el intérprete interactivo. Ahí podés escribir línea por línea y Python te responde al instante:

```
Python 3.7.3 (default, ...)
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hola mundo")
Hola mundo
>>> exit()
```

Ejecutar varias líneas de código

¿Qué pasa si queremos ejecutar varias líneas? No es práctico escribirlas una por una en el intérprete. Para eso, los IDEs y editores tienen editores de texto integrados donde podés escribir programas completos.

¿Qué es un programa? (en la práctica)

Un programa es un archivo de texto (con extensión `.py`) que contiene muchas líneas de código. Ese archivo se puede ejecutar desde cualquier IDE, editor o terminal.

```
+-----+
|
|  PROGRAMA.py (archivo de texto)
|  +-----+
|  | print("Hola mundo")
|  | print("Estoy aprendiendo python")
|  | print("Adios")
|  +-----+
|
|  Este archivo se puede ejecutar en CUALQUIER
|  IDE o editor que soporte Python
|
+-----+
```

Ejecutar el mismo programa en diferentes IDEs/Editores

Una de las mejores cosas de Python es que el **mismo código** funciona en todos los IDEs y editores. No importa si usás IDLE, VS Code, PyCharm, Sublime Text, o cualquier otro. El código es el mismo.

En IDLE

1. Abrís IDLE
2. Vas a **File** → **New File** (o **Ctrl+N**)
3. Escribís el código
4. Guardás con **Ctrl+S** como `programa.py`
5. Presionás **F5** para ejecutar

En Visual Studio Code

1. Abrís VS Code
2. **Ctrl+N** para nuevo archivo
3. Escribís el código
4. Guardás como `programa.py`
5. Presionás **Ctrl+F5** o el botón Play

En PyCharm

1. Abrís PyCharm
2. Creás un nuevo proyecto
3. Creás un nuevo archivo Python (`programa.py`)
4. Escribís el código
5. Hacés clic derecho en el archivo y elegís Run”
6. O presionás **Shift+F10**

En Jupyter Notebook / JupyterLab

1. Abrís Jupyter (desde terminal con `jupyter notebook`)
2. Creás un nuevo notebook
3. Escribís el código en una celda
4. Presionás **Shift+Enter** para ejecutar

En Google Colab

1. Vas a `colab.research.google.com`
2. Creás un nuevo notebook
3. Escribís el código en una celda
4. Presionás **Shift+Enter**

En Spyder

1. Abrís Spyder
2. Escribís el código en el editor
3. Guardás como `programa.py`
4. Presionás **F5** o el botón Run”

En Thonny

1. Abrís Thonny
2. Escribís el código
3. Guardás como `programa.py`
4. Presionás **F5** o el botón Run”

En Sublime Text

1. Abrís Sublime Text
2. Escribís el código
3. Guardás como `programa.py`
4. Presionás **Ctrl+B** para ejecutar

En la terminal (sin IDE)

También podés ejecutar el programa directamente desde la terminal:

1. Abrís la terminal (cmd en Windows, Terminal en Mac/Linux)
2. Navegás hasta la carpeta donde guardaste `programa.py`
3. Escribís: `python programa.py`
4. Presionás Enter

```
+-----+
| Terminal (cmd / bash) |
| +-----+ |
| | C:\Users\Nahuel> cd Desktop | | | |
| | C:\Users\Nahuel\Desktop> python programa.py | |
| | | | | |
| | Hola mundo | |
| | Estoy aprendiendo python | |
| | Adios | |
| | | | | |
| | C:\Users\Nahuel\Desktop> | |
| +-----+ |
+-----+
```

En Replit (IDE en la nube)

1. Vas a `replit.com`
2. Creás un nuevo Repl con Python
3. Escribís el código en el editor web
4. Presionás Run”

Tabla resumen de ejecución en diferentes IDEs/Editores

IDE / EDITOR	NUEVO ARCHIVO	EJECUTAR
IDLE	Ctrl+N	F5
Visual Studio Code	Ctrl+N	Ctrl+F5 o Play
PyCharm	Ctrl+Alt+N	Shift+F10
Jupyter Notebook	New Notebook	Shift+Enter
Google Colab	New Notebook	Shift+Enter
Spyder	Ctrl+N	F5
Thonny	Ctrl+N	F5
Sublime Text	Ctrl+N	Ctrl+B
Terminal	(crear con nano)	python programa.py
Replit (web)	New Repl	Click "Run"

Ejemplo completo

Acá te muestro un programa completo de ejemplo que podés probar en CUALQUIER IDE o editor:

```
print("Hola mundo")
print("Estoy aprendiendo python")
print("Adios")
```

Lo guardás como **programa.py** y lo ejecutás. El resultado SIEMPRE será el mismo, sin importar dónde lo ejecutes:

```
Hola mundo
Estoy aprendiendo python
Adios
```

Conclusión

En conclusión, un programa de computadora escrito en un lenguaje de programación el cual es legible para un humano se llama **código fuente**, y dicho código fuente se puede convertir en un ejecutable por un **compilador** o bien ejecutarse inmediatamente con la ayuda de un **intérprete**.

¿Y Python qué usa?

Python es un lenguaje **interpretado**. Esto significa que no necesitás compilar tu código antes de ejecutarlo. Escribís, guardás, y ejecutás. El resultado es el mismo en IDLE, VS Code, PyCharm, Jupyter, o cualquier otro IDE/editor.

El MISMO código → funciona en TODOS	
IDLE	→ Hola mundo
VS Code	→ Hola mundo

```
| PyCharm      → Hola mundo |
| Jupyter     → Hola mundo |
| Spyder      → Hola mundo |
| Thonny      → Hola mundo |
| Terminal    → Hola mundo |
| Replit      → Hola mundo |
|
| ¡No importa qué uses! El código es el mismo! |
|
+-----+-----+-----+
```

Pequeño resumen para agilizar ideas

En resumen, los conceptos del capítulo son los siguientes:

- Un **programa** es una serie de instrucciones
- El **código fuente** es lo que escribe el programador
- Un **compilador** traduce todo el programa de una vez
- Un **intérprete** traduce línea por línea
- Python es un lenguaje **interpretado**
- Para programas de varias líneas, usás un editor
- En IDLE ejecutás con F5
- En VS Code ejecutás con Ctrl+F5
- En PyCharm ejecutás con Shift+F10
- En Jupyter/Colab ejecutás con Shift+Enter
- En la terminal ejecutás con `python programa.py`

El código que escribas en Python va a funcionar igual en cualquier editor que elijas. Lo importante es que empieces a programar.

Ejercicios del capítulo

Antes de pasar al siguiente tema, es hora de poner en práctica lo que aprendiste. Te preparé 20 ejercicios. Mi recomendación: primero intentá resolverlos solo, sin mirar las soluciones. Si te trabás, volvé a leer la sección correspondiente. Así se aprende de verdad.

1. Definí con tus propias palabras qué es un programa de computadora.
2. ¿Cuál es la diferencia entre código fuente y código máquina? ¿Para qué sirve cada uno?
3. Explicá qué es un compilador y mencioná dos lenguajes que usen compilador.
4. Explicá qué es un intérprete y mencioná dos lenguajes interpretados además de Python.
5. ¿Por qué decimos que Python es un lenguaje interpretado? ¿Cómo afecta eso a la forma en que ejecutamos un programa?

6. Imaginá que tenés un programa de 10 líneas y hay un error en la línea 7. ¿Qué pasa si lo ejecutás con un intérprete? ¿Y si estuviera compilado?
7. Describí paso a paso cómo ejecutarías un programa simple (tres líneas) en IDLE, desde abrir el programa hasta ver el resultado.
8. ¿Cuál es la extensión de archivo que deben tener los programas Python? ¿Por qué es importante?
9. En Visual Studio Code, ¿con qué atajo de teclado se ejecuta un archivo Python? ¿Y si preferís usar el mouse?
10. Abrí la terminal de tu sistema y escribí un programa Python que imprima tu nombre. Contame los pasos exactos que diste y el comando que usaste para ejecutarlo.
11. Si un amigo te dice "yo quiero programar Python pero no quiero instalar nada", ¿qué herramienta le recomendarías y cómo le explicarías que la use?
12. ¿Qué significa el símbolo `<code></code>`

Nombrá al menos tres IDEs o editores donde puedas ejecutar un programa Python, y decí cuál es el atajo de teclado para correr el código en cada uno.

Usando una analogía de la vida cotidiana, explicá la diferencia entre un compilador y un intérprete. (Por ejemplo, con una receta de cocina, una traducción, etc.)

Escribí un pequeño programa de al menos 4 líneas en Python (puede ser cualquier cosa) y mostrá cómo se vería ejecutado en una terminal, indicando el prompt y la salida.

¿Por qué el mismo código Python funciona sin cambios en Windows, macOS y Linux? ¿Qué característica de Python lo permite?

Si tuvieras que enseñarle a una persona mayor que nunca usó una computadora qué es un programa, ¿qué ejemplo de la vida real usarías?

Investigá (o recordá) el comando para ejecutar un script Python en Linux y en Windows. ¿Hay alguna diferencia? Escribilos.

¿Qué pasa si intentás ejecutar un programa Python desde la terminal sin haber instalado Python? ¿Por qué ocurre eso?

Creá un programa Python que muestre la frase "Aprendiendo a programar con Python". Ejecutalo en dos entornos distintos (por ejemplo, IDLE y terminal). Describí la experiencia: ¿notaste alguna diferencia en el resultado?

Soluciones a los ejercicios

Ahora sí, las respuestas. Leelas con atención y comparalas con lo que vos pensaste. Si encontrás diferencias, es un buen momento para repasar.

Ejercicio 1

¿Qué es un programa?

Un programa es como una lista de órdenes que le das a la computadora. Por ejemplo, cuando le decís "mostrá esto en pantalla", "esperá a que el usuario apriete una tecla", "calculá $2 + 2$ ". Esas órdenes, escritas en un lenguaje que entiende (en nuestro caso Python), forman un programa.

Ejercicio 2

Código fuente vs. código máquina

El **código fuente** es lo que escribimos los humanos, en un lenguaje como Python. Es legible para nosotros: `print("Hola")`. El **código máquina** es la traducción de eso a ceros y unos, que es lo único que la computadora entiende realmente. El compilador o el intérprete hacen de traductores entre ambos mundos.

Ejercicio 3

Compilador y lenguajes

Un compilador toma el código fuente completo y lo convierte en un archivo ejecutable. Después, ese ejecutable corre directamente en la máquina. Ejemplos de lenguajes que usan compilador: C, C++, Rust, Go.

Ejercicio 4

Intérprete y lenguajes interpretados

Un intérprete lee el código línea por línea, lo traduce y lo ejecuta al instante. No genera un archivo ejecutable aparte. Además de Python, otros lenguajes interpretados son JavaScript, Ruby, PHP, y R.

Ejercicio 5

Python como lenguaje interpretado

Decimos que es interpretado porque no necesitamos compilar el programa antes de correrlo. Escribimos el `.py` y lo ejecutamos directamente. El intérprete de Python se encarga de leer y ejecutar cada línea al vuelo. Esto hace que el ciclo “escribir – probar – corregir” sea mucho más rápido, porque no hay un paso extra de compilación.

Ejercicio 6

Error en la línea 7 – intérprete vs compilador

Con un intérprete (Python), el programa se ejecuta hasta la línea 7. Las primeras 6 instrucciones se cumplieron, y cuando llega al error se detiene y muestra un mensaje. Con un compilador, el programa ni siquiera llega a ejecutarse; el compilador revisa todo el código antes de generar el ejecutable, detecta el error y te avisa. Ninguna línea se ejecuta hasta que arregles el error.

Ejercicio 7

Pasos para ejecutar en IDLE

1. Abro IDLE desde el menú de inicio (o con `idle` en la terminal).
2. Voy a `File > New File` (o aprieto `Ctrl+N`).
3. Escribo:

```
print("Línea 1")
print("Línea 2")
print("Línea 3")
```

4. Guardo con `Ctrl+S`, le pongo nombre `programa.py`.

5. Presiono F5 para ejecutar.
6. En la shell de IDLE aparecen las tres líneas impresas.

Ejercicio 8

Extensión .py

Los programas Python deben tener extensión `.py`. Esto le avisa al sistema operativo y a los editores que el archivo contiene código Python, y permite que el intérprete lo reconozca y ejecute. Si lo guardás como `.txt` no va a funcionar (a menos que le cambies la extensión).

Ejercicio 9

Ejecutar en VS Code

Atajo de teclado: `Ctrl+F5` (o `F5` si ya configuraste el depurador). Con el mouse: clic en el botón Play (triángulo) que aparece arriba a la derecha del editor.

Ejercicio 10

Programa con tu nombre desde la terminal

Pasos que di yo (en Windows):

1. Abrí el `cmd` con `Win+R`, escribí `cmd`, Enter.
2. Me moví a la carpeta donde quería crear el archivo, por ejemplo `cd Desktop`.
3. Abrí el Bloc de notas, escribí `print("Nahuel")`, guardé como `nombre.py` en el Escritorio.
4. Volví a la terminal y ejecuté: `python nombre.py`.
5. La salida fue `Nahuel`.

En Linux/Mac sería similar: abris la terminal, creás el archivo con `nano nombre.py`, escribís `print("Nahuel")`, guardás, salís, y ejecutás `python3 nombre.py`.

Ejercicio 11

Programar sin instalar nada

Le recomendaría **Google Colab** o **Replit**. Le diría: "Abrís el navegador, entrás a `colab.research.google.com`, iniciás sesión con Google, creás un notebook nuevo y ya podés escribir `print("Hola")` y ejecutarlo con `Shift+Enter`. No necesitás instalar nada."

Ejercicio 12

El símbolo `>`

Es el **prompt** del intérprete interactivo de Python. Indica que Python está esperando que escribas una instrucción. Lo ves en IDLE, en la terminal cuando escribís `python` sin un archivo, y en cualquier consola Python interactiva.

Ejercicio 13

Atajos de teclado en diferentes IDEs

- IDLE: F5
- VS Code: Ctrl+F5
- PyCharm: Shift+F10
- Jupyter / Colab: Shift+Enter
- Spyder: F5
- Thonny: F5
- Sublime Text: Ctrl+B

Ejercicio 14

Analogía compilador vs intérprete

Imaginá que tenés una receta de cocina en inglés y querés cocinar un plato.

Compilador: Llamás a un traductor profesional que te traduce toda la receta de una vez y te da un librito en español. Después vos, con el librito en mano, cocinás. La traducción se hace una sola vez, y cada vez que querés cocinar usás el librito.

Intérprete: Tenés al lado a un amigo que sabe inglés. Vas leyendo la receta en inglés y él te va diciendo en español línea por línea lo que tenés que hacer. Cocinas mientras él te traduce. No te queda un librito, y cada vez que cocinás necesitás que tu amigo te traduzca.

Ejercicio 15

Programa de 4 líneas en la terminal

Mi programa ejemplo.py:

```
print("Primera linea")
nombre = "Nahuel"
print("Hola," + nombre)
print("Chau!")
```

Ejecución en terminal:

```
C:\> python ejemplo.py
Primera linea
Hola, Nahuel
Chau!
```

Ejercicio 16

Multiplataforma

Python es multiplataforma gracias a que está implementado para distintos sistemas operativos. El intérprete de Python para Windows, para macOS y para Linux se encargan de traducir el mismo código fuente a las instrucciones específicas de cada sistema. El código `print("Hola")` es interpretado de la misma forma en todos lados.

Ejercicio 17

Analogía para explicar un programa a una persona mayor

Le diría: Un programa es como una lista de instrucciones que le das a un robot muy obediente pero un poco tonto. Por ejemplo, si le decís: 'Andá a la cocina, abrí la heladera, agarrá la leche, serví un vaso', el robot lo hace. El programador es el que escribe esa lista en un idioma que el robot entiende."

Ejercicio 18

Comando en Linux y Windows

En ambos: `python nombre_del_archivo.py`. En algunas distribuciones Linux el comando es `python3 nombre.py` porque `python` suele ser Python 2. La diferencia no es del sistema operativo en sí, sino de cómo está instalado.

Ejercicio 19

Intentar ejecutar sin Python instalado

Si no tenés Python instalado y escribís `python programa.py`, el sistema te va a decir "no se reconoce como comando interno o externo" (en Windows) o `command not found` (en Linux/Mac). Porque el sistema no encuentra el intérprete de Python. Necesitás instalarlo y agregarlo al PATH, como vimos en el capítulo anterior.

Ejercicio 20

Ejecutar en IDLE y en terminal

Programa `frase.py`:

```
print("Aprendiendo a programar con Python")
```

Ejecuté en IDLE con F5 y en la terminal con `python frase.py`. La salida fue exactamente la misma: `Aprendiendo a programar con Python`. No noté ninguna diferencia, salvo que en IDLE el resultado aparece en una ventana de shell y en la terminal en la misma consola.

Entender qué es un programa y cómo se ejecuta es la base de todo. Ahora ya podés empezar a escribir tus propias instrucciones. ¡Vamos al siguiente capítulo!

¿Qué es una variable?

El primer gran 'ahá' que tuve

Cuando arranqué a programar, me la pasaba escribiendo cosas como `print(2 + 3)` y lo ejecutaba. Veía el resultado, todo bien. Pero un día quise hacer una suma y después usar ese resultado para otra cosa, y me di cuenta de que no podía. El número desaparecía. Fue como cocinar un plato y no tener dónde apoyarlo: se me caía al piso.

Ahí aparecieron las **variables**. Una variable es un espacio en la memoria donde podés guardar un dato y ponerle un nombre para usarlo después. Es como una cajita con etiqueta: guardás lo que quieras adentro, y cuando lo necesitás, solo tenés que recordar el nombre de la etiqueta.

```
+-----+
|       +-----+
|       | MIS DATOS |
|       +-----+
|       | nombre_de_variable = 42 |
|       +-----+
|
| La variable guarda el valor
| y lo mantiene hasta que vos
| lo cambies.
+-----+
```

Reglas para ponerle nombre a una variable

En Python, el nombre de una variable no puede ser cualquier cosa. Hay que respetar algunas reglas, y también hay buenas costumbres. Vamos por partes.

Lo que NO podés hacer:

- Usar palabras reservadas del lenguaje: `print`, `if`, `for`, `while`, `import`, etc. Si intentás usar una de esas como nombre, Python se confunde.
- Poner espacios en blanco. `mi variable` no funciona. Para separar palabras, usá guiones bajos.
- Empezar con un número. `1er_lugar` está mal. Pero podés poner números después del primer carácter: `lugar_1` es válido.
- Usar caracteres raros: `@`, `#`, `$`, etc. Solo letras, números y guiones bajos.

Lo que SÍ podés hacer (y es recomendado):

- Nombres en minúsculas, separados por guiones bajos: `mi_variable`, `resultado_suma`. Esta convención se llama **snake_case**.

- Nombres que tengan significado: `edad_usuario` es mejor que `x` o `var1`. Así, cuando vuelvas al código dentro de un mes, sabrás qué carajo era.

Nombres INCORRECTOS:

```
print = 5
mi variable = "Hola"
2do_lugar = "segundo"
direccion@casa = "Avenida Siempreviva 742"
```

Nombres CORRECTOS:

```
mi_variable = "Hola"
numero1 = 10
nombre_de_usuario = "Nahuel"
resultado_suma = 100
```

Reflexión: ¿Por qué pensás que Python no permite espacios en los nombres? Si tuvieras que explicárselo a un amigo no programador, ¿cómo se lo dirías?

Los tipos de variables más comunes

No todas las variables guardan lo mismo. A veces querés guardar un número, a veces un texto, a veces un sí/no. En Python, el tipo de dato lo detecta solo, pero vos tenés que entender qué estás guardando.

Variables numéricas

Dentro de los números hay dos grandes grupos:

- **Enteros (int):** números sin decimales. Ej: 2, 10, -5, 1000.
- **Decimales o reales (float):** números con punto decimal. Ej: 3.14, 2.5, -0.75.

Para Python, 5 y 5.0 son distintos. El primero es `int`, el segundo es `float`. No te vuelvas loco con eso ahora, pero es bueno saberlo.

Variables de texto

Para guardar texto usamos **strings (str)**. Lo fundamental: el texto tiene que ir entre comillas, simples o dobles. Si no ponés comillas, Python piensa que es el nombre de otra variable.

```
>>> mensaje = "Hola, ¿cómo andás?"
>>> print(mensaje)
Hola, ¿cómo andás?
```

Variables booleanas (verdadero/falso)

Un tipo que aparece mucho en programación es el **booleano (bool)**. Solo puede tener dos valores: `True` o `False` (con mayúscula inicial). Más adelante lo vas a ver cuando hablemos de condiciones.

```
>>> casado = False
>>> print(casado)
False
```

Cómo crear (declarar) una variable

Crear una variable en Python es tan simple como elegir un nombre y usar el signo = para asignarle un valor. Fijate este ejemplo:

```
>>> numero_uno = 2
```

¿Qué pasó acá? Python reservó un espacio en memoria, guardó el número 2 y lo asoció con el nombre `numero_uno`. No tuviste que decirle “esto es un número entero”; lo dedujo solo.

numero_uno	2
(nombre)	(dato)

Podés crear varias variables seguidas, incluso en la misma línea:

```
>>> nombre, edad, altura = "Nahuel", 25, 1.75
```

Esto es útil cuando tenés varios valores que van juntos.

Las variables pueden cambiar (y eso es genial)

Una variable no está atada de por vida a su valor inicial. Podés cambiar su contenido cuando quieras. Incluso podés cambiar de tipo (aunque no es recomendable, te puede traer dolores de cabeza).

```
>>> numero_uno = 2
>>> print(numero_uno)
2
>>> numero_uno = "Ahora soy texto"
>>> print(numero_uno)
Ahora soy texto
```

¿Ves? `numero_uno` empezó siendo un entero y terminó siendo un string. Python no se queja, pero si después querés sumar algo con `numero_uno`, va a saltar error. Por eso, en general mantenemos un tipo fijo para cada variable.

Anécdota: Una vez cambié el tipo de una variable sin darme cuenta y me pasé media hora buscando por qué una suma no funcionaba. Era un string. ¡Maldición!

Operaciones con variables

Las variables no solo sirven para guardar datos, sino también para operar con ellos. Acá es donde empieza la magia.

Operaciones numéricas

Con números podés sumar, restar, multiplicar, dividir, elevar a potencias, etc. Todo lo que harías con una calculadora.

```
>>> a = 10
>>> b = 3
>>> suma = a + b
>>> resta = a - b
>>> multiplicacion = a * b
>>> division = a / b          # Siempre devuelve float
>>> division_entera = a // b  # Descarta decimales
>>> potencia = a ** b         # 10 elevado a la 3
>>> print(suma, resta, multiplicacion, division, division_entera, potencia)
13 7 30 3.3333333333333335 3 1000
```

Operaciones con strings

Con texto también podés hacer algunas operaciones, como **concatenar** (pegar una cadena detrás de otra) usando el operador +.

```
>>> saludo = "Hola"
>>> nombre = "Nahuel"
>>> mensaje = saludo + ", " + nombre + "!"
>>> print(mensaje)
Hola, Nahuel!
```

También podés multiplicar un string por un número entero para repetirlo:

```
>>> linea = "-" * 20
>>> print(linea)
-----
```

Cuidado: No podés sumar strings con números directamente. `edad + 25` da error. Primero tenés que convertir el número a string con `str(25)`.

Ejemplo completo: calculando el promedio

Vamos a armar un programa que pida tres notas y calcule el promedio. Pero como todavía no vimos cómo ingresar datos del usuario, vamos a usar valores fijos.

```
# Calculadora de promedio
nota_1 = 8.5
nota_2 = 7.0
nota_3 = 9.2

suma_total = nota_1 + nota_2 + nota_3
promedio = suma_total / 3

print("Notas:", nota_1, nota_2, nota_3)
print("El promedio es:", promedio)
```

Salida:

Notas: 8.5 7.0 9.2

El promedio es: 8.233333333333334

Ejercicio rápido: Cambiá las notas y probalo en tu máquina. ¿Qué pasa si las notas son números enteros? ¿Y si ponés una nota como string? Probá y anotá el error.

Conversión entre tipos

Muchas veces necesitás transformar un tipo en otro. Para eso Python te da funciones como `int()`, `float()`, `str()`, `bool()`. Por ejemplo:

```
>>> numero_texto = "123"
>>> numero_entero = int(numero_texto)
>>> print(numero_entero + 1)
124
```

```
>>> decimal = float("3.14")
>>> print(decimal * 2)
6.28
```

```
>>> texto = str(42)
>>> print("La respuesta es " + texto)
La respuesta es 42
```

Reflexión: ¿Por qué no se puede hacer directamente "123" + 1? ¿Qué error te da? Probá en tu terminal.

Errores comunes que me mandé yo (y vos también te vas a mandar)

1. **Olvidar las comillas en strings:** Si escribís `mensaje = Hola` sin comillas, Python buscará una variable llamada `Hola`. Solución: `"Hola"`.
2. **Confundir `=` con `==`.** El primero asigna, el segundo compara. Si escribís `if edad = 18:` te va a saltar error.
3. **Usar nombres de variables que ya existen como funciones.** `print = "hola"` arruina la función `print`.
4. **Mayúsculas y minúsculas.** Python distingue: `Nombre` y `nombre` son dos variables diferentes.
5. **Olvidar el guión bajo o escribirlo mal.** `mi_variable` no es lo mismo que `mi.variable`.

Resumen rápido

- Una **variable** es un espacio con nombre para guardar datos.
- Se crea con `nombre = valor`.

- Tipos básicos: **int** (entero), **float** (decimal), **str** (texto), **bool** (True/False).
- Los nombres van en minúsculas, sin espacios, sin símbolos raros.
- Podés operar con variables: sumar, restar, concatenar, etc.
- Podés convertir tipos con `int()`, `float()`, `str()`.
- Cuidado con cambiar el tipo sin querer y con las comillas.

Las variables son la base de todo. Si las dominás, ya podés empezar a construir programas interesantes.

Ejercicios del capítulo

1. Explicá con tus propias palabras qué es una variable y para qué sirve. Usá una analogía de la vida cotidiana.
2. ¿Cuáles de los siguientes nombres de variable son válidos en Python? Justificá.

```
mi_variable
2variable
_variable
nombre completo
print
variable$
```

3. Declará tres variables: una con tu nombre, otra con tu edad y otra con tu altura en metros. Luego mostralas con `print`.
4. ¿Qué tipo de dato (`int`, `float`, `str`, `bool`) tiene cada una de estas variables?

```
a = 10
b = 3.0
c = "10"
d = True
e = "Hola"
f = 0.5
```

5. Corregí el siguiente código para que funcione:

```
nombre = Nahuel
print("Hola " + nombre)
```

6. ¿Qué imprime este programa? Pensalo antes de ejecutar.

```
x = 5
y = x + 2
x = 10
print(y)
```

7. Escribí un programa que calcule el área de un rectángulo de base 8 y altura 3. El resultado debe guardarse en una variable y luego imprimirse.
8. ¿Qué error tiene este código y cómo lo arreglarías?

```
total = "100"
descuento = 20
final = total - descuento
print(final)
```

9. Concatená tu nombre y apellido en una sola variable y mostralos con un espacio entre ellos.
10. ¿Qué diferencia hay entre 10 y "10"? Explicá qué pasa si los sumás con otro número.
11. Creá una variable llamada `mensaje` que contenga la frase `.Aprendiendo Python` repetida tres veces, separada por guiones. Usá solo operaciones con strings.
12. Declará tres variables en una sola línea: `pais`, `capital`, `habitantes`. Asigná valores de Argentina, Buenos Aires y 46 millones. Imprimí una frase que las use.
13. ¿Cuál es el resultado de la siguiente operación? `7 / 2` y `7 // 2`. Explicá la diferencia.
14. Convertí el número 3.1416 a string y concatenálo con la frase `.El valor de pi es aproximadamente`. Imprimilo.
15. ¿Qué pasa si ejecutás `print("Hola"* 3)`? ¿Y `print("Hola" 3)`? Explicá la diferencia.
16. Creá una variable booleana `es_mayor` que indique si 18 es mayor que 21. Mostrá su valor.
17. Escribí un programa que intercambie los valores de dos variables. Partí de `a = 5` y `b = 10`. Al final, `a` debe valer 10 y `b` 5. Ayuda: necesitás una variable temporal.
18. ¿Qué imprime este código?

```
a = 2
b = a
a = 5
print(b)
```

¿Cambió `b` cuando `a` cambió?

19. Explicá con un ejemplo por qué la línea `print = 100` es peligrosa y qué consecuencias trae.
20. Creá un programa que calcule el índice de masa corporal (IMC). Usá variables para peso (en kg) y altura (en metros). Fórmula: `imc = peso / (altura ** 2)`. Probá con tus datos y mostrá el resultado redondeado a dos decimales. Pista: usá `round(imc, 2)`.

Soluciones a los ejercicios

Ejercicio 1

Una variable es como un frasco etiquetado en la cocina. Le ponés `.azúcar` adentro guardás azúcar. Cuando necesitás azúcar, buscás el frasco por su nombre. En programación, guardás datos bajo un nombre para usarlos después.

Ejercicio 2

Válidos: `mi_variable`, `_variable`. No válidos: `2variable` (empieza con número), `nombre completo` (tiene espacio), `print` (palabra reservada), `variable$` (carácter no permitido).

Ejercicio 3

```
nombre = "Nahuel"
edad = 25
altura = 1.75
print(nombre, edad, altura)
```

Ejercicio 4

a: int, b: float, c: str, d: bool, e: str, f: float.

Ejercicio 5

Falta comillas: `nombre = "Nahuel"`. El código corregido:

```
nombre = "Nahuel"
print("Hola " + nombre)
```

Ejercicio 6

Imprime 7. Porque y se calculó como $x + 2$ cuando x valía 5. Luego x cambió a 10, pero eso no afecta a y.

Ejercicio 7

```
base = 8
altura = 3
area = base * altura
print("El área es:", area)
```

Ejercicio 8

Error: `total` es string, no se puede restar un número a un string. Hay que convertir `total` a número: `final = int(total) - descuento`.

Ejercicio 9

```
nombre = "Nahuel"
apellido = "Pérez"
nombre_completo = nombre + " " + apellido
print(nombre_completo)
```

Ejercicio 10

10 es un entero, "10" es un texto. Si sumás $10 + 5$ da 15. Si intentás `"10" + 5` da error (no se puede concatenar str con int). Necesitás convertir: `int("10") + 5` da 15.

Ejercicio 11

```
frase = "Aprendiendo Python"
mensaje = (frase + " - ") * 2 + frase
print(mensaje)
```

O también: `mensaje = frase + " " + frase + " " + frase`.

Ejercicio 12

```
pais, capital, habitantes = "Argentina", "Buenos Aires", 46000000
print(pais, "tiene como capital a", capital, "y aproximadamente", habitantes, "habitantes.")
```

Ejercicio 13

`7 / 2` devuelve 3.5 (float). `7 // 2` devuelve 3 (división entera, descarta decimales).

Ejercicio 14

```
pi = 3.1416
mensaje = "El valor de pi es aproximadamente " + str(pi)
print(mensaje)
```

Ejercicio 15

`"Hola"* 3` repite el string tres veces: `HolaHolaHola`. `"Hola" * 3` da error: no se puede concatenar str con int.

Ejercicio 16

```
es_mayor = 18 > 21
print(es_mayor) # False
```

Ejercicio 17

```
a = 5
b = 10
temp = a
a = b
b = temp
print(a, b) # 10 5
```

Ejercicio 18

Imprime 2. `b` se quedó con el valor que tenía `a` en el momento de la asignación. Cambiar `a` después no modifica `b`.

Ejercicio 19

Al hacer `print = 100`, la función `print` deja de ser una función y se convierte en una variable que guarda 100. Si después intentás usar `print("hola")` te va a dar error porque `print` ya no es la función original. Es un lío.

Ejercicio 20

```
peso = 70 # kg
altura = 1.75 # metros
imc = peso / (altura ** 2)
imc_redondeado = round(imc, 2)
print("Tu IMC es:", imc_redondeado)
```

Cadena de caracteres

¿Qué es una cadena de caracteres?

Mirá, cuando empecé a programar, creía que los únicos datos importantes eran los números. Pero un día tuve que hacer un programa que saludara al usuario y me di cuenta de que necesitaba manejar texto. Ahí entré en el mundo de las **cadenas de caracteres** o **strings**.

Pensá en un string como un collar de perlas, donde cada perla es una letra, un número, un espacio o un signo. Todas juntas forman algo con sentido. En criollo, un string es cualquier cosa que va entre comillas. Puede ser tu nombre, una frase, un código postal, lo que sea.

```
+-----+
|
|  "Hola, este es un string o cadena de texto"  |
|
|  Letras, numeros (123), signos (!@#) y simbolos |
|
+-----+
```

Las herramientas para trabajar con texto

Python te da un montón de operaciones para jugar con strings. Como cuando tenés una caja de herramientas y cada una sirve para algo distinto. Las principales que vamos a ver son:

- La **asignación** (**+=**): para agregar más texto a una variable.
- La **concatenación** (**+**): para unir dos o más cadenas en una nueva.
- La **búsqueda** (**find()**): para encontrar dónde está una palabra dentro de otra.
- La **extracción** (**[]**): para cortar un pedazo de texto.
- La **comparación** (**==**): para ver si dos textos son iguales.

La asignación (**+=**)

Problema que tuve: Estaba armando un mensaje de bienvenida y quería agregar el nombre de la persona al final, pero sin borrar lo que ya tenía. Si usaba **=** solamente, me pisaba todo. Entonces aprendí **+=**.

La **asignación** con **+=** es como ir llenando un carrito de compras. Empezás con un carrito vacío y le vas agregando productos uno a uno. El carrito es la variable, y **+=** es el acto de meter algo adentro.

Ejemplo paso a paso

```
mensaje = 'Hola'
mensaje += ' '
mensaje += 'Nahuel'
print(mensaje)
```

Acompañame a ver qué pasó por dentro:

Paso 1: mensaje = 'Hola'	-> El carrito tiene "Hola"
Paso 2: mensaje += ' '	-> Ahora tiene "Hola "
Paso 3: mensaje += 'Nahuel'	-> Termina con "Hola Nahuel"
Paso 4: print(mensaje)	-> Muestra: Hola Nahuel

Reflexión: ¿Qué pasaría si en lugar de += usaras = en los pasos 2 y 3? ¡Probá en tu cabeza o en la terminal! Te darías cuenta de que cada vez se borra lo anterior y solo queda lo último.

La concatenación (+)

Otro problema: Quería unir el nombre y el apellido que estaban en dos variables distintas sin modificar ninguna de las dos. La solución fue la **concatenación** con +.

Concatenar es como pegar dos papelitos con cinta adhesiva. Tenés el papelito A y el B, los unís y formás uno nuevo más largo. Ni A ni B se modifican; creás un tercer papelito.

Ejemplo de concatenación

```
mensaje = 'Hola'
espacio = ' '
nombre = 'Nahuel'
mensaje_completo = mensaje + espacio + nombre
print(mensaje_completo)
```

Fijate cómo funciona:

```
mensaje tiene 'Hola', espacio tiene ' ', nombre tiene 'Nahuel'
La suma: 'Hola' + ' ' + 'Nahuel' da 'Hola Nahuel'
Ese valor nuevo se guarda en mensaje_completo
```

Ejercicio rápido: Creá tres variables con las palabras "Me", "gusta", "Pythonz" concatenalas con espacios. Después mostrá la frase completa.

La búsqueda (find())

Me pasó: Tenía una frase larguísima y necesitaba saber en qué posición exacta aparecía la palabra "error". No podía contarlo a mano. Entonces conocí find().

find() es como cuando buscás una palabra en un libro usando el índice. El índice te dice la página donde empieza. Acá te dice la **posición** (contando desde 0) donde arranca la subcadena.

Ejemplo

```
mensaje = 'Hola Nahuel'
buscar_subcadena = mensaje.find('Nahuel')
print(buscar_subcadena)
```

Pensemos en la cadena como una fila de sillas numeradas:

Posiciones: 0:H 1:o 2:l 3:a 4: 5:N 6:a 7:h 8:u 9:e 10:l
 "find('Nahuel')" devuelve 5, porque la 'N' está en la silla 5.

Ojo: Si la palabra no está, find() te devuelve -1. Por ejemplo, mensaje.find('Maria') da -1.
 ¡No hay coincidencia!

La extracción ([inicio:fin])

Una necesidad: De una fecha escrita como "2024-01-25" quería sacar solo el año. Es decir, cortar un pedazo del string. Eso es la **extracción**.

Imaginá que tenés una barra de pan y querés cortar una rebanada. Con [] le decís desde dónde hasta dónde. El cuchillo corta en el inicio y deja de cortar justo antes del fin.

Ejemplo

```
mensaje = 'Hola Nahuel'
extraer_subcadena = mensaje[1:8]
print(extraer_subcadena)
```

Veamos gráficamente:

```
0:H 1:o 2:l 3:a 4: 5:N 6:a 7:h 8:u 9:e 10:l
  |_____|
  inicio=1      fin=8 (no incluido)
```

Las sillas que agarra son 1,2,3,4,5,6,7: "la Nah". La silla 8 (u) queda afuera.

Reglas que me enseñaron:

- [:5] → desde el principio hasta la posición 4 (inclusive).
- [3:] → desde la posición 3 hasta el final.
- [::2] → toma una sí, una no (paso 2).

Ejercicio: Dado frase = "prender es clave", extraé "prender" (pista: frase[:8] o frase[0:8]).

La comparación (==)

Confusión de novato: Yo usaba = para comparar y Python me tiraba error. Hasta que me dijeron que = es para guardar, == es para preguntar si son iguales. Es como comparar dos fotos para ver si son la misma persona.

La **comparación** con == devuelve True o False (verdadero/falso).

Ejemplo de igualdad

```
mensaje_uno = 'Hola'
mensaje_dos = 'Hola'
print(mensaje_uno == mensaje_dos) # True
```

Ejemplo de diferencia

```
print('Hola' == 'Nahuel') # False
```

Importante: Python diferencia mayúsculas y minúsculas. 'Python' no es igual a 'python'. Además, '123' (texto) no es igual a 123 (número). Cuidado con eso.

Resumen de operaciones con strings

Dale, repasemos rápido lo importante:

OPERACION	OPERADOR	EJEMPLO
Asignacion	+=	texto += " agregado"
Concatenacion	+	texto1 + texto2
Busqueda	.find()	texto.find("buscar")
Extraccion	[inicio:fin]	texto[2:7]
Comparacion	==	texto1 == texto2

Ejercicios del capítulo

1. Creá una variable `nombre` con tu nombre y otra `apellido` con tu apellido. Concatenarlos para mostrar el nombre completo con un espacio en el medio.
2. Creá una variable `saludo` con "Hola" y usando += agregale tu nombre. Mostrá el resultado.
3. Dada la variable `mensaje = "Python es genial"`, extraé la palabra "Python".
4. Con `mensaje = ".^prender Python es divertido"`, usá `find()` para encontrar la posición de "Python".
5. Compará dos variables con tu nombre y mostrá si son iguales.
6. Construí un saludo concatenando "Buenos dias", una coma, un espacio y tu nombre.
7. Usando `find()`, buscá la letra 'a' en "programacion". ¿Qué posición devuelve? ¿Es la primera 'a'?
8. Extraé los primeros 5 caracteres de "Me encanta programar".
9. Compará "Python" con "python". ¿Son iguales? Explicá por qué.

10. Creá una variable `frase = "Hola mundo"` y agregale "desde Python" usando `+=`.
11. Extraé los últimos 6 caracteres de `."prendiendo Python paso a paso"` (pista: índices negativos).
12. Concatená tres variables: `nombre`, `edad` (como texto), y `ciudad`. Mostrá el resultado en una frase.
13. Usá `find()` para buscar la letra 'z' en "manzana".
14. Extraé desde la posición 3 hasta la 10 de "Programacion es divertida".
15. ¿Qué pasa si comparás "123" (texto) con 123 (número) usando `==`? Explicá por qué.
16. Creá dos variables `texto1 = "Hola"` y `texto2 = "Mundo"`. Concatenalas con un espacio en el medio y guardá el resultado en `texto3`.
17. Usá `find()` para buscar la palabra "facil" en "Python es facil de aprender".
18. Extraé los caracteres en posiciones pares de `."abcdefghij"` (recordá que empezamos desde 0).
19. Compará "Hola" con "HOLA" usando `==`. ¿Por qué el resultado es False?
20. Armá un programa que:
 - a) Asigne tu nombre a una variable.
 - b) Asigne tu ciudad.
 - c) Concatene todo en un mensaje: "Hola, soy [nombre] y vivo en [ciudad]".
 - d) Muestre el mensaje.

Soluciones paso a paso

Ejercicio 1

```
nombre = "Juan"
apellido = "Perez"
nombre_completo = nombre + " " + apellido
print(nombre_completo)
```

Ejercicio 2

```
saludo = "Hola"
saludo += " Juan"
print(saludo)
```

Ejercicio 3

```
mensaje = "Python es genial"
palabra = mensaje[0:6]
print(palabra)
```

Ejercicio 4

```
mensaje = "Aprender Python es divertido"
posicion = mensaje.find("Python")
print(posicion) # 9
```

Ejercicio 5

```
nombre1 = "Juan"
nombre2 = "Juan"
print(nombre1 == nombre2) # True
```

Ejercicio 6

```
nombre = "Juan"
saludo = "Buenos dias, " + nombre
print(saludo)
```

Ejercicio 7

```
palabra = "programacion"
posicion = palabra.find("a")
print(posicion) # 3, es la primera 'a'
```

Ejercicio 8

```
frase = "Me encanta programar"
primeros_cinco = frase[:5]
print(primeros_cinco) # "Me en"
```

Ejercicio 9

```
print("Python" == "python") # False
# Porque la P mayúscula no es igual a la p minúscula.
```

Ejercicio 10

```
frase = "Hola mundo"
frase += " desde Python"
print(frase)
```

Ejercicio 11

```
frase = "Aprendiendo Python paso a paso"
ultimos = frase[-6:] # los últimos 6
print(ultimos) # "a paso"
```

Ejercicio 12

```
nombre = "Juan"
edad = "25"
ciudad = "Buenos Aires"
resultado = nombre + ", " + edad + " años, " + ciudad
print(resultado)
```

Ejercicio 13

```
palabra = "manzana"
posicion = palabra.find("z")
print(posicion) # 3
```

Ejercicio 14

```
texto = "Programacion es divertida"
extraccion = texto[3:11]
print(extraccion) # "gramacio"
```

Ejercicio 15

```
print("123" == 123) # False
# Son de distinto tipo: string vs entero.
# Python no los considera iguales.
```

Ejercicio 16

```
texto1 = "Hola"
texto2 = "Mundo"
texto3 = texto1 + " " + texto2
print(texto3)
```

Ejercicio 17

```
frase = "Python es facil de aprender"
posicion = frase.find("facil")
print(posicion) # 10
```

Ejercicio 18

```
texto = "abcdefghij"
resultado = texto[::2] # toma índices 0,2,4,6,8
print(resultado) # "acegi"
```

Ejercicio 19

```
print("Hola" == "HOLA") # False
# Las mayúsculas y minúsculas son caracteres distintos.
```


Ejercicio 20

```
nombre = "Juan"
ciudad = "Buenos Aires"
mensaje = "Hola, soy " + nombre + " y vivo en " + ciudad
print(mensaje)
```

Los strings son la voz de tus programas. Cuanto mejor los manipules, más claro hablarán. ¡Seguí practicando!

Palabras reservadas

El día que rompí el print

Cuando recién arrancaba con Python, me pasó algo que me dejó rascándome la cabeza. Estaba probando variables y se me ocurrió hacer `print = 5`. Inocente, ¿no? Después quise imprimir algo y ¡pum! error. No entendía nada. El mismísimo `print` se había vuelto un número y ya no servía para mostrar nada en pantalla.

Ese día aprendí sobre las **palabras reservadas**. Y ahora te lo cuento para que no te comás el mismo garrón.

¿Qué son las palabras reservadas?

Pensá en un micro de larga distancia. Hay asientos que ya están reservados para ciertas personas. Vos podés elegir cualquier otro, pero esos no los podés ocupar porque ya tienen dueño. En Python, las **palabras reservadas** son esos asientos: el lenguaje ya las usa para sus propias tareas, y vos no podés usarlas para nombrar variables, funciones ni nada que inventes.

```
+-----+
|
|  PALABRAS RESERVADAS = asientos con dueño
|  en el micro de Python.
|
|  NO SE PUEDEN USAR COMO NOMBRES DE VARIABLES
|
+-----+
```

¿Por qué no puedo usarlas como variables?

Porque Python necesita esas palabras para funcionar. Si vos le cambiás el significado, es como si en un partido de fútbol un jugador se pusiera la camiseta del árbitro y empezara a cobrar faltas. Se arma un quilombo.

Ejemplo clásico (y doloroso):

```
>>> print = 6
>>> print(5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'int' object is not callable
```

¿Qué pasó ahí? Simple: le dijiste a Python que `print` ahora es el número 6. La función original que muestra cosas en pantalla quedó tapada, como si le hubieras puesto una caja encima. Cuando intentaste `print(5)`, Python se quejó: “¡Flaco, esto es un número, no una función!”.

La lista completa de palabras reservadas

Estas son todas las que Python tiene ocupadas. Te conviene echarles un ojo aunque no las memorices; con el tiempo las vas a ir reconociendo.

False	None	True	and	as	assert	async
await	break	class	continue	def	del	elif
else	except	finally	for	from	global	if
import	in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with	yield

Dato: En las versiones más recientes de Python hay 35 palabras reservadas.

Errores comunes que me mandé (y vos también te vas a mandar)

Usar input como variable

```
>>> input = 10
>>> dato = input("Ingrese algo: ")
TypeError: 'int' object is not callable
```

Yo quería pedirle algo al usuario, pero como `input` ahora era el número 10, Python no tenía forma de llamar a la función original. Solución: `entrada_usuario = 10`, por ejemplo.

Usar sum como variable

```
>>> sum = 5
>>> resultado = sum([1, 2, 3])
TypeError: 'int' object is not callable
```

`sum` es una función que suma los elementos de una lista. Al hacer `sum = 5`, la pisé. Un dolor de cabeza hasta que me di cuenta.

Usar if, for o while como variables

Estas ni siquiera te dejan ejecutar: Python te frena de una con un `SyntaxError`. Son tan importantes para la estructura del programa que el intérprete no te deja ni intentarlo.

```
>>> if = 10
File "<stdin>", line 1
    if = 10
    ^
SyntaxError: invalid syntax
```

¿Cómo darme cuenta si una palabra está reservada?

Hay dos trucos muy útiles:

1. **Los colores en el editor:** en VS Code, IDLE y casi cualquier editor, las palabras reservadas aparecen de un color distinto (generalmente azul o violeta). Si escribís una variable y se tiñe de ese color, ¡alerta!
2. **Preguntarle a Python:** podés ejecutar esto en tu terminal para ver la lista oficial:

```
>>> import keyword
>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',
'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',
'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
'try', 'while', 'with', 'yield']
```

Guardá ese `import keyword` como una herramienta más en tu caja. Te va a salvar más de una vez.

¿Qué hago si ya usé una palabra reservada?

No te asustes, tiene arreglo. Lo más sencillo es cambiarle el nombre a tu variable por algo que no esté en la lista. Si estás en una shell interactiva y ya hiciste el desastre, a veces lo más rápido es cerrar y volver a abrir para empezar de cero.

INCORRECTO	CORRECTO
<code>print = 5</code>	<code>mi_print = 5</code>
<code>if = 10</code>	<code>condicion_if = 10</code>
<code>for = "texto"</code>	<code>variable_for = "texto"</code>
<code>while = 3</code>	<code>contador_while = 3</code>
<code>input = 20</code>	<code>dato_input = 20</code>
<code>sum = 2 + 3</code>	<code>resultado_suma = 5</code>

Regla de oro: ante la duda, poné un nombre descriptivo. `resultado_final` no está reservado; `sum` sí.

¿Y si quiero usar una palabra reservada pero con otro significado?

Podés usar variaciones. Python distingue mayúsculas y minúsculas, así que `Print` (con mayúscula) no es lo mismo que `print`. Sin embargo, no te lo recomiendo: puede confundir a quien lea tu código (incluso a vos mismo mañana). Mejor buscá un sinónimo: `mostrar`, `imprimir`, `mensaje`, etc.

Ejercicio rápido: ¿Cuáles de estos nombres son válidos? `print`, `mi_print`, `Print`, `if`, `condicion`. (Respuesta más abajo, ¡pero pensalo antes!)

Dale, repasemos rápido lo importante

- **Palabras reservadas:** son las que Python ya usa y no podés reutilizar.
- Si las usás como variables, Python tira error o se comporta raro.
- Hay 35 en total. Las más comunes: `print`, `if`, `for`, `while`, `input`, `import`, `def`, `True`, `False`, `None`.
- Para ver la lista completa: `import keyword; print(keyword.kwlist)`.
- Los editores las colorean distinto.
- Si la pifiaste, cambiale el nombre a tu variable.

Las palabras reservadas son como los asientos con dueño en un micro: no te podés sentar ahí, pero hay un montón de otros libres.

Ejercicios del capítulo

1. ¿Cuáles de estos nombres son válidos para una variable? Justificá.

- a) `print`
- b) `mi_print`
- c) `if`
- d) `condicion`
- e) `for`
- f) `lista_for`

2. ¿Qué error da este código? ¿Por qué?

```
if = 10
print(if)
```

3. Renombra estas variables mal escritas:

- a) `print = "Hola"`
- b) `input = 25`
- c) `while = True`
- d) `sum = 1 + 2`

4. ¿Qué imprime este programa? Explicá cada paso.

```
print("Hola")
print = 5
print("Mundo")
```

5. Escribí el comando para ver todas las palabras reservadas de Python.
6. ¿Cuántas palabras reservadas hay en Python 3.12 aproximadamente?

7. ¿Qué pasa si intentás usar `class` como nombre de variable? Probá mentalmente.
8. Explicá con una analogía por qué no se puede usar `print` como variable.
9. Corregí este código para que funcione:

```
print("Ingrese su nombre: ")
input = input()
print("Hola", input)
```

10. ¿Cuál es la diferencia entre `print` como función y `print` como variable?
11. ¿Es válido usar `Python` como nombre de variable? ¿Por qué?
12. Proponé un buen nombre para una variable que guarde un mensaje de error.
13. Proponé un buen nombre para una variable que guarde la edad ingresada por el usuario.
14. ¿Es válido `_print`? ¿Por qué?
15. ¿Es válido `Print` (con mayúscula)? ¿Es recomendable?
16. ¿Qué hace `len(keyword.kwlist)`?
17. Escribí 5 nombres de variables correctos que describan bien lo que guardan.
18. Escribí 5 nombres de variables que den error por usar palabras reservadas.
19. ¿Qué nombre es mejor: `a` o `promedio_notas`? Justificá.
20. Averiguá: ¿las palabras reservadas de Python son las mismas que las de C++ o Java? Comentá lo que encuentres.

Soluciones paso a paso

Ejercicio 1

- `print`: no válido, es palabra reservada.
- `mi_print`: válido, no está en la lista.
- `if`: no válido, palabra reservada.
- `condicion`: válido.
- `for`: no válido, palabra reservada.
- `lista_for`: válido, aunque tiene “for” al final, no es exactamente la palabra reservada.

Ejercicio 2

`SyntaxError: invalid syntax`. No se puede asignar un valor a `if` porque es una palabra clave para estructuras condicionales.

Ejercicio 3

```
1. mensaje = "Hola"
2. dato_ingresado = 25
3. condicion_while = True
4. resultado = 1 + 2
```

Ejercicio 4

Primero imprime "Hola" correctamente. Luego, `print` se convierte en el entero 5. Al intentar `print("Mundo")`, Python lanza `TypeError` porque un entero no es una función.

Ejercicio 5

```
import keyword
print(keyword.kwlist)
```

Ejercicio 6

Hay 35 palabras reservadas en Python 3.12.

Ejercicio 7

Da `SyntaxError`. `class` está reservada para definir clases.

Ejercicio 8

Es como querer usar el banco del conductor para sentarte a leer un libro: ese asiento ya tiene una función específica. `print` ya es la función que muestra cosas en pantalla, no podés darle otro uso.

Ejercicio 9

```
print("Ingrése su nombre: ")
nombre = input()
print("Hola", nombre)
```

Ejercicio 10

`print` como función es la herramienta que muestra datos. Como variable, es una caja donde guardás un valor. No pueden convivir con el mismo nombre en el mismo programa.

Ejercicio 11

Sí, `Python` (con mayúscula) no es palabra reservada. Las reservadas están en minúscula.

Ejercicio 12

```
mensaje_error, error_usuario, alerta, notificacion.
```

Ejercicio 13

`edad_ingresada, edad_usuario, edad_input, anios_usuario.`

Ejercicio 14

Sí, `_print` es válido. El guión bajo lo hace distinto a la palabra reservada. Pero puede ser confuso.

Ejercicio 15

`Print` es válido porque Python distingue mayúsculas. Pero no es recomendable: puede llevar a confusiones con la función `print`.

Ejercicio 16

Devuelve 35, la cantidad de elementos de la lista de palabras reservadas.

Ejercicio 17

Ejemplos: `nombre_completo, temperatura_actual, cantidad_productos, email_usuario, direccion_envio.`

Ejercicio 18

Ejemplos: `import = "datos", from = 10, None = 0, True = "verdadero", lambda = 5.`

Ejercicio 19

`promedio_notas` es mucho mejor porque cualquiera que lea el código entiende qué guarda. `a` no dice nada y hace que el programa sea difícil de mantener.

Ejercicio 20

No, cada lenguaje tiene sus propias palabras reservadas, aunque muchas coinciden (por ejemplo `if`, `else`, `while` son comunes en muchos). Pero `print` en C es una función de biblioteca, no una palabra reservada. En Python, sí lo es.

Dominar las palabras reservadas es conocer las reglas de la cancha. Ahora que las tenés claras, podés jugar sin miedo a que te cobren foul.

Operadores aritméticos

La calculadora más poderosa del mundo

Cuando empecé a programar, lo primero que pensé fue: "Esto es una calculadora gigante". Y sí, en parte tenés razón. Pero lo lindo no es solo hacer cuentas, sino que la computadora las haga por vos, rápido y sin quejarse. Para eso están los **operadores aritméticos**.

Pensá en ellos como las herramientas básicas de cualquier taller: el martillo (+), el destornillador (-), la llave inglesa (*)... Cada uno hace una tarea específica con números. Y cuando los combinás, armás cualquier mueble.

```
+-----+
|                                     |
|  OPERADORES ARITMETICOS          |
|                                     |
|  +  -  *  /  **  //  %           |
|                                     |
|  Las siete herramientas del taller numérico |
|                                     |
+-----+
```

Los siete operadores, uno por uno

Antes de tirarte una tabla fría, dejame que te cuente cada uno con un problema que yo mismo tuve que resolver cuando arranqué.

Suma (+)

Mi primer problema: Quería sumar la cantidad de páginas que leí en dos días distintos. Nada loco: 25 un día, 30 el otro. La **suma** con + junta dos o más números y te da el total.

```
>>> 25 + 30
55
```

Con variables es igual de simple:

```
>>> dia1 = 25
>>> dia2 = 30
>>> total = dia1 + dia2
>>> print(total)
55
```

Resta (-)

Mi segundo problema: Tenía \$100 en la billetera digital y gasté \$35 en una cena. ¿Cuánto me quedaba? La **resta** (-) quita un número de otro.

```
>>> billetera = 100
>>> gasto = 35
>>> saldo = billetera - gasto
>>> print(saldo)
65
```

Ojo con algo que me pasó: Si restás un número más grande de uno más chico, el resultado es negativo. Perfectamente válido, pero al principio me confundía.

```
>>> 10 - 25
-15
```

Multiplicación (*)

Problema clásico: Tengo 8 cajas y cada una trae 6 manzanas. ¿Cuántas manzanas tengo? La **multiplicación** (*) es una suma repetida.

```
>>> cajas = 8
>>> manzanas_por_caja = 6
>>> total_manzanas = cajas * manzanas_por_caja
>>> print(total_manzanas)
48
```

Anécdota: Al principio yo usaba `x` para multiplicar, como en el colegio. ¡No! Python no entiende `x`, entiende `*`.

División (/)

Problema: Una pizza de 8 porciones para 4 personas. ¿Cuánto come cada una? La **división** (/) reparte.

```
>>> porciones = 8
>>> personas = 4
>>> por_persona = porciones / personas
>>> print(por_persona)
2.0
```

Detalle importante que me costó entender: La división / **siempre** devuelve un número con decimales (float), incluso si la cuenta es exacta. Fijate que `8 / 4` da `2.0`, no `2`. Al principio pensaba que era un error, pero está bien.

División entera (//)

Otro problema: Tengo 10 sillas y quiero armar filas de a 3. ¿Cuántas filas completas puedo armar? La **división entera** (//) se queda con la parte entera del resultado y tira los decimales a la basura.

```
>>> sillars = 10
>>> por_fila = 3
>>> filas_completas = sillars // por_fila
>>> print(filas_completas)
3
```

Compará con la división normal:

```
>>> 10 / 3
3.3333333333333335
>>> 10 // 3
3
```

Módulo (%)

Necesidad real: Con las mismas 10 sillas y filas de a 3, ¿cuántas sillas me sobran? El **módulo** (%) te da el **resto** de la división.

```
>>> sillars = 10
>>> por_fila = 3
>>> sobran = sillars % por_fila
>>> print(sobran)
1
```

Truco que usé mucho: El módulo sirve para saber si un número es par o impar. Si `num % 2` da 0, es par. Si da 1, es impar.

```
>>> 14 % 2
0    # 14 es par
>>> 15 % 2
1    # 15 es impar
```

Exponente (**)

Problema: Quería calcular 2 elevado a la 5 (2 multiplicado por sí mismo 5 veces). La **potencia** se hace con ******.

```
>>> 2 ** 5
32
```

Equivale a `2 * 2 * 2 * 2 * 2`. Imaginate hacerlo a mano con potencias más grandes... ¡por eso tenemos Python!

Combinando operadores: la jerarquía importa

Cuando mezclás varios operadores en una misma línea, Python sigue el mismo orden que aprendiste en la escuela: primero paréntesis, después exponentes, después multiplicación/división/módulo, y al final suma/resta. En criollo: **PEMDAS** (o **PEMDASR** si le ponemos el módulo).

```
>>> 2 + 3 * 4
14    # 3*4=12, luego 2+12=14
>>> (2 + 3) * 4
20    # 2+3=5, luego 5*4=20
```

Reflexión: Probá en tu cabeza (y después en la terminal) cuánto da $10 - 2 ** 2 * 3 + 5$. Después verificá con Python. ¿Coincidió? Si no, revisá el orden.

Errores comunes que yo también me mandé

1. **Mezclar texto y números:** "5" + 3 da error. Python no sabe si querés sumar o concatenar. Solución: convertí con `int("5") + 3`.
2. **Dividir por cero:** `10 / 0` tira `ZeroDivisionError`. Es imposible. Siempre chequeá que el divisor no sea cero.
3. **Confundir ** con *:** `5 ** 2` es 25 (potencia), `5 * 2` es 10. Me pasé horas buscando un error por eso.
4. **Olvidar los paréntesis:** `10 + 5 * 2` no es lo mismo que `(10 + 5) * 2`. Si tenés duda, poné paréntesis.

Tabla resumen para la pared

Operador	Operacion	Ejemplo	Resultado
+	Suma	<code>5 + 3</code>	8
-	Resta	<code>5 - 3</code>	2
*	Multiplicacion	<code>5 * 3</code>	15
/	Division	<code>10 / 3</code>	3.333...
**	Exponente	<code>2 ** 3</code>	8
//	Division entera	<code>10 // 3</code>	3
%	Modulo	<code>10 % 3</code>	1

Dale, repasemos rápido lo importante

- + suma, - resta, * multiplica.
- / divide y siempre devuelve decimal (`float`).
- // divide y devuelve solo la parte entera.
- % te da el resto de la división. Ideal para par/impar.

- `**` eleva un número a una potencia.
- La jerarquía de operaciones es igual a la matemática: paréntesis ¿exponentes ¿multiplicación/división/módulo ¿suma/resta.
- Si mezclas texto y números sin convertir, ¡explota! Usá `int()` o `str()`.

Los operadores aritméticos son el primer paso para que la computadora haga el trabajo pesado por vos. Dominá estas herramientas y ya vas a poder construir programas útiles.

Ejercicios del capítulo

1. Calculá la suma de 38 y 56 usando variables y mostrá el resultado.
2. Calculá la resta de 200 menos 87 y mostrala.
3. Calculá la multiplicación de 12 por 9.
4. Dividí 156 entre 13. ¿Qué tipo de dato devuelve? ¿Por qué?
5. Calculá 3 elevado a la 6 (3 multiplicado por sí mismo 6 veces).
6. Calculá la división entera de 100 entre 7.
7. Calculá el módulo (resto) de 100 entre 7.
8. Hacé esta cuenta en Python: $(8 + 4) * (15 - 6)$. Mostrá el proceso mental.
9. Calculá $100 / 5 / 4$. ¿Qué resultado te da? ¿Por qué?
10. ¿Cuál es el resto de dividir 256 entre 10? ¿Cómo lo usarías para obtener el último dígito de un número?
11. ¿Cuál es la parte entera de 256 dividido 10? ¿Cómo lo usarías para obtener todos los dígitos menos el último?
12. Calculá el área de un triángulo de base 12 y altura 7 (fórmula: `base * altura / 2`).
13. Calculá el perímetro de un rectángulo de lados 8 y 5 (fórmula: `2 * (lado1 + lado2)`).
14. Si un producto cuesta \$250 y tiene un descuento del 30 %, ¿cuánto se descuenta? ¿Cuánto se paga al final?
15. Una clase tiene 35 estudiantes. Si se forman grupos de a 4, ¿cuántos grupos completos hay y cuántos estudiantes quedan sin grupo?
16. ¿Cuántas horas, minutos y segundos hay en 3 días? Calculá cada cosa por separado.
17. Creá un programa que pida dos números (asignados con variables) y muestre la suma, resta, multiplicación y división de esos dos números.
18. Creá un programa que convierta grados Celsius a Fahrenheit. Fórmula: $F = C * 9/5 + 32$. Probá con 0°C, 25°C y 100°C.
19. Si un viaje en auto consume 1 litro de nafta cada 15 kilómetros, ¿cuántos litros necesito para recorrer 320 kilómetros? ¿Y si el litro cuesta 1.20? Calculá el costo total.
20. Usando solo los operadores `//` y `%`, descomponé el número 4937 en sus dígitos individuales (miles, centenas, decenas, unidades). Ayuda: pensá en divisiones sucesivas por 10.

Soluciones paso a paso

Ejercicio 1

```
a = 38
b = 56
resultado = a + b
print("La suma es:", resultado) # 94
```

Ejercicio 2

```
a = 200
b = 87
resultado = a - b
print("La resta es:", resultado) # 113
```

Ejercicio 3

```
resultado = 12 * 9
print(resultado) # 108
```

Ejercicio 4

```
resultado = 156 / 13
print(resultado) # 12.0, devuelve float siempre
```

Ejercicio 5

```
resultado = 3 ** 6
print(resultado) # 729
```

Ejercicio 6

```
resultado = 100 // 7
print(resultado) # 14
```

Ejercicio 7

```
resultado = 100 % 7
print(resultado) # 2 (porque 14*7=98, sobran 2)
```

Ejercicio 8

```
resultado = (8 + 4) * (15 - 6)
# (12) * (9) = 108
print(resultado) # 108
```

Ejercicio 9

```
resultado = 100 / 5 / 4
# 100 / 5 = 20.0, luego 20.0 / 4 = 5.0
print(resultado) # 5.0
```

Ejercicio 10

```
resto = 256 % 10
print(resto) # 6 (es el ultimo digito del numero)
```

Ejercicio 11

```
parte_entera = 256 // 10
print(parte_entera) # 25 (todos los digitos menos el ultimo)
```

Ejercicio 12

```
base = 12
altura = 7
area = base * altura / 2
print(area) # 42.0
```

Ejercicio 13

```
lado1 = 8
lado2 = 5
perimetro = 2 * (lado1 + lado2)
print(perimetro) # 26
```

Ejercicio 14

```
precio = 250
descuento = precio * 30 / 100
precio_final = precio - descuento
print("Descuento:", descuento) # 75.0
print("Precio final:", precio_final) # 175.0
```

Ejercicio 15

```
estudiantes = 35
grupo_de = 4
grupos_completos = estudiantes // grupo_de
sobran = estudiantes % grupo_de
print("Grupos completos:", grupos_completos) # 8
print("Sobran:", sobran) # 3
```

Ejercicio 16

```
dias = 3
horas = dias * 24
minutos = horas * 60
segundos = minutos * 60
print("Horas:", horas)          # 72
print("Minutos:", minutos)      # 4320
print("Segundos:", segundos)    # 259200
```

Ejercicio 17

```
a = 15
b = 7
print("Suma:", a + b)           # 22
print("Resta:", a - b)          # 8
print("Multiplicacion:", a * b) # 105
print("Division:", a / b)        # 2.142857...
```

Ejercicio 18

```
c1 = 0
c2 = 25
c3 = 100
f1 = c1 * 9/5 + 32
f2 = c2 * 9/5 + 32
f3 = c3 * 9/5 + 32
print("0°C =", f1, "°F")        # 32.0 °F
print("25°C =", f2, "°F")        # 77.0 °F
print("100°C =", f3, "°F")       # 212.0 °F
```

Ejercicio 19

```
km = 320
litros_necesarios = km / 15
precio_por_litro = 1.20
costo_total = litros_necesarios * precio_por_litro
print("Litros:", litros_necesarios) # 21.333...
print("Costo:", costo_total)        # 25.6
```

Ejercicio 20

```
numero = 4937
miles = numero // 1000
resto1 = numero % 1000
centenas = resto1 // 100
resto2 = resto1 % 100
decenas = resto2 // 10
unidades = resto2 % 10
print("Miles:", miles)             # 4
print("Centenas:", centenas)       # 9
```



```
print("Decenas:", decenas)    # 3  
print("Unidades:", unidades) # 7
```

Ahora que tenés el poder de los números, podés empezar a construir cosas útiles: calculadoras, conversores, repartidores... ¡Lo que se te ocurra! En el próximo capítulo, a tomar decisiones con condiciones.

Sentencias condicionales simples - if

El día que mi programa tuvo que decidir

Cuando arranqué a programar, mis programas eran un embole. Siempre hacían lo mismo, sin importar nada. Ponías datos, tiraba resultados, chau. Pero un día necesité que el programa hiciera una cosa si el usuario era mayor de edad y otra si no. Ahí descubrí que los programas pueden **tomar decisiones**. Y eso cambió todo.

En este capítulo vamos a ver la herramienta más básica para decidir: el **if**. En criollo, es como decirle a Python: “Che, si pasa tal cosa, hacé esto”.

¿Qué es una sentencia condicional?

Una **sentencia condicional** es una bifurcación en el camino. Imaginate que vas por una ruta y ves un cartel: “Si el clima está lindo, doblá a la derecha para ir a la playa”. Si se cumple la condición (clima lindo), tomás ese camino. Si no, seguí derecho.

En Python, la condición siempre se evalúa como **True** (verdadero) o **False** (falso). Con eso ya sabés si doblás o no.

```
+-----+
| SENTENCIA CONDICIONAL = bifurcación en el |
| camino del programa.                       |
| SI se cumple la condición → hacé A         |
| SI NO se cumple → seguí de largo          |
+-----+
```

Estructura del if: la receta

La sintaxis del if tiene tres partes que no podés olvidar:

```
if condicion_logica:
    instrucción_1
    instrucción_2
    instrucción_3 # y todas las que quieras
```

Las reglas de oro:

- La línea del if termina con **dos puntos (:)**.

- Todo lo que querés que pase cuando se cumple la condición va **tabulado** (4 espacios o un TAB). Ese bloque tabulado es el “cuerpo” del `if`.
- Lo que no esté tabulado se ejecuta siempre, haya o no condición.

Reflexión: ¿Por qué Python necesita los dos puntos y la tabulación? Porque así sabe dónde empieza y termina el bloque que depende del `if`. Otros lenguajes usan llaves {}, Python usa la indentación. Al principio me volvía loco, después lo amás.

Probemos el if con un ejemplo concreto

Primer problema real: Quiero que el programa me avise si un número es igual a 5.

```
>>> num_uno = 5
>>> if num_uno == 5:
...     print('El numero es cinco')
...     print('Fin del if')
...
El numero es cinco
Fin del if
```

¿Qué pasó? Python evaluó `num_uno == 5`, vio que era `True`, y ejecutó todo lo tabulado. Simple. Ahora probemos con un número distinto:

```
>>> num_dos = 10
>>> if num_dos == 5:
...     print('El numero es cinco')
...     print('Fin del if')
...
>>> # No imprime nada porque la condición es False
```

Como `num_dos == 5` es `False`, Python se saltó todo el bloque tabulado y no pasó nada.

Los operadores que te preguntan “¿son iguales?”, “¿cuál es más grande?”

Para armar condiciones, necesitás **operadores de comparación**. Son como las preguntas que hacés en un formulario. Acá tenés todos:

OPERADOR	SIGNIFICADO	EJEMPLO
<code>==</code>	igual a	<code>5 == 5 → True</code>
<code>!=</code>	diferente de	<code>5 != 3 → True</code>
<code>></code>	mayor que	<code>5 > 3 → True</code>
<code><</code>	menor que	<code>3 < 5 → True</code>
<code>>=</code>	mayor o igual a	<code>5 >= 5 → True</code>
<code><=</code>	menor o igual a	<code>3 <= 5 → True</code>

Error que me persiguió: Yo escribía `if numero = 5:` y Python me tiraba error. ¿Por qué? Porque `=` es **asignación** (guardar), no comparación. Para preguntar si son iguales se usa `==`. Son dos caracteres, pero te pueden costar una tarde entera de debugging.

Ejemplo completo: ¿aprobaste o no?

Vamos a armar un programita que pide tres notas, calcula el promedio y te dice si aprobaste. Este fue uno de los primeros programas “útiles” que hice. Todavía lo recuerdo.

```
print('Sistema para calcular promedio')

nombre = input('¿Cual es tu nombre? ')

quimica = float(input('¿Cual es tu nota en quimica? '))
matematicas = float(input('¿Cual es tu nota en matematicas? '))
biologia = float(input('¿Cual es tu nota en biologia? '))

promedio = (quimica + matematicas + biologia) / 3

print('Tu promedio fue', promedio)

if promedio >= 7:
    print('Felicitaciones', nombre, 'aprobaste con un promedio de', promedio)
    print('¡Seguí así!')

print('Fin del programa')
```

Ejecución cuando aprobás:

```
Sistema para calcular promedio
¿Cual es tu nombre? Nahuel
¿Cual es tu nota en quimica? 8
¿Cual es tu nota en matematicas? 9
¿Cual es tu nota en biologia? 9
Tu promedio fue 8.666666666666666
Felicitaciones Nahuel aprobaste con un promedio de 8.666666666666666
¡Seguí así!
Fin del programa
```

Ejecución cuando no llegás:

```
Sistema para calcular promedio
¿Cual es tu nombre? Nahuel
¿Cual es tu nota en quimica? 5
¿Cual es tu nota en matematicas? 6
¿Cual es tu nota en biologia? 4
Tu promedio fue 5.0
Fin del programa
```

Fijate que en el segundo caso no se imprimen las felicitaciones. La condición `promedio >= 7` dio `False` y Python saltó todo el bloque tabulado. Pero “Fin del programa” sí se ve, porque está afuera del `if`.

Más ejemplos con if para que practiques

¿Es positivo?

```
numero = int(input('Ingrese un numero: '))
if numero > 0:
    print('El numero es positivo')
```

¿Es par?

```
numero = int(input('Ingrese un numero: '))
if numero % 2 == 0:
    print('El numero es par')
```

¿Sos mayor de edad?

```
edad = int(input('Ingrese su edad: '))
if edad >= 18:
    print('Sos mayor de edad')
```

¿El texto es largo?

```
texto = input('Ingrese un texto: ')
if len(texto) > 10:
    print('El texto tiene mas de 10 caracteres')
```

Ejercicio rápido: Con lo que viste hasta ahora, escribí un programa que pida tu ciudad y, si es “Buenos Aires”, muestre “¡Yo también soy de ahí!”. Después verificá con la solución.

Errores comunes con if (me los sé de memoria)

1. **Olvidar los dos puntos:** `if numero == 5 sin : al final.` `SyntaxError`.
2. **No tabular el bloque:** `if numero == 5: print('ok')` sin tabulación. `IndentationError`.
3. **Usar = en vez de ==:** `if numero = 5:.` `SyntaxError`. Recordá: = asigna, == pregunta.
4. **Condiciones imposibles:** `if numero >10 and numero <5:.` Ningún número cumple eso. Siempre da `False`.
5. **Olvidar convertir el input:** `edad = input(); if edad >= 18:.` Comparás texto con número y da `TypeError`. Usá `int(input())`.

Dale, repasemos rápido lo importante

- El if ejecuta código solo si una condición es `True`.
- Sintaxis: `if condicion: + bloque tabulado`.
- Operadores de comparación: `==, !=, >, <, >=, <=`.
- `=` asigna, `==` compara. No los mezcles.

- Lo que no esté tabulado se ejecuta siempre, no depende del `if`.

Con el `if`, tus programas empiezan a tener personalidad. Ya no son robots tontos, ahora pueden decidir. En el próximo capítulo vamos a ver el `else`, para hacer “si no, hacé esto otro”.

Ejercicios del capítulo

1. Escribí un programa que pida un número y diga si es igual a 10.
2. Escribí un programa que pida un número y diga si es mayor que 100.
3. Escribí un programa que pida un número y diga si es negativo (menor que 0).
4. Escribí un programa que pida la edad y diga si es mayor de edad (18 o más).
5. Escribí un programa que pida dos números y diga si el primero es mayor que el segundo.
6. Escribí un programa que pida una letra y diga si es la letra 'a'.
7. Escribí un programa que pida un número y diga si es par (usá el módulo %).
8. Escribí un programa que pida un número y diga si es múltiplo de 5.
9. Escribí un programa que pida la temperatura y diga si hace calor (más de 30 grados).
10. Escribí un programa que pida una contraseña y diga si es exactamente “python123”.
11. Escribí un programa que pida tres notas y diga si el promedio es mayor o igual a 7.
12. Escribí un programa que pida un número y diga si es diferente de 0.
13. Escribí un programa que pida un año y diga si es bisiesto (múltiplo de 4). Ojo: hay reglas más complejas, pero por ahora usá solo esa.
14. Escribí un programa que pida un número y diga si está entre 10 y 20 (inclusive). Usá `and`.
15. Escribí un programa que pida un carácter y diga si es una vocal (a, e, i, o, u). Usá `or`.
16. Escribí un programa que pida el total de una compra y, si es mayor a \$10000, diga “Tenés envío gratis”.
17. Escribí un programa que pida un número y diga si es cero.
18. Escribí un programa que pida un nombre y, si es exactamente “Python”, diga “¡Bienvenido!”.
19. Escribí un programa que pida dos números y diga si son iguales.
20. Escribí un programa que pida las notas de 3 materias, calcule el promedio y, si es mayor a 8, imprima “Cuadro de honor”.

Soluciones paso a paso

Ejercicio 1

```
numero = int(input("Ingrese un numero: "))
if numero == 10:
    print("El numero es igual a 10")
```

Ejercicio 2

```
numero = int(input("Ingrese un numero: "))
if numero > 100:
    print("El numero es mayor que 100")
```

Ejercicio 3

```
numero = int(input("Ingrese un numero: "))
if numero < 0:
    print("El numero es negativo")
```

Ejercicio 4

```
edad = int(input("Ingrese su edad: "))
if edad >= 18:
    print("Sos mayor de edad")
```

Ejercicio 5

```
num1 = int(input("Primer numero: "))
num2 = int(input("Segundo numero: "))
if num1 > num2:
    print("El primer numero es mayor que el segundo")
```

Ejercicio 6

```
letra = input("Ingrese una letra: ")
if letra == 'a':
    print("Es la letra a")
```

Ejercicio 7

```
numero = int(input("Ingrese un numero: "))
if numero % 2 == 0:
    print("El numero es par")
```

Ejercicio 8

```
numero = int(input("Ingrese un numero: "))
if numero % 5 == 0:
    print("El numero es multiplo de 5")
```

Ejercicio 9

```
temperatura = float(input("Ingrese la temperatura: "))
if temperatura > 30:
    print("Hace calor")
```

Ejercicio 10

```
contrasena = input("Ingrese la contraseña: ")
if contrasena == "python123":
    print("Contraseña correcta")
```

Ejercicio 11

```
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))
promedio = (n1 + n2 + n3) / 3
if promedio >= 7:
    print("Aprobaste con promedio", promedio)
```

Ejercicio 12

```
numero = int(input("Ingrese un numero: "))
if numero != 0:
    print("El numero es diferente de cero")
```

Ejercicio 13

```
anio = int(input("Ingrese un año: "))
if anio % 4 == 0:
    print("Es año bisiesto")
```

Ejercicio 14

```
numero = int(input("Ingrese un numero: "))
if numero >= 10 and numero <= 20:
    print("El numero esta entre 10 y 20")
```

Ejercicio 15

```
caracter = input("Ingrese un caracter: ")
if caracter == 'a' or caracter == 'e' or caracter == 'i' or caracter == 'o' or caracter == 'u':
    print("Es una vocal")
```

Ejercicio 16

```
total = float(input("Total de la compra: $"))
if total > 10000:
    print("¡Tenés envío gratis!")
```

Ejercicio 17

```
numero = int(input("Ingrese un numero: "))
if numero == 0:
    print("El numero es cero")
```


Ejercicio 18

```
nombre = input("Ingrese un nombre: ")
if nombre == "Python":
    print("¡Bienvenido!")
```

Ejercicio 19

```
num1 = int(input("Primer numero: "))
num2 = int(input("Segundo numero: "))
if num1 == num2:
    print("Los numeros son iguales")
```

Ejercicio 20

```
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))
promedio = (n1 + n2 + n3) / 3
if promedio > 8:
    print("Cuadro de honor. Promedio:", promedio)
```

El if te abre las puertas a programas que reaccionan. Ya podés simular un cajero automático, un sistema de notas o un detector de contraseñas. Practicá estos ejercicios y después inventá los tuyos. ¡La práctica hace al programador!

Sentencias condicionales compuestas (if-else)

El día que necesité un “plan B”

Cuando empecé con el `if` estaba chocho. Ya podía hacer que el programa reaccionara. Pero enseguida me topé con una limitación: ¿qué pasa cuando la condición no se cumple? Mi programa se quedaba mudo, sin hacer nada. Era como un semáforo que solo tiene luz verde.

Un día tuve que hacer un sistema de login. Si la contraseña era correcta, todo bien. Pero si no, quería mostrar un mensaje de error. Para eso necesitaba un “plan B”. Y ahí apareció el `else`.

El `if-else` es como un semáforo completo: si es verde, avanzás; **si no** (else), te frenás. En criollo: “si pasa esto, hacé A; si no, hacé B”.

¿Cómo funciona el if-else?

La sentencia condicional compuesta `if-else` evalúa una condición. Si es `True`, ejecuta el bloque del `if`. Si es `False`, ejecuta el bloque del `else`. Nunca los dos. Es una cosa o la otra.

```
+-----+
|
|  SI la condicion es VERDADERA → camino A
|  SINO (else) → camino B
|
|  No hay grises. Es blanco o negro.
|
+-----+
```

Estructura del if-else: la receta

La sintaxis es muy parecida a la del `if` simple, pero le agregás el bloque `else`:

```
if condicion_logica:
    # bloque que se ejecuta si la condicion es True
    instruccion_1
    instruccion_2
else:
    # bloque que se ejecuta si la condicion es False
    instruccion_3
    instruccion_4
```

Reglas que no podés olvidar:

- El **else** va al **mismo nivel** que el **if**, sin tabular.
- Después del **else** van **dos puntos (:)**, igual que el **if**.
- El bloque del **else** va **tabulado**, igual que el del **if**.
- Solo **uno** de los dos bloques se ejecuta. Nunca los dos.

Ejemplo 1: ¿Es cinco o no es cinco?

Arranquemos con el ejemplo más simple. Quiero saber si un número es igual a 5 o no.

```
num_uno = 20
if num_uno == 5:
    print("El numero es cinco")
else:
    print("El numero NO es cinco")
```

¿Qué pasó? Python evaluó `num_uno == 5`. Como `num_uno` vale 20, la comparación da `False`. Entonces se saltó el bloque del `if` y ejecutó el del `else`. La salida es “El numero NO es cinco”.

Ahora cambiá el valor a 5 y volvé a correrlo mentalmente. ¿Qué bloque se ejecuta ahora? Exacto, el del `if`. Vas a ver “El numero es cinco”.

Reflexión: Fijate que sin el `else`, si el número no era 5 el programa no hacía nada. Con el `else` siempre hay una respuesta, pase lo que pase.

Ejemplo 2: Mayor o menor de edad

Este fue mi primer programa “con consecuencias”. Según la edad, te decía si podías votar o no.

```
edad = int(input("Ingrese su edad: "))
if edad >= 18:
    print("Eres mayor de edad")
    print("Puedes votar")
else:
    print("Eres menor de edad")
    print("No puedes votar")
```

Ejecución con 20:

```
Ingrese su edad: 20
Eres mayor de edad
Puedes votar
```

Ejecución con 15:

```
Ingrese su edad: 15
Eres menor de edad
No puedes votar
```

Anécdota: La primera vez que probé esto puse mal la tabulación y el `else` me tiró error de indentación. Estuve diez minutos mirando el código sin entender nada. El problema era un espacio de más. Python es quisquilloso con eso, pero una vez que te acostumbras, te ayuda a leer mejor el código.

Ejemplo 3: Promedio con final feliz o triste

Vamos a mejorar el sistema de promedios del capítulo anterior. Ahora, si aprobás te felicita, y si no, te dice que estudiés más.

```
print("Sistema para calcular promedio")

nombre = input("¿Cual es tu nombre? ")

quimica = float(input("Nota de quimica: "))
matematicas = float(input("Nota de matematicas: "))
biologia = float(input("Nota de biologia: "))

promedio = (quimica + matematicas + biologia) / 3

if promedio >= 7:
    print("Felicitaciones", nombre, "aprobaste!")
    print("Tu promedio fue:", promedio)
else:
    print("Lo siento", nombre, "no aprobaste")
    print("Tu promedio fue:", promedio)
    print("Deberias estudiar mas")
```

Probá en tu cabeza: Si sacaste 8, 9 y 7, el promedio es 8.0. ¿Qué mensajes ves? ¿Y si sacaste 5, 6 y 4?

El problemita de los decimales largos

Cuando ejecuté el programa anterior con notas 8, 9 y 9, el promedio me dio 8.666666666666666. Un espanto visual. Para dejarlo más lindo, Python nos da la función `round()`.

`round()` es como un peluquero: le decís cuántos decimales querés y él corta el resto.

```
>>> promedio = 8.666666666666666
>>> print(round(promedio, 2))
8.67
>>> print(round(promedio, 1))
8.7
>>> print(round(promedio, 0))
9.0
```

Ejemplo aplicado al sistema de promedios:

```
promedio = (quimica + matematicas + biologia) / 3
promedio_redondeado = round(promedio, 2)

if promedio >= 7:
    print("Felicitaciones", nombre, "aprobaste con", promedio_redondeado)
else:
    print("Lo siento", nombre, "no aprobaste. Promedio:", promedio_redondeado)
```

Ejercicio rápido: Modificá el programa para que también redondee a 1 decimal. ¿Qué cambia en la salida?

Ejemplo 4: Par o impar

Un clásico que nunca falla. Queremos saber si un número es par o impar.

```
numero = int(input("Ingrese un numero: "))
if numero % 2 == 0:
    print(numero, "es un numero par")
else:
    print(numero, "es un numero impar")
```

¿Recordás el operador%? Da el resto de la división. Si al dividir por 2 el resto es 0, es par. Si es 1, es impar. Así de simple.

Ejecución de prueba:

```
Ingrese un numero: 7
7 es un numero impar
```

```
Ingrese un numero: 10
10 es un numero par
```

Ejemplo 5: Con tres caminos (adelanto de elif)

Hay veces que dos ramas no alcanzan. Por ejemplo, para clasificar un número en positivo, negativo o cero. En el próximo capítulo vamos a ver `elif` en detalle, pero te lo muestro de regalo:

```
numero = float(input("Ingrese un numero: "))
if numero > 0:
    print("El numero es positivo")
elif numero < 0:
    print("El numero es negativo")
else:
    print("El numero es cero")
```

`elif` es “else if”, otro escalón de decisión. Pero por ahora concentrémonos en el `if-else` de dos caminos.

if simple vs if-else: el duelo

	IF SIMPLE	IF-ELSE
Estructura	if cond: accion	if cond: accion_A else: accion_B
Cuando se ejecuta algo	Solo si es True	Siempre, sea True o False
Analogia	Si llueve, llevo paraguas (si no, nada)	Si llueve, llevo paraguas Si no, llevo anteojos de sol

Errores que me hicieron putear (y cómo evitarlos)

1. **Else sin if:** El `else` no puede ir solo. Siempre necesita un `if` antes. Es como un vagón que no puede ir sin locomotora.
2. **Olvidar los dos puntos en el else:** `else sin :` tira `SyntaxError`.
3. **Tabulación desapareja:** El `if` y el `else` tienen que estar al mismo nivel. Si uno está más adentro que el otro, `IndentationError`.
4. **Mezclar `=` y `==`:** En la condición va `==`. Si ponés `=`, Python piensa que querés asignar un valor dentro del `if` y explota.
5. **No convertir el input:** Si comparás `input()` con un número sin convertirlo, comparás texto con número y el resultado siempre es `False` (o `TypeError`, según el caso).

Dale, repasemos rápido lo importante

- El `if-else` ejecuta **un bloque u otro** según la condición.
- Estructura: `if cond: (bloque True) + else: (bloque False)`.
- Ambos bloques van tabulados. `else` va al mismo nivel que `if`.
- `round(numero, decimales)` redondea para mostrar resultados más limpios.
- Con `if-else` siempre hay una respuesta. Tu programa nunca se queda mudo.

El if-else es la base de cualquier decisión binaria: prendido/apagado, aprobado/desaprobado, adentro/afuera. Dominá esto y vas a poder construir lógica más compleja con `elif`.

Ejercicios del capítulo

1. Escribí un programa que pida un número y diga si es positivo o negativo (el 0 como positivo por ahora).
2. Escribí un programa que pida la edad y diga si es mayor o menor de edad.
3. Escribí un programa que pida dos números y diga cuál es el mayor. Si son iguales, decí que son iguales.
4. Escribí un programa que pida una contraseña y diga si es correcta ("**secreta**") o incorrecta.
5. Escribí un programa que pida un número y diga si es par o impar.
6. Escribí un programa que pida la temperatura y diga si hace calor (más de 25) o frío (25 o menos).
7. Escribí un programa que pida una calificación (0-10) y diga si aprobó (6 o más) o desaprobó.
8. Escribí un programa que pida un año y diga si es bisesto (múltiplo de 4) o no.
9. Escribí un programa que pida un carácter y diga si es vocal o consonante (simplificado).
10. Escribí un programa que pida el total de una compra y diga si tiene descuento (más de \$10000) o no.

11. Escribí un programa que pida tres notas, calcule el promedio y diga si aprobó (≥ 7) o desaprobó.
12. Escribí un programa que pida un nombre y, si es "Nahuel", diga "Hola Nahuel"; si no, diga "Vos no sos Nahuel".
13. Escribí un programa que pida un número y diga si es múltiplo de 3 o no.
14. Escribí un programa que pida una letra y diga si es mayúscula o minúscula. Pista: `letra.isupper()`.
15. Escribí un programa que pida un número y diga si es mayor que 100 o menor/igual.
16. Escribí un programa que pida la base y la altura de un rectángulo y diga si es un cuadrado o no.
17. Escribí un programa que pida un número y diga si es divisible por 2 y por 3 a la vez, o no.
18. Escribí un programa que pida 3 notas, calcule el promedio redondeado a 2 decimales, y diga si aprobó (≥ 7) o no.
19. Escribí un programa que pida la velocidad de un auto y diga si excede el límite de 80 km/h o no.
20. Escribí un programa que pida un número y diga si es cero o no es cero.

Soluciones paso a paso

Ejercicio 1

```
numero = float(input("Ingrese un numero: "))
if numero >= 0:
    print("Positivo")
else:
    print("Negativo")
```

Ejercicio 2

```
edad = int(input("Ingrese su edad: "))
if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

Ejercicio 3

```
a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))
if a > b:
    print(a, "es mayor que", b)
elif a < b:
    print(b, "es mayor que", a)
else:
    print("Son iguales")
```

Ejercicio 4

```
clave = input("Ingrese contraseña: ")
if clave == "secreta":
    print("Correcta")
else:
    print("Incorrecta")
```

Ejercicio 5

```
num = int(input("Ingrese un numero: "))
if num % 2 == 0:
    print("Par")
else:
    print("Impar")
```

Ejercicio 6

```
temp = float(input("Ingrese la temperatura: "))
if temp > 25:
    print("Hace calor")
else:
    print("Hace frio")
```

Ejercicio 7

```
nota = float(input("Ingrese la calificacion (0-10): "))
if nota >= 6:
    print("Aprobado")
else:
    print("Desaprobado")
```

Ejercicio 8

```
anio = int(input("Ingrese un año: "))
if anio % 4 == 0:
    print("Es bisiesto")
else:
    print("No es bisiesto")
```

Ejercicio 9

```
letra = input("Ingrese una letra: ")
if letra.lower() in 'aeiou':
    print("Vocal")
else:
    print("Consonante")
```


Ejercicio 10

```
total = float(input("Total de la compra: $"))
if total > 10000:
    print("Tiene descuento")
else:
    print("No tiene descuento")
```

Ejercicio 11

```
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))
promedio = (n1 + n2 + n3) / 3
if promedio >= 7:
    print("Aprobado con", promedio)
else:
    print("Desaprobado con", promedio)
```

Ejercicio 12

```
nombre = input("Ingrese su nombre: ")
if nombre == "Nahuel":
    print("Hola Nahuel")
else:
    print("Vos no sos Nahuel")
```

Ejercicio 13

```
num = int(input("Ingrese un numero: "))
if num % 3 == 0:
    print("Es multiplo de 3")
else:
    print("No es multiplo de 3")
```

Ejercicio 14

```
letra = input("Ingrese una letra: ")
if letra.isupper():
    print("Mayuscula")
else:
    print("Minuscula")
```

Ejercicio 15

```
num = float(input("Ingrese un numero: "))
if num > 100:
    print("Mayor que 100")
else:
    print("Menor o igual a 100")
```

Ejercicio 16

```
base = float(input("Base: "))
altura = float(input("Altura: "))
if base == altura:
    print("Es un cuadrado")
else:
    print("No es un cuadrado")
```

Ejercicio 17

```
num = int(input("Ingrese un numero: "))
if num % 2 == 0 and num % 3 == 0:
    print("Divisible por 2 y por 3")
else:
    print("No es divisible por ambos")
```

Ejercicio 18

```
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))
promedio = round((n1 + n2 + n3) / 3, 2)
if promedio >= 7:
    print("Aprobado con", promedio)
else:
    print("Desaprobado con", promedio)
```

Ejercicio 19

```
velocidad = float(input("Velocidad en km/h: "))
if velocidad > 80:
    print("Excede el límite de velocidad")
else:
    print("Velocidad permitida")
```

Ejercicio 20

```
num = float(input("Ingrese un numero: "))
if num == 0:
    print("Es cero")
else:
    print("No es cero")
```

Sentencias condicionales múltiples (if-elif-else)

Cuando dos caminos no alcanzan

Con el `if-else` ya podía tomar decisiones binarias: prendido o apagado, aprobado o desaprobado. Pero un día quise clasificar notas en Excelente, Muy Bien, Bien, Regular y Mal. Cinco categorías. Con `if-else` pelado no me alcanzaba. Ahí descubrí el **elif**, una bendición para cuando el árbol de decisiones se ramifica.

Pensá en un menú de restaurante: no es solo carne o no carne”. Es milanesa, fideos, pizza, ensalada... Cada plato es una opción. El `if-elif-else` es justamente eso: un menú de opciones. Elegís una, y Python te sirve ese plato.

```
+-----+
|
|  MULTIPLES CONDICIONES = muchas ramas
|
|  SI tiene mucha hambre → milanesa
|  SI quiere algo ligero → ensalada
|  SI tiene antojo de algo rico → pizza
|  SINO → lo que sobre del mediodía
|
+-----+
```

Estructura del if-elif-else: la receta

La sintaxis es una extensión natural del `if-else` que ya conocés. Le agregás tantos `elif` como necesites:

```
if condicion_1:
    # bloque si condicion_1 es True
    instruccion_1
elif condicion_2:
    # bloque si condicion_2 es True
    instruccion_2
elif condicion_3:
    # bloque si condicion_3 es True
    instruccion_3
else:
    # bloque si ninguna de las anteriores es True
    instruccion_4
```

Reglas fundamentales:

- `elif` significa **else if** (sino, si...). Es una segunda oportunidad.
- Podés poner todos los `elif` que quieras. Uno, cinco, veinte.
- Se evalúan en orden. Python empieza por el primer `if`, sigue con los `elif`, y si ninguno se cumple, va al `else`.
- Cuando encuentra **la primera condición verdadera**, ejecuta ese bloque y se sale. No sigue evaluando las demás.
- El `else` es opcional. Si no lo ponés y ninguna condición se cumple, no pasa nada.

Reflexión: ¿Por qué se sale en la primera verdadera? Porque las condiciones son excluyentes. Si sos mayor de 60, también sos mayor de 18. Pero no queremos que te clasifique como adulto y como adulto mayor al mismo tiempo. Por eso el orden importa.

Ejemplo 1: De notas numéricas a palabras

Este fue el primer `if-elif-else` que escribí. Quería que el programa me diga `.Excelente` si sacaba más de 9, `.Aprobado` si más de 7, y así.

```
nota = float(input("Ingrese su nota: "))

if nota >= 9:
    print("Excelente! Muy bien")
elif nota >= 7:
    print("Aprobado. Bien hecho")
elif nota >= 5:
    print("Regular. Puedes mejorar")
else:
    print("Desaprobado. Estudia mas")
```

¿Por qué funciona así? Suponé que ponés 9.5. Python evalúa `nota >= 9`, es `True`, imprime `.Excelente` y sale. No evalúa `nota >= 7` ni `nota >= 5`. Si ponés 8, la primera es `False`, pasa al `elif nota >= 7`, que es `True`, imprime y sale.

Error clásico que me mandé: Poner las condiciones en orden inverso.

```
if nota >= 5:          # un 9.5 entra acá y dice "Regular". ¡Mal!
    print("Regular")
elif nota >= 7:
    print("Aprobado")
elif nota >= 9:
    print("Excelente")
```

Un 9.5 es mayor o igual a 5, así que entra al primer `if` y nunca llega a `.Excelente`. **Moraleja:** las condiciones más estrictas van primero.

Ejemplo 2: Días de la semana

Otro clásico. Querés traducir un número del 1 al 7 al nombre del día.

```
dia = int(input("Ingrese un numero del 1 al 7: "))

if dia == 1:
    print("Lunes")
elif dia == 2:
    print("Martes")
elif dia == 3:
    print("Miercoles")
elif dia == 4:
    print("Jueves")
elif dia == 5:
    print("Viernes")
elif dia == 6:
    print("Sabado")
elif dia == 7:
    print("Domingo")
else:
    print("Numero invalido. Debe ser entre 1 y 7")
```

Fijate que acá todas las condiciones son excluyentes por naturaleza: un número no puede ser 1 y 2 al mismo tiempo. Así que el orden no es tan crítico, pero igual conviene ponerlas ordenadas.

Ejemplo 3: Clasificador de IMC

Un ejemplo más real. El índice de masa corporal (IMC) clasifica el peso en categorías según rangos.

```
print("Calculadora de IMC")
peso = float(input("Ingrese su peso (kg): "))
altura = float(input("Ingrese su altura (m): "))

imc = peso / (altura ** 2)
imc_redondeado = round(imc, 2)

print("Su IMC es:", imc_redondeado)

if imc < 18.5:
    print("Bajo peso")
elif imc < 25:
    print("Peso normal")
elif imc < 30:
    print("Sobrepeso")
else:
    print("Obesidad")
```

Ojo con el orden acá: Si pusieras `if imc < 25` primero, un IMC de 17 (bajo peso) caería en "Peso normal". Error grave. Siempre de menor a mayor (o de más específico a más general).

Ejemplo 4: Positivo, negativo o cero

Un problema de tres caminos. No alcanza con `if-else`.

```
numero = int(input("Ingrese un numero: "))

if numero > 0:
    print("El numero es positivo")
elif numero < 0:
    print("El numero es negativo")
else:
    print("El numero es cero")
```

¿Qué pasa si ponés 0? El primer `if` da `False` (0 no es mayor a 0), el `elif` también da `False` (0 no es menor a 0), así que va al `else` y dice `cero`. Perfecto.

Ejemplo 5: Etapas de la vida

Este me gusta porque muestra cómo los rangos se construyen con límites.

```
edad = int(input("Ingrese su edad: "))

if edad < 12:
    print("Niño")
elif edad < 18:
    print("Adolescente")
elif edad < 60:
    print("Adulto")
else:
    print("Adulto mayor")
```

Pensá en una edad de 10. Entra al primer `if` porque `10 < 12`. Una de 15: `False` el primero, `True` el segundo (`15 < 18`). Una de 55: `False` los dos primeros, `True` el tercero. Una de 70: `False` todos, va al `else`.

Ejemplo 6: Notas a letras al estilo universitario

```
nota = float(input("Ingrese su nota (0-10): "))

if nota >= 9.5:
    letra = "A+"
elif nota >= 9:
    letra = "A"
elif nota >= 8:
    letra = "B+"
elif nota >= 7:
    letra = "B"
elif nota >= 6:
    letra = "C"
elif nota >= 5:
```

```

    letra = "D"
else:
    letra = "F"

print("Tu calificacion es:", letra)

```

Ejercicio rápido: ¿Qué letra corresponde a un 8.5? ¿Y a un 9.3? ¿Y a un 9.5? Hacé la traza mental antes de ejecutar.

Tabla comparativa: if, if-else, if-elif-else

TIPO	ESTRUCTURA	CUANDO USAR
if simple	if cond: accion	Una sola condición Si no se cumple, nada
if-else	if cond: accion1 else: accion2	Dos opciones Una cosa o la otra
if-elif-else	if cond1: accion1 elif cond2: accion2 elif cond3: accion3 else: accion4	Tres o más opciones Cada una es un camino

Errores que me hicieron renegar

- Orden de las condiciones:** Como ya te conté, si ponés lo general antes que lo específico, nunca vas a llegar a lo específico. Pensá en un filtro: lo más fino primero.
- Olvidar el else:** A veces es necesario. Si hay una posibilidad que no cubriste con ningún if ni elif, el programa no hace nada y queda un vacío. El **else** es tu red de seguridad.
- Poner elif sin if:** No se puede. **elif** es un vagón que necesita la locomotora del **if** adelante.
- Condiciones superpuestas sin querer:** Por ejemplo, `if edad < 18` y después `elif edad > 12`. Los de 15 caen en el primero y nunca llegan al segundo, aunque ahí también serían adolescentes. Solución: ordenar bien.
- Mezclar elif con if separados:** Si ponés varios if sueltos en lugar de **elif**, Python evalúa todos y puede ejecutar más de un bloque. A veces eso es lo que querés, pero generalmente no.

Dale, repasemos rápido lo importante

- `if-elif-else` permite evaluar múltiples condiciones en orden.
- `elif` es “else if”: una condición alternativa.
- Python ejecuta **solo el primer bloque** cuya condición sea `True`.
- El `else` final es opcional y cubre “todo lo demás”.
- **Ordená las condiciones de la más específica a la más general.**
- Si necesitás tres o más caminos, `if-elif-else` es tu herramienta.

Con el `if-elif-else` tus programas ya pueden clasificar, categorizar y tomar decisiones complejas. Estás cada vez más cerca de construir aplicaciones reales.

Ejercicios del capítulo

1. Escribí un programa que pida un número del 1 al 12 y muestre el nombre del mes correspondiente. Si no está en ese rango, avisá.
2. Escribí un programa que pida una nota (0-10) y muestre: Excelente (9-10), Aprobado (7-8), Regular (5-6), Desaprobado (0-4).
3. Escribí un programa que pida la edad y clasifique: Niño (0-12), Adolescente (13-17), Adulto (18-59), Adulto mayor (60 o más).
4. Escribí un programa que pida un número y diga si es positivo, negativo o cero.
5. Escribí un programa que pida dos números y muestre el mayor, el menor o si son iguales.
6. Escribí un programa que pida una letra y diga si es vocal, consonante o “no es una letra válida”.
7. Escribí un programa que pida la hora (0-23) y diga: Mañana (6-11), Tarde (12-17), Noche (18-23), Madrugada (0-5).
8. Escribí un programa que calcule el IMC y clasifique: Bajo peso (<18.5), Normal (18.5-24.9), Sobrepeso (25-29.9), Obesidad (≥ 30).
9. Escribí un programa que pida tres números distintos y muestre cuál es el mayor.
10. Escribí un programa que pida el total de una compra y aplique descuento según el monto: más de 50000 (20 %), más de 20000 (15 %), más de 10000 (10 %), sino sin descuento.
11. Escribí un programa que pida un carácter y diga si es número, letra mayúscula, letra minúscula o símbolo.
12. Escribí un programa que pida el precio de un producto y pregunte si es comida (sí/no). Si no es comida, sumale 21 % de IVA.
13. Escribí un programa que pida la distancia de un viaje en km y calcule el costo: hasta 10 km \$100 c/u, hasta 50 km \$80 c/u, hasta 100 km \$60 c/u, más de 100 km \$50 c/u.
14. Escribí un programa que pida un año y diga si es bisiesto. Regla: divisible por 4, pero no por 100, excepto si es divisible por 400.

15. Escribí un programa que pida tres notas, calcule el promedio y diga: Sobresaliente (≥ 9), Aprobado (≥ 7), Regular (≥ 5), Desaprobado (≥ 5).
16. Escribí un programa que pida dos números y una operación (+, -, *, /) y muestre el resultado. Si la división es por cero, avisá.
17. Escribí un programa que pida la estación del año (1=Verano, 2=Otoño, 3=Invierno, 4=Primavera) y muestre los meses típicos de esa estación.
18. Escribí un programa que pida un número del 1 al 100 y diga en qué rango cae: 1-25, 26-50, 51-75, 76-100.
19. Escribí un programa que pida la velocidad de un auto y calcule la multa: hasta 80 sin multa, 81-100 leve, 101-120 grave, más de 120 muy grave.
20. Escribí un programa que pida un número del 1 al 7 y diga si es día laboral (1-5) o fin de semana (6-7). Si el número es inválido, avisá.

Soluciones paso a paso

Ejercicio 1

```
mes = int(input("Numero del 1 al 12: "))
if mes == 1:
    print("Enero")
elif mes == 2:
    print("Febrero")
elif mes == 3:
    print("Marzo")
elif mes == 4:
    print("Abril")
elif mes == 5:
    print("Mayo")
elif mes == 6:
    print("Junio")
elif mes == 7:
    print("Julio")
elif mes == 8:
    print("Agosto")
elif mes == 9:
    print("Septiembre")
elif mes == 10:
    print("Octubre")
elif mes == 11:
    print("Noviembre")
elif mes == 12:
    print("Diciembre")
else:
    print("Numero invalido")
```

Ejercicio 2

```
nota = float(input("Nota (0-10): "))
```

```
if nota >= 9:
    print("Excelente")
elif nota >= 7:
    print("Aprobado")
elif nota >= 5:
    print("Regular")
else:
    print("Desaprobado")
```

Ejercicio 3

```
edad = int(input("Edad: "))
if edad < 13:
    print("Niño")
elif edad < 18:
    print("Adolescente")
elif edad < 60:
    print("Adulto")
else:
    print("Adulto mayor")
```

Ejercicio 4

```
num = float(input("Numero: "))
if num > 0:
    print("Positivo")
elif num < 0:
    print("Negativo")
else:
    print("Cero")
```

Ejercicio 5

```
a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))
if a > b:
    print(a, "es mayor que", b)
elif b > a:
    print(b, "es mayor que", a)
else:
    print("Son iguales")
```

Ejercicio 6

```
letra = input("Ingrese una letra: ").lower()
if len(letra) != 1:
    print("Ingrese un solo caracter")
elif letra in 'aeiou':
    print("Es vocal")
elif letra.isalpha():
    print("Es consonante")
```

```
else:  
    print("No es una letra valida")
```

Ejercicio 7

```
hora = int(input("Hora (0-23): "))  
if hora < 6:  
    print("Madrugada")  
elif hora < 12:  
    print("Mañana")  
elif hora < 18:  
    print("Tarde")  
else:  
    print("Noche")
```

Ejercicio 8

```
peso = float(input("Peso (kg): "))  
altura = float(input("Altura (m): "))  
imc = peso / (altura ** 2)  
if imc < 18.5:  
    print("Bajo peso")  
elif imc < 25:  
    print("Peso normal")  
elif imc < 30:  
    print("Sobrepeso")  
else:  
    print("Obesidad")
```

Ejercicio 9

```
a = float(input("Numero 1: "))  
b = float(input("Numero 2: "))  
c = float(input("Numero 3: "))  
if a >= b and a >= c:  
    print("El mayor es", a)  
elif b >= a and b >= c:  
    print("El mayor es", b)  
else:  
    print("El mayor es", c)
```

Ejercicio 10

```
total = float(input("Total de la compra: $"))  
if total > 50000:  
    desc = 0.20  
elif total > 20000:  
    desc = 0.15  
elif total > 10000:  
    desc = 0.10  
else:
```

```
desc = 0
print("Descuento: $", total * desc)
print("Total a pagar: $", total - (total * desc))
```

Ejercicio 11

```
car = input("Ingrese un caracter: ")
if car.isdigit():
    print("Es un numero")
elif car.isupper():
    print("Letra mayuscula")
elif car.islower():
    print("Letra minuscula")
else:
    print("Es un simbolo")
```

Ejercicio 12

```
precio = float(input("Precio del producto: $"))
comida = input("¿Es comida? (si/no): ")
if comida.lower() == "si":
    print("Precio final: $", precio)
else:
    print("Precio final con IVA: $", round(precio * 1.21, 2))
```

Ejercicio 13

```
dist = float(input("Distancia (km): "))
if dist <= 10:
    costo = dist * 100
elif dist <= 50:
    costo = dist * 80
elif dist <= 100:
    costo = dist * 60
else:
    costo = dist * 50
print("Costo del viaje: $", costo)
```

Ejercicio 14

```
anio = int(input("Año: "))
if anio % 400 == 0:
    print("Bisiesto")
elif anio % 100 == 0:
    print("No bisiesto")
elif anio % 4 == 0:
    print("Bisiesto")
else:
    print("No bisiesto")
```

Ejercicio 15

```
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))
prom = (n1 + n2 + n3) / 3
print("Promedio:", round(prom, 2))
if prom >= 9:
    print("Sobresaliente")
elif prom >= 7:
    print("Aprobado")
elif prom >= 5:
    print("Regular")
else:
    print("Desaprobado")
```

Ejercicio 16

```
a = float(input("Primer numero: "))
b = float(input("Segundo numero: "))
op = input("Operacion (+, -, *, /): ")
if op == "+":
    print("Resultado:", a + b)
elif op == "-":
    print("Resultado:", a - b)
elif op == "*":
    print("Resultado:", a * b)
elif op == "/":
    if b != 0:
        print("Resultado:", a / b)
    else:
        print("Error: division por cero")
else:
    print("Operacion no valida")
```

Ejercicio 17

```
est = int(input("1=Verano 2=Otoño 3=Invierno 4=Primavera: "))
if est == 1:
    print("Enero, Febrero, Marzo")
elif est == 2:
    print("Abril, Mayo, Junio")
elif est == 3:
    print("Julio, Agosto, Septiembre")
elif est == 4:
    print("Octubre, Noviembre, Diciembre")
else:
    print("Opcion invalida")
```

Ejercicio 18

```
num = int(input("Numero del 1 al 100: "))
```

```
if num <= 25:
    print("Rango 1-25")
elif num <= 50:
    print("Rango 26-50")
elif num <= 75:
    print("Rango 51-75")
else:
    print("Rango 76-100")
```

Ejercicio 19

```
vel = float(input("Velocidad (km/h): "))
if vel <= 80:
    print("Sin multa")
elif vel <= 100:
    print("Multa leve: $50,000")
elif vel <= 120:
    print("Multa grave: $100,000")
else:
    print("Multa muy grave: $200,000")
```

Ejercicio 20

```
dia = int(input("Numero del 1 al 7: "))
if dia < 1 or dia > 7:
    print("Numero invalido")
elif dia <= 5:
    print("Dia laboral")
else:
    print("Fin de semana")
```

El if-elif-else es como un árbol de decisiones. Ya podés clasificar, categorizar y hacer que tus programas respondan de muchas maneras distintas. Practicá fuerte estos ejercicios, y después inventá tus propios menús de opciones. ¡El límite es tu imaginación!

Operadores relacionales (de comparación)

El día que confundí = con ==

Mirá, cuando arranqué a programar tenía muy claro que para guardar un valor en una variable usaba el `=`. Pero después, en las condiciones del `if`, empecé a poner `if numero = 5:` y Python me sacaba error. Para mí era lógico: “si número es igual a 5...”. Pero no, Python entiende `=` como “guardá” y `==` como “¿es igual?”. Ese día aprendí que los símbolos tienen doble vida.

Los **operadores relacionales** o de comparación son justamente esos signos que nos permiten hacerle preguntas a los datos. ¿Es mayor? ¿Es menor? ¿Son iguales? La respuesta siempre es `True` (sí) o `False` (no). Sin estos operadores, los `if` serían ciegos.

Pensá en un árbitro de fútbol. Cuando el defensor toca la pelota con la mano, el árbitro evalúa: ¿fue mano? (`mano == True`). Si la comparación da `True`, cobra penal. Si no, sigue el juego. Así funcionan los condicionales.

```
+-----+
|
|  OPERADORES RELACIONALES = preguntas al dato
|
|  > , < , >= , <= , == , !=
|
|  La respuesta siempre es True o False
|
+-----+
```

Los seis operadores, uno por uno

Te presento a los seis mosqueteros de la comparación. Cada uno hace una pregunta puntual.

- `<` (menor que): pregunta si el izquierdo es más chico que el derecho.
- `>` (mayor que): pregunta si el izquierdo es más grande que el derecho.
- `==` (igual que): pregunta si los dos son iguales.
- `!=` (distinto que): pregunta si son diferentes.
- `<=` (menor o igual que): pregunta si el izquierdo no es mayor que el derecho.
- `>=` (mayor o igual que): pregunta si el izquierdo no es menor que el derecho.

Probándolos en la terminal

Abrí tu intérprete de Python y acompañame:

```
>>> 5 < 4
False
>>> 7 > 5
True
>>> 5 == 5
True
>>> 4 != 5
True
>>> 6 <= 6
True
>>> 9 >= 5
True
```

Fijate que cada línea devuelve únicamente True o False. Así de simple.

Reflexión: ¿Qué creés que devuelve "manzana" < "pera"? ¿Y 5 == 5.0? Probá en tu máquina y sacate la duda.

Comparando números y también texto

Los operadores no discriminan: funcionan con enteros, decimales y strings. Eso sí, con texto el criterio es alfabético y sensible a mayúsculas.

```
>>> 5 == 5.0          # True, mismo valor numérico
>>> 5 == "5"          # False, tipos distintos
>>> "a" < "b"         # True, orden alfabético
>>> "Hola" > "Chau"   # True, la 'H' viene después que la 'C'
```

Cuidado con los strings: Python compara carácter por carácter según su código Unicode. Por eso "B" da False (las minúsculas son mayores que las mayúsculas en la tabla). No te asustes, con el tiempo le tomás la mano.

Metiendo los operadores en los if

Los operadores relacionales son el corazón de las condiciones. Acá tenés ejemplos que seguramente ya usaste sin darte cuenta.

Ejemplo 1: ¿Es positivo?

```
numero = int(input("Ingrese un número: "))
if numero > 0:
    print("Positivo")
else:
    print("No positivo")
```

Ejemplo 2: ¿Sos mayor de edad?

```
edad = int(input("Edad: "))
if edad >= 18:
    print("Adelante, podés votar.")
else:
    print("Todavía no, pero ya va a llegar.")
```


Ejemplo 3: El mayor de dos

```
a = int(input("Primer número: "))
b = int(input("Segundo número: "))
if a == b:
    print("Son iguales")
elif a > b:
    print("El mayor es", a)
else:
    print("El mayor es", b)
```

Ejercicio rápido: Modificá el ejemplo 3 para que también diga cuál es el menor. Probá distintas combinaciones.

Tabla de verdad de bolsillo

Cuando tengas dudas, esta tabla resume todo:

Expresión	Resultado	Interpretación
5 < 10	True	¿5 es menor que 10? Sí
5 > 10	False	¿5 es mayor que 10? No
5 == 5	True	¿Son iguales? Sí
5 != 5	False	¿Son diferentes? No
5 <= 5	True	¿5 es menor o igual a 5? Sí
5 <= 10	False	¿5 es mayor o igual a 10? No

Mezclando con and, or y not

Muchas veces necesitás preguntar dos cosas al mismo tiempo. Ahí aparecen los operadores lógicos `and`, `or` y `not`. No son relacionales, pero son los mejores amigos.

```
edad = 25
if edad >= 18 and edad <= 60:
    print("Estás en edad laboral activa")

nota = 8
if nota >= 9 or nota == 10:
    print("Excelente")
else:
    print("Bien, pero puede mejorar")
```

Traducción: `and` es “y además”, `or` es “o bien”, `not` es “no”. Más adelante los veremos a fondo.

Errores que me hicieron golpear la cabeza

1. **Usar = en lugar de ==:** El más clásico. Ya te lo conté. Para Python `if x = 5` significa “asigne 5 a x” dentro de la condición, lo cual no tiene sentido.
2. **Comparar tipos incompatibles:** `"10" > 5` da error. Convertí antes: `int("10") > 5`.

3. **Olvidar que $\geq$ incluye el igual:** Muchas veces ponemos `if nota >6` en lugar de `>= 6` y el que sacó 6 reprueba por un pelo. Revisá bien el enunciado.
4. **Crear que `==` funciona con listas de la misma manera:** Dos listas se comparan elemento a elemento, pero eso lo dejamos para más adelante.

Dale, repasemos rápido lo importante

- Los **operadores relacionales** comparan dos valores y devuelven `True` o `False`.
- Los seis: `<`, `>`, `<=`, `>=`, `==`, `!=`.
- Se usan dentro de condiciones (`if`, `elif`, `while`) para decidir.
- Funcionan con números y con texto (orden alfabético).
- `=` asigna, `==` compara. No te confundas más.
- Se pueden combinar con `and`, `or` para preguntas más complejas.

Los operadores relacionales son las preguntas que le hacés a los datos. Con ellos, tus programas aprenden a comparar, y comparar es el primer paso para decidir.

Ejercicios del capítulo

1. Evaluá mentalmente las siguientes expresiones y decí si son `True` o `False`:

- a) `5 >3`
- b) `2 <1`
- c) `7 == 7`
- d) `4 != 4`
- e) `10 <= 10`
- f) `8 >= 12`

Después comprobá con Python.

2. Escribí un programa que pida dos números y diga si el primero es mayor que el segundo. Mostrá ambos números y el resultado.
3. Hacé un programa que pregunte un número y diga si es igual a 10.
4. Pedí un número y respondé si es diferente de 0.
5. Pedí la edad y mostrá si es mayor o igual a 18.
6. Pedí dos números y decí si son iguales, o cuál es mayor.
7. Pedí un número y avisá si es menor que 100.
8. Pedí la temperatura y si es mayor a 30 grados, decí “Hace calor”. Si no, “Está fresco”.
9. Pedí la velocidad de un auto y si supera 80 km/h, mostrá “Excede el límite”.

10. Pedí dos palabras y decí si son exactamente iguales (sin importar mayúsculas). Pista: podés usar `.lower()`.
11. Pedí una nota y decí si es mayor o igual a 7 (aprobado) o no.
12. Pedí un número y decí si está entre 1 y 10 inclusive, usando `and`.
13. Pedí el precio de un producto y si es mayor a 10000, decí "Con descuento". Si no, "Sin descuento".
14. Pedí un carácter y decí si es exactamente la letra 'a'.
15. Pedí tres números y contá cuántos de ellos son mayores que 0. Mostrá el resultado.
16. Pedí un número y decí si es menor o igual a 50.
17. Pedí dos números y mostrá si el segundo es mayor que el primero.
18. Pedí un nombre y si es "Nahuel", saludá con un "¡Hola Nahuel!". Si no, saludá con "Hola, [nombre]".
19. Pedí un número y decí si es mayor que 10 y menor que 20 (usando `and`).
20. Pedí tres números y decí si el primero es igual a la suma de los otros dos.

Soluciones paso a paso

Ejercicio 1

```
5 > 3   → True   (porque 5 es mayor que 3)
2 < 1   → False  (porque 2 no es menor que 1)
7 == 7  → True   (son iguales)
4 != 4  → False  (no son diferentes, son iguales)
10 <= 10 → True   (10 es menor o igual a 10, se cumple el "o igual")
8 >= 12 → False  (8 no es mayor ni igual a 12)
```

Ejercicio 2

```
a = int(input("Primer número: "))
b = int(input("Segundo número: "))
if a > b:
    print(f"{a} es mayor que {b}")
else:
    print(f"{a} NO es mayor que {b}")
```

Ejercicio 3

```
num = int(input("Número: "))
if num == 10:
    print("Es igual a 10")
else:
    print("No es igual a 10")
```

Ejercicio 4

```
num = int(input("Número: "))
if num != 0:
    print("Es diferente de 0")
else:
    print("Es cero")
```

Ejercicio 5

```
edad = int(input("Edad: "))
if edad >= 18:
    print("Sos mayor de edad")
else:
    print("Sos menor de edad")
```

Ejercicio 6

```
a = int(input("Primer número: "))
b = int(input("Segundo número: "))
if a == b:
    print("Son iguales")
elif a > b:
    print(f"{a} es mayor que {b}")
else:
    print(f"{b} es mayor que {a}")
```

Ejercicio 7

```
num = int(input("Número: "))
if num < 100:
    print("Es menor que 100")
else:
    print("No es menor que 100 (es mayor o igual)")
```

Ejercicio 8

```
temp = float(input("Temperatura: "))
if temp > 30:
    print("Hace calor")
else:
    print("Está fresco")
```

Ejercicio 9

```
vel = float(input("Velocidad (km/h): "))
if vel > 80:
    print("Excede el límite")
else:
    print("Velocidad dentro del límite")
```

Ejercicio 10

```
p1 = input("Primera palabra: ").lower()
p2 = input("Segunda palabra: ").lower()
if p1 == p2:
    print("Son iguales (ignorando mayúsculas)")
else:
    print("Son diferentes")
```

Ejercicio 11

```
nota = float(input("Nota: "))
if nota >= 7:
    print("Aprobado")
else:
    print("Desaprobado")
```

Ejercicio 12

```
num = int(input("Número entre 1 y 10: "))
if num >= 1 and num <= 10:
    print("Está en el rango")
else:
    print("Fuera de rango")
```

Ejercicio 13

```
precio = float(input("Precio: $"))
if precio > 10000:
    print("Con descuento")
else:
    print("Sin descuento")
```

Ejercicio 14

```
car = input("Un carácter: ")
if car == 'a':
    print("Es la letra 'a'")
else:
    print("No es 'a'")
```

Ejercicio 15

```
a = int(input("Número 1: "))
b = int(input("Número 2: "))
c = int(input("Número 3: "))
cont = 0
if a > 0:
    cont = cont + 1
if b > 0:
    cont += 1
```

```
if c > 0:
    cont += 1
print("Cantidad de positivos:", cont)
```

Ejercicio 16

```
num = int(input("Número: "))
if num <= 50:
    print("Es menor o igual a 50")
else:
    print("Es mayor a 50")
```

Ejercicio 17

```
a = int(input("Primero: "))
b = int(input("Segundo: "))
if b > a:
    print("El segundo es mayor que el primero")
else:
    print("El segundo NO es mayor que el primero")
```

Ejercicio 18

```
nombre = input("Nombre: ")
if nombre == "Nahuel":
    print("¡Hola Nahuel!")
else:
    print("Hola,", nombre)
```

Ejercicio 19

```
num = int(input("Número: "))
if num > 10 and num < 20:
    print("Está entre 10 y 20 (excluidos 10 y 20)")
else:
    print("No está en ese rango abierto")
```

Ejercicio 20

```
a = int(input("Primero: "))
b = int(input("Segundo: "))
c = int(input("Tercero: "))
if a == b + c:
    print("El primero es igual a la suma de los otros dos")
else:
    print("No se cumple la igualdad")
```

Los operadores relacionales son los sentidos de tu programa: tacto, vista, oído. Con ellos percibe el mundo de los datos y actúa en consecuencia. Ya los tenés dominados. Ahora, a seguir tomando decisiones.

Operadores lógicos

Cuando una sola pregunta no alcanza

Cuando ya manejaba el `if-elif-else` y los operadores relacionales, me sentía un campeón. Pero enseguida me topé con problemas que necesitaban más de una condición a la vez. Por ejemplo, quería saber si un número estaba entre 10 y 20, o si una persona era mayor de edad *y* tenía licencia. No podía poner dos `if` separados porque no era lo mismo. Ahí aparecieron los **operadores lógicos**, esos conectores que unen condiciones como eslabones de una cadena.

Pensá en las reglas de un club: para entrar tenés que ser socio **AND** mayor de edad. Si falla una, no entrás. En cambio, para un sorteo alcanza con cumplir **OR** otro requisito. Y si querés lo contrario de algo, usás **NOT**. Así de simple.

OPERADORES LOGICOS	
AND	-> se cumplen TODAS las condiciones
OR	-> se cumple AL MENOS UNA condicion
NOT	-> invierte el resultado (lo contrario)

Los tres mosqueteros: AND, OR, NOT

Python nos da tres operadores lógicos para armar expresiones compuestas. Cada uno funciona como una llave diferente.

- **AND (conjunción)**: Exige que *todas* las condiciones sean verdaderas. Es como un portero que solo abre si se cumplen todos los requisitos.
- **OR (disyunción)**: Se conforma con que *al menos una* sea verdadera. Como un amigo que dice “con que vengas vos o Juan, ya alcanza”.
- **NOT (negación)**: Da vuelta el resultado. Si era `True`, lo convierte en `False` y viceversa. Es el “no” de toda la vida.

Operador AND: el más exigente

El **AND** solo devuelve `True` cuando **ambas** condiciones son verdaderas. Basta con que una falle para que todo sea `False`.

Tabla de verdad del AND

Condicion A	Condicion B	A AND B
True	True	True
True	False	False
False	True	False
False	False	False

Probando el AND en código

```
num_uno = 5
num_dos = 7
```

```
if num_uno == 5 and num_dos >= 5:
    print("Ambas condiciones se han cumplido")
else:
    print("Una o ambas condiciones no se han cumplido")
```

¿Qué pasó? `num_uno == 5` es `True`, `num_dos >= 5` también es `True`. Como las dos son verdaderas, el `and` da `True` y se imprime el primer mensaje.

Otro ejemplo clásico: ¿Está el número entre 2 y 5?

```
num = int(input("Escribe un numero mayor a 2 y menor a 5: "))
```

```
if num > 2 and num < 5:
    print("El numero", num, "cumple la condicion")
else:
    print("El numero", num, "NO cumple la condicion")
```

Si ingresás 3, ambas condiciones son `True` y se cumple. Con 7, `num < 5` es `False`, y el `and` falla.

Operador OR: el conformista

El `OR` devuelve `True` si **al menos una** condición es verdadera. Solo es `False` cuando las dos son falsas.

Tabla de verdad del OR

Condicion A	Condicion B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

OR en acción

```
num_uno = 5
num_dos = 3

if num_uno == 5 or num_dos >= 5:
    print("Una o ambas condiciones se han cumplido")
else:
    print("Ninguna condicion se ha cumplido")
```

Explicación: `num_uno == 5` es `True`, eso ya alcanza. El `or` no necesita mirar la segunda. Sale con `True`.

Caso donde ninguna se cumple:

```
num_uno = 3
num_dos = 2

if num_uno == 5 or num_dos >= 5:
    print("Una o ambas condiciones se han cumplido")
else:
    print("Ninguna condicion se ha cumplido")
```

Acá las dos son `False`, por lo tanto el `or` también lo es y se va al `else`.

Operador NOT: el rebelde

El `NOT` simplemente invierte el valor lógico de lo que tiene adelante. Si la condición es `True`, `not` la vuelve `False`, y al revés.

Tabla de verdad del NOT

Condicion	NOT Condicion
True	False
False	True

Dándole vuelta a la tortilla

```
num_dos = 3

if not num_dos > 5:
    print("La condicion se cumple al ser un numero menor o igual a 5")
else:
    print("No se cumple la condicion porque el numero es mayor a 5")
```

Razonamiento: `num_dos > 5` es `False` (3 no es mayor que 5). Al ponerle `not` adelante, se convierte en `True`, y se ejecuta el `if`.

Otro ejemplo, pidiendo un número:

```
num_uno = int(input("Introduce un numero igual a 5: "))

if not num_uno == 5:
    print("El numero es diferente de 5 y se cumple la condicion")
else:
    print("El numero es igual a 5 y no cumple la condicion")
```

Si ingresás 3, `num_uno == 5` es `False`, `not False` es `True` → se ejecuta el primer `print`.

Mezclando AND, OR y NOT en la misma coctelera

Lo lindo es que podés combinar estos operadores para armar condiciones más complejas, como si estuvieras armando un rompecabezas. Eso sí, poné paréntesis cuando tengas dudas; así dejás claro el orden de evaluación.

```
edad = int(input("Ingrese su edad: "))
tiene_licencia = input("¿Tiene licencia? (si/no): ")

if edad >= 18 and (tiene_licencia == "si" or tiene_licencia == "SI"):
    print("Puede conducir")
else:
    print("No puede conducir")
```

¿Qué hace? Verifica que la edad sea al menos 18 AND que la licencia sea “si” o “SI”. Si falla cualquiera de las dos, no puede conducir.

Otro ejemplo con rangos:

```
num = int(input("Ingrese un numero: "))

if (num > 0 and num < 10) or (num > 20 and num < 30):
    print("El numero esta en el rango 1-9 o 21-29")
else:
    print("El numero NO esta en los rangos")
```

Acá el `or` separa dos bloques que a su vez contienen `and`. Si el número cumple el primer par o el segundo, entra.

Tabla resumen para la pared

OPERADOR	EJEMPLO	DESCRIPCION
AND	<code>a > 5 and b < 10</code>	Verdadero si ambas condiciones son verdaderas
OR	<code>a == 3 or b == 4</code>	Verdadero si al menos una condicion es verdadera
NOT	<code>not a > 5</code>	Verdadero si la condicion es falsa (invierte)

Errores comunes que me dejaron pensando

1. **Olvidar los paréntesis en combinaciones:** Sin paréntesis, Python evalúa `not`, luego `and`, luego `or`. Si no es lo que querés, el resultado puede ser inesperado. Mejor poné paréntesis para que quede claro.
2. **Confundir `and` con `or`:** “Para entrar, la persona debe ser mayor de 18 años y tener entrada.” Si ponés `or`, con que cumpla una sola entra cualquiera. Cuidado con la lógica del negocio.
3. **Usar `not` sin pensar:** `not a > 5` es más difícil de leer que `a <= 5`. A veces conviene escribir la condición directa sin negación.
4. **Dejar condiciones redundantes:** `if num > 10 and num > 5` es lo mismo que `if num > 10`. La segunda sobra.

Dale, repasemos rápido lo importante

- **AND:** todas las condiciones deben ser `True`. Sin excepción.
- **OR:** al menos una debe ser `True`. Si una alcanza, listo.
- **NOT:** da vuelta el valor. `True` se vuelve `False` y viceversa.
- Se pueden combinar con operadores relacionales y entre sí.
- Usá paréntesis para evitar confusiones de precedencia.
- Son la base para construir condiciones complejas en `if`, `while`, etc.

Los operadores lógicos son los hilos que tejen tus condiciones. Con ellos podés hacer preguntas cada vez más finas y precisas.

Ejercicios del capítulo

1. Escribí un programa que pida dos números y diga si ambos son mayores que 10.
2. Escribí un programa que pida dos números y diga si al menos uno es mayor que 10.
3. Escribí un programa que pida un número y diga si NO es igual a 5 (usando `not`).
4. Escribí un programa que pida la edad y si tiene carnet. Diga si puede conducir (edad $\geq$ 18 Y carnet == "sí").
5. Escribí un programa que pida una letra y diga si es vocal minúscula (a, e, i, o, u) usando `or`.
6. Escribí un programa que pida un número y diga si está entre 10 y 20 inclusive, usando `and`.
7. Escribí un programa que pida dos números y diga si son iguales o diferentes (usando `or` o `and`, aunque no sea necesario, para practicar).
8. Escribí un programa que pida un año y diga si es bisiesto. Regla: divisible por 4, pero no por 100, excepto si es divisible por 400. Usá `and` y `or`.
9. Escribí un programa que pida tres números y diga si todos son positivos (`and`).

10. Escribí un programa que pida un número y diga si es múltiplo de 3 y de 5 a la vez.
11. Escribí un programa que pida una letra y diga si es mayúscula o minúscula (`isupper()` y `islower()`) con `if-elif`.
12. Escribí un programa que pida un número y diga si es impar usando `not`.
13. Escribí un programa que pida tres números y diga si al menos uno es negativo (`or`).
14. Escribí un programa que pida la temperatura y si está lloviendo. Diga si es recomendable salir (temperatura ≥ 15 Y no está lloviendo). Usá `and` y `not`.
15. Escribí un programa que pida un carácter y diga si es letra (usá `isalpha()`).
16. Escribí un programa que pida dos números y diga si el primero es mayor que el segundo Y además el primero es positivo.
17. Escribí un programa que pida un número y diga si está fuera del rango 1-100 usando `not` con un `and`.
18. Escribí un programa que pida tres números y diga si alguno es cero (`or`).
19. Escribí un programa que pida usuario y contraseña. Acceso permitido si usuario es "admin" Y contraseña es "1234".
20. Escribí un programa que pida un número y diga si es divisible por 2, 3 o 5 (al menos uno).

Soluciones paso a paso

Ejercicio 1

```
a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))
if a > 10 and b > 10:
    print("Ambos son mayores que 10")
else:
    print("No ambos son mayores que 10")
```

Ejercicio 2

```
a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))
if a > 10 or b > 10:
    print("Al menos uno es mayor que 10")
else:
    print("Ninguno es mayor que 10")
```

Ejercicio 3

```
num = int(input("Numero: "))
if not num == 5:
    print("No es igual a 5")
else:
    print("Es igual a 5")
```

Ejercicio 4

```
edad = int(input("Edad: "))
carnet = input("¿Tiene carnet? (si/no): ")
if edad >= 18 and carnet == "si":
    print("Puede conducir")
else:
    print("No puede conducir")
```

Ejercicio 5

```
letra = input("Letra: ")
if letra == 'a' or letra == 'e' or letra == 'i' or letra == 'o' or letra == 'u':
    print("Es vocal minuscula")
else:
    print("No es vocal minuscula")
```

Ejercicio 6

```
num = int(input("Numero: "))
if num >= 10 and num <= 20:
    print("Esta entre 10 y 20")
else:
    print("No esta entre 10 y 20")
```

Ejercicio 7

```
a = int(input("Primero: "))
b = int(input("Segundo: "))
if a == b:
    print("Son iguales")
else:
    print("Son diferentes")
# (or no era necesario, pero practicamos el uso de ==)
```

Ejercicio 8

```
anio = int(input("Año: "))
if (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0):
    print("Es bisiesto")
else:
    print("No es bisiesto")
```

Ejercicio 9

```
a = int(input("Primero: "))
b = int(input("Segundo: "))
c = int(input("Tercero: "))
if a > 0 and b > 0 and c > 0:
    print("Todos son positivos")
else:
```

```
print("No todos son positivos")
```

Ejercicio 10

```
num = int(input("Numero: "))
if num % 3 == 0 and num % 5 == 0:
    print("Es multiplo de 3 y de 5")
else:
    print("No es multiplo de 3 y de 5")
```

Ejercicio 11

```
letra = input("Letra: ")
if letra.isupper():
    print("Es mayuscula")
elif letra.islower():
    print("Es minuscula")
else:
    print("No es una letra")
```

Ejercicio 12

```
num = int(input("Numero: "))
if not num % 2 == 0:
    print("Es impar")
else:
    print("Es par")
```

Ejercicio 13

```
a = int(input("Primero: "))
b = int(input("Segundo: "))
c = int(input("Tercero: "))
if a < 0 or b < 0 or c < 0:
    print("Al menos uno es negativo")
else:
    print("Ninguno es negativo")
```

Ejercicio 14

```
temp = float(input("Temperatura: "))
lluvia = input("¿Esta lloviendo? (si/no): ")
if temp > 15 and lluvia == "no":
    print("Es recomendable salir")
else:
    print("No es recomendable salir")
```

Ejercicio 15

```
car = input("Caracter: ")
if car.isalpha():
```

```
print("Es letra")
else:
    print("No es letra")
```

Ejercicio 16

```
a = int(input("Primero: "))
b = int(input("Segundo: "))
if a > b and a > 0:
    print("El primero es mayor que el segundo y es positivo")
else:
    print("No cumple la condicion")
```

Ejercicio 17

```
num = int(input("Numero: "))
if not (num >= 1 and num <= 100):
    print("Esta fuera del rango 1-100")
else:
    print("Esta dentro del rango 1-100")
```

Ejercicio 18

```
a = int(input("Primero: "))
b = int(input("Segundo: "))
c = int(input("Tercero: "))
if a == 0 or b == 0 or c == 0:
    print("Alguno es cero")
else:
    print("Ninguno es cero")
```

Ejercicio 19

```
usuario = input("Usuario: ")
contrasena = input("Contraseña: ")
if usuario == "admin" and contrasena == "1234":
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

Ejercicio 20

```
num = int(input("Numero: "))
if num % 2 == 0 or num % 3 == 0 or num % 5 == 0:
    print("Es divisible por 2, 3 o 5")
else:
    print("No es divisible por 2, 3 ni 5")
```

Con AND, OR y NOT podés armar las preguntas más rebuscadas. Practicá estos ejercicios y después inventá tus propias reglas. La lógica es el motor de cualquier programa. ¡A darle átomos!

Operadores de asignación

El día que me cansé de escribir largo

Cuando empecé a hacer programas un poco más largos, me di cuenta de que escribir `x = x + 1` todo el tiempo era un embole. Además, quedaba un código ruidoso y difícil de leer. Un compañero me dijo: “Che, ¿por qué no usás +=?”. Lo probé y fue como pasar de la bici al auto. Eso era un **operador de asignación compuesto**, y desde ese día no los solté más.

En este capítulo vamos a ver todos los operadores que combinan una operación aritmética con la asignación. Son atajos que hacen tu código más corto, más limpio y más profesional. ¡Vamos a darles!

¿Qué es un operador de asignación?

Un **operador de asignación** toma el valor de la derecha y lo guarda en la variable de la izquierda. El más simple y el que ya conocés es el `=`.

Pensá en una caja etiquetada. Si ponés `x = 5`, estás metiendo un papelito con el número 5 dentro de la caja llamada `x`. Si después hacés `x = 8`, tirás el papelito anterior y metés uno nuevo. Así de fácil.

```
x = 5          -> la caja 'x' ahora contiene 5
nombre = "Nahuel" -> la caja 'nombre' contiene "Nahuel"
precio = 99.99  -> la caja 'precio' contiene 99.99
```

Operadores de asignación compuestos: el verdadero atajo

Python nos regala un montón de operadores que **hacen una cuenta y guardan el resultado en la misma variable**. Son como una navaja suiza: ahorran tiempo y espacio. La tabla completa es esta:

OPERADOR	EJEMPLO	SIGNIFICADO
<code>=</code>	<code>x = 5</code>	<code>x = 5</code>
<code>+=</code>	<code>x += 3</code>	<code>x = x + 3</code>
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code>	<code>x /= 3</code>	<code>x = x / 3</code>
<code>//=</code>	<code>x //= 3</code>	<code>x = x // 3</code>
<code>**=</code>	<code>x **= 3</code>	<code>x = x ** 3</code>
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>

Fijate que todos siguen la misma lógica: `variable operador= valor` es lo mismo que `variable = variable operador valor`. O sea, `x += 5` es idéntico a `x = x + 5`. Una maravilla.

Suma y asigna (+=)

Problema: Quería ir armando un mensaje de bienvenida agregando partes sin borrar lo anterior. Con = perdía todo cada vez. Con += lo solucioné.

```
mensaje = "Hola"
mensaje += " "
mensaje += "Mundo"
print(mensaje) # Hola Mundo
```

Con números también funciona. Si estás sumando puntos en un juego, es un golazo:

```
puntaje = 0
puntaje += 10 # el jugador encontró una moneda
puntaje += 50 # mató a un enemigo
print(puntaje) # 60
```

Resta y asigna (--)

Problema: Tenía que descontar un gasto de un saldo. En vez de `saldo = saldo - gasto`, usé `--`.

```
saldo = 1000
saldo -= 250 # pagué el alquiler
print(saldo) # 750
```

Ejercicio rápido: ¿Cuánto vale `x` después de `x = 20`; `x -= 7`; `x -= 3`? Pensá antes de ejecutar.

Multiplicación y asigna (*=)

Problema: Necesitaba duplicar un valor sin crear variables nuevas.

```
factor = 5
factor *= 3 # factor ahora es 15
print(factor) # 15
```

Dato curioso: Con strings, `*=` repite el texto. ¿Querés una línea de 20 guiones? `linea = "`

```
>>> risa = "ja"
>>> risa *= 3
>>> print(risa)
jajaja
```

División y asigna (/=)

Cuidado: `/=` siempre devuelve un `float`, aunque la división sea exacta. Es lo mismo que `/`.

```
pastel = 10
personas = 4
pastel /= personas # pastel = 10 / 4 = 2.5
print(pastel) # 2.5
```

División entera y asigna (//=)

Si solo te interesa la parte entera (por ejemplo, para repartir sillas y que no queden medios traseros), usá //.

```
sillas = 10
sillas //= 3    # 10 // 3 = 3
print(sillas)  # 3
```

Exponente y asigna (**=)

Para elevar un número a una potencia y guardarlo de una:

```
base = 2
base **= 5    # 2 elevado a la 5 = 32
print(base)  # 32
```

Reflexión: ¿Por qué `x **= 2` es equivalente a `x = x * x`? Solo si entendés eso, ya sos Gardel.

Módulo y asigna (%=)

¿Te acordás del resto de la división? Con %= podés quedarte con el resto y actualizar la variable.

```
numero = 17
numero %= 5    # 17 % 5 = 2
print(numero)  # 2
```

Muy útil para ciclos de reloj: `hora %= 24` te mantiene en el rango 0–23.

Tabla resumen de los compuestos

Pegá esto en la pared, cerca del monitor:

OPERADOR	EJEMPLO	EQUIVALE A
+=	<code>x += y</code>	<code>x = x + y</code>
-=	<code>x -= y</code>	<code>x = x - y</code>
*=	<code>x *= y</code>	<code>x = x * y</code>
/=	<code>x /= y</code>	<code>x = x / y</code>
//=	<code>x //= y</code>	<code>x = x // y</code>
**=	<code>x **= y</code>	<code>x = x ** y</code>
%=	<code>x %= y</code>	<code>x = x % y</code>

Ejemplos completos para verlos en acción

Acumulador de compras (ideal para el super)

```
total = 0
total += 1500
total += 800
total += 2500
print("Total a pagar:", total)    # 4800
```

Barra de energía de un personaje

```
energia = 100
energia -= 15    # recibió un golpe
energia -= 20    # cayó en una trampa
energia += 50    # tomó una poción
print("Energía restante:", energia) # 115
```

Repetir texto como un lorito

```
eco = "eco... "
eco *= 3
print(eco) # eco... eco... eco...
```

Errores que me hicieron putear (y cómo evitarlos)

1. **Confundir += con +=.** $x += 3$ no es lo mismo que $x += 3$. El primero asigna +3 a x (como ponerle un cartelito), el segundo suma. Ojo al piojo.
2. **Intentar += con dos variables que no son números ni strings compatibles.** $x += [1, 2]$ con x entero da error. Solo se puede si son del mismo tipo.
3. **Olvidar que /= devuelve float.** Si después usás el resultado en una función que espera entero, te va a dar problemas.
4. **No reinicializar el acumulador.** Si reutilizás una variable sin ponerla a cero, los valores se arrastran.

Dale, repasemos rápido lo importante

- El operador = asigna un valor a una variable.
- Los operadores compuestos (+=, -=, *=, /=, //=, **=, %=) hacen una operación y guardan el resultado en la misma variable.
- Son atajos que hacen el código más limpio y evitan repetir el nombre de la variable.
- += también sirve para concatenar strings; *= para repetirlos.
- Cuidado con el orden: $a += b$ es $a = a + b$, no al revés.

Ejercicios del capítulo

1. Declará una variable x con valor 10 y usando += sumale 5. Mostrá el resultado.
2. Declará una variable y con 20, restale 7 con -= y mostrá el resultado.
3. Tomá $z = 4$ y multiplicá por 3 con *= . ¿Cuánto queda?
4. Usando /=, dividí $w = 15$ por 2 y mostrá el resultado.
5. Con $v = 17$, aplicá división entera //= por 4 e imprimí.

6. A partir de `u = 2`, elevá a la cuarta potencia con `**=` y mostrá.
7. Calculá el resto de 25 dividido 4 con `%=` sobre una variable `t = 25`.
8. Construí el string “Hola mundo” empezando con `texto = "Hola"` y usando `+=` dos veces.
9. Llevá un contador desde 0 hasta 3 sumando 1 tres veces con `+=`. Mostrá cada paso (opcional) y el final.
10. Tenés un saldo de 1000. Restale 250, después sumale 100, todo con operadores compuestos. Mostrá el saldo final.
11. Empezá con `numero = 2`. Elevá al cuadrado y mostrá; después elevá al cubo y mostrá (usá `**=`).
12. Partí de `multiplicador = 1`. Multiplicalo por 2, luego por 3, luego por 4 con `*=`. Mostrá el resultado final.
13. Creá una variable `frase` vacía (). Agregale “Aprendiendo”, un espacio y “Python” con `+=`. Imprimí.
14. Acumulá una compra: variable `total` en 0, sumale 2500, 1800 y 3200 con `+=`. Mostrá el total.
15. Empezá con 50, dividí por 2 y luego por 5 usando `/=`. Mostrá el resultado después de cada división.
16. Repetí la cadena "ja" cinco veces usando `*=`. Mostrá el resultado.
17. Inventario inicial 100. Vendé 15 (`-=`), repusiste 30 (`+=`). Mostrá el stock final.
18. Energía inicial 100. Gastá 10, 20 y 15 con `-=`. ¿Cuánta energía queda?
19. Experiencia inicial 0. Sumá 50, 30, 20 con `+=`. Mostrá la experiencia total.
20. Calculá el factorial de 5 usando `*=`: partí de `resultado = 1` y multiplicalo por 1, 2, 3, 4, 5 sucesivamente. Imprimí el resultado (debe dar 120).

Soluciones paso a paso

Ejercicio 1

```
x = 10
x += 5      # x = 10 + 5
print(x)    # 15
```

Ejercicio 2

```
y = 20
y -= 7      # y = 20 - 7
print(y)    # 13
```

Ejercicio 3

```
z = 4
z *= 3      # z = 4 * 3
print(z)    # 12
```

Ejercicio 4

```
w = 15
w /= 2          # w = 15 / 2 = 7.5 (float)
print(w)        # 7.5
```

Ejercicio 5

```
v = 17
v //= 4         # v = 17 // 4 = 4
print(v)        # 4
```

Ejercicio 6

```
u = 2
u **= 4         # u = 2 ** 4 = 16
print(u)        # 16
```

Ejercicio 7

```
t = 25
t %= 4          # t = 25 % 4 = 1
print(t)        # 1
```

Ejercicio 8

```
texto = "Hola"
texto += " "    # "Hola "
texto += "mundo" # "Hola mundo"
print(texto)
```

Ejercicio 9

```
contador = 0
contador += 1
contador += 1
contador += 1
print(contador) # 3
```

Ejercicio 10

```
saldo = 1000
saldo -= 250    # 750
saldo += 100    # 850
print(saldo)    # 850
```

Ejercicio 11

```
numero = 2
numero **= 2   # 2**2 = 4
print(numero)  # 4
```

```
numero **= 3      # 4**3 = 64
print(numero)     # 64
```

Ejercicio 12

```
multiplicador = 1
multiplicador *= 2   # 2
multiplicador *= 3   # 6
multiplicador *= 4   # 24
print(multiplicador) # 24
```

Ejercicio 13

```
frase = ""
frase += "Aprendiendo"
frase += " "
frase += "Python"
print(frase)          # Aprendiendo Python
```

Ejercicio 14

```
total = 0
total += 2500
total += 1800
total += 3200
print(total)          # 7500
```

Ejercicio 15

```
num = 50
num /= 2               # 25.0
print(num)             # 25.0
num /= 5               # 5.0
print(num)             # 5.0
```

Ejercicio 16

```
repetir = "ja"
repetir *= 5           # "jajajajaja"
print(repetir)
```

Ejercicio 17

```
inventario = 100
inventario -= 15       # 85
inventario += 30       # 115
print(inventario)     # 115
```

Ejercicio 18

```
energia = 100
energia -= 10
energia -= 20
energia -= 15
print(energia)          # 55
```

Ejercicio 19

```
experiencia = 0
experiencia += 50
experiencia += 30
experiencia += 20
print(experiencia)      # 100
```

Ejercicio 20

```
resultado = 1
resultado *= 1    # 1
resultado *= 2    # 2
resultado *= 3    # 6
resultado *= 4    # 24
resultado *= 5    # 120
print(resultado) # 120 (factorial de 5)
```

Los operadores de asignación compuestos son como una cinta de correr: te ahorran pasos y te llevan al mismo lugar, pero más rápido. Ahora que los dominás, tus programas van a ser más elegantes y fáciles de seguir.

Los parámetros sep y end

¿Qué son los parámetros sep y end?

La función `print()` es una de las herramientas más útiles en el lenguaje de programación Python al momento de interactuar con los usuarios de nuestros programas. Por tal motivo, es indispensable saber cómo podemos manipular dichas impresiones en pantalla, con la finalidad de tener el control completo del texto a mostrar.

Python nos proporciona dos parámetros muy útiles para controlar cómo se muestra la información:

- **end**: controla qué se imprime al final.
- **sep**: controla qué se imprime entre los valores.

```
+-----+
|
|  print(valores, sep=" ", end="\n")
|
|  sep → separador entre valores (default: " ")
|  end → lo que va al final (default: "\n")
|
+-----+
```

El parámetro end

El parámetro **end** se utiliza para agregar cualquier cadena de caracteres al final de la salida e impresión en pantalla de la función `print()`.

Por defecto, la función `print()` genera un salto de línea al terminar su ejecución. Sin embargo, con ayuda del parámetro **end**, podemos cambiar ese comportamiento.

Ejemplo básico de end

```
print('Esto es un ', end="")
print('ejemplo')
```

Salida:

Esto es un ejemplo

Ejemplo con end personalizado

```
print('Esto es un ', end="-*")  
print('ejemplo')
```

Salida:

Esto es un -*ejemplo

Ejemplo: sin salto de línea

```
print("Hola", end="")  
print("Mundo")
```

Salida:

HolaMundo

Ejemplo: con espacio al final

```
print("Hola", end=" ")  
print("Mundo", end=" ")  
print("Python")
```

Salida:

Hola Mundo Python

Ejemplo: con guión al final

```
print("Uno", end="-")  
print("Dos", end="-")  
print("Tres")
```

Salida:

Uno-Dos-Tres

Ejemplo: barra de carga simulada

```
print("Cargando", end="")  
print(".", end="")  
print(".", end="")  
print(".")
```

Salida:

Cargando...

El parámetro sep

En ocasiones, es posible que deseemos imprimir múltiples valores de manera legible utilizando la función `print()`. Para ello contamos con el parámetro **sep**, que se utiliza para dar formato a las cadenas de caracteres que deben imprimirse en pantalla, agregando un separador entre las cadenas que se imprimen.

Por defecto, **sep** tiene un espacio en blanco ().

Ejemplo básico (sep por defecto)

```
print("1", "2", "3", "4", "5")
```

Salida:

```
1 2 3 4 5
```

Ejemplo con sep vacío

```
print("1", "2", "3", "4", "5", sep="")
```

Salida:

```
12345
```

Ejemplo con sep personalizado (guión)

```
print("1", "2", "3", "4", "5", sep="-")
```

Salida:

```
1-2-3-4-5
```

Ejemplo con sep personalizado (coma y espacio)

```
print("Lunes", "Martes", "Miercoles", sep=", ")
```

Salida:

```
Lunes, Martes, Miercoles
```

Ejemplo con sep personalizado (asterisco)

```
print("Hola", "Mundo", "Python", sep=" * ")
```

Salida:

```
Hola * Mundo * Python
```

Combinando sep y end

Podemos usar **sep** y **end** juntos en el mismo `print()` para tener un control total sobre la salida.

Ejemplo combinado

```
print("A", "B", "C", sep="-", end=" FIN")
```

Salida:

A-B-C FIN

Ejemplo: tabla con separadores

```
print("Nombre", "Edad", "Ciudad", sep=" | ", end="\n\n")
print("Nahuel", "25", "Buenos Aires", sep=" | ")
```

Salida:

Nombre | Edad | Ciudad

Nahuel | 25 | Buenos Aires

Ejemplo: construyendo una línea

```
for i in range(1, 6):
    print(i, end=" ")
```

Salida:

1 2 3 4 5

Ejemplos prácticos

Ejemplo 1: Imprimir en la misma línea

```
print("Procesando", end=" ")
print("archivo...", end=" ")
print("listo!")
```

Ejemplo 2: Separador personalizado para lista

```
frutas = ["manzana", "pera", "uva", "sandia"]
print(*frutas, sep=", ")
```

Salida:

manzana, pera, uva, sandia

Ejemplo 3: Crear una línea de caracteres

```
print("*" * 20)
print("Menu Principal")
print("*" * 20)
```

Ejemplo 4: Usar sep para formato de tabla

```
print("Producto", "Precio", "Stock", sep="\t")
print("Mouse", "$1500", "10", sep="\t")
print("Teclado", "$2500", "5", sep="\t")
print("Monitor", "$5000", "3", sep="\t")
```

Tabla resumen de sep y end

PARAMETRO	DESCRIPCION
sep=""	Sin separador: "12345"
sep=" "	Espacio (default): "1 2 3 4 5"
sep="-"	Guión: "1-2-3-4-5"
sep=", "	Coma y espacio: "1, 2, 3, 4, 5"
sep="\n"	Salto de línea entre cada valor
end=""	Sin salto de línea al final
end=" "	Espacio al final
end="\n"	Salto de línea (default)
end="FIN"	Termina con "FIN"

Ejercicios propuestos

- Usando `end=`, haz que tres `print()` impriman en la misma línea: "Hola", , "Mundo".
- Usando `sep=`, imprime "Python" separado en letras: "P", "y", "t", "h", "o", "n".
- Imprime los números del 1 al 5 separados por guiones usando `sep=`.
- Imprime "Hola" y "Mundo" con un espacio entre ellos usando `end` (sin usar `sep`).
- Imprime los números 1, 2, 3, 4, 5 sin espacios entre ellos usando `sep=`.
- Imprime los días de la semana separados por coma y espacio: "Lunes, Martes, Miercoles...".
- Usando `end="..."`, haz que un `print()` termine con puntos suspensivos.
- Combina `sep` y `end` para imprimir ".A-B-C" seguido de "FIN".
- Imprime una cuenta regresiva del 3 al 1 en la misma línea: "3 2 1" usando `end`.
- Imprime los números del 1 al 5 separados por "*" usando `sep`.
- Usa `end=` para imprimir ".Esto ", ".es ", "una ", "prueba" en la misma línea.
- Imprime una tabla simple con `sep="\t"` para alinear columnas: Nombre, Edad, Ciudad.
- Imprime "1+2+3+4+5" usando `sep="+"` (con números, no texto).
- Usa `end=` y `print()` para simular una barra de carga que imprime "..." progresivamente.
- Imprime "Hola", "Python", "2024" separados por » usando `sep`.

Haz que un `print()` termine con un signo de exclamación usando `end`.

Imprime una línea de asteriscos usando `print("*** 20)` y luego un mensaje centrado.

Imprime los números del 1 al 10 en la misma línea, separados por coma y espacio, usando un bucle y `end`.

Usa `sep="\n"` para imprimir cada número del 1 al 5 en una línea diferente usando un solo `print()`.

Crea un menú con formato usando `sep` y `end`.

Soluciones paso a paso

Ejercicio 1

```
print("Hola", end="")
print(" ", end="")
print("Mundo")
```

Ejercicio 2

```
print("P", "y", "t", "h", "o", "n", sep="")
```

Ejercicio 3

```
print(1, 2, 3, 4, 5, sep="-")
```

Ejercicio 4

```
print("Hola", end=" ")
print("Mundo")
```

Ejercicio 5

```
print(1, 2, 3, 4, 5, sep="")
```

Ejercicio 6

```
print("Lunes", "Martes", "Miercoles", "Jueves", "Viernes", sep=", ")
```

Ejercicio 7

```
print("Cargando", end="...")
```

Ejercicio 8

```
print("A", "B", "C", sep="-", end="FIN")
```

Ejercicio 9

```
print("3", end=" ")
print("2", end=" ")
print("1")
```

Ejercicio 10

```
print(1, 2, 3, 4, 5, sep=" * ")
```

Ejercicio 11

```
print("Esto", end=" ")
print("es", end=" ")
print("una", end=" ")
print("prueba")
```

Ejercicio 12

```
print("Nombre", "Edad", "Ciudad", sep="\t")
print("Nahuel", "25", "Buenos Aires", sep="\t")
```

Ejercicio 13

```
print(1, 2, 3, 4, 5, sep="+")
```

Ejercicio 14

```
print("Cargando", end="")
print(".", end="")
print(".", end="")
print(".")
```

Ejercicio 15

```
print("Hola", "Python", "2024", sep=" >>> ")
```

Ejercicio 16

```
print("Bienvenido", end="!")
```

Ejercicio 17

```
print("*" * 20)
print("    MENU PRINCIPAL")
print("*" * 20)
```

Ejercicio 18

```
for i in range(1, 11):
    if i < 10:
        print(i, end=", ")
    else:
        print(i)
```

Ejercicio 19

```
print(1, 2, 3, 4, 5, sep="\n")
```

Ejercicio 20

```
print("=== MENU ===")
print("1.", "Opcion A", sep="")
print("2.", "Opcion B", sep="")
print("3.", "Opcion C", sep="")
print("=====")
```

Resumen del capítulo

En este capítulo aprendiste:

- El parámetro **end** controla qué se imprime al final de cada `print()`. Por defecto es `"\n"` (salto de línea).
- El parámetro **sep** controla qué se imprime entre los valores dentro de un mismo `print()`. Por defecto es (espacio).
- Con **end=** podemos evitar el salto de línea y hacer que múltiples `print()` escriban en la misma línea.
- Con **sep=** podemos eliminar los espacios entre valores.
- Podemos usar **sep** y **end** juntos para dar formato personalizado a la salida.
- Estos parámetros son muy útiles para crear menús, tablas, barras de carga y todo tipo de salida formateada.

Con sep y end tenés el control total de cómo se ve la salida de tus programas. Pequeños detalles que hacen la diferencia.

Bucle while

¿Por qué necesitamos los bucles?

Imaginá que necesitás escribir “Hola mundo” 100 veces. Podrías escribir 100 veces la misma línea de código. Sin embargo, esta alternativa no es la más óptima, ya que duplicaría líneas generando archivos más extensos y difíciles de entender al momento de querer realizar alguna modificación o actualización.

Ante esto, se cuenta con **sentencias de repetición de código**, las cuales permiten ejecutar una serie de instrucciones o líneas de código de manera controlada y repetitiva. A esto se llama **bucle** o **ciclo**.

```
+-----+
|      SIN BUCLE (100 líneas)      CON BUCLE (5 líneas)      |
|      |      |      |      |      |      |      |      |      |
| print("Hola mundo")      i = 1      |      |      |      |
| print("Hola mundo")      while i <= 100:      |      |      |
| ... (100 veces)      print("Hola mundo")      |      |      |
| print("Hola mundo")      i = i + 1      |      |      |
|      |      |      |      |      |      |      |      |      |
+-----+
```

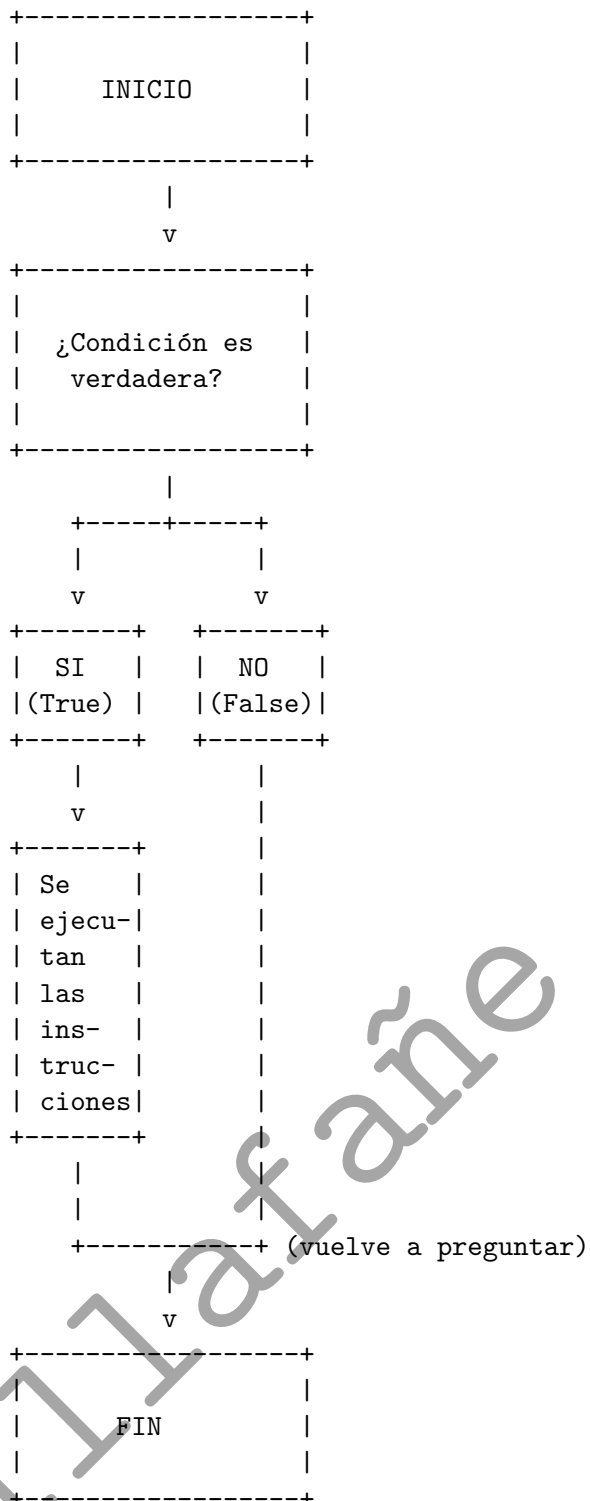
¿Qué es un bucle o ciclo?

Un **ciclo** o **bucle** permite ejecutar en repetidas ocasiones las instrucciones o líneas de código indicadas por el programador, con lo cual no existe la necesidad de duplicar líneas de código para ejecutarlas en más de una ocasión.

En Python, el bucle **while** permite repetir la ejecución de un grupo de instrucciones **mientras se cumpla una condición**. Es decir, mientras la condición del ciclo sea verdadera, las instrucciones se seguirán ejecutando.

```
+-----+
| while condicion:      |
|     instruccion1      |
|     instruccion2      |
|     ...      |
|      |      |      |      |      |      |      |      |
| MIENTRAS condicion sea True → se repite      |
| CUANDO condicion sea False → termina      |
|      |      |      |      |      |      |      |      |
+-----+
```


Diagrama de flujo del while



Sintaxis del while

La sintaxis básica del bucle `while` en Python es:

```
while condicion:
    # instrucciones a repetir
```

```
instruccion1
instruccion2
# IMPORTANTE: algo debe cambiar la condicion
```

Ejemplo básico

```
x = 1
while x <= 5:
    print("El valor de x es:", x)
    x = x + 1
```

Salida:

```
El valor de x es: 1
El valor de x es: 2
El valor de x es: 3
El valor de x es: 4
El valor de x es: 5
```

Ejemplo 1: Contador ascendente

```
contador = 1
while contador <= 10:
    print("Numero:", contador)
    contador = contador + 1
print("Fin del conteo")
```

Ejemplo 2: Contador descendente

```
contador = 10
while contador >= 1:
    print(contador)
    contador = contador - 1
print("Despegue!")
```

Salida:

```
10
9
8
7
6
5
4
3
2
1
Despegue!
```

Ejemplo 3: Suma de números

```
suma = 0
numero = 1
while numero <= 10:
    suma = suma + numero
    print("Sumo", numero, "→ total:", suma)
    numero = numero + 1
print("La suma total es:", suma)
```

Ejemplo 4: Validación de entrada

```
edad = int(input("Ingrese su edad (debe ser mayor a 0): "))
while edad <= 0:
    print("Error: la edad debe ser mayor a 0")
    edad = int(input("Ingrese su edad nuevamente: "))
print("Edad valida:", edad)
```

Ejemplo 5: Juego de adivinanza

```
secreto = 7
intento = int(input("Adivina el numero (1-10): "))

while intento != secreto:
    print("Incorrecto! Intenta de nuevo")
    intento = int(input("Adivina el numero (1-10): "))

print("Felicitaciones! Adivinaste el", secreto)
```

Ejemplo 6: Tabla de multiplicar

```
numero = int(input("Que tabla quieres ver? "))
i = 1
while i <= 10:
    print(numero, "x", i, "=", numero * i)
    i = i + 1
```

Ejemplo 7: Suma hasta que el usuario quiera

```
total = 0
continuar = "si"

while continuar == "si":
    num = float(input("Ingrese un numero: "))
    total = total + num
    continuar = input("Quiere seguir sumando? (si/no): ")

print("El total acumulado es:", total)
```

Ejemplo 8: Factorial de un número

```
num = int(input("Ingrese un numero: "))
factorial = 1
i = 1

while i <= num:
    factorial = factorial * i
    i = i + 1

print("El factorial de", num, "es:", factorial)
```

Error común: bucle infinito

```
# ESTO ES UN ERROR! BUCLE INFINITO
x = 1
while x <= 5:
    print("Hola")
    # Falta x = x + 1 → x siempre es 1, nunca cambia
```

```
+-----+
|
|  ADVERTENCIA:
|
|  Si la condicion nunca se vuelve falsa,
|  el bucle se ejecuta para siempre (o hasta
|  que forces la detencion con Ctrl+C)
|
|  Siempre debes modificar la variable que
|  controla la condicion dentro del bucle
|
+-----+
```

Ejemplo correcto vs incorrecto

```
# INCORRECTO (bucle infinito)
i = 1
while i <= 5:
    print("Esto no termina nunca")
    # Falta i = i + 1

# CORRECTO
i = 1
while i <= 5:
    print("Esto termina")
    i = i + 1
```

Ejercicios propuestos

Ejercicio 1

Escribí un bucle que imprima los números del 1 al 10.

Ejercicio 2

Escribí un bucle que imprima los números del 10 al 1 (descendente).

Ejercicio 3

Escribí un bucle que imprima los números pares del 2 al 20.

Ejercicio 4

Escribí un bucle que sume los números del 1 al 100 y muestre el resultado.

Ejercicio 5

Escribí un bucle que pida números al usuario hasta que ingrese 0, luego muestre la suma.

Ejercicio 6

Escribí un bucle que imprima la tabla de multiplicar del 5.

Ejercicio 7

Escribí un bucle que calcule el factorial de un número ingresado por el usuario.

Ejercicio 8

Escribí un bucle que pida una contraseña hasta que el usuario ingrese "python123".

Ejercicio 9

Escribí un bucle que imprima los números impares del 1 al 30.

Ejercicio 10

Escribí un bucle que cuente cuántos números pares hay del 1 al 50.

Ejercicio 11

Escribí un bucle que imprima los múltiplos de 3 del 1 al 50.

Ejercicio 12

Escribí un bucle que pida números al usuario y calcule el promedio hasta que ingrese -1.

Ejercicio 13

Escribí un bucle que imprima un triángulo de asteriscos con 5 filas: `*** ** *` `**** *` `*****`__

Ejercicio 14

Escribí un bucle que encuentre el número más grande de una lista de números ingresados por el usuario (termina con 0).

Ejercicio 15

Escribí un bucle que imprima la secuencia de Fibonacci hasta el término 10: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

Ejercicio 16

Escribí un bucle que pida un número y diga si es primo.

Ejercicio 17

Escribí un bucle que imprima los números del 1 al 50, pero que reemplace los múltiplos de 3 por “Fizz” y los de 5 por “Buzz”.

Ejercicio 18

Escribí un bucle que invierta un número ingresado por el usuario (ej: 12345 → 54321).

Ejercicio 19

Escribí un bucle que calcule la potencia de un número (base y exponente) sin usar el operador `**`.

Ejercicio 20

Escribí un programa que simule un cajero automático con saldo inicial de \$50000, que permita retirar dinero mientras el saldo sea mayor a 0.

Soluciones paso a paso

Solución 1

```
i = 1
while i <= 10:
    print(i)
    i = i + 1
```

Solución 2

```
i = 10
while i >= 1:
    print(i)
    i = i - 1
```

Solución 3

```
i = 2
while i <= 20:
    print(i)
    i = i + 2
```

Solución 4

```
suma = 0
i = 1
while i <= 100:
    suma = suma + i
    i = i + 1
print(suma)
```

Solución 5

```
suma = 0
num = int(input("Numero (0 para terminar): "))
while num != 0:
    suma = suma + num
    num = int(input("Numero (0 para terminar): "))
print("Suma total:", suma)
```

Solución 6

```
num = 5
i = 1
while i <= 10:
    print(num, "x", i, "=", num * i)
    i = i + 1
```

Solución 7

```
num = int(input("Numero: "))
factorial = 1
i = 1
while i <= num:
    factorial = factorial * i
    i = i + 1
print("Factorial:", factorial)
```

Solución 8

```
clave = input("Contraseña: ")
while clave != "python123":
    print("Incorrecta")
    clave = input("Contraseña: ")
print("Acceso permitido")
```

Solución 9

```
i = 1
while i <= 30:
    if i % 2 != 0:
        print(i)
    i = i + 1
```

Solución 10

```
contador = 0
i = 1
while i <= 50:
    if i % 2 == 0:
        contador = contador + 1
    i = i + 1
print("Cantidad de pares:", contador)
```

Solución 11

```
i = 1
while i <= 50:
    if i % 3 == 0:
        print(i)
    i = i + 1
```

Solución 12

```
suma = 0
cantidad = 0
num = int(input("Numero (-1 para terminar): "))
while num != -1:
    suma = suma + num
    cantidad = cantidad + 1
    num = int(input("Numero (-1 para terminar): "))
if cantidad > 0:
    promedio = suma / cantidad
    print("Promedio:", promedio)
```

Solución 13

```
fila = 1
while fila <= 5:
```



```
print("*" * fila)
fila = fila + 1
```

Solución 14

```
mayor = 0
num = int(input("Numero (0 para terminar): "))
while num != 0:
    if num > mayor:
        mayor = num
    num = int(input("Numero (0 para terminar): "))
print("El mayor es:", mayor)
```

Solución 15

```
a, b = 0, 1
contador = 1
while contador <= 10:
    print(a, end=" ")
    a, b = b, a + b
    contador = contador + 1
```

Solución 16

```
num = int(input("Numero: "))
es_primo = True
i = 2
while i < num:
    if num % i == 0:
        es_primo = False
        i = i + 1
if es_primo and num > 1:
    print("Es primo")
else:
    print("No es primo")
```

Solución 17

```
i = 1
while i <= 50:
    if i % 3 == 0 and i % 5 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
    i = i + 1
```

Solución 18

```
num = int(input("Numero: "))
invertido = 0
while num > 0:
    digito = num % 10
    invertido = invertido * 10 + digito
    num = num // 10
print("Invertido:", invertido)
```

Solución 19

```
base = int(input("Base: "))
exponente = int(input("Exponente: "))
resultado = 1
i = 1
while i <= exponente:
    resultado = resultado * base
    i = i + 1
print(base, "^", exponente, "=", resultado)
```

Solución 20

```
saldo = 50000
while saldo > 0:
    print("Saldo actual:", saldo)
    retiro = int(input("Monto a retirar: "))
    if retiro <= saldo:
        saldo = saldo - retiro
        print("Retiro exitoso")
    else:
        print("Saldo insuficiente")
print("Saldo agotado. Cuenta cerrada")
```

Resumen del capítulo

En este capítulo aprendiste:

- Un **bucle** o **ciclo** permite repetir instrucciones múltiples veces.
- El bucle **while** repite instrucciones **mientras** una condición sea verdadera.
- Es importante que dentro del bucle algo **cambie la condición** para evitar bucles infinitos.
- Los bucles ayudan a evitar duplicar código y hacen los programas más compactos y mantenibles.
- Se pueden usar bucles para: contar, sumar, validar entradas, procesar listas de datos, etc.

Los bucles son como un robot que hace una y otra vez la misma tarea sin quejarse. Vos le decís cuándo parar.

Sentencias break y continue

El día que mi bucle se volvió rebelde

Cuando empecé a jugar con los bucles, tenía un problema: quería buscar un número en una lista enorme y detenerme apenas lo encontrara. Puse un `while` y empecé a recorrer, pero no sabía cómo salir antes. Alguien me dijo: “Probá con `break`, es como un botón de pánico”. Lo probé y ¡funcionó! De repente mi programa dejaba de dar vueltas al pedo y se frenaba justo donde yo quería. Después descubrí `continue`, que no frena del todo sino que salta una vuelta, como cuando en una fila dejás pasar a alguien y seguís adelante. En este capítulo te cuento cómo usar estas dos joyitas.

¿Qué son las iteraciones?

En programación, una **iteración** es la repetición de un segmento de código dentro de un programa. Los ciclos (como `while` o `for`) pueden ser **interrumpidos**, dejar de ejecutar el código que se encuentra dentro del bucle e iniciar una nueva iteración.

Para lograr la interrupción de una iteración, contamos con las sentencias `break` y `continue`.

+-----+	
break → detiene el bucle por completo	
continue → salta a la siguiente iteración	
Ambos se usan DENTRO de bucles (while o for)	
+-----+	

Sentencia break

La sentencia `break` se utiliza para detener la ejecución de una iteración y **salir del bucle completamente**, con lo cual el programa podrá continuar con la ejecución del código que se encuentre fuera del bucle.

Pensá en un buscador que revisa una lista de invitados. Apenas encuentra a la persona que busca, se detiene, no sigue revisando el resto de la lista. Así funciona `break`: en cuanto se ejecuta, el bucle muere y el programa sigue con lo que viene después.

Ejemplo básico de break

```
print("Ejemplo con break")
contador = 0
```

```
while contador < 10:
    contador += 1

    if contador == 5:
        break

    print('Valor actual de la variable:', contador)

print('Fin del programa, la sentencia break se ha ejecutado')
```

Salida:

```
Ejemplo con break
Valor actual de la variable: 1
Valor actual de la variable: 2
Valor actual de la variable: 3
Valor actual de la variable: 4
Fin del programa, la sentencia break se ha ejecutado
```

Cuando `contador` llega a 5, se ejecuta `break` y el bucle se detiene inmediatamente. Las iteraciones 5, 6, 7, 8, 9, 10 nunca ocurren.

Ejemplo: adivinar un número secreto

```
secreto = 7
intentos = 0

while True:
    intento = int(input("Adivina el numero (1-10): "))
    intentos += 1

    if intento == secreto:
        print("Felicitaciones! Lo lograste en", intentos, "intentos")
        break
    else:
        print("Incorrecto, intenta de nuevo")

print("Fin del juego")
```

Acá usamos un `while True` (bucle infinito) y confiamos en que el `break` nos sacará cuando se acierte.

Ejemplo: salir con “salir”

```
while True:
    nombre = input("Ingrese un nombre (o 'salir' para terminar): ")
    if nombre == "salir":
        break
    print("Hola", nombre)
print("Programa terminado")
```

Sentencia continue

La sentencia **continue** permite detener la iteración actual y **volver al principio del bucle** para comenzar una nueva iteración, si la condición que rige a nuestro bucle lo permite.

Volviendo a la fila del supermercado: si estás contando cuántas personas llevan caramelos, y de repente ves a alguien que solo compra verduras, lo ignoras y pasas a la siguiente persona. Eso es **continue**: se saltea lo que queda del bloque y arranca la vuelta siguiente.

Ejemplo básico de continue

```
print("Ejemplo con continue")
contador = 0

while contador < 10:
    contador += 1

    if contador == 5:
        continue

    print('Valor actual de la variable:', contador)

print('Fin del programa')
```

Salida:

```
Ejemplo con continue
Valor actual de la variable: 1
Valor actual de la variable: 2
Valor actual de la variable: 3
Valor actual de la variable: 4
Valor actual de la variable: 6
Valor actual de la variable: 7
Valor actual de la variable: 8
Valor actual de la variable: 9
Valor actual de la variable: 10
Fin del programa
```

Cuando **contador** llega a 5, se ejecuta **continue** y el **print()** se salta. El bucle sigue con el 6, 7, 8, 9, 10. ¡El 5 queda afuera!

Ejemplo: solo impares

```
i = 0
while i < 10:
    i += 1

    if i % 2 == 0:
        continue

    print(i, "es impar")
```

Salida:

```
1 es impar
3 es impar
5 es impar
7 es impar
9 es impar
```

Ejemplo: filtrar números positivos

```
numeros = [1, -5, 3, -2, 8, -1, 4]
i = 0
```

```
while i < len(numeros):
    if numeros[i] < 0:
        i += 1
        continue

    print("Numero positivo:", numeros[i])
    i += 1
```

¡Atención! Cuando usamos `continue`, hay que asegurarse de que el contador avance igual, porque si no nos quedamos en un bucle infinito. Fijate que en el ejemplo incrementamos `i` antes del `continue`.

Diferencia entre `break` y `continue`

	break	continue
Qué hace	Sale del bucle completamente	Salta a la siguiente iteración
El bucle sigue?	Termina (no sigue)	Sigue ejecutándose (con la próxima vuelta)
Cuándo usarlo	Encontraste lo que buscabas	Querés saltar un caso específico
Analogía	Bajar del colectivo	Pasar de largo un pasajero sin parar

Comparación en vivo

```
print("=== DEMOSTRACION BREAK ===")
i = 1
while i <= 10:
    if i == 5:
        break
    print(i, end=" ")
    i += 1
```

```
print("\n\n=== DEMOSTRACION CONTINUE ===")
i = 1
while i <= 10:
    if i == 5:
        i += 1
        continue
    print(i, end=" ")
    i += 1
```

Salida:

```
=== DEMOSTRACION BREAK ===
```

```
1 2 3 4
```

```
=== DEMOSTRACION CONTINUE ===
```

```
1 2 3 4 6 7 8 9 10
```

Ejercicios propuestos

1. Escribí un bucle que imprima números del 1 al 10 pero que se detenga cuando llegue a 5 (usando break).
2. Escribí un bucle que imprima números del 1 al 10 pero que salte el número 5 (usando continue).
3. Hacé un programa que pida números al usuario hasta que ingrese 0, usando break.
4. Escribí un bucle que imprima solo los números impares del 1 al 20 (usando continue).
5. Buscá el número 7 en la lista [3, 5, 2, 7, 9, 1] y detenete al encontrarlo.
6. Imprimí los números del 1 al 15 pero salteá los múltiplos de 3.
7. Pedí una contraseña y terminá cuando sea "python123".
8. Imprimí números del 1 al 20 pero detenete al superar el 15.
9. Sumá números ingresados por el usuario hasta que ingrese -1 (sin sumar el -1).
10. Recorré la palabra "Python" y detenete en la letra "h" sin imprimirla.
11. Imprimí los números del 1 al 10 saltando los pares.
12. Encontrá el primer múltiplo de 7 entre 1 y 100.
13. Imprimí los números del 1 al 20, pero salteá el 13 (mala suerte).
14. Pedí nombres hasta que se ingrese "fin".
15. Recorré "Hola mundo" e imprimí las letras hasta el primer espacio.
16. Imprimí los números del 1 al 30 saltando los múltiplos de 5.
17. Calculá la suma solo de los números positivos ingresados (termina con 0).
18. Contá del 10 al 1 pero salteá el 5.
19. Ingresá números y mostrá solo los mayores a 10; terminá con 0.
20. Simulá un menú que se repita hasta que el usuario elija "salir".

Soluciones paso a paso

Solución 1

```
i = 1
while i <= 10:
    if i == 5:
        break
    print(i)
    i += 1
```

Solución 2

```
i = 1
while i <= 10:
    if i == 5:
        i += 1
        continue
    print(i)
    i += 1
```

Solución 3

```
while True:
    num = int(input("Numero (0 para terminar): "))
    if num == 0:
        break
    print("Ingresaste:", num)
```

Solución 4

```
i = 1
while i <= 20:
    if i % 2 == 0:
        i += 1
        continue
    print(i)
    i += 1
```

Solución 5

```
lista = [3, 5, 2, 7, 9, 1]
for num in lista:
    if num == 7:
        print("Encontre el 7!")
        break
```

Solución 6

```
i = 1
while i <= 15:
```



```
if i % 3 == 0:
    i += 1
    continue
print(i)
i += 1
```

Solución 7

```
while True:
    clave = input("Contraseña: ")
    if clave == "python123":
        print("Acceso permitido")
        break
    print("Contraseña incorrecta")
```

Solución 8

```
i = 1
while i <= 20:
    if i > 15:
        break
    print(i)
    i += 1
```

Solución 9

```
suma = 0
while True:
    num = int(input("Numero (-1 para terminar): "))
    if num == -1:
        break
    suma += num
print("Suma total:", suma)
```

Solución 10

```
palabra = "Python"
for letra in palabra:
    if letra == 'h':
        break
    print(letra)
```

Solución 11

```
i = 1
while i <= 10:
    if i % 2 == 0:
        i += 1
        continue
    print(i)
    i += 1
```

Solución 12

```
i = 1
while i <= 100:
    if i % 7 == 0:
        print("Primer multiplo de 7:", i)
        break
    i += 1
```

Solución 13

```
i = 1
while i <= 20:
    if i == 13:
        i += 1
        continue
    print(i)
    i += 1
```

Solución 14

```
while True:
    nombre = input("Nombre (o 'fin' para terminar): ")
    if nombre == "fin":
        break
    print("Hola", nombre)
```

Solución 15

```
texto = "Hola mundo"
for letra in texto:
    if letra == " ":
        break
    print(letra)
```

Solución 16

```
i = 1
while i <= 30:
    if i % 5 == 0:
        i += 1
        continue
    print(i)
    i += 1
```

Solución 17

```
suma = 0
while True:
    num = int(input("Numero (0 para terminar): "))
    if num == 0:
```

```
        break
    if num > 0:
        suma += num
print("Suma de positivos:", suma)
```

Solución 18

```
i = 10
while i >= 1:
    if i == 5:
        i -= 1
        continue
    print(i)
    i -= 1
```

Solución 19

```
while True:
    num = int(input("Numero (0 para terminar): "))
    if num == 0:
        break
    if num > 10:
        print("Mayor a 10:", num)
```

Solución 20

```
while True:
    print("\n=== MENU ===")
    print("1. Opcion A")
    print("2. Opcion B")
    print("3. Salir")
    opcion = input("Elija: ")
    if opcion == "3":
        print("Hasta luego!")
        break
    elif opcion == "1":
        print("Elegiste A")
    elif opcion == "2":
        print("Elegiste B")
    else:
        print("Opcion invalida")
```

Resumen del capítulo

En este capítulo aprendiste:

- Una **iteración** es la repetición de un segmento de código.
- **break**: detiene el bucle por completo y sale de él.
- **continue**: salta la iteración actual y continúa con la siguiente.

- Ambos se usan dentro de bucles (`while` o `for`).
- `break` es ideal cuando encuentras lo que buscabas.
- `continue` sirve para ignorar ciertos casos sin romper el bucle.
- ¡Cuidado con los contadores! Si usas `continue`, asegurate de que el contador avance igual para no caer en un bucle infinito.

break y continue son el freno y el embrague de tus bucles. Con ellos tomás el control total de cada vuelta.

Función len()

El día que dejé de contar con los dedos

Cuando empecé a hacer programas que procesaban texto, tenía un problema recurrente: quería saber cuántas letras tenía una palabra. Al principio, lo hacía a ojo y ponía el número a mano. Pero claro, si cambiaba la palabra, el número quedaba mal. Después intenté contar con un bucle, pero era un lío. Hasta que un amigo me dijo: “Usá `len()`, te da la longitud de una”. ¡Y era tan simple! `len("Hola")` devuelve 4. De repente, contar caracteres, elementos de una lista o claves de un diccionario se volvió una pavada. En este capítulo te cuento todo sobre esta función que te va a salvar las papas más veces de las que imaginás.

¿Qué es la función len()?

La función `len()` es una función incorporada en Python que devuelve la cantidad de elementos (longitud) de un objeto. Se puede usar con strings, listas, tuplas, diccionarios y otros tipos de datos que sean secuencias o colecciones. Es como una cinta métrica universal: la ponés sobre lo que quieras medir y te da la medida.

```
+-----+
| len(objeto) → devuelve la cantidad de elementos |
| len("Hola") → 4 |
| len([1, 2, 3]) → 3 |
| len((1, 2, 3, 4)) → 4 |
+-----+
```

len() con strings (cadenas de texto)

Cuando usamos `len()` con un string, nos devuelve la cantidad de caracteres que contiene, incluyendo letras, números, espacios y símbolos. Cada espacio cuenta como un carácter, y los saltos de línea también.

Ejemplo básico

```
print('Hola tiene', len('Hola'), 'caracteres')
```

Salida:

```
Hola tiene 4 caracteres
```

Más ejemplos con strings

```
palabra1 = "Python"
palabra2 = "Hola mundo"
palabra3 = ""
palabra4 = "12345"

print(len(palabra1)) # 6
print(len(palabra2)) # 10 (incluye el espacio)
print(len(palabra3)) # 0 (string vacío)
print(len(palabra4)) # 5 (los números como texto cuentan)
```

len() con espacios y caracteres especiales

```
texto1 = "a b c"
texto2 = "Hola\nMundo"
texto3 = "   " # tres espacios

print(len(texto1)) # 5 (a, espacio, b, espacio, c)
print(len(texto2)) # 9 (H,o,l,a,\n,M,u,n,d,o)
print(len(texto3)) # 3 (tres espacios)
```

Reflexión: ¿Cuánto vale len()? ¿Y len()? Probá en tu cabeza y después en Python.

len() con listas

Cuando usamos len() con una lista, nos devuelve la cantidad de elementos que contiene, sin importar el tipo de cada elemento.

Ejemplos con listas

```
lista1 = [1, 2, 3, 4, 5]
lista2 = ["manzana", "pera", "uva"]
lista3 = []
lista4 = [10, "hola", True, 3.14]

print(len(lista1)) # 5
print(len(lista2)) # 3
print(len(lista3)) # 0
print(len(lista4)) # 4
```

len() con tuplas

Las tuplas son similares a las listas pero inmutables. len() también funciona con ellas.

Ejemplos con tuplas

```
tupla1 = (1, 2, 3, 4)
tupla2 = ("a", "b", "c")
tupla3 = ()
```

```
print(len(tupla1)) # 4
print(len(tupla2)) # 3
print(len(tupla3)) # 0
```

len() con diccionarios

Cuando usamos `len()` con un diccionario, nos devuelve la cantidad de pares clave-valor que contiene.

Ejemplos con diccionarios

```
dic1 = {"nombre": "Nahuel", "edad": 25}
dic2 = {}
dic3 = {1: "uno", 2: "dos", 3: "tres", 4: "cuatro"}

print(len(dic1)) # 2
print(len(dic2)) # 0
print(len(dic3)) # 4
```

Usos prácticos de len()

Validar que un string no esté vacío

```
nombre = input("Ingrese su nombre: ")
if len(nombre) > 0:
    print("Hola", nombre)
else:
    print("No ingresaste ningun nombre")
```

Validar longitud mínima de contraseña

```
contrasena = input("Ingrese su contraseña: ")
if len(contrasena) >= 8:
    print("Contraseña valida")
else:
    print("La contraseña debe tener al menos 8 caracteres")
```

Recorrer una lista usando len()

```
frutas = ["manzana", "pera", "uva", "sandia"]
i = 0
while i < len(frutas):
    print(frutas[i])
    i += 1
```

Verificar si una lista está vacía

```
carrito = []
```

```

if len(carrito) == 0:
    print("El carrito está vacío")
else:
    print("Tienes", len(carrito), "productos")

```

Ejemplos combinados

Ejemplo 1: Contar vocales en una palabra

```

palabra = input("Ingrese una palabra: ")
vocales = 0
for letra in palabra:
    if letra in "aeiouAEIOU":
        vocales += 1
print("La palabra tiene", vocales, "vocales")
print("Longitud total:", len(palabra))

```

Ejemplo 2: Validar que el nombre tenga más de 3 letras

```

nombre = input("Ingrese su nombre: ")
if len(nombre) > 3:
    print("Bienvenido", nombre)
else:
    print("El nombre debe tener más de 3 caracteres")

```

Ejemplo 3: Contar caracteres sin espacios

```

frase = input("Ingrese una frase: ")
sin_espacios = frase.replace(" ", "")
print("La frase tiene", len(frase), "caracteres totales")
print("La frase tiene", len(sin_espacios), "caracteres sin espacios")

```

Tabla resumen de len()

TIPO DE DATO	EJEMPLO
string	len("Hola") → 4
string	len("Hola mundo") → 10
string	len("") → 0
lista	len([1, 2, 3]) → 3
lista	len([]) → 0
tupla	len((1, 2, 3, 4)) → 4
diccionario	len({"a":1, "b":2}) → 2

Errores comunes con len()

Error: len() con números directamente

```
numero = 123
print(len(numero)) # TypeError: object of type 'int' has no len()
```

Solución: Convertir a string primero: `len(str(numero))`

Error: olvidar los paréntesis

```
print(len "Hola") # SyntaxError: invalid syntax
```

Solución: Siempre usar paréntesis: `len("Hola")`

Ejercicios propuestos

1. Usá `len()` para mostrar cuántos caracteres tiene la palabra "Python".
2. Pedí al usuario su nombre y mostrá cuántas letras tiene.
3. Pedí al usuario una frase y mostrá cuántos caracteres tiene (incluyendo espacios).
4. Creá una lista con 5 nombres y usá `len()` para mostrar cuántos hay.
5. Pedí al usuario una contraseña y verificá que tenga al menos 6 caracteres.
6. Mostrá la longitud de una lista vacía usando `len()`.
7. Pedí al usuario dos palabras y mostrá cuál es más larga (tiene más caracteres).
8. Creá una tupla con los días de la semana y mostrá cuántos días tiene.
9. Pedí al usuario un texto y mostrá la cantidad de caracteres sin contar los espacios.
10. Creá un diccionario con 3 datos personales (nombre, edad, ciudad) y mostrá cuántas claves tiene.
11. Pedí al usuario un número y mostrá cuántos dígitos tiene (usando `str()`).
12. Usá `len()` dentro de un `if` para verificar si un string está vacío.
13. Pedí al usuario una palabra y mostrá la primera y la última letra usando `len()`.
14. Creá una lista de números y usá `len()` en un bucle `while` para recorrerla.
15. Pedí al usuario un texto y mostrá cuántas veces aparece la letra 'a' usando `len()` y `count()`.
16. Creá un programa que pida nombres hasta que se ingrese "fin" y luego muestre cuántos nombres se ingresaron.
17. Pedí al usuario una palabra y mostrá si es capicúa (se lee igual al revés) usando `len()` y slicing.
18. Creá una lista de compras con 5 items y usá `len()` para mostrar cuántos items hay.
19. Pedí al usuario un texto y mostrá el caracter central (si la longitud es impar).
20. Creá un programa que simule un cajero automático que permita hasta 3 intentos de PIN usando `len()` para contar intentos (sin usar `len()` directamente, pero sí en el contexto).

Soluciones paso a paso

Solución 1

```
print(len("Python")) # 6
```

Solución 2

```
nombre = input("Ingrese su nombre: ")
print("Tu nombre tiene", len(nombre), "letras")
```

Solución 3

```
frase = input("Ingrese una frase: ")
print("La frase tiene", len(frase), "caracteres")
```

Solución 4

```
nombres = ["Ana", "Luis", "Maria", "Juan", "Carlos"]
print("Cantidad de nombres:", len(nombres))
```

Solución 5

```
clave = input("Ingrese contraseña: ")
if len(clave) >= 6:
    print("Contraseña válida")
else:
    print("Debe tener al menos 6 caracteres")
```

Solución 6

```
lista_vacia = []
print(len(lista_vacia)) # 0
```

Solución 7

```
palabra1 = input("Primera palabra: ")
palabra2 = input("Segunda palabra: ")
if len(palabra1) > len(palabra2):
    print(palabra1, "es más larga")
elif len(palabra2) > len(palabra1):
    print(palabra2, "es más larga")
else:
    print("Tienen la misma longitud")
```

Solución 8

```
dias = ("Lunes", "Martes", "Miercoles", "Jueves", "Viernes", "Sabado", "Domingo")
print("Cantidad de días:", len(dias))
```

Solución 9

```
texto = input("Ingrese un texto: ")
sin_espacios = texto.replace(" ", "")
print("Caracteres sin espacios:", len(sin_espacios))
```

Solución 10

```
datos = {"nombre": "Nahuel", "edad": 25, "ciudad": "Buenos Aires"}
print("Cantidad de datos:", len(datos))
```

Solución 11

```
numero = input("Ingrese un número: ")
print("Cantidad de dígitos:", len(numero))
```

Solución 12

```
texto = input("Ingrese algo: ")
if len(texto) == 0:
    print("No ingresaste nada")
else:
    print("Ingresaste:", texto)
```

Solución 13

```
palabra = input("Ingrese una palabra: ")
print("Primera letra:", palabra[0])
print("Última letra:", palabra[len(palabra) - 1])
```

Solución 14

```
numeros = [10, 20, 30, 40, 50]
i = 0
while i < len(numeros):
    print(numeros[i])
    i += 1
```

Solución 15

```
texto = input("Ingrese un texto: ")
print("Cantidad de 'a':", texto.count('a'))
print("Longitud total:", len(texto))
```

Solución 16

```
nombres = []
while True:
    nombre = input("Nombre (o 'fin' para terminar): ")
    if nombre == "fin":
        break
```

```
nombres.append(nombre)
print("Ingresaste", len(nombres), "nombres")
```

Solución 17

```
palabra = input("Ingrese una palabra: ")
if palabra == palabra[::-1]:
    print("Es capicúa")
else:
    print("No es capicúa")
print("Longitud:", len(palabra))
```

Solución 18

```
compras = ["pan", "leche", "huevos", "fruta", "verdura"]
print("Items en la lista de compras:", len(compras))
```

Solución 19

```
texto = input("Ingrese un texto: ")
if len(texto) % 2 != 0:
    centro = len(texto) // 2
    print("Caracter central:", texto[centro])
else:
    print("La longitud es par, no hay un solo caracter central")
```

Solución 20

```
pin_correcto = "1234"
intentos = 0
while intentos < 3:
    pin = input("Ingrese su PIN: ")
    intentos += 1
    if pin == pin_correcto:
        print("Acceso permitido")
        break
    else:
        print("PIN incorrecto. Intentos restantes:", 3 - intentos)
if intentos == 3 and pin != pin_correcto:
    print("Tarjeta bloqueada")
```

Resumen del capítulo

En este capítulo aprendiste:

- La función **len()** devuelve la cantidad de elementos de un objeto.
- Con strings: devuelve el número de caracteres (incluyendo espacios y signos).
- Con listas: devuelve el número de elementos.

- Con tuplas: devuelve el número de elementos.
- Con diccionarios: devuelve el número de pares clave-valor.
- `len()` es muy útil para validar entradas, recorrer colecciones y contar elementos.
- No funciona directamente con números; hay que convertirlos a string.

len() es como una regla mágica que mide lo que le pongas delante: palabras, listas, diccionarios... ¡y nunca se equivoca!

Concatenación con el método format()

El día que el + me dejó de garpe

Cuando empecé a armar mensajes con variables, usaba el + y andaba bien. Pero un día quise mostrar un número y una palabra juntos: "Hola Γ nombre + ", tenés Γ edad + .años". Y ¡pum! error. Porque `edad` era entero y el + no puede sumar strings con enteros. Me vi obligado a usar `str(edad)` cada vez, y el código se volvía un choclo de signos de más. Hasta que un amigo me dijo: "Che, usá `format()`, te olvidás de esos problemas". Y era cierto: con `format()` mezclás números y texto sin convertir nada, y queda todo clarito. En este capítulo te muestro cómo dominar este método y sus tres formas de uso.

¿Qué es el método format()?

El método `.format()` es una herramienta que Python incluye en los strings para construir textos dinámicos. En vez de pegar pedazos con +, vos escribís un molde con **marcadores** (`{}`) y después le pasás los valores que van en esos huecos. Es como un formulario preimpreso: vos tenés el texto base y dejás casilleros vacíos que se completan después.

```
+-----+
|
|  "Hola {} tenés {} años".format(nombre, edad) |
|          ↑          ↑          ↑          ↑   |
|   marcador1  marcador2      argumento1  arg2  |
|
+-----+
```

Las ventajas sobre el + son varias:

- No necesitás convertir números a string: `format()` lo hace solo.
- El código queda más ordenado, sobre todo cuando hay muchas variables.
- Podés reutilizar y reordenar valores sin duplicar variables.

Alternativa 1: Sustitución por posición relativa

La forma más directa es dejar las llaves vacías y pasar los valores en orden. Python reemplaza la primera `{}` con el primer argumento, la segunda con el segundo, y así.

Ejemplo:

```
nombre = "Carlos"
edad = 22

print("Hola {} tienes {} años".format(nombre, edad))
```

Salida:

Hola Carlos tienes 22 años

Podés mezclar varios tipos sin problema:

```
ciudad = "Buenos Aires"
pais = "Argentina"
poblacion = 15000000

print("Vivo en {}, que está en {} y tiene {} habitantes".format(ciudad, pais, poblacion))
```

Salida:

Vivo en Buenos Aires, que está en Argentina y tiene 15000000 habitantes

Incluso podés poner operaciones adentro:

```
a = 10
b = 5
print("La suma de {} y {} es {}".format(a, b, a + b))
```

Salida:

La suma de 10 y 5 es 15

Reflexión: ¿Qué pasa si ponés más marcadores que argumentos? Probá en tu máquina y fijate el error que tira.

Alternativa 2: Referencia por identificador (keywords)

Cuando manejas muchas variables, a veces es fácil confundir el orden. Para eso podés poner nombres dentro de las llaves y después asignarles valores con `clave=valor`. Así no importa el orden en que los pasás.

```
print("Hola {n} tienes {e} años".format(n="Carlos", e=22))
```

Salida:

Hola Carlos tienes 22 años

Acá `n` y `e` son etiquetas que vos inventás. También podés usar variables que ya tengas:

```
nombre = "Nahuel"
edad = 25
print("Me llamo {nom} y mi edad es {ed}".format(nom=nombre, ed=edad))
```

Incluso podés dar vuelta los argumentos y el resultado sigue siendo el deseado:

```
print("{segundo} {primero}".format(primero="Hola", segundo="Mundo"))
# Salida: Mundo Hola
```

Esta modalidad es ideal para frases largas donde la claridad es fundamental.

Alternativa 3: Referencia por índice numérico

Si querés más control, podés usar números dentro de las llaves que indican la posición del argumento empezando desde 0. Esto permite reordenar los valores e incluso repetirlos sin necesidad de volver a pasarlos.

```
nombre = "Carlos"
edad = 22
print("El usuario {1} se llama {0}".format(nombre, edad))
```

Salida:

El usuario 22 se llama Carlos

Fijate que {0} es nombre (el primer argumento) y {1} es edad. Invertimos el orden de impresión sin tocar `.format()`.

Podés repetir el mismo índice:

```
print("{0} {0} {1} {1}".format("Hola", "Mundo"))
# Salida: Hola Hola Mundo Mundo
```

Y también usar índices en cualquier orden:

```
fruta1 = "manzana"
fruta2 = "pera"
fruta3 = "uva"
print("{2} - {0} - {1}".format(fruta1, fruta2, fruta3))
# Salida: uva - manzana - pera
```

Ejercicio rápido: ¿Qué imprimiría `print("1 0 1".format(.a, "b"))`? Pensá antes de probar.

Comparación de las tres alternativas

ALTERNATIVA	SINTAXIS	EJEMPLO
Posición	"{ } { }".format	"Hola { } { }".format(nombre, edad)
relativa	(a, b)	
Keywords	"{x}{y}".format	"Hola {n} {e}".format(n="A", e=25)
	(x=a, y=b)	
Índices	"{0}{1}".format	"Hola {0} {1}".format(nombre, edad)
numéricos	(a, b)	

Ejemplos que te van a dar hambre (de código)

Factura sencilla con índices

```
producto = "Teclado"
cantidad = 3
precio = 2500
total = cantidad * precio

factura = "Producto: {0} | Cantidad: {1} | Precio: ${2} | Total: ${3}".format(
    producto, cantidad, precio, total
)
print(factura)
```

Mensaje clarito con keywords

```
print("El {tipo} {nom} vale ${precio}".format(
    tipo="mouse",
    nom="Logitech",
    precio=3500
))
```

Repetición con índices

```
print("{0} + {0} = {1}".format(5, 10)) # 5 + 5 = 10
```

Alineación para tablas

```
print("{:<10} | {:^10} | {:>10}".format("Nombre", "Edad", "Ciudad"))
print("{:<10} | {:^10} | {:>10}".format("Nahuel", "25", "Bs As"))
# Alinea izquierda, centro, derecha en campos de 10 caracteres.
```

Dale, repasemos rápido lo importante

- `.format()` usa llaves `{}` como marcadores de posición.
- Podés referenciar los valores por **posición relativa**, **keywords** o **índices numéricos**.
- No requiere convertir tipos; acepta enteros, flotantes, strings, etc.
- Hace el código más legible que la concatenación con `+`.
- Ideal para armar mensajes dinámicos, reportes, facturas y tablas.

Con `format()` dejás de pegar con plasticola y empezás a encajar las piezas como en un tetris.

Ejercicios del capítulo

1. Usando posición relativa, mostrá: “Mi nombre es [nombre] y tengo [edad] años”.
2. Con posición relativa, imprimí la suma de dos números: “La suma de [a] y [b] es [resultado]”.
3. Usando keywords, mostrá: “El producto [producto] cuesta [precio] pesos”.
4. Con índices numéricos, invertí nombre y apellido: “[apellido], [nombre]”.
5. Con posición relativa, armá un saludo: “Hola [nombre], bienvenido a [ciudad]”.
6. Usando keywords, describí un auto: “El [auto] de color [color] cuesta [precio]”.
7. Con índices, repetí un valor dos veces: “[valor] es igual a [valor]”.
8. Calculá el área de un rectángulo y mostralo con posición relativa: “El área de [base] x [altura] es [area]”.
9. Usá keywords para mostrar nombre, edad y ciudad en un solo print.
10. Con índices numéricos, imprimí tres palabras en orden inverso.
11. Creá una factura simple con producto, cantidad, precio unitario y total usando posición relativa.
12. Usá keywords para mostrar una operación matemática (ej: “ $5 + 3 = 8$ ”).
13. Con índices, repetí un mismo nombre tres veces en un solo print.
14. Pedí nombre, edad y ciudad al usuario y mostralos con `format()`.
15. Calculá y mostrá el promedio de tres notas con posición relativa.
16. Con keywords, creá un mensaje de bienvenida a un curso.
17. Con índices, invertí el orden de tres palabras: `rojo verde azul` → `.azul verde rojo`.
18. Usá `format()` y alineación para hacer una pequeña tabla de dos filas con tres columnas.
19. Calculá el IMC ($\text{peso}/\text{altura}^2$) y mostralo con dos decimales usando `format()`.
20. Escribí un programa que use las tres formas de `format()` (posición, keywords, índices) cada una en un print distinto.

Soluciones paso a paso

Ejercicio 1

```
nombre = "Nahuel"
edad = 25
print("Mi nombre es {} y tengo {} años".format(nombre, edad))
```

Ejercicio 2

```
a = 10
b = 5
print("La suma de {} y {} es {}".format(a, b, a + b))
```

Ejercicio 3

```
print("El producto {prod} cuesta {precio} pesos".format(prod="Mouse", precio=1500))
```

Ejercicio 4

```
nombre = "Nahuel"  
apellido = "Zanini"  
print("{1}, {0}".format(nombre, apellido))    # apellido, nombre
```

Ejercicio 5

```
nombre = "Nahuel"  
ciudad = "Buenos Aires"  
print("Hola {}, bienvenido a {}".format(nombre, ciudad))
```

Ejercicio 6

```
print("El {auto} de color {color} cuesta {precio}".format(auto="Fiat", color="rojo", precio=50000))
```

Ejercicio 7

```
valor = 42  
print("{0} es igual a {0}".format(valor))
```

Ejercicio 8

```
base = 10  
altura = 5  
area = base * altura  
print("El área de {} x {} es {}".format(base, altura, area))
```

Ejercicio 9

```
print("{nom} tiene {ed} años y vive en {ciu}".format(nom="Nahuel", ed=25, ciu="Bs As"))
```

Ejercicio 10

```
print("{2} {1} {0}".format("rojo", "verde", "azul"))    # azul verde rojo
```

Ejercicio 11

```
producto = "Teclado"  
cantidad = 2  
precio = 2500  
total = cantidad * precio  
print("Producto: {} | Cantidad: {} | Precio: ${} | Total: ${}".format(producto, cantidad, precio, total))
```

Ejercicio 12

```
a = 5
b = 3
print("{op1} + {op2} = {res}".format(op1=a, op2=b, res=a + b))
```

Ejercicio 13

```
nombre = "Nahuel"
print("{0} {0} {0}".format(nombre))
```

Ejercicio 14

```
nombre = input("Nombre: ")
edad = input("Edad: ")
ciudad = input("Ciudad: ")
print("{} tiene {} años y vive en {}".format(nombre, edad, ciudad))
```

Ejercicio 15

```
n1, n2, n3 = 7, 8, 9
promedio = (n1 + n2 + n3) / 3
print("El promedio de {}, {} y {} es {}".format(n1, n2, n3, promedio))
```

Ejercicio 16

```
print("Bienvenido {nombre} al curso de {curso}".format(nombre="Nahuel", curso="Python"))
```

Ejercicio 17

```
print("{2} {1} {0}".format("rojo", "verde", "azul")) # azul verde rojo
```

Ejercicio 18

```
print("{:<10} | {:~10} | {:>10}".format("Nombre", "Edad", "Ciudad"))
print("{:<10} | {:~10} | {:>10}".format("Nahuel", "25", "Bs As"))
```

Ejercicio 19

```
peso = 75
altura = 1.75
imc = peso / (altura ** 2)
print("IMC: {:.2f}".format(imc))
```

Ejercicio 20

```
nombre = "Nahuel"
edad = 25

# Posición relativa
print("Hola {} tenés {} años".format(nombre, edad))
```

```
# Keywords
print("Hola {n} tenés {e} años".format(n=nombre, e=edad))

# Índices numéricos
print("El usuario {1} se llama {0}".format(nombre, edad))
```

El método format() es una navaja suiza para armar textos. Dominá las tres formas y elegí la que te quede más clara en cada caso.

Concatenación con f-strings

¿Qué son las f-strings?

Cuando empecé a programar en Python, uno de los primeros dolores de cabeza que tuve fue juntar texto con variables. Quería mostrar algo como “Hola, me llamo Nahuel y tengo 25 años”, y lo primero que se me ocurrió fue usar el signo + para pegar pedazos de frase:

```
nombre = "Nahuel"
edad = 25
print("Hola, me llamo " + nombre + " y tengo " + edad + " años")
```

Y ahí sonó el error: `TypeError: can only concatenate str (not int) to str`. Me quedé duro. ¿Por qué no podía mezclar palabras con números? Resulta que + solo une cadenas con cadenas, no con números. Había que convertir la edad con `str(edad)`. Un lío, sobre todo cuando tenés muchas variables.

Después descubrí el método `.format()` que ya era más limpio, pero todavía me parecía un poco enrevesado. Hasta que aparecieron las **f-strings** y me cambiaron la vida. Son la forma más moderna, rápida y legible de armar cadenas dinámicas en Python. Se incorporaron en la versión **Python 3.6**, así que si tenés una versión más vieja, tendrías que actualizar (pero hoy en día casi todo el mundo usa Python 3.6 o superior).

```
+-----+
|
| f-strings = la forma más moderna y eficiente |
| de concatenar en Python                      |
|
| f"Hola {nombre}, tenés {edad} años"         |
|
| Ventajas:                                    |
| - Más legible que + y format()              |
| - Más rápida en ejecución                   |
| - Permite expresiones dentro de {}          |
|
+-----+
```

Analogía: Imaginá que tenés un formulario en papel con espacios en blanco para completar. Ese formulario es la plantilla base, y vos solo rellenás los espacios con los datos del día. Las f-strings son justamente eso: una plantilla donde ponés las llaves {} como espacios en blanco, y Python los completa automáticamente con el valor de las variables en tiempo de ejecución. ¡Magia pura!

Sintaxis básica

La regla es re simple: ponés una `f` (o `F`, da igual) justo antes de las comillas que abren el string, y dentro del string, todo lo que quieras que sea dinámico lo metés entre llaves `{}`. No puede haber ni un espacio entre la `f` y la comilla, así: `f"texto {variable}"`.

Nota: *Regla de Oro: Nada de espacios entre la `f` y la comilla de apertura. Si escribís `f "..."` con un espacio, Python te va a mirar con cara rara y te va a tirar un error de sintaxis.*

Primeros pasos: mezclando palabras y números

Mirá este ejemplo, que retoma el problema de mi anécdota:

```
nombre = "Carlos"
edad = 22
estatura = 1.8

# Concatenación directa de múltiples tipos de datos
print(f"Hola {nombre}, tenés {edad} años y medís {estatura} metros.")
```

Si lo ejecutás, la consola te devuelve:

```
Hola Carlos, tenés 22 años y medís 1.8 metros.
```

¿Viste qué limpio? Ni un solo `+`, ni conversiones extrañas. Python se encarga de convertir automáticamente los números a texto y meterlos donde corresponde.

Ahora te muestro unos ejemplos más para que agarres confianza:

```
# Con strings
nombre = "Nahuel"
print(f"Mi nombre es {nombre}")

# Con números
edad = 25
print(f"Tengo {edad} años")

# Con múltiples variables
ciudad = "Buenos Aires"
pais = "Argentina"
print(f"Vivo en {ciudad}, {pais}")
```

Ejercicio rápido: Copiá estos ejemplos en tu editor, ejecutalos y después cambiá los valores de las variables. Fijate cómo Python actualiza todo automáticamente. Es como cambiar la tinta del sello: la plantilla sigue igual, pero los datos cambian.

Evaluación de expresiones

Hasta acá parecen simples sustituciones, pero la verdadera potencia es que dentro de las llaves podés poner **expresiones completas**, y Python las calcula en el momento de ejecutar el `print`. O sea, no solo variables, también operaciones matemáticas, llamadas a funciones, ¡lo que se te ocurra!

Operaciones aritméticas dentro de las llaves

¿Te acordás de cuando en la escuela te pedían “El resultado de 4 más 1 es...”? Con f-strings lo resolvés en un renglón:

```
print(f"El resultado de 4 + 1 es: {4 + 1}")
```

La consola muestra:

```
El resultado de 4 + 1 es: 5
```

Python primero calcula $4 + 1$, obtiene 5, y lo incrusta en la cadena. Así de simple. Podés hacer restas, multiplicaciones, divisiones, potencias, módulo... lo que quieras.

```
a = 10
b = 5
print(f"{a} + {b} = {a + b}")
print(f"{a} - {b} = {a - b}")
print(f"{a} * {b} = {a * b}")
print(f"{a} / {b} = {a / b}")

# Potencias
print(f"2 elevado a 3 es {2 ** 3}")

# Módulo (resto de la división)
print(f"10 dividido 3 da resto {10 % 3}")
```

Reflexión: ¿Notás la diferencia con el operador +? Antes tenías que escribir algo horrible como `"10 + 5 = `str(10+5)`"`. Ahora todo queda encapsulado en un único string, mucho más fácil de leer y modificar.

Llamadas a funciones dentro de f-strings

También podés meter funciones que devuelvan un valor directamente en las llaves. Por ejemplo, `len()` para saber la cantidad de caracteres de un nombre, o `upper()` para pasarlo a mayúsculas.

```
nombre = "Nahuel"
print(f"Tu nombre {nombre} tiene {len(nombre)} letras")
print(f"Mayúscula: {nombre.upper()}")
print(f"Minúscula: {nombre.lower()}")

# Redondear decimales
pi = 3.14159265
print(f"Pi redondeado a 2 decimales: {round(pi, 2)}")
```

Acordate: las funciones se ejecutan en el momento, así que si `nombre` cambia, `len(nombre)` se ajusta.

Formato de números

A veces necesitas mostrar un número con cierta cantidad de decimales, o con separadores de miles. Las f-strings te lo ponen fácil usando dos puntos y una especificación de formato dentro de las llaves, después del valor.

```
# Mostrar siempre 2 decimales
precio = 123.4567
print(f"Precio: {precio:.2f}")

# Porcentaje con un decimal
aprobados = 7
total = 10
print(f"Porcentaje de aprobados: {aprobados / total * 100:.1f}%")

# Separador de miles (cambia según la configuración regional, pero suele ser coma)
poblacion = 15000000
print(f"Población: {poblacion:,}")
```

Dato curioso: El formato `:.2f` le dice a Python: “mostrame este número con exactamente dos dígitos después del punto decimal”. La `f` es de *float*. Y `:` activa el separador de miles. Estos trucos te salvan cuando trabajas con plata o estadísticas.

Interacción con el usuario

Ahora juntemos f-strings con `input()` para hacer programas que hablen con quien los usa. La idea es: pedimos datos, los guardamos en variables, y armamos la respuesta con una f-string.

Ejemplo práctico: calculadora de suma

```
usuario = input("¿Cuál es tu nombre?: ")
num1 = int(input("Introduce un número: "))
num2 = int(input("Introduce otro número: "))

# Mezcla de variables y una expresión de suma al final
print(f"Hola {usuario}, el resultado de {num1} + {num2} es {num1 + num2}")
```

Una ejecución típica se vería así:

```
¿Cuál es tu nombre?: Nahuel
Introduce un número: 8
Introduce otro número: 3
Hola Nahuel, el resultado de 8 + 3 es 11
```

Fijate que convertimos la entrada a número con `int()` porque `input()` siempre devuelve texto, y si no lo convertís, el `+` dentro de la f-string no suma, sino que concatenaría strings (si ambos fueran string). Pero como estamos dentro de una expresión matemática, `num1 + num2` necesita que ambos sean números. Es un detalle importante: la f-string respeta los tipos de datos.

Más ejemplos con input()

```
# Calculadora de IMC (índice de masa corporal)
nombre = input("Nombre: ")
peso = float(input("Peso (kg): "))
altura = float(input("Altura (m): "))
imc = peso / (altura ** 2)

print(f"{nombre}, tu IMC es {imc:.2f}")

# Verificación de edad proyectada
edad = int(input("Edad: "))
print(f"En 10 años tendrás {edad + 10} años")
```

Ejercicio: Hacé un programa que pida el año de nacimiento y el año actual, y muestre cuántos años cumple (o cumplió) la persona con un mensaje simpático.

Comparativa: ¿Por qué usar f-strings?

Para que veas con claridad el antes y el después, te pongo los tres métodos lado a lado.

MÉTODO	SINTAXIS	LEGIBILIDAD
Operador +	"Hola " + nom + " " + str(edad)	Baja (muchas comillas y +)
Método .format()	"Hola {} {}".format(nom, edad)	Media
f-strings	f"Hola {nom} {edad}"	Alta (muy intuitivo)

La diferencia de legibilidad es enorme cuando tenés muchas variables. Con f-strings el código se lee casi como una frase en español, con los huecos justos donde van los datos. Y además, son más rápidas, porque Python las evalúa directamente en el momento sin tener que buscar índices como en .format().

Ejemplo completo: Sistema de pedidos

Para bajar todo a tierra, vamos a armar un mini sistema que toma los datos de un pedido y muestra una factura formateada.

```
print("=== SISTEMA DE PEDIDOS ===")

# Datos del cliente
cliente = input("Nombre del cliente: ")
producto = input("Producto: ")
```

```
cantidad = int(input("Cantidad: "))
precio_unitario = float(input("Precio unitario: "))

# Cálculos
subtotal = cantidad * precio_unitario
iva = subtotal * 0.21
total = subtotal + iva

# Mostrar factura con f-strings
print("\n=== FACTURA ===")
print(f"Cliente: {cliente}")
print(f"Producto: {producto}")
print(f"Cantidad: {cantidad}")
print(f"Precio unitario: ${precio_unitario:.2f}")
print(f"Subtotal: ${subtotal:.2f}")
print(f"IVA (21%): ${iva:.2f}")
print(f"TOTAL: ${total:.2f}")
```

Ejecutalo, jugá con los valores, y vas a ver cómo los cálculos y los formatos se actualizan solos. Es la base para armar presupuestos, tickets, informes, todo lo que se te ocurra.

f-strings con condiciones (operador ternario)

Python permite meter una estructura condicional compacta dentro de las llaves usando el **operador ternario**. La sintaxis es: `<valor_si_verdadero> if <condición> else <valor_si_falso>`.

```
edad = 18
print(f"Eres {'mayor' if edad >= 18 else 'menor'} de edad")

nota = 8
print(f"Estado: {'Aprobado' if nota >= 7 else 'Desaprobado'}")
```

Esto es muy útil para mensajes que cambian según una condición, sin necesidad de armar un if aparte.

f-strings con diccionarios

Si tenés un diccionario, podés acceder a sus claves directamente dentro de las llaves usando la sintaxis de corchetes.

```
persona = {"nombre": "Nahuel", "edad": 25, "ciudad": "Buenos Aires"}
print(f"{persona['nombre']} tiene {persona['edad']} años y vive en {persona['ciudad']}")
```

Ojo: como las comillas dentro del diccionario son simples, no entran en conflicto con las comillas dobles de la f-string. Esa es una buena práctica: si el string principal usa comillas dobles, dentro usá simples, y al revés.

Nota importante sobre versiones

Nota: Requisito de versión: Siempre verificá que tu entorno de ejecución sea **Python 3.6 o superior**. Si estás en una máquina con Python 3.5 o anterior, el intérprete no va a reconocer el prefijo `f` y te marcará un error de sintaxis. Podés chequear tu versión corriendo `python -version` en la terminal (o `python3 -version` en Linux/Mac). Si necesitás actualizar, descargá la última desde python.org.

Ejercicios propuestos

Llegó la hora de ponerte manos a la obra. Cada ejercicio está pensado para que apliques lo que vimos. No te saltees ninguno, porque la práctica hace al maestro.

1. Usando f-strings, mostrá "Mi nombre es [nombre] y tengo [edad] años".
2. Usando f-strings, mostrá `.E1 resultado de 5 + 3 es [resultado]`".
3. Pedí al usuario su nombre y mostrá "Hola [nombre]" usando f-strings.
4. Pedí dos números y mostrá su suma usando f-strings.
5. Mostrá el área de un rectángulo de base 10 y altura 5 usando f-strings y la expresión dentro de las llaves.
6. Pedí el nombre y la edad, y mostrá "Te llamas [nombre] y tenés [edad] años".
7. Usando f-strings, mostrá `.E1 promedio de 7, 8 y 9 es [promedio]`".
8. Pedí el precio de un producto y mostrá `.E1 precio con IVA (21%) es [precio_con_iva]`".
9. Usando f-strings, mostrá la longitud de la palabra "Python".
10. Pedí una palabra y mostrá "La palabra [palabra] tiene [longitud] letras".
11. Mostrá el resultado de 2 elevado a la 5 usando f-strings con la expresión dentro.
12. Pedí la base y la altura de un triángulo y mostrá el área ($\text{base} \times \text{altura} / 2$).
13. Usando f-strings, mostrá un número con 2 decimales (ej: $10.5678 \rightarrow 10.57$).
14. Pedí el nombre y la nota de un alumno, y mostrá si aprobó ($\text{nota} \geq 7$) usando un ternario dentro de f-string.
15. Usando f-strings, mostrá el resto de dividir 17 entre 5.
16. Pedí la cantidad de productos y el precio unitario, mostrá el total usando f-strings.
17. Mostrá el porcentaje que representa 25 sobre 200 usando f-strings.
18. Pedí un número y mostrá su cuadrado y su cubo usando f-strings.
19. Usando f-strings, mostrá el resultado de $(10 + 5) * 2$.
20. Creá un programa que pida nombre, edad, ciudad y altura, y muestre todos los datos en un solo `print` usando f-strings.

Soluciones paso a paso

A continuación te dejo las soluciones, pero hacé el esfuerzo de resolverlos vos primero. Si te trabás, acá están explicadas.

Solución 1

```
nombre = "Nahuel"
edad = 25
print(f"Mi nombre es {nombre} y tengo {edad} años")
```

Explicación: Definimos las variables y las metemos dentro de las llaves. Python las reemplaza.

Solución 2

```
print(f"El resultado de 5 + 3 es {5 + 3}")
```

Explicación: La expresión 5+3 se calcula dentro de las llaves y el resultado (8) se inserta.

Solución 3

```
nombre = input("Ingrese su nombre: ")
print(f"Hola {nombre}")
```

Explicación: Capturamos lo que escribe el usuario y lo mostramos con el saludo.

Solución 4

```
num1 = int(input("Primer número: "))
num2 = int(input("Segundo número: "))
print(f"La suma es {num1 + num2}")
```

Explicación: Convertimos la entrada a int para poder sumar, y la operación va dentro de las llaves.

Solución 5

```
base = 10
altura = 5
print(f"El área es {base * altura}")
```

Explicación: Podés escribir directamente la fórmula dentro de la f-string.

Solución 6

```
nombre = input("Nombre: ")
edad = input("Edad: ")
print(f"Te llamas {nombre} y tenés {edad} años")
```

Explicación: No necesitamos convertir la edad a número porque no la vamos a operar, solo mostrarla.

Solución 7

```
print(f"El promedio de 7, 8 y 9 es {(7 + 8 + 9) / 3}")
```

Explicación: Los paréntesis agrupan la suma antes de dividir, respetando el orden matemático.

Solución 8

```
precio = float(input("Precio: "))
precio_con_iva = precio * 1.21
print(f"El precio con IVA es ${precio_con_iva:.2f}")
```

Explicación: Calculamos el IVA y usamos `:.2f` para mostrar dos decimales y el signo peso.

Solución 9

```
print(f"Python tiene {len('Python')} letras")
```

Explicación: La función `len()` cuenta los caracteres del string.

Solución 10

```
palabra = input("Ingrese una palabra: ")
print(f"La palabra {palabra} tiene {len(palabra)} letras")
```

Explicación: Calculamos la longitud de la palabra ingresada en tiempo real.

Solución 11

```
print(f"2 elevado a la 5 es {2 ** 5}")
```

Explicación: El operador `**` es la potencia.

Solución 12

```
base = float(input("Base: "))
altura = float(input("Altura: "))
print(f"El área del triángulo es {base * altura / 2}")
```

Explicación: La expresión se evalúa directamente dentro de las llaves.

Solución 13

```
numero = 10.5678
print(f"Número con 2 decimales: {numero:.2f}")
```

Explicación: `:.2f` redondea a dos decimales.

Solución 14

```
nombre = input("Nombre: ")
nota = float(input("Nota: "))
print(f"{nombre} {'Aprobó' if nota >= 7 else 'Desaprobó'} con {nota}")
```

Explicación: El ternario elige entre “Aprobó” y “Desaprobó” según la nota.

Solución 15

```
print(f"El resto de 17 dividido 5 es {17 % 5}")
```

Explicación: El operador % devuelve el resto de la división entera.

Solución 16

```
cantidad = int(input("Cantidad: "))
precio = float(input("Precio unitario: "))
print(f"Total: ${cantidad * precio:.2f}")
```

Explicación: Multiplicamos y formateamos con dos decimales.

Solución 17

```
print(f"25 es el {25 / 200 * 100}% de 200")
```

Explicación: La operación da 12.5, se muestra directamente.

Solución 18

```
num = int(input("Número: "))
print(f"Cuadrado: {num ** 2}")
print(f"Cubo: {num ** 3}")
```

Explicación: Calculamos potencias con los operadores correspondientes.

Solución 19

```
print(f"El resultado de (10 + 5) * 2 es {(10 + 5) * 2}")
```

Explicación: Los paréntesis internos aseguran que primero se suma y luego se multiplique.

Solución 20

```
nombre = input("Nombre: ")
edad = input("Edad: ")
ciudad = input("Ciudad: ")
altura = input("Altura: ")
print(f"Me llamo {nombre}, tengo {edad} años, vivo en {ciudad} y mido {altura} metros")
```

Explicación: Juntamos todas las variables en una sola f-string, bien prolijo.

Resumen del capítulo

Dale, repasemos rápido lo importante:

- Las **f-strings** son la forma más moderna y recomendada de formatear cadenas en Python (desde versión 3.6).
- Se crean anteponiendo **f** antes de las comillas: `f"texto"`.
- Las variables y expresiones van dentro de `{}` y Python las reemplaza por su valor en tiempo de ejecución.
- Podés incluir **expresiones matemáticas** y **llamadas a funciones** dentro de las llaves.
- Le podés dar **formato a números** con especificaciones como `:.2f` (dos decimales) o `:,` (separador de miles).
- Son más **legibles** y **rápidas** que el viejo `+` o el método `format()`.
- Combinadas con `input()` te permiten crear programas interactivos de manera muy clara.

Las f-strings son como tener una plantilla mágica donde escribís directamente las variables y Python las completa automáticamente. ¡Disfrutalas!

Métodos `rstrip()` y `lstrip()`

¿Por qué necesitamos limpiar cadenas?

Cuando arranqué a programar, me topé con un problema muy común: leer datos desde un archivo. Yo estaba en Windows y armaba un programita que procesaba una lista de nombres. Todo parecía andar joya, pero cuando comparaba un nombre leído con uno que yo escribía a mano, Python me decía que eran distintos. ¡Estaba desesperado! Después de mucho debuggear (y putear en silencio), me di cuenta de que la línea que leía del archivo traía un salto de línea invisible al final: `\n`. En Windows a veces también venía un `\r` antes. Entonces, "Nahuel" no era lo mismo que "Nahuel\n".

Fue ahí cuando descubrí los métodos para “podar” los extremos de una cadena, como si estuvieras cortando los bordes gastados de una hoja. Python tiene tres grandes aliados: `strip()` (que limpia ambos lados), `rstrip()` (right strip, limpia solo la derecha) y `lstrip()` (left strip, limpia solo la izquierda). En este capítulo vamos a profundizar en estos dos últimos, porque a veces necesitás un bisturí, no una motosierra.

```
+-----+
|
|  "  Hola mundo  "
|      ↑      ↑
|   izquierda  derecha
|
|  lstrip() → elimina espacios de la izquierda
|  rstrip() → elimina espacios de la derecha
|  strip()  → elimina de ambos lados
|
+-----+
```

Analogía: Imaginá que tenés una barra de pan con los extremos un poco quemados. `lstrip()` le saca la parte quemada de la izquierda, `rstrip()` la de la derecha, y `strip()` corta las dos puntas. Dependiendo de cómo quede el pan después de hornear, elegís cuál usar.

El método `rstrip()` (Right Strip)

`rstrip()` es tu herramienta para eliminar caracteres **solamente del final** de la cadena, es decir, del lado derecho. En criollo: “sacale lo que sobre a la derecha”.

Comportamiento por defecto

Si no le especificás nada, `rstrip()` remueve todos los **espacios en blanco** que encuentre al final: espacios comunes, tabulaciones (`\t`) y saltos de línea (`\n`). Es justo lo que necesitaba

yo cuando leía archivos, porque la línea terminaba con `\n` y yo quería que desaparezca.

Sintaxis

```
cadena.rstrip()
```

Ejemplo de código

```
cadena = "\tHola Nahuel\n"
# Elimina el salto de línea al final, pero mantiene la tabulación inicial
print(repr(cadena.rstrip()))
```

En la consola vas a ver:

```
'\tHola Nahuel'
```

Fijate que la tabulación del principio sigue ahí, intacta. Solo se fue el `\n` final. Usé `repr()` para que puedas ver los caracteres especiales que normalmente no se imprimen. En un uso normal, los métodos funcionan igual aunque no lo veas.

Nota: *No te asustes por `repr()`; es solo una lupa para que vos y yo veamos lo invisible. En tu código diario, usá `print` común y corriente.*

Más ejemplos de rstrip()

```
# Eliminar espacios al final
texto1 = "Hola mundo  "
print(f'{texto1.rstrip()}') # 'Hola mundo'

# Eliminar saltos de línea
texto2 = "Python\n"
print(f'{texto2.rstrip()}') # 'Python'

# Eliminar tabulaciones al final
texto3 = "Programacion\t\t"
print(f'{texto3.rstrip()}') # 'Programacion'
```

Ejercicio rápido: Probá vos mismo con una variable que termine en varios espacios y fijate cómo `rstrip()` los limpia sin tocar los que pueda haber en el medio de la frase.

El método lstrip() (Left Strip)

Ahora demos vuelta la torta. `lstrip()` hace exactamente lo opuesto: elimina caracteres **solamente del principio** (izquierda). Imaginate que tenés una sangría o un prefijo molesto al comienzo.

Comportamiento por defecto

Al igual que su gemelo, `lstrip()` sin argumentos barre espacios, tabulaciones y saltos de línea que estén **al inicio** de la cadena.

Sintaxis

```
cadena.lstrip()
```

Ejemplo de código

```
cadena = "\tHola Nahuel\n"  
# Elimina la tabulación inicial, pero mantiene el salto de línea al final  
print(repr(cadena.lstrip()))
```

En consola:

```
'Hola Nahuel\n'
```

La tabulación del principio se esfumó, y el `\n` sigue ahí al fondo, bien cómodo.

Más ejemplos de lstrip()

```
# Eliminar espacios al inicio  
texto1 = "    Hola mundo"  
print(f'"{texto1.lstrip()}"') # 'Hola mundo'  
  
# Eliminar tabulaciones al inicio  
texto2 = "\t\tPython"  
print(f'"{texto2.lstrip()}"') # 'Python'  
  
# Eliminar saltos de línea al inicio  
texto3 = "\n\nProgramacion"  
print(f'"{texto3.lstrip()}"') # 'Programacion'
```

Reflexión: ¿Te acordás de cuando copiabas y pegabas código de internet y te quedaban espacios raros al principio? Con `lstrip()` podés arreglarlo en dos segundos.

Eliminación de caracteres específicos

Hasta acá hablamos de espacios y esos bichos invisibles. Pero también podés pedirle a `rstrip()` y `lstrip()` que eliminen letras o símbolos concretos de los extremos. Les pasás una cadena con todos los caracteres que querés eliminar, y Python los va sacando mientras aparezcan al comienzo o al final, uno por uno, sin importar el orden.

Analogía: Es como decirle a un barrendero: “Limpiame de la vereda todo lo que sea hojas secas, envoltorios de caramelos y piedritas”. El tipo va a ir sacando cada cosita que encuentre hasta que se tope con un objeto que no está en la lista.

Ejemplo con rstrip() y caracteres específicos

```
cadena = "    Hola Nahuel  oo"  
  
# Elimina espacios y la letra 'o' al final  
print(cadena.rstrip(" o"))
```

En consola verás:

```
Hola Nahuel
\end{center}
Explicación: empieza a mirar desde el final. Encuentra 'o', la borra. Otra 'o', la
borra. Luego un espacio, también está en el conjunto ".", así que lo borra. El
siguiente es otro espacio, también lo borra. Cuando llega a la letra 'l' de Nahuel,
como 'l' no está en ".", se detiene. Los espacios del principio se mantienen
intactos.
```

Ejemplo con lstrip() y caracteres específicos

```
cadena = "  Hola Nahuel  "
```

```
# Elimina espacios y la letra 'H' al inicio
print(cadena.lstrip(" H"))
```

El resultado será:

```
ola Nahuel
```

Acá Python arranca por la izquierda: borra espacios, borra la 'H', y cuando se topa con la 'o', que no está en "H", frena en seco. ¡Pum! La 'H' desapareció y ahora tenemos ".ola".

Más ejemplos de eliminación específica

```
# rstrip: eliminar puntos y signos de exclamación al final
texto = "Hola mundo...!!!"
print(texto.rstrip("!!!")) # Hola mundo
```

```
# lstrip: eliminar dígitos del principio
texto = "12345Hola"
print(texto.lstrip("1234567890")) # Hola
```

```
# rstrip: eliminar múltiples caracteres (x y 3)
texto = "Python123xxx"
print(texto.rstrip("x3")) # Python12 (borró xxx pero el 3 también lo saca si aparece)
```

Ojo: En el último caso, `rstrip("x3")` elimina cualquier combinación de 'x' y '3' que encuentre al final. Como el final era "xxx", lo borró por completo; si hubiera un '3' también lo liquidaba.

Comparativa: lstrip() vs rstrip() vs strip()

MÉTODO	DIRECCIÓN DE LIMPIEZA	USO COMÚN
lstrip()	Inicio (Izquierda)	Limpiar sangrías o prefijos innecesarios

rstrip()	Final (Derecha)	Eliminar saltos de línea al leer archivos
strip()	Ambos lados	Limpieza general de entradas de usuario

Ejemplos

prácticos

Para que te des un festín de casos reales, acá van algunos escenarios donde `rstrip()` y `lstrip()` te salvan la vida.

Ejemplo 1: Limpiar entrada de usuario

```
nombre = input("Ingresá tu nombre: ")
# Eliminar espacios accidentales al inicio o al final
nombre_limpio = nombre.strip()
print(f"Hola {nombre_limpio}")

# Si solo queremos eliminar espacios al final (porque el inicio está bien)
nombre_sin_espacios_final = nombre.rstrip()
```

Ejemplo 2: Procesar líneas de un archivo

```
# Simulamos una línea leída de un archivo
linea = "datos importantes\n"
# Eliminamos el salto de línea al final
linea_limpia = linea.rstrip()
print(f'{linea_limpia}') # 'datos importantes'
```

Nota mental: En Windows, a veces las líneas terminan en `\r\n`. `rstrip()` sin argumentos elimina ambos, porque `\r` y `\n` son considerados caracteres de espacio en blanco. Pero si estás en Linux o Mac, típicamente solo tenés `\n`. Por eso siempre usá `rstrip()` para leer archivos y curarte en salud.

Ejemplo 3: Limpiar prefijos numéricos

```
codigo = "0012345Producto"
codigo_limpio = codigo.lstrip("0")
print(codigo_limpio) # 12345Producto
```

Ejemplo 4: Eliminar signos de puntuación al final

```
frase = "Hola mundo!!!"
frase_limpia = frase.rstrip("!")
print(frase_limpia) # Hola mundo
```

Ejemplo 5: Procesar datos de un CSV simulado

```
# Simulamos datos con espacios y saltos de línea
datos = [" Nahuel \n", " 25 \n", " Buenos Aires \n"]

nombre = datos[0].strip()
edad = datos[1].strip()
ciudad = datos[2].strip()

print(f"Nombre: '{nombre}'")
print(f"Edad: '{edad}'")
print(f"Ciudad: '{ciudad}'")
```

Ejemplo 6: Eliminar caracteres específicos de un extremo

```
# Eliminar guiones del final
texto = "Python----"
print(texto.rstrip("-")) # Python

# Eliminar corchetes del inicio
texto = "[[[Hola]]]"
print(texto.lstrip("[")) # Hola]]
```

Consejo

importante

Nota: Estos métodos son fundamentales cuando procesás datos que vienen de fuentes externas (archivos, redes, bases de datos). Los saltos de línea (\n) o los caracteres de retorno de carro (\r) suelen colarse al final y te arruinan comparaciones o te meten líneas en blanco en un archivo de salida. Acostumbrate a limpiar siempre las cadenas que lees del mundo exterior.

Ejercicios

propuestos

Ahora te toca a vos. Agarrá la terminal y ponete a practicar.

1. Usá `rstrip()` para eliminar los espacios al final de la cadena "Hola mundo ". Mostrá el resultado con comillas para verificar.
2. Usá `lstrip()` para eliminar los espacios al inicio de "Python".
3. Usá `rstrip()` para eliminar el salto de línea de "Programacion\n". Usá `repr()` para verlo.
4. Usá `lstrip()` para eliminar las tabulaciones de "\t\tHola".
5. Usá `rstrip()` para eliminar los caracteres '!' y '?' del final de "Hola!?!?".
6. Usá `lstrip()` para eliminar los números del inicio de "12345Nahuel".
7. Usá `rstrip()` para eliminar los puntos al final de "Python...".
8. Usá `lstrip()` para eliminar los corchetes del inicio de "[[[Mensaje]]]".

9. Pedile al usuario su nombre y usá `strip()` para eliminar espacios accidentales. Salúdalo.
10. Procesá la línea `"datos importantes\n"` eliminando solo el salto de línea con `rstrip()`.
11. Eliminá los guiones bajos del final de `"variable_"` con `rstrip()`.
12. Eliminá los asteriscos del inicio de `"***Hola"` con `lstrip()`.
13. Eliminá los espacios y la letra `'a'` del final de `"Hola aa"`. Observá qué queda.
14. Eliminá el prefijo `'www.'` del inicio de `"www.python.com"` (letra por letra).
15. Procesá una lista de nombres que tienen espacios y saltos de línea: `[. 'na \n', "Luis \n", "Maria \n"]`. Dejá los nombres limpios.
16. Eliminá los caracteres `'x'`, `'y'`, `'z'` del final de `"Pythonxyz"`.
17. Eliminá los caracteres `'0'` y `'1'` del inicio de `"001100Python"`.
18. Usá `rstrip()` para eliminar comas y puntos al final de `"Hola mundo,."`.
19. Usá `lstrip()` para eliminar el prefijo `'>'` del inicio de `>Hola"`.

Creá un programa que limpie una entrada de usuario eliminando espacios al inicio y al final, y también elimine saltos de línea si los hubiera. Mostrá el texto limpio.

Soluciones paso a paso

No mires las soluciones hasta que hayas intentado cada ejercicio por tu cuenta. Si te trabás, acá están, explicadas.

Solución 1

```
texto = "Hola mundo  "
print(f'"{texto.rstrip()}"') # 'Hola mundo'
```

Explicación: `rstrip()` elimina los espacios del final. Las comillas en el `print` ayudan a ver que no quedaron espacios ocultos.

Solución 2

```
texto = " Python"
print(f'"{texto.lstrip()}"') # 'Python'
```

Explicación: Similar, pero actúa sobre la izquierda.

Solución 3

```
texto = "Programacion\n"
print(repr(texto.rstrip())) # 'Programacion'
```

Explicación: `rstrip()` elimina el `\n` final. `repr` muestra que ya no está.

Solución 4

```
texto = "\t\tHola"
print(repr(texto.lstrip())) # 'Hola'
```

Explicación: Las tabulaciones del principio desaparecen.

Solución 5

```
texto = "Hola!!!?"
print(texto.rstrip("!?")) # Hola
```

Explicación: Al pasarle `!?"`, elimina cualquier combinación de esos dos caracteres al final.

Solución 6

```
texto = "12345Nahuel"
print(texto.lstrip("1234567890")) # Nahuel
```

Explicación: Todos los dígitos están en el conjunto; se eliminan hasta que aparece una letra.

Solución 7

```
texto = "Python..."
print(texto.rstrip(".")) # Python
```

Explicación: Solo elimina los puntos del final.

Solución 8

```
texto = "[[Mensaje]]]"
print(texto.lstrip("[")) # Mensaje]]
```

Explicación: Borra los corchetes de apertura de la izquierda; los del final quedan porque lstrip no los mira.

Solución 9

```
nombre = input("Ingresa tu nombre: ")
nombre_limpio = nombre.strip()
print(f"Hola {nombre_limpio}")
```

Explicación: strip() limpia ambos extremos; ideal para entradas de usuario.

Solución 10

```
linea = "datos importantes\n"
linea_limpia = linea.rstrip()
print(repr(linea_limpia)) # 'datos importantes'
```

Explicación: Solo el salto de línea final es eliminado; los datos se conservan.

Solución 11

```
texto = "variable_"
print(texto.rstrip("_")) # variable
```

Solución 12

```
texto = "***Hola"
print(texto.lstrip("*")) # Hola
```

Solución 13

```
texto = "Hola aa"
print(texto.rstrip(" a")) # Hol
```

Explicación: Se eliminan espacios y letras 'a' del final. La última 'a' antes de la 'l' se va, y luego el método se detiene.

Solución 14

```
texto = "www.python.com"
print(texto.lstrip("w.")) # python.com
```

Explicación: Se eliminan las 'w' y los puntos del principio. Como '.' está en el conjunto, también se come el punto después de **www**.

Solución 15

```
nombres = [" Ana \n", " Luis \n", " Maria \n"]
nombres_limpios = [nombre.strip() for nombre in nombres]
print(nombres_limpios) # ['Ana', 'Luis', 'Maria']
```

Explicación: Usamos una comprensión de listas para limpiar cada nombre con `strip()`, que elimina espacios y saltos de ambos lados.

Solución 16

```
texto = "Pythonxyz"
print(texto.rstrip("xyz")) # Python
```

Explicación: Cualquier combinación de 'x', 'y', 'z' al final se va.

Solución 17

```
texto = "001100Python"
print(texto.lstrip("01")) # Python
```

Explicación: Todos los ceros y unos del inicio desaparecen, hasta que llega a 'P'.

Solución 18

```
texto = "Hola mundo,.."
print(texto.rstrip(",..")) # Hola mundo
```

Solución 19

```
texto = ">>>Hola"
print(texto.lstrip(">")) # Hola
```

Solución 20

```
entrada = input("Ingresa algo: ")
entrada_limpia = entrada.strip()
print(f"Texto limpio: '{entrada_limpia}'")
```

Explicación: `strip()` ya elimina espacios y saltos de línea, así que con eso alcanza. Si quisieras ser más específico, podrías usar `entrada.rstrip("\n")`, pero `strip()` generalmente es suficiente.

Resumen del capítulo

Dale, repasemos rápido lo importante:

- `rstrip()` elimina caracteres solo del **inicio** (izquierda).
- `rstrip()` elimina caracteres solo del **final** (derecha).
- Por defecto, ambos eliminan espacios en blanco (`' ', \t, \n, \r`).
- Podés pasar una cadena de caracteres para que eliminen esos caracteres puntuales del extremo correspondiente, de a uno, mientras los encuentren.
- Son ideales para limpiar líneas leídas de archivos (`rstrip("\n")`), sacar sangrías molestas (`rstrip()`), o remover signos de puntuación que sobran.
- No modifican la cadena original (las cadenas en Python son inmutables); devuelven una nueva cadena limpia.
- `strip()` hace el trabajo en ambos lados, pero a veces necesitás control fino.

rstrip() y lstrip() son como tijeras quirúrgicas: cortan solo donde les indicás, sin afectar el resto del texto.

Métodos `title()` e `istitle()`

¿Por qué necesitamos formatear títulos?

Cuando empecé a trabajar en un sistema de registro de usuarios, me llegaban nombres de todas las formas posibles: en minúsculas, en mayúsculas, mezclados como `çARLOS` o `"JUAN pEREZ"`. Quería mostrarlos bien prolijos en la pantalla y guardarlos en la base de datos con un formato uniforme. Mi primera idea fue usar `upper()` o `lower()` a lo bruto, pero el resultado quedaba horrible: todo mayúsculas es como estar gritando, y todo minúsculas parece informal. Después probé combinarlos manualmente con un bucle, pero era un quilombo y no andaba bien con palabras con mayúsculas en el medio.

Hasta que un colega me dijo: “Che, ¿no conocés `title()`?”. Y ahí se me abrió la cabeza. Python ya viene preparado con métodos específicos para trabajar con el **formato título**: la primera letra de cada palabra en mayúscula y el resto en minúscula. Además, podés verificar si una cadena ya tiene ese formato con `istitle()`. ¡Un golazo!

Analogía: Pensá en una vidriera de negocio. `title()` es el empleado que acomoda cada cartel para que la inicial de cada palabra quede bien derecha y legible, sin importar cómo se la hayan entregado. `istitle()` es el supervisor que revisa que todo esté en orden antes de abrir al público.

En este capítulo vamos a dominar estos dos métodos para que puedas normalizar cualquier texto sin despeinarte.

```
+-----+
|
|  Formato título: "Carlos Gallardo"
|  (primera letra de cada palabra en mayúscula)
|
|  title() → convierte a formato título
|  istitle() → verifica si ya está en formato
|  título
|
+-----+
```

El método `title()`

`title()` es un método de **transformación**. Toma una cadena y te devuelve una nueva donde cada palabra empieza con mayúscula y el resto de sus letras son minúsculas. No toca la cadena original; en Python los strings son inmutables, así que siempre genera una copia.

Cómo funciona

El algoritmo es simple: recorre la cadena, detecta dónde comienza cada “palabra” (después de un espacio, un número, un guion, etc.), pone esa letra en mayúscula y todas las siguientes hasta el próximo separador en minúscula.

Sintaxis

```
variable.title()
```

Primeros pasos

Mirá este ejemplo que me sacó del apuro aquella vez:

```
nombre = "cARlos gALLardo"  
# Muestro el nombre formateado, pero la variable sigue igual  
print(nombre.title())
```

En la consola vas a ver:

```
Carlos Gallardo
```

Fijate que no importaba que las mayúsculas estuvieran en cualquier lado; `title()` las reacomodó perfecto.

Nota: Recordá que `title()` no modifica la variable original. Si querés conservar el cambio, tenés que reasignarlo: `nombre = nombre.title()`. Esto es clave y suele ser fuente de confusión al principio.

Más ejemplos para agarrarle la mano

```
texto1 = "hola mundo"  
print(texto1.title())    # Hola Mundo  
  
texto2 = "HOLA MUNDO"  
print(texto2.title())    # Hola Mundo  
  
texto3 = "h0lA mUnDo"  
print(texto3.title())    # Hola Mundo  
  
texto4 = "python es genial"  
print(texto4.title())    # Python Es Genial
```

Ejercicio rápido: Probá vos con tu propio nombre y apellido, y también con el título de tu libro favorito. ¿Qué pasa si ponés todo en minúsculas? ¿Y si alternás mayúsculas y minúsculas? Anotá los resultados.

¿Qué pasa con números y caracteres especiales?

El separador de palabras no es solo el espacio. Cualquier caracter que no sea una letra puede actuar como “frontera” para que `title()` ponga mayúscula después de él. Esto incluye números, guiones y hasta signos de puntuación.

```
# Después de un número
print("hola123mundo".title())    # Hola123Mundo

# Con un guion
print("hola-mundo".title())      # Hola-Mundo

# Con signos al final (no afecta)
print("hola mundo!".title())     # Hola Mundo!
```

Reflexión: Este comportamiento es útil si tenés códigos como `id123cliente` y querés que quede `Id123Cliente`, pero a veces puede no ser lo que esperabas. Siempre probá con datos reales.

El método istitle()

Después de normalizar un nombre, ¿cómo sabés si realmente quedó en formato título? Para eso existe `istitle()`, un método de **validación** que devuelve `True` o `False`.

Funcionamiento

`istitle()` devuelve `True` si **toda** la cadena cumple con la regla del formato título: cada palabra debe empezar con una letra mayúscula y las letras siguientes deben ser minúsculas. Si al menos una palabra no respeta eso, devuelve `False`.

Sintaxis

```
variable.istitle()
```

Ejemplo de verificación

```
nombre1 = "Carlos Gallardo"
print(nombre1.istitle())    # True

nombre2 = "carlos Gallardo"
print(nombre2.istitle())    # False, primera letra minuscula

nombre3 = "Carlos gallardo"
print(nombre3.istitle())    # False, segunda palabra con minuscula
```

Salida en la terminal:

```
True
False
False
```

Analogía: `istitle()` es como el inspector de vestimenta en una fiesta elegante: mira que cada palabra tenga el saco puesto (mayúscula) y que el resto de la ropa no llame la atención (minúsculas). Si alguna palabra va en pantuflas, lo rechaza.

Casos típicos de True y False

```
# Estos dan True
print("Hola Mundo".istitle())      # True
print("Python Es Genial".istitle()) # True
print("Mi Nombre Es Juan".istitle()) # True

# Estos dan False
print("hola Mundo".istitle())      # False (primera palabra en baja)
print("Hola mundo".istitle())      # False (segunda palabra en baja)
print("HOLA MUNDO".istitle())      # False (todo en mayuscula)
print("Hola Mundo!".istitle())     # True (el signo no participa)
```

Ojo: Si una palabra está completamente en mayúsculas, `istitle()` dice **False** porque las letras internas no son minúsculas. El formato título exige que solo la inicial sea mayúscula.

El cambio permanente: reasignación

Te repito porque es importantísimo: los strings en Python no se pueden modificar una vez creados. Si hacés `nombre.title()` sin capturar el resultado, la variable sigue sucia. Para limpiarla de verdad, tenés que reasignarla.

```
nombre_usuario = input("Escribe tu nombre: ")

# Si el formato no es título, lo corregimos y guardamos
if not nombre_usuario.istitle():
    print("Corrigiendo formato...")
    nombre_usuario = nombre_usuario.title()    # ¡Acá reasignamos!

print(f"Nombre almacenado: {nombre_usuario}")
```

↙ Dos ejecuciones de ejemplo:

```
Ejecucion 1:
Escribe tu nombre: carlos
Corrigiendo formato...
Nombre almacenado: Carlos
```

```
Ejecucion 2:
Escribe tu nombre: Carlos Gallardo
Nombre almacenado: Carlos Gallardo
```

Reflexión: Si olvidás reasignar, verás que el `print` sigue mostrando el original sucio, y te vas a rascar la cabeza. ¡A todos nos pasó!

Ejemplos prácticos

Ahora bajemos a tierra con algunos casos de uso real.

Ejemplo 1: Normalizar nombres de clientes

Tenés una lista de nombres que vienen de distintos formularios, todos con formatos locos. Con `title()` los dejás impecables.

```
clientes = ["juan perez", "MARIA GOMEZ", "carlos lopez", "Ana Martinez"]

print("Nombres originales:")
for cliente in clientes:
    print(f" {cliente}")

print("\nNombres normalizados:")
for cliente in clientes:
    print(f" {cliente.title()}")
```

Ejemplo 2: Validación antes de guardar en base de datos

Una función que recibe un nombre, lo normaliza solo si hace falta, y devuelve la versión lista para almacenar.

```
def normalizar_nombre(nombre):
    if not nombre.istitle():
        return nombre.title()
    return nombre

# Pruebas
nombres = ["juan", "MARIA", "Carlos", "ana", "PEDRO"]
for nombre in nombres:
    nombre_ok = normalizar_nombre(nombre)
    print(f"{nombre} -> {nombre_ok}")
```

Ejemplo 3: Limpiar títulos de libros

Los títulos de libros a veces vienen en mayúsculas fijas o minúsculas raras. `title()` los deja como se debe.

```
titulos = [
    "el señor de los anillos",
    "CIEN AÑOS DE SOLEDAD",
    "La casa de los espíritus",
    "1984 george orwell"
]

print("Títulos normalizados:")
for titulo in titulos:
    print(f" {titulo.title()}")
```


Ejemplo 4: Sistema de registro con verificación final

Pedimos nombre y apellido, normalizamos, y al final comprobamos con `istitle()` que todo esté en orden.

```
print("=== SISTEMA DE REGISTRO ===")
nombre = input("Nombre: ")
apellido = input("Apellido: ")

# Normalizar formato
nombre = nombre.title()
apellido = apellido.title()

nombre_completo = f"{nombre} {apellido}"
print(f"\nUsuario registrado: {nombre_completo}")
print(f"¿Formato correcto? {nombre_completo.istitle()}")
```

Resumen comparativo

MÉTODO	TIPO	FUNCIÓN
<code>title()</code>	Transformación	Convierte a formato título (primera letra mayúscula)
<code>istitle()</code>	Validación	Verifica si ya está en formato título

Consejo importante

Nota: Estos métodos son tus mejores aliados para limpiar datos que vienen de formularios web, archivos CSV o cualquier fuente externa. Normalizar con `title()` te asegura una estética profesional y evita que un nombre como *¿CARLOS* grite en tus reportes. Y `istitle()` te permite validar antes de enviar a producción, ahorrándote dolores de cabeza posteriores.

Ejercicios propuestos

Ahora es tu turno. Agarrando el teclado y poné en práctica cada ejercicio. No te saltees ninguno, porque la práctica es la que fija los conceptos.

- Usá `title()` para convertir "hola mundo" a formato título.
- Usá `title()` para convertir "PYTHON ES GENIAL" a formato título.
- Usá `istitle()` para verificar si "Hola Mundo" está en formato título.
- Usá `istitle()` para verificar si "hola mundo" está en formato título.

5. Convertí "jUaN pErEz" a formato título usando `title()`.
6. Verifiqué si "Python Es Genial" está en formato título con `istitle()`.
7. Convertí `.el señor de los anillos` a formato título.
8. Pedile al usuario su nombre y normalízalo con `title()`. Mostrá el resultado.
9. Pedile al usuario una frase y mostrá si ya está en formato título usando `istitle()`.
10. Creá una lista de nombres con formatos inconsistentes y normalizalos todos con `title()`.
11. Convertí "1984 ORWELL" a formato título.
12. Verifiqué si "Hola mundo" (con segunda palabra en minúscula) es formato título.
13. Convertí "hola-mundo" a formato título.
14. Pedile al usuario su nombre completo y apellido, normalizá cada uno con `title()` y mostrá el nombre completo.
15. Creá una función que reciba una cadena y devuelva `True` si está en formato título, `False` si no.
16. Convertí `.ESTA ES UNA PRUEBA` a formato título.
17. Procesá la lista `[.ªna", "LUIS", çarlos", "MARIA"]` y mostrá cada nombre normalizado.
18. Verifiqué si "La Casa De Los Espíritus" está en formato título.
19. Convertí "mi nombre es nahuel" a formato título.
20. Creá un programa que pida al usuario un título de libro, lo normalice a formato título, muestre el resultado y verifique si quedó correcto.

Soluciones paso a paso

Antes de leer las soluciones, intentá cada ejercicio por tu cuenta. Recién después compará.

Solución 1

```
texto = "hola mundo"
print(texto.title()) # Hola Mundo
```

Explicación: title() pone en mayúscula la h y la m, y el resto en minúscula.

Solución 2

```
texto = "PYTHON ES GENIAL"
print(texto.title()) # Python Es Genial
```

Explicación: Todas las mayúsculas bajan, salvo las iniciales.

Solución 3

```
texto = "Hola Mundo"
print(texto.istitle()) # True
```

Explicación: Ambas palabras cumplen el formato título.

Solución 4

```
texto = "hola mundo"
print(texto.istitle()) # False
```

Explicación: La primera letra de cada palabra es minúscula.

Solución 5

```
nombre = "jUaN pErEz"
print(nombre.title()) # Juan Perez
```

Explicación: title() normaliza sin importar el desorden.

Solución 6

```
texto = "Python Es Genial"
print(texto.istitle()) # True
```

Explicación: Cada palabra comienza con mayúscula y sigue con minúscula.

Solución 7

```
texto = "el señor de los anillos"
print(texto.title()) # El Señor De Los Anillos
```

Explicación: Todas las iniciales suben, incluyendo artículos y preposiciones.

Solución 8

```
nombre = input("Ingrese su nombre: ")
nombre = nombre.title()
print(f"Nombre normalizado: {nombre}")
```

Explicación: Reasignamos el resultado para que el cambio se mantenga.

Solución 9

```
frase = input("Ingrese una frase: ")
if frase.istitle():
    print("La frase ya esta en formato titulo")
else:
    print("La frase NO esta en formato titulo")
```

Explicación: El if utiliza el booleano devuelto por istitle().

Solución 10

```
nombres = ["juan", "MARIA", "carlos", "ANA", "PEDRO"]
nombres_normalizados = [nombre.title() for nombre in nombres]
print(nombres_normalizados)
```

Explicación: Comprensión de listas para aplicar title() a cada elemento.

Solución 11

```
texto = "1984 ORWELL"
print(texto.title()) # 1984 Orwell
```

Explicación: Después del número, la O pasa a mayúscula; el resto baja.

Solución 12

```
texto = "Hola mundo"
print(texto.istitle()) # False
```

Explicación: La segunda palabra empieza con minúscula.

Solución 13

```
texto = "hola-mundo"
print(texto.title()) # Hola-Mundo
```

Explicación: El guion actúa como separador, cada fragmento se capitaliza.

Solución 14

```
nombre = input("Nombre: ")
apellido = input("Apellido: ")
nombre = nombre.title()
apellido = apellido.title()
print(f"Nombre completo: {nombre} {apellido}")
```

Explicación: Cada parte se normaliza por separado antes de armar el saludo.

Solución 15

```
def es_formato_titulo(texto):
    return texto.istitle()

# Prueba
print(es_formato_titulo("Hola Mundo")) # True
print(es_formato_titulo("hola mundo")) # False
```

Explicación: La función simplemente envuelve a `istitle()`, útil para claridad.

Solución 16

```
texto = "ESTA ES UNA PRUEBA"
print(texto.title()) # Esta Es Una Prueba
```

Explicación: De mayúsculas a formato título en un solo paso.

Solución 17

```
nombres = ["ana", "LUIS", "carlos", "MARIA"]
for nombre in nombres:
    print(nombre.title())
```

Explicación: Iteramos y normalizamos cada uno.

Solución 18

```
texto = "La Casa De Los Espiritus"
print(texto.istitle()) # True
```

Explicación: Cada palabra cumple la regla.

Solución 19

```
texto = "mi nombre es nahuel"
print(texto.title()) # Mi Nombre Es Nahuel
```

Explicación: Todas las iniciales suben.

Solución 20

```
titulo = input("Ingrese el titulo del libro: ")
titulo_normalizado = titulo.title()
print(f"Titulo original: {titulo}")
print(f"Titulo normalizado: {titulo_normalizado}")
print(f"¿Formato correcto? {titulo_normalizado.istitle()}")
```

Explicación: Normalizamos y verificamos en un mismo programa.

Resumen del capítulo

Dale, repasemos rápido lo importante:

- `title()`: transforma una cadena a formato título (inicial mayúscula de cada palabra, resto minúscula). No modifica la original; hay que reasignar para conservar el cambio.
- `istitle()`: verifica si la cadena ya cumple el formato título; devuelve `True` o `False`.
- Ambos métodos consideran “palabras” separadas por espacios, números, guiones y otros no alfabéticos.
- Son ideales para normalizar nombres, títulos, direcciones y cualquier dato que requiera una presentación uniforme.
- Combinados con estructuras de control y listas, podés automatizar la limpieza de datos de forma muy elegante.

`title()` e `istitle()` son como el corrector de estilo de tus textos: aseguran que los nombres y títulos luzcan profesionales y uniformes, sin que vos muevas un dedo de más.

Métodos de Strings: Mayúsculas, Minúsculas y Validación

Introducción a los Métodos

Cuando empecé a programar, uno de los primeros errores ridículos que tuve fue comparar una contraseña ingresada por el usuario contra la guardada en una lista, y nunca coincidía. Después de un rato largo, me di cuenta de que yo escribía "Python" y el programa esperaba "PYTHON" o viceversa. ¡El problema era la diferencia entre mayúsculas y minúsculas! En Linux y Mac esto es particularmente estricto; en Windows a veces los paths no distinguen, pero para strings de texto, Python distingue siempre. Así que me propuse dominar los métodos que permiten transformar y preguntar sobre el "case" de las letras.

En este capítulo vamos a ver cuatro métodos esenciales:

- **Transformadores:** cambian el texto (crean una nueva versión).
- **Validadores:** verifican el estado y devuelven un valor booleano (True o False).

Como decía un colega en la facultad (UBA Exactas): "Los validadores son como preguntas que le hacés al texto. Los transformadores son como órdenes que le das para que se convierta en otra cosa."

+-----+ VALIDADORES (preguntan, NO modifican) - islower() → ¿está todo en minúsculas? - isupper() → ¿está todo en mayúsculas? TRANSFORMADORES (modifican, hay que reasignar) - lower() → convierte a minúsculas - upper() → convierte a mayúsculas +-----+	
--	--

Analogía: Imaginá que tenés una prenda de ropa con etiquetas que dicen cómo se debe lavar. `islower()` y `isupper()` son las etiquetas que te dicen en qué estado está la tela (todo en minúsculas o todo en mayúsculas). `lower()` y `upper()` son el lavarropas que la transforma a un solo programa de lavado.

Métodos islower() y lower()

islower() – El validador de minúsculas

El método `islower()` verifica si **todas las letras** de la cadena están en minúsculas. Si hay una sola mayúscula, devuelve `False`.

- **Sintaxis:** `nombre_variable.islower()`
- **Devuelve:** `True` (si todo es minúscula) o `False`
- **Importante:** Los números, espacios y símbolos se ignoran, no afectan la verificación.

```
# Ejemplos de islower()
texto1 = "la geekipedia"
texto2 = "La Geekipedia"
texto3 = "la geekipedia 123"
```

```
print(f" '{texto1}' es todo minúsculas? {texto1.islower()}") # True
print(f" '{texto2}' es todo minúsculas? {texto2.islower()}") # False (L y G mayúsculas)
print(f" '{texto3}' es todo minúsculas? {texto3.islower()}") # True (los números no afectan)
```

En la consola verías:

```
'la geekipedia' es todo minúsculas? True
'La Geekipedia' es todo minúsculas? False
'la geekipedia 123' es todo minúsculas? True
```

Reflexión: `islower()` es como preguntarle al texto: “¿estás hablando bajito?”. Si todas las letras son minúsculas, te dice `True`; si alguna levanta la voz, `False`.

lower() – El transformador a minúsculas

El método `lower()` toma la cadena y la pasa toda a minúsculas. Te devuelve una copia nueva, sin modificar la original.

- **Sintaxis:** `nombre_variable.lower()`
- **Devuelve:** Un nuevo string con todo en minúsculas.
- **NO modifica** la variable original. Si querés que cambie, tenés que reasignar.

```
texto = "La Geekipedia"
print(f"Original: {texto}")
print(f"lower(): {texto.lower()}") # "la geekipedia"
print(f"Original sigue igual: {texto}") # No se modificó
```

```
Original: La Geekipedia
lower(): la geekipedia
Original sigue igual: La Geekipedia
```

Ejercicio rápido: Creá una variable con tu nombre en mayúsculas, aplicá `lower()` y fijate que la variable no cambia si no reasignás. Después reasignala y compará los resultados.

Métodos `isupper()` y `upper()`

`isupper()` – El validador de mayúsculas

`isupper()` funciona exactamente al revés: pregunta si **todas las letras** son mayúsculas.

```
# Ejemplos de isupper()
texto1 = "LA GEEKIPEDIA"
texto2 = "La Geekipedia"
texto3 = "LA GEEKIPEDIA 123"

print(f"{'texto1'} es todo mayúsculas? {texto1.isupper()}") # True
print(f"{'texto2'} es todo mayúsculas? {texto2.isupper()}") # False
print(f"{'texto3'} es todo mayúsculas? {texto3.isupper()}") # True (números no afectan)

'LA GEEKIPEDIA' es todo mayúsculas? True
'La Geekipedia' es todo mayúsculas? False
'LA GEEKIPEDIA 123' es todo mayúsculas? True
```

`upper()` – El transformador a mayúsculas

Convierte toda la cadena a letras mayúsculas.

```
texto = "La Geekipedia"
print(f"Original: {texto}")
print(f"upper(): {texto.upper()}") # "LA GEEKIPEDIA"
print(f"Original sigue igual: {texto}")

Original: La Geekipedia
upper(): LA GEEKIPEDIA
Original sigue igual: La Geekipedia
```

Simulación de ejecución y persistencia de datos

Esta distinción es fundamental y me hizo patear el teclado más de una vez. Tenés que distinguir entre **mostrar** un cambio y **hacer** el cambio.

Reasignación (cambio permanente)

Si hacés `texto = texto.lower()`, la variable ahora almacena la nueva versión. Perdés la original para siempre.

```
texto = "La Geekipedia"
print(f"Antes: {texto}")

# REASIGNACIÓN: el cambio es permanente
texto = texto.lower()
print(f"Después de reasignar: {texto}")

Antes: La Geekipedia
Después de reasignar: la geekipedia
```

Solo visualización (cambio temporal)

Si solo hacés `print(variable.upper())`, ves mayúsculas en la pantalla, pero la variable sigue intacta.

```
texto = "La Geekipedia"
print(f"Mostrando en mayúsculas: {texto.upper()}")
print(f"La variable original sigue igual: {texto}")
```

```
Mostrando en mayúsculas: LA GEEKIPEDIA
La variable original sigue igual: La Geekipedia
```

Nota: *Regla de oro:*

- *Reasignás si querés que el cambio sea permanente.*
- *Solo visualizás si querés mostrar algo distinto sin perder el original.*

Programa completo de demostración

Vamos a juntar todo lo anterior en un programa que pida un texto al usuario y muestre validaciones y transformaciones, con f-strings y saltos de línea.

```
print("\n=== VALIDACIÓN Y TRANSFORMACIÓN DE TEXTO ===\n")

# Solicitamos un texto al usuario
texto = input("Introducí un texto: ")

print("\n--- RESULTADOS ---")

# Validaciones (preguntan, no modifican)
print(f"¿Está todo en minúsculas? {texto.islower()}")
print(f"¿Está todo en mayúsculas? {texto.isupper()}")

# Transformaciones (solo visualización, no se guardan)
print(f"Texto en minúsculas: {texto.lower()}")
print(f"Texto en mayúsculas: {texto.upper()}")

# Estado final de la variable (no cambió)
print(f"\nEl texto original se mantiene: {texto}")

# Con reasignación (cambio permanente)
texto = texto.lower()
print(f"Si reasignamos: {texto}")
```

Ejecutá este programita con distintos valores y vas a ver clarito el comportamiento. Por ejemplo:

=== VALIDACIÓN Y TRANSFORMACIÓN DE TEXTO ===

Introducí un texto: La Geekipedia

--- RESULTADOS ---

```
¿Está todo en minúsculas? False
¿Está todo en mayúsculas? False
Texto en minúsculas: la geekipedia
Texto en mayúsculas: LA GEEKIPEDIA
```

El texto original se mantiene: La Geekipedia
Si reasignamos: la geekipedia

Ejemplo práctico: Normalización de entrada de usuario

Un uso real y cotidiano es cuando pedís una respuesta al usuario y no querés que importe si escribe en mayúsculas o minúsculas. Por ejemplo, en un sistema de encuesta.

```
print("\n=== SISTEMA DE ENCUESTA ===")
respuesta = input("¿Te gusta programar en Python? (si/no): ")

# Normalizamos la respuesta a minúsculas para comparar fácilmente
respuesta_normalizada = respuesta.lower()

if respuesta_normalizada == "si" or respuesta_normalizada == "sí":
    print("¡Excelente! A mí también me encanta")
elif respuesta_normalizada == "no":
    print("¡Qué lástima! Quizás después cambies de opinión")
else:
    print("Respuesta no válida. Respondé 'si' o 'no'")
```

Ejecuciones:

```
¿Te gusta programar en Python? (si/no): SI
¡Excelente! A mí también me encanta

¿Te gusta programar en Python? (si/no): No
¡Qué lástima! Quizás después cambies de opinión

¿Te gusta programar en Python? (si/no): si
¡Excelente! A mí también me encanta
```

Reflexión: Gracias a `lower()`, no importa cómo el usuario escriba la respuesta, siempre la llevamos a un formato común y podemos comparar tranquilamente.

Ejemplo Integrador: Sistema de Login sin distinción de mayúsculas

Ahora un caso más completo, donde usamos todo lo aprendido para armar un pequeño sistema de login que no le da importancia a las mayúsculas.

```
# Base de datos de usuarios (guardamos todo en minúsculas)
usuarios = {
    "nahuel": "exactas2024",
    "ana": "password123",
    "luis": "qwerty"
}

print("\n=== SISTEMA DE LOGIN ===")
usuario = input("Usuario: ")
contrasena = input("Contraseña: ")

# Normalizamos a minúsculas para comparar
usuario_normalizado = usuario.lower()

if usuario_normalizado in usuarios:
    if contrasena == usuarios[usuario_normalizado]:
        print("Bienvenido al sistema")
    else:
        print("Contraseña incorrecta")
else:
    print("Usuario no registrado")
```

Fíjate que no importa si el usuario escribe “NAHUEL” o “nahuel”, `lower()` lo convierte y la búsqueda funciona.

Ejercicios propuestos

Ahora es momento de que pongas en práctica todo esto. Son 20 ejercicios; hazlos de a uno y no mires las soluciones hasta haberlo intentado.

1. Creá un string con mayúsculas y minúsculas mezcladas. Usá `islower()` e `isupper()` para verificar su estado.
2. Pedile al usuario un texto y mostralo completamente en minúsculas (guardá el cambio).
3. Pedile al usuario un texto y mostralo completamente en mayúsculas (solo visual, sin modificar la original).
4. Hacé un programa que pregunte al usuario “¿Desea continuar?” y acepte “sí”, “SI”, “sÍ”, “SÍ”. Normalizá la respuesta a minúsculas para comparar.
5. Pedí al usuario dos palabras y comparalas sin importar mayúsculas/minúsculas usando `lower()`. Decí si son iguales o no.
6. Creá una lista de 5 nombres en minúsculas. Pedí al usuario un nombre en cualquier formato y decí si está en la lista (normalizando la entrada).
7. Pedí al usuario una frase y mostrá cuántas letras mayúsculas tiene (pista: usá `isupper()` letra por letra).

8. Creá un programa que valide una contraseña: debe tener al menos una mayúscula y una minúscula.
9. Pedí al usuario un texto y mostralo en mayúsculas si está en minúsculas, o en minúsculas si está en mayúsculas (invertí el caso según su estado).
10. Normalizá una lista de nombres desordenados a formato “Primera letra en mayúscula, el resto minúscula” (usá índices de string, no `title()`).
11. Pedí un nombre de usuario y convertilo siempre a minúsculas, guardalo y mostrá el original y el normalizado.
12. Pedí una frase y mostrala con la primera letra de cada palabra en mayúscula y el resto en minúscula, pero sin usar `title()`. Usá `lower()` y `upper()` combinados con índices.
13. Pedí una palabra y verificá si es completamente mayúscula o completamente minúscula. Mostrá un mensaje que diga “Es mayúscula”, “Es minúscula” o “Es mixta”.
14. Pedí una lista de palabras separadas por espacios y mostrala con cada palabra en minúsculas.
15. Pedí una oración y contá cuántas palabras empiezan con mayúscula (pista: `istitle()` no sirve; necesitás recorrer y verificar la primera letra).
16. Escribí una función `contiene_mayus_y_minus(texto)` que devuelva `True` si la cadena contiene al menos una mayúscula y una minúscula.
17. Pedí un texto y reemplazá todos los espacios por guiones bajos (`_`), y después pasá todo a mayúsculas. Mostrá el resultado.
18. Creá un menú simple: “Elegí opción: A) Mayúsculas, B) Minúsculas, C) Salir”. Luego pedí un texto y mostralo transformado según la opción elegida.
19. Pedí un nombre de archivo con extensión (ej: “MiArchivo.TXT”). Convertí solo la parte del nombre (sin extensión) a minúsculas, y mantené la extensión intacta. Ayuda: `split(".")`.
20. Pedí una frase y convertí cada letra: las mayúsculas a minúsculas y viceversa, sin usar `swapcase()`. Recorré el string y usé `isupper()` e `islower()`.

Soluciones paso a paso

A continuación las soluciones con explicaciones. Leelas solo después de haber peleado con cada ejercicio.

Solución 1

```
texto = "Hola Mundo"
print(f"¿Está todo en minúsculas? {texto.islower()}")
print(f"¿Está todo en mayúsculas? {texto.isupper()}")
```

Explicación: `islower()` devuelve `False` porque hay mayúsculas; `isupper()` también `False` porque hay minúsculas.

Solución 2

```
texto = input("Ingresa un texto: ")
texto = texto.lower()
print(f"Texto en minúsculas: {texto}")
```

Explicación: Reasignamos para que el cambio quede permanente.

Solución 3

```
texto = input("Ingresa un texto: ")
print(f"Texto en mayúsculas (solo visual): {texto.upper()}")
print(f"Texto original: {texto}")
```

Explicación: Solo imprimimos el resultado de `upper()`, sin modificar la variable.

Solución 4

```
respuesta = input("¿Desea continuar? (si/no): ")
if respuesta.lower() == "si" or respuesta.lower() == "sí":
    print("Continuando...")
else:
    print("Saliendo...")
```

Explicación: Normalizamos con `lower()` antes de comparar, así aceptamos todas las variantes.

Solución 5

```
p1 = input("Primera palabra: ")
p2 = input("Segunda palabra: ")
if p1.lower() == p2.lower():
    print("Son iguales (sin importar mayúsculas)")
else:
    print("Son diferentes")
```

Explicación: Comparamos las versiones en minúsculas para que sea insensible al case.

Solución 6

```
nombres = ["ana", "luis", "pedro", "marta", "juan"]
buscar = input("Nombre a buscar: ")
if buscar.lower() in nombres:
    print(f"{buscar} está en la lista")
else:
    print(f"{buscar} no está en la lista")
```

Explicación: Convertimos la entrada a minúscula y la buscamos en la lista que ya está en minúsculas.

Solución 7

```
frase = input("Ingresa una frase: ")
contador = 0
for letra in frase:
    if letra.isupper():
        contador += 1
print(f"La frase tiene {contador} letras mayúsculas")
```

Explicación: Recorremos carácter por carácter y usamos `isupper()` para contar.

Solución 8

```
clave = input("Ingresa la contraseña: ")
tiene_mayuscula = False
tiene_minuscula = False
for letra in clave:
    if letra.isupper():
        tiene_mayuscula = True
    if letra.islower():
        tiene_minuscula = True
if tiene_mayuscula and tiene_minuscula:
    print("Contraseña válida")
else:
    print("Contraseña inválida: debe tener mayúsculas y minúsculas")
```

Explicación: Verificamos la presencia de al menos una mayúscula y una minúscula con un bucle.

Solución 9

```
texto = input("Ingresa un texto: ")
if texto.islower():
    print(f"En mayúsculas: {texto.upper()}")
elif texto.isupper():
    print(f"En minúsculas: {texto.lower()}")
else:
    print("El texto tiene mezcla de mayúsculas y minúsculas")
```

Explicación: Usamos `islower()` e `isupper()` para decidir cómo transformar.

Solución 10

```
nombres = ["ANA", "luis", "PEDRO", "marta", "JUAN"]
nombres_normalizados = []
for nombre in nombres:
    nombre_normalizado = nombre[0].upper() + nombre[1:].lower()
    nombres_normalizados.append(nombre_normalizado)
print("Nombres originales:", nombres)
print("Nombres normalizados:", nombres_normalizados)
```

Explicación: Combinamos `upper()` para la primera letra y `lower()` para el resto.

Solución 11

```
usuario = input("Elegí un nombre de usuario: ")
usuario_guardado = usuario.lower()
print(f"Original: {usuario}")
print(f"Guardado como: {usuario_guardado}")
```

Explicación: Guardamos la versión en minúsculas para evitar duplicados por case.

Solución 12

```
frase = input("Frase: ")
palabras = frase.split()
frase_normalizada = []
for palabra in palabras:
    if palabra: # por si hay espacios extra
        nueva = palabra[0].upper() + palabra[1:].lower()
        frase_normalizada.append(nueva)
    else:
        frase_normalizada.append("")
print("Frase normalizada:", " ".join(frase_normalizada))
```

Explicación: Separamos por espacios, capitalizamos cada palabra manualmente.

Solución 13

```
palabra = input("Ingresá una palabra: ")
if palabra.isupper():
    print("Es completamente mayúscula")
elif palabra.islower():
    print("Es completamente minúscula")
else:
    print("Es mixta")
```

Explicación: `isupper()` e `islower()` devuelven `False` si hay mezcla.

Solución 14

```
entrada = input("Ingresa palabras separadas por espacios: ")
palabras = entrada.split()
for palabra in palabras:
    print(palabra.lower(), end=" ")
```

Explicación: Recorremos la lista y aplicamos `lower()` a cada elemento.

Solución 15

```
oracion = input("Ingresa una oración: ")
contador = 0
palabras = oracion.split()
for palabra in palabras:
    if palabra and palabra[0].isupper():
        contador += 1
print(f"Cantidad de palabras que empiezan con mayúscula: {contador}")
```

Explicación: Verificamos la primera letra de cada palabra con `isupper()`.

Solución 16

```
def contiene_mayus_y_minus(texto):
    tiene_mayus = False
    tiene_minus = False
    for letra in texto:
        if letra.isupper():
            tiene_mayus = True
        if letra.islower():
            tiene_minus = True
    return tiene_mayus and tiene_minus

# Pruebas
print(contiene_mayus_y_minus("Python"))    # True
print(contiene_mayus_y_minus("python"))    # False
print(contiene_mayus_y_minus("PYTHON"))    # False
```

Explicación: La función recorre y verifica ambas condiciones.

Solución 17

```
texto = input("Ingresa un texto: ")
texto_procesado = texto.replace(" ", "_").upper()
print(texto_procesado)
```

Explicación: Primero reemplazamos espacios, luego pasamos a mayúsculas.

Solución 18

```
print("Menú:")
print("A) Convertir a mayúsculas")
print("B) Convertir a minúsculas")
print("C) Salir")
opcion = input("Elegí una opción: ").upper()
if opcion in ("A", "B", "C"):
    texto = input("Ingresá un texto: ")
    if opcion == "A":
        print(texto.upper())
    elif opcion == "B":
        print(texto.lower())
    else:
        print("Chau")
else:
    print("Opción no válida")
```

Explicación: Normalizamos la opción con `upper()` y aplicamos el método correspondiente.

Solución 19

```
archivo = input("Nombre del archivo (con extensión): ")
partes = archivo.rsplit(".", 1) # dividimos solo en el último punto
if len(partes) == 2:
    nombre = partes[0].lower()
    extension = partes[1]
    nuevo_archivo = nombre + "." + extension
else:
    nuevo_archivo = archivo.lower() # sin extensión, todo minúsculas
print("Archivo normalizado:", nuevo_archivo)
```

Explicación: Separamos nombre y extensión, convertimos solo el nombre a minúsculas.

Solución 20

```
frase = input("Ingresá una frase: ")
resultado = ""
for letra in frase:
    if letra.isupper():
        resultado += letra.lower()
    elif letra.islower():
        resultado += letra.upper()
    else:
        resultado += letra # espacios, números, etc.
print("Frase invertida:", resultado)
```

Explicación: Recorremos cada letra, la transformamos según su estado usando los validadores.

Resumen del capítulo

Dale, repasemos lo importante:

- **Validadores:**

- `islower()` devuelve `True` si **todas las letras** son minúsculas.
- `isupper()` devuelve `True` si **todas las letras** son mayúsculas.

- **Transformadores:**

- `lower()` convierte todo a minúsculas.
- `upper()` convierte todo a mayúsculas.

- Los strings son **inmutables**; estos métodos no modifican la original, devuelven una nueva.
- Para que el cambio sea permanente, hay que reasignar: `texto = texto.lower()`.
- Son herramientas imprescindibles para normalizar entradas de usuario, comparar sin distinguir mayúsculas/minúsculas, y validar formatos.

lower() y upper() son como un traductor instantáneo: convierten todo a un mismo idioma para que las comparaciones sean justas. islower() e isupper() son las preguntas que le hacés al texto: “¿estás hablando bajito o gritando?”

Método `swapcase()` - Invertir Mayúsculas y Minúsculas

¿Qué es el método `swapcase()`?

Cuando estaba empezando a programar en Python, me encontré con un desafío que parecía una tontería pero me tuvo un buen rato pensando. Necesitaba tomar un texto que un usuario había escrito y convertirlo en una versión con las mayúsculas y minúsculas intercambiadas, como un espejo. Por ejemplo, si el usuario ponía "Hola Mundo", yo quería mostrar "hOLA mUNDO". Al principio, me puse a recorrer el string carácter por carácter, con un `if` largo que preguntaba si era mayúscula o minúscula. Funcionaba, pero era un código feo y largo. Hasta que un día, revisando la documentación de Python, descubrí el método `swapcase()`. ¡Me sentí un tonto por no haberlo buscado antes! En una sola línea hacía justo lo que yo necesitaba.

Analogía: `swapcase()` es como un espejo que invierte la ropa: lo que estaba dado vuelta ahora se ve al derecho, y viceversa. Las letras mayúsculas pasan a minúsculas, y las minúsculas a mayúsculas, todo automáticamente.

El método `swapcase()` se usa para **invertir el estado** de todas las letras de una cadena. O sea:

- Las **mayúsculas** se convierten en **minúsculas**.
- Las **minúsculas** se convierten en **mayúsculas**.
- Todo lo demás (números, espacios, signos) queda igual.

Es como si al texto le dieras un sacudón y cada letra se transformara en su opuesta.

```
+-----+
|
| texto = "Hola Mundo"
|
| texto.swapcase()
|
| Resultado: "hOLA mUNDO"
|
| H -> h (mayuscula a minuscula)
| o -> O (minuscula a mayuscula)
| l -> L (minuscula a mayuscula)
| a -> A (minuscula a mayuscula)
| espacio -> espacio (no cambia)
| M -> m (mayuscula a minuscula)
|
```

```
| u -> U (minuscula a mayuscula) |
| n -> N (minuscula a mayuscula) |
| d -> D (minuscula a mayuscula) |
| o -> O (minuscula a mayuscula) |
|                                     |
+-----+-----+-----+-----+
```

Sintaxis

La sintaxis es re sencilla, igual que `lower()` y `upper()`:

`nombre_variable.swapcase()`

- No lleva argumentos entre los paréntesis.
- Devuelve un **nuevo string** con los casos invertidos.
- **NO modifica** la variable original (como todo método de string en Python).

Si querés que el cambio se guarde, tenés que reasignar: `texto = texto.swapcase()`. Esto ya lo vimos antes, pero nunca está de más repetirlo.

Funcionamiento y Comportamiento

`swapcase()` analiza **cada carácter** de la cadena. Si es una letra (a-z o A-Z), la transforma a su versión opuesta. Si no es una letra (números, espacios, signos de puntuación), la deja exactamente igual. Esto es buenísimo porque no tenés que preocuparte por que se rompa nada.

```
# Caracteres especiales y numeros NO cambian
texto = "1234! - * abc ABC"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}")
```

En la terminal verías:

```
Original: 1234! - * abc ABC
swapcase(): 1234! - * ABC abc
```

Ejercicio rápido: Probá con un string que tenga emojis o caracteres raros. ¿Qué creés que va a pasar? (Spoiler: tampoco los toca).

Ejemplos Prácticos

A. Inversión Mixta

Cuando tenés una frase con mezcla de mayúsculas y minúsculas, `swapcase()` invierte cada una.

```
texto = "la GEEKIPEDIA de NAHUEL"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}")
```

```
Original: la GEEKIPEDIA de NAHUEL
swapcase(): LA geekipedia DE nahuel
```

B. Inversión Total (todo mayúsculas)

Si el texto está todo en mayúsculas, `swapcase()` lo pasa todo a minúsculas.

```
texto = "CADENA DE MAYUSCULAS"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}")
```

```
Original: CADENA DE MAYUSCULAS
swapcase(): cadena de mayusculas
```

Reflexión: En este caso, `swapcase()` hace lo mismo que `lower()`. Pero si hubiera alguna minúscula colada, `lower()` la dejaría en minúscula mientras que `swapcase()` la subiría a mayúscula. Esa es la diferencia clave.

C. Inversión Total (todo minúsculas)

Si está todo en minúsculas, pasa a mayúsculas.

```
texto = "cadena con minusculas"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}")
```

```
Original: cadena con minusculas
swapcase(): CADENA CON MINUSCULAS
```

D. Caracteres Especiales y Números

Cualquier cosa que no sea letra se mantiene inalterada.

```
texto = "Hola Mundo 123! Como estas?"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}")
```

```
Original: Hola Mundo 123! Como estas?
swapcase(): hOLA mUNDO 123! cOMO ESTAS?
```

```
texto = "1234! - *"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}") # Queda igual
```

```
Original: 1234! - *
swapcase(): 1234! - *
```

Recordatorio sobre la Persistencia

Ya lo vimos con `lower()` y `upper()`, pero te lo repito porque es un error muy común: si solo hacés `print(variable.swapcase())`, la variable original **no cambia**. Para que el cambio quede guardado, tenés que reasignar la variable.

```
# Solo visualizacion (NO cambia la original)
texto = "Hola Mundo"
print(texto.swapcase())    # hOLA mUNDO (solo se ve)
print(texto)               # Hola Mundo (sigue igual)

# Reasignacion (cambio permanente)
texto = texto.swapcase()
print(texto)               # hOLA mUNDO (ahora si cambio)

hOLA mUNDO
Hola Mundo
hOLA mUNDO
```

Nota: *Regla de oro:*

- `print(texto.swapcase())` → muestra el resultado invertido, la variable sigue original.
- `texto = texto.swapcase()` → guarda el cambio de forma permanente.

Ejemplo práctico: Alternar formato de texto

Podés usar `swapcase()` junto con `lower()` y `upper()` para mostrar todas las variantes de un texto.

```
# Generador de variaciones de texto
texto_original = "El Lenguaje Python"
print(f"Original: {texto_original}")
print(f"Todo minusculas: {texto_original.lower()}")
print(f"Todo mayusculas: {texto_original.upper()}")
print(f"Invertido (swapcase): {texto_original.swapcase()}")
```

```
Original: El Lenguaje Python
Todo minusculas: el lenguaje python
Todo mayusculas: EL LENGUAJE PYTHON
Invertido (swapcase): eL LENGUAJE pYTHON
```

Ejemplo práctico: Efecto *.Espejo.* en textos

Podés jugar con `swapcase()` para hacer efectos visuales. Por ejemplo, partir un texto por la mitad y aplicar distintas transformaciones.


```
# Efecto espejo combinado
texto = "Python"
mitad = len(texto) // 2
primera_mitad = texto[:mitad].upper()
segunda_mitad = texto[mitad:].lower()
print(f"Texto original: {texto}")
print(f"Efecto personalizado: {primera_mitad}{segunda_mitad}")
print(f"Con swapcase(): {texto.swapcase()}")
```

```
Texto original: Python
Efecto personalizado: PYthon
Con swapcase(): pYTHON
```

Reflexión: `swapcase()` te da el resultado opuesto directamente; el efecto personalizado es solo para mostrar que podés hacer lo mismo a mano, pero con más trabajo.

Ejemplo práctico: Procesamiento de datos de usuario

En un sistema donde el usuario escribe sin orden, podés usar `swapcase()` para ofrecer una variante alternativa. Por ejemplo, como sugerencia de contraseña "invertida".

```
nombre = input("Ingresa tu nombre: ")

print(f"\n--- VARIACIONES DE {nombre} ---")
print(f"Normal: {nombre}")
print(f"Minúsculas: {nombre.lower()}")
print(f"Mayúsculas: {nombre.upper()}")
print(f"Alternado: {nombre.swapcase()}")

# Guardamos la version invertida si el usuario quiere
opcion = input("\nQuieres guardar la version alternada? (si/no): ")
if opcion.lower() == "si":
    nombre = nombre.swapcase()
    print(f"Nombre guardado: {nombre}")
```

Ejercicios Propuestos

Ahora poné manos a la obra. Intentá cada uno antes de mirar las soluciones.

1. Creá un string con mayúsculas y minúsculas mezcladas. Usá `swapcase()` y mostrá el resultado.
2. Creá un string completamente en mayúsculas. Usá `swapcase()` y mostrá el resultado.
3. Creá un string completamente en minúsculas. Usá `swapcase()` y mostrá el resultado.
4. Creá un string con números y caracteres especiales. Usá `swapcase()` y observá qué pasa.
5. Usá `swapcase()` solo para visualizar (sin modificar la variable). Mostrá la variable original después para verificar que no cambió.

6. Usá `swapcase()` con reasignación para modificar permanentemente la variable. Mostrala antes y después.
7. Pedile al usuario una frase y mostrá la versión con `swapcase()`.
8. Creá un programa que pida un texto y luego muestre el original, en minúsculas, en mayúsculas y con `swapcase()`.
9. Pedí una contraseña y mostrá su versión con `swapcase()` como sugerencia alternativa.
10. Tomá una lista de palabras y devolvé una nueva lista con cada palabra pasada por `swapcase()`.
11. Pedí un texto y contá cuántas letras cambiaron de mayúscula a minúscula y viceversa después de aplicar `swapcase()`. Ayuda: compará carácter a carácter.
12. Creá una función `espejo(texto)` que devuelva el texto con `swapcase()` pero conservando intacta la primera letra de cada palabra (con su caso original).
13. Pedí un texto y mostrá el resultado de aplicar `swapcase()` dos veces. ¿Qué obtenés?
14. Combiná `swapcase()` con `title()`: primero pasá a formato título y luego invertí los casos. ¿Cómo queda?
15. Pedí un nombre completo y convertí la primera letra de cada palabra a mayúscula con `title()`, y después invertí los casos con `swapcase()`. Mostrá el resultado.
16. Creá un programa que pida dos cadenas y diga si una es la versión `swapcase()` de la otra (sin aplicar el método, usando comparación lógica).
17. Usá `swapcase()` dentro de una f-string para formatear un mensaje dinámico con el texto invertido.
18. Tomá una lista de frases y creá una nueva donde cada frase tenga sus casos invertidos. Guardala y mostrala.
19. Pedí un texto, contá cuántas letras mayúsculas y minúsculas tiene originalmente, luego aplicá `swapcase()` y contá de nuevo. Comprabá que las cuentas se intercambian.
20. Escribí un programa que tome un texto del usuario y lo muestre en mayúsculas, minúsculas y con `swapcase`. Luego preguntá si quiere guardar una de las versiones. Si elige “swapcase”, reasignala la variable y mostrá que el original se perdió.

Soluciones paso a paso

Solución 1

```
texto = "Hola Mundo"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}") # hOLA mUNDO
```

Explicación: Cada letra cambia: H->h, o->O, l->L, a->A, espacio, M->m, etc.

Solución 2

```
texto = "HOLA MUNDO"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}") # hola mundo
```

Explicación: Todo mayúsculas pasa a minúsculas.

Solución 3

```
texto = "hola mundo"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}") # HOLA MUNDO
```

Explicación: Todo minúsculas pasa a mayúsculas.

Solución 4

```
texto = "Hola123! Mundo*"
print(f"Original: {texto}")
print(f"swapcase(): {texto.swapcase()}") # hOLA123! mUNDO*
```

Explicación: Las letras cambian, números y signos quedan igual.

Solución 5

```
texto = "Python Es Genial"
print(f"Mostrando swapcase(): {texto.swapcase()}")
print(f"Original (sin cambios): {texto}")
```

Explicación: El método se usa dentro de print, no se reasigna, por lo que la variable permanece intacta.

Solución 6

```
texto = "Python Es Genial"
print(f"Antes de reasignar: {texto}")
texto = texto.swapcase()
print(f"Despues de reasignar: {texto}")
```

Explicación: Al hacer `texto = texto.swapcase()`, el nuevo string invertido reemplaza al original.

Solución 7

```
frase = input("Ingresa una frase: ")
print(f"Frase con swapcase(): {frase.swapcase()}")
```

Explicación: Directo y simple.

Solución 8

```
texto = input("Ingresa un texto: ")
print("\n--- RESULTADOS ---")
print(f"Original: {texto}")
print(f"Minúsculas: {texto.lower()}")
print(f"Mayúsculas: {texto.upper()}")
print(f"Swapcase: {texto.swapcase()}")
```

Explicación: Mostramos todas las variantes en un solo programa.

Solución 9

```
clave = input("Ingresa una contraseña: ")
print(f"Contraseña original: {clave}")
print(f"Sugerencia alternativa: {clave.swapcase()}")
```

Explicación: Se puede usar para sugerir contraseñas derivadas.

Solución 10

```
palabras = ["Hola", "MUNDO", "PyThOn", "casa", "CASA"]
palabras_swap = []
for palabra in palabras:
    palabras_swap.append(palabra.swapcase())
print("Originales:", palabras)
print("Con swapcase:", palabras_swap)
```

Explicación: Recorremos la lista y aplicamos el método a cada elemento.

Solución 11

```
texto = input("Ingresa un texto: ")
texto_swap = texto.swapcase()
may_a_min = 0
min_a_may = 0
for i in range(len(texto)):
    if texto[i].isupper() and texto_swap[i].islower():
        may_a_min += 1
    elif texto[i].islower() and texto_swap[i].isupper():
        min_a_may += 1
print(f"Letras que pasaron de mayúscula a minúscula: {may_a_min}")
print(f"Letras que pasaron de minúscula a mayúscula: {min_a_may}")
```

Explicación: Comparamos carácter a carácter y contamos los cambios.

Solución 12

```
def espejo(texto):
    resultado = ""
    for palabra in texto.split():
        # Mantener primera letra original, invertir el resto
        palabra_swap = palabra[0] + palabra[1:].swapcase() if len(palabra) > 1 else palabra
        resultado += palabra_swap + " "
    return resultado.strip()

texto = input("Ingresa un texto: ")
print("Texto espejo:", espejo(texto))
```

Explicación: Separamos por palabras, conservamos la primera letra y al resto le aplicamos `swapcase()`.

Solución 13

```
texto = input("Ingresa un texto: ")
doble = texto.swapcase().swapcase()
print(f"Texto original: {texto}")
print(f"Despues de swapcase dos veces: {doble}")
print("Es igual al original?", doble == texto) # True
```

Explicación: Aplicar `swapcase` dos veces vuelve al original, porque invierte y luego vuelve a invertir.

Solución 14

```
texto = input("Ingresa un texto: ")
primero_title = texto.title()
luego_swap = primero_title.swapcase()
print(f"Original: {texto}")
print(f"title(): {primero_title}")
print(f"title + swapcase: {luego_swap}")
```

Explicación: Primero capitalizamos cada palabra (`title`), después invertimos casos.

Solución 15

```
nombre = input("Ingresa tu nombre completo: ")
nombre_title = nombre.title()
nombre_invertido = nombre_title.swapcase()
print(f"Original: {nombre}")
print(f"Formato titulo: {nombre_title}")
print(f"Invertido: {nombre_invertido}")
```

Explicación: Similar al anterior, aplicado a nombres.

Solución 16

```
cadena1 = input("Primera cadena: ")
cadena2 = input("Segunda cadena: ")
es_swap = cadena1 == cadena2.swapcase()
if es_swap:
    print("La segunda es la version swapcase de la primera.")
else:
    print("No coinciden como swapcase.")
```

Explicación: Comparamos sin modificar, usando el método en la comparación.

Solución 17

```
texto = input("Ingresa un texto: ")
print(f"Texto invertido: {texto.swapcase()} y su longitud es {len(texto)}")
```

Explicación: F-string que incluye directamente el resultado de swapcase().

Solución 18

```
frases = ["Hola Mundo", "Python ES genial", "CASA de PAPEL"]
frases_invertidas = [frase.swapcase() for frase in frases]
print("Originales:", frases)
print("Invertidas:", frases_invertidas)
```

Explicación: Comprensión de listas para aplicar swapcase a cada elemento.

Solución 19

```
texto = input("Ingresa un texto: ")
mays_orig = sum(1 for c in texto if c.isupper())
mins_orig = sum(1 for c in texto if c.islower())
texto_swap = texto.swapcase()
mays_swap = sum(1 for c in texto_swap if c.isupper())
mins_swap = sum(1 for c in texto_swap if c.islower())
print(f"Original -> Mayus: {mays_orig}, Minus: {mins_orig}")
print(f"Invertido -> Mayus: {mays_swap}, Minus: {mins_swap}")
print("Se intercambiaron? ", mays_orig == mins_swap and mins_orig == mays_swap)
```

Explicación: Contamos usando generadores y verificamos que las cuentas se intercambian.

Solución 20

```
texto = input("Ingresa un texto: ")
print("\nOpciones:")
print(f"1. Mayusculas: {texto.upper()}")
print(f"2. Minusculas: {texto.lower()}")
print(f"3. Swapcase: {texto.swapcase()}")
opcion = input("Elegi la opcion para guardar (1/2/3): ")
```

```
if opcion == "1":
    texto = texto.upper()
elif opcion == "2":
    texto = texto.lower()
elif opcion == "3":
    texto = texto.swapcase()
print(f"Texto final guardado: {texto}")
print(f"El original se perdio? Si, ahora vale: {texto}")
```

Explicación: Reasignamos según la opción, demostrando que el original cambia permanentemente.

Tabla Resumen de Métodos de Caso

METODO	DESCRIPCION
<code>lower()</code>	Convierte TODO a minusculas
<code>upper()</code>	Convierte TODO a mayusculas
<code>swapcase()</code>	Invierte: mayusculas -> minusculas, minusculas -> mayusculas
<code>islower()</code>	Todo en minusculas? (True/False)
<code>isupper()</code>	Todo en mayusculas? (True/False)

METODO	"Hola Mundo"	"HOLA MUNDO"	"hola mundo"
<code>lower()</code>	"hola mundo"	"hola mundo"	"hola mundo"
<code>upper()</code>	"HOLA MUNDO"	"HOLA MUNDO"	"HOLA MUNDO"
<code>swapcase()</code>	"hOLA mUNDO"	"hola mundo"	"HOLA MUNDO"
<code>islower()</code>	False	False	True
<code>isupper()</code>	False	True	False

Ejemplo Integrador: Procesamiento de texto completo

```
# =====
# PROCESADOR DE TEXTO COMPLETO
# =====

print("\n=== PROCESADOR DE TEXTO ===")
texto = input("Ingrese un texto: ")

print("\n--- ANALISIS DEL TEXTO ---")
print(f"Original: {texto}")
print(f"Longitud: {len(texto)} caracteres")
print(f"Todo minusculas? {texto.islower()}")
print(f"Todo mayusculas? {texto.isupper()}")

print("\n--- TRANSFORMACIONES ---")
print(f"Minusculas: {texto.lower()}")
print(f"Mayusculas: {texto.upper()}")
print(f"Invertido (swapcase): {texto.swapcase()}")

print("\n--- OPCIONES ---")
print("1. Guardar version en minusculas")
print("2. Guardar version en mayusculas")
print("3. Guardar version invertida")
print("4. Mantener original")

opcion = input("Elija una opcion (1-4): ")

if opcion == "1":
    texto = texto.lower()
    print(f"Texto guardado: {texto}")
elif opcion == "2":
    texto = texto.upper()
    print(f"Texto guardado: {texto}")
elif opcion == "3":
    texto = texto.swapcase()
    print(f"Texto guardado: {texto}")
elif opcion == "4":
    print(f"Texto original guardado: {texto}")
else:
    print("Opcion no valida")
```

Resumen del Capítulo

En este capítulo aprendiste:

- `swapcase()` invierte el caso de todas las letras: mayúsculas a minúsculas y viceversa.
- No afecta números, espacios, ni caracteres especiales.
- Es un método de string que devuelve una **nueva cadena**; no modifica la original a menos que reasignes.
- Se puede combinar con otros métodos como `lower()`, `upper()`, `title()`, etc.
- Es muy útil para generar variantes de texto, sugerencias de contraseñas, o simplemente para voltear el formato.

swapcase() es como un interruptor de luz: apaga lo que está encendido y enciende lo que está apagado. Perfecto para invertir el caso de un texto en una sola línea.

Método `capitalize()` – Primera Letra Mayúscula

¿Qué es el método `capitalize()`?

Cuando empecé a programar, una de las tareas que más me daba por las pelotas era tener que formatear los textos que me llegaban desde formularios. La gente escribe como se le canta: a veces todo en mayúsculas, otras todo en minúsculas, o mezclado como "h01A mUnDo". Quería dejarlo prolijo, con la primera letra mayúscula y el resto en minúscula, como una oración normal. Al principio armé un bucle con `upper()` en la primera letra y `lower()` en el resto, pero era un código feo y largo. Hasta que un día, revisando la guía de métodos de string, me topé con `capitalize()`. ¡Era justo lo que necesitaba! En una sola línea hacía todo el trabajo.

Analogía: `capitalize()` es como un corrector automático de textos: tomás una hoja escrita en mayúsculas y minúsculas revoltosas, y la máquina te la devuelve con la primera letra bien puesta y el resto bajado, lista para entregar.

El método `capitalize()` se usa para dar un **formato específico** a una cadena de texto:

- Convierte el **primer carácter** del string en una **letra mayúscula**.
- Convierte **todas las demás letras** de la cadena en **minúsculas**.

Como decía un colega en la facultad (UBA Exactas): “`capitalize()` es como tener un corrector de textos: agarra cualquier frase desordenada y la deja prolija, con la primera letra mayúscula y el resto en minúsculas.”

```
+-----+
|
|  texto = "eL vIaJe Es La ReCoMpEnSa"
|
|  texto.capitalize()
|
|  Resultado: "El viaje es la recompensa"
|
|  Primera letra -> mayuscula (E)
|  Resto del texto -> todo minusculas
|
+-----+
```

Sintaxis

Es fundamental escribir bien el método (todo en minúsculas):

```
nombre_variable.capitalize()
```

- No lleva argumentos entre los paréntesis.
- Devuelve un **nuevo string** con el formato aplicado.
- **NO modifica** la variable original (hay que reasignar).

Si querés guardar el cambio, tenés que reasignar la variable, como vimos con `lower()` y `upper()`. En criollo: si solo lo imprimís, la variable sigue igual.

Comportamiento detallado

El método `capitalize()` es ideal para **normalizar oraciones**. Si tenés una frase con mayúsculas y minúsculas desordenadas, `capitalize()` la “limpia” dejando solo la inicial como corresponde a una oración estándar.

Ejemplo con texto desordenado

```
texto = "eL vIaJe Es La ReCoMpEnSa"
print(f"Original: {texto}")
print(f"capitalize(): {texto.capitalize()}")
```

En consola vas a ver:

```
Original: eL vIaJe Es La ReCoMpEnSa
capitalize(): El viaje es la recompensa
```

Ejemplo con todo mayúsculas

```
texto = "EL VIAJE ES LA RECOMPENSA"
print(f"Original: {texto}")
print(f"capitalize(): {texto.capitalize()}")
```

```
Original: EL VIAJE ES LA RECOMPENSA
capitalize(): El viaje es la recompensa
```

Ejemplo con todo minúsculas

```
texto = "el viaje es la recompensa"
print(f"Original: {texto}")
print(f"capitalize(): {texto.capitalize()}")
```

```
Original: el viaje es la recompensa
capitalize(): El viaje es la recompensa
```

Reflexión: Fijate que sin importar cómo venía el texto, siempre termina igual: arranca con mayúscula y el resto en minúscula. Es un cable a tierra.

Práctica en Código (Con reasignación)

Este es el punto más importante: **si no reasignás, el cambio no se guarda** en la variable para futuros usos. Muchas veces vi programadores principiantes que se sorprendían porque después de un `print(variable.capitalize())` la variable seguía igual.

Solo visualización (NO guarda el cambio)

```
texto = "el viaje es la recompensa"
print(f"Original: {texto}")

# Solo visualizacion (la variable original NO cambia)
print(f"Mostrando capitalize(): {texto.capitalize()}")
print(f"Original sigue igual: {texto}")
```

```
Original: el viaje es la recompensa
Mostrando capitalize(): El viaje es la recompensa
Original sigue igual: el viaje es la recompensa
```

Con reasignación (guarda el cambio)

```
texto = "el viaje es la recompensa"
print(f"Antes de reasignar: {texto}")

# Aplicamos capitalize() Y REASIGNAMOS ✓
texto = texto.capitalize()

print(f"Despues de reasignar: {texto}")

Antes de reasignar: el viaje es la recompensa
Despues de reasignar: El viaje es la recompensa
```

Nota: *Regla de oro:*

- `print(texto.capitalize())` → solo muestra el resultado.
- `texto = texto.capitalize()` → guarda el cambio en la variable.

Ejemplo completo con comparación de estados

Ahora te muestro cómo ver el antes y el después usando los validadores que ya conocés.

```
frase = "eL vIaJe Es La ReCoMpEnSa"

print("=== ANTES DE capitalize() ===")
print(f"Original: {frase}")
print(f"Empieza con mayuscula? {frase[0].isupper()}")
print(f"Esta todo en minusculas? {frase.islower()}")
```

```
# Aplicamos y reasignamos
frase = frase.capitalize()

print("\n=== DESPUES DE capitalize() ===")
print(f"Resultado: {frase}")
print(f"Empieza con mayuscula? {frase[0].isupper()}")
print(f"Esta todo en minusculas? {frase.islower()}")

=== ANTES DE capitalize() ===
Original: eL vIaJe Es La ReCoMpEnSa
Empieza con mayuscula? False
Esta todo en minusculas? False

=== DESPUES DE capitalize() ===
Resultado: El viaje es la recompensa
Empieza con mayuscula? True
Esta todo en minusculas? True
```

Ejercicio rápido: Probá vos con tu nombre en mayúsculas, después con tu nombre en minúsculas, y anotá cómo cambia la validación de `islower()`.

¿Qué pasa con números y caracteres especiales?

`capitalize()` afecta principalmente a las letras. Si el primer carácter no es una letra, no se convierte a mayúscula. El resto de la cadena sí se pasa a minúsculas, pero la primera posición queda intacta si no es alfabética.

```
texto1 = "123 hola mundo"
print(f"Original: {texto1}")
print(f"capitalize(): {texto1.capitalize()}")

texto2 = "* hola mundo"
print(f"Original: {texto2}")
print(f"capitalize(): {texto2.capitalize()}")

Original: 123 hola mundo
capitalize(): 123 hola mundo
Original: * hola mundo
capitalize(): * hola mundo
```

Observá que el “123” y el “*” no se transforman; simplemente se quedan como están. Las letras siguientes sí se vuelven minúsculas.

Ejemplo práctico: Normalización de nombres de personas

Este método es útil para normalizar nombres, siempre que entiendas que solo pone mayúscula la primera letra de todo el texto. Si tenés nombre y apellido, cada uno debería ir con mayúscula, y para eso se usa `title()`, no `capitalize()`. Pero si tenés un solo nombre o una frase completa, `capitalize()` cumple.

```
nombres = ["jUAn pErEz", "MARIA GONZALEZ", "carlos lopez", "ANA MARTINEZ"]

print("=== NORMALIZACION DE NOMBRES ===")
print("Nombres originales:")
for nombre in nombres:
    print(f" {nombre}")

print("\nNombres normalizados con capitalize():")
for nombre in nombres:
    print(f" {nombre.capitalize()}")

# Guardamos los normalizados
nombres_normalizados = [nombre.capitalize() for nombre in nombres]
print("\nLista guardada:", nombres_normalizados)

=== NORMALIZACION DE NOMBRES ===
Nombres originales:
jUAn pErEz
MARIA GONZALEZ
carlos lopez
ANA MARTINEZ

Nombres normalizados con capitalize():
Juan perez
Maria gonzalez
Carlos lopez
Ana martinez

Lista guardada: ['Juan perez', 'Maria gonzalez', 'Carlos lopez', 'Ana martinez']
```

Nota: Observación: *Notá que `capitalize()` solo pone la PRIMERA letra de todo el string en mayúscula. Para nombres propios compuestos (donde cada palabra debe tener mayúscula) usamos `title()`, no `capitalize()`.*

Ejemplo práctico: Procesamiento de títulos de libros

```
libros = [
    "el SEÑOR de los anillos",
    "CIEN años de SOLEDAD",
    "la CASA de los ESPÍRITUS",
    "1984 george orwell"
]

print("=== NORMALIZACION DE TITULOS ===")
for titulo in libros:
    print(f" Original: {titulo}")
    print(f" capitalize(): {titulo.capitalize()}\n")
```

Original: el SEÑOR de los anillos
capitalize(): El señor de los anillos

Original: CIEN años de SOLEDAD
capitalize(): Cien años de soledad

Original: la CASA de los ESPÍRITUS
capitalize(): La casa de los espíritus

Original: 1984 george orwell
capitalize(): 1984 george orwell

Ejemplo práctico: Limpiar entrada del usuario

Un uso cotidiano: pedís nombre y apellido y los dejás con el formato deseado.

```
print("=== REGISTRO DE USUARIO ===")
nombre = input("Ingresa tu nombre: ")
apellido = input("Ingresa tu apellido: ")

nombre = nombre.capitalize()
apellido = apellido.capitalize()

print(f"\nUsuario registrado: {nombre} {apellido}")
```

Ejecucion:

Ingresa tu nombre: jUAn
Ingresa tu apellido: pErEz

Usuario registrado: Juan Perez

Ejercicios propuestos

Llegó la hora de que pongas las manos en la masa. Hacé cada uno, no te saltees, y recién después mirá las soluciones.

1. Creá un string con la primera letra en minúscula y el resto mezclado. Usá `capitalize()` y mostrá el resultado.
2. Creá un string completamente en mayúsculas. Usá `capitalize()` y mostrá el resultado.
3. Creá un string completamente en minúsculas. Usá `capitalize()` y mostrá el resultado.
4. Usá `capitalize()` solo para visualizar (sin modificar la variable). Mostrá la variable original después para verificar que no cambió.
5. Usá `capitalize()` con reasignación para modificar permanentemente la variable.
6. Pedile al usuario una frase y mostrá la versión con `capitalize()`.

7. Creá un programa que pida al usuario un nombre y un apellido, los normalice con `capitalize()`, y los muestre formateados.
8. Creá una lista de 5 frases desordenadas (con mayúsculas y minúsculas mezcladas). Usá un bucle para normalizarlas con `capitalize()` y mostrá los resultados.
9. ¿Qué pasa con `capitalize()` si el texto comienza con un número? Probá y explicá lo que observás.
10. Creá un programa que reciba una oración del usuario y la muestre primero con `capitalize()` y luego con `title()`. Compará los resultados y explicá la diferencia.
11. Pedí una palabra y mostrá el resultado de aplicarle `capitalize()` aunque la palabra ya tenga una mayúscula en otra posición. Ejemplo: "h01A".
12. Tomá una lista de nombres completos (nombre y apellido separados por espacio) y aplicá `capitalize()` a cada elemento. Guardá los resultados en una nueva lista y mostrala.
13. Pedí un texto y verificá si después de aplicarle `capitalize()` la primera letra es mayúscula y el resto son minúsculas. Usá `islower()` en la parte que corresponde.
14. Pedí dos textos distintos y mostrá cuál queda más largo después de aplicar `capitalize()`. Pista: compará longitudes.
15. Creá una función `normalizar_oracion(oracion)` que reciba una oración y devuelva la versión con `capitalize()` y, si no termina con punto, le agregue uno al final.
16. Usá comprensión de listas para normalizar todas las frases de una lista `frases = ["h01A mUnDo", ".^DIOS", "Que Tal"]` y guardalas en una nueva variable.
17. Pedí un texto y contá cuántas letras cambiaron de mayúscula a minúscula (exceptuando la primera) después de aplicar `capitalize()`. Compará carácter a carácter.
18. Pedí un texto, convertilo con `capitalize()` y después aplicá `swapcase()` al resultado. Mostrá el texto original, el intermedio y el final.
19. Escribí un programa que pida una oración y, si la primera letra es minúscula, le aplique `capitalize()`; si ya empieza con mayúscula, que la deje igual.
20. Creá un programita que permita ingresar varias líneas de texto (hasta que se ingrese la palabra "fin") y luego muestre todas las líneas normalizadas con `capitalize()`. Guardalas en una lista.

Soluciones paso a paso

Solución 1

```
texto = "h0lA mUnDo"  
print(f"Original: {texto}")  
print(f"capitalize(): {texto.capitalize()}") # Hola mundo
```

Explicación: La primera letra pasa a mayúscula, todas las demás a minúscula.

Solución 2

```
texto = "HOLA MUNDO"  
print(f"Original: {texto}")  
print(f"capitalize(): {texto.capitalize()}") # Hola mundo
```

Explicación: Aunque estaba todo en mayúsculas, solo la primera se mantiene mayúscula y el resto baja.

Solución 3

```
texto = "hola mundo"  
print(f"Original: {texto}")  
print(f"capitalize(): {texto.capitalize()}") # Hola mundo
```

Explicación: La primera letra sube a mayúscula, lo demás sigue igual porque ya era minúscula.

Solución 4

```
texto = "hola mundo"  
print(f"Mostrando capitalize(): {texto.capitalize()}")  
print(f"Original sin cambios: {texto}") # Sigue "hola mundo"
```

Explicación: No reasignamos, solo vemos el resultado.

Solución 5

```
texto = "hola mundo"  
print(f"Antes de reasignar: {texto}")  
texto = texto.capitalize()  
print(f"Despues de reasignar: {texto}") # "Hola mundo"
```

Explicación: Ahora la variable apunta al nuevo string.

Solución 6

```
frase = input("Ingresa una frase: ")  
print(f"Frase con capitalize(): {frase.capitalize()}")
```

Explicación: Directo, mostramos la versión normalizada.

Solución 7

```
nombre = input("Nombre: ")
apellido = input("Apellido: ")
nombre = nombre.capitalize()
apellido = apellido.capitalize()
print(f"Nombre completo: {nombre} {apellido}")
```

Explicación: Normalizamos cada parte por separado (aunque para apellido tal vez quieras title, pero así cumple el ejercicio).

Solución 8

```
frases = [
    "eL vIaJe Es La ReCoMpEnSa",
    "lA vIdA eS bElLa",
    "pYtHoN eS gEnIaL",
    "uBA eXaCtAs",
    "pRoGrAmAr Es DiVeRtIdO"
]
print("Frases normalizadas:")
for frase in frases:
    print(f" {frase.capitalize()}")
```

Explicación: Bucle simple, mostramos cada una ya capitalizada.

Solución 9

```
texto = "123 hola mundo"
print(f"Original: {texto}")
print(f"capitalize(): {texto.capitalize()}") # 123 hola mundo (no cambia el primer caract
```

Explicación: El primer carácter es un número, así que no se convierte a mayúscula; el resto sí baja a minúsculas.

Solución 10

```
oracion = input("Ingresa una oracion: ")
print(f"Con capitalize(): {oracion.capitalize()}")
print(f"Con title(): {oracion.title()}")
print("\nDiferencia: capitalize() solo pone mayuscula la primera letra de toda la cadena
print("title() pone mayuscula la primera letra de CADA palabra.")
```

Explicación: Así se ve claramente la diferencia entre ambos métodos.

Solución 11

```
palabra = "h01a"
print(f"Original: {palabra}")
print(f"capitalize(): {palabra.capitalize()}") # Hola
```

Explicación: La H inicial se vuelve mayúscula, el resto minúsculas.

Solución 12

```
nombres = ["jUAn pErEz", "ANA GOMEZ", "pepe martinez"]
capitalizados = [nombre.capitalize() for nombre in nombres]
print("Originales:", nombres)
print("Capitalizados:", capitalizados)
```

Explicación: Comprensión de listas aplica el método a cada elemento.

Solución 13

```
texto = input("Ingresa un texto: ")
capitalizado = texto.capitalize()
primera_mayus = capitalizado[0].isupper() if len(capitalizado) > 0 else False
resto = capitalizado[1:] if len(capitalizado) > 1 else ""
resto_minus = resto.islower() if resto else True
print("Primera mayuscula:", primera_mayus)
print("Resto en minusculas:", resto_minus)
```

Explicación: Verificamos carácter a carácter, salvo que la cadena sea muy corta.

Solución 14

```
texto1 = input("Primer texto: ")
texto2 = input("Segundo texto: ")
t1 = texto1.capitalize()
t2 = texto2.capitalize()
if len(t1) > len(t2):
    print(f"El primero es mas largo: '{t1}'")
elif len(t2) > len(t1):
    print(f"El segundo es mas largo: '{t2}'")
else:
    print("Tienen la misma longitud.")
```

Explicación: Normalizamos ambos y comparamos sus longitudes.

Solución 15

```
def normalizar_oracion(oracion):
    oracion = oracion.capitalize()
    if not oracion.endswith('.'):
        oracion += '.'
    return oracion
```

```
frase = input("Ingresa una frase: ")
print(normalizar_oracion(frase))
```

Explicación: La función capitaliza y agrega punto si falta.

Solución 16

```
frases = ["h01A mUnDo", "ADIOS", "Que Tal"]
normalizadas = [f.capitalize() for f in frases]
print(normalizadas)
```

Explicación: Otra vez comprensión de listas, práctico y corto.

Solución 17

```
texto = input("Texto: ")
capitalizado = texto.capitalize()
contador = 0
# Empezamos desde índice 1 para ignorar la primera letra
for i in range(1, len(texto)):
    if texto[i].isupper() and capitalizado[i].islower():
        contador += 1
print(f"Letras que pasaron de mayuscula a minuscula: {contador}")
```

Explicación: Recorremos y comparamos carácter a carácter después del índice 0.

Solución 18

```
texto = input("Texto: ")
capitalizado = texto.capitalize()
invertido = capitalizado.swapcase()
print(f"Original: {texto}")
print(f"Capitalize: {capitalizado}")
print(f"Swapcase del capitalize: {invertido}")
```

Explicación: Encadenamos métodos para ver el efecto acumulado.

Solución 19

```
oracion = input("Oracion: ")
if len(oracion) > 0 and oracion[0].islower():
    oracion = oracion.capitalize()
print("Resultado:", oracion)
```

Explicación: Solo aplicamos si empieza con minúscula; si no, se deja igual.

Solución 20

```
lineas = []
print("Ingresa lineas (escribi 'fin' para terminar):")
while True:
    linea = input("> ")
    if linea.lower() == "fin":
        break
    lineas.append(linea.capitalize())
```

```
print("\nLineas normalizadas:")
for i, linea in enumerate(lineas, 1):
    print(f"{i}. {linea}")
```

Explicación: Acumulamos en una lista y luego mostramos todas ya capitalizadas.

Tabla Comparativa: capitalize() vs title()

METODO	EJEMPLO	RESULTADO
capitalize()	"el viaje es la recompensa"	"El viaje es la recompensa" (solo primera letra mayuscula)
title()	"el viaje es la recompensa"	"El Viaje Es La Recompensa" (primera letra de CADA palabra)

Ejemplo Integrador: Sistema de Normalización de Textos

Vamos a armar un pequeño programa que permite ingresar frases y normalizarlas con `capitalize()`.

```
# =====
# SISTEMA DE NORMALIZACION DE TEXTOS
# =====

print("\n=== NORMALIZADOR DE TEXTOS ===")
print("Este programa normaliza frases al formato estandar")
print("(primera letra mayuscula, resto minusculas)")

frases = []
while True:
    print("\n1. Ingresar una frase")
    print("2. Normalizar todas las frases guardadas")
    print("3. Mostrar frases originales")
    print("4. Mostrar frases normalizadas")
    print("5. Salir")

    opcion = input("Elegi una opcion: ")

    if opcion == "1":
        frase = input("Ingresa la frase: ")
        frases.append(frase)
        print("Frase guardada.")
    elif opcion == "2":
        if not frases:
            print("No hay frases para normalizar.")
        else:
            for i in range(len(frases)):
                frases[i] = frases[i].capitalize()
            print("¡Todas las frases fueron normalizadas!")
    elif opcion == "3":
        if not frases:
            print("No hay frases guardadas.")
        else:
            print("\n--- FRASES ORIGINALES ---")
            for i, frase in enumerate(frases, 1):
                print(f"{i}. {frase}")
    elif opcion == "4":
        if not frases:
            print("No hay frases guardadas.")
        else:
            print("\n--- FRASES NORMALIZADAS ---")
            for i, frase in enumerate(frases, 1):
                print(f"{i}. {frase.capitalize()}")
    elif opcion == "5":
```



```
        print("¡Hasta luego!")
        break
    else:
        print("Opcion no valida")
```

Resumen del Capítulo

En este capítulo aprendiste:

- `capitalize()` convierte la **primera letra** del string en **mayúscula** y el **resto** en **minúsculas**.
- No modifica la cadena original; hay que reasignar si querés conservar el cambio.
- Si el primer carácter no es una letra, no se transforma, pero el resto igual pasa a minúsculas.
- Es útil para normalizar oraciones simples, pero no para nombres compuestos (ahí usamos `title()`).
- Combinalo con `input()` para limpiar entradas de usuario.
- La diferencia con `title()` es clave: `capitalize()` solo sube la primera letra de todo el texto, `title()` sube la primera de cada palabra.

capitalize() es como un corrector ortográfico instantáneo: agarra cualquier frase desordenada y la deja prolija, lista para ser mostrada al usuario.

Alineación de Cadenas: `center()`, `ljust()`, `rjust()`

¿Por qué necesitamos alinear texto?

La primera vez que tuve que mostrar una tabla en la consola, me quedó un desastre. Había armado una lista de productos con sus precios y quería que se viera prolijo, pero los números bailaban y el texto no quedaba alineado. Me acuerdo de que estaba en Windows y usaba espacios a lo bruto, tratando de contar caracteres manualmente. ¡Un horror! Cuando al fin un profe me dijo que Python ya venía con métodos para centrar y alinear, sentí un alivio enorme. En criollo, Python te da las herramientas para que tus textos queden ordenados como si fueran columnas de una planilla de cálculo.

Python ofrece tres métodos principales para alinear texto rellenando con espacios o con el carácter que vos elijas:

- `center()`: centra el texto en un espacio determinado.
- `ljust()`: alinea el texto a la izquierda (left justify).
- `rjust()`: alinea el texto a la derecha (right justify).

Analogía: Imaginá que tenés un estante de largo fijo y querés colocar un cartel. `ljust()` lo manda bien contra la izquierda, `rjust()` lo empuja a la derecha, y `center()` lo pone justo en el medio. Si sobra lugar, podés rellenar con espuma (espacios) o con bloqueitos decorativos (el carácter que vos digas).

```
+-----+
|
|  center() -> centra el texto
|  ljust()  -> alinea a la izquierda (left)
|  rjust()  -> alinea a la derecha (right)
|
|  Todos requieren argumentos para funcionar
|
+-----+
```

Reglas Comunes a los Tres Métodos

Antes de zambullirnos en cada método, mirá estas reglas que son iguales para todos:

1. El **primer argumento** debe ser un **número entero**: la longitud total deseada del string final.
2. Ese número debe ser **mayor** que la longitud del string original. Si ponés un número menor o igual, el método no agrega nada y te devuelve el string original tal cual.
3. El **segundo argumento** (opcional) debe ser un **solo carácter**. Si ponés más de uno, Python te va a putear con un error.
4. Si no ponés el segundo argumento, Python asume que querés rellenar con **espacios en blanco**.

Nota: Longitud del string: Podés averiguarla con `len(texto)`. Si querés centrar "Hola" (4 letras) en un espacio de 10, Python va a agregar 3 espacios antes y 3 después (porque $4 + 3 + 3 = 10$, y si sobra 1 suele ir a la derecha).

El Método center() – Centrar texto

`center()` hace lo que su nombre indica: centra el string dentro de una cajita del ancho que vos determines.

Sintaxis

```
variable.center(longitud, "character")
```

Funcionamiento

Supone que tu string mide 4 y le pedís ancho 10. Python calcula cuánto espacio sobra (6), lo divide en dos mitades (3 y 3) y rodea tu texto con esos espacios o caracteres. Si la división no es exacta, el espacio extra va a parar a la derecha.

Ejemplos básicos

```
texto = "Hola"

# Un solo argumento: rellena con espacios
print(f'"{texto.center(10)}"')

# Dos argumentos: rellena con un carácter específico
print(f'"{texto.center(10, '-')}')
print(f'"{texto.center(10, '*')}')
```

En la consola vas a ver:

```
'  Hola  '
'---Hola---'
'***Hola***'
```

Reflexión: Las comillas en el `print` son para que puedas ver bien los espacios agregados. En un uso normal, si no las ponés, el usuario solo ve el texto centrado, sin comillas.

Ejemplo práctico: Título centrado para un menú

```

titulo = "MENU PRINCIPAL"
print(titulo.center(30, "="))
print(titulo.center(30, " "))
print(titulo.center(30, "-"))

=====MENU PRINCIPAL=====
      MENU PRINCIPAL
-----MENU PRINCIPAL-----

```

Ejercicio rápido: Probá centrar tu nombre con asteriscos, cambiando el ancho total de 20 a 40. ¿Cómo se ve?

El Método ljust() – Alinear a la izquierda

`ljust()` viene de *left justify* (justificado a la izquierda). Pone el texto original bien contra la pared izquierda y rellena el espacio restante a la derecha.

Sintaxis

```
variable.ljust(longitud, "caracter")
```

Funcionamiento

Si tu string mide 4 y pedís ancho 10, `ljust()` pone los 4 caracteres a la izquierda y luego 6 caracteres de relleno a la derecha.

Ejemplos básicos

```

texto = "Hola"

print(f'"{texto.ljust(10)}"')
print(f'"{texto.ljust(10, "-')}")
print(f'"{texto.ljust(10, "*')}")

'Hola      '
'Hola-----'
'Hola*****'

```

Ejemplo práctico: Lista de productos con nombre alineado a la izquierda

```

productos = [("Manzana", 1500), ("Pera", 1200), ("Sandia", 3000)]

print("=== LISTA DE PRODUCTOS ===")
for producto, precio in productos:
    print(f"{producto.ljust(15)} ${precio}")

```

```
=== LISTA DE PRODUCTOS ===
Manzana      $1500
Pera          $1200
Sandia        $3000
```

Reflexión: Así los nombres quedan alineados y los precios arrancan todos en la misma columna. Es mucho más limpio que imprimir todo pegado.

El Método rjust() – Alinear a la derecha

rjust() es justo lo opuesto a ljust(). Apoya el texto contra la derecha y rellena a la izquierda.

Sintaxis

```
variable.rjust(longitud, "caracter")
```

Funcionamiento

String de 4, ancho 10: primero van 6 caracteres de relleno, después tu texto.

Ejemplos básicos

```
texto = "Hola"

print(f'{{texto.rjust(10)}}')
print(f'{{texto.rjust(10, '-')}}')
print(f'{{texto.rjust(10, '*')}}')

'      Hola'
'-----Hola'
'*****Hola'
```

Ejemplo práctico: Precios alineados a la derecha

```
productos = [("Manzana", 1500), ("Pera", 12000), ("Sandia", 300000)]

print("=== LISTA DE PRODUCTOS ===")
for producto, precio in productos:
    # Convertimos el precio a string y lo alineamos a la derecha
    print(f'{{producto.ljust(15)}} ${{str(precio).rjust(8)}}

```

Ejercicio rápido: Modificá el ancho del rjust() para los precios y fijate cómo afecta el formato. ¿Qué pasa si ponés un ancho menor que la cantidad de dígitos del precio? (Spoiler: no lo corta, simplemente no modifica nada).

La Importancia de la Reasignación

Como todo método de string, `center()`, `ljust()` y `rjust()` devuelven un nuevo string y no modifican el original, a menos que vos reasignes. Si te olvidás de esto, vas a ver el texto alineado en el `print` pero la variable va a seguir igual.

Solo visualización (NO guarda el cambio)

```
texto = "Hola"
print(f"Original: '{texto}'")

# Solo lo muestro, pero no lo guardo
print(f"Centrado temporal: '{texto.center(10)}'")
print(f"Original sigue igual: '{texto}'") # Sigue siendo "Hola"

Original: 'Hola'
Centrado temporal: '  Hola  '
Original sigue igual: 'Hola'
```

Con reasignación (guarda el cambio)

```
texto = "Hola"
print(f"Antes: '{texto}'")

# Reasignamos el valor centrado
texto = texto.center(10)
print(f"Después: '{texto}'") # Ahora si, "  Hola  "

Antes: 'Hola'
Después: '  Hola  '
```

Nota: Regla de oro: Si querés conservar el texto formateado para usarlo después, hacé `texto = texto.center(20)`. Si solo lo querés mostrar una vez, con el `print` alcanza.

Ejemplos Prácticos con Tres Escenarios

Vamos a probar los tres métodos en acción con distintos parámetros.

Escenario 1: Un solo argumento (relleno con espacios)

```
palabra = "Python"
print(f"Original: '{palabra}'")
print(f"center(15): '{palabra.center(15)}'")
print(f"ljust(15): '{palabra.ljust(15)}'")
print(f"rjust(15): '{palabra.rjust(15)}'")

Original: 'Python'
center(15): '    Python    '
ljust(15): 'Python      '
rjust(15): '      Python'
```

Escenario 2: Dos argumentos (relleno con caracteres especiales)

```
palabra = "Python"
print(f"center(20, '='): '{palabra.center(20, '=')}'")
print(f"ljust(20, '-'): '{palabra.ljust(20, '-')}'")
print(f"rjust(20, '*'): '{palabra.rjust(20, '*')}'")

center(20, '='): '====Python===='
ljust(20, '-'): 'Python-----'
rjust(20, '*'): '*****Python'
```

Escenario 3: Creación de separadores visuales para menús

Acá es donde estos métodos brillan. Imaginá que estás haciendo una aplicación de consola y querés un menú limpio.

```
menu = "MENU PRINCIPAL"
opcion1 = "1. Sumar"
opcion2 = "2. Restar"
opcion3 = "3. Multiplicar"
opcion4 = "4. Salir"

print(menu.center(30, "="))
print(opcion1.ljust(30, "."))
print(opcion2.ljust(30, "."))
print(opcion3.ljust(30, "."))
print(opcion4.ljust(30, "."))
print("".center(30, "="))

=====MENU PRINCIPAL=====
1. Sumar.....
2. Restar.....
3. Multiplicar.....
4. Salir.....
=====
```

Ejemplo práctico: Tabla de datos formateada

Vamos a combinar ljust(), rjust() y center() para armar una tabla con encabezados.

```
# Datos de ejemplo
datos = [
    ["Nahuel", 25, "Buenos Aires"],
    ["Ana", 30, "Cordoba"],
    ["Luis", 22, "Rosario"],
    ["Maria", 28, "Mendoza"]
]

print("=== TABLA DE DATOS ===")
```



```
# Encabezados alineados
print(f"{'Nombre'.ljust(15)} {'Edad'.center(6)} {'Ciudad'.rjust(15)}")
print("-" * 40)

# Datos alineados
for nombre, edad, ciudad in datos:
    print(f"{nombre.ljust(15)} {str(edad).center(6)} {ciudad.rjust(15)}")

=== TABLA DE DATOS ===
Nombre          Edad          Ciudad
-----
Nahuel           25          Buenos Aires
Ana              30           Cordoba
Luis             22           Rosario
Maria            28           Mendoza
```

Reflexión: Fijate cómo cada pieza se ubica en su columna. Los nombres a la izquierda, las edades centradas, las ciudades a la derecha. Así queda profesional con poquitas líneas.

Ejercicios Propuestos

Ahora es tu turno de practicar. Hacé cada ejercicio en tu editor, corregí los errores que aparezcan, y solo después revisá las soluciones.

1. Usá `center()` para centrar la palabra “Python” en un espacio de 20 caracteres. Mostrala entre comillas para ver los espacios.
2. Usá `ljust()` para alinear “Holá” a la izquierda en un espacio de 15 caracteres.
3. Usá `rjust()` para alinear “Mundo” a la derecha en un espacio de 15 caracteres.
4. Usá `center()` con el carácter “*” para centrar “Python” en 20 caracteres.
5. Usá `ljust()` con guiones para rellenar “Python” hasta 20 caracteres.
6. Usá `rjust()` con puntos para rellenar “Final” hasta 20 caracteres a la izquierda.
7. Creá un título de menú centrado con `center()` y bordes de “=” de ancho 40.
8. Creá una lista de productos con nombre (`ljust`) y precio (`rjust`) para que se vean como columnas. Ancho: nombre 20, precio 10.
9. Pedile al usuario un texto y mostralo centrado en un espacio de 50 caracteres.
10. Generá un menú de opciones con formato profesional: título centrado con “=”, opciones alineadas a la izquierda con puntos, y pie de “=”.
11. Usá `rjust()` para mostrar números de 1 a 5 alineados a la derecha en un ancho fijo, uno debajo del otro.
12. Pedí un nombre y un apellido, y mostralos en dos columnas: nombre a la izquierda (ancho 15) y apellido a la derecha (ancho 15).

13. Creá una función `encuadrar_titulo(titulo, ancho)` que devuelva el título centrado entre líneas de “-” arriba y abajo, del ancho dado.
14. Dado un texto, mostralo centrado en un ancho que sea el doble de su longitud + 2, usando “.” como relleno.
15. Pedí una lista de 3 nombres y mostralos en una columna alineada a la izquierda de ancho 20 y a la derecha la longitud de cada nombre, alineada a la derecha con ancho 10.
16. Tomá la frase “Python es genial” y muestra cada palabra en una línea: la primera centrada (ancho 20), la segunda a la izquierda (ancho 20), la tercera a la derecha (ancho 20).
17. Pedí una palabra y un carácter, y mostrá la palabra centrada en un ancho de 30 con ese carácter.
18. Simulá un ticket de compra: encabezado “TICKET” centrado con “*”, ítems alineados izquierda/derecha, y total con doble línea.
19. Escribí un programa que reciba un texto y un ancho, y muestre el texto alineado de las tres formas (center, ljust, rjust) cada una en una línea, con un carácter de relleno distinto.
20. Creá una función `tabla_multiplicar(n, ancho)` que muestre la tabla del 1 al 10 con el formato “`n x i = resultado`”, alineando cada parte: el primer número a la izquierda, el “`x i`” centrado, y el resultado a la derecha, todo dentro del ancho total.

Soluciones paso a paso

No mires las soluciones hasta que no hayas intentado cada ejercicio. Si te trabás, acá están con explicaciones.

Solución 1

```
texto = "Python"
print(f' {texto.center(20)} ')
```

Explicación: `center(20)` agrega 7 espacios a cada lado (o similar) para llegar a 20 caracteres. Las comillas simples muestran dónde empieza y termina el string resultante.

Solución 2

```
texto = "Hola"
print(f' {texto.ljust(15)} ')
```

Explicación: `ljust(15)` pone “Hola” a la izquierda y completa con 11 espacios a la derecha.

Solución 3

```
texto = "Mundo"
print(f' {texto.rjust(15)} ')
```

Explicación: `rjust(15)` empuja “Mundo” contra la derecha y agrega 10 espacios a la izquierda.

Solución 4

```
texto = "Python"
print(f' {texto.center(20, '*')} ')
```

Explicación: Al dar un segundo argumento, rellena con ese carácter en lugar de espacios.

Solución 5

```
texto = "Python"
print(f' {texto.ljust(20, '-')} ')
```

Explicación: Guiones a la derecha hasta completar 20 caracteres.

Solución 6

```
texto = "Final"
print(f' {texto.rjust(20, '.')} ')
```

Explicación: Puntos a la izquierda hasta completar 20 caracteres.

Solución 7

```
titulo = "MENU PRINCIPAL"  
print(titulo.center(40, "="))
```

Explicación: Centramos el título con signos igual a ambos lados.

Solución 8

```
productos = [("Manzana", 1500), ("Pera", 12000), ("Sandia", 300000)]  
for prod, precio in productos:  
    print(f"{prod.ljust(20)} ${str(precio).rjust(10)}")
```

Explicación: El nombre se fija a la izquierda con ancho 20, el precio a la derecha con ancho 10. El signo peso queda afuera de la alineación pero fijo.

Solución 9

```
texto = input("Ingresa un texto: ")  
print(texto.center(50))
```

Explicación: Leemos, centramos en 50 espacios y mostramos.

Solución 10

```
print("".center(40, "="))  
print("MENU PRINCIPAL".center(40))  
print("".center(40, "-"))  
print("1. Sumar".ljust(40, "."))  
print("2. Restar".ljust(40, "."))  
print("3. Multiplicar".ljust(40, "."))  
print("4. Salir".ljust(40, "."))  
print("".center(40, "="))  
print("Elija una opcion: ".rjust(40))
```

Explicación: Combinamos los métodos para construir un menú con bordes y opciones con puntos hasta el final.

Solución 11

```
for i in range(1, 6):  
    print(str(i).rjust(3))
```

Explicación: Cada número se convierte a string y se alinea a la derecha en un ancho de 3.

Solución 12

```
nombre = input("Nombre: ")  
apellido = input("Apellido: ")  
print(f"{nombre.ljust(15)} {apellido.rjust(15)}")
```

Explicación: Dos columnas de ancho fijo.

Solución 13

```
def encuadrar_titulo(titulo, ancho):  
    borde = "-" * ancho  
    titulo_centrado = titulo.center(ancho)  
    return f"{borde}\n{titulo_centrado}\n{borde}"  
  
print(encuadrar_titulo("MI PROGRAMA", 30))
```

Explicación: Creamos bordes con repetición de carácter y devolvemos el texto formateado.

Solución 14

```
texto = input("Texto: ")  
ancho = len(texto) * 2 + 2  
print(f"'{texto.center(ancho, ' ')}'")
```

Explicación: Calculamos el ancho dinámicamente y centramos con puntos.

Solución 15

```
nombres = input("Ingresa 3 nombres separados por coma: ").split(",")  
# Limpiamos espacios extra  
nombres = [n.strip() for n in nombres]  
for nombre in nombres:  
    lon = str(len(nombre))  
    print(f"{nombre.ljust(20)} {lon.rjust(10)}")
```

Explicación: Por cada nombre mostramos su texto a la izquierda y la longitud a la derecha.

Solución 16

```
frase = "Python es genial"  
palabras = frase.split()  
print(palabras[0].center(20))  
print(palabras[1].ljust(20))  
print(palabras[2].rjust(20))
```

Explicación: Cada palabra tiene un método distinto y se imprime en su línea.

Solución 17

```
palabra = input("Palabra: ")  
caracter = input("Caracter: ")  
print(palabra.center(30, caracter))
```

Explicación: Usamos el segundo argumento ingresado por el usuario.

Solución 18

```
print(".".center(40, "."))
print("TICKET".center(40))
print(".".center(40, "."))
items = [("Producto A", 1230), ("Producto B", 2450)]
total = 0
for item, precio in items:
    print(f"{item.ljust(30)} {str(precio).rjust(10)}")
    total += precio
print("-".center(40, "-"))
print(f"{'TOTAL'.ljust(30)} {str(total).rjust(10)}")
print("=".center(40, "="))
```

Explicación: Simula un ticket con encabezado, items, línea de total.

Solución 19

```
texto = input("Texto: ")
ancho = int(input("Ancho: "))
print("center:", texto.center(ancho, '-'))
print("ljust:", texto.ljust(ancho, '~'))
print("rjust:", texto.rjust(ancho, '#'))
```

Explicación: Mostramos las tres alineaciones con rellenos distintos.

Solución 20

```
def tabla_multiplicar(n, ancho=30):
    print(f"Tabla del {n}".center(ancho, "="))
    for i in range(1, 11):
        izq = str(n).ljust(4)
        medio = f"x {i}".center(8)
        der = f"={n*i}".rjust(8)
        print(izq + medio + der)
    print("=" * ancho)

tabla_multiplicar(5, 40)
```

Explicación: Dividimos el ancho en tres secciones y las concatenamos para armar la línea.

Ejemplo Integrador: Generador de Menús Profesionales

```
# =====
# GENERADOR DE MENU PROFESIONAL
# =====

def generar_menu(titulo, opciones, ancho=50):
    """Genera un menu formateado profesionalmente"""
    print("=" * ancho)
    print(titulo.upper().center(ancho))
    print("=" * ancho)

    for i, opcion in enumerate(opciones, 1):
        texto = f"{i}. {opcion}"
        print(texto.ljust(ancho, "."))

    print("=" * ancho)
    print("".rjust(ancho))

# Programa principal
print("=== GENERADOR DE MENUS ===")
titulo_menu = input("Titulo del menu: ")
print("\nIngresa las opciones (deja vacio para terminar):")

opciones_menu = []
while True:
    opcion = input(f"Opcion {len(opciones_menu) + 1}: ")
    if opcion == "":
        break
    opciones_menu.append(opcion)

if opciones_menu:
    generar_menu(titulo_menu, opciones_menu)
else:
    print("No se ingresaron opciones.")
```

Resumen del Capítulo

Dale, repasemos lo más importante:

- `center(longitud, carácter)`: centra el texto. Ej: `"Hola".center(10, " ")` → `-Hola--`.
- `ljust(longitud, carácter)`: alinea a la izquierda. Ej: `"Hola".ljust(10, " ")` → `"Hola----`.
- `rjust(longitud, carácter)`: alinea a la derecha. Ej: `"Hola".rjust(10, " ")` → `---Hola"`.
- El primer argumento es la longitud total deseada (debe ser mayor que la longitud original).
- El segundo argumento es opcional; si no se pone, rellena con espacios.
- Si la longitud pedida es menor o igual a la original, el método devuelve el string sin cambios.
- Para conservar el texto alineado en una variable, recordá reasignar: `texto = texto.center(20)`.
- Son ideales para menús, tablas, reportes y cualquier salida de texto que necesite formato.

center(), ljust() y rjust() son como herramientas de maquetación: te permiten ordenar el texto en la pantalla como si estuvieras armando un cartel o una tabla en un procesador de textos.

Método count() - Contar Subcadenas en un String

¿Qué es el método count()?

El método `count()` es una herramienta de Python que se utiliza para saber **cuántas veces aparece una subcadena** (un carácter o una frase) dentro de un texto principal.

Como decía Nahuel en la facultad (UBA Exactas): `count()` es como tener un contador automático: le pedís que busque una palabra o letra, y te dice exactamente cuántas veces aparece."

```
+-----+
|      |
| texto = "mi mamá me mima"      |
|      |
| texto.count("m") → 6            |
|      |
| Cuenta cuántas veces aparece la letra "m" |
| (distingue mayúsculas de minúsculas)    |
|      |
+-----+
```

Sintaxis y Argumentos

El método puede recibir desde **uno hasta tres argumentos**:

`variable.count("subcadena", inicio, fin)`

- **Subcadena (Obligatorio)**: El texto que querés buscar.
- **Inicio (Opcional)**: El índice desde donde querés empezar a buscar.
- **Fin (Opcional)**: El índice donde querés que termine la búsqueda (el índice `fin` no se incluye).

Comportamiento del conteo

Búsqueda básica (un argumento)

```
texto = "mi mamá me mima"
```

```
# Contar cuántas veces aparece la letra "m"
print(f"La letra 'm' aparece: {texto.count('m')} veces")
```

```
# Contar cuántas veces aparece la letra "a"
print(f"La letra 'a' aparece: {texto.count('a')} veces")
```

Salida:

La letra 'm' aparece: 6 veces

La letra 'a' aparece: 2 veces

Caso especial: cadena vacía

Un detalle curioso: si buscás una cadena vacía en un texto, el método te devolverá la cantidad de caracteres + 1.

```
texto = "Hola"
print(f"Buscando cadena vacía en '{texto}': {texto.count('')}")
```

Salida:

Buscando cadena vacía en 'Hola': 5

Nota: *¿Por qué? El método cuenta los .^{espacios}.º posiciones de memoria entre los caracteres (desde el extremo izquierdo del primero hasta el extremo derecho del último). Para un texto de 4 caracteres, hay 5 "huecos" donde podría ir una cadena vacía.*

Búsqueda con inicio (dos argumentos)

Podés indicar desde qué posición empezar a contar.

```
texto = "mi mamá me mima"

# Contar desde la posición 0 (por defecto)
print(f"Desde el inicio: {texto.count('m')}")

# Contar desde la posición 10
print(f"Desde la posición 10: {texto.count('m', 10)}")
```

Salida:

Desde el inicio: 6

Desde la posición 10: 1

Nota: *Posiciones:*

```
texto = "m i   m a m á   m e   m i m a"
índice: 0 1 2 3 4 5 6 7 8 9 10 11 12...
```

La letra 'm' en la posición 13 aparece después de los primeros 13 caracteres.

Búsqueda con rango (tres argumentos)

Podés especificar un rango exacto donde buscar (el índice **fin** no se incluye).

```
texto = "mi mamá me mima"

# Buscar solo entre las posiciones 3 y 7
print(f"Entre posiciones 3 y 7: {texto.count('m', 3, 7)}")
```

Salida:

Entre posiciones 3 y 7: 2

Consideraciones importantes

Case Sensitive (Distingue mayúsculas/minúsculas)

El método **distingue entre mayúsculas y minúsculas**. Si buscás "M" en "mi mamá", te devolverá 0 porque solo hay "m" minúsculas.

```
texto = "Mi Mamá Me Mima"

print(f"Buscando 'm' minúscula: {texto.count('m')}")
print(f"Buscando 'M' mayúscula: {texto.count('M')}")
```

Salida:

Buscando 'm' minúscula: 2

Buscando 'M' mayúscula: 3

Acentos

Los caracteres con tilde son diferentes para Python. "ma" no es lo mismo que "má".

```
texto = "mi mamá me mima"

print(f"Buscando 'ma': {texto.count('ma')}")
print(f"Buscando 'má': {texto.count('má')}")
```

Salida:

Buscando 'ma': 1

Buscando 'má': 1

Fuera de rango

Si ponés un índice de inicio muy grande (por ejemplo, 100 en una frase corta), Python no da error; simplemente devuelve 0 porque no hay nada que buscar ahí.

```
texto = "Hola mundo"
print(f"Buscando desde la posición 100: {texto.count('o', 100)}")
```

Salida:

Buscando desde la posición 100: 0

Índices negativos

Podés usar números negativos para contar desde el final del texto hacia atrás (-1 es el último carácter).

```
texto = "mi mamá me mima"

# Desde la posición -8 hasta el final
print(f"Desde posición -8: {texto.count('m', -8)}")

# Desde el inicio hasta la posición -5
print(f"Hasta posición -5: {texto.count('m', 0, -5)}")
```

Salida:

```
Desde posición -8: 2
Hasta posición -5: 4
```

Ejemplo práctico completo

```
# =====
# EJEMPLO COMPLETO CON count()
# =====

texto = "mi mamá me mima"

print("=" * 40)
print("MÉTODO COUNT()".center(40))
print("=" * 40)

print(f"\nTexto original: '{texto}'")
print(f"Longitud del texto: {len(texto)} caracteres\n")

# 1. Un argumento
print("--- 1 argumento ---")
print(f"Veces que aparece 'm': {texto.count('m')}")
print(f"Veces que aparece 'a': {texto.count('a')}")
print(f"Veces que aparece 'me': {texto.count('me')}")

# 2. Dos argumentos (con inicio)
print("\n--- 2 argumentos (inicio) ---")
print(f"Apariciones de 'm' desde posición 10: {texto.count('m', 10)}")

# 3. Tres argumentos (con rango)
print("\n--- 3 argumentos (rango) ---")
print(f"Apariciones de 'm' entre posiciones 3 y 7: {texto.count('m', 3, 7)}")

# 4. Distinción mayúsculas/minúsculas
print("\n--- Case sensitive ---")
texto_mayus = texto.upper()
```

```
print(f"Texto en mayúsculas: '{texto_mayus}'")
print(f"Buscando 'M' en mayúsculas: {texto_mayus.count('M')}")
```

Salida:

```
=====
                        MÉTODO COUNT()
=====

Texto original: 'mi mamá me mima'
Longitud del texto: 15 caracteres

--- 1 argumento ---
Veces que aparece 'm': 6
Veces que aparece 'a': 2
Veces que aparece 'me': 2

--- 2 argumentos (inicio) ---
Apariciones de 'm' desde posición 10: 1

--- 3 argumentos (rango) ---
Apariciones de 'm' entre posiciones 3 y 7: 2

--- Case sensitive ---
Texto en mayúsculas: 'MI MAMÁ ME MIMA'
Buscando 'M' en mayúsculas: 6
```

Ejemplo práctico: Analizando un texto

```
# =====
# ANALIZADOR DE TEXTO
# =====

print("\n=== ANALIZADOR DE TEXTO ===")
texto = input("Ingrese un texto: ")

print("\n--- ANÁLISIS ---")
print(f"Texto original: '{texto}'")
print(f"Longitud: {len(texto)} caracteres")

# Contar vocales
vocales = "aeiou"
for vocal in vocales:
    cantidad = texto.lower().count(vocal)
    if cantidad > 0:
        print(f"Vocal '{vocal}': {cantidad} veces")

# Contar espacios
espacios = texto.count(" ")
print(f"Espacios: {espacios}")
```

```
# Contar palabras (espacios + 1)
if len(texto) > 0:
    palabras = espacios + 1
    print(f"Cantidad aproximada de palabras: {palabras}")
```

Ejercicios Propuestos

Ejercicio 1

Contá cuántas veces aparece la letra `.a` en la frase "la casa es blanca".

Ejercicio 2

Contá cuántas veces aparece la palabra "mundo" en "hola mundo, mundo Python".

Ejercicio 3

Usando el texto "Python es genial, Python es poderoso", contá cuántas veces aparece "Python" desde la posición 10.

Ejercicio 4

En el texto "programacion", contá cuántas veces aparece la letra `.a` entre las posiciones 2 y 8.

Ejercicio 5

¿Qué devuelve `"Hola".count()`? Probá y explicá por qué.

Ejercicio 6

Contá cuántas veces aparece la letra "m" en "mi mamá me mima" pero distinguiendo mayúsculas (probá con "M").

Ejercicio 7

Contá cuántas veces aparece la letra `.e` desde la posición -10 hasta el final.

Ejercicio 8

Creá un programa que pida un texto al usuario y cuente cuántas vocales tiene.

Ejercicio 9

Creá un programa que pida un texto y una letra, y muestre cuántas veces aparece esa letra en el texto (sin importar mayúsculas).

Ejercicio 10

Creá un programa que cuente cuántas palabras tiene una frase (contando los espacios).

Manuel Villafañe - Python paso a paso

Soluciones Paso a Paso

Solución 1

```
texto = "la casa es blanca"
print(f"La letra 'a' aparece: {texto.count('a')} veces")
```

Solución 2

```
texto = "hola mundo, mundo Python"
print(f"La palabra 'mundo' aparece: {texto.count('mundo')} veces")
```

Solución 3

```
texto = "Python es genial, Python es poderoso"
print(f"Desde posición 10: {texto.count('Python', 10)} veces")
```

Solución 4

```
texto = "programacion"
print(f"Entre posiciones 2 y 8: {texto.count('a', 2, 8)}")
```

Solución 5

```
print(f"'Hola'.count(''): {'Hola'.count('')}")
# Devuelve 5 (longitud + 1). Cuenta las posiciones entre caracteres.
```

Solución 6

```
texto = "mi mamá me mima"
print(f"Minúscula 'm': {texto.count('m')}")
print(f"Mayúscula 'M': {texto.count('M')}")
```

Solución 7

```
texto = "mi mamá me mima"
print(f"Desde -10: {texto.count('e', -10)}")
```

Solución 8

```
texto = input("Ingrese un texto: ").lower()
vocales = "aeiou"
total = 0
for vocal in vocales:
    total = total + texto.count(vocal)
print(f"El texto tiene {total} vocales")
```


Solución 9

```
texto = input("Ingrese un texto: ")
letra = input("Ingrese una letra: ")
cantidad = texto.lower().count(letra.lower())
print(f"La letra '{letra}' aparece {cantidad} veces")
```

Solución 10

```
frase = input("Ingrese una frase: ").strip()
espacios = frase.count(" ")
if frase:
    palabras = espacios + 1
    print(f"La frase tiene aproximadamente {palabras} palabras")
else:
    print("No ingresaste ninguna palabra")
```

Ejemplo Integrador: Analizador de Texto Completo

```
# =====
# ANALIZADOR DE TEXTO COMPLETO
# =====

print("\n" + "=" * 50)
print("ANALIZADOR DE TEXTO".center(50))
print("=" * 50)

texto = input("\nIngrese un texto: ")

if not texto:
    print("No ingresaste ningún texto.")
else:
    print("\n" + "-" * 40)
    print("ESTADÍSTICAS BÁSICAS".center(40))
    print("-" * 40)

    print(f"Longitud del texto: {len(texto)} caracteres")
    print(f"Cantidad de palabras: {texto.count(' ') + 1}")
    print(f"Cantidad de espacios: {texto.count(' ')}")

    print("\n" + "-" * 40)
    print("CONTEO DE LETRAS".center(40))
    print("-" * 40)

    # Contar cada letra del alfabeto
    for letra in "abcdefghijklmnopqrstuvwxyz":
        cantidad = texto.lower().count(letra)
        if cantidad > 0:
            print(f"Letra '{letra}': {cantidad} veces")

    print("\n" + "-" * 40)
    print("CONTEO DE VOCALES".center(40))
    print("-" * 40)

    vocales = "aeiou"
    total_vocales = 0
    for vocal in vocales:
        cantidad = texto.lower().count(vocal)
        total_vocales = total_vocales + cantidad
        print(f"Vocal '{vocal}': {cantidad} veces")
    print(f"TOTAL DE VOCALES: {total_vocales}")

    print("\n" + "=" * 50)
```

Resumen del Capítulo

En este capítulo aprendiste:

- `count()` cuenta cuántas veces aparece una **subcadena** dentro de un string.
- **Sintaxis:** `texto.count(subcadena, inicio, fin)`
- **Argumentos:**
 - `subcadena` (obligatorio): lo que querés buscar.
 - `inicio` (opcional): desde dónde empezar a buscar.
 - `fin` (opcional): hasta dónde buscar (sin incluir).
- **Comportamiento clave:**
 - Distingue **mayúsculas** de **minúsculas**.
 - Distingue **acentos** (".a" "á").
 - Si el inicio es mayor que la longitud, devuelve 0.
 - Se pueden usar índices **negativos**.
 - Buscar cadena vacía devuelve `len(texto) + 1`.

count() es como un contador automático: le pedís que busque una palabra o letra, y te dice exactamente cuántas veces aparece en el texto.

Métodos `startswith()` y `endswith()` - Verificar el Inicio y Final de un String

¿Para qué sirven estos métodos?

Cuando empecé a manipular archivos, necesitaba separar los que eran imágenes de los que no. Abría el explorador, miraba las extensiones y pensaba: “¿No habrá una forma automática de preguntarle a Python si un archivo termina en `.jpg`?”. Y sí, la hay. Los métodos `startswith()` y `endswith()` son justo para eso: verificar si un texto empieza o termina con una subcadena determinada. Son como un portero que solo deja pasar a los que cumplen con el requisito de entrada o salida.

En criollo, `startswith()` pregunta “¿esto arranca con...?” y `endswith()` pregunta “¿esto termina con...?”. Los dos te devuelven `True` o `False`, y podés usarlos para validar formatos, filtrar listas o clasificar datos al vuelo.

```
+-----+
|      texto = "Diana se peina sola"      |
|      texto.startswith("Diana") → True   |
|      texto.endswith("sola") → True      |
|      ¡Devuelven True o False (booleanos)! |
+-----+
```

El Método `startswith()`

`startswith()` verifica si la cadena **empieza** con la subcadena que le indiques. Es case-sensitive, así que "Hola" y "hola" son distintos.

Sintaxis

```
variable.startswith("subcadena", inicio, fin)
```

- **subcadena** (obligatorio): el texto que querés buscar al comienzo.
- **inicio** (opcional): desde qué posición empezar a mirar.
- **fin** (opcional): hasta qué posición mirar (sin incluir este índice).

Si usás solo el primer argumento, examina el comienzo de toda la cadena. Si agregás **inicio**, examina a partir de esa posición. Con **fin**, examina dentro de un rango, verificando si en el **inicio de ese rango** aparece la subcadena.

Primeros ejemplos

```
texto = "Diana se peina sola"
```

```
print(f"¿Empieza con 'Diana'? {texto.startswith('Diana')}") # True
print(f"¿Empieza con 'D'? {texto.startswith('D')}")         # True
print(f"¿Empieza con 'diana'? {texto.startswith('diana')}") # False (case sensitive)
print(f"¿Empieza con 'a'? {texto.startswith('a')}")         # False
```

```
¿Empieza con 'Diana'? True
¿Empieza con 'D'? True
¿Empieza con 'diana'? False
¿Empieza con 'a'? False
```

Usando el argumento inicio

```
texto = "Diana se peina sola"
# Queremos saber si en la posición 6 (donde está 'se') empieza con "se"
print(texto.startswith("se", 6)) # True
print(texto.startswith("pe", 9)) # True (en posición 9 empieza "pe")
```

```
True
True
```

Reflexión: Acordate que los índices arrancan en cero. En "Diana se peina sola", la letra 's' de 'se' está en la posición 6.

Usando rango con inicio y fin

Cuando ponés los tres argumentos, `startswith()` examina el segmento desde **inicio** hasta **fin-1** y comprueba si ese segmento **empieza** con la subcadena. El **fin** no se incluye.

```
texto = "Diana se peina sola"
# Rango 6 a 8: solo contiene "se" (índices 6 y 7)
print(texto.startswith("se", 6, 8)) # True
# Rango 6 a 7: solo contiene "s"
print(texto.startswith("se", 6, 7)) # False
```

Índices negativos

Podés usar números negativos para contar desde el final. Por ejemplo, -10 se refiere al carácter que está a 10 posiciones del final.

```
texto = "Diana se peina sola"
# Desde la posición -10 hasta el final
print(texto.startswith("peina", -10)) # True
```

El Método endswith()

endswith() funciona igual que startswith(), pero verifica el **final** de la cadena o del segmento indicado. También es case-sensitive.

Sintaxis

```
variable.endswith("subcadena", inicio, fin)
```

Ejemplos básicos

```
texto = "Diana se peina sola"
```

```
print(f"¿Termina con 'sola'? {texto.endswith('sola')}") # True
print(f"¿Termina con 'a'? {texto.endswith('a')}")       # True
print(f"¿Termina con 'sol'? {texto.endswith('sol')}")   # True (porque 'sola' incluye
print(f"¿Termina con 'A'? {texto.endswith('A')}")       # False
```

Con inicio y fin

Al igual que con startswith(), podés restringir la verificación a un rango. endswith() revisa si el segmento **termina** con la subcadena.

```
texto = "Diana se peina sola"
# Del índice 9 al 14 (sin incluir 14) tenemos "peina" (índices 9-13)
print(texto.endswith("a", 9, 14)) # True, porque termina con 'a'
print(texto.endswith("s", 9, 14)) # False
```

Ejemplo práctico completo

Vamos a juntar los dos métodos en un mismo script para verlos en acción.

```
texto = "Diana se peina sola"

print("=" * 50)
print("MÉTODOS startswith() Y endswith().center(50))
print("=" * 50)

• print(f"\nTexto original: '{texto}'\n")

print("--- startswith() ---")
print(f"¿Empieza con 'Diana'? {texto.startswith('Diana')}")
print(f"¿Empieza con 'diana'? {texto.startswith('diana')}")
print(f"¿Empieza con 'se' desde posición 6? {texto.startswith('se', 6)}")
print(f"¿Empieza con 'se' entre 6 y 8? {texto.startswith('se', 6, 8)}")

print("\n--- endswith() ---")
print(f"¿Termina con 'sola'? {texto.endswith('sola')}")
```

```
print(f"¿Termina con 'sol'? {texto.endswith('sol')}")
print(f"¿Termina con 'a'? {texto.endswith('a')}")
print(f"¿En rango 9-14 termina con 'a'? {texto.endswith('a', 9, 14)}")
```

Ejemplos de la vida real

Validación de archivos por extensión

```
archivos = ["documento.pdf", "imagen.jpg", "script.py", "datos.csv", "video.mp4"]

print("=== FILTRADO DE ARCHIVOS ===")
```

```
# Solo PDF
print("\nArchivos PDF:")
for archivo in archivos:
    if archivo.endswith(".pdf"):
        print(f"  - {archivo}")

# Solo imágenes
print("\nArchivos de imagen:")
for archivo in archivos:
    if archivo.endswith((".jpg", ".png", ".gif")):
        print(f"  - {archivo}")
```

Nota: Podés pasar una tupla de extensiones a `endswith()` para verificar varias posibilidades a la vez.

Validación de nombres de variables

```
variables = ["mi_variable", "_privada", "123_variable", "nombre", "2numero"]

print("=== VALIDACIÓN DE NOMBRES DE VARIABLES ===")
print("(Las variables no pueden empezar con números)\n")

for var in variables:
    if var[0].isdigit():
        # empieza con número
        print(f" '{var}' → Inválida (empieza con número)")
    elif var.startswith("_"):
        print(f" '{var}' → Privada (empieza con _)")
    else:
        print(f" '{var}' → Válida")
```

Decoración con `rjust()`

Para darle un toque visual a la salida, podés usar `rjust()` que vimos antes. Por ejemplo:

```
print("Hola".rjust(50, '-'))
print("MÉTODO STARTSWITH".center(50))
```

```
print("FIN".rjust(50, '-'))
```

```
-----Hola
                MÉTODO STARTSWITH
-----FIN
```

Ejercicios propuestos

Llegó la hora de que vos practiques. Cada ejercicio está pensado para que uses `startswith()` y `endswith()` en situaciones reales.

1. Verificá si la cadena "Python es genial" empieza con "Python".
2. Verificá si la cadena "Python es genial" termina con "genial".
3. Verificá si "programacion" empieza con "pro" y termina con "cion".
4. Dada la cadena "hola mundo", comprobá si desde la posición 2 empieza con "la".
5. Dada la cadena "hola mundo", comprobá si en el rango 2 a 5 termina con "l".
6. Escribí un programa que pida una palabra y diga si empieza con vocal.
7. Escribí un programa que pida una frase y diga si termina con un signo de puntuación (., ?, !).
8. Filtra una lista de nombres y mostrá solo los que empiezan con 'A'.
9. Filtra una lista de archivos y mostrá solo los que son imágenes (.jpg, .png, .gif) usando una tupla.
10. Pedí un email y verificá si termina con "@gmail.com".
11. Verificá si la frase "Hola, ¿cómo estás?" empieza con mayúscula y termina con signo de interrogación.
12. Dada la lista ["manzana", "banana", "pera", "uva"], contá cuántas frutas terminan con "a".
13. Escribí una función `es_python(archivo)` que devuelva `True` si el nombre del archivo termina en `.py`.
14. Pedí una URL y mostrá si usa protocolo seguro (`https://`) o no.
15. Usá `startswith` con índice para ver si en la cadena `."bracadabra"` la subcadena `cad` empieza en la posición 4.
16. Comprobá si el string `."nana"` termina con `."na"` tanto si lo evaluás completo como si lo restringís a los primeros 4 caracteres.
17. Dado un número de teléfono como string ``5491134567890"`, verificá si empieza con ``54"`.

18. Creá un programa que reciba una ruta de archivo y muestre si es una ruta absoluta (empieza con "/" en Linux/Mac o con "C:\\" en Windows). Averiguá cómo detectar el sistema operativo.
19. Escribí un programa que lea una lista de comandos y ejecute los que empiezan con `run` (simulalo con `prints`).
20. Desafío: Sin usar `in`, determiná si una subcadena está contenida en un string usando `startswith` en un bucle (simulando `find`).

Soluciones paso a paso

Solución 1

```
texto = "Python es genial"
print(texto.startswith("Python")) # True
```

Explicación: La frase comienza exactamente con "Python".

Solución 2

```
texto = "Python es genial"
print(texto.endswith("genial")) # True
```

Explicación: La frase finaliza con "genial".

Solución 3

```
texto = "programacion"
print(texto.startswith("pro")) # True
print(texto.endswith("cion")) # True
```

Solución 4

```
texto = "hola mundo"
print(texto.startswith("la", 2)) # True
```

Explicación: Desde el índice 2, la subcadena es "la mundo", que empieza con "la".

Solución 5

```
texto = "hola mundo"
print(texto.endswith("l", 2, 5)) # False
# El rango 2-5 corresponde a "la " (índices 2,3,4) y termina con 'a', no con 'l'.
```

Solución 6

```
palabra = input("Ingrese una palabra: ").lower()
if palabra.startswith(("a","e","i","o","u")):
    print("Empieza con vocal")
else:
    print("No empieza con vocal")
```

Solución 7

```
frase = input("Ingrese una frase: ")
if frase.endswith((".", "?", "!", "")):
    print("Termina con signo de puntuación")
else:
    print("No termina con signo")
```

Solución 8

```
nombres = ["Ana", "Luis", "Andrea", "Pedro", "Alberto"]
for nombre in nombres:
    if nombre.startswith("A"):
        print(nombre)
```

Solución 9

```
archivos = ["foto.jpg", "video.mp4", "imagen.png", "doc.pdf"]
for archivo in archivos:
    if archivo.endswith((".jpg", ".png", ".gif")):
        print(archivo)
```

Solución 10

```
email = input("Email: ")
if email.endswith("@gmail.com"):
    print("Es de Gmail")
else:
    print("No es de Gmail")
```

Solución 11

```
frase = "Hola, ¿cómo estás?"
print(frase[0].isupper())           # True (empieza con mayúscula)
print(frase.endswith("?"))         # True (termina con ?)
```

Solución 12

```
frutas = ["manzana", "banana", "pera", "uva"]
cont = sum(1 for f in frutas if f.endswith("a"))
print(cont) # 3 (manzana, pera, uva)
```

Solución 13

```
def es_python(archivo):
    return archivo.endswith(".py")

print(es_python("script.py")) # True
print(es_python("app.exe"))   # False
```

Solución 14

```
url = input("URL: ")
if url.startswith("https://"):
    print("Protocolo seguro")
elif url.startswith("http://"):
    print("Protocolo HTTP")
```

```
else:
    print("Otro protocolo")
```

Solución 15

```
print("abracadabra".startswith("cad", 4)) # True (posición 4 es 'c')
```

Solución 16

```
texto = "anana"
print(texto.endswith("ana")) # True
print(texto.endswith("ana", 0, 4)) # True? "anan" termina con "an", no "ana" → False
# El rango 0-4 cubre "anan", termina con "n", por lo tanto False.
```

Solución 17

```
tel = "+5491134567890"
if tel.startswith("+54"):
    print("Número argentino")
else:
    print("Otro país")
```

Solución 18

```
import os
ruta = input("Ruta: ")
if os.name == "nt": # Windows
    if ruta.startswith("C:\\") or ruta.startswith("C:/"):
        print("Ruta absoluta en Windows")
    else:
        print("Ruta relativa")
else: # Unix/Linux/Mac
    if ruta.startswith("/"):
        print("Ruta absoluta en Unix")
    else:
        print("Ruta relativa")
```

Solución 19

```
comandos = ["runscript", "stop", "runbackup", "status"]
for cmd in comandos:
    if cmd.startswith("run"):
        print(f"Ejecutando: {cmd}")
    else:
        print(f"Ignorando: {cmd}")
```

Solución 20

```
texto = "abracadabra"
sub = "cad"
encontrado = False
for i in range(len(texto)):
    if texto.startswith(sub, i):
        encontrado = True
        break
print(encontrado)  # True
```

Ejemplo Integrador: Validador de URLs

```
urls = [  
    "https://www.google.com",  
    "http://python.org",  
    "ftp://archivos.uba.ar",  
    "www.ejemplo.com",  
    "https://github.com/Nahuel"  
]  
  
print("=== VALIDADOR DE URLs ===\n")  
for url in urls:  
    print(f"URL: {url}")  
    if url.startswith("https://"):  
        print(" Protocolo: HTTPS (seguro)")  
    elif url.startswith("http://"):  
        print(" Protocolo: HTTP")  
    elif url.startswith("ftp://"):  
        print(" Protocolo: FTP")  
    else:  
        print(" Protocolo no especificado")  
  
    if url.endswith(".com"):  
        print(" Dominio: .com")  
    elif url.endswith(".org"):  
        print(" Dominio: .org")  
    elif url.endswith(".ar"):  
        print(" Dominio: .ar")  
    print()
```

Resumen del capítulo

Dale, repasemos rápido lo importante:

- `startswith()` y `endswith()` verifican si un string empieza o termina con una subcadena.
- Devuelven `True` o `False`.
- Aceptan argumentos opcionales `inicio` y `fin` para acotar la búsqueda.
- Distinguen mayúsculas y minúsculas.
- Son muy útiles para validar formatos (emails, URLs, extensiones de archivo) y filtrar listas.
- Recordá que podés pasar una tupla de opciones a ambos métodos.

startswith() y endswith() son como revisar la primera y la última letra de una palabra. Te dicen si empieza o termina como esperabas.

La clase range() en Python

¿Qué es range()?

Cuando empecé a programar en Python, venía de otros lenguajes donde para hacer un bucle que contara hasta diez tenías que declarar una variable, inicializarla en cero y aumentarla a mano. Era un ritual mecánico. Cuando me mostraron `for i in range(10):` pensé: “¿Así de simple? ¿No falta nada?”. Y no, no faltaba nada. Python ya se encargaba de generar los números por mí.

La clase `range()` es una herramienta para **generar secuencias de números** de manera eficiente. A diferencia de una lista, no guarda todos los números en memoria; los va entregando uno por uno cuando se los pedís. En criollo, es como un surtidor de números: vos le decís cuántos querés y desde dónde, y él te los va dando de a uno, sin hacer bulto.

Analogía: Pensá en un pelotero automático de tenis. No necesita tener todas las pelotas en la cancha; las va lanzando de a una, a medida que las necesitás. `range()` hace lo mismo: genera el próximo número justo cuando el bucle lo pide, sin ocupar espacio de más.

+-----+	
range() -> genera secuencias de numeros	
Es MUY eficiente: no almacena todos los	
numeros a la vez, los genera bajo demanda	
Se usa MUCHO con bucles for	
+-----+	

Estructura y argumentos

`range()` puede recibir uno, dos o tres argumentos. Según cuántos le pases, se comporta distinto:

+-----+	
SINTAXIS	SIGNIFICADO
+-----+	
range(stop)	Genera desde 0 hasta stop-1 (paso 1)
range(start, stop)	Genera desde start hasta stop-1 (paso 1)
range(start, stop, step)	Genera desde start hasta stop-1,
	saltando de a "step"
+-----+	

El primer número (start) **siempre se incluye**; el último (stop) **nunca se incluye**. Esto al principio confunde, pero ya te vas a acostumbrar.

Un solo argumento: range(stop)

Cuando solo le pasás un número, este actúa como **límite superior** (stop). La secuencia arranca desde 0 y va sumando 1 hasta llegar a **stop - 1**.

- **Inicio:** siempre es 0.
- **Incremento:** siempre es 1.
- **Límite:** el número que pasaste nunca se incluye.

Ejemplo

```
# range(5) genera: 0, 1, 2, 3, 4
for i in range(5):
    print(i)
```

En la consola vas a ver:

```
0
1
2
3
4
```

Nota: *Importante: range(5) NO incluye el 5. La secuencia siempre es desde 0 hasta stop - 1.*

Ejercicio rápido: Ejecutá `for i in range(3): print(i)` y después `for i in range(10): print(i)`. Contá cuántos números imprime cada uno.

Más ejemplos con un argumento

```
# range(3) -> 0, 1, 2
for i in range(3):
    print(i)

# range(10) -> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
for i in range(10):
    print(i)
```

Dos argumentos: range(start, stop)

Con dos argumentos definís vos mismo desde dónde arrancar. El **start** se incluye, el **stop** no.

Ejemplo

```
# range(5, 11) genera: 5, 6, 7, 8, 9, 10
for i in range(5, 11):
    print(i)

5
6
7
8
9
10
```

Nota: `range(5, 11)` comienza en 5, termina en 10. El 11 no se incluye.

Más ejemplos con dos argumentos

```
# range(1, 6) -> 1, 2, 3, 4, 5
for i in range(1, 6):
    print(i)
```

```
# range(10, 15) -> 10, 11, 12, 13, 14
for i in range(10, 15):
    print(i)
```

Reflexión: `range(1, 6)` arranca en 1 y frena antes del 6. Son 5 números en total. ¿Se parece a los slices que vimos antes? Sí, exactamente la misma lógica.

Tres argumentos: `range(start, stop, step)`

El tercer argumento, **step** (paso), controla de a cuánto saltamos entre número y número. Puede ser positivo o negativo.

Ejemplo con step positivo (incremento)

```
# Numeros pares del 0 al 10
for i in range(0, 11, 2):
    print(i)

0
2
4
6
8
10
```

Más ejemplos con step positivo

```
# range(1, 20, 3) -> 1, 4, 7, 10, 13, 16, 19
for i in range(1, 20, 3):
    print(i)

# range(10, 21, 5) -> 10, 15, 20
for i in range(10, 21, 5):
    print(i)
```

Secuencias en decremento (cuenta regresiva)

Para ir hacia atrás, **start** debe ser **mayor** que **stop** y el **step** debe ser **negativo**. Si no cumpls estas dos condiciones, `range()` no da error, simplemente no genera nada (queda vacío).

Ejemplo de cuenta regresiva

```
# Cuenta regresiva para un lanzamiento
for i in range(10, 0, -1):
    print(i)
```

```
10
9
8
7
6
5
4
3
2
1
```

Nota: *Nota: El 0 no se incluye porque es el **stop**. Si querés incluir el 0, usá `range(10, -1, -1)`.*

Más ejemplos con step negativo

```
# range(20, 9, -2) -> 20, 18, 16, 14, 12, 10
for i in range(20, 9, -2):
    print(i)

# range(5, -1, -1) -> 5, 4, 3, 2, 1, 0
for i in range(5, -1, -1):
    print(i)
```

Ejercicio rápido: ¿Qué creés que pasa si hacés `range(0, 10, -1)`? Probá y sacá conclusiones.

¿Por qué range() es especial?

A diferencia de una lista común, `range()` no ocupa memoria para guardar todos los números. Solo almacena los parámetros (start, stop, step) y calcula el siguiente número cuando lo necesitás. Esto es clave cuando trabajás con rangos enormes.

```
+-----+
|
|  lista = [0, 1, 2, 3, 4, 5, ...] -> OCUPA      |
|  MEMORIA (almacena todo)                     |
|
|  range(1000000) -> NO OCUPA MEMORIA          |
|  (genera los numeros cuando los necesita)    |
|
|  ¡MUCHO MAS EFICIENTE!                      |
|
+-----+
```

Ejemplo de eficiencia

```
# Esto ocupa MUCHA memoria (no recomendado para rangos grandes)
lista_grande = list(range(1000000))

# Esto NO ocupa memoria (super eficiente)
for i in range(1000000):
    # procesar i
    pass
```

En mis primeras épocas, una vez creé una lista de millones de elementos y la máquina se quedó sin memoria. Un amigo me dijo: “¡Usá `range()` directamente en el for!”. Desde ese día, nunca más armé una lista innecesaria.

Convertir range() a lista

Si necesitás ver todos los números juntos, podés convertir el `range()` a lista con `list()`. Pero ojo, solo hazlo con rangos chicos para depurar o mostrar.

```
secuencia = list(range(5))
print(secuencia)  # [0, 1, 2, 3, 4]

pares = list(range(0, 11, 2))
print(pares)  # [0, 2, 4, 6, 8, 10]

regresiva = list(range(10, 0, -1))
print(regresiva)  # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Usos prácticos de range()

Repetir una acción N veces

```
# Imprimir "Hola" 5 veces
for i in range(5):
    print("Hola")
```

Recorrer índices de una lista

```
frutas = ["manzana", "pera", "uva", "sandia"]
for i in range(len(frutas)):
    print(f"Indice {i}: {frutas[i]}")
```

Generar números pares e impares

```
# Numeros pares
for i in range(0, 21, 2):
    print(i)
```

```
# Numeros impares
for i in range(1, 20, 2):
    print(i)
```

Sumar los primeros N números

```
n = 10
suma = 0
for i in range(n + 1): # n+1 para incluir el propio n
    suma += i
print(f"Suma de 0 a {n}: {suma}") # 55
```

Tabla resumen de range()

EJEMPLO	SECUENCIA
range(5)	0, 1, 2, 3, 4
range(2, 8)	2, 3, 4, 5, 6, 7
range(1, 10, 2)	1, 3, 5, 7, 9
range(0, 11, 2)	0, 2, 4, 6, 8, 10
range(10, 0, -1)	10, 9, 8, 7, 6, 5, 4, 3, 2, 1
range(5, -1, -1)	5, 4, 3, 2, 1, 0
range(20, 9, -2)	20, 18, 16, 14, 12, 10

Ejercicios propuestos

No saltees ninguno. La práctica con `range()` es clave para dominar los bucles.

1. Usá `range()` para imprimir los números del 0 al 9.
2. Usá `range()` para imprimir los números del 5 al 10.
3. Usá `range()` para imprimir los números pares del 0 al 20.
4. Usá `range()` para imprimir los números impares del 1 al 19.
5. Usá `range()` para imprimir una cuenta regresiva del 10 al 1.
6. Usá `range()` para imprimir los múltiplos de 3 del 0 al 30.
7. Usá `range()` para imprimir los números del 50 al 40 (descendente).
8. Usá `range()` para imprimir los números del 1 al 100 de 10 en 10.
9. Creá una lista con `range(1, 11)` y mostrala.
10. Creá una lista con los números pares del 2 al 20 usando `range()`.
11. Usá `range()` para sumar los números del 1 al 100.
12. Usá `range()` para imprimir “Hola” 7 veces.
13. Usá `range()` para imprimir los números del 0 al 10 en orden inverso.
14. Usá `range()` para imprimir los números negativos del -1 al -10.
15. Creá una lista con los cuadrados de los números del 1 al 10 usando `range()`.
16. Usá `range()` para sumar solo los números pares del 1 al 50.
17. Usá `range()` para imprimir los números del 100 al 200 que sean múltiplos de 20.
18. Usá `range()` con step negativo para imprimir 10, 7, 4, 1.
19. Creá una lista con los primeros 10 números impares usando `range()`.
20. Usá `range()` para calcular el factorial de 5 ($1 \times 2 \times 3 \times 4 \times 5$).

Soluciones paso a paso

Solución 1

```
for i in range(10):  
    print(i)
```

Explicación: range(10) va de 0 a 9. Se imprimen diez números.

Solución 2

```
for i in range(5, 11):  
    print(i)
```

Explicación: Empieza en 5, termina en 10. El 11 no se incluye.

Solución 3

```
for i in range(0, 21, 2):  
    print(i)
```

Explicación: Paso 2 desde 0 hasta 20 inclusive.

Solución 4

```
for i in range(1, 20, 2):  
    print(i)
```

Explicación: Paso 2 empezando en 1 genera todos los impares.

Solución 5

```
for i in range(10, 0, -1):  
    print(i)
```

Explicación: Paso negativo -1 desde 10 bajando hasta 1.

Solución 6

```
for i in range(0, 31, 3):  
    print(i)
```

Explicación: Paso 3, el stop es 31 para que incluya al 30.

Solución 7

```
for i in range(50, 39, -1):  
    print(i)
```

Explicación: Va de 50 a 40 descendente. El stop 39 no se incluye.

Solución 8

```
for i in range(1, 101, 10):  
    print(i)
```

Explicación: Salta de a 10: 1, 11, 21, ..., 91. ¿100? No, 101 no incluye el 100 porque arranca en 1, no en 0. Si quisieras incluir 100, podrías hacer `range(0, 101, 10)`.

Solución 9

```
lista = list(range(1, 11))  
print(lista)
```

Explicación: `list()` convierte el rango en lista.

Solución 10

```
pares = list(range(2, 21, 2))  
print(pares)
```

Explicación: Desde 2 al 20 inclusive, paso 2.

Solución 11

```
suma = 0  
for i in range(1, 101):  
    suma += i  
print(suma) # 5050
```

Explicación: Acumulamos todos los números del 1 al 100.

Solución 12

```
for i in range(7):  
    print("Hola")
```

Explicación: La variable `i` no se usa; solo repetimos la acción 7 veces.

Solución 13

```
for i in range(10, -1, -1):  
    print(i)
```

Explicación: De 10 a 0 inclusive. Fijate que el stop es -1 para que incluya el 0.

Solución 14

```
for i in range(-1, -11, -1):  
    print(i)
```

Explicación: Empieza en -1, termina en -10. El stop -11 no se incluye.

Solución 15

```
cuadrados = []
for i in range(1, 11):
    cuadrados.append(i ** 2)
print(cuadrados)
```

Explicación: Calculamos el cuadrado de cada i y lo guardamos en una lista.

Solución 16

```
suma = 0
for i in range(2, 51, 2):
    suma += i
print(suma) # 650
```

Explicación: Recorremos solo los pares desde 2 hasta 50 inclusive.

Solución 17

```
for i in range(100, 201, 20):
    print(i)
```

Explicación: 100, 120, 140, 160, 180, 200. El 201 no se incluye.

Solución 18

```
for i in range(10, 0, -3):
    print(i)
```

Explicación: 10 menos 3 es 7, menos 3 es 4, menos 3 es 1. El 0 no se incluye.

Solución 19

```
impares = list(range(1, 20, 2))
print(impares) # [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

Explicación: Los primeros 10 impares se generan con start=1, step=2, stop=20.

Solución 20

```
factorial = 1
for i in range(1, 6):
    factorial *= i
print(factorial) # 120
```

Explicación: Multiplicamos $1 \times 2 \times 3 \times 4 \times 5 = 120$.

Resumen del capítulo

Dale, repasemos lo importante:

- `range(stop)`: 0 hasta `stop-1`, paso 1.
- `range(start, stop)`: `start` hasta `stop-1`, paso 1.
- `range(start, stop, step)`: `start` hasta `stop-1`, con paso `step`.
- Para secuencias **decrecientes**: `start` > `stop` y `step` negativo.
- `range()` no crea una lista, es un objeto que genera números bajo demanda. ¡Ultra eficiente!
- Se lleva de maravillas con el bucle `for` y es la base para repetir tareas o generar secuencias.

range() es como un generador de números bajo pedido: no ocupa espacio hasta que lo necesitas, y te da los números uno por uno cuando los pedís.

El Poder del Bucle for

¿Por qué necesitamos el for?

Cuando empecé a programar en serio, mi primer encuentro con la repetición fue el **while**. Lo usaba para todo: contar, recorrer listas, leer archivos. Pero cada vez que necesitaba hacer algo una cantidad fija de veces, terminaba escribiendo un contador, una condición y un incremento manual. Era como caminar con muletas cuando podías ir en bicicleta.

Un día, en la facultad (UBA Exactas), un ayudante me mostró el bucle **for** y me dijo: “Che, Nahuel, ¿para qué sufrís con el **while** si tenés esto?”. Me quedé boquiabierto. En dos líneas hacía lo mismo que yo en cinco. Desde entonces, el **for** se convirtió en mi herramienta favorita para recorrer secuencias.

Analogía: Imaginá que tenés una pila de carpetas y querés revisarlas una por una. Con **while** vos mismo tenés que agarrar cada carpeta, fijarte si es la última y pasar a la siguiente. Con **for** es como si un asistente automático te fuera alcanzando cada carpeta una tras otra, sin que vos tengas que controlar nada.

En este capítulo vas a dominar el bucle **for** y su fiel compañero **range()** para que puedas repetir tareas de manera limpia y elegante.

La Estrella: La Función range()

El secreto del **for** en Python es la **función range()**. Es ella quien genera la secuencia de números sobre la que el bucle itera. No guarda todos los números en memoria, los va produciendo a medida que se necesitan. En criollo, es como un surtidor de números bajo demanda.

range() puede recibir uno, dos o tres argumentos:

- **range(stop)**: Empieza en 0 y termina en **stop - 1**. El **stop** nunca se incluye.
- **range(start, stop)**: Empieza en **start** y termina en **stop - 1**.
- **range(start, stop, step)**: Empieza en **start**, termina en **stop - 1**, y avanza de a **step** pasos. Si el **step** es negativo, la secuencia va hacia atrás.

Nota: Pensá en **range()** como una cinta transportadora que entrega números. El bucle **for** es el operario que toma cada número y hace algo con él. No importa cuán grande sea el rango, la cinta ocupa casi nada de espacio.

Ejemplos básicos con range()

```
# range(stop): imprime 0, 1, 2, 3, 4
print("range(5):")
for numero in range(5):
    print(f"Numero: {numero}")

# range(start, stop): imprime 5, 6, 7, 8, 9
print("\nrange(5, 10):")
for valor in range(5, 10):
    print(f"Valor: {valor}")

# range(start, stop, step): imprime 0, 2, 4, 6, 8
print("\nrange(0, 10, 2):")
for par in range(0, 10, 2):
    print(f"Numero par: {par}")
```

Reflexión: Fijate que `range(5)` imprime cinco números, del 0 al 4. ¿Por qué? Porque en Python los índices siempre arrancan en 0. Ya te vas a acostumbrar.

Más Allá de los Números: Iterar sobre Colecciones

Lo que vuelve realmente poderoso al `for` es que no se limita a números. Podés usarlo para recorrer directamente cualquier **iterable**: listas, tuplas, strings, diccionarios y más. No necesitás andar manejando índices a mano.

Recorriendo una lista

Cuando empecé a trabajar con listas de nombres, el `for` me salvó. En lugar de andar con `while` y `i`, podía hacerlo así:

```
frutas = ['Manzana', 'Banana', 'Pera', 'Kiwi', 'Naranja']
print("Mi lista de frutas:")
for fruta in frutas:
    print(fruta)
```

```
Manzana
Banana
Pera
Kiwi
Naranja
```

Ejercicio rápido: Cambiá la lista por `['Mate', 'Termo', 'Bombilla']` y ejecutá el código. ¿Ves cómo se adapta solo?

Recorriendo un string carácter por carácter

También podés tratar un string como una secuencia de caracteres. Por ejemplo, para analizar un nombre letra por letra:

```
nombre = "Nahuel"
print(f"Analizando el nombre '{nombre}':")
for letra in nombre:
    print(f"Letra encontrada: {letra}")
```

```
Letra encontrada: N
Letra encontrada: a
Letra encontrada: h
Letra encontrada: u
Letra encontrada: e
Letra encontrada: l
```

Un Primer Proyecto: Calculando Promedios

Una de las primeras aplicaciones útiles que hice fue un calculador de promedios para las notas de la facultad. Usando `for` y `range()`, el código queda clarísimo.

```
suma_notas = 0
cantidad_notas = 5

print(f"Vamos a calcular el promedio de {cantidad_notas} notas.")
for i in range(cantidad_notas):
    nota = float(input(f"Ingrese la nota {i+1}: "))
    suma_notas = suma_notas + nota

promedio = suma_notas / cantidad_notas
print(f"El promedio final es: {promedio:.2f}")

if promedio >= 7:
    print("¡Felicitaciones! Aprobaste la materia.")
else:
    print("No alcanzaste el 7. ¡A estudiar mas para el recuperatorio!")
```

Fijate que usé `i+1` para mostrar el número de nota de forma humana (1 a 5), aunque internamente `i` vaya de 0 a 4. Es un truquito que vas a usar muchas veces.

Bucles Anidados: for dentro de for

Cuando necesitás repetir algo dentro de otra repetición, ponés un `for` adentro de otro. El ejemplo clásico son las tablas de multiplicar.

```
print("=== TABLAS DE MULTIPLICAR ===")
for tabla in range(2, 6):
    print(f"\nTabla del {tabla}:")
    for multiplicador in range(1, 11):
        resultado = tabla * multiplicador
        print(f"{tabla} x {multiplicador} = {resultado}")
```

Nota: La indentación (los espacios al inicio de cada línea) es sagrada en Python. El segundo `for` está adentro del primero porque tiene más espacios. Si te confundís, el programa no funciona o hace cualquier cosa.

Reflexión: Al principio, los bucles anidados marean un poco. Te recomiendo que agarres papel y lápiz y sigas la ejecución paso a paso para un valor chico de tabla (por ejemplo, 2). Así entendés el flujo.

Ejercicios Propuestos

Ahora la parte más importante: que vos practiques. Tomate tu tiempo, no copies y pegues. Escribí cada uno a mano, ejecutalo y corregí los errores.

1. Imprimí los números del 1 al 20 usando un bucle `for` y `range()`.
2. Dada la lista `ciudades = ["Buenos Aires", "Córdoba", "Rosario", "Mendoza"]`, imprimí cada ciudad con un mensaje como "Visité la ciudad de Buenos Aires".
3. Pedile al usuario una palabra y, con un `for`, recorré sus letras. Si encontrás una letra 'a', mostrá "¡Encontré la letra 'a'!" y detené el bucle con `break`.
4. Sumá todos los números pares del 1 al 100 usando un bucle `for` y un `if`.
5. Escribí un programa que pida 5 precios de productos y calcule el total de la compra.
6. Generá una lista con los cuadrados de los primeros 10 números naturales (1 al 10) usando un `for` y el método `.append()`. Mostrala.
7. Escribí un programa que cuente cuántas vocales tiene una frase ingresada por el usuario.
8. Imprimí la siguiente figura usando bucles `for` anidados:

```
*
* *
* * *
* * * *
* * * * *
```

9. Mostrá los números del 10 al 1 en orden descendente usando `for` y `range()`.
10. Dada la lista `numeros = [4, 2, 9, 7, 5, 1]`, encontrá el número más grande (sin usar `max()`) recorriéndola con `for`.
11. Pedí un número entero `n` y mostrá su factorial ($n! = 1*2*\dots*n$) usando un `for`.
12. Creá un programa que pida una frase y muestre cuántos espacios en blanco contiene.
13. Dado un string `palabra = "programacion"`, mostrá cada letra junto con su índice usando `for` y `range(len(...))`.
14. Escribí un programa que muestre todos los múltiplos de 7 entre 1 y 100 usando `for` y `range` con paso.

15. Pedí al usuario 6 números y, usando un `for`, guardalos en una lista. Luego mostraré la lista y su longitud.
16. Imprimí un cuadrado de asteriscos de tamaño 5x5 usando bucles anidados.
17. Recorré una lista de nombres e imprimí solo aquellos que tengan más de 5 letras.
18. Simulé una cuenta regresiva desde 10 hasta 0 imprimiendo también “¡Despegue!” al final.
19. Pedí una palabra e imprimí sus letras una por línea, pero saltando las vocales (usando `continue`).
20. Combiné dos listas: `nombres = [".na", "Luis"]` y `edades = [25, 30]`, e imprimí “Ana tiene 25 años”, “Luis tiene 30 años” usando un solo `for` y `range(len(...))`.

Soluciones Paso a Paso

Solución 1

```
for i in range(1, 21):  
    print(i)
```

Explicación: range(1, 21) incluye el 1 y llega hasta el 20. El 21 no se muestra.

Solución 2

```
ciudades = ["Buenos Aires", "Cordoba", "Rosario", "Mendoza"]  
for ciudad in ciudades:  
    print(f"Visite la ciudad de {ciudad}")
```

Explicación: El for recorre directamente los elementos de la lista, sin necesidad de índices.

Solución 3

```
palabra = input("Ingrese una palabra: ")  
for letra in palabra:  
    if letra == 'a':  
        print("¡Encontre la letra 'a'!")  
        break
```

Explicación: break detiene el bucle inmediatamente al encontrar la primera 'a'.

Solución 4

```
suma = 0  
for i in range(1, 101):  
    if i % 2 == 0:  
        suma += i  
print(f"Suma de pares: {suma}") # 2550
```

Explicación: El operador % obtiene el resto. Si $i \% 2 == 0$, es par.

Solución 5

```
total = 0  
for i in range(5):  
    precio = float(input(f"Precio del producto {i+1}: "))  
    total += precio  
print(f"Total de la compra: ${total:.2f}")
```

Explicación: Pedimos 5 precios y los vamos sumando.

Solución 6

```
cuadrados = []
for i in range(1, 11):
    cuadrados.append(i ** 2)
print(cuadrados) # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Explicación: Creamos una lista vacía y le agregamos cada cuadrado con `append()`.

Solución 7

```
frase = input("Ingrese una frase: ").lower()
vocales = "aeiou"
contador = 0
for letra in frase:
    if letra in vocales:
        contador += 1
print(f"La frase tiene {contador} vocales")
```

Explicación: Pasamos a minúsculas para simplificar la comparación.

Solución 8

```
for i in range(1, 6):
    for j in range(i):
        print('*', end=' ')
    print()
```

Explicación: El `for` externo controla las filas (1 a 5). El interno imprime `i` asteriscos. `end=' '` evita el salto de línea, y `print()` vacío al final de la fila hace el salto.

Solución 9

```
for i in range(10, 0, -1):
    print(i)
```

Explicación: `range(10, 0, -1)` arranca en 10, termina antes de 0 (en 1) con paso -1.

Solución 10

```
numeros = [4, 2, 9, 7, 5, 1]
maximo = numeros[0]
for n in numeros:
    if n > maximo:
        maximo = n
print(f"El maximo es: {maximo}") # 9
```

Explicación: Inicializamos `maximo` con el primer elemento y lo actualizamos si encontramos algo mayor.

Solución 11

```
n = int(input("Ingrese un numero: "))
factorial = 1
for i in range(1, n+1):
    factorial *= i
print(f"{n}! = {factorial}")
```

Explicación: Multiplicamos desde 1 hasta n inclusive.

Solución 12

```
frase = input("Frase: ")
espacios = 0
for car in frase:
    if car == ' ':
        espacios += 1
print(f"Espacios en blanco: {espacios}")
```

Explicación: Comparamos cada carácter con un espacio.

Solución 13

```
palabra = "programacion"
for i in range(len(palabra)):
    print(f"Indice {i}: {palabra[i]}")
```

Explicación: `range(len(palabra))` nos da los índices de 0 hasta la longitud-1.

Solución 14

```
for i in range(7, 101, 7):
    print(i)
```

Explicación: Empezamos en 7 y damos pasos de 7 hasta 100 inclusive.

Solución 15

```
numeros = []
for i in range(6):
    num = int(input(f"Numero {i+1}: "))
    numeros.append(num)
print("Lista:", numeros)
print("Longitud:", len(numeros))
```

Explicación: Acumulamos en una lista vacía usando `append()`.

Solución 16

```
for fila in range(5):
    for col in range(5):
        print('*', end=' ')
    print()
```

Explicación: Cinco filas, en cada una imprimimos cinco asteriscos separados por espacio.

Solución 17

```
nombres = ["Ana", "Federico", "Luis", "Mariana", "Pepe"]
for nombre in nombres:
    if len(nombre) > 5:
        print(nombre)
```

Explicación: len() devuelve la cantidad de caracteres. Solo imprimimos los que pasan el filtro.

Solución 18

```
for i in range(10, -1, -1):
    print(i)
print(";Despegue!")
```

Explicación: Bajamos de 10 a 0 inclusive, con paso -1.

Solución 19

```
palabra = input("Palabra: ")
for letra in palabra:
    if letra.lower() in "aeiou":
        continue
    print(letra)
```

Explicación: continue salta a la siguiente iteración sin imprimir la vocal.

Solución 20

```
nombres = ["Ana", "Luis"]
edades = [25, 30]
for i in range(len(nombres)):
    print(f"{nombres[i]} tiene {edades[i]} años")
```

Explicación: Usamos el índice i para acceder simultáneamente a ambas listas.

Resumen del Capítulo

En este capítulo aprendiste:

- El bucle `for` recorre automáticamente cualquier secuencia (listas, strings, `range()`, etc.).
- La función `range()` es ideal para generar secuencias numéricas de manera eficiente.
- Podés anidar bucles `for` para trabajar con dos o más dimensiones.
- `break` detiene el bucle inmediatamente; `continue` salta a la siguiente iteración.
- Recorrer una lista por sus elementos directamente es más limpio y pitónico que usar índices, aunque a veces necesitás ambos.

El bucle for es como un asistente infatigable: recorre lo que le pidas sin quejarse y sin errores de conteo. ¡Aprovechalo!

Bucles Anidados: While dentro de While, While con For y While con Listas

Introducción

Cuando empecé a programar, después de dominar el `while` y el `for` por separado, me creía que podía resolver cualquier cosa. Hasta que me pidieron que arme un juego de batalla por turnos: cada turno había que repetir acciones mientras los dos personajes estuvieran vivos, y adentro de cada turno, a veces quería mostrar un menú de opciones con una cantidad fija de ítems. Me quedé trabado. Un compañero me dijo: “¿Probaste poner un `for` adentro del `while`?”. No lo había pensado. Era el famoso **bucle anidado**. Y ahí se me abrió un mundo nuevo.

Un **bucle anidado** es sencillamente un bucle dentro de otro. El bucle exterior se ejecuta una vez y, por cada vuelta de él, el bucle interior se ejecuta **completo**, desde el inicio hasta el final. Es como un reloj: la manecilla de los minutos da una vuelta entera por cada tictac de la manecilla de las horas.

En este capítulo vamos a combinar `while` con `while`, `while` con `for` y `while` con listas. Son herramientas que te van a permitir crear programas mucho más interesantes y reales. Mirá este esquema para que lo visualices:

```
+-----+
|      |
| BUCLE EXTERIOR      |
| while condicion1:   |
|     # codigo exterior |
|     while condicion2: # BUCLE INTERIOR |
|         # codigo interior |
|         # mas codigo exterior |
|      |
| El bucle interior se ejecuta COMPLETO |
| por CADA vuelta del bucle exterior  |
|      |
+-----+
```

While anidado (while dentro de while)

El `while` anidado es ideal cuando necesitamos repetir una tarea una cantidad de veces que depende de un estado, y dentro de esa tarea, a su vez, repetir algo que también depende de otro estado.

Ejemplo básico

El siguiente programa muestra cómo el bucle exterior cuenta del 1 al 3, y el interior, del 1 al 2. Fíjate que por cada número exterior se imprimen dos números interiores.

```
i = 1
while i <= 3:
    print(f"Vuelta exterior {i}")
    j = 1
    while j <= 2:
        print(f"    Interior {j}")
        j = j + 1
    i = i + 1
```

```
Vuelta exterior 1
    Interior 1
    Interior 2
Vuelta exterior 2
    Interior 1
    Interior 2
Vuelta exterior 3
    Interior 1
    Interior 2
```

Reflexión: Si te olvidás de incrementar la variable interior `j`, entrarás en un bucle infinito que te va a volver loco. ¡A todos nos pasó!

Tabla de multiplicar completa

Un clásico: generar las tablas del 1 al 5 usando dos `while`. El exterior recorre las tablas, el interior los multiplicadores del 1 al 10.

```
i = 1
while i <= 5:
    print(f"--- Tabla del {i} ---")
    j = 1
    while j <= 10:
        print(f"{i} x {j} = {i * j}")
        j = j + 1
    print()
    i = i + 1
```

Triángulo de asteriscos

Vamos a dibujar un triángulo de altura 5 usando `while` anidados. Por cada fila, imprimimos tantos asteriscos como el número de fila.

```
fila = 1
while fila <= 5:
    columna = 1
```

```
while columna <= fila:
    print("*", end="")
    columna = columna + 1
print()
fila = fila + 1
```

```
*
**
***
****
*****
```

Ejercicio rápido: Cambiá la condición para que en vez de asteriscos imprima números consecutivos en cada fila, o para que el triángulo salga invertido. Jugá un rato.

While con for combinados

A veces el exterior necesita repetir hasta que el usuario se canse, pero una tarea interna tiene un número fijo de pasos. Ahí combinamos `while` (no sabemos cuántas vueltas) con `for` (sí sabemos cuántas vueltas dentro).

Pedir números y mostrar sus tablas

El usuario ingresa números hasta que pone 0. Para cada número, mostramos su tabla del 1 al 10 con un `for`.

```
num = int(input("Numero (0 para terminar): "))
while num != 0:
    print(f"--- Tabla del {num} ---")
    for i in range(1, 11):
        print(f"{num} x {i} = {num * i}")
    print()
    num = int(input("Otro numero (0 para terminar): "))
print("¡Chau!")
```

Sistema de login con reintentos y menú

Un ejemplo de la vida real: un login con hasta 3 intentos. Si se logra, mostramos un menú fijo usando `for`.

```
usuario_correcto = "admin"
contrasena_correcta = "1234"
intentos = 0

while intentos < 3:
    usuario = input("Usuario: ")
    contrasena = input("Contrasena: ")

    if usuario == usuario_correcto and contrasena == contrasena_correcta:
```

```

    print(";Bienvenido!")
    print("\n--- MENU ---")
    for opcion in range(1, 4):
        print(f"{opcion}. Opcion {opcion}")
        break
    else:
        intentos = intentos + 1
        print(f"Error. Intentos restantes: {3 - intentos}")

if intentos == 3:
    print("Cuenta bloqueada")

```

Analogía: El `while` es como el portero del edificio, que deja intentar entrar varias veces. El `for` es como el ascensor una vez adentro: siempre tenés los mismos pisos para elegir.

While con listas

Las listas son la compañera perfecta del `while` cuando querés procesar elementos hasta que se cumpla alguna condición (por ejemplo, vaciar una lista, buscar y eliminar todos los que cumplan algo, o recorrer mientras quede algo).

Vaciar una lista con `pop()`

```

numeros = [1, 2, 3, 4, 5]
print("Lista original:", numeros)

while len(numeros) > 0:
    eliminado = numeros.pop()
    print(f"Eliminando: {eliminado}")
print("Lista vacia:", numeros)

```

Reflexión: `pop()` saca el último elemento. Si usáramos `pop(0)` sacaría el primero, pero sería más lento. Por suerte no nos importa mientras aprendemos.

Recorrer una lista con `while` e índice

```

compras = ["pan", "leche", "huevos", "carne"]
indice = 0
while indice < len(compras):
    print(f"Comprando: {compras[indice]}")
    indice = indice + 1
print(";Lista completada!")

```

Buscar y eliminar todas las ocurrencias

Un clásico: queremos sacar todas las apariciones de un elemento de una lista. Con `while` y el operador `in` queda muy limpio.


```
nombres = ["Ana", "Luis", "Pedro", "Ana", "Maria", "Ana"]
print("Lista original:", nombres)

a_eliminar = input("Nombre a eliminar: ")
while a_eliminar in nombres:
    nombres.remove(a_eliminar)
    print(f"Se elimino '{a_eliminar}'. Lista: {nombres}")

print(f"Lista final sin '{a_eliminar}': {nombres}")
```

Ejercicio rápido: ¿Qué pasaría si en lugar de `while a_eliminar in nombres` usaras un `for` que recorre la lista? Probá y fijate por qué no funciona bien (pista: modificar una lista mientras la recorrés es mala idea).

Ejemplos integradores: Programas que armé en mis primeras prácticas

Acá te voy a mostrar fragmentos reales de programas que hice cuando estaba en la facultad (UBA Exactas). Son imperfectos, pero me sirvieron un montón.

Simulador de carrito de compras

Un menú que se repite hasta que el usuario elige salir. Se agregan productos a una lista y se actualiza el total.

```
lista_de_compra = []
total = 0

while True:
    op = input("1.Pan 2.Carne 9.Ver total 0.Salir: ")
    if op == '1':
        lista_de_compra.append('pan')
        total = total + 2599
        print(f"Total: {total}")
    elif op == '2':
        lista_de_compra.append('carne')
        total = total + 30000
        print(f"Total: {total}")
    elif op == '9':
        print(f"Importe: {total}")
    elif op == '0':
        print("¡Adios!")
        break
```

Piedra, papel o tijera al mejor de 3

Un `while` que repite rondas mientras nadie haya ganado 3 partidas. Dentro usamos un poco de lógica con `if`.

```
import random
formas = ['piedra', 'papel', 'tijera']
ganadas_jugador = 0
ganadas_cpu = 0
ronda = 1

while ganadas_jugador < 3 and ganadas_cpu < 3:
    print(f"\n--- RONDA {ronda} ---")
    jugador = input("Elegi piedra, papel o tijera: ").lower()
    cpu = random.choice(formas)
    print(f"CPU: {cpu}")

    if jugador == cpu:
        print("Empate")
    elif (jugador == 'piedra' and cpu == 'tijera') or \
         (jugador == 'papel' and cpu == 'piedra') or \
         (jugador == 'tijera' and cpu == 'papel'):
        print("Ganaste esta ronda")
        ganadas_jugador = ganadas_jugador + 1
    else:
        print("Perdiste esta ronda")
        ganadas_cpu = ganadas_cpu + 1
    ronda = ronda + 1

print(f"\nResultado: Jugador {ganadas_jugador} - CPU {ganadas_cpu}")
```

Batalla por turnos

Un combate de fantasía donde mientras ambos tengan vida >0, se ejecutan turnos con un menú de ataque o defensa. Adentro hay condicionales y cálculos aleatorios.

```
import random
vida_jugador = 100
vida_monstruo = 100
turno = 1

print("BATALLA: JUGADOR VS MONSTRUO")
while vida_jugador > 0 and vida_monstruo > 0:
    print(f"\nTurno {turno} - Vida jugador: {vida_jugador}, monstruo: {vida_monstruo}")
    accion = input("Atacar o defender? ").lower()
    ataque_jugador = random.randint(10, 30)
    ataque_monstruo = random.randint(5, 25)

    if accion == 'atacar':
        vida_monstruo = vida_monstruo - ataque_jugador
        print(f"Atacaste! Daño: {ataque_jugador}")
        vida_jugador = vida_jugador - ataque_monstruo
        print(f"El monstruo te pego: {ataque_monstruo}")
    elif accion == 'defender':
```

```

        vida_jugador = vida_jugador - (ataque_monstruo // 2)
        print(f"Te defendiste. Recibiste: {ataque_monstruo // 2}")
    turno = turno + 1

if vida_jugador <= 0:
    print(";GAME OVER!")
else:
    print(";VICTORIA!")

```

Mirá cómo se combinan: el `while` principal por turnos, dentro el manejo de la acción y los cálculos. Es una base para cualquier juego sencillo.

Ejercicios propuestos

Ahora es tu turno de ensuciarte las manos. Intentá resolver cada uno antes de mirar las soluciones. Todos usan alguna forma de bucle anidado, combinación `while/for` o `while` con listas.

1. Usá un `while` anidado para mostrar un rectángulo de asteriscos de 3 filas y 5 columnas.
2. Usá un `while` anidado para mostrar la figura:

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

3. Escribí un programa que pida números al usuario hasta que ingrese 0, y para cada número muestre su tabla de multiplicar del 1 al 10 usando `for`.
4. Creá una lista de frutas. Usá un `while` para recorrer la lista y mostrar cada fruta numerada.
5. Usá un `while` para eliminar todos los elementos impares de una lista de números (cuidado con el índice).
6. Simulá un cajero automático. Saldo inicial \$5000. El usuario puede depositar, retirar o consultar saldo mientras no elija salir. Usá `while True`.
7. Pedile al usuario una contraseña (3 intentos). Si acierta, mostrará un menú con 3 opciones usando `for`. Si no, bloqueá la sesión.
8. Creá dos listas: `pares = []` e `impares = []`. Pedí números hasta que ingrese 0 y guardá cada número en la lista correspondiente. Luego mostrará ambas listas.
9. Usá un `while` anidado para mostrar el triángulo:

```

1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

```

10. Hacé el juego “Piedra, Papel o Tijera” hasta que el usuario decida salir. Llevá el puntaje.
11. Pedí una frase al usuario. Usá un **while** para recorrer la frase carácter por carácter y contá cuántas vocales tiene (sin usar **count**).
12. Construí una lista de compras. Permití al usuario agregar productos (nombre y precio), ver el carrito y calcular el total. Usá **while True**.
13. Con un **while** anidado, generá la figura:

```

*****
****
***
**
*

```

14. Simulá una calculadora que permita hacer múltiples operaciones. Después de cada resultado, que pregunte “¿Desea continuar? (s/n)”.
15. Generá la secuencia de Fibonacci mientras el término sea menor a 1000 usando un **while**.
16. La computadora piensa un número entre 1 y 100. El usuario tiene intentos ilimitados; después de cada uno, recibe pista “más alto” o “más bajo”. Al final, clasificá la performance según intentos.
17. Implementá un sistema de registro de usuarios. Usá una lista de diccionarios con nombre y edad. Permití agregar, eliminar y listar usuarios con un menú **while**.
18. Dibujá un triángulo invertido de números con **while** anidado:

```

1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

```

19. Pedí 5 números y guardalos en una lista. Usá un **while** para calcular el promedio y mostrá si está aprobado (mayor o igual a 7).
20. Simulá un cajero automático completo: ver saldo, depositar, retirar, historial de movimientos (guardado en una lista) y salir. Todo dentro de un **while True**.

Soluciones paso a paso

Solución 1

```
fila = 1
while fila <= 3:
    columna = 1
    while columna <= 5:
        print("*", end="")
        columna = columna + 1
    print()
    fila = fila + 1
```

Explicación: El `while` exterior controla las filas (1 a 3), el interior imprime 5 asteriscos en cada fila y luego un salto de línea.

Solución 2

```
fila = 1
while fila <= 5:
    columna = 1
    while columna <= fila:
        print(columna, end=" ")
        columna = columna + 1
    print()
    fila = fila + 1
```

Explicación: En cada fila, el bucle interior imprime números desde 1 hasta el número de fila.

Solución 3

```
num = int(input("Numero (0 para terminar): "))
while num != 0:
    print(f"\nTabla del {num}:")
    for i in range(1, 11):
        print(f"{num} x {i} = {num * i}")
    num = int(input("\nOtro numero (0 para terminar): "))
```

Explicación: El `while` se repite hasta que el usuario pone 0; el `for` ejecuta las 10 multiplicaciones fijas.

Solución 4

```
frutas = ["manzana", "pera", "uva", "sandia", "melon"]
indice = 0
while indice < len(frutas):
    print(f"{indice + 1}. {frutas[indice]}")
    indice = indice + 1
```

Explicación: Se recorre la lista con un índice que incrementamos manualmente dentro del `while`.

Solución 5

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
indice = 0
while indice < len(numeros):
    if numeros[indice] % 2 != 0:
        numeros.pop(indice) # no incrementamos indice porque se corrió la lista
    else:
        indice = indice + 1
print("Pares:", numeros)
```

Explicación: Al eliminar un elemento, el siguiente toma su lugar; por eso solo avanzamos el índice cuando no eliminamos nada.

Solución 6

```
saldo = 5000
print("Saldo inicial: $5000")
while True:
    print("\n1. Depositar")
    print("2. Retirar")
    print("3. Consultar")
    print("4. Salir")
    op = input("Opcion: ")
    if op == "1":
        monto = float(input("Monto: "))
        saldo = saldo + monto
        print(f"Nuevo saldo: ${saldo}")
    elif op == "2":
        monto = float(input("Monto: "))
        if monto <= saldo:
            saldo = saldo - monto
            print(f"Nuevo saldo: ${saldo}")
        else:
            print("Saldo insuficiente")
    elif op == "3":
        print(f"Saldo: ${saldo}")
    elif op == "4":
        print("Gracias")
        break
    else:
        print("Opcion invalida")
```

Explicación: Un while True infinito que se rompe con break al elegir salir.

Solución 7

```
usuario_correcto = "admin"
clave_correcta = "1234"
```

```

intentos = 0
while intentos < 3:
    usuario = input("Usuario: ")
    clave = input("Clave: ")
    if usuario == usuario_correcto and clave == clave_correcta:
        print("Bienvenido!")
        print("\n--- MENU ---")
        for i in range(1, 4):
            print(f"{i}. Opcion {i}")
        break
    else:
        intentos = intentos + 1
        print(f"Error. Intentos: {3 - intentos}")
if intentos == 3:
    print("Cuenta bloqueada")

```

Explicación: El while controla los intentos, y el for imprime un menú fijo.

Solución 8

```

pares = []
impares = []
num = int(input("Numero (0 para terminar): "))
while num != 0:
    if num % 2 == 0:
        pares.append(num)
    else:
        impares.append(num)
    num = int(input("Otro numero (0 para terminar): "))
print("Pares:", pares)
print("Impares:", impares)

```

Explicación: El while sigue pidiendo hasta 0, y mediante un if decidimos a qué lista va cada número.

Solución 9

```

fila = 1
while fila <= 5:
    columna = 1
    while columna <= fila:
        print(fila, end=" ")
        columna = columna + 1
    print()
    fila = fila + 1

```

Explicación: Similar al triángulo de la solución 2, pero imprimimos el número de fila en lugar de la columna.

Solución 10

```
import random
opciones = ["piedra", "papel", "tijera"]
jugador = 0
cpu = 0
while True:
    print(f"\nMarcador: Jugador {jugador} - CPU {cpu}")
    print("1. Jugar")
    print("2. Salir")
    op = input("Opcion: ")
    if op == "2":
        print("Gracias por jugar!")
        break
    elif op == "1":
        j_elige = input("Elegi: ").lower()
        c_elige = random.choice(opciones)
        print(f"CPU: {c_elige}")
        if j_elige == c_elige:
            print("Empate")
        elif (j_elige == "piedra" and c_elige == "tijera") or \
            (j_elige == "papel" and c_elige == "piedra") or \
            (j_elige == "tijera" and c_elige == "papel"):
            print("Ganaste!")
            jugador = jugador + 1
        else:
            print("Perdiste!")
            cpu = cpu + 1
    else:
        print("Opcion invalida")
```

Explicación: El while principal permite jugar partidas hasta que el usuario elige salir.

Solución 11

```
frase = input("Frase: ")
indice = 0
vocales = 0
while indice < len(frase):
    letra = frase[indice].lower()
    if letra in "aeiou":
        vocales = vocales + 1
    indice = indice + 1
print(f"Vocales: {vocales}")
```

Explicación: Recorremos con while y un índice manual, chequeando vocal por vocal.

Solución 12

```
carrito = []
```



```
while True:
    print("\n--- CARRITO ---")
    print("1. Agregar")
    print("2. Ver")
    print("3. Total")
    print("4. Salir")
    op = input("Opcion: ")
    if op == "1":
        prod = input("Producto: ")
        precio = float(input("Precio: "))
        carrito.append((prod, precio))
        print("Agregado!")
    elif op == "2":
        if not carrito:
            print("Carrito vacio")
        else:
            for item in carrito:
                print(f"{item[0]}: ${item[1]}")
    elif op == "3":
        total = 0
        for item in carrito:
            total = total + item[1]
        print(f"Total: ${total}")
    elif op == "4":
        print("Gracias!")
        break
    else:
        print("Opcion invalida")
```

Explicación: El carrito se maneja con una lista de tuplas; el `for` recorre los ítems para mostrarlos o sumar el total.

Solución 13

```
fila = 5
while fila >= 1:
    columna = 1
    while columna <= fila:
        print("*", end="")
        columna = columna + 1
    print()
    fila = fila - 1
```

Explicación: Empezamos con 5 asteriscos y vamos disminuyendo la cantidad en cada fila.

Solución 14

```
while True:
    print("\n--- CALCULADORA ---")
```

```
a = float(input("Primer numero: "))
b = float(input("Segundo numero: "))
op = input("Operacion (+, -, *, /): ")
if op == "+":
    print(f"{a} + {b} = {a + b}")
elif op == "-":
    print(f"{a} - {b} = {a - b}")
elif op == "*":
    print(f"{a} * {b} = {a * b}")
elif op == "/":
    if b != 0:
        print(f"{a} / {b} = {a / b}")
    else:
        print("No se puede dividir por cero")
else:
    print("Operacion invalida")
cont = input("¿Desea continuar? (s/n): ").lower()
if cont != "s":
    print("Hasta luego!")
    break
```

Explicación: Un while que solo se corta cuando la respuesta no es "s".

Solución 15

```
a, b = 0, 1
print("Fibonacci (< 1000):")
while a < 1000:
    print(a, end=" ")
    a, b = b, a + b
print()
```

Explicación: Con dos variables y un while vamos generando la serie.

Solución 16

```
import random
secreto = random.randint(1, 100)
intentos = 0
print("Adivina el numero (1-100)")
while True:
    num = int(input("Numero: "))
    intentos = intentos + 1
    if num == secreto:
        if intentos < 5:
            print(f"Excelente! Adivinaste en {intentos} intentos")
        elif intentos < 10:
            print(f"Bien! Adivinaste en {intentos} intentos")
        else:
```

```

        print(f"Regular. Adivinaste en {intentos} intentos")
    break
elif num < secreto:
    print("Mas alto")
else:
    print("Mas bajo")

```

Explicación: El bucle infinito se rompe con `break` al acertar.

Solución 17

```

usuarios = []
while True:
    print("\n1. Agregar usuario")
    print("2. Eliminar usuario")
    print("3. Listar usuarios")
    print("4. Salir")
    op = input("Opcion: ")
    if op == "1":
        nombre = input("Nombre: ")
        edad = int(input("Edad: "))
        usuarios.append({"nombre": nombre, "edad": edad})
        print("Usuario agregado")
    elif op == "2":
        nombre = input("Nombre a eliminar: ")
        i = 0
        while i < len(usuarios):
            if usuarios[i]["nombre"] == nombre:
                usuarios.pop(i)
                print("Eliminado")
            else:
                i = i + 1
    elif op == "3":
        for u in usuarios:
            print(f"{u['nombre']}: {u['edad']} años")
    elif op == "4":
        print("Adios!")
        break

```

• **Explicación:** Combinamos un menú `while`, un `for` para listar y otro `while` para eliminar correctamente.

Solución 18

```

fila = 5
while fila >= 1:
    columna = 1
    while columna <= fila:
        print(columna, end=" ")

```

```
        columna = columna + 1
    print()
    fila = fila - 1
```

Explicación: Similar al triángulo de asteriscos pero con números.

Solución 19

```
numeros = []
i = 0
while i < 5:
    num = float(input(f"Numero {i+1}: "))
    numeros.append(num)
    i = i + 1

suma = 0
i = 0
while i < len(numeros):
    suma = suma + numeros[i]
    i = i + 1
promedio = suma / 5
print(f"Promedio: {promedio:.2f}")
print("Aprobado" if promedio >= 7 else "Reprobado")
```

Explicación: Dos while: uno para cargar y otro para sumar. Luego evaluamos el promedio.

Solución 20

```
saldo = 5000
historial = []
print("Saldo inicial: $5000")
while True:
    print("\n1. Ver saldo")
    print("2. Depositar")
    print("3. Retirar")
    print("4. Historial")
    print("5. Salir")
    op = input("Opcion: ")
    if op == "1":
        print(f"Saldo: ${saldo}")
    elif op == "2":
        monto = float(input("Monto: "))
        saldo = saldo + monto
        historial.append(f"Deposito: +${monto}")
        print(f"Nuevo saldo: ${saldo}")
    elif op == "3":
        monto = float(input("Monto: "))
        if monto <= saldo:
            saldo = saldo - monto
```

```
        historial.append(f"Retiro: -${monto}")
        print(f"Nuevo saldo: ${saldo}")
    else:
        print("Saldo insuficiente")
elif op == "4":
    if not historial:
        print("Sin movimientos")
    else:
        for mov in historial:
            print(mov)
elif op == "5":
    print("Gracias por usar el cajero!")
    break
else:
    print("Opcion invalida")
```

Explicación: El historial se guarda en una lista que se va llenando y luego se muestra con un for.

Resumen del capítulo

Dale, repasemos lo importante:

- Un **bucle anidado** es meter un bucle dentro de otro. El interior corre completo por cada vuelta del exterior.
- **While anidado** sirve para figuras, tablas, y cualquier patrón que dependa de un estado doble.
- **While con for**: la combinación perfecta cuando la tarea externa es indefinida y la interna tiene pasos fijos.
- **While con listas**: procesamiento de colecciones mientras se cumpla una condición (vaciar, buscar y eliminar, recorrer).
- Siempre actualizá las variables de control en los **while** para evitar bucles infinitos.
- La indentación es fundamental: el código que está dentro de un bucle debe tener más espacios que el bucle mismo.

¿Qué sigue?

Ahora que dominás los bucles y sus combinaciones, estás listo para abordar estructuras de datos más complejas.

Los bucles anidados son como las muñecas rusas: una dentro de otra, cada una con su propio trabajo. Si las entendés, podés modelar casi cualquier proceso repetitivo.

Listas y sus Métodos en Python

¿Qué es una lista?

Cuando empecé a programar, uno de los primeros desafíos fue guardar varios nombres sin tener que crear una variable por cada uno. Recuerdo que en una práctica en la UBA Exactas necesitaba almacenar los asistentes a un curso y me pregunté: “¿Cómo hago para tenerlos todos juntos?”. Ahí aparecieron las **listas**, una de las estructuras más útiles de Python.

En criollo, una **lista** es como un estante con cajones numerados. Cada cajón tiene una posición, que llamamos **índice**, y adentro podés meter cualquier cosa: números, texto, booleanos, e incluso otras listas. Lo mejor es que podés modificar el contenido cuando quieras: agregar, sacar, cambiar... por eso decimos que son **mutables**.

```
+-----+
|
|  frutas = ["manzana", "banana", "pera", "kiwi"] |
|
|  indice:      0          1          2          3      |
|
|  Las listas se crean con corchetes [] |
|  Los elementos se separan con comas |
|
+-----+
```

Creación de listas

Para armar una lista, usás corchetes `[]` y ponés los elementos separados por comas. Si no tenés nada, podés empezar con una lista vacía, que es como un estante listo para recibir cosas.

```
# Lista con elementos
frutas = ['Manzana', 'Banana', 'Pera', 'Kiwi', 'Naranja']

# Lista vacia (ideal para ir agregando despues)
Nom = []

# Lista con distintos tipos de datos
mezcla = [1, "hola", True, 3.14]

print(frutas) # ['Manzana', 'Banana', 'Pera', 'Kiwi', 'Naranja']
```

Las listas pueden ser **homogéneas** (todos los elementos del mismo tipo, como solo strings) o **heterogéneas** (mezcla de enteros, strings, lo que se te ocurra). No hay restricciones, y eso las vuelve muy flexibles.

Acceder a elementos por índice

En Python, los índices arrancan desde 0. El primer elemento está en la posición 0, el segundo en 1, y así. Si te pasás de la cantidad, te salta un `IndexError` que te deja congelado. ¡A todos nos pasó!

```
frutas = ['Manzana', 'Banana', 'Pera', 'Kiwi', 'Naranja']

print(frutas[0])    # Manzana (primer elemento)
print(frutas[2])    # Pera (tercer elemento)
print(frutas[-1])   # Naranja (último elemento, índice negativo)
print(frutas[-2])   # Kiwi (penúltimo)
```

```
Manzana
Pera
Naranja
Kiwi
```

Nota: ¡Cuidado con el `IndexError`! Si intentás acceder a un índice que no existe, Python va a putear. Ejemplo: `frutas[10]` cuando la lista tiene 5 elementos. Siempre chequeá la cantidad con `len()`.

Ejercicio rápido: Creá una lista con tus tres comidas favoritas y accedé al segundo elemento. ¿Cuál es su índice?

La función `len()`

Para saber cuántos elementos tiene una lista sin tener que contarlos a mano, usás `len()`. Es como preguntarle al estante cuántos cajones hay.

```
marcas = ["apple", "samsung", "xiaomi", "huawei"]
print(f"La lista tiene {len(marcas)} elementos")
```

Rebanadas de Lista (Slicing)

Así como con los strings, podés obtener una **porción** de la lista usando el operador `[inicio:fin]`. El elemento en la posición `fin` **no se incluye**. También podés jugar con el paso.

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Desde el índice 2 hasta el 5 (el 5 no se incluye)
print(numeros[2:6])  # [2, 3, 4, 5]

# Desde el principio hasta el índice 5
print(numeros[:5])   # [0, 1, 2, 3, 4]

# Desde el índice 5 hasta el final
print(numeros[5:])   # [5, 6, 7, 8, 9]
```



```
# Con paso (cada 2 elementos)
print(numeros[::2]) # [0, 2, 4, 6, 8]

# Invertir la lista (truco magico)
print(numeros[::-1]) # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Analogía: Si la lista es una góndola de supermercado, el slicing te permite agarrar los productos que están entre tal estantería y tal otra, sin tocar el resto.

Modificar elementos de una lista

Las listas son **mutables**, o sea que podés cambiar un elemento sin problemas.

Cambiar un elemento existente

Simplemente le asignás un nuevo valor con el índice.

```
frutas = ['Manzana', 'Banana', 'Pera', 'Kiwi']
frutas[1] = 'Sandia' # Cambia el elemento en posición 1
print(frutas) # ['Manzana', 'Sandia', 'Pera', 'Kiwi']
```

Modificar varios elementos a la vez (con slicing)

Incluso podés reemplazar un rango entero por otra lista.

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print("Original:", numeros)

# Reemplazar desde índice 1 hasta 4 (sin incluir el 4) con otros valores
numeros[1:4] = [20, 30, 40]
print("Despues del cambio:", numeros)
```

Recorriendo Listas con el bucle for

El **for** es el compañero inseparable de las listas. Hay varias formas de recorrerlas, cada una con su gracia.

Forma 1: Directamente sobre los elementos

La más pitónica y clara. Si no necesitás el índice, esta es la que va.

```
frutas = ["manzana", "banana", "pera", "kiwi"]
print("Mis frutas favoritas:")
for fruta in frutas:
    print(f"- {fruta}")
```

Forma 2: Usando índices con `range(len())`

Cuando necesitas la posición, usás `range()` y `len()`.

```
frutas = ["manzana", "banana", "pera", "kiwi"]
for i in range(len(frutas)):
    print(f"Posicion {i}: {frutas[i]}")
```

Forma 3: Con `enumerate()` (la más elegante)

`enumerate()` te da el índice y el elemento al mismo tiempo. Te ahorra usar `range(len())`.

```
frutas = ["manzana", "banana", "pera", "kiwi"]
for indice, fruta in enumerate(frutas):
    print(f"Posicion {indice}: {fruta}")
```

Métodos de listas

Los métodos son acciones que las listas entienden. Como un control remoto: apretás un botón y la lista hace algo. Vamos a ver los más útiles, que usé hasta el cansancio en la facultad y en el trabajo.

`append()` – Agregar al final

`append()` es el rey. Agrega un elemento al **final** de la lista, como poner un producto más en el último lugar de la cinta del supermercado.

```
lista = [1, 2, 3]
lista.append(4)
print(lista) # [1, 2, 3, 4]

# Ejemplo práctico: pedir nombres y guardarlos
nombres = []
for i in range(3):
    nombre = input('Ingresa un nombre: ')
    nombres.append(nombre)
print(nombres)
```

`insert()` – Insertar en una posición

Si querés meter un elemento en el medio de la lista, usás `insert(posicion, elemento)`. Los demás se corren hacia la derecha.

```
lista = [1, 2, 3]
lista.insert(1, 'hola')
print(lista) # [1, 'hola', 2, 3]

# Para ponerlo al principio (posicion 0)
lista.insert(0, 'inicio')
print(lista) # ['inicio', 1, 'hola', 2, 3]
```

extend() – Concatenar listas

`extend()` agrega todos los elementos de otra lista al final, como si volcás un carrito entero sobre la cinta.

```
lista = [1, 2, 3]
lista.extend([4, 5, 6])
print(lista)  # [1, 2, 3, 4, 5, 6]

# Diferencia entre append() y extend()
lista = [1, 2]
lista.append([3, 4])  # Agrega la lista entera como un solo elemento -> [1, 2, [3, 4]]
print(lista)

lista = [1, 2]
lista.extend([3, 4])  # Agrega cada elemento por separado -> [1, 2, 3, 4]
print(lista)
```

pop() – Eliminar el último elemento (o uno específico)

`pop()` saca el último elemento de la lista y te lo devuelve. Si le pasás un índice, saca ese en particular.

```
lista = [1, 2, 3, 4]
eliminado = lista.pop()
print(f"Elemento eliminado: {eliminado}")  # 4
print(lista)  # [1, 2, 3]

# Eliminar por posicion
lista = [10, 20, 30, 40]
eliminado = lista.pop(1)
print(f"Eliminado: {eliminado}")  # 20
print(lista)  # [10, 30, 40]
```

remove() – Eliminar por valor

`remove(valor)` borra la **primera aparición** del valor que le digas. Si no existe, da error. Para eliminar todas las ocurrencias, podés combinarlo con un `while`.

```
lista = [1, 'jorge', 2, 'jorge']
lista.remove('jorge')
print(lista)  # [1, 2, 'jorge']

# Eliminar todas las "Ana" de una lista
nombres = ["Ana", "Luis", "Ana", "Pedro", "Ana", "Carlos"]
while "Ana" in nombres:
    nombres.remove("Ana")
print(nombres)  # ['Luis', 'Pedro', 'Carlos']
```

clear() – Vaciar la lista

`clear()` vacía completamente la lista, como pasar una escoba y dejar el estante pelado.

```
lista = [1, 2, 3, 4]
lista.clear()
print(lista)  # []
```

sort() – Ordenar la lista

`sort()` ordena de menor a mayor (o alfabéticamente si son strings). Con `reverse=True` lo hacés al revés.

```
numeros = [4, 2, 7, 1, 5]
numeros.sort()
print(numeros)  # [1, 2, 4, 5, 7]
```

```
numeros.sort(reverse=True)
print(numeros)  # [7, 5, 4, 2, 1]
```

```
colores = ['rojo', 'azul', 'verde', 'amarillo']
colores.sort()
print(colores)  # ['amarillo', 'azul', 'rojo', 'verde']
```

```
# sorted() crea una nueva lista sin modificar la original
original = [5, 2, 8, 1, 9, 3]
nueva = sorted(original)
print("Original:", original)  # [5, 2, 8, 1, 9, 3]
print("Nueva ordenada:", nueva)  # [1, 2, 3, 5, 8, 9]
```

reverse() – Invertir el orden

`reverse()` da vuelta a la lista: el primero pasa a ser el último y viceversa.

```
lista = [0, 1, 2, 3, 4]
lista.reverse()
print(lista)  # [4, 3, 2, 1, 0]
```

```
# Para invertir sin modificar la original, usas slicing
original = [1, 2, 3, 4, 5]
invertida = original[::-1]
print(original)  # [1, 2, 3, 4, 5]
print(invertida)  # [5, 4, 3, 2, 1]
```

index() – Encontrar la posición de un elemento

`index(valor)` te dice el índice de la primera aparición del valor. Si no está, da error. ¡Ojo!

```
lista = [10, 20, 30, 40]
print(lista.index(30)) # 2
```

```
lista = [5, 10, 15, 10, 20]
print(lista.index(10)) # 1 (la primera)
```

count() – Contar apariciones

count(valor) cuenta cuántas veces aparece un valor en la lista.

```
lista = [1, 2, 1, 3, 1, 4]
print(lista.count(1)) # 3
```

copy() – Copiar una lista

Si hacés `lista2 = lista1`, ambas variables apuntan a la **misma lista**. Para tener una copia independiente, usás `copy()` o el slicing `[:]`.

```
original = [1, 2, 3]
mala_copia = original
mala_copia.append(4)
print("Original modificado:", original) # [1, 2, 3, 4]
```

```
original = [1, 2, 3]
buena_copia = original.copy()
buena_copia.append(4)
print("Original:", original) # [1, 2, 3]
print("Buena copia:", buena_copia) # [1, 2, 3, 4]
```

sum() – Sumar los elementos

sum() devuelve la suma de todos los elementos de una lista numérica. Mucho más rápido que hacer un bucle.

```
numeros = [10, 20, 30, 40, 50]
total = sum(numeros)
print(f"La suma es {total}")
```

Operadores con listas

El operador in

Con `in` preguntás si un elemento está en la lista. Te devuelve `True` o `False`.

```
materias = ["Matematica", "Fisica", "Quimica", "Algoritmos"]
print("'Fisica' esta?", "Fisica" in materias) # True
print("'Literatura' esta?", "Literatura" in materias) # False
```

```
if "Algoritmos" in materias:
    print("¡Si, tenemos Algoritmos!")
```

Concatenación con +

Podés unir dos listas con el signo +. Es como poner dos tandas de compras en una misma bolsa.

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
lista3 = lista1 + lista2
print(lista3) # [1, 2, 3, 4, 5, 6]
```

Repetición con *

Si querés repetir una lista varias veces, usás el operador *.

```
lista = [1, 2, 3]
lista_repetida = lista * 3
print(lista_repetida) # [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Uso de listas en programas reales

Acá te muestro algunos ejemplos que escribí cuando daba mis primeros pasos. Son simples pero efectivos.

Listas paralelas (múltiples listas relacionadas)

Cuando necesitás asociar datos pero todavía no viste diccionarios, usar listas paralelas es un salvavidas.

```
NOMBRE = []
APELLIDO = []
EDAD = []

for i in range(2):
    nombre = input('Ingresa nombre: ')
    apellido = input('Ingresa apellido: ')
    edad = input('Ingresa edad: ')
    NOMBRE.append(nombre)
    APELLIDO.append(apellido)
    EDAD.append(edad)

for i in range(len(NOMBRE)):
    print(f"Nombre: {NOMBRE[i]}, Apellido: {APELLIDO[i]}, Edad: {EDAD[i]}")
```

Separar pares e impares con listas

Un clásico: pedir números y clasificarlos.

```
pares = []
impares = []

for i in range(5):
    numero = int(input('Ingresa un numero: '))
    if numero % 2 == 0:
        pares.append(numero)
    else:
        impares.append(numero)

print('Numeros pares:', pares)
print('Numeros impares:', impares)
```

Simular un carrito de compras

Este lo usé en una práctica para entender cómo se manejan acumuladores y menús.

```
carrito = []

while True:
    print("\n--- MENU ---")
    print("1. Agregar producto")
    print("2. Ver carrito")
    print("3. Total")
    print("4. Salir")
    op = input("Opcion: ")

    if op == "1":
        producto = input("Producto: ")
        precio = float(input("Precio: "))
        carrito.append((producto, precio))
        print(f"{producto} agregado")
    elif op == "2":
        if carrito:
            for item in carrito:
                print(f"{item[0]}: ${item[1]}")
        else:
            print("Carrito vacio")
    elif op == "3":
        total = sum(precio for _, precio in carrito)
        print(f"Total: ${total}")
    elif op == "4":
        print("Gracias por su compra!")
        break
```

Simular una pila (LIFO) con listas

Una pila sigue la regla “último en entrar, primero en salir”. Con `append()` y `pop()` se implementa fácil.

```
pila = []

while True:
    print("\n--- PILA (LIFO) ---")
    print("1. Apilar (push)")
    print("2. Desapilar (pop)")
    print("3. Ver pila")
    print("4. Salir")
    op = input("Opcion: ")

    if op == "1":
        elemento = input("Elemento a apilar: ")
        pila.append(elemento)
        print(f"{elemento} apilado")
    elif op == "2":
        if pila:
            eliminado = pila.pop()
            print(f"Desapilado: {eliminado}")
        else:
            print("Pila vacia")
    elif op == "3":
        print("Pila:", pila)
    elif op == "4":
        break
```

Eliminar con del

Además de los métodos, podés usar la instrucción `del` para borrar un elemento o una porción.

```
nombres = ["Ana", "Luis", "Pedro", "Marta"]
print("Original:", nombres)
del nombres[1] # Elimina a Luis
print("Despues de del nombres[1]:", nombres)

numeros = [1,2,3,4,5]
del numeros[1:3] # Borra indices 1 y 2
print(numeros) # [1, 4, 5]
```

Ejercicios propuestos

Ahora te toca a vos. No mires las soluciones hasta que hayas peleado un rato con cada uno. Son 20, de fácil a difícil.

1. Creá una lista con los nombres de 5 amigos y mostralos uno por uno con un `for`.
2. Dada `numeros = [10, 20, 30, 40, 50]`, cambiá el tercer elemento (índice 2) a 100 y mostrá la lista.

3. Creá una lista vacía, pedí al usuario 4 colores y agregalos con `append()`. Mostrá la lista.
4. Usando `insert()`, agregá la palabra "Python" en la posición 1 de `["Hola", "mundo"]`.
5. Usando `pop()`, eliminá el último elemento de `[1, 2, 3, 4, 5]` y mostrá el elemento eliminado y la lista resultante.
6. Dada `nombres = [".^na", "Luis", ".^na", "Pedro", ".^na", "Carlos"]`, eliminá todas las apariciones de "Ana" usando un `while` y `remove()`.
7. Creá una lista desordenada, usá `sort()` para ordenarla de menor a mayor, y luego `sort(reverse=True)` para ordenarla al revés.
8. Pedí 5 números, guardalos en una lista, y mostrá la lista original, la ordenada ascendente y la invertida.
9. Con `index()`, encontrá la posición del número 30 en `[10, 20, 30, 40, 50]`.
10. Con `count()`, contá cuántas veces aparece el 3 en `[1, 3, 2, 3, 4, 3, 5]`.
11. Pedí números hasta que se ingrese 0, guardalos en una lista, y luego mostrá la lista completa, la suma total (sin usar `sum()`) y el promedio.
12. Creá dos listas paralelas (nombres y notas), y mostrá los nombres de los que aprobaron (nota ≥ 6).
13. Eliminá los duplicados de `[1, 2, 2, 3, 4, 4, 4, 5]` sin usar `set()`.
14. Implementá una lista de tareas (to-do list) con menú: agregar, eliminar, mostrar y salir.
15. Escribí una función que reciba una lista de números y devuelva una nueva solo con los pares, usando `append()`.
16. Simulá una cola FIFO (primero en entrar, primero en salir) con una lista, usando `pop(0)` para desencolar.
17. Creá una lista de diccionarios para un inventario (nombre, precio, cantidad). Hacé menú para agregar producto, ver inventario, calcular valor total de stock y salir.
18. Ingresá nombres y edades en dos listas paralelas. Luego mostrá la persona más joven, la más mayor y el promedio de edad.
19. Implementá el algoritmo de ordenamiento burbuja para ordenar una lista sin usar `sort()`.
20. Tomá dos listas de números y devolvé la unión (sin duplicados), la intersección y la diferencia (elementos en la primera pero no en la segunda).

Soluciones paso a paso

Solución 1

```
amigos = ["Ana", "Luis", "Carlos", "Maria", "Pedro"]
for nombre in amigos:
    print(nombre)
```

Explicación: El for recorre directamente los elementos de la lista.

Solución 2

```
numeros = [10, 20, 30, 40, 50]
numeros[2] = 100
print(numeros) # [10, 20, 100, 40, 50]
```

Explicación: Cambiamos el elemento en la posición 2 por 100.

Solución 3

```
colores = []
for i in range(4):
    color = input(f"Color {i+1}: ")
    colores.append(color)
print("Colores:", colores)
```

Explicación: Empezamos con lista vacía y agregamos cada color al final.

Solución 4

```
lista = ["Hola", "mundo"]
lista.insert(1, "Python")
print(lista) # ["Hola", "Python", "mundo"]
```

Explicación: insert(1,...) mete el elemento en la posición 1, corriendo al resto.

Solución 5

```
lista = [1, 2, 3, 4, 5]
eliminado = lista.pop()
print(f"Eliminado: {eliminado}") # 5
print(lista) # [1, 2, 3, 4]
```

Explicación: pop() sin argumentos saca el último.

Solución 6

```
nombres = ["Ana", "Luis", "Ana", "Pedro", "Ana", "Carlos"]
while "Ana" in nombres:
    nombres.remove("Ana")
print(nombres) # ['Luis', 'Pedro', 'Carlos']
```

Explicación: Mientras “Ana” esté en la lista, la eliminamos.

Solución 7

```
numeros = [5, 2, 8, 1, 9, 3]
numeros.sort()
print("Ascendente:", numeros)
numeros.sort(reverse=True)
print("Descendente:", numeros)
```

Explicación: `sort()` ordena in-place; con `reverse=True` da vuelta el orden.

Solución 8

```
numeros = []
for i in range(5):
    num = int(input(f"Numero {i+1}: "))
    numeros.append(num)

print("Original:", numeros)
numeros.sort()
print("Ordenado:", numeros)
numeros.reverse()
print("Invertido:", numeros)
```

Explicación: Guardamos, ordenamos ascendente, luego invertimos.

Solución 9

```
lista = [10, 20, 30, 40, 50]
posicion = lista.index(30)
print(f"El 30 esta en la posicion: {posicion}") # 2
```

Explicación: `index()` devuelve el índice de la primera aparición.

Solución 10

```
lista = [1, 3, 2, 3, 4, 3, 5]
cantidad = lista.count(3)
print(f"El 3 aparece {cantidad} veces") # 3
```

Explicación: `count()` cuenta las ocurrencias del valor.

Solución 11

```
numeros = []
while True:
    num = int(input("Numero (0 para terminar): "))
    if num == 0:
        break
    numeros.append(num)
```

```
print("Lista:", numeros)
suma = 0
for num in numeros:
    suma += num
print("Suma:", suma)
if numeros:
    promedio = suma / len(numeros)
    print("Promedio:", promedio)
```

Explicación: Acumulamos en lista, sumamos con un bucle y calculamos promedio.

Solución 12

```
nombres = []
notas = []
for i in range(5):
    nombre = input(f"Nombre {i+1}: ")
    nota = float(input("Nota: "))
    nombres.append(nombre)
    notas.append(nota)

print("\nAprobados (nota >= 6):")
for i in range(len(nombres)):
    if notas[i] >= 6:
        print(f"{nombres[i]}: {notas[i]}")
```

Explicación: Listas paralelas; recorreremos por índice y filtramos.

Solución 13

```
original = [1, 2, 2, 3, 4, 4, 4, 5]
sin_duplicados = []
for num in original:
    if num not in sin_duplicados:
        sin_duplicados.append(num)
print("Sin duplicados:", sin_duplicados) # [1, 2, 3, 4, 5]
```

Explicación: Solo agregamos si no está ya en la nueva lista.

Solución 14

```
tareas = []
while True:
    print("\n1. Agregar 2. Eliminar 3. Mostrar 4. Salir")
    op = input("Opcion: ")
    if op == "1":
        tarea = input("Tarea: ")
        tareas.append(tarea)
    elif op == "2":
```

```

    tarea = input("Tarea a eliminar: ")
    if tarea in tareas:
        tareas.remove(tarea)
        print("Eliminada")
    else:
        print("No encontrada")
elif op == "3":
    if tareas:
        for i, t in enumerate(tareas, 1):
            print(f"{i}. {t}")
    else:
        print("No hay tareas")
elif op == "4":
    break

```

Explicación: Menú clásico con while; usamos `remove()` con verificación previa.

Solución 15

```

def solo_pares(lista):
    pares = []
    for num in lista:
        if num % 2 == 0:
            pares.append(num)
    return pares

print(solo_pares([1,2,3,4,5,6])) # [2, 4, 6]

```

Explicación: Filtramos con `if` y `append()`.

Solución 16

```

cola = []
while True:
    print("\n1. Encolar  2. Desencolar  3. Ver  4. Salir")
    op = input("Opcion: ")
    if op == "1":
        elem = input("Elemento: ")
        cola.append(elem)
    elif op == "2":
        if cola:
            print("Desencolado:", cola.pop(0))
        else:
            print("Cola vacia")
    elif op == "3":
        print("Cola:", cola)
    elif op == "4":
        break

```

Explicación: `pop(0)` elimina y devuelve el primer elemento (FIFO).

Solución 17

```

inventario = []
while True:
    print("\n1. Agregar  2. Ver  3. Valor stock  4. Salir")
    op = input("Opcion: ")
    if op == "1":
        prod = {
            "nombre": input("Nombre: "),
            "precio": float(input("Precio: ")),
            "cantidad": int(input("Cantidad: "))
        }
        inventario.append(prod)
    elif op == "2":
        for p in inventario:
            print(f"{p['nombre']} - ${p['precio']} x {p['cantidad']} = ${p['precio'] * p['cantidad']}")
    elif op == "3":
        total = sum(p['precio'] * p['cantidad'] for p in inventario)
        print(f"Valor total del stock: ${total}")
    elif op == "4":
        break

```

Explicación: Lista de diccionarios; iteramos para calcular y mostrar.

Solución 18

```

nombres = []
edades = []
while True:
    nombre = input("Nombre (vacío para terminar): ")
    if nombre == "":
        break
    edad = int(input("Edad: "))
    nombres.append(nombre)
    edades.append(edad)

if nombres:
    menor_edad = edades[0]
    mayor_edad = edades[0]
    menor_nombre = nombres[0]
    mayor_nombre = nombres[0]
    for i in range(len(edades)):
        if edades[i] < menor_edad:
            menor_edad = edades[i]
            menor_nombre = nombres[i]
        if edades[i] > mayor_edad:
            mayor_edad = edades[i]
            mayor_nombre = nombres[i]
    promedio = sum(edades) / len(edades)

```

```
print(f"Mas joven: {menor_nombre} ({menor_edad} años)")
print(f"Mas mayor: {mayor_nombre} ({mayor_edad} años)")
print(f"Promedio edad: {promedio:.1f}")
```

Explicación: Buscamos mínimo y máximo manualmente y calculamos promedio.

Solución 19

```
def burbuja(lista):
    n = len(lista)
    for i in range(n-1):
        for j in range(n-1-i):
            if lista[j] > lista[j+1]:
                lista[j], lista[j+1] = lista[j+1], lista[j]
    return lista
```

```
datos = [64, 34, 25, 12, 22, 11, 90]
print("Original:", datos)
print("Ordenado:", burbuja(datos.copy()))
```

Explicación: Algoritmo clásico de intercambio por pares.

Solución 20

```
def union_sin_dup(a, b):
    res = []
    for x in a:
        if x not in res: res.append(x)
    for x in b:
        if x not in res: res.append(x)
    return res

def interseccion(a, b):
    res = []
    for x in a:
        if x in b and x not in res:
            res.append(x)
    return res

def diferencia(a, b):
    res = []
    for x in a:
        if x not in b and x not in res:
            res.append(x)
    return res
```

```
L1 = [1,2,3,4,5]
L2 = [4,5,6,7,8]
print("Union:", union_sin_dup(L1, L2))
```

```
print("Interseccion:", interseccion(L1, L2))  
print("Diferencia (1-2):", diferencia(L1, L2))
```

Explicación: Implementamos las tres operaciones recorriendo manualmente.

Resumen del capítulo

Dale, repasemos lo más importante:

- Las **listas** son colecciones ordenadas y mutables; se crean con `[ ]`.
- Se accede por **índice** (empieza en 0) y se pueden **rebanar** con `[inicio:fin:paso]`.
- Métodos clave:
 - `append()`, `insert()`, `extend()` para agregar.
 - `pop()`, `remove()`, `clear()` para eliminar.
 - `sort()`, `reverse()`, `index()`, `count()`, `copy()`.
- Funciones útiles: `len()`, `sum()`, `sorted()`.
- Operadores: `in`, `+`, `*`.
- Se recorren con `for` (elemento a elemento o con `enumerate`).
- Ideales para pilas, colas, carritos, registros y un montón de situaciones reales.

*Las listas son el corazón de muchos programas; dominarlas te abre las puertas a casi cualquier lógica.
Practicá, equivocate y volvé a intentarlo.*

Funciones: El Corazón del Código Re-utilizable

¿Qué es una función?

Cuando empecé a programar en la facu (UBA Exactas), todo era un desastre: copiaba y pegaba bloques de código por todos lados. Si necesitaba saludar a cinco personas distintas, escribía cinco veces el mismo `print`. Un día un ayudante me dijo: “Che, Nahuel, ¿por qué no armás una función? Escribís una vez y la usás cuando quieras, como una receta de cocina.” Me quedé duro. No entendía cómo algo tan simple me había pasado desapercibido.

Desde ese momento, las funciones se convirtieron en mi pan de cada día. Una **función** es simplemente un bloque de código que realiza una tarea y que podés llamar desde cualquier lado del programa. Ya conocés muchas: `print()`, `input()`, `len()`, `range()`, `sum()`. Pero ahora vas a crear las tuyas.

Analogía: Imaginá una fábrica. Cada función es una máquina que recibe materiales (parámetros), los procesa y entrega un producto terminado (valor de retorno). Algunas máquinas solo muestran el producto en pantalla (funciones impuras), otras te lo entregan en una caja para que lo uses después (funciones puras).

```
+-----+
|
|  def nombre_de_la_funcion(parametros):
|      # Código que hace algo
|      return resultado (opcional)
|
|  Las funciones ayudan a:
|  - Reutilizar código
|  - Organizar mejor el programa
|  - Hacer el código más fácil de leer y mantener
|
+-----+
```

¿Por qué usar funciones?

Te cuento una anécdota: una vez hice un programita que pedía el nombre, apellido y edad de diez personas, y después los mostraba. Eran dos líneas de código repetidas diez veces. Funcionaba, pero cuando quise cambiar el formato de salida, tuve que corregirlo en diez lugares distintos. ¡Una locura! Si hubiera escrito una función, modificaba una sola vez y listo.

Ventajas de usar funciones:

- **Reutilización:** escribís una vez, usás mil veces.
- **Organización:** el código queda limpio, como un placard ordenado.
- **Mantenimiento:** si hay un error, lo arreglás en un solo lugar.
- **Abstracción:** podés usar una función sin saber cómo funciona por dentro, como hacés con `print()`.

Funciones Puras vs Funciones Impuras

Uno de los conceptos que más me costó fue entender la diferencia entre una función que imprime y una que devuelve un valor. Me acuerdo que una vez llamé a una función que solo imprimía y esperé que me diera un número para seguir calculando... ¡y me dio `None`! No entendía nada. Hasta que me explicaron lo de las funciones puras e impuras.

¿Qué es un efecto secundario?

Un **efecto secundario** es cualquier cosa que la función hace además de devolver un valor. Por ejemplo:

- **Imprimir en pantalla** con `print()`.
- **Modificar una variable global** (fuera de la función).
- **Modificar una lista o diccionario** que se pasó como parámetro.
- **Escribir en un archivo** o leer del teclado (`input()`).

Función Pura

Una función **pura** cumple dos reglas de oro:

1. **No tiene efectos secundarios:** no imprime, no modifica variables externas, no pide datos al usuario.
2. **Siempre devuelve el mismo resultado** para los mismos argumentos (es "determinística").

FUNCIÓN PURA:

```
def saludo(nombre):  
    return "Hola " + nombre
```

- No imprime nada
- No modifica nada externo
- Simplemente calcula y devuelve
- Llamar `saludo("Juan")` siempre da "Hola Juan"

Función Impura

Una función **impura** tiene al menos un efecto secundario. Son las más comunes cuando recién empezás, y no está mal usarlas. Solo tenés que saber qué esperar de cada una.

```
+-----+
|
|  FUNCIÓN IMPURA:
|
|  def saludar(nombre):
|      print("Hola", nombre)
|
|  - Imprime algo (efecto secundario)
|  - No devuelve nada (o devuelve None)
|  - Hace algo "de paso" además de procesar
|
+-----+
```

Ejemplo de tu propio código

En tus apuntes tenías estos dos ejemplos perfectos que muestran la diferencia:

```
# FUNCIÓN IMPURA
def salud(nombre):
    # Esta funcion NO retorna un valor, solo imprime (efecto secundario)
    print('Hola', nombre)

# FUNCIÓN PURA
def saludo(nombre):
    # Esta funcion retorna un string sin imprimir nada
    return 'Hola ' + nombre

# Llamamos a la funcion pura e imprimimos su resultado
print(saludo("Juan")) # Hola Juan
```

Nota: *Diferencia clave:*

- `salud("Juan")` → *imprime "Hola Juan" en la pantalla, pero no devuelve nada. Si hacés `x = salud("Juan")`, `x` vale `None`.*
- `saludo("Juan")` → *devuelve "Hola Juan", pero no imprime nada. Si hacés `x = saludo("Juan")`, `x` vale "Hola Juan".*

Funciones que NO devuelven nada (solo imprimen)

De tus primeros ejercicios de la facultad, tenías funciones de suma, resta, multiplicación y división que eran **impuras** porque solo imprimían el resultado. Mirá este ejemplo:

```
# FUNCIÓN DE SUMA (IMPURA)
def suma(num1, num2):
    num3 = num1 + num2
    print('el resultado es:', num3)

# Llamamos a la funcion suma
res = suma(20, 3)  # Imprime: el resultado es: 23

# NOTA: res vale None porque suma no retorna nada
print(res)  # None
```

Fijate que `suma` hace su trabajo, pero no podés usar ese resultado después. Si querías el doble, no podías, porque `res` era `None`. Por eso, con el tiempo vas a preferir funciones con `return`.

Funciones con `return` (que devuelven algo)

Una función con `return` te entrega un valor que podés guardar en una variable y reutilizar. Es mucho más potente.

```
# Versión PURA (con return)
def suma_pura(num1, num2):
    resultado = num1 + num2
    return resultado

# Ahora podemos guardar el resultado y usarlo despues
total = suma_pura(20, 3)
print(f"El resultado guardado es: {total}")
print(f"El doble del resultado es: {total * 2}")
```

Reflexión: Siempre que puedas, preferí que tus funciones devuelvan el resultado con `return` en lugar de imprimir. Te da más flexibilidad.

Más ejemplos de tus apuntes

En tus clases viste varios ejemplos prácticos que vale la pena repasar.

Función que determina si un número es par o impar (impura)

```
def es_o_impar(numero):
    if numero % 2 == 0:
        print('Es par')
    else:
        print('Es impar')

numero = int(input('Introduzca un numero: '))
es_o_impar(numero)
```

Funciones que devuelven valores (puras)

Estas sí se pueden reutilizar porque tienen `return`.

```
def suma_entre_ellos(num1, num2):  
    return num1 + num2  
  
def resta_entre_ellos(num4, num5):  
    return num4 - num5  
  
def multiplicacion_entre_ellos(num7, num8):  
    return num7 * num8  
  
def dividir_entre_ellos(num10, num11):  
    if num11 != 0:  
        return num10 / num11  
    else:  
        return "Error: division por cero"  
  
# Ejemplo de uso  
var1 = int(input('Primer numero: '))  
var2 = int(input('Segundo numero: '))  
print(f"La suma es: {suma_entre_ellos(var1, var2)}")
```

Funciones con listas

Las funciones también pueden trabajar con listas. Podés pasar una lista como parámetro y modificarla (efecto secundario) o devolver una nueva.

Función que verifica el tamaño de una lista (impura)

```
def la_lista_es_mayor_a_6(lista):  
    if len(lista) > 6:  
        print('La lista es mayor a 6')  
    else:  
        print('La lista es menor a 6')  
  
lista_a_prueba = [1, 2, 3, 4, 5, 6, 7]  
la_lista_es_mayor_a_6(lista_a_prueba) # La lista es mayor a 6
```

Función que agrega un elemento a una lista (impura)

Esta función modifica la lista original, lo cual es un efecto secundario. A veces es justo lo que necesitás.

```
def agregar_al_final_de_la_lista(lista, cosa_a_agregar):  
    lista.append(cosa_a_agregar)  
    print(lista)
```

```
listaI = [1, 2, 3, 4, 5, 6, 7]
cosa = input('Introduzca una palabra: ')
agregar_al_final_de_la_lista(listaI, cosa)
```

Ejemplo Completo: Calculadora con Funciones

Juntando todo, armé una calculadora con funciones puras para cada operación. Así el código queda clarísimo.

```
def suma(num1, num2):
    return num1 + num2

def resta(num1, num2):
    return num1 - num2

def multiplicacion(num1, num2):
    return num1 * num2

def division(num1, num2):
    if num2 != 0:
        return num1 / num2
    else:
        return "Error: Division por cero"

# PROGRAMA PRINCIPAL
print("=== CALCULADORA ===")
print("1. Suma")
print("2. Resta")
print("3. Multiplicacion")
print("4. Division")

opcion = input("Elegi una operacion (1-4): ")
num1 = float(input("Primer numero: "))
num2 = float(input("Segundo numero: "))

if opcion == "1":
    resultado = suma(num1, num2)
    print(f"{num1} + {num2} = {resultado}")
elif opcion == "2":
    resultado = resta(num1, num2)
    print(f"{num1} - {num2} = {resultado}")
elif opcion == "3":
    resultado = multiplicacion(num1, num2)
    print(f"{num1} x {num2} = {resultado}")
elif opcion == "4":
    resultado = division(num1, num2)
    print(f"{num1} / {num2} = {resultado}")
else:
```

```
print("Opcion no valida")
```

Ejemplo Completo: Simulador de Cajero Automático

Un cajero automático con funciones impuras porque modifican una variable global `saldo` y muestran mensajes.

```
saldo = 1000
```

```
def mostrar_saldo():  
    print(f"Su saldo actual es: ${saldo}")
```

```
def depositar():  
    global saldo  
    monto = float(input("Monto a depositar: "))  
    if monto > 0:  
        saldo = saldo + monto  
        print(f"Deposito exitoso. Nuevo saldo: ${saldo}")  
    else:  
        print("Monto invalido")
```

```
def retirar():  
    global saldo  
    monto = float(input("Monto a retirar: "))  
    if monto > 0 and monto <= saldo:  
        saldo = saldo - monto  
        print(f"Retiro exitoso. Nuevo saldo: ${saldo}")  
    elif monto > saldo:  
        print("Saldo insuficiente")  
    else:  
        print("Monto invalido")
```

```
while True:  
    print("\n=== CAJERO AUTOMATICO ===")  
    print("1. Ver saldo")  
    print("2. Depositar")  
    print("3. Retirar")  
    print("4. Salir")  
    opcion = input("Elegi una opcion: ")  
    if opcion == "1":  
        mostrar_saldo()  
    elif opcion == "2":  
        depositar()  
    elif opcion == "3":  
        retirar()  
    elif opcion == "4":  
        print("Gracias por usar el cajero. ¡Hasta luego!")  
        break
```



```
else:
    print("Opcion no valida")
```

Ejemplo Completo: Lista de Compras con Funciones

Este es uno de mis favoritos. Combina funciones puras e impuras, listas y menús. Lo usé para hacer las compras del supermercado (sí, posta).

```
def agregar_producto(lista, producto, precio):
    lista.append({"producto": producto, "precio": precio})
    print(f"'{producto}' agregado al carrito.")

def mostrar_carrito(lista):
    if len(lista) == 0:
        print("El carrito esta vacio.")
    else:
        print("\n--- CARRITO DE COMPRAS ---")
        for i, item in enumerate(lista, 1):
            print(f"{i}. {item['producto']} - ${item['precio']:.2f}")

def calcular_total(lista):
    if len(lista) == 0:
        return 0
    total = 0
    for item in lista:
        total = total + item["precio"]
    return total

def eliminar_producto(lista, indice):
    if 0 <= indice < len(lista):
        eliminado = lista.pop(indice)
        print(f"'{eliminado['producto']}' eliminado del carrito.")
        return True
    else:
        print("Indice invalido.")
        return False

carrito = []
while True:
    print("\n=== SUPERMERCADO ===")
    print("1. Agregar producto")
    print("2. Ver carrito")
    print("3. Calcular total")
    print("4. Eliminar producto")
    print("5. Salir")
    opcion = input("Elegi una opcion: ")
    if opcion == "1":
        producto = input("Nombre del producto: ")
```

```

    precio = float(input("Precio: $"))
    agregar_producto(carrito, producto, precio)
elif opcion == "2":
    mostrar_carrito(carrito)
elif opcion == "3":
    total = calcular_total(carrito)
    print(f"Total a pagar: ${total:.2f}")
elif opcion == "4":
    mostrar_carrito(carrito)
    if len(carrito) > 0:
        indice = int(input("Numero del producto a eliminar: ")) - 1
        eliminar_producto(carrito, indice)
elif opcion == "5":
    print("¡Gracias por su compra!")
    break
else:
    print("Opcion no valida")

```

Diferencia clave entre funciones puras e impuras

En criollo, la diferencia es simple:

	FUNCIÓN PURA	FUNCIÓN IMPURA
¿Que hace?	Solo calcula y devuelve un valor	Puede imprimir, modificar variables, etc.
¿Usa return?	SI, siempre devuelve algo	Puede NO devolver nada (None)
¿Modifica algo externo?	NO	SI (listas, variables globales)
¿Depende de factores externos?	NO	A veces (random, hora actual)
Ejemplo	def suma(a, b): return a + b	def saludar(nombre): print(f"Hola {nombre}")

¿Cuándo usar una y cuándo usar la otra?

Usá funciones PURAS cuando:

- Necesitás calcular algo y usar el resultado más adelante.
- Estás haciendo operaciones matemáticas.
- Querés que tu código sea fácil de probar.

Usá funciones IMPURAS cuando:

- Necesitás mostrar algo en pantalla (ahí sí o sí `print`).
- Necesitás modificar una lista global (como en el carrito de compras).
- Estás leyendo datos del usuario o escribiendo archivos.

En la práctica, mezclamos ambas. Lo importante es entender qué devuelve cada una y si tiene efectos secundarios.

Ejercicios propuestos

Ahora te toca a vos. Escribí cada función y probala. No te saltees ninguno.

1. Escribí una función pura que reciba dos números y devuelva su producto.
2. Escribí una función impura que reciba un número y muestre en pantalla si es par o impar.
3. Escribí una función que reciba una lista de números y devuelva el promedio (pura).
4. Escribí una función que reciba un nombre y un apellido y devuelva el nombre completo (pura).
5. Escribí una función impura que reciba un nombre y muestre “Bienvenido [nombre]”.
6. Escribí una función que reciba base y altura de un triángulo y devuelva el área (pura).
7. Escribí una función `es_mayor_de_edad(edad)` que devuelva `True` si la edad es ≥ 18 , sino `False` (pura).
8. Escribí una función `maximo(a, b, c)` que devuelva el número más grande entre tres (pura).
9. Escribí una función que reciba una lista y devuelva una nueva lista solo con los números pares (pura).
10. Escribí una función `invertir_palabra(palabra)` que devuelva la palabra al revés (pura).
11. Escribí una función que reciba el total de una compra y devuelva el total con IVA (21 %) (pura).
12. Escribí una función impura que muestre un menú con 3 opciones y pida al usuario elegir una.
13. Escribí una función que reciba una lista de números y devuelva el número más chico (pura, sin usar `min()`).
14. Escribí una función que reciba una lista de strings y devuelva una nueva lista con los strings en mayúscula (pura).
15. Escribí una función `contar_vocales(texto)` que devuelva cuántas vocales tiene el string (pura).

16. Escribí una función `es_capicua(numero)` que devuelva `True` si el número se lee igual al revés (pura).
17. Escribí una función `generar_tabla(numero)` que muestre la tabla de multiplicar del número (impura).
18. Escribí una función `factorial(n)` que devuelva el factorial de `n` (pura).
19. Escribí una función `es_primo(n)` que devuelva `True` si `n` es primo (pura).
20. Escribí un programa completo que use al menos 3 funciones propias (puras e impuras) para simular una calculadora de propinas: total de cuenta, porcentaje de propina, calcular total final y mostrar resultados.

Soluciones paso a paso

Solución 1

```
def producto(a, b):  
    return a * b
```

Explicación: Recibe dos números y los multiplica directamente.

Solución 2

```
def es_par_o_impar(numero):  
    if numero % 2 == 0:  
        print(f"{numero} es par")  
    else:  
        print(f"{numero} es impar")
```

Explicación: Imprime en pantalla sin devolver nada (impura).

Solución 3

```
def promedio(lista):  
    if len(lista) == 0:  
        return 0  
    return sum(lista) / len(lista)
```

Explicación: Suma todos los elementos y divide por la cantidad.

Solución 4

```
def nombre_completo(nombre, apellido):  
    return f"{nombre} {apellido}"
```

Explicación: Concatena con f-string y devuelve.

Solución 5

```
def bienvenida(nombre):  
    print(f"Bienvenido {nombre}")
```

Explicación: Solo imprime, no devuelve.

Solución 6

```
def area_triangulo(base, altura):  
    return (base * altura) / 2
```

Explicación: Fórmula clásica.

Solución 7

```
def es_mayor_de_edad(edad):  
    return edad >= 18
```

Explicación: Comparación directa que devuelve booleano.

Solución 8

```
def maximo(a, b, c):  
    if a >= b and a >= c:  
        return a  
    elif b >= a and b >= c:  
        return b  
    else:  
        return c
```

Explicación: Compara los tres y devuelve el mayor.

Solución 9

```
def numeros_pares(lista):  
    resultado = []  
    for num in lista:  
        if num % 2 == 0:  
            resultado.append(num)  
    return resultado
```

Explicación: Filtra los pares y los devuelve en una nueva lista.

Solución 10

```
def invertir_palabra(palabra):  
    return palabra[::-1]
```

Explicación: Slicing con paso -1 invierte el string.

Solución 11

```
def con_iva(total):  
    return total * 1.21
```

Explicación: Multiplica por 1.21 para agregar el 21 %.

Solución 12

```
def mostrar_menu():  
    print("1. Opcion A")  
    print("2. Opcion B")  
    print("3. Opcion C")  
    return input("Elegi una opcion: ")
```

Explicación: Imprime el menú y devuelve la elección del usuario.

Solución 13

```
def minimo(lista):
    if len(lista) == 0:
        return None
    menor = lista[0]
    for num in lista:
        if num < menor:
            menor = num
    return menor
```

Explicación: Recorre la lista buscando el menor.

Solución 14

```
def mayusculas(lista):
    resultado = []
    for palabra in lista:
        resultado.append(palabra.upper())
    return resultado
```

Explicación: Convierte cada string a mayúsculas.

Solución 15

```
def contar_vocales(texto):
    vocales = "aeiouAEIOU"
    contador = 0
    for letra in texto:
        if letra in vocales:
            contador += 1
    return contador
```

Explicación: Cuenta mayúsculas y minúsculas.

Solución 16

```
def es_capicua(numero):
    texto = str(numero)
    return texto == texto[::-1]
```

Explicación: Compara el string con su reverso.

Solución 17

```
def generar_tabla(numero):
    for i in range(1, 11):
        print(f"{numero} x {i} = {numero * i}")
```

Explicación: Impura, imprime la tabla completa.

Solución 18

```
def factorial(n):
    resultado = 1
    for i in range(1, n + 1):
        resultado = resultado * i
    return resultado
```

Explicación: Acumula el producto desde 1 hasta n.

Solución 19

```
def es_primo(n):
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Explicación: Verifica divisores hasta la raíz cuadrada.

Solución 20

```
def calcular_propina(total, porcentaje):
    return total * porcentaje / 100

def calcular_total_final(total, propina):
    return total + propina

def mostrar_resultado(total, propina, total_final, porcentaje):
    print(f"Total de la cuenta: ${total:.2f}")
    print(f"Propina ({porcentaje}%): ${propina:.2f}")
    print(f"Total final: ${total_final:.2f}")

# Programa principal
total_cuenta = float(input("Total de la cuenta: $"))
porcentaje = float(input("Porcentaje de propina: "))
propina = calcular_propina(total_cuenta, porcentaje)
total_con_propina = calcular_total_final(total_cuenta, propina)
mostrar_resultado(total_cuenta, propina, total_con_propina, porcentaje)
```

Explicación: Dos funciones puras calculan, una impura muestra.

Resumen del capítulo

Dale, repasemos rápido lo importante:

- Una **función** es un bloque de código reutilizable que hace una tarea.
- Las funciones pueden ser **puras** (sin efectos secundarios, devuelven algo) o **impuras** (con efectos secundarios, como imprimir o modificar algo externo).
- Las funciones puras usan **return** para entregar un valor; las impuras a veces no retornan nada (**None**).
- Las funciones con listas pueden modificar la lista original (efecto secundario) o devolver una nueva.
- Organizar el código en funciones lo vuelve más limpio, mantenible y reutilizable.
- Combinando funciones puras e impuras podés armar programas completos, como calculadoras, cajeros automáticos o carritos de compras.

Las funciones son como ladrillos: con ellos podés construir programas grandes y complejos de manera ordenada y reutilizable.

Diccionarios: Organizando Datos con Claves

¿Qué es un diccionario?

Cuando empecé a programar, uno de los primeros dolores de cabeza fue encontrar un dato rápido. Tenía listas con nombres y edades, y para buscar la edad de “Ana” tenía que recorrer toda la lista hasta dar con ella. Era como buscar una aguja en un pajar. Un compañero de la facu (UBA Exactas) me dijo: “Che, Nahuel, ¿por qué no usás un diccionario? Es como un índice de libro: vas directo al capítulo sin hojear todo”. Y tenía razón. Desde ese día, los diccionarios se volvieron mis favoritos.

En criollo, un **diccionario** es una estructura que guarda información en pares de **clave** y **valor**. A diferencia de las listas, que usan índices numéricos (0, 1, 2...), acá las claves pueden ser texto, números, lo que vos quieras. Es como tener casilleros etiquetados: en el casillero “nombre” guardás “Nahuel”, en “edad” guardás 25.

```
+-----+
|
|  DICCIONARIO = { clave1: valor1,
|                  clave2: valor2,
|                  clave3: valor3 }
|
|  Las claves son UNICAS (no se repiten)
|  Los valores pueden ser cualquier cosa
|
+-----+
```

Analogía: Imaginá una guía telefónica antigua. Buscás el nombre (clave) y encontrás el número (valor). En Python es igual de simple.

¿Cómo se crea un diccionario?

Se arma con **llaves** {} y dos puntos : entre clave y valor. Cada par se separa con comas. Fijate:

```
# Diccionario vacío
diccionario_vacio = {}
```

```
# Diccionario con datos de una persona
persona = {
```

```

    "nombre": "Nahuel",
    "edad": 25,
    "ciudad": "Buenos Aires"
}

print(persona)

{'nombre': 'Nahuel', 'edad': 25, 'ciudad': 'Buenos Aires'}
```

Ejercicio rápido: Creá un diccionario con tus datos: nombre, edad y tu banda favorita. Mostralo.

Un ejemplo real: la bolsa de la compra

En mis primeros programas, cuando quería armar un carrito de compras, usaba listas de diccionarios. Cada producto era un diccionario con 'nombre', 'precio' y 'cantidad'. Después los metía todos en una lista. Es una técnica que vas a usar un montón.

```

# Creamos la lista vacía
bolsa = []

# Cada producto es un diccionario
producto_pan = {
    'nombre': 'pan',
    'precio': 70,
    'cantidad': 2
}

producto_ajo = {
    'nombre': 'ajo',
    'precio': 701,
    'cantidad': 2
}

producto_donas = {
    'nombre': 'donas',
    'precio': 704,
    'cantidad': 2
}

# Los agregamos a la lista
bolsa.append(producto_pan)
bolsa.append(producto_ajo)
bolsa.append(producto_donas)

print("Bolsa de compras:", bolsa)

Bolsa de compras: [{'nombre': 'pan', 'precio': 70, 'cantidad': 2},
                    {'nombre': 'ajo', 'precio': 701, 'cantidad': 2},
                    {'nombre': 'donas', 'precio': 704, 'cantidad': 2}]
```

Reflexión: ¿Ves la potencia? Tenés una lista donde cada elemento es un diccionario, como una mini base de datos. Más adelante podés recorrerla, filtrar, sumar precios... ¡un golazo!

Acceder a los valores de un diccionario

Para sacar un valor, usás el nombre del diccionario y, entre corchetes, la **clave**. Es como decirle: “Dame lo que hay en el casillero tal”.

```
# Acceder a valores de un producto
print("Nombre del producto:", producto_pan['nombre'])
print("Precio del producto:", producto_pan['precio'])
print("Cantidad:", producto_pan['cantidad'])
```

```
Nombre del producto: pan
Precio del producto: 70
Cantidad: 2
```

Si tenés una lista de diccionarios, combinás el índice de la lista con la clave del diccionario. ¡Ojo al orden!

```
# Accedo al nombre del producto que está en la posición 1 (ajo)
print(bolsa[1]['nombre']) # ajo
print(bolsa[2]['precio']) # 704 (precio de las donas)
```

Nota: *Diferencia clave entre listas y diccionarios:*

- **Listas:** se accede por *índice numérico* (0, 1, 2...).
- **Diccionarios:** se accede por *clave* (“nombre”, “edad”...).

Error común: Si ponés una clave que no existe, Python te tira un `KeyError`. Para evitarlo, más adelante vas a conocer `.get()`.

Recorrer diccionarios

Una de las cosas que más hacía en la facultad era recorrer diccionarios para mostrar datos. Hay varias formas, dependiendo de lo que necesites.

Recorrer solo las claves

```
for clave in producto_pan:
    print(f"Clave: {clave}")
```

Recorrer solo los valores

```
for valor in producto_pan.values():
    print(f"Valor: {valor}")
```

Recorrer claves y valores juntos con `.items()`

Este es el método estrella. Te devuelve cada par clave-valor como una tupla. En criollo, te da la etiqueta y el contenido al mismo tiempo.

```
for clave, valor in producto_pan.items():
    print(f"{clave} -> {valor}")
```

```
nombre -> pan
precio -> 70
cantidad -> 2
```

Reflexión: `.items()` es ideal cuando necesitás hacer algo con la clave y el valor al mismo tiempo, como imprimir una ficha completa.

El método `.items()` en detalle

Cuando usás `.items()`, Python te devuelve un objeto especial, pero podés convertirlo a lista para manipularlo. Fijate:

```
diccionario = {"a": 1, "b": 2, "c": 3}
print("Items originales:", diccionario.items())

# Lo convierto a lista y puedo acceder por índice
items_lista = list(diccionario.items())
print("Posición 1:", items_lista[1])    # ('b', 2)
```

Nota: *¿Por qué usar `.items()`?*

- Para recorrer el diccionario cuando necesitás clave Y valor.
- Para inspeccionar todo el contenido de una.
- Porque es la forma más pitónica de hacerlo.

Modificar un diccionario

Los diccionarios son mutables: podés agregar, cambiar o borrar elementos a piacere.

Agregar o modificar un valor

Con la misma sintaxis de acceso, si la clave no existe se crea; si existe, se modifica.

```
producto_pan['marca'] = 'Bimbo'    # Agrega una nueva clave
producto_pan['precio'] = 80        # Modifica el valor de 'precio'
print(producto_pan)
```

```
{'nombre': 'pan', 'precio': 80, 'cantidad': 2, 'marca': 'Bimbo'}
```

Eliminar una clave con del

```
del producto_pan['marca']    # Elimina la clave 'marca'
print(producto_pan)
```

Eliminar una clave con pop()

pop() elimina la clave y te devuelve el valor que tenía, por si lo querés guardar.

```
precio_eliminado = producto_pan.pop('precio')
print(f"Precio eliminado: {precio_eliminado}")
print(producto_pan)    # Ya no tiene 'precio'
```

Métodos útiles de diccionarios

METODO	DESCRIPCION
.keys()	Devuelve todas las claves del diccionario
.values()	Devuelve todos los valores del diccionario
.items()	Devuelve todos los pares (clave, valor)
.get(clave)	Devuelve el valor o None si no existe
.pop(clave)	Elimina una clave y devuelve su valor
.update(otro)	Actualiza el diccionario con otro
.clear()	Vacia el diccionario

Probemos algunos:

```
producto = {'nombre': 'pan', 'precio': 70, 'cantidad': 2}

print("Claves:", producto.keys())
print("Valores:", producto.values())

# get() es mas seguro que [] porque no da error si la clave no existe
print(producto.get('marca', 'No tengo esa clave'))

# update() agrega o actualiza varias claves de una
producto.update({'precio': 85, 'marca': 'Bimbo'})
print(producto)

# clear() vacia todo
producto.clear()
print(producto)    # {}
```

Sistema de Login con Diccionario (proyecto real)

Este es un programita que hice en mis primeras prácticas. Usa un diccionario para guardar usuarios y contraseñas. Fijate cómo se combinan las claves con las herramientas que vimos.

```
usuarios_registrados = {}

while True:
    print("\n=== SISTEMA DE LOGIN ===")
    print("1. Registrarse")
    print("2. Iniciar sesion")
    print("3. Ver todos los usuarios")
    print("4. Salir")

    opcion = input("Elegi una opcion: ")

    if opcion == "1":
        usuario = input("Ingresa un nombre de usuario: ")
        if usuario in usuarios_registrados:
            print("Ese usuario ya existe. Elegi otro.")
        else:
            contrasena = input("Ingresa una contrasena: ")
            usuarios_registrados[usuario] = contrasena
            print(f"¡Usuario {usuario} registrado con exito!")

    elif opcion == "2":
        usuario = input("Usuario: ")
        contrasena = input("Contrasena: ")
        if usuario in usuarios_registrados and usuarios_registrados[usuario] == contrasena:
            print(f"¡Bienvenido {usuario}!")
        else:
            print("Usuario o contrasena incorrectos.")

    elif opcion == "3":
        if len(usuarios_registrados) == 0:
            print("No hay usuarios registrados.")
        else:
            print("\n--- USUARIOS REGISTRADOS ---")
            for user in usuarios_registrados:
                print(f"- {user}")

    elif opcion == "4":
        print("¡Hasta luego!")
        break
    else:
        print("Opcion no valida.")
```

Carrito de Compras con Diccionesarios (proyecto integrador)

Otro clásico: un carrito de compras que usa una lista de diccionarios. Cada producto es un diccionario, y la lista los agrupa. Incluye funciones para todo, como hacíamos en los capítulos

anteriores.

```
carrito = []
```

```
def agregar_producto():
    nombre = input("Nombre del producto: ")
    precio = float(input("Precio: $"))
    cantidad = int(input("Cantidad: "))
    producto = {"nombre": nombre, "precio": precio, "cantidad": cantidad}
    carrito.append(producto)
    print(f"'{nombre}' agregado al carrito.")
```

```
def ver_carrito():
    if not carrito:
        print("El carrito esta vacio.")
        return
    print("\n--- CARRITO ---")
    for i, item in enumerate(carrito, 1):
        subtotal = item["precio"] * item["cantidad"]
        print(f"{i}. {item['nombre']} - ${item['precio']:.2f} x {item['cantidad']} = ${s
```

```
def calcular_total():
    if not carrito:
        print("El carrito esta vacio.")
        return
    total = sum(item["precio"] * item["cantidad"] for item in carrito)
    print(f"Total a pagar: ${total:.2f}")
```

```
def eliminar_producto():
    if not carrito:
        print("El carrito esta vacio.")
        return
    ver_carrito()
    try:
        indice = int(input("Numero del producto a eliminar: ")) - 1
        if 0 <= indice < len(carrito):
            eliminado = carrito.pop(indice)
            print(f"'{eliminado['nombre']}' eliminado.")
        else:
            print("Numero invalido.")
    except ValueError:
        print("Ingresa un numero valido.")
```

```
# Programa principal
while True:
    print("\n=== CARRITO DE COMPRAS ===")
    print("1. Agregar producto")
    print("2. Ver carrito")
    print("3. Calcular total")
```



```
print("4. Eliminar producto")
print("5. Salir")
opcion = input("Elegi una opcion: ")

if opcion == "1":
    agregar_producto()
elif opcion == "2":
    ver_carrito()
elif opcion == "3":
    calcular_total()
elif opcion == "4":
    eliminar_producto()
elif opcion == "5":
    print("¡Gracias por la compra!")
    break
else:
    print("Opcion no valida.")
```

Ejercicios propuestos

Ahora es tu turno. Agarrá la terminal y no mires las soluciones hasta haber peleado un rato.

1. Creá un diccionario `persona` con las claves `"nombre"`, `"edad"`, `"ciudad"`. Asignales valores y mostralo.
2. Accedé al valor de la clave `"edad"` de ese diccionario y mostralo.
3. Agregá una nueva clave `"profesion"` al diccionario y asignale un valor.
4. Modificá el valor de la clave `"ciudad"` del diccionario.
5. Eliminá la clave `"edad"` usando `del`.
6. Creá un diccionario con 3 productos (clave: nombre del producto, valor: precio). Mostrá todos.
7. Recorré el diccionario anterior con `.items()` y mostrá cada producto con su precio.
8. Creá una lista de 3 diccionarios, cada uno representando una persona con nombre, edad y ciudad. Recorré la lista y mostrá los datos de cada una.
9. Agregá un nuevo diccionario (persona) a la lista del ejercicio anterior.
10. Usando el diccionario de productos del ejercicio 6, calculá el precio total de comprar 2 de cada producto.
11. Creá un diccionario que guarde las notas de 3 materias (clave: materia, valor: nota). Calculá el promedio.
12. Usando `.get()`, intentá acceder a una clave que no existe y mostrá un mensaje personalizado.

13. Creá un diccionario vacío. Pedí al usuario 3 nombres y edades, y guardalos en el diccionario.
14. Creá un programa que permita buscar un producto por su nombre en un diccionario de precios.
15. Mostrá todas las claves de un diccionario usando `.keys()`.
16. Mostrá todos los valores de un diccionario usando `.values()`.
17. Creá un diccionario con los meses del año (1: "Enero", 2: "Febrero", ...). Pedí un número y mostrá el mes correspondiente.
18. Creá un diccionario que guarde el inventario de una tienda (producto: cantidad). Simulá una venta: pedí un producto y restá 1 a su cantidad.
19. Uní dos diccionarios con datos de personas usando `.update()`.
20. Creá una agenda telefónica completa con diccionario (nombre: teléfono). Debe permitir agregar, buscar, eliminar y mostrar todos los contactos.

Soluciones paso a paso

Solución 1

```
persona = {"nombre": "Nahuel", "edad": 25, "ciudad": "Buenos Aires"}  
print(persona)
```

Explicación: Usamos llaves y separamos con comas cada par clave-valor.

Solución 2

```
print(persona["edad"])    # 25
```

Explicación: Accedemos con corchetes y la clave exacta.

Solución 3

```
persona["profesion"] = "programador"  
print(persona)
```

Explicación: Al asignar a una clave nueva, Python la agrega automáticamente.

Solución 4

```
persona["ciudad"] = "La Plata"  
print(persona)
```

Explicación: Si la clave ya existe, se sobrescribe el valor.

Solución 5

```
del persona["edad"]  
print(persona)
```

Explicación: del elimina la clave y su valor asociado.

Solución 6

```
productos = {"pan": 70, "leche": 90, "huevos": 120}  
print(productos)
```

Explicación: Las claves son los nombres, los valores los precios.

Solución 7

```
for producto, precio in productos.items():  
    print(f"{producto}: ${precio}")
```

Explicación: .items() nos da cada par en cada vuelta.

Solución 8

```
personas = [  
    {"nombre": "Ana", "edad": 22, "ciudad": "CABA"},  
    {"nombre": "Luis", "edad": 25, "ciudad": "Cordoba"},  
    {"nombre": "Marta", "edad": 21, "ciudad": "Rosario"}  
]  
for p in personas:  
    print(f"{p['nombre']} tiene {p['edad']} años y vive en {p['ciudad']}")
```

Explicación: Recorremos la lista y por cada diccionario accedemos a sus claves.

Solución 9

```
personas.append({"nombre": "Pedro", "edad": 23, "ciudad": "Mendoza"})  
print(personas)
```

Explicación: Agregamos un diccionario nuevo a la lista con `append()`.

Solución 10

```
total = 0  
for producto, precio in productos.items():  
    total += precio * 2  
print(f"Total por 2 de cada producto: ${total}")
```

Explicación: Multiplicamos cada precio por 2 y sumamos.

Solución 11

```
notas = {"matematica": 8, "fisica": 7, "quimica": 9}  
promedio = sum(notas.values()) / len(notas)  
print(f"Promedio: {promedio:.2f}")
```

Explicación: `sum()` suma todos los valores de una; `len()` cuenta las materias.

Solución 12

```
valor = persona.get("altura", "La clave 'altura' no existe")  
print(valor)
```

Explicación: `.get()` con segundo argumento evita el error y devuelve un mensaje alternativo.

Solución 13

```
datos = {}  
for i in range(3):  
    nombre = input("Nombre: ")  
    edad = input("Edad: ")  
    datos[nombre] = edad  
print(datos)
```

Explicación: El usuario va llenando el diccionario; la clave es el nombre, el valor la edad.

Solución 14

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
buscar = input("Producto a buscar: ")
if buscar in productos:
    print(f"Precio: ${productos[buscar]}")
else:
    print("Producto no encontrado")
```

Explicación: Verificamos pertenencia con `in` y luego accedemos.

Solución 15

```
print(persona.keys())
```

Explicación: Devuelve una vista con todas las claves.

Solución 16

```
print(persona.values())
```

Explicación: Similar pero con los valores.

Solución 17

```
meses = {1:"Enero",2:"Febrero",3:"Marzo",4:"Abril",5:"Mayo",6:"Junio",
        7:"Julio",8:"Agosto",9:"Septiembre",10:"Octubre",11:"Noviembre",12:"Diciembre"}
num = int(input("Numero de mes: "))
print(meses.get(num, "Numero invalido"))
```

Explicación: Mapeamos número a nombre y usamos `get()` para evitar errores.

Solución 18

```
inventario = {"pan": 10, "leche": 5, "huevos": 12}
producto = input("Producto a vender: ")
if producto in inventario and inventario[producto] > 0:
    inventario[producto] -= 1
    print(f"Vendido. Quedan {inventario[producto]} de {producto}")
else:
    print("Producto no disponible")
```

Explicación: Verificamos disponibilidad y descontamos una unidad.

Solución 19

```
p1 = {"nombre": "Ana", "edad": 22}
p2 = {"ciudad": "CABA", "profesion": "ingeniera"}
p1.update(p2)
print(p1)
```

Explicación: `update()` vuelca las claves de un diccionario en otro.

Solución 20

```
agenda = {}
while True:
    print("\n1. Agregar\n2. Buscar\n3. Eliminar\n4. Mostrar\n5. Salir")
    op = input("Opcion: ")
    if op == "1":
        nombre = input("Nombre: ")
        tel = input("Telefono: ")
        agenda[nombre] = tel
    elif op == "2":
        nombre = input("Buscar: ")
        print(agenda.get(nombre, "No encontrado"))
    elif op == "3":
        nombre = input("Eliminar: ")
        if nombre in agenda:
            del agenda[nombre]
            print("Eliminado")
        else:
            print("No encontrado")
    elif op == "4":
        for n, t in agenda.items():
            print(f"{n}: {t}")
    elif op == "5":
        break
```

Explicación: Menú clásico que utiliza todas las operaciones básicas de diccionarios.

Resumen del capítulo

En este capítulo aprendiste:

- Un **diccionario** es una colección de pares clave-valor, ideal para acceder rápido a los datos sin importar el orden.
- Se crean con `{}` y cada elemento se separa con comas; clave y valor van unidos por `:`.
- Las claves son únicas y pueden ser de varios tipos; los valores también.
- Se accede con `diccionario[clave]`, pero con `.get()` evitás errores si la clave no existe.
- Podés modificar, agregar y eliminar elementos libremente.
- Los métodos `.keys()`, `.values()` e `.items()` te permiten recorrerlos a placer.
- Las listas de diccionarios son una herramienta poderosa para manejar colecciones de objetos (productos, personas, etc.).
- En la práctica, combinar diccionarios con funciones y menús te permite armar programas completos como sistemas de login, carritos de compras o agendas.

Los diccionarios son como un placard con cajones etiquetados: ponés lo que quieras y lo encontrás al toque.

Método items() - Recorrer Diccionarios

¿Qué es el método items()?

El método `.items()` se utiliza para obtener todos los elementos de un diccionario. Un **item** es el conjunto compuesto por una clave y su valor correspondiente.

- **Qué devuelve:** Un objeto de tipo `dict_items`, que contiene una lista de **tuplas** (pares clave-valor).
- **Sintaxis:** `nombre_diccionario.items()`

Como decía Nahuel en la facultad (UBA Exactas): `items()` es como sacar una foto de todo el diccionario: ves cada clave con su valor al lado."

```
+-----+
|
|  diccionario = {"a": 1, "b": 2, "c": 3}
|
|  diccionario.items()
|
|  Resultado: dict_items([('a', 1), ('b', 2),
|                      ('c', 3)])
|
+-----+
```

¿Para qué sirve?

El método `.items()` es fundamental cuando necesitás:

- **Recorrer un diccionario** y necesitás tanto la clave como el valor al mismo tiempo.
- **Inspeccionar** rápidamente el contenido completo de un diccionario.
- Cuando **desconocés qué claves** tiene el diccionario.

Ejemplo básico

```
# 1. Creación del diccionario
diccionario = {"a": 1, "b": 2, "c": 3}
print(f"Diccionario original: {diccionario}")
```



```
# 2. Uso de .items() directamente
print(f"Los items del diccionario son: {diccionario.items()}")
```

Salida:

Diccionario original: {'a': 1, 'b': 2, 'c': 3}

Los items del diccionario son: dict_items([('a', 1), ('b', 2), ('c', 3)])

Convertir items() a lista

El objeto que devuelve `.items()` (`dict_items`) no permite manipular los elementos de forma independiente (por ejemplo, mediante índices). Para poder hacerlo, se suele convertir a una **lista real** usando el constructor `list()`.

```
# Conversión a lista para manipulación independiente
items_lista = list(diccionario.items())
print(f"La lista convertida es: {items_lista}")
```

```
# Acceso por posición (índice)
print(f"Posición 0 de la lista de items: {items_lista[0]}")
print(f"Posición 1 de la lista de items: {items_lista[1]}")
```

Salida:

La lista convertida es: [('a', 1), ('b', 2), ('c', 3)]

Posición 0 de la lista de items: ('a', 1)

Posición 1 de la lista de items: ('b', 2)

Nota: *¿Qué es una tupla? Los pares devueltos por `.items()` vienen encerrados en paréntesis ('clave', 'valor'). Eso es una **tupla**. Es como una lista, pero NO se puede modificar (es *immutable*).*

Recorrer diccionarios con items()

La forma más común de usar `.items()` es dentro de un bucle `for` para recorrer el diccionario.

```
# Diccionario de productos
productos = {"manzana": 500, "pera": 600, "uva": 700}

# Recorremos con .items()
for producto, precio in productos.items():
    print(f"El producto '{producto}' cuesta ${precio}")
```

Salida:

El producto 'manzana' cuesta \$500

El producto 'pera' cuesta \$600

El producto 'uva' cuesta \$700

Ejemplo práctico: Agenda de contactos

```
agenda = {  
    "Nahuel": "11-1234-5678",  
    "Ana": "11-8765-4321",  
    "Luis": "11-5678-1234"  
}  
  
print("--- AGENDA DE CONTACTOS ---")  
for nombre, telefono in agenda.items():  
    print(f"{nombre}: {telefono}")
```

Salida:

```
--- AGENDA DE CONTACTOS ---  
Nahuel: 11-1234-5678  
Ana: 11-8765-4321  
Luis: 11-5678-1234
```

Ejercicios

Ejercicio 1

Creá un diccionario con 3 países y sus capitales. Usá `.items()` para mostrar "La capital de [país] es [capital]".

Ejercicio 2

Creá un diccionario con 5 alumnos y sus notas. Usá `.items()` para mostrar cada alumno con su nota.

Ejercicio 3

Usando el diccionario del ejercicio 2, convertí `.items()` a lista y mostrá el tercer elemento.

Ejercicio 4

Creá un diccionario con 4 productos y sus precios. Usá `.items()` para mostrar solo los productos que cuesten más de \$1000.

Ejercicio 5

Creá un diccionario con 3 personas (nombre, edad). Usá `.items()` para calcular el promedio de edad.

Ejercicio 6

Creá un diccionario con 5 ciudades y su población. Usá `.items()` para mostrar la ciudad con mayor población.

Ejercicio 7

Creá un diccionario con los resultados de un examen (nombre: nota). Usá `.items()` para contar cuántos aprobaron (nota ≥ 7).

Ejercicio 8

Creá un diccionario con 3 materias y sus notas. Usá `.items()` para calcular cuántas materias están aprobadas (nota ≥ 7).

Ejercicio 9

Creá un diccionario con 4 empleados y sus sueldos. Usá `.items()` para calcular el sueldo total de todos.

Ejercicio 10

Creá un diccionario con los nombres de 5 amigos y sus edades. Usá `.items()` para mostrar quién es el más joven.

Soluciones

Solución 1

```
capitales = {"Argentina": "Buenos Aires", "Brasil": "Brasilia", "Chile": "Santiago"}
for pais, capital in capitales.items():
    print(f"La capital de {pais} es {capital}")
```

Solución 2

```
notas = {"Ana": 8, "Luis": 7, "Pedro": 9, "Marta": 6, "Juan": 10}
for alumno, nota in notas.items():
    print(f"{alumno} sacó {nota}")
```

Solución 3

```
notas = {"Ana": 8, "Luis": 7, "Pedro": 9, "Marta": 6, "Juan": 10}
lista_items = list(notas.items())
print(f"El tercer elemento es: {lista_items[2]}")
```

Solución 4

```
productos = {"Mouse": 1500, "Teclado": 2500, "Monitor": 50000, "Pendrive": 800}
for producto, precio in productos.items():
    if precio > 1000:
        print(f"{producto}: ${precio}")
```

Solución 5

```
personas = {"Ana": 22, "Luis": 25, "Pedro": 21}
suma = 0
for nombre, edad in personas.items():
    suma = suma + edad
promedio = suma / len(personas)
print(f"El promedio de edad es: {promedio}")
```

Solución 6

```
ciudades = {"Buenos Aires": 15000000, "Córdoba": 1500000, "Rosario": 1200000,
            "Mendoza": 1000000, "La Plata": 800000}
max_poblacion = 0
ciudad_max = ""
for ciudad, poblacion in ciudades.items():
    if poblacion > max_poblacion:
        max_poblacion = poblacion
        ciudad_max = ciudad
print(f"La ciudad con mayor población es {ciudad_max} con {max_poblacion} habitantes")
```

Solución 7

```
examen = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 4, "Juan": 7}
aprobados = 0
for nombre, nota in examen.items():
    if nota >= 7:
        aprobados = aprobados + 1
print(f"Aprobaron {aprobados} alumnos")
```

Solución 8

```
materias = {"Matemática": 8, "Física": 5, "Química": 9}
aprobadas = 0
for materia, nota in materias.items():
    if nota >= 7:
        aprobadas = aprobadas + 1
print(f"Materias aprobadas: {aprobadas}")
```

Solución 9

```
suealdos = {"Ana": 50000, "Luis": 60000, "Pedro": 55000, "Marta": 65000}
total = 0
for empleado, sueldo in suealdos.items():
    total = total + sueldo
print(f"Total de suealdos: ${total}")
```

Solución 10

```
edades = {"Ana": 22, "Luis": 25, "Pedro": 21, "Marta": 24, "Juan": 23}
mas_joven = None
edad_min = float('inf')
for nombre, edad in edades.items():
    if edad < edad_min:
        edad_min = edad
        mas_joven = nombre
print(f"El más joven es {mas_joven} con {edad_min} años")
```

Resumen

- `.items()` devuelve todos los pares clave-valor de un diccionario.
- Devuelve un objeto `dict_items` que se puede convertir a lista.
- Es ideal para recorrer diccionarios con `for` cuando necesitás tanto la clave como el valor.
- Los elementos son **tuplas**: (clave, valor).

Método `keys()` – Obtener las Claves de un Diccionario

¿Qué es el método `keys()`?

Cuando empecé a programar con diccionarios, siempre me olvidaba qué claves le había puesto. Tenía que andar imprimiendo todo el diccionario o adivinando, y más de una vez me comí un `KeyError`. Hasta que un compañero me tiró: “¿Probaste con `.keys()`? Te lista todas las etiquetas sin el contenido”. Y ahí me cambió la vida.

El método `.keys()` devuelve una **vista** de todas las claves de un diccionario. Es como mirar el índice de un libro: ves los títulos de los capítulos, pero no las páginas. En criollo, te muestra qué hay en los casilleros, sin abrir cada uno.

- **Qué devuelve:** Un objeto de tipo `dict_keys`, que es una vista de las claves.
- **Sintaxis:** `diccionario.keys()`
- **Utilidad:** Sirve para inspeccionar qué campos existen, verificar pertenencia, ordenar claves, recorrer solo los nombres, etc.

Como decía Nahuel en la facultad (UBA Exactas): “`keys()` es como mirar solo los títulos de las columnas de una tabla, sin ver los datos.”

```
+-----+
|
|  diccionario = {"a": 1, "b": 2, "c": 3}  |
|
|  diccionario.keys()                    |
|
|  Resultado: dict_keys(['a', 'b', 'c']) |
|
|  ¡Solo las claves, los valores se ignoran! |
|
+-----+
```

¿Para qué sirve?

`.keys()` te saca de apuros cuando:

- Necesitás saber qué **claves existen** en un diccionario.

- Querés **verificar** si una clave está antes de acceder (evitando errores).
- Querés **recorrer solo las claves** sin que los valores molesten.
- Necesitás **ordenar** las claves alfabéticamente o trabajar con índices.

Analogía: Imaginá que tenés un guardarropas con cajones etiquetados. `keys()` te da la lista de etiquetas: “camisas”, “pantalones”, “medias”. No te dice cuántas prendas hay adentro, solo los nombres de los cajones.

Ejemplo básico

```
# Creación del diccionario
diccionario = {"a": 1, "b": 2, "c": 3}

# Obtener las claves
print("Las claves son:", diccionario.keys())

Las claves son: dict_keys(['a', 'b', 'c'])
```

Reflexión: El resultado parece una lista, pero es un objeto especial. Si querés manipularlo con índices, tenés que convertirlo a lista.

Conversión a lista

El objeto `dict_keys` no soporta acceso por índice. Para trabajar con él como una lista común, usás `list()`. Una vez convertido, podés ordenarlo, acceder por posición, etc.

```
# Convertir a lista real
lista_de_claves = list(diccionario.keys())
print("Lista de claves:", lista_de_claves)

# Acceder por índice
print("Primera clave:", lista_de_claves[0])
print("Segunda clave:", lista_de_claves[1])
print("Tercera clave:", lista_de_claves[2])

Lista de claves: ['a', 'b', 'c']
Primera clave: a
Segunda clave: b
Tercera clave: c
```

Nota: *Conversión a lista:* Una vez que tenés `lista_de_claves`, podés:

- Ordenar: `lista_de_claves.sort()`
- Eliminar un elemento: `lista_de_claves.pop(0)` (pero no afecta al diccionario original)
- Recorrerla como cualquier lista.

Recordá que la lista es una copia de las claves; los cambios no repercuten en el diccionario.

Verificar si una clave existe

Una de las aplicaciones más útiles es evitar el temido `KeyError`. Usando `in` sobre `keys()` (o directamente sobre el diccionario) podés comprobar si una clave está presente antes de intentar acceder.

```
persona = {"nombre": "Nahuel", "edad": 25, "ciudad": "Buenos Aires"}

# Usando .keys()
if "nombre" in persona.keys():
    print("La clave 'nombre' existe")

# También podés usar el diccionario directamente (más corto)
if "apellido" not in persona:
    print("'apellido' no está, agrego uno por defecto")
    persona["apellido"] = "Desconocido"

print(persona)

La clave 'nombre' existe
'apellido' no está, agrego uno por defecto
{'nombre': 'Nahuel', 'edad': 25, 'ciudad': 'Buenos Aires', 'apellido': 'Desconocido'}
```

Recorrer solo las claves de un diccionario

Cuando solo te interesan los nombres de los campos, un `for` con `.keys()` es la opción más clara.

```
productos = {"manzana": 500, "pera": 600, "uva": 700, "naranja": 400}

print("=== LISTA DE PRODUCTOS ===")
for producto in productos.keys():
    print(f"- {producto}")

=== LISTA DE PRODUCTOS ===
- manzana
- pera
- uva
- naranja
```

Reflexión: ¿Es necesario usar `.keys()` acá? Si solo escribís `for producto in productos:` también itera sobre las claves por defecto. Pero usar `.keys()` lo hace explícito y legible.

Ejemplo práctico: Sistema de notas

Imaginá que querés mostrar los nombres de los alumnos que aprobaron. Con `.keys()` y un `if` lo tenés servido.


```
notas = {
    "Ana": 8,
    "Luis": 5,
    "Pedro": 9,
    "Marta": 6,
    "Juan": 10
}

print("=== ALUMNOS QUE APROBARON (nota >= 7) ===")
for alumno in notas.keys():
    if notas[alumno] >= 7:
        print(f"- {alumno}")

=== ALUMNOS QUE APROBARON (nota >= 7) ===
- Ana
- Pedro
- Juan
```

Ordenar las claves de un diccionario

Muchas veces necesitás mostrar los datos ordenados, por ejemplo, alfabéticamente. Convertís las claves a lista, las ordenás, y luego recorrés esa lista.

```
edades = {"Luis": 25, "Ana": 22, "Pedro": 30, "Marta": 21}

# Obtener claves ordenadas
nombres_ordenados = sorted(edades.keys()) # sorted() ya devuelve una lista

print("Nombres ordenados alfabeticamente:")
for nombre in nombres_ordenados:
    print(f"- {nombre}: {edades[nombre]} años")

Nombres ordenados alfabeticamente:
- Ana: 22 años
- Luis: 25 años
- Marta: 21 años
- Pedro: 30 años
```

• Ejemplo práctico: Inventario de tienda

• Otro uso clásico: detectar productos con stock bajo.

```
inventario = {
    "manzanas": 50,
    "peras": 30,
    "naranjas": 40,
    "bananas": 25
}
```

```
print("=== PRODUCTOS CON STOCK BAJO (< 30) ===")
for producto in inventario.keys():
    if inventario[producto] < 30:
        print(f"¡Quedan pocas {producto}! Solo {inventario[producto]} unidades")

=== PRODUCTOS CON STOCK BAJO (< 30) ===
¡Quedan pocas bananas! Solo 25 unidades
```

Ejercicios propuestos

Ahora practicá vos. Son 20 ejercicios para que `keys()` te quede grabado en el ADN. No mires las soluciones hasta que los hayas intentado.

1. Creá un diccionario con 3 personas y sus edades. Usá `.keys()` para mostrar solo los nombres.
2. Creá un diccionario con 5 países y sus capitales. Usá `.keys()` para verificar si “Argentina” está en el diccionario.
3. Creá un diccionario con 4 productos y sus precios. Convertí `.keys()` a lista y mostrá el segundo producto.
4. Creá un diccionario con 3 materias y sus notas. Usá `.keys()` para mostrar cuántas materias hay.
5. Creá un diccionario con 5 amigos y sus teléfonos. Usá `.keys()` para mostrar todos los nombres en mayúsculas.
6. Creá un diccionario con 4 empleados y sus sueldos. Usá `.keys()` para verificar si “Pedro” está en el diccionario.
7. Creá un diccionario con 3 ciudades y su población. Usá `.keys()` para crear una lista con las ciudades.
8. Creá un diccionario con 5 alumnos y sus notas. Usá `.keys()` para mostrar los alumnos que aprobaron (nota ≥ 7).
9. Creá un diccionario con 4 productos y sus precios. Usá `.keys()` para verificar si “Monitor” existe en el diccionario.
10. Creá un diccionario con 3 libros y sus autores. Usá `.keys()` para mostrar el primer libro de la lista (usando conversión a lista).
11. Pedí al usuario 4 nombres y edades, guardalos en un diccionario y luego mostrá todas las claves.
12. A partir de un diccionario de frutas y precios, creá una lista con las frutas que tienen más de 6 letras, usando `.keys()`.
13. Ordená alfabéticamente las claves de un diccionario de asignaturas y mostralas junto con sus valores correspondientes.

14. Verificá si una clave introducida por el usuario existe en un diccionario dado; si no existe, agregala con un valor por defecto.
15. Contá cuántas claves de un diccionario de meses empiezan con la letra “m”.
16. Creá un diccionario de configuración (host, port, user). Usá `.keys()` y un bucle para generar una cadena de conexión.
17. Dado un diccionario de notas, eliminá la clave de los alumnos que hayan desaprobado (nota <4) recorriendo una copia de `keys()`.
18. Convertí las claves de un diccionario de sinónimos a una lista, reordenalas al azar y mostrá el diccionario en ese orden.
19. Escribí una función `listar_claves(diccionario)` que reciba un diccionario y devuelva una lista con sus claves ordenadas de mayor a menor (según el valor).
20. Simulá una agenda telefónica donde puedas agregar contactos y luego ver todos los nombres ordenados, usando `keys()` y `list()`.

Soluciones paso a paso

Solución 1

```
personas = {"Ana": 22, "Luis": 25, "Pedro": 21}
print("Nombres:", list(personas.keys()))
```

Explicación: Convertimos las claves a lista para mostrarlas de una.

Solución 2

```
países = {"Argentina": "Buenos Aires", "Brasil": "Brasilia", "Chile": "Santiago"}
if "Argentina" in países.keys():
    print("Argentina está en el diccionario")
else:
    print("Argentina NO está")
```

Explicación: Usamos in sobre la vista de claves para chequear existencia.

Solución 3

```
productos = {"Mouse": 1500, "Teclado": 2500, "Monitor": 50000, "Pendrive": 800}
lista_productos = list(productos.keys())
print("Segundo producto:", lista_productos[1]) # Teclado
```

Explicación: La lista conserva el orden de inserción (a partir de Python 3.7).

Solución 4

```
materias = {"Matemática": 8, "Física": 7, "Química": 9}
print(f"Cantidad de materias: {len(materias.keys())}")
```

Explicación: len() sobre las claves da el número de materias.

Solución 5

```
amigos = {"Ana": "1234", "Luis": "5678", "Pedro": "9012", "Marta": "3456", "Juan": "7890"}
for amigo in amigos.keys():
    print(amigo.upper())
```

Explicación: Iteramos sobre las claves y aplicamos upper().

Solución 6

```
empleados = {"Ana": 50000, "Luis": 60000, "Marta": 55000}
if "Pedro" in empleados.keys():
    print("Pedro está en el diccionario")
else:
    print("Pedro NO está")
```

Explicación: Verificamos con in.

Solución 7

```
ciudades = {"Buenos Aires": 15000000, "Córdoba": 1500000, "Rosario": 1200000}
lista_ciudades = list(ciudades.keys())
print("Ciudades:", lista_ciudades)
```

Explicación: Conversión directa a lista.

Solución 8

```
alumnos = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 6, "Juan": 10}
print("Alumnos aprobados:")
for alumno in alumnos.keys():
    if alumnos[alumno] >= 7:
        print(f"- {alumno}")
```

Explicación: Recorremos las claves y accedemos al valor con el diccionario.

Solución 9

```
productos = {"Mouse": 1500, "Teclado": 2500, "Pendrive": 800}
if "Monitor" in productos.keys():
    print("Monitor está en el diccionario")
else:
    print("Monitor NO está")
```

Explicación: Chequeo de existencia.

Solución 10

```
libros = {"Cien años de soledad": "García Márquez",
          "El Quijote": "Cervantes",
          "La sombra del viento": "Ruiz Zafón"}
lista_libros = list(libros.keys())
print("Primer libro:", lista_libros[0])
```

Explicación: Accedemos al índice 0 de la lista de claves.

Solución 11

```
datos = {}
for i in range(4):
    nombre = input("Nombre: ")
    edad = int(input("Edad: "))
    datos[nombre] = edad
print("Claves:", list(datos.keys()))
```

Explicación: Construimos el diccionario y luego mostramos sus claves.

Solución 12

```
frutas = {"manzana": 500, "banana": 300, "naranja": 400, "mandarina": 350}
largas = []
for fruta in frutas.keys():
    if len(fruta) > 6:
        largas.append(fruta)
print("Frutas con más de 6 letras:", largas)
```

Explicación: Filtramos por longitud de la clave.

Solución 13

```
asignaturas = {"Python": 90, "HTML": 85, "CSS": 80, "JavaScript": 95}
claves_ordenadas = sorted(asignaturas.keys())
for clave in claves_ordenadas:
    print(f"{clave}: {asignaturas[clave]}")
```

Explicación: sorted() crea una lista ordenada de claves.

Solución 14

```
configuracion = {"host": "localhost", "port": "5432"}
nueva = input("Nombre de la clave: ")
if nueva not in configuracion:
    configuracion[nueva] = "predeterminado"
    print(f"Clave '{nueva}' agregada.")
else:
    print(f"La clave '{nueva}' ya existe.")
print(configuracion)
```

Explicación: Verificamos y agregamos si no está.

Solución 15

```
meses = {"enero": 1, "febrero": 2, "marzo": 3, "abril": 4,
         "mayo": 5, "junio": 6}
contador = 0
for mes in meses.keys():
    if mes.startswith('m'):
        contador += 1
print(f"Claves que empiezan con 'm': {contador}")
```

Explicación: Usamos startswith() sobre cada clave.

Solución 16

```
config = {"host": "192.168.0.1", "port": "8080", "user": "admin"}
partes = []
for key in config.keys():
```

```
partes.append(f"{key}={config[key]}")
cadena = "&".join(partes)
print("Cadena de conexión:", cadena)
```

Explicación: Armamos la cadena "host=...port=...user=...".

Solución 17

```
notas = {"Ana": 8, "Luis": 3, "Pedro": 9, "Marta": 2, "Juan": 7}
# Recorremos una copia de las claves para no modificar el diccionario mientras iteramos
for alumno in list(notas.keys()):
    if notas[alumno] < 4:
        del notas[alumno]
print("Notas después de eliminar desaprobados:", notas)
```

Explicación: Copiamos las claves a una lista para eliminar de forma segura.

Solución 18

```
import random
sinonimos = {"alegre": "contento", "triste": "apenado", "rápido": "veloz"}
claves = list(sinonimos.keys())
random.shuffle(claves)
for clave in claves:
    print(f"{clave}: {sinonimos[clave]}")
```

Explicación: Mezclamos la lista de claves y mostramos.

Solución 19

```
def listar_claves(diccionario):
    # Ordenamos las claves por el valor (de mayor a menor)
    return sorted(diccionario, key=diccionario.get, reverse=True)

notas = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 6}
print(listar_claves(notas)) # ['Pedro', 'Ana', 'Marta', 'Luis']
```

Explicación: sorted() puede recibir una función key; diccionario.get devuelve el valor.

Solución 20

```
agenda = {}
while True:
    print("\n1. Agregar contacto")
    print("2. Ver nombres ordenados")
    print("3. Salir")
    op = input("Opcion: ")
    if op == "1":
        nombre = input("Nombre: ")
```

```
tel = input("Telefono: ")
agenda[nombre] = tel
elif op == "2":
    if agenda:
        nombres = sorted(agenda.keys())
        for n in nombres:
            print(f"{n}: {agenda[n]}")
    else:
        print("Agenda vacia")
elif op == "3":
    break
```

Explicación: Menú clásico usando keys() para mostrar ordenado.

Resumen del capítulo

En este capítulo aprendiste:

- `.keys()` devuelve una **vista de todas las claves** de un diccionario (ignora los valores).
- Devuelve un objeto `dict_keys` que podés convertir a **lista** con `list()`.
- Sirve para **verificar existencia** de claves con `in`.
- Se puede **recorrer solo las claves** con un bucle `for`.
- Al convertir a lista, podés **ordenar**, acceder por índice, etc.
- Es una herramienta básica pero fundamental para trabajar con diccionarios de forma segura y ordenada.

keys() es como mirar el índice de un libro: solo ves los títulos de los capítulos, no el contenido.

Método values() – Obtener los Valores de un Diccionario

¿Qué es el método values()?

Cuando empecé a trabajar con diccionarios, al principio solo me interesaban los valores para hacer cuentas: sumar edades, encontrar el precio más alto, calcular promedios. Pero me encontraba escribiendo `diccionario[clave]` todo el tiempo, incluso cuando la clave no importaba. Un compañero de la facultad (UBA Exactas) me dijo: “Che, Nahuel, si solo necesitás los números, usá `.values()` y te olvidás de las claves”. Y la verdad que me simplificó la vida.

El método `.values()` devuelve una **vista de todos los valores** de un diccionario, ignorando por completo las claves. Es como si miraras solo la columna de precios de una lista de productos, sin ver los nombres.

- **Qué devuelve:** Un objeto de tipo `dict_values` (una vista de los valores).
- **Sintaxis:** `diccionario.values()`
- **Utilidad:** Sirve para hacer cálculos sobre los datos, como sumar, promediar, encontrar máximos y mínimos, o simplemente recorrer valores en limpio.

Analogía: Pensá en un ticket del supermercado. `.values()` te da solo los precios, sin los nombres de los productos. Si querés saber cuánto gastaste, eso es lo que necesitás.

```
+-----+
| diccionario = {"a": 1, "b": 2, "c": 3} |
| diccionario.values() |
| Resultado: dict_values([1, 2, 3]) |
| ¡Solo los valores, las claves se ignoran! |
+-----+
```

¿Para qué sirve?

`.values()` es la herramienta ideal cuando:

- Querés hacer **operaciones matemáticas** con los datos (suma, promedio, máximo, mínimo).
- Necesitás **recorrer solo los valores** sin que las claves te molesten.
- Te interesa **analizar tendencias** o comparar números.
- Querés generar listas de valores para pasárselas a otras funciones.

Ejemplo básico

```
diccionario = {"a": 1, "b": 2, "c": 3}
print("Los valores son:", diccionario.values())
```

```
Los valores son: dict_values([1, 2, 3])
```

Como con `keys()` e `items()`, el resultado no es una lista común, sino un objeto vista. Para operar con él, casi siempre lo convertimos a lista.

Conversión a lista

Convertir a lista es un paso sencillo y te da acceso a todas las herramientas de las listas: índices, ordenamiento, sumas, etc.

```
# Convertir a lista real
lista_de_valores = list(diccionario.values())
print("Lista de valores:", lista_de_valores)
```

```
# Acceso por índice
print("Primer valor:", lista_de_valores[0])
print("Segundo valor:", lista_de_valores[1])
```

```
Lista de valores: [1, 2, 3]
Primer valor: 1
Segundo valor: 2
```

Nota: *Conversión a lista:* Una vez que tenés la lista de valores, podés:

- `sum(lista)` para obtener la suma total.
- `max(lista)` y `min(lista)` para extremos.
- `sorted(lista)` para ordenarlos.
- Acceder por posición como en cualquier lista.

Ojo, los cambios que hagas en esa lista no afectan al diccionario original.

Operaciones matemáticas con values()

Acá es donde `.values()` se vuelve imprescindible. Imaginá que tenés las notas de un curso y querés sacar el promedio, la nota más alta y la más baja.

```
notas = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 6, "Juan": 10}
lista_notas = list(notas.values())
```

```
print("Notas:", lista_notas)
print("Máxima:", max(lista_notas))
print("Mínima:", min(lista_notas))
print("Suma:", sum(lista_notas))
print("Promedio:", sum(lista_notas) / len(lista_notas))
```

```
Notas: [8, 5, 9, 6, 10]
Máxima: 10
Mínima: 5
Suma: 38
Promedio: 7.6
```

Reflexión: ¿Ves? En ningún momento usamos los nombres de los alumnos. Solo nos importaban los números. Eso es justamente lo que `.values()` te permite hacer.

Recorrer solo los valores

Si necesitás hacer algo con cada valor, un `for` sobre `values()` es más limpio que andar sacando clave por clave.

```
precios = {"manzana": 500, "pera": 600, "uva": 700}
for precio in precios.values():
    print(f"Precio: ${precio}")
```

```
Precio: $500
Precio: $600
Precio: $700
```

Ejemplos prácticos

Análisis de ventas mensuales

Tenés las ventas de varios meses y querés saber el total vendido y el promedio.

```
ventas = {"Enero": 15000, "Febrero": 12000, "Marzo": 18000,
          "Abril": 14000, "Mayo": 20000}
```

```
valores = list(ventas.values())
print("Venta total: $", sum(valores))
print("Promedio: $", sum(valores) / len(valores))
print("Mejor mes: $", max(valores))
print("Peor mes: $", min(valores))
```

Venta total: \$ 79000
 Promedio: \$ 15800.0
 Mejor mes: \$ 20000
 Peor mes: \$ 12000

Conteo de aprobados

Con `values()` podés filtrar y contar sin preocuparte por los nombres.

```
calificaciones = {"Mate": 8, "Fisica": 5, "Quimica": 9, "Bio": 7, "Lengua": 6}
aprobadas = sum(1 for nota in calificaciones.values() if nota >= 7)
print(f"Materias aprobadas: {aprobadas} de {len(calificaciones.values())}")
```

Materias aprobadas: 3 de 5

Ejercicio rápido: ¿Cómo harías para contar cuántos precios superan los \$600 en el diccionario de productos de la sección anterior? Intentá hacerlo con `.values()` y un bucle.

Cuadro Comparativo de Métodos

Para que no te queden dudas, acá va un resumen de cuándo usar cada uno:

MÉTODO	¿QUÉ RETORNA?	EJEMPLO DE SALIDA (convertido a lista)
<code>.items()</code>	Clave y Valor	<code>[('a', 1), ('b', 2)]</code>
<code>.keys()</code>	Solo las Claves	<code>['a', 'b']</code>
<code>.values()</code>	Solo los Valores	<code>[1, 2]</code>

Nota: ¿Cuándo usar cada uno?

- `.items()`: cuando necesitás la clave Y el valor (ej: mostrar “El producto X cuesta Y”).
- `.keys()`: cuando solo necesitás las claves (ej: mostrar solo los nombres de productos).
- `.values()`: cuando solo necesitás los valores (ej: calcular el promedio de notas, sumar ventas).

Ejercicios propuestos

Ahora te toca practicar. Son 20 ejercicios; arrancan sencillos y van subiendo de dificultad. No mires las soluciones antes de intentarlos.

1. Creá un diccionario con 3 personas y sus edades. Usá `.values()` para calcular el promedio de edad.

2. Creá un diccionario con 5 productos y sus precios. Usá `.values()` para encontrar el producto más caro (solo el precio máximo).
3. Creá un diccionario con 4 materias y sus notas. Usá `.values()` para contar cuántas materias tienen nota mayor o igual a 7.
4. Creá un diccionario con 6 alumnos y sus alturas. Usá `.values()` para calcular la altura total y el promedio.
5. Creá un diccionario con 3 ciudades y sus poblaciones. Usá `.values()` para encontrar la ciudad con mayor población (mostrá solo la población máxima).
6. Creá un diccionario con 5 empleados y sus sueldos. Usá `.values()` para calcular el sueldo total de todos.
7. Creá un diccionario con 4 años y las ventas de esos años. Usá `.values()` para calcular el aumento entre el primer y último año.
8. Creá un diccionario con 6 materias y sus notas. Usá `.values()` para contar cuántas materias están desaprobadas (nota <7).
9. Creá un diccionario con 5 personas y sus pesos. Usá `.values()` para encontrar el peso mínimo y máximo.
10. Creá un diccionario con 4 productos y sus precios. Usá `.values()` para calcular el precio promedio.
11. Pedí al usuario 5 países y sus superficies (en km²). Guardalos en un diccionario y luego mostrá la superficie total sumando los valores.
12. Dado un diccionario de frutas y cantidades, calculá el total de frutas. Ejemplo: "manzana": 3, "pera": 2, "naranja": 5.
13. Usá `.values()` y una lista por comprensión para crear una nueva lista con los valores incrementados en un 10 %.
14. A partir de un diccionario de temperaturas diarias, encontrá la temperatura más alta y la más baja sin usar `max/min`.
15. Ordená los valores de un diccionario de mayor a menor y mostralos como lista.
16. Verificá si hay algún valor repetido en un diccionario (sin usar colecciones especiales).
17. Creá una función `analizar_notas(dic)` que devuelva un diccionario con el promedio, la nota máxima y la mínima, usando `.values()`.
18. Dado un diccionario de productos y precios, aplicá un descuento del 20 % a todos los valores y guardá el resultado en un nuevo diccionario. Mostrá el total antes y después.
19. Simulá una encuesta donde varias personas dan su puntuación (1 a 10). Calculá cuántas personas dieron una puntuación mayor o igual a 8 usando solo `.values()`.
20. Construí un pequeño sistema de estadísticas para un equipo de fútbol: ingresá los goles de 5 partidos (fecha: goles). Mostrá total de goles, promedio, partido con más goles y partido con menos goles, utilizando exclusivamente `.values()` y la información de las claves cuando sea necesario.

Soluciones paso a paso

Solución 1

```
personas = {"Ana": 22, "Luis": 25, "Pedro": 21}
edades = list(personas.values())
promedio = sum(edades) / len(edades)
print(f"Promedio de edad: {promedio:.2f}")
```

Explicación: Convertimos los valores a lista, sumamos y dividimos por la cantidad.

Solución 2

```
productos = {"Mouse": 1500, "Teclado": 2500, "Monitor": 50000, "Pendrive": 800, "Cable": 1000}
precios = list(productos.values())
print(f"Precio máximo: ${max(precios)}")
```

Explicación: La función `max()` sobre la lista de valores devuelve el mayor.

Solución 3

```
materias = {"Matemática": 8, "Física": 5, "Química": 9, "Biología": 7}
aprobadas = 0
for nota in materias.values():
    if nota >= 7:
        aprobadas += 1
print(f"Materias aprobadas: {aprobadas}")
```

Explicación: Iteramos directamente sobre los valores y contamos los que cumplen la condición.

Solución 4

```
alumnos = {"Ana": 1.65, "Luis": 1.75, "Pedro": 1.70, "Marta": 1.60, "Juan": 1.80, "Lucía": 1.72}
alturas = list(alumnos.values())
print(f"Altura total: {sum(alturas):.2f} m")
print(f"Altura promedio: {sum(alturas) / len(alturas):.2f} m")
```

Explicación: Usamos `sum()` y dividimos para el promedio.

Solución 5

```
ciudades = {"Buenos Aires": 15000000, "Córdoba": 1500000, "Rosario": 1200000}
poblaciones = list(ciudades.values())
print(f"Población máxima: {max(poblaciones)}")
```

Explicación: Solo mostramos el valor máximo, no la ciudad (para eso necesitaríamos las claves).

Solución 6

```
empleados = {"Ana": 50000, "Luis": 60000, "Pedro": 55000, "Marta": 65000, "Juan": 58000}
sueldos = list(empleados.values())
print(f"Total de sueldos: ${sum(sueldos)}")
```

Explicación: Sumamos todos los valores.

Solución 7

```
ventas = {2020: 10000, 2021: 12000, 2022: 15000, 2023: 18000}
lista_ventas = list(ventas.values())
aumento = lista_ventas[-1] - lista_ventas[0]
print(f"Aumento total: ${aumento}")
```

Explicación: Accedemos al primer y último elemento de la lista de valores.

Solución 8

```
materias = {"Mate": 8, "Fisica": 5, "Quimica": 4, "Bio": 7, "Lengua": 6, "Historia": 3}
desaprobadas = 0
for nota in materias.values():
    if nota < 7:
        desaprobadas += 1
print(f"Materias desaprobadas: {desaprobadas}")
```

Explicación: Similar al de aprobados, pero con condición opuesta.

Solución 9

```
personas = {"Ana": 55, "Luis": 70, "Pedro": 65, "Marta": 52, "Juan": 80}
pesos = list(personas.values())
print(f"Peso mínimo: {min(pesos)} kg")
print(f"Peso máximo: {max(pesos)} kg")
```

Explicación: min() y max() sobre la lista de pesos.

Solución 10

```
productos = {"Mouse": 1500, "Teclado": 2500, "Monitor": 50000, "Pendrive": 800}
precios = list(productos.values())
promedio = sum(precios) / len(precios)
print(f"Precio promedio: ${promedio:.2f}")
```

Explicación: Suma dividida por cantidad.

Solución 11

```

países = {}
for i in range(5):
    país = input(f"País {i+1}: ")
    superficie = float(input("Superficie (km²): "))
    países[país] = superficie
superficie_total = sum(países.values())
print(f"Superficie total: {superficie_total} km²")

```

Explicación: Construimos el diccionario y sumamos sus valores directamente.

Solución 12

```

frutas = {"manzana": 3, "pera": 2, "naranja": 5}
total_frutas = sum(frutas.values())
print(f"Total de frutas: {total_frutas}")

```

Explicación: sum() sobre la vista de valores.

Solución 13

```

precios = {"pan": 100, "leche": 80, "café": 120}
aumentados = [precio * 1.10 for precio in precios.values()]
print("Precios con 10% aumento:", aumentados)

```

Explicación: Comprensión de listas aplicando el aumento a cada valor.

Solución 14

```

temps = {"lunes": 23, "martes": 26, "miercoles": 21, "jueves": 27, "viernes": 25}
lista = list(temps.values())
mayor = lista[0]
menor = lista[0]
for temp in lista:
    if temp > mayor: mayor = temp
    if temp < menor: menor = temp
print(f"Máxima: {mayor} °C, Mínima: {menor} °C")

```

- **Explicación:** Recorremos manualmente para encontrar máximo y mínimo sin funciones pre-definidas.

Solución 15

```

edades = {"Ana": 22, "Luis": 25, "Pedro": 21, "Marta": 24}
valores_ordenados = sorted(edades.values(), reverse=True)
print("Edades ordenadas desc:", valores_ordenados)

```

Explicación: sorted() con reverse=True sobre la vista de valores.

Solución 16

```
datos = {"a": 10, "b": 20, "c": 10, "d": 30}
vistos = []
repetido = False
for valor in datos.values():
    if valor in vistos:
        repetido = True
        break
    vistos.append(valor)
print("¿Hay repetidos?", repetido)
```

Explicación: Usamos una lista auxiliar para detectar repeticiones.

Solución 17

```
def analizar_notas(dic):
    vals = list(dic.values())
    return {
        "promedio": sum(vals) / len(vals),
        "maxima": max(vals),
        "minima": min(vals)
    }
```

```
notas = {"A": 7, "B": 9, "C": 6}
print(analizar_notas(notas))
```

Explicación: Función que empaqueta las estadísticas en un nuevo diccionario.

Solución 18

```
productos = {"Mouse": 1000, "Teclado": 2000, "Monitor": 5000}
precios_original = list(productos.values())
total_antes = sum(precios_original)
descontados = {prod: precio * 0.8 for prod, precio in productos.items()}
total_despues = sum(descontados.values())
print(f"Total antes: ${total_antes}")
print(f"Total después del 20% descuento: ${total_despues}")
```

- **Explicación:** Usamos un diccionario por comprensión para aplicar el descuento y luego sumamos.

Solución 19

```
encuesta = {"Persona1": 9, "Persona2": 7, "Persona3": 8, "Persona4": 10}
altos = sum(1 for v in encuesta.values() if v >= 8)
print(f"Puntuaciones >= 8: {altos}")
```

Explicación: Expresión generadora que cuenta los valores que cumplen.

Solución 20

```
goles = {}
for i in range(5):
    fecha = input(f"Fecha del partido {i+1}: ")
    goles_fecha = int(input("Goles a favor: "))
    goles[fecha] = goles_fecha

valores = list(goles.values())
total = sum(valores)
promedio = total / len(valores)
max_goles = max(valores)
min_goles = min(valores)

# Para saber la fecha, necesitamos claves: usamos items() para el detalle
print(f"Total goles: {total}")
print(f"Promedio: {promedio:.2f}")
# Buscamos la(s) fecha(s) con max y min (por simplicidad una sola)
for fecha, cantidad in goles.items():
    if cantidad == max_goles:
        print(f"Partido con más goles: {fecha} ({max_goles})")
        break
for fecha, cantidad in goles.items():
    if cantidad == min_goles:
        print(f"Partido con menos goles: {fecha} ({min_goles})")
        break
```

Explicación: Usamos `.values()` para los cálculos y luego `.items()` para asociar fechas con los extremos. Es un caso real donde combinamos ambos.

Resumen del capítulo

En este capítulo aprendiste:

- `.values()` devuelve una **vista de todos los valores** de un diccionario, ignorando las claves.
- Podés convertirlo a **lista** con `list()` para manipularlo con índices o funciones como `sum()`, `max()`, `min()`.
- Es muy útil para **operaciones numéricas** sobre los datos (promedios, totales, extremos).
- También sirve para **recorrer valores** en un bucle y tomar decisiones basadas en ellos.
- Combinado con otras herramientas, te permite analizar rápidamente colecciones de datos sin enredarte con las claves.

values() es como mirar solo los números de una factura: te interesa cuánto pagaste, no qué cosa compraste.

Método `clear()` – Vaciar un Diccionario

¿Qué es el método `clear()`?

Cuando empecé a trabajar con grandes volúmenes de datos en la facultad (UBA Exactas), me pasó más de una vez que quería reutilizar un diccionario para una nueva tanda de experimentos sin estar creando variables nuevas a cada rato. Al principio, lo que hacía era asignarle un diccionario vacío (`{}`) a la variable, pero eso generaba un objeto nuevo y perdía la referencia original. Después descubrí `.clear()` y fue un antes y un después.

El método `clear()` se usa para **eliminar TODOS los elementos** de un diccionario (o lista, o conjunto) de forma simultánea, dejando el objeto **vacío** pero sin borrar el objeto en sí de la memoria.

- **Qué hace:** Vacía el contenido del diccionario.
- **Resultado:** El diccionario queda vacío: `{}`.
- **Sintaxis:** `nombre_del_diccionario.clear()`
- **Objetos compatibles:** Funciona con diccionarios, listas y conjuntos (sets).

Analogía: Imaginá que tenés una caja llena de juguetes. `clear()` es como volcar la caja y dejarla completamente vacía, pero la caja sigue ahí para que la vuelvas a llenar con lo que quieras.

```
+-----+
|
|  diccionario = {"a": 1, "b": 2, "c": 3}
|
|  diccionario.clear()
|
|  Resultado: {}
|
|  ¡El diccionario queda vacío pero existe!
|
+-----+
```

¿Para qué sirve?

El método `.clear()` es muy práctico cuando necesitás:

- **Reiniciar** una base de datos temporal.

- **Limpiar** una lista de resultados antes de iniciar un nuevo cálculo o proceso.
- **Reutilizar** un diccionario sin tener que crear uno nuevo.
- **Liberar memoria** de los elementos sin perder la variable.

Ejemplo básico

```
# Creación del diccionario
diccionario = {"a": 1, "b": 2, "c": 3}
print("Diccionario original:", diccionario)

# Vaciamos con clear()
diccionario.clear()

print("Diccionario después de clear():", diccionario)

Diccionario original: {'a': 1, 'b': 2, 'c': 3}
Diccionario después de clear(): {}
```

Ejemplo práctico: Diccionario de empleados anidado

El método también limpia diccionarios anidados (diccionarios dentro de otros). No importa la complejidad, `clear()` los deja completamente vacíos.

```
empleados = {
    "Juan": {"edad": 28, "salario": 2500},
    "Maria": {"edad": 24, "salario": 3000},
    "Pedro": {"edad": 32, "salario": 3500}
}

print("=== DICCIONARIO ORIGINAL ===")
for nombre, datos in empleados.items():
    print(f"{nombre}: edad {datos['edad']}, salario ${datos['salario']}")

empleados.clear()

print("\nDespués de clear():", empleados)
print("¿Está vacío?", len(empleados) == 0)

=== DICCIONARIO ORIGINAL ===
Juan: edad 28, salario $2500
Maria: edad 24, salario $3000
Pedro: edad 32, salario $3500

Después de clear(): {}
¿Está vacío? True
```

Diferencia clave: clear() vs del

Es muy común confundir `clear()` con la instrucción `del`. En criollo: `clear()` vacía el cajón, `del` rompe el cajón.

MÉTODO	QUÉ HACE
<code>clear()</code>	Vacía el contenido pero MANTIENE la variable El diccionario sigue existiendo, vacío <code>{}</code>
<code>del nombre</code>	Borra la variable COMPLETAMENTE de la memoria Si intentás usarla después, da error

```
# clear() permite reutilizar la variable
datos = {"nombre": "Nahuel", "edad": 25}
datos.clear()
datos["nueva"] = "cosa"    # Funciona sin problemas
print(datos)

# del elimina la variable
datos2 = {"a": 1, "b": 2}
del datos2
# print(datos2) daría NameError: la variable ya no existe
```

Nota: Regla de oro:

- Usá `.clear()` si querés **reutilizar** el diccionario.
- Usá `del` si querés **eliminar** el diccionario para siempre.

Ejemplo práctico: Reiniciar un carrito de compras

Uno de los usos más comunes: el cliente termina su compra y el carrito debe quedar limpio para la siguiente.

```
carrito = {"manzanas": 3, "peras": 2, "naranjas": 5}
print("Carrito actual:", carrito)

# Finaliza la compra
print("\nFinalizando compra...")
carrito.clear()
print("Carrito vaciado:", carrito)

# Nueva compra
carrito["uvas"] = 4
carrito["sandias"] = 1
print("Nuevo carrito:", carrito)
```

```
Carrito actual: {'manzanas': 3, 'peras': 2, 'naranjas': 5}
```

```
Finalizando compra...
```

```
Carrito vaciado: {}
```

```
Nuevo carrito: {'uvas': 4, 'sandias': 1}
```

Ejemplo práctico: Limpiar resultados de experimentos

En Exactas hacíamos simulaciones y necesitábamos limpiar los resultados entre cada corrida. `clear()` nos ahorra crear un nuevo diccionario cada vez.

```
resultados = {}

# Experimento 1
resultados["prueba1"] = 25.5
resultados["prueba2"] = 26.1
resultados["prueba3"] = 24.8
print("Experimento 1:", resultados)

# Limpiamos para el siguiente
resultados.clear()
print("Después de limpiar:", resultados)

# Experimento 2
resultados["pruebaA"] = 32.4
resultados["pruebaB"] = 33.1
print("Experimento 2:", resultados)
```

Ejercicios propuestos

Practicá con estos 20 ejercicios. No mires las soluciones hasta haberlos intentado.

1. Creá un diccionario con 3 productos y sus precios. Usá `.clear()` para vaciarlo y mostralo.
2. Creá un diccionario anidado (con datos de empleados). Usá `.clear()` y verificá que quedó vacío.
3. Creá una lista de nombres y usá `.clear()` para vaciarla (las listas también tienen este método).
4. Creá un diccionario con 5 alumnos y sus notas. Usá `.clear()` y luego agregá 3 nuevos alumnos.
5. Explicá la diferencia entre `clear()` y `del` con tus propias palabras (escribí un ejemplo con código).
6. Creá un diccionario con 4 materias y sus notas. Usá `.clear()` y después verificá si el diccionario está vacío con `len()`.

7. Creá un diccionario con datos de una persona (nombre, edad, ciudad). Aplicá `clear()`, luego agregá un nuevo dato. ¿Funciona?
8. Creá un diccionario con 3 productos. Usá `del` para eliminar el diccionario. Intentá usarlo después (comentá la línea que daría error).
9. Creá un diccionario con inventario de una tienda. Simulá un reinicio de inventario usando `.clear()`.
10. Creá un diccionario con 5 pares clave-valor. Usá `.clear()` y luego mostrá la cantidad de elementos con `len()`.
11. Pedile al usuario 3 nombres y edades, guardalos en un diccionario, mostralos, luego vacialo y mostralo de nuevo.
12. Simulá una agenda telefónica: creá un diccionario con 4 contactos, vacialo con `clear()` y luego agregá un contacto nuevo. Comprabá que solo quede el nuevo.
13. Demostrá que `clear()` también funciona en conjuntos: creá un set con números, vacialo y mostralo.
14. ¿Qué pasa si llamás a `clear()` sobre un diccionario que ya está vacío? Probá y contá qué observaste.
15. Creá dos diccionarios que referencien al mismo objeto (asignación directa). Si aplicás `clear()` a uno, ¿qué le pasa al otro? Anotá tus conclusiones.
16. Escribí una función `reiniciar_estadisticas(dic)` que reciba un diccionario, lo vacíe con `clear()` y devuelva el diccionario vacío.
17. En un bucle, llená un diccionario con 5 números al cuadrado (clave = número, valor = cuadrado). Imprimilo, limpialo con `clear()` y volvé a llenar con los mismos números pero al cubo.
18. Creá un diccionario con varias llaves. Usá `clear()` y después intentá acceder a una llave. ¿Qué sucede? (Probá con `get` y con corchetes).
19. Armá un pequeño sistema de puntos de un juego: ganás una partida, se suma un punto a un jugador. Al terminar la sesión, reiniciá todos los puntajes con `clear()`. Mostrá el proceso.
20. Combiná `clear()` con un menú interactivo: el usuario puede agregar productos a un diccionario, ver el carrito, y al elegir “finalizar compra”, se vacía el carrito automáticamente.

Soluciones paso a paso

Solución 1

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
print("Antes:", productos)
productos.clear()
print("Después:", productos)
```

Explicación: clear() elimina todos los pares clave-valor.

Solución 2

```
empleados = {"Juan": {"edad": 25}, "Maria": {"edad": 30}}
print("Antes:", empleados)
empleados.clear()
print("Después:", empleados)
# Confirmamos vacío
print("¿Vacío?", len(empleados) == 0)
```

Explicación: Se vacía todo el contenido anidado.

Solución 3

```
nombres = ["Ana", "Luis", "Pedro"]
print("Lista antes:", nombres)
nombres.clear()
print("Lista después:", nombres)
```

Explicación: Las listas también tienen clear(), funciona igual.

Solución 4

```
alumnos = {"Ana": 8, "Luis": 7, "Pedro": 9, "Marta": 6, "Juan": 10}
print("Original:", alumnos)
alumnos.clear()
print("Después de clear:", alumnos)
alumnos["Nahuel"] = 9
alumnos["Lucía"] = 8
alumnos["Carlos"] = 7
print("Nuevos:", alumnos)
```

Explicación: Se puede volver a llenar sin problema.

Solución 5

```
# clear() vacía pero mantiene la variable
datos = {"a": 1}
datos.clear()          # datos ahora es {}
```

```
datos["b"] = 2          # funciona

# del elimina la variable
datos2 = {"x": 10}
del datos2              # datos2 ya no existe
# print(datos2)         # NameError
```

Explicación: clear() conserva la variable, del la borra.

Solución 6

```
materias = {"Mate": 8, "Fisica": 7, "Quimica": 9, "Historia": 6}
print("Cantidad antes:", len(materias))
materias.clear()
print("Cantidad después:", len(materias))
```

Explicación: len() da 0 después de limpiar.

Solución 7

```
persona = {"nombre": "Nahuel", "edad": 25, "ciudad": "Bs As"}
print("Original:", persona)
persona.clear()
print("Después de clear:", persona)
persona["nuevo"] = "valor"
print("Agregando:", persona)
# Funciona perfectamente
```

Explicación: El diccionario sigue siendo válido.

Solución 8

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
del productos
# La variable productos ya no existe
# print(productos) # Esto daría NameError
print("La variable fue eliminada con del")
```

• **Explicación:** Con del no podemos acceder más.

Solución 9

```
inventario = {"manzanas": 50, "peras": 30, "naranjas": 40}
print("Inventario original:", inventario)
inventario.clear()
print("Inventario reiniciado:", inventario)
```

Explicación: Reinicia todo a cero.

Solución 10

```
dic = {1: "a", 2: "b", 3: "c", 4: "d", 5: "e"}
print("Elementos antes:", len(dic))
dic.clear()
print("Elementos después:", len(dic))
```

Explicación: len(dic) pasa a 0.

Solución 11

```
datos = {}
for i in range(3):
    nombre = input("Nombre: ")
    edad = int(input("Edad: "))
    datos[nombre] = edad
print("Diccionario lleno:", datos)
datos.clear()
print("Vacío:", datos)
```

Explicación: El usuario ingresa datos, se muestran y luego se vacían.

Solución 12

```
agenda = {"Ana": "1234", "Luis": "5678", "Marta": "9012", "Pedro": "3456"}
print("Agenda original:", agenda)
agenda.clear()
print("Agenda vacía:", agenda)
agenda["Nahuel"] = "1111"
print("Agenda con nuevo contacto:", agenda)
```

Explicación: Vaciamos y agregamos solo el nuevo.

Solución 13

```
mi_set = {1, 2, 3, 4, 5}
print("Set antes:", mi_set)
mi_set.clear()
print("Set después:", mi_set)
# Los sets también tienen clear()
```

Explicación: clear() funciona en conjuntos también.

Solución 14

```
dic_vacio = {}
print("Contenido antes de clear:", dic_vacio)
dic_vacio.clear()
print("Contenido después de clear:", dic_vacio)
# No pasa nada, simplemente sigue vacío
```

Explicación: Si el diccionario ya está vacío, `clear()` no provoca errores; simplemente no hace nada.

Solución 15

```
original = {"a": 1, "b": 2}
copia = original    # apuntan al mismo objeto
original.clear()
print("original:", original)
print("copia:", copia)
# Ambos se vacían porque son el mismo diccionario
```

Explicación: Al compartir la misma referencia, limpiar uno afecta al otro.

Solución 16

```
def reiniciar_estadisticas(dic):
    dic.clear()
    return dic

estadisticas = {"goles": 5, "faltas": 3}
print("Antes:", estadisticas)
estadisticas = reiniciar_estadisticas(estadisticas)
print("Después:", estadisticas)
```

Explicación: La función limpia y devuelve el mismo diccionario (ya vacío).

Solución 17

```
numeros = {}
for i in range(1, 6):
    numeros[i] = i ** 2
print("Al cuadrado:", numeros)

numeros.clear()
print("Después de clear:", numeros)

for i in range(1, 6):
    numeros[i] = i ** 3
print("Al cubo:", numeros)
```

Explicación: Reutilizamos el mismo diccionario para distintos datos.

Solución 18

```
datos = {"clave1": 10, "clave2": 20}
datos.clear()
print(datos.get("clave1")) # None (no existe)
# print(datos["clave1"])   # KeyError
```

Explicación: Después de `clear()`, las claves ya no están.

Solución 19

```
puntajes = {"jugador1": 0, "jugador2": 0}
print("Inicio:", puntajes)

# Jugador1 anota
puntajes["jugador1"] += 1
print("Tras gol:", puntajes)

# Fin de sesión: reiniciamos
puntajes.clear()
print("Reinicio:", puntajes)
# Nuevo juego
puntajes["jugador1"] = 0
puntajes["jugador2"] = 0
print("Listos para otra partida:", puntajes)
```

Explicación: clear() permite reiniciar la sesión sin borrar la variable.

Solución 20

```
carrito = {}
while True:
    print("\n1. Agregar producto")
    print("2. Ver carrito")
    print("3. Finalizar compra")
    print("4. Salir")
    op = input("Opcion: ")
    if op == "1":
        prod = input("Producto: ")
        cant = int(input("Cantidad: "))
        carrito[prod] = carrito.get(prod, 0) + cant
        print("Agregado")
    elif op == "2":
        if not carrito:
            print("Carrito vacio")
        else:
            for p, c in carrito.items():
                print(f"{p}: {c}")
    elif op == "3":
        if carrito:
            print("Compra finalizada. Gracias!")
            carrito.clear()
        else:
            print("El carrito ya esta vacio")
    elif op == "4":
        break
```

Explicación: Al finalizar la compra, el carrito se limpia con clear() automáticamente.

Resumen del capítulo

En este capítulo aprendiste:

- `.clear()` elimina **TODOS los elementos** de un diccionario, dejándolo vacío (`{}`).
- El diccionario **sigue existiendo** en memoria, a diferencia de `del` que lo elimina por completo.
- Funciona en diccionarios, listas y conjuntos.
- Es útil para **reutilizar** variables y reiniciar datos (carritos, inventarios, resultados experimentales).
- Si dos variables apuntan al mismo diccionario, `clear()` afecta a ambas.
- No causa error si el diccionario ya está vacío.

clear() es como barrer una habitación: la habitación sigue ahí, solo que ahora está vacía y lista para volver a llenarse.

Agregar y Modificar Elementos en un Diccionario

La belleza de Python: misma sintaxis para dos tareas

Cuando empecé a trabajar con diccionarios en la UBA Exactas, lo primero que me enseñaron fue que para cambiar un valor había que hacer una cosa, y para agregar un dato nuevo, otra distinta. En mi cabeza era lógico. Pero cuando vi que en Python con una misma línea podías hacer ambas cosas según existiera o no la clave, pensé: “Esto es tramposo... ¡pero me encanta!”.

En criollo, **la misma sintaxis** se usa para **modificar** un elemento existente o para **agregar** uno nuevo. La única diferencia es si la clave que ponés entre los corchetes ya está en el diccionario o no.

```
+-----+
|
|  nombre_diccionario[clave] = nuevo_valor  |
|
|  Si la CLAVE EXISTE -> MODIFICA el valor  |
|  Si la CLAVE NO EXISTE -> AGREGA nuevo par |
|
|  ¡Misma línea de código, dos comportamientos! |
|
+-----+
```

Analogía: Pensá en una caja con compartimentos etiquetados. Si abrís un compartimento que ya tiene algo y le ponés otra cosa, cambia el contenido. Si el compartimento está vacío (la etiqueta no existía), lo creás y ponés la cosa. Python hace eso automático.

Modificar un elemento existente

Si la clave que usás **ya existe** en el diccionario, Python simplemente **reemplaza** el valor anterior por el nuevo. El viejo se pierde, así que asegurate de que sea lo que querés.

Ejemplo básico

```
frutas = {
    "manzana": 2.50,
    "pera": 3.00,
    "naranja": 1.50,
    "platano": 2.00
```



```
}

print("=== DICCIONARIO ORIGINAL ===")
print(frutas)

# Modificar el valor de "manzana" (la clave YA existe)
frutas["manzana"] = 3.50

print("\n=== DESPUES DE MODIFICAR 'manzana' ===")
print(frutas)

=== DICCIONARIO ORIGINAL ===
{'manzana': 2.5, 'pera': 3.0, 'naranja': 1.5, 'platano': 2.0}

=== DESPUES DE MODIFICAR 'manzana' ===
{'manzana': 3.5, 'pera': 3.0, 'naranja': 1.5, 'platano': 2.0}
```

Reflexión: Fijate que la clave "manzanaza" estaba, así que solo se actualizó su precio.

Agregar un nuevo elemento

Si la clave **no existe** en el diccionario, Python entiende que querés crear un nuevo par clave-valor y lo añade al final (a partir de Python 3.7 se mantiene el orden de inserción).

Ejemplo básico

```
frutas = {
    "manzana": 2.50,
    "pera": 3.00,
    "naranja": 1.50,
    "platano": 2.00
}

print("=== DICCIONARIO ORIGINAL ===")
print(frutas)

# Agregar "uva" (la clave NO existe)
frutas["uva"] = 4.00

print("\n=== DESPUES DE AGREGAR 'uva' ===")
print(frutas)

=== DICCIONARIO ORIGINAL ===
{'manzana': 2.5, 'pera': 3.0, 'naranja': 1.5, 'platano': 2.0}

=== DESPUES DE AGREGAR 'uva' ===
{'manzana': 2.5, 'pera': 3.0, 'naranja': 1.5, 'platano': 2.0, 'uva': 4.0}
```

Nota: *¿Y si no sé si la clave existe? Podés usar `in` para verificarlo antes, pero no es obligatorio. Si querés agregar sin miedo a sobrescribir, usá `.get()` o `if clave not in dic:`. Pero para aprender, lo directo es lo mejor.*

Ejemplo completo: Diccionario de frutas

Para que veas el panorama completo, acá tenés un ejemplo que modifica, agrega y muestra:

```
frutas = {
    "manzana": 2.50,
    "pera": 3.00,
    "naranja": 1.50,
    "platano": 2.00
}

print("=== PRECIOS INICIALES ===")
for fruta, precio in frutas.items():
    print(f"{fruta}: ${precio}")

# Modificar "manzana"
frutas["manzana"] = 3.50
print(f"\nAhora la manzana cuesta ${frutas['manzana']}")

# Agregar "uva"
frutas["uva"] = 4.00
print(f"Se agrego la uva con precio ${frutas['uva']}")

# Agregar otra más
frutas["kiwi"] = 5.50

print("\n=== DICCIONARIO FINAL ===")
for fruta, precio in frutas.items():
    print(f"{fruta}: ${precio}")

=== PRECIOS INICIALES ===
manzana: $2.5
pera: $3.0
naranja: $1.5
platano: $2.0

Ahora la manzana cuesta $3.5
Se agrego la uva con precio $4.0

=== DICCIONARIO FINAL ===
manzana: $3.5
pera: $3.0
naranja: $1.5
platano: $2.0
uva: $4.0
```

```
kiwi: $5.5
```

¿Qué pasa si la clave ya existe?

Si la clave ya existe, el valor anterior se **sobrescribe** y se pierde para siempre. A veces querés eso (como actualizar un precio), otras veces querés evitar perder datos. Por eso es importante saber qué claves tenés.

```
datos = {"nombre": "Nahuel", "edad": 25}
print(f"Original: {datos}")

# La clave "nombre" ya existe -> se modifica
datos["nombre"] = "Carlos"
print(f"Despues de modificar: {datos}")
```

¿Qué pasa si la clave no existe?

Si la clave no existe, simplemente se crea un nuevo par al final. No da error. Es lo más normal del mundo.

```
productos = {"pan": 70, "leche": 90}
print(f"Original: {productos}")

productos["huevos"] = 120
print(f"Despues de agregar: {productos}")
```

Ejemplos prácticos

Registro de temperaturas en el laboratorio

Cuando hacíamos experimentos en la facu, llevábamos un diccionario para anotar temperaturas y luego corregirlas si repetíamos la medición.

```
temperaturas = {}
temperaturas["Exp1"] = 25.5
temperaturas["Exp2"] = 26.1
temperaturas["Exp3"] = 24.8

print("Antes de corregir:", temperaturas)

# Corrección de Exp2
temperaturas["Exp2"] = 26.3
# Nuevo experimento
temperaturas["Exp4"] = 27.2

print("Final:", temperaturas)
```

Sistema de notas

```
notas = {}
notas["Ana"] = 8
notas["Luis"] = 7
notas["Pedro"] = 9

# Recuperatorio de Luis
notas["Luis"] = 8.5

# Alumna nueva
notas["Marta"] = 10

for alumno, nota in notas.items():
    print(f"{alumno}: {nota}")
```

Ejercicio rápido: Creá un diccionario vacío con `precios = {}` y agregá dos productos. Después modificá uno. Mostrá antes y después.

Ejercicios propuestos

Ahora practicá. Estos 20 ejercicios te van a dar fluidez con la agregación y modificación de diccionarios.

1. Creá un diccionario con 3 productos y sus precios. Modificá el precio de uno de ellos y mostralo.
2. Creá un diccionario con 3 personas y sus edades. Agregá una nueva persona.
3. Creá un diccionario vacío. Agregá 4 pares clave-valor usando la sintaxis de corchetes.
4. Creá un diccionario con 5 materias y sus notas. Modificá la nota de una materia.
5. Creá un diccionario con 3 ciudades y su población. Agregá una nueva ciudad.
6. Creá un diccionario con 4 empleados y sus sueldos. Modificá el sueldo de uno de ellos.
7. Creá un diccionario con 2 productos. Intentá modificar una clave que NO existe. ¿Qué pasa? Explicá el resultado.
8. Creá un diccionario con 3 países y sus capitales. Agregá 2 países nuevos.
9. Creá un diccionario con 4 libros y sus autores. Modificá el autor de un libro.
10. Creá un diccionario vacío. Usá un bucle `for` para pedir al usuario 5 nombres y edades, y agregalos al diccionario.
11. Dado un diccionario de stock `{"manzana": 10, "pera": 5}`, simulá una venta restando 2 manzanas y una pera, modificando los valores correspondientes.
12. Pedí al usuario el nombre de una fruta y un precio. Si la fruta ya existe en un diccionario fijo, modificá el precio; si no, agregala. Mostrá el diccionario final.

13. Creá un diccionario con los días de la semana y sus temperaturas. Después, actualizá la temperatura del miércoles y agregá un día extra ("feriado": 18) y mostrá todo.
14. Escribí una función `agregar_o_modificar(dic, clave, valor)` que haga justamente eso y devuelva el diccionario.
15. Creá un diccionario de configuración {"host": "localhost", "port": 80}. Modificá el puerto a 443 y agregá una clave "ssl" con valor True.
16. A partir de una lista de tuplas `[("a", 1), ("b", 2)]`, construí un diccionario agregando una por una con corchetes en un bucle.
17. Simulá un carrito de compras donde el usuario ingresa el producto y la cantidad. Si el producto ya está, sumá la cantidad; si no, agregalo.
18. ¿Qué pasa si intentás agregar una clave que es un número? Hacé la prueba: `dic = {};`
`dic[1] = "uno"; dic[2] = "dos"` y mostrá el resultado.
19. Creá un diccionario con tus datos personales. Luego modificá uno de ellos y mostrá el cambio.
20. Construí un programa que pida indefinidamente al usuario un par clave-valor (nombre y teléfono) y los guarde en un diccionario. Si el usuario ingresa un nombre que ya existe, debe preguntar si quiere sobrescribir el teléfono o no.

Soluciones paso a paso

Solución 1

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
print("Original:", productos)
productos["leche"] = 95
print("Modificado:", productos)
```

Explicación: Accedemos a la clave existente "leche" le asignamos un nuevo valor.

Solución 2

```
personas = {"Ana": 22, "Luis": 25, "Pedro": 21}
print("Original:", personas)
personas["Marta"] = 24
print("Con Marta:", personas)
```

Explicación: La clave "Marta" no existe, así que Python la agrega.

Solución 3

```
datos = {}
datos["nombre"] = "Nahuel"
datos["edad"] = 25
datos["ciudad"] = "Buenos Aires"
datos["profesion"] = "programador"
print(datos)
```

Explicación: Cada asignación agrega un nuevo par al diccionario vacío.

Solución 4

```
materias = {"Mate": 8, "Fisica": 7, "Quimica": 9, "Historia": 6, "Lengua": 8}
print("Original:", materias)
materias["Historia"] = 7
print("Modificado:", materias)
```

• **Explicación:** Cambiamos la nota de Historia a 7.

Solución 5

```
ciudades = {"Buenos Aires": 15000000, "Cordoba": 1500000, "Rosario": 1200000}
print("Original:", ciudades)
ciudades["Mendoza"] = 1000000
print("Con Mendoza:", ciudades)
```

Explicación: Se agrega una nueva ciudad con su población.

Solución 6

```
empleados = {"Ana": 50000, "Luis": 60000, "Pedro": 55000, "Marta": 65000}
print("Original:", empleados)
empleados["Pedro"] = 58000
print("Actualizado:", empleados)
```

Explicación: Actualizamos el sueldo de Pedro.

Solución 7

```
productos = {"pan": 70, "leche": 90}
productos["huevos"] = 120 # "huevos" no existe, se AGREGARA
print(productos) # {'pan': 70, 'leche': 90, 'huevos': 120}
# Si queríamos modificar una clave inexistente, en realidad la estamos agregando.
```

Explicación: Como la clave "huevos" no estaba, se crea una nueva entrada, no se modifica nada existente.

Solución 8

```
países = {"Argentina": "Bs As", "Brasil": "Brasilia", "Chile": "Santiago"}
print("Original:", países)
países["Uruguay"] = "Montevideo"
países["Paraguay"] = "Asuncion"
print("Actualizado:", países)
```

Explicación: Se agregan dos nuevos países.

Solución 9

```
libros = {"1984": "Orwell", "Fahrenheit": "Bradbury", "Rayuela": "Cortazar", "Principito"}
print("Original:", libros)
libros["1984"] = "George Orwell"
print("Corregido:", libros)
```

Explicación: La clave "1984" ya existía, así que su valor se actualiza.

Solución 10

```
agenda = {}
for i in range(5):
    nombre = input(f"Nombre {i+1}: ")
    edad = input(f"Edad de {nombre}: ")
    agenda[nombre] = edad
print("\nAgenda:")
for nombre, edad in agenda.items():
    print(f"{nombre}: {edad} años")
```

Explicación: El bucle pide datos y los va agregando al diccionario vacío.

Solución 11

```
stock = {"manzana": 10, "pera": 5}
print("Stock inicial:", stock)
stock["manzana"] = stock["manzana"] - 2    # queda en 8
stock["pera"] = stock["pera"] - 1          # queda en 4
print("Stock final:", stock)
```

Explicación: Modificamos los valores haciendo la resta directamente.

Solución 12

```
frutas = {"manzana": 3.0, "pera": 2.5}
nombre = input("Fruta: ")
precio = float(input("Precio: "))
frutas[nombre] = precio    # Si existe modifica, si no agrega
print("Diccionario:", frutas)
```

Explicación: Confiamos en el comportamiento dual de Python.

Solución 13

```
temps = {"lunes": 22, "martes": 23, "miercoles": 20, "jueves": 19, "viernes": 21}
print("Original:", temps)
temps["miercoles"] = 24
temps["feriado"] = 18
print("Actualizado:", temps)
```

Explicación: Modificamos miércoles y agregamos feriado.

Solución 14

```
def agregar_o_modificar(dic, clave, valor):
    dic[clave] = valor
    return dic

config = {"debug": False}
config = agregar_o_modificar(config, "debug", True)
config = agregar_o_modificar(config, "version", "2.0")
print(config)
```

Explicación: La función encapsula la operación y devuelve el diccionario.

Solución 15

```
config = {"host": "localhost", "port": 80}
print("Original:", config)
config["port"] = 443
config["ssl"] = True
print("Actualizada:", config)
```

Explicación: Modificamos el puerto y agregamos la clave "ssl".

Solución 16

```
tuplas = [("a", 1), ("b", 2), ("c", 3)]
dic = {}
for k, v in tuplas:
    dic[k] = v
print(dic)
```

Explicación: Recorremos la lista de tuplas y agregamos cada par al diccionario.

Solución 17

```
carrito = {}
while True:
    prod = input("Producto (enter para salir): ")
    if prod == "":
        break
    cant = int(input("Cantidad: "))
    if prod in carrito:
        carrito[prod] += cant    # suma si ya existia
    else:
        carrito[prod] = cant    # agrega si no existia
print("Carrito:", carrito)
```

Explicación: Usamos in para verificar si la clave ya existe y actuar en consecuencia.

Solución 18

```
dic = {}
dic[1] = "uno"
dic[2] = "dos"
print(dic)  # {1: 'uno', 2: 'dos'}
```

Explicación: Las claves pueden ser números enteros sin problema.

Solución 19

```
yo = {"nombre": "Nahuel", "edad": 25, "pais": "Argentina"}
print("Original:", yo)
yo["edad"] = 26
print("Cambio de edad:", yo)
```

Explicación: Modificamos la edad.

Solución 20

```
contactos = {}
while True:
    nombre = input("Nombre (o 'salir')"): ")
    if nombre.lower() == "salir":
```

```
        break
    tel = input("Telefono: ")
    if nombre in contactos:
        resp = input("El contacto ya existe. Sobrescribir? (s/n): ")
        if resp.lower() != "s":
            print("No se modifiko.")
            continue
        contactos[nombre] = tel
    print("Guardado.")
print("Contactos:", contactos)
```

Explicación: Preguntamos antes de sobrescribir para evitar perder datos.

Resumen del capítulo

Dale, repasemos lo importante:

- **Misma sintaxis** para agregar y modificar: `dic[clave] = valor`.
- Si la **clave existe**: el valor se **reemplaza**.
- Si la **clave no existe**: se **crea un nuevo par** al final del diccionario.
- No hay métodos especiales obligatorios: es simple e intuitivo.
- Esto permite construir y actualizar diccionarios de manera muy dinámica, ideal para formularios, inventarios, registros, etc.

Agregar y modificar en diccionarios es como tener un cuaderno de hojas móviles: podés escribir en una hoja que ya existe o agregar una hoja nueva donde antes no había nada.

Método fromkeys() – Crear Diccionarios de Forma Masiva

¿Qué es el método fromkeys()?

Cuando arranqué con Python en la UBA Exactas, recuerdo que tenía que anotar las notas de un montón de materias todas de una vez. Al principio hacía un diccionario vacío y con un `for` iba agregando cada materia con 0. Pero un profe me dijo: “Che, Nahuel, ¿no conocés `fromkeys()`? Te crea el diccionario entero de una, con todas las claves y el mismo valor inicial. Es un golazo.” Y tenía razón. Desde ese día, cada vez que necesito inicializar una estructura con valores por defecto, uso `fromkeys()`.

El método `fromkeys()` es una función que crea un **nuevo diccionario** a partir de una **secuencia** (una lista, tupla, string, o incluso otro diccionario) y un **valor opcional** que se asignará a cada una de esas claves. En criollo: vos le das los nombres de los cajones y un contenido estándar, y él te arma el mueble entero.

Analogía: Imaginá que tenés un formulario donde todos los campos empiezan en blanco o con un valor predeterminado. `fromkeys()` es como darte ese formulario ya armado con todos los campos listos, y vos después solo completás los que cambian.

```
+-----+
| dict.fromkeys(secuencia, valor) |
|                               |
| secuencia: lista, tupla, string o diccionario |
|           (sus elementos serán las claves)   |
|                               |
| valor: (opcional) valor para todas las claves |
|           si no se pone, es None             |
+-----+
```

Sintaxis fundamental

La forma más común y recomendada es llamar al método directamente desde la clase `dict`:

```
nuevo_diccionario = dict.fromkeys(secuencia, valor)
```

- **Secuencia (obligatorio):** Los elementos que se convertirán en las claves del diccionario. Puede ser una lista, tupla, string, range, etc.
- **Valor (opcional):** El valor que se le dará a cada clave. Si no lo ponés, por defecto es `None`.

Dos formas de invocarlo

Hay dos maneras de llamar a `fromkeys()`, pero la primera es la más clara y la que vas a ver en todos lados:

```
# FORMA 1: Usando la clase dict (RECOMENDADA)
diccionario1 = dict.fromkeys(["a", "b", "c"], 0)
print(diccionario1) # {'a': 0, 'b': 0, 'c': 0}

# FORMA 2: Usando llaves vacías (menos comun)
diccionario2 = {}.fromkeys(["x", "y", "z"], 1)
print(diccionario2) # {'x': 1, 'y': 1, 'z': 1}
```

Salida:

```
{'a': 0, 'b': 0, 'c': 0}
{'x': 1, 'y': 1, 'z': 1}
```

Nota: Recomendación: Siempre usá la primera forma `dict.fromkeys()`. Es más explícita y se entiende inmediatamente que estás creando un diccionario nuevo. La segunda forma con `{}.fromkeys()` funciona, pero es menos legible.

Comportamientos según la secuencia

`fromkeys()` es bastante versátil y se adapta al tipo de secuencia que le entregues.

Con una lista

Cada elemento de la lista se convierte en una clave.

```
claves = ["fisica", "mate", "quimica"]
materias = dict.fromkeys(claves, 0)
print(materias) # {'fisica': 0, 'mate': 0, 'quimica': 0}
```

Con una tupla

Funciona exactamente igual que con una lista.

```
colores = dict.fromkeys( ("rojo", "verde", "azul"), "color")
print(colores) # {'rojo': 'color', 'verde': 'color', 'azul': 'color'}
```

Con un string

Cuando le pasás un string, Python lo toma como una secuencia de caracteres, por lo que cada letra se vuelve una clave. ¡Ojo con esto!

```
texto = "Hola"
dic = dict.fromkeys(texto, 1)
print(dic) # {'H': 1, 'o': 1, 'l': 1, 'a': 1}
```

Esto es muy útil para crear contadores de letras, como un contador de vocales.

```
vocales = dict.fromkeys("aeiou", 0)
print(vocales)    # {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}
```

Con otro diccionario

Si le pasás un diccionario como secuencia, `fromkeys()` solo toma las **claves** de ese diccionario e **ignora completamente sus valores**. Esto es buenísimo para replicar la estructura de un diccionario pero con nuevos valores iniciales.

```
original = {"a": 100, "b": 200, "c": 300}
nuevo = dict.fromkeys(original, 0)
print(nuevo)    # {'a': 0, 'b': 0, 'c': 0}
```

Tipos de valores

El segundo argumento puede ser cualquier objeto de Python: un número, un string, `None`, una lista, otro diccionario, etc. Pero hay que tener cuidado con los valores mutables.

```
# Con números
numeros = dict.fromkeys([1, 2, 3], 0)
print("Numeros:", numeros)

# Con strings
estados = dict.fromkeys(["activo", "inactivo"], "pendiente")
print("Estados:", estados)

# Omitiendo el valor (resulta None)
sin_valor = dict.fromkeys(["a", "b", "c"])
print("Sin valor:", sin_valor)

Numeros: {1: 0, 2: 0, 3: 0}
Estados: {'activo': 'pendiente', 'inactivo': 'pendiente'}
Sin valor: {'a': None, 'b': None, 'c': None}
```

¡Cuidado con los valores mutables!

Si ponés una lista o un diccionario como valor inicial, todas las claves van a apuntar al **mismo objeto en memoria**. No se crean copias independientes. Esto puede ser una trampa si después modificás una de ellas.

```
# ¡PELIGRO! Todas las claves comparten la MISMA lista
dic = dict.fromkeys(["a", "b", "c"], [])
print("Original:", dic)

dic["a"].append(1)
print("Despues de modificar 'a':", dic)
# {'a': [1], 'b': [1], 'c': [1]} -> ¡se modificaron todas!
```

Nota: Moraleja: Si necesitas que cada clave tenga su propia lista o diccionario independiente, no uses `fromkeys()` con un valor mutable. En su lugar, construí el diccionario con un bucle o una comprensión.

Ejemplos prácticos

Inicializar materias para un experimento

En la facultad, cuando arrancás un cuatrimestre, todas las notas empiezan en cero. Con `fromkeys()` lo resolvés en una línea.

```
materias = ["Fisica", "Matematica", "Quimica", "Biologia", "Programacion"]
notas = dict.fromkeys(materias, 0)
```

```
print("=== NOTAS INICIALES ===")
for mate, nota in notas.items():
    print(f"{mate}: {nota}")
```

```
# Despues cargamos notas reales
notas["Fisica"] = 8
notas["Matematica"] = 3
notas["Programacion"] = 10
```

```
print("\n=== NOTAS FINALES ===")
for mate, nota in notas.items():
    print(f"{mate}: {nota}")
```

Contador de vocales

```
contador = dict.fromkeys("aeiou", 0)
frase = "hola mundo desde python"
for letra in frase:
    if letra in contador:
        contador[letra] = contador[letra] + 1
print(contador) # muestra cuantas veces aparece cada vocal
```

Inicializar estados de empleados

```
empleados = ["Ana", "Luis", "Pedro", "Marta", "Juan"]
estado = dict.fromkeys(empleados, "presente")
```

```
# Alguien falta
estado["Luis"] = "ausente"
estado["Pedro"] = "vacaciones"
```

```
for emp, est in estado.items():
    print(f"{emp}: {est}")
```

Ejercicios propuestos

Ahora vos. Agarrá el intérprete y probá cada uno. No mires las soluciones hasta que lo hayas intentado.

1. Usá `dict.fromkeys()` para crear un diccionario con las claves “a”, “b”, “c” y valor inicial 0.
2. Creá un diccionario con las claves de una lista de frutas: “manzana”, “pera”, “uva” y valor inicial “disponible”.
3. Usá un string como secuencia para crear un diccionario con las letras de “Python” como claves y valor inicial 0.
4. Creá un diccionario a partir de otro diccionario existente, tomando solo sus claves y asignando valor inicial 100.
5. Creá un diccionario con una lista de 5 materias y valor inicial “pendiente”.
6. Usá `dict.fromkeys()` para crear un diccionario con las claves 1, 2, 3, 4, 5 y valor inicial `None` (sin poner el valor).
7. Creá un diccionario con los meses del año (1 a 12) como claves y valor inicial 0.
8. ¿Qué pasa si usás una lista como valor inicial? Probá y explicá el resultado.
9. Creá un diccionario con las claves de una tupla (“rojo”, “verde”, “azul”) y valor inicial “color”.
10. Inicializá un diccionario para contar las vocales en una frase usando `dict.fromkeys()` con las vocales como claves y valor inicial 0. Después probá contar las vocales de una frase cualquiera.
11. ¿Funciona `fromkeys()` con un `range`? Probá `dict.fromkeys(range(1,6), "valor")` y analizá el resultado.
12. Creá un diccionario con las claves de una lista de letras `[.a, "b", ç]` y como valor una lista vacía `[]`. Luego agregá el número 5 a la lista de la clave “a”. ¿Qué observás? Explicá por qué ocurre.
13. Escribí un programa que pida una lista de palabras separadas por comas y las convierta en un diccionario donde cada palabra sea una clave con el valor 0, usando `fromkeys()`. Mostrá el resultado.
14. Usá `fromkeys()` para crear un diccionario que represente un tablero de puntuaciones para 10 jugadores (nombres “jugador1” a “jugador10”) todos con puntaje inicial 0.
15. A partir de un diccionario de productos con precios, creá un nuevo diccionario con las mismas claves pero con todos los precios en 0 para un reinicio de promoción.
16. ¿Se puede usar un conjunto (set) como secuencia en `fromkeys()`? Probá y explicá qué sucede.

17. Creá una función `inicializar_contadores(lista)` que reciba una lista de palabras y devuelva un diccionario.
17. Combiná `fromkeys()` con un bucle para llevar un conteo de las veces que aparece cada palabra en una lista, partiendo de un diccionario inicializado con cero.
18. Desarrollá un programa que pida al usuario 5 nombres y los inicialice en un diccionario de asistencia (todos “ausente”). Luego permita marcar algunos como “presente” y muestre el resultado final.
19. Investigá y explicá la diferencia entre `dict.fromkeys(seq, [])` y usar un bucle para asignar listas independientes. Escribí un ejemplo que muestre la diferencia.

Soluciones paso a paso

Solución 1

```
dic = dict.fromkeys(["a", "b", "c"], 0)
print(dic) # {'a': 0, 'b': 0, 'c': 0}
```

Explicación: La lista provee las claves y el 0 el valor para todas.

Solución 2

```
frutas = dict.fromkeys(["manzana", "pera", "uva"], "disponible")
print(frutas)
```

Explicación: Misma idea, con un string como valor.

Solución 3

```
letras = dict.fromkeys("Python", 0)
print(letras) # {'P': 0, 'y': 0, 't': 0, 'h': 0, 'o': 0, 'n': 0}
```

Explicación: El string “Python” es una secuencia de caracteres.

Solución 4

```
original = {"a": 10, "b": 20, "c": 30}
nuevo = dict.fromkeys(original, 100)
print(nuevo) # {'a': 100, 'b': 100, 'c': 100}
```

Explicación: Toma solo las claves del diccionario original.

Solución 5

```
materias = dict.fromkeys(["Mate", "Fisica", "Quimica", "Historia", "Lengua"], "pendiente")
print(materias)
```

Explicación: Un ejemplo típico de inicialización.

Solución 6

```
numeros = dict.fromkeys([1, 2, 3, 4, 5])
print(numeros) # {1: None, 2: None, 3: None, 4: None, 5: None}
```

Explicación: Al omitir el valor, cada clave recibe None.

Solución 7

```
meses = dict.fromkeys(range(1, 13), 0)
print(meses)
```

Explicación: range(1,13) genera los números 1 a 12.

Solución 8

```
# ¡Cuidado! Todas las claves comparten la misma lista
d = dict.fromkeys(["a", "b", "c"], [])
d["a"].append(1)
print(d) # {'a': [1], 'b': [1], 'c': [1]}
# Modificar una modifica todas porque es el mismo objeto.
```

Explicación: Al ser el mismo objeto mutable, los cambios se reflejan en todas las claves.

Solución 9

```
colores = dict.fromkeys( ("rojo", "verde", "azul"), "color")
print(colores)
```

Explicación: Funciona exactamente igual con una tupla.

Solución 10

```
contador = dict.fromkeys("aeiou", 0)
frase = "aprendiendo python en la uba"
for letra in frase.lower():
    if letra in contador:
        contador[letra] += 1
print(contador)
```

Explicación: Inicializamos el contador y luego sumamos si la letra es vocal.

Solución 11

```
d = dict.fromkeys(range(1,6), "valor")
print(d) # {1: 'valor', 2: 'valor', 3: 'valor', 4: 'valor', 5: 'valor'}
```

Explicación: range es una secuencia iterable, así que funciona perfecto.

Solución 12

```
d = dict.fromkeys(["a","b","c"], [])
d["a"].append(5)
print(d)
# {'a': [5], 'b': [5], 'c': [5]}
# Ocurre porque todas las claves apuntan a la misma lista vacía.
```

Explicación: Se debe a la mutabilidad compartida; si querés listas independientes debés usar un bucle o comprensión de diccionarios.

Solución 13

```
entrada = input("Ingresa palabras separadas por comas: ")
palabras = entrada.split(",")
dic = dict.fromkeys(palabras, 0)
print(dic)
```

Explicación: `split(",")` crea una lista que luego usamos como secuencia.

Solución 14

```
jugadores = [f"jugador{i}" for i in range(1,11)]
puntos = dict.fromkeys(jugadores, 0)
print(puntos)
```

Explicación: Generamos los nombres con una comprensión y luego inicializamos.

Solución 15

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
rebajados = dict.fromkeys(productos, 0)
print(rebajados)
```

Explicación: Las mismas claves, ahora con valor 0.

Solución 16

```
conjunto = {"a", "b", "c"}
dic = dict.fromkeys(conjunto, 1)
print(dic) # {'a': 1, 'b': 1, 'c': 1}
```

Explicación: Un set también es iterable, así que funciona. Pero el orden no está garantizado (aunque a veces se ve ordenado).

Solución 17

```
def inicializar_contadores(lista):
    return dict.fromkeys(lista, 0)

print(inicializar_contadores(["manzana", "pera"]))
```

Explicación: Encapsulamos la lógica en una función reutilizable.

Solución 18

```
palabras = ["hola", "mundo", "hola", "python", "mundo", "mundo"]
contador = dict.fromkeys(set(palabras), 0)
for palabra in palabras:
    contador[palabra] += 1
print(contador)
```

Explicación: Creamos el diccionario con las palabras únicas (usando set) y luego contamos.

Solución 19

```
asistencia = dict.fromkeys(["Ana","Luis","Pedro","Marta","Juan"], "ausente")
print("Asistencia inicial:", asistencia)
```

```
asistencia["Ana"] = "presente"
asistencia["Luis"] = "presente"
print("Asistencia final:", asistencia)
```

Explicación: Inicializamos y luego modificamos los que llegaron.

Solución 20

```
# Con fromkeys (problemático)
d1 = dict.fromkeys(["a","b"], [])
d1["a"].append(1)
print("Con fromkeys:", d1) # ambos tienen [1]
```

```
# Con bucle (independiente)
d2 = {}
for key in ["a","b"]:
    d2[key] = []
d2["a"].append(1)
print("Con bucle:", d2) # {'a': [1], 'b': []}
```

Explicación: El bucle crea una nueva lista en cada iteración, mientras que `fromkeys` asigna el mismo objeto a todas las claves.

Resumen del capítulo

En este capítulo aprendiste:

- `dict.fromkeys()` crea un nuevo diccionario a partir de una secuencia y un valor opcional.
- La secuencia puede ser lista, tupla, string, rango, etc., y cada elemento se convierte en clave.
- Si se omite el valor, todas las claves quedan con `None`.
- Es ideal para inicializar estructuras de datos (notas, contadores, estados).
- Cuidado con los valores mutables: todas las claves comparten el mismo objeto.
- La forma recomendada es `dict.fromkeys(seq, valor)`, no `{}.fromkeys()`.

fromkeys() es como un molde para hacer galletitas: todas salen con la misma forma (claves) y el mismo sabor inicial (valor), hasta que las decorás una por una.

Método fromkeys() – Crear Diccionarios de Forma Masiva

¿Qué es el método fromkeys()?

Cuando arranqué con Python en la UBA Exactas, recuerdo que tenía que anotar las notas de un montón de materias todas de una vez. Al principio hacía un diccionario vacío y con un `for` iba agregando cada materia con 0. Pero un profe me dijo: “Che, Nahuel, ¿no conocés `fromkeys()`? Te crea el diccionario entero de una, con todas las claves y el mismo valor inicial. Es un golazo.” Y tenía razón. Desde ese día, cada vez que necesito inicializar una estructura con valores por defecto, uso `fromkeys()`.

El método `fromkeys()` es una función que crea un **nuevo diccionario** a partir de una **secuencia** (una lista, tupla, string, o incluso otro diccionario) y un **valor opcional** que se asignará a cada una de esas claves. En criollo: vos le das los nombres de los cajones y un contenido estándar, y él te arma el mueble entero.

Analogía: Imaginá que tenés un formulario donde todos los campos empiezan en blanco o con un valor predeterminado. `fromkeys()` es como darte ese formulario ya armado con todos los campos listos, y vos después solo completás los que cambian.

```
+-----+
| dict.fromkeys(secuencia, valor) |
|                               |
| secuencia: lista, tupla, string o diccionario |
|           (sus elementos serán las claves)   |
|                               |
| valor: (opcional) valor para todas las claves |
|           si no se pone, es None             |
+-----+
```

Sintaxis fundamental

La forma más común y recomendada es llamar al método directamente desde la clase `dict`:

```
nuevo_diccionario = dict.fromkeys(secuencia, valor)
```

- **Secuencia (obligatorio):** Los elementos que se convertirán en las claves del diccionario. Puede ser una lista, tupla, string, range, etc.
- **Valor (opcional):** El valor que se le dará a cada clave. Si no lo ponés, por defecto es `None`.

Dos formas de invocarlo

Hay dos maneras de llamar a `fromkeys()`, pero la primera es la más clara y la que vas a ver en todos lados:

```
# FORMA 1: Usando la clase dict (RECOMENDADA)
diccionario1 = dict.fromkeys(["a", "b", "c"], 0)
print(diccionario1) # {'a': 0, 'b': 0, 'c': 0}
```

```
# FORMA 2: Usando llaves vacías (menos comun)
diccionario2 = {}.fromkeys(["x", "y", "z"], 1)
print(diccionario2) # {'x': 1, 'y': 1, 'z': 1}
```

Salida:

```
{'a': 0, 'b': 0, 'c': 0}
{'x': 1, 'y': 1, 'z': 1}
```

Nota: Recomendación: Siempre usá la primera forma `dict.fromkeys()`. Es más explícita y se entiende inmediatamente que estás creando un diccionario nuevo. La segunda forma con `{}.fromkeys()` funciona, pero es menos legible.

Comportamientos según la secuencia

`fromkeys()` es bastante versátil y se adapta al tipo de secuencia que le entregues.

Con una lista

Cada elemento de la lista se convierte en una clave.

```
claves = ["fisica", "mate", "quimica"]
materias = dict.fromkeys(claves, 0)
print(materias) # {'fisica': 0, 'mate': 0, 'quimica': 0}
```

Con una tupla

Funciona exactamente igual que con una lista.

```
colores = dict.fromkeys( ("rojo", "verde", "azul"), "color")
print(colores) # {'rojo': 'color', 'verde': 'color', 'azul': 'color'}
```

Con un string

Cuando le pasás un string, Python lo toma como una secuencia de caracteres, por lo que cada letra se vuelve una clave. ¡Ojo con esto!

```
texto = "Hola"
dic = dict.fromkeys(texto, 1)
print(dic) # {'H': 1, 'o': 1, 'l': 1, 'a': 1}
```


Esto es muy útil para crear contadores de letras, como un contador de vocales.

```
vocales = dict.fromkeys("aeiou", 0)
print(vocales)    # {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}
```

Con otro diccionario

Si le pasás un diccionario como secuencia, `fromkeys()` solo toma las **claves** de ese diccionario e **ignora completamente sus valores**. Esto es buenísimo para replicar la estructura de un diccionario pero con nuevos valores iniciales.

```
original = {"a": 100, "b": 200, "c": 300}
nuevo = dict.fromkeys(original, 0)
print(nuevo)    # {'a': 0, 'b': 0, 'c': 0}
```

Tipos de valores

El segundo argumento puede ser cualquier objeto de Python: un número, un string, `None`, una lista, otro diccionario, etc. Pero hay que tener cuidado con los valores mutables.

```
# Con números
numeros = dict.fromkeys([1, 2, 3], 0)
print("Numeros:", numeros)

# Con strings
estados = dict.fromkeys(["activo", "inactivo"], "pendiente")
print("Estados:", estados)

# Omitiendo el valor (resulta None)
sin_valor = dict.fromkeys(["a", "b", "c"])
print("Sin valor:", sin_valor)

Numeros: {1: 0, 2: 0, 3: 0}
Estados: {'activo': 'pendiente', 'inactivo': 'pendiente'}
Sin valor: {'a': None, 'b': None, 'c': None}
```

¡Cuidado con los valores mutables!

Si ponés una lista o un diccionario como valor inicial, todas las claves van a apuntar al **mismo objeto en memoria**. No se crean copias independientes. Esto puede ser una trampa si después modificás una de ellas.

```
# ¡PELIGRO! Todas las claves comparten la MISMA lista
dic = dict.fromkeys(["a", "b", "c"], [])
print("Original:", dic)

dic["a"].append(1)
print("Despues de modificar 'a':", dic)
# {'a': [1], 'b': [1], 'c': [1]} -> ¡se modificaron todas!
```

Nota: Moraleja: Si necesitas que cada clave tenga su propia lista o diccionario independiente, no uses `fromkeys()` con un valor mutable. En su lugar, construí el diccionario con un bucle o una comprensión.

Ejemplos prácticos

Inicializar materias para un experimento

En la facultad, cuando arrancás un cuatrimestre, todas las notas empiezan en cero. Con `fromkeys()` lo resolvés en una línea.

```
materias = ["Fisica", "Matematica", "Quimica", "Biologia", "Programacion"]
notas = dict.fromkeys(materias, 0)
```

```
print("=== NOTAS INICIALES ===")
for mate, nota in notas.items():
    print(f"{mate}: {nota}")
```

```
# Despues cargamos notas reales
notas["Fisica"] = 8
notas["Matematica"] = 3
notas["Programacion"] = 10
```

```
print("\n=== NOTAS FINALES ===")
for mate, nota in notas.items():
    print(f"{mate}: {nota}")
```

Contador de vocales

```
contador = dict.fromkeys("aeiou", 0)
frase = "hola mundo desde python"
for letra in frase:
    if letra in contador:
        contador[letra] = contador[letra] + 1
print(contador) # muestra cuantas veces aparece cada vocal
```

Inicializar estados de empleados

```
empleados = ["Ana", "Luis", "Pedro", "Marta", "Juan"]
estado = dict.fromkeys(empleados, "presente")
```

```
# Alguien falta
estado["Luis"] = "ausente"
estado["Pedro"] = "vacaciones"
```

```
for emp, est in estado.items():
    print(f"{emp}: {est}")
```

Ejercicios propuestos

Ahora vos. Agarrá el intérprete y probá cada uno. No mires las soluciones hasta que lo hayas intentado.

1. Usá `dict.fromkeys()` para crear un diccionario con las claves “a”, “b”, “c” y valor inicial 0.
2. Creá un diccionario con las claves de una lista de frutas: “manzana”, “pera”, “uva” y valor inicial “disponible”.
3. Usá un string como secuencia para crear un diccionario con las letras de “Python” como claves y valor inicial 0.
4. Creá un diccionario a partir de otro diccionario existente, tomando solo sus claves y asignando valor inicial 100.
5. Creá un diccionario con una lista de 5 materias y valor inicial “pendiente”.
6. Usá `dict.fromkeys()` para crear un diccionario con las claves 1, 2, 3, 4, 5 y valor inicial `None` (sin poner el valor).
7. Creá un diccionario con los meses del año (1 a 12) como claves y valor inicial 0.
8. ¿Qué pasa si usás una lista como valor inicial? Probá y explicá el resultado.
9. Creá un diccionario con las claves de una tupla (“rojo”, “verde”, “azul”) y valor inicial “color”.
10. Inicializá un diccionario para contar las vocales en una frase usando `dict.fromkeys()` con las vocales como claves y valor inicial 0. Después probá contar las vocales de una frase cualquiera.
11. ¿Funciona `fromkeys()` con un `range`? Probá `dict.fromkeys(range(1,6), "valor")` y analizá el resultado.
12. Creá un diccionario con las claves de una lista de letras `[.a, "b", ç]` y como valor una lista vacía `[]`. Luego agregá el número 5 a la lista de la clave “a”. ¿Qué observás? Explicá por qué ocurre.
13. Escribí un programa que pida una lista de palabras separadas por comas y las convierta en un diccionario donde cada palabra sea una clave con el valor 0, usando `fromkeys()`. Mostrá el resultado.
14. Usá `fromkeys()` para crear un diccionario que represente un tablero de puntuaciones para 10 jugadores (nombres “jugador1” a “jugador10”) todos con puntaje inicial 0.
15. A partir de un diccionario de productos con precios, creá un nuevo diccionario con las mismas claves pero con todos los precios en 0 para un reinicio de promoción.
16. ¿Se puede usar un conjunto (set) como secuencia en `fromkeys()`? Probá y explicá qué sucede.

17. Creá una función `inicializar_contadores(lista)` que reciba una lista de palabras y devuelva un diccionario.
17. Combiná `fromkeys()` con un bucle para llevar un conteo de las veces que aparece cada palabra en una lista, partiendo de un diccionario inicializado con cero.
18. Desarrollá un programa que pida al usuario 5 nombres y los inicialice en un diccionario de asistencia (todos “ausente”). Luego permita marcar algunos como “presente” y muestre el resultado final.
19. Investigá y explicá la diferencia entre `dict.fromkeys(seq, [])` y usar un bucle para asignar listas independientes. Escribí un ejemplo que muestre la diferencia.

Soluciones paso a paso

Solución 1

```
dic = dict.fromkeys(["a", "b", "c"], 0)
print(dic) # {'a': 0, 'b': 0, 'c': 0}
```

Explicación: La lista provee las claves y el 0 el valor para todas.

Solución 2

```
frutas = dict.fromkeys(["manzana", "pera", "uva"], "disponible")
print(frutas)
```

Explicación: Misma idea, con un string como valor.

Solución 3

```
letras = dict.fromkeys("Python", 0)
print(letras) # {'P': 0, 'y': 0, 't': 0, 'h': 0, 'o': 0, 'n': 0}
```

Explicación: El string “Python” es una secuencia de caracteres.

Solución 4

```
original = {"a": 10, "b": 20, "c": 30}
nuevo = dict.fromkeys(original, 100)
print(nuevo) # {'a': 100, 'b': 100, 'c': 100}
```

Explicación: Toma solo las claves del diccionario original.

Solución 5

```
materias = dict.fromkeys(["Mate", "Fisica", "Quimica", "Historia", "Lengua"], "pendiente")
print(materias)
```

Explicación: Un ejemplo típico de inicialización.

Solución 6

```
numeros = dict.fromkeys([1, 2, 3, 4, 5])
print(numeros) # {1: None, 2: None, 3: None, 4: None, 5: None}
```

Explicación: Al omitir el valor, cada clave recibe None.

Solución 7

```
meses = dict.fromkeys(range(1, 13), 0)
print(meses)
```

Explicación: range(1,13) genera los números 1 a 12.

Solución 8

```
# ¡Cuidado! Todas las claves comparten la misma lista
d = dict.fromkeys(["a", "b", "c"], [])
d["a"].append(1)
print(d) # {'a': [1], 'b': [1], 'c': [1]}
# Modificar una modifica todas porque es el mismo objeto.
```

Explicación: Al ser el mismo objeto mutable, los cambios se reflejan en todas las claves.

Solución 9

```
colores = dict.fromkeys( ("rojo", "verde", "azul"), "color")
print(colores)
```

Explicación: Funciona exactamente igual con una tupla.

Solución 10

```
contador = dict.fromkeys("aeiou", 0)
frase = "aprendiendo python en la uba"
for letra in frase.lower():
    if letra in contador:
        contador[letra] += 1
print(contador)
```

Explicación: Inicializamos el contador y luego sumamos si la letra es vocal.

Solución 11

```
d = dict.fromkeys(range(1,6), "valor")
print(d) # {1: 'valor', 2: 'valor', 3: 'valor', 4: 'valor', 5: 'valor'}
```

Explicación: range es una secuencia iterable, así que funciona perfecto.

Solución 12

```
d = dict.fromkeys(["a","b","c"], [])
d["a"].append(5)
print(d)
# {'a': [5], 'b': [5], 'c': [5]}
# Ocurre porque todas las claves apuntan a la misma lista vacía.
```

Explicación: Se debe a la mutabilidad compartida; si querés listas independientes debés usar un bucle o comprensión de diccionarios.

Solución 13

```
entrada = input("Ingresa palabras separadas por comas: ")
palabras = entrada.split(",")
dic = dict.fromkeys(palabras, 0)
print(dic)
```

Explicación: `split(",")` crea una lista que luego usamos como secuencia.

Solución 14

```
jugadores = [f"jugador{i}" for i in range(1,11)]
puntos = dict.fromkeys(jugadores, 0)
print(puntos)
```

Explicación: Generamos los nombres con una comprensión y luego inicializamos.

Solución 15

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
rebajados = dict.fromkeys(productos, 0)
print(rebajados)
```

Explicación: Las mismas claves, ahora con valor 0.

Solución 16

```
conjunto = {"a", "b", "c"}
dic = dict.fromkeys(conjunto, 1)
print(dic) # {'a': 1, 'b': 1, 'c': 1}
```

Explicación: Un set también es iterable, así que funciona. Pero el orden no está garantizado (aunque a veces se ve ordenado).

Solución 17

```
def inicializar_contadores(lista):
    return dict.fromkeys(lista, 0)

print(inicializar_contadores(["manzana", "pera"]))
```

Explicación: Encapsulamos la lógica en una función reutilizable.

Solución 18

```
palabras = ["hola", "mundo", "hola", "python", "mundo", "mundo"]
contador = dict.fromkeys(set(palabras), 0)
for palabra in palabras:
    contador[palabra] += 1
print(contador)
```

Explicación: Creamos el diccionario con las palabras únicas (usando set) y luego contamos.

Solución 19

```
asistencia = dict.fromkeys(["Ana","Luis","Pedro","Marta","Juan"], "ausente")
print("Asistencia inicial:", asistencia)
```

```
asistencia["Ana"] = "presente"
asistencia["Luis"] = "presente"
print("Asistencia final:", asistencia)
```

Explicación: Inicializamos y luego modificamos los que llegaron.

Solución 20

```
# Con fromkeys (problemático)
d1 = dict.fromkeys(["a","b"], [])
d1["a"].append(1)
print("Con fromkeys:", d1) # ambos tienen [1]
```

```
# Con bucle (independiente)
d2 = {}
for key in ["a","b"]:
    d2[key] = []
d2["a"].append(1)
print("Con bucle:", d2) # {'a': [1], 'b': []}
```

Explicación: El bucle crea una nueva lista en cada iteración, mientras que `fromkeys` asigna el mismo objeto a todas las claves.

Resumen del capítulo

En este capítulo aprendiste:

- `dict.fromkeys()` crea un nuevo diccionario a partir de una secuencia y un valor opcional.
- La secuencia puede ser lista, tupla, string, rango, etc., y cada elemento se convierte en clave.
- Si se omite el valor, todas las claves quedan con `None`.
- Es ideal para inicializar estructuras de datos (notas, contadores, estados).
- Cuidado con los valores mutables: todas las claves comparten el mismo objeto.
- La forma recomendada es `dict.fromkeys(seq, valor)`, no `{}.fromkeys()`.

fromkeys() es como un molde para hacer galletitas: todas salen con la misma forma (claves) y el mismo sabor inicial (valor), hasta que las decorás una por una.

Método `get()` – Acceder a Valores sin Riesgo de Error

¿Para qué sirve `get()`?

Cuando empecé a trabajar con diccionarios en la facultad (UBA Exactas), mi primer instinto fue acceder a los valores con `diccionario[clave]`. Funcionaba bárbaro hasta que un día quise mostrar el precio de un producto que no existía en el catálogo y el programa se me cayó con un `KeyError` horrible. Me sentí un tonto, pero un compañero me dijo: “Che, Nahuel, para eso está `get()`, no te rompe el programa, solo te devuelve `None` o lo que vos le digas”. Y desde entonces, `get()` se volvió mi método favorito para consultar diccionarios.

El método `.get()` se usa para acceder al valor de una clave dentro de un diccionario, pero con la gran ventaja de que si la clave no existe, **no lanza un error**, sino que devuelve un valor por defecto. En criollo, es como preguntar “¿tenés esto?” en lugar de exigir “¡dame esto o reviento!”.

Analogía: Es como cuando llamás por teléfono a un amigo: si atiende, te da la información; si no, podés dejar un mensaje o simplemente no pasa nada. El programa sigue andando sin cortarse.

```
+-----+
|
|  diccionario.get(clave, valor_por_defecto)  |
|
|  clave: la clave que querés buscar          |
|  valor_por_defecto: (opcional) lo que devuelve |
|                      si no encuentra la clave |
|                      (default: None)         |
|
|  ¡NO lanza KeyError si la clave no existe!  |
|
+-----+
```

Sintaxis

```
diccionario.get(clave, valor_por_defecto)
```

- **Clave (obligatorio):** La clave que querés buscar en el diccionario.
- **Valor por defecto (opcional):** Lo que querés que devuelva si no encuentra la clave. Si no ponés nada, devuelve `None`.

¿Por qué es mejor que el acceso directo?

Cuando hacés `diccionario[clave]`, si la clave no está, el programa **rompe** lanzando un `KeyError`:

```
precios = {"manzana": 1.55, "banana": 1.20, "pera": 1.40}

# Acceso directo - puede romper
print(precios["manzana"])    # 1.55
print(precios["mango"])      # ¡KeyError! El programa se detiene
```

Usando `.get()`, el flujo continúa normalmente:

```
precios = {"manzana": 1.55, "banana": 1.20, "pera": 1.40}

# Usando get() - seguro
print(precios.get("manzana"))    # 1.55
print(precios.get("mango"))      # None (sin error)
print(precios.get("mango", 4.55)) # 4.55 (valor personalizado)
```

Salida:

```
1.55
None
4.55
```

Nota: *Diferencia clave:*

- `diccionario[clave]` → si no existe, *ERROR (KeyError)*
- `diccionario.get(clave)` → si no existe, devuelve *None*

Usá `.get()` cuando no estés 100% seguro de que la clave exista.

Ejemplo práctico del video: Precios de frutas

Ernesto (de la UBA Exactas) utiliza un diccionario de precios de frutas para mostrar los tres escenarios posibles:

```
# Diccionario de precios
• precios = {"manzana": 1.55, "banana": 1.20, "pera": 1.40}

print("=== PRECIOS DE FRUTAS ===")

# Escenario 1: La clave existe
p_manzana = precios.get("manzana")
print(f"Precio de la manzana: ${p_manzana}")

# Escenario 2: La clave NO existe (sin valor por defecto)
p_mango = precios.get("mango")
```

```
print(f"Precio del mango: {p_mango}")

# Escenario 3: La clave NO existe (CON valor por defecto)
p_mango_promo = precios.get("mango", 4.55)
print(f"Precio promocional del mango: ${p_mango_promo}")

Salida:
=== PRECIOS DE FRUTAS ===
Precio de la manzana: $1.55
Precio del mango: None
Precio promocional del mango: $4.55
```

Ejemplo práctico: Sistema de notas

```
notas = {"Ana": 8, "Luis": 7, "Pedro": 9, "Marta": 10}

print("=== CONSULTA DE NOTAS ===")

# Alumnos que existen
print(f"Nota de Ana: {notas.get('Ana')}")
print(f"Nota de Luis: {notas.get('Luis')}")

# Alumno que NO existe (sin valor por defecto)
print(f"Nota de Carlos: {notas.get('Carlos')}")

# Alumno que NO existe (con valor por defecto)
print(f"Nota de Carlos (con default): {notas.get('Carlos', 'Alumno no encontrado')}")
```

Ejemplo práctico: Inventario de tienda

```
inventario = {"manzanas": 50, "peras": 30, "naranjas": 40}

print("=== CONSULTA DE STOCK ===")

print(f"Manzanas: {inventario.get('manzanas', 0)} unidades")
print(f"Peras: {inventario.get('peras', 0)} unidades")
print(f"Uvas: {inventario.get('uvas', 0)} unidades")
```

Ejemplo práctico: Contar frecuencias sin errores

El método `.get()` es ideal para contar cosas sin miedo a que falle.

```
frase = "hola mundo desde python"
conteo = {}

print("=== CONTADOR DE LETRAS ===")
print(f"Frase: {frase}")
```

```
for letra in frase:
    if letra != " ": # ignoramos espacios
        conteo[letra] = conteo.get(letra, 0) + 1

for letra, cantidad in conteo.items():
    print(f"'{letra}': {cantidad} veces")
```

Ejemplo práctico: Configuración con valores por defecto

```
config = {"tema": "oscuro", "idioma": "español"}

tema = config.get("tema", "claro")
idioma = config.get("idioma", "inglés")
tamano_fuente = config.get("tamano_fuente", 12)
notificaciones = config.get("notificaciones", True)

print(f"Tema: {tema}, Idioma: {idioma}")
print(f"Fuente: {tamano_fuente}, Notificaciones: {notificaciones}")
```

Comparativa: acceso directo vs get()

ACCESO DIRECTO	get()
dic["clave"]	dic.get("clave")
Si la clave existe → devuelve el valor	Si la clave existe → devuelve el valor
Si la clave NO existe → LANZA KeyError (el programa se detiene)	Si la clave NO existe → devuelve None (o el valor por defecto que le pongas)
Usar cuando SABES que la clave existe	Usar cuando NO estás SEGURO de que la clave exista

Ejercicios Propuestos

Ejercicio 1

Creá un diccionario con 3 productos y sus precios. Usá `.get()` para obtener el precio de un producto que existe.

Ejercicio 2

Usá `.get()` para obtener el precio de un producto que NO existe. ¿Qué devuelve?

Ejercicio 3

Usá `.get()` con un valor por defecto personalizado para un producto que no existe.

Ejercicio 4

Creá un diccionario con las notas de 5 alumnos. Usá `.get()` para consultar la nota de un alumno que no está en el diccionario.

Ejercicio 5

Creá un programa que pida al usuario una clave y use `.get()` para mostrar un mensaje personalizado si la clave no existe.

Ejercicio 6

Usá `.get()` dentro de un bucle para contar frecuencias de letras en una palabra.

Ejercicio 7

Creá un diccionario de inventario. Usá `.get()` para consultar stock de productos, retornando 0 si no existen.

Ejercicio 8

Creá un diccionario de configuración. Usá `.get()` para acceder a valores con defaults razonables.

Ejercicio 9

Compará qué pasa cuando usás `dic[clave]` vs `dic.get(clave)` con una clave que no existe. Explicá la diferencia.

Ejercicio 10

Creá un diccionario con 3 ciudades y sus poblaciones. Usá `.get()` con un valor por defecto de "Dato no disponible".

Ejercicio 11

Tenés un diccionario `estudiantes = "Juan": 19, "Maria": 21, "Pedro": 22`. Pedile al usuario un nombre y mostrá la edad, pero si no está, decí `.Es`a persona no cursa".

Ejercicio 12

Escribí una función `obtener_stock(producto)` que recorra un diccionario de inventario global y devuelva la cantidad. Usá `get()` para devolver 0 si el producto no existe.

Ejercicio 13

Con el diccionario `precios = {"azucar": 80, "zerba": 120, "cafe": 150}`, armá un programa que pida un producto y una cantidad, y calcule el total. Si el producto no existe, que muestre un mensaje y no calcule nada.

Ejercicio 14

Usá `get()` para sumar solo los valores de las claves `"a"` y `"b"` de un diccionario, ignorando claves faltantes (usá 0 por defecto para que la suma no falle).

Ejercicio 15

Tenés dos diccionarios con precios de diferentes sucursales. Usá `get()` para crear un tercer diccionario que contenga la suma de los precios de los productos comunes (si un producto falta en uno, se toma su precio como 0).

Ejercicio 16

Pedí al usuario que ingrese una frase y contá cuántas veces aparece cada vocal (a, e, i, o, u) usando `get()`. Mostrá el resultado sin importar mayúsculas.

Ejercicio 17

En una encuesta, las respuestas se guardan en un diccionario: `respuestas = {"si": 0, "no": 0, "tal_vez": 0}`. Cada vez que un usuario contesta, se incrementa la cuenta de esa respuesta usando `get(respuesta, 0) + 1` y se actualiza. Escribí un programa que simule 10 respuestas aleatorias y muestre el resultado final.

Ejercicio 18

Creá un diccionario `config = {"host": "localhost", "port": 8080}`. Escribí una función `conectar(config)` que obtenga host y port con `get()`, usando `"127.0.0.1"` y `80` como defaults, y simule una conexión imprimiendo los valores.

Ejercicio 19

Implementá un traductor simple: tenés un diccionario `ingles_espanol` con algunas palabras. El usuario ingresa una palabra en inglés y, usando `get()`, mostrás la traducción o un mensaje de que no se encontró. Además, permití que el usuario agregue nuevas palabras al diccionario si no existen.

Ejercicio 20

Usando `get()` y un bucle, generá un diccionario que contenga la frecuencia de cada palabra en una lista. Luego, mostrá las 3 palabras más frecuentes (sin librerías externas). Ayuda: armá el diccionario con `get()` y después ordená por valor.

Soluciones Paso a Paso

Solución 1

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
precio = productos.get("pan")
print(f"Precio del pan: ${precio}")
```

Explicación: La clave "pan" existe, por lo que get() devuelve 70.

Solución 2

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
precio = productos.get("coca")
print(f"Precio de la coca: {precio}") # None
```

Explicación: Como "coca" no existe, devuelve None sin error.

Solución 3

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
precio = productos.get("coca", 150)
print(f"Precio de la coca: ${precio}") # 150
```

Explicación: Al poner un valor por defecto, obtenemos 150 en lugar de None.

Solución 4

```
notas = {"Ana": 8, "Luis": 7, "Pedro": 9, "Marta": 10, "Juan": 6}
nota = notas.get("Carlos", "Alumno no encontrado")
print(f"Nota de Carlos: {nota}")
```

Explicación: Usamos un string como valor por defecto para un mensaje claro.

Solución 5

```
datos = {"nombre": "Nahuel", "edad": 25}
clave = input("Qué clave querés buscar? ")
valor = datos.get(clave, "Esa clave no existe en el diccionario")
print(f"Resultado: {valor}")
```

Explicación: El programa no falla ante una entrada desconocida.

Solución 6

```
palabra = "python"
conteo = {}
for letra in palabra:
    conteo[letra] = conteo.get(letra, 0) + 1
print(conteo)
```

Explicación: Cada letra arranca con 0 si no existe y se va incrementando.

Solución 7

```
inventario = {"manzanas": 50, "peras": 30}
stock = inventario.get("uvas", 0)
print(f"Stock de uvas: {stock}")
```

Explicación: Al consultar un producto ausente, recibimos 0 unidades.

Solución 8

```
config = {"tema": "oscuro"}
tema = config.get("tema", "claro")
fuente = config.get("fuente", "Arial")
tamano = config.get("tamano", 12)
print(f"Tema: {tema}, Fuente: {fuente}, Tamaño: {tamano}")
```

Explicación: Para claves no especificadas, se aplican valores por defecto.

Solución 9

```
dic = {"a": 1}
# print(dic["b"]) # Esto daría KeyError
print(dic.get("b")) # None
print(dic.get("b", "no existe")) # "no existe"
```

Explicación: Con get() el programa no se interrumpe; con acceso directo sí.

Solución 10

```
ciudades = {"Buenos Aires": 15000000, "Córdoba": 1500000, "Rosario": 1200000}
poblacion = ciudades.get("Mendoza", "Dato no disponible")
print(f"Población de Mendoza: {poblacion}")
```

Explicación: Al no estar Mendoza, se muestra el mensaje personalizado.

Solución 11

```
estudiantes = {"Juan": 19, "Maria": 21, "Pedro": 22}
nombre = input("Nombre del estudiante: ")
edad = estudiantes.get(nombre, "Esa persona no cursa")
print(f"{nombre}: {edad}")
```

Explicación: get() devuelve el mensaje si el nombre no está.

Solución 12

```
inventario = {"tornillos": 350, "tuercas": 500}
def obtener_stock(producto):
    return inventario.get(producto, 0)

print(obtener_stock("tornillos")) # 350
print(obtener_stock("clavos"))    # 0
```

Explicación: La función encapsula la consulta segura.

Solución 13

```
precios = {"azucar": 80, "yerba": 120, "cafe": 150}
producto = input("Producto: ")
cantidad = int(input("Cantidad: "))
precio_unitario = precios.get(producto)
if precio_unitario is not None:
    total = precio_unitario * cantidad
    print(f"Total: ${total}")
else:
    print("Producto no encontrado.")
```

Explicación: Validamos la existencia antes de calcular.

Solución 14

```
datos = {"a": 5, "c": 10}
suma = datos.get("a", 0) + datos.get("b", 0)
print(suma) # 5
```

Explicación: La clave "b" no existe, así que aporta 0.

Solución 15

```
suc1 = {"manzana": 100, "pera": 80}
suc2 = {"manzana": 120, "naranja": 90}
totales = {}
for clave in set(list(suc1.keys()) + list(suc2.keys())):
    totales[clave] = suc1.get(clave, 0) + suc2.get(clave, 0)
print(totales) # {'manzana': 220, 'pera': 80, 'naranja': 90}
```

Explicación: set junta todas las claves únicas; get asegura que no falle.

Solución 16

```
frase = input("Frase: ").lower()
vocales = "aeiou"
contador = {}
for v in vocales:
    contador[v] = 0
for letra in frase:
    if letra in vocales:
        contador[letra] = contador.get(letra, 0) + 1
for vocal, cantidad in contador.items():
    print(f"{vocal}: {cantidad}")
```

Explicación: Inicializamos con 0 y contamos solo vocales.

Solución 17

```
import random
respuestas = {"si": 0, "no": 0, "tal_vez": 0}
for _ in range(10):
    respuesta = random.choice(["si", "no", "tal_vez"])
    respuestas[respuesta] = respuestas.get(respuesta, 0) + 1
print(respuestas)
```

Explicación: get nos da el valor actual o 0 para incrementar.

Solución 18

```
def conectar(config):
    host = config.get("host", "127.0.0.1")
    port = config.get("port", 80)
    print(f"Conectando a {host} en puerto {port}...")

config = {"host": "localhost", "port": 8080}
conectar(config)
conectar({}) # usa defaults
```

Explicación: Si la clave falta, se emplean valores seguros.

Solución 19

```
ingles_espanol = {"hello": "hola", "cat": "gato", "dog": "perro"}
while True:
    palabra = input("Inglés (enter para salir): ")
    if palabra == "":
        break
    trad = ingles_espanol.get(palabra)
    if trad:
        print(f"Traducción: {trad}")
    else:
        print("No encontrada.")
        agregar = input("¿Querés agregarla? (s/n): ")
        if agregar.lower() == "s":
            significado = input("Traducción: ")
            ingles_espanol[palabra] = significado
            print("Agregada.")
```

Explicación: get() evita KeyError y permite ampliar el diccionario.

Solución 20

```
palabras = ["hola", "mundo", "hola", "python", "mundo", "mundo"]
frecuencia = {}
for p in palabras:
    frecuencia[p] = frecuencia.get(p, 0) + 1
```

```
ordenadas = sorted(frecuencia.items(), key=lambda x: x[1], reverse=True)
print("Top 3:")
for palabra, cant in ordenadas[:3]:
    print(f"{palabra}: {cant}")
```

Explicación: `get()` construye el diccionario; luego ordenamos por frecuencia descendente y mostramos los primeros 3.

Resumen del capítulo

En este capítulo aprendiste:

- `.get()` accede a valores de un diccionario de forma **segura**: no lanza `KeyError` si la clave no existe.
- Su sintaxis es `diccionario.get(clave, valor_por_defecto)`.
- Si la clave está, devuelve el valor; si no, devuelve `None` o el valor que le indiques.
- Es ideal para consultas inciertas, contadores, configuraciones con defaults y para evitar interrupciones en el programa.

get() es como llamar a alguien por teléfono: si atiende, le hablás; si no, dejás un mensaje y seguís con tu vida. El programa nunca se corta.

Método popitem() – Eliminar y Obtener el Último Elemento

¿Qué hace popitem()?

Cuando empecé a trabajar con diccionarios, muchas veces necesitaba procesar elementos en orden inverso al que los había cargado. Por ejemplo, en un sistema de tareas quería atender primero la última que había entrado, como una pila de platos. Al principio usaba `pop()` con una clave, pero tenía que saber cuál era la última. Un día un compañero de la UBA Exactas me dijo: “Che, Nahuel, ¿no conocés `popitem()`? Te saca automáticamente el último que pusiste, sin necesidad de recordar la clave”. ¡Y era justo lo que necesitaba!

El método `popitem()` se encarga de **eliminar y devolver** el último par clave-valor (ítem) que se agregó al diccionario. En criollo: saca el elemento más reciente y te lo entrega, dejando el diccionario sin ese par.

Analogía: Imaginá una pila de platos limpios. Vas apilando platos uno arriba del otro. Cuando necesitás uno, sacás el de arriba, que fue el último que pusiste. `popitem()` hace exactamente eso: saca el último plato (el último ítem agregado) y te lo da.

```
+-----+
|
|  diccionario.popitem()
|
|  Elimina y DEVUELVE el último par clave-valor
|  que se agregó al diccionario
|
|  Devuelve: una TUPLA con (clave, valor)
|
|  ¡No requiere argumentos!
|
+-----+
```

¿Qué devuelve?

`popitem()` devuelve una **tupla** con dos elementos: (clave, valor).

Nota: *¿Qué es una tupla? Es como una lista, pero no se puede modificar (es inmutable). Se escribe con paréntesis en lugar de corchetes: (clave, valor).*

Sintaxis

```
variable = diccionario.popitem()
```

- No necesita ningún argumento entre los paréntesis.
- Es muy recomendable **asignarlo a una variable** para poder aprovechar la información del elemento que se borró. Si no lo hacés, el par se elimina pero no podés acceder a él.

Ejemplo básico

```
# Diccionario inicial
datos = {"a": 1, "b": 2, "c": 3}
print("Diccionario original:", datos)

# Eliminamos el último elemento
eliminado = datos.popitem()

print(f"El ítem eliminado es: {eliminado}")
print(f"Diccionario actualizado: {datos}")
```

Salida:

```
Diccionario original: {'a': 1, 'b': 2, 'c': 3}
El ítem eliminado es: ('c', 3)
Diccionario actualizado: {'a': 1, 'b': 2}
```

¿Qué pasa si no asignamos a una variable?

El elemento se elimina igual, pero la información se pierde.

```
datos = {"x": 10, "y": 20, "z": 30}
print("Antes:", datos)

# Eliminamos sin guardar
datos.popitem()

print("Después:", datos)
# No sabemos qué se eliminó
```

Salida:

```
Antes: {'x': 10, 'y': 20, 'z': 30}
Después: {'x': 10, 'y': 20}
```

Ejemplo práctico: Simulación de una pila (LIFO)

El comportamiento de `popitem()` es **LIFO** (Last In, First Out), igual que una pila. El último en entrar es el primero en salir.

```
# Simulamos una pila de tareas
tareas = {
    "tarea1": "Estudiar Python",
    "tarea2": "Hacer ejercicios",
    "tarea3": "Repasar listas"
}

print("=== PILA DE TAREAS (LIFO) ===")
print("Tareas originales:", tareas)

# Procesamos de la más reciente a la más antigua
print("\nProcesando tareas...")
while tareas:
    tarea = tareas.popitem()
    print(f"Procesando: {tarea[1]} (clave: {tarea[0]})")

print("\nTareas pendientes:", tareas)
```

Ejemplo práctico: Deshacer acciones (Undo)

Ideal para sistemas de “deshacer”, donde querés revertir la última acción.

```
historial = {
    1: "Escribir 'Hola'",
    2: "Escribir ' mundo'",
    3: "Borrar palabra",
    4: "Agregar signo de exclamacion"
}

print("=== SISTEMA DE DESHACER (UNDO) ===")
print("Historial:", historial)

# Deshacer última acción
ultima = historial.popitem()
print(f"\nDeshaciendo: {ultima[1]}")
print("Historial actualizado:", historial)

# Deshacer otra
ultima = historial.popitem()
print(f"Deshaciendo: {ultima[1]}")
print("Historial actualizado:", historial)
```

Ejemplo práctico: Procesar pedidos (LIFO)

```
pedidos = {
    1001: "Pizza",
    1002: "Hamburguesa",
```



```

    1003: "Ensalada",
    1004: "Pastas"
}

print("=== SISTEMA DE PEDIDOS (LIFO) ===")
while pedidos:
    pedido = pedidos.popitem()
    print(f"Atendiendo pedido #{pedido[0]}: {pedido[1]}")

```

Error común: popitem() en diccionario vacío

Si intentás usar `popitem()` en un diccionario vacío, Python lanza un `KeyError`. Por eso siempre conviene verificar antes.

```

# Forma segura
diccionario = {"a": 1}

if diccionario:
    ultimo = diccionario.popitem()
    print(f"Se eliminó: {ultimo}")
else:
    print("El diccionario está vacío")

```

Ejercicios propuestos

1. Creá un diccionario con 5 elementos. Usá `popitem()` para eliminar y mostrar el último elemento.
2. Creá un diccionario. Usá `popitem()` sin asignarlo a una variable. Mostrá el diccionario antes y después. ¿Se perdió la información?
3. Creá un diccionario con 3 elementos. Ejecutá `popitem()` dos veces. Mostrá el diccionario después de cada eliminación.
4. ¿Qué tipo de dato devuelve `popitem()`? Comprobalo con `type()`.
5. Creá un diccionario vacío. Intentá usar `popitem()`. ¿Qué error ocurre? ¿Cómo lo evitarías?
6. Simulá una pila (LIFO) con 4 elementos usando `popitem()`. Mostrá cada elemento a medida que lo eliminás.
7. Creá un diccionario de productos y sus precios. Usá `popitem()` para “vender” el último producto agregado, mostrando cuál fue.
8. Creá un diccionario con 5 alumnos y sus notas. Usá `popitem()` para eliminar al último alumno agregado y mostrá su nota.
9. Creá un diccionario. Usá un bucle `while` para eliminar todos los elementos uno por uno con `popitem()`.

10. Guardá el resultado de `popitem()` en una variable y luego accedé a la clave y al valor por separado.
11. ¿En qué orden se eliminan los elementos? ¿Primero el más nuevo o el más viejo? Hacé una prueba y explicá.
12. Usá `popitem()` para deshacer la última operación en una lista de acciones representada como diccionario (ej.: `acciones = 1: .abrir", 2: .escribir", 3: cerrar`).
13. Escribí un programa que pida al usuario ingresar 5 palabras y las guarde como claves con valor igual a su longitud. Luego, usando `popitem()`, mostrálas en orden inverso al que las ingresó.
14. Creá un diccionario con claves numéricas incrementales (1, 2, 3, 4, 5) y valores sus cuadrados. Eliminá el último y mostrá el diccionario actualizado.
15. ¿Qué sucede si mezclás `popitem()` y `pop(clave)` en un mismo bucle? Creá un ejemplo.
16. Con el diccionario `letras = dict.fromkeys('abcdef', 0)`, eliminá y mostrá las últimas 3 letras usando `popitem()`.
17. Escribí una función `vaciar_pila(dic)` que recorra el diccionario con `popitem()` y devuelva una lista con los valores eliminados (en orden LIFO).
18. Simulá una bandeja de entrada de correos: cada nuevo correo tiene un número consecutivo y un asunto. Usá `popitem()` para leer los correos empezando por el más reciente.
19. Investigá: ¿Qué diferencia hay entre `popitem()` y `pop()` en un diccionario? Describí con tus palabras y da un ejemplo de cada uno.
20. Creá un programa de menú que permita al usuario agregar productos a un carrito (diccionario), y después, al elegir “vaciar carrito”, elimine todos los productos usando `popitem()` mostrando cada uno que se quita.

Soluciones paso a paso

Solución 1

```
datos = {"a": 1, "b": 2, "c": 3, "d": 4, "e": 5}
ultimo = datos.popitem()
print(f"Eliminado: {ultimo}") # ('e', 5)
print(datos)
```

Explicación: El último elemento agregado es el que se elimina.

Solución 2

```
d = {"x": 10, "y": 20}
print("Antes:", d)
d.popitem()
print("Después:", d)
# La información del elemento eliminado se pierde.
```

Explicación: No guardamos el resultado, así que no sabemos qué se eliminó exactamente (aunque podemos deducirlo por el orden).

Solución 3

```
d = {"a": 1, "b": 2, "c": 3}
d.popitem()
print("1er pop:", d) # {'a': 1, 'b': 2}
d.popitem()
print("2do pop:", d) # {'a': 1}
```

Explicación: Cada llamada elimina el último par presente.

Solución 4

```
d = {"a": 1}
resultado = d.popitem()
print(type(resultado)) # <class 'tuple'>
```

Explicación: popitem() devuelve una tupla de dos elementos.

Solución 5

```
d = {}
# d.popitem() # KeyError: 'popitem(): dictionary is empty'
# Solución: verificar antes
if d:
    d.popitem()
else:
    print("Está vacío, no se puede eliminar.")
```

Explicación: El error se evita comprobando si el diccionario tiene elementos.

Solución 6

```
pila = {"t1": "A", "t2": "B", "t3": "C", "t4": "D"}
print("Pila inicial:", pila)
while pila:
    elemento = pila.popitem()
    print(f"Sacado: {elemento}")
print("Pila vacía:", pila)
```

Explicación: Se sacan en orden inverso al de inserción (D, C, B, A).

Solución 7

```
productos = {"pan": 70, "leche": 90, "huevos": 120, "coca": 150}
vendido = productos.popitem()
print(f"Se vendió {vendido[0]} a ${vendido[1]}")
print("Restan:", productos)
```

Explicación: Eliminamos el último producto agregado ("coca").

Solución 8

```
alumnos = {"Ana": 8, "Luis": 7, "Pedro": 9, "Marta": 10, "Juan": 6}
ultimo = alumnos.popitem()
print(f"Último alumno agregado: {ultimo[0]} con nota {ultimo[1]}")
```

Explicación: Accedemos a la clave y al valor por separado con índices.

Solución 9

```
d = {"a": 1, "b": 2, "c": 3}
while d:
    eliminado = d.popitem()
    print(f"Eliminando: {eliminado}")
print("Quedó vacío:", d)
```

Explicación: La condición `while d:` se vuelve `False` cuando el diccionario queda vacío.

Solución 10

```
d = {"a": 1, "b": 2, "c": 3}
par = d.popitem()
clave = par[0]
valor = par[1]
print(f"Clave: {clave}, Valor: {valor}")
```

Explicación: Desempaquetamos la tupla accediendo por índice.

Solución 11

```
d = {}
d["primero"] = 1
d["segundo"] = 2
d["tercero"] = 3
while d:
    print(d.popitem())
# Salida: ('tercero', 3) ('segundo', 2) ('primero', 1) -> el más nuevo primero.
```

Explicación: popitem() elimina en orden LIFO (el último ingresado es el primero en salir).

Solución 12

```
acciones = {1: "abrir", 2: "escribir", 3: "cerrar"}
print("Historial:", acciones)
accion_deshecha = acciones.popitem()
print(f"Deshacer: {accion_deshecha[1]}")
print("Historial actualizado:", acciones)
```

Explicación: La última acción ("cerrar") se deshace y desaparece del registro.

Solución 13

```
palabras = {}
for i in range(5):
    p = input("Palabra: ")
    palabras[p] = len(p)

print("\nEn orden inverso:")
while palabras:
    par = palabras.popitem()
    print(f"{par[0]} (longitud {par[1]})")
```

Explicación: Al eliminar con popitem(), mostramos en orden contrario a la inserción.

Solución 14

```
d = {i: i**2 for i in range(1, 6)}
print("Original:", d)
d.popitem()
print("Después de popitem:", d)
```

Explicación: Se elimina el par (5, 25).

Solución 15

```
d = {"a": 1, "b": 2, "c": 3, "d": 4}
print("Sacando con clave 'b':", d.pop("b"))
print("Ahora popitem:", d.popitem())
print("Resultado:", d)
```

Explicación: `pop("b")` elimina la clave 'b' sin importar el orden; `popitem()` luego elimina el último que quedó.

Solución 16

```
letras = dict.fromkeys("abcdef", 0)
for _ in range(3):
    print("Eliminado:", letras.popitem())
print("Restantes:", letras)
```

Explicación: Se eliminan 'f', 'e', 'd' (orden inverso a la creación).

Solución 17

```
def vaciar_pila(dic):
    valores = []
    while dic:
        _, valor = dic.popitem()
        valores.append(valor)
    return valores

pila = {"a": 100, "b": 200, "c": 300}
print(vaciar_pila(pila)) # [300, 200, 100]
```

Explicación: Usamos `_` para ignorar la clave; recolectamos los valores en orden LIFO.

Solución 18

```
correos = {}
cont = 1
for asunto in ["Reunión", "Informe", "Saludo"]:
    correos[cont] = asunto
    cont += 1

print("Bandeja:", correos)
while correos:
    num, asunto = correos.popitem()
    print(f"Leyendo correo #{num}: {asunto}")
```

• **Explicación:** Simula la lectura de correos del más reciente al más viejo.

Solución 19

```
# popitem() elimina el último añadido y devuelve tupla
d = {"a": 1, "b": 2}
ultimo = d.popitem() # ('b', 2)

# pop(clave) elimina una clave específica y devuelve su valor
d = {"a": 1, "b": 2}
```

```
val = d.pop("a")          # 1
```

popitem no requiere argumento; pop sí.

Explicación: popitem() siempre saca el último; pop() saca la clave que le indiques.

Solución 20

```
carrito = {}
while True:
    print("\n1. Agregar producto")
    print("2. Vaciar carrito")
    print("3. Ver carrito")
    print("4. Salir")
    op = input("Opción: ")

    if op == "1":
        prod = input("Producto: ")
        cant = int(input("Cantidad: "))
        carrito[prod] = cant
    elif op == "2":
        while carrito:
            prod, cant = carrito.popitem()
            print(f"Eliminando {prod}: {cant} unidades")
        print("Carrito vaciado.")
    elif op == "3":
        if carrito:
            for p, c in carrito.items():
                print(f"{p}: {c}")
        else:
            print("Carrito vacío")
    elif op == "4":
        break
```

Explicación: Al vaciar el carrito, se usa popitem() para quitar de a uno y mostrarlo.

Resumen del capítulo

En este capítulo aprendiste:

- `popitem()` elimina y devuelve el **último par clave-valor** agregado al diccionario.
- Devuelve una **tupla** (`clave`, `valor`).
- Se comporta como una **pila LIFO**: último en entrar, primero en salir.
- No necesita argumentos; si el diccionario está vacío lanza `KeyError`.
- Es ideal para procesar elementos en orden inverso, deshacer acciones o vaciar una estructura paso a paso mostrando qué se quita.

popitem() es como sacar el último plato de una pila: lo sacás, lo ves y la pila se achica. El plato que sacaste es el que pusiste último.

Método `setdefault()` – Buscar o Crear en una Sola Línea

¿Qué es `setdefault()`?

Cuando empecé a trabajar con diccionarios en la UBA Exactas, había una situación que me sacaba de quicio: quería incrementar un contador para cada palabra que aparecía en un texto, pero antes tenía que verificar si la palabra ya existía en el diccionario. Era un `if` y un `else` que repetía una y otra vez. Hasta que un compañero me dijo: “Che, Nahuel, ¿no viste `setdefault()`? Te ahorra todo eso”. Y tenía razón. `setdefault()` combina búsqueda y creación en una misma línea.

El método `setdefault()` busca una clave en el diccionario. Si la clave existe, devuelve su valor **sin modificar nada**. Si la clave no existe, la **crea** con un valor por defecto que vos le indiqués (o `None` si no le pasás nada) y devuelve ese valor.

Analogía: Imaginá que le pedís a un asistente que vaya al depósito a buscar una herramienta. Si la herramienta ya está, te la alcanza. Si no está, la compra en el momento, la guarda en el depósito y te la entrega nuevita. Así funciona `setdefault()`.

```
+-----+
|
|  diccionario.setdefault(clave, valor_por_defecto)
|
|  Si la clave EXISTE -> devuelve su valor
|                    -> NO modifica nada
|
|  Si la clave NO EXISTE -> crea la clave con
|                        valor_por_defecto
|                        -> devuelve ese valor
|
+-----+
```

Sintaxis

```
diccionario.setdefault(clave, valor_por_defecto)
```

- **clave:** la clave que querés buscar o crear.
- **valor_por_defecto:** el valor que se le asignará si la clave no está. Si no ponés nada, se asigna `None`.

Las tres situaciones típicas

Veamos cómo se comporta `setdefault()` con un ejemplo de precios de frutas.

```
precios = {"manzana": 2.50, "banana": 3.00, "pera": 2.80}
print("Diccionario original:", precios)

# Escenario 1: La clave YA EXISTE
print("\n--- Clave 'banana' ya existe ---")
valor_banana = precios.setdefault("banana", 4.00)
print(f"Valor devuelto: {valor_banana}") # 3.00 (el original)
print(f"Diccionario (sin cambios): {precios}")

# Escenario 2: Clave NO existe, sin valor por defecto
print("\n--- Clave 'kiwi' no existe, sin valor ---")
valor_kiwi = precios.setdefault("kiwi")
print(f"Valor devuelto: {valor_kiwi}") # None
print(f"Diccionario actualizado: {precios}")

# Escenario 3: Clave NO existe, con valor por defecto
print("\n--- Clave 'mango' no existe, con valor ---")
valor_mango = precios.setdefault("mango", 5.00)
print(f"Valor devuelto: {valor_mango}") # 5.00
print(f"Diccionario actualizado: {precios}")
```

Salida:

```
Diccionario original: {'manzana': 2.5, 'banana': 3.0, 'pera': 2.8}
```

```
--- Clave 'banana' ya existe ---
```

```
Valor devuelto: 3.0
```

```
Diccionario (sin cambios): {'manzana': 2.5, 'banana': 3.0, 'pera': 2.8}
```

```
--- Clave 'kiwi' no existe, sin valor ---
```

```
Valor devuelto: None
```

```
Diccionario actualizado: {'manzana': 2.5, 'banana': 3.0, 'pera': 2.8, 'kiwi': None}
```

```
--- Clave 'mango' no existe, con valor ---
```

```
Valor devuelto: 5.0
```

```
Diccionario actualizado: {'manzana': 2.5, 'banana': 3.0, 'pera': 2.8, 'kiwi': None, 'mango': 5.0}
```

Tabla resumen de comportamientos

ESCAPENARIO	CLAVE EXISTE?	VALOR QUE LE PASAMOS	RESULTADO
Clave existe	SI	Cualquiera	Devuelve el valor original.
			NO se modifica.

Clave NO existe (sin valor)	NO	Ninguno (None)	Crea clave con None. Devuelve None.
Clave NO existe (con valor)	NO	Valor específico	Crea clave con ese valor. Devuelve ese valor.

Ejemplo práctico: Contar frecuencias

Uno de los usos más comunes de `setdefault()` es contar elementos sin preocuparse por inicializar el contador.

```
datos = ["manzana", "pera", "manzana", "uva", "pera", "manzana", "banana"]
conteo = {}
```

```
for elemento in datos:
    conteo[elemento] = conteo.setdefault(elemento, 0) + 1
```

```
print("Resultado del conteo:")
for clave, valor in conteo.items():
    print(f"{clave}: {valor} veces")
```

```
manzana: 3 veces
pera: 2 veces
uva: 1 veces
banana: 1 veces
```

Reflexión: Sin `setdefault()`, tendrías que hacer `if elemento in conteo: ... else: ...`. Ahora lo resuelves en una línea.

Ejemplo práctico: Agrupar elementos por categoría

`setdefault()` es genial para crear listas de cosas agrupadas por algún criterio.

```
• productos = [
    ("manzana", "fruta"),
    ("pera", "fruta"),
    ("zanahoria", "verdura"),
    ("banana", "fruta"),
    ("lechuga", "verdura")
]
```

```
agrupados = {}
```

```
for producto, categoria in productos:
    agrupados.setdefault(categoria, []).append(producto)

print("Productos agrupados:")
for cat, lista in agrupados.items():
    print(f"{cat}: {lista}")

fruta: ['manzana', 'pera', 'banana']
verdura: ['zanahoria', 'lechuga']
```

Ejercicios propuestos

Ahora es tu turno. Practicá con estos 20 ejercicios. Los primeros 10 son los que ya conocés, y los últimos 10 te van a desafiar un poco más.

1. Creá un diccionario con 3 elementos. Usá `setdefault()` con una clave que YA EXISTE. ¿Qué devuelve? ¿Se modifica el diccionario?
2. Creá un diccionario con 3 elementos. Usá `setdefault()` con una clave que NO EXISTE y sin valor por defecto. ¿Qué devuelve? ¿Qué pasa con el diccionario?
3. Creá un diccionario vacío. Usá `setdefault()` con una clave que no existe y valor 100. Mostrá el diccionario resultante.
4. Usá `setdefault()` para contar frecuencias de las letras en la palabra “programacion”.
5. Creá un diccionario. Usá `setdefault()` para agregar una lista vacía como valor de una clave que no existe.
6. ¿Qué diferencia hay entre `dic.setdefault(".", 5)` y `dic["."] = 5`? Probá con una clave que ya existe y explicá.
7. Creá un diccionario de productos. Usá `setdefault()` para asegurar que un producto tenga precio por defecto 0 si no existe.
8. Usá `setdefault()` para agrupar una lista de palabras por su primera letra.
9. Creá un diccionario de notas. Usá `setdefault()` para agregar un alumno nuevo solo si no existe.
10. Compará el código necesario para contar frecuencias con `if` vs con `setdefault()`. ¿Cuál es más corto? Escribí ambos.
11. Tenés un diccionario `stock = "lapiceras": 10, "cuadernos": 5`. Usá `setdefault()` para intentar obtener el stock de “gomas”; si no existe, que se agregue con valor 15. Luego mostrá el diccionario actualizado.
12. Ingresá una frase por teclado y contá la frecuencia de cada palabra usando `setdefault()`. Mostrá el resultado.
13. Tenés una lista de estudiantes y el curso al que pertenecen: `estudiantes = [(".na", "1A"), ("Luis", "1B"), ("Marta", "1A")]`. Agrupalos por curso usando `setdefault()` y mostrá el diccionario resultante.

14. Escribí una función `contar_letras(texto)` que devuelva un diccionario con la frecuencia de cada letra (solo alfabéticas) usando `setdefault()`.
15. Creá un diccionario `config = {"debug": True, "version": "1.0"}`.
Usá `setdefault()` para obtener el valor de "timeout", y si no existe, establecelo en 30.
16. Simulá un carrito de compras donde el usuario ingresa producto y cantidad. Si el producto ya está en el carrito, ignorá la nueva cantidad (no la pises). Usá `setdefault()` para agregar solo si no existía.
17. Tenés una lista de direcciones web con protocolo: `["http://python.org", "https://google.com", "http://example.com"]`. Usá `setdefault()` para crear un diccionario que agrupe las URLs por su protocolo (http o https).
18. Escribí un programa que lea palabras de una lista y construya un diccionario de sinónimos usando `setdefault()`: cada palabra es clave y su valor es una lista de sinónimos que se van agregando. Partí de una lista de pares (`palabra, sinonimo`).
19. Usá `setdefault()` para construir un grafo simple: tenés una lista de aristas `aristas = [(.A, "B"), (.A, "C"), ("B", "C")]` y querés un diccionario donde cada nodo tenga una lista de nodos adyacentes.
20. Implementá un sistema de puntuaciones para un juego: cuando un jugador hace un punto, se incrementa su puntuación. Si el jugador no existía, se lo agrega con 1 punto. Usá `setdefault()` para hacerlo en una línea.

Soluciones paso a paso

Solución 1

```
dic = {"a": 10, "b": 20, "c": 30}
valor = dic.setdefault("b", 999)
print(f"Valor devuelto: {valor}") # 20
print(f"Diccionario: {dic}") # {'a': 10, 'b': 20, 'c': 30} (sin cambios)
```

Explicación: Como “b” ya existía, se devuelve su valor original y el diccionario no se modifica.

Solución 2

```
dic = {"a": 1, "b": 2, "c": 3}
valor = dic.setdefault("d")
print(f"Valor devuelto: {valor}") # None
print(f"Diccionario: {dic}") # {'a': 1, 'b': 2, 'c': 3, 'd': None}
```

Explicación: Al no existir “d”, se crea con None y se devuelve None.

Solución 3

```
dic = {}
valor = dic.setdefault("clave_nueva", 100)
print(f"Valor devuelto: {valor}") # 100
print(f"Diccionario: {dic}") # {'clave_nueva': 100}
```

Explicación: Se crea la clave y se le asigna el valor indicado.

Solución 4

```
palabra = "programacion"
conteo = {}
for letra in palabra:
    conteo[letra] = conteo.setdefault(letra, 0) + 1
print(conteo)
```

Explicación: Cada letra se inicializa en 0 la primera vez y luego se incrementa.

Solución 5

```
dic = {}
dic.setdefault("lista_vacia", [])
print(dic) # {'lista_vacia': []}
dic["lista_vacia"].append(1)
print(dic) # {'lista_vacia': [1]}
```

Explicación: Se crea la clave con una lista vacía, que luego podemos modificar porque es una referencia.

Solución 6

```
dic = {"a": 10}
# setdefault() NO modifica si la clave ya existe
v1 = dic.setdefault("a", 999) # devuelve 10
# Asignación directa MODIFICA aunque ya exista
dic["a"] = 999 # ahora "a" vale 999
print(dic) # {'a': 999}
```

Explicación: setdefault() protege el valor existente; la asignación directa lo pisa.

Solución 7

```
productos = {"pan": 70, "leche": 90}
precio = productos.setdefault("huevos", 0)
print(f"Precio de huevos: {precio}") # 0
print(productos) # {'pan': 70, 'leche': 90, 'huevos': 0}
```

Explicación: Si el producto no existe, se agrega con precio 0.

Solución 8

```
palabras = ["auto", "avion", "casa", "arbol", "coche"]
agrupadas = {}
for palabra in palabras:
    agrupadas.setdefault(palabra[0], []).append(palabra)
print(agrupadas) # {'a': ['auto', 'avion', 'arbol'], 'c': ['casa', 'coche']}
```

Explicación: Se crea una lista por cada primera letra y se agrega la palabra.

Solución 9

```
notas = {"Ana": 8, "Luis": 7}
notas.setdefault("Pedro", 6)
print(notas) # {'Ana': 8, 'Luis': 7, 'Pedro': 6}
# Si ya existía, no lo toca
notas.setdefault("Ana", 10)
print(notas) # {'Ana': 8, 'Luis': 7, 'Pedro': 6}
```

Explicación: Solo se agrega si es nuevo.

Solución 10

```
# Con if (más largo)
conteo1 = {}
for letra in "python":
    if letra in conteo1:
        conteo1[letra] += 1
    else:
        conteo1[letra] = 1
```

```
# Con setdefault() (más corto)
conteo2 = {}
for letra in "python":
    conteo2[letra] = conteo2.setdefault(letra, 0) + 1

print("Con if:", conteo1)
print("Con setdefault:", conteo2)
```

Explicación: setdefault() reduce 4 líneas a 1.

Solución 11

```
stock = {"lapiceras": 10, "cuadernos": 5}
cant = stock.setdefault("gomas", 15)
print(f"Stock de gomas: {cant}") # 15
print(stock) # {'lapiceras': 10, 'cuadernos': 5, 'gomas': 15}
```

Explicación: Como “gomas” no existía, se agrega con 15.

Solución 12

```
frase = input("Frase: ").lower().split()
frec = {}
for palabra in frase:
    frec[palabra] = frec.setdefault(palabra, 0) + 1
print("Frecuencia de palabras:")
for palabra, cant in frec.items():
    print(f"{palabra}: {cant}")
```

Explicación: Separamos la frase en palabras y contamos cada una.

Solución 13

```
estudiantes = [("Ana","1A"), ("Luis","1B"), ("Marta","1A")]
cursos = {}
for nombre, curso in estudiantes:
    cursos.setdefault(curso, []).append(nombre)
print(cursos) # {'1A': ['Ana', 'Marta'], '1B': ['Luis']}
```

Explicación: Agrupamos estudiantes por su curso.

Solución 14

```
def contar_letras(texto):
    cont = {}
    for car in texto.lower():
        if car.isalpha():
            cont[car] = cont.setdefault(car, 0) + 1
```



```

    return cont

print(contar_letras("Hola Mundo!"))

```

Explicación: Filtramos solo letras y contamos con `setdefault()`.

Solución 15

```

config = {"debug": True, "version": "1.0"}
timeout = config.setdefault("timeout", 30)
print(f"Timeout: {timeout}") # 30
print(config) # {'debug': True, 'version': '1.0', 'timeout': 30}

```

Explicación: Agrega un valor por defecto para una configuración faltante.

Solución 16

```

carrito = {}
while True:
    prod = input("Producto (enter para terminar): ")
    if prod == "":
        break
    cant = int(input("Cantidad: "))
    # Solo agrega si no existe, ignora si ya está
    carrito.setdefault(prod, cant)
    print("Carrito:", carrito)

```

Explicación: Si el producto ya existe, `setdefault()` no modifica el valor.

Solución 17

```

urls = ["http://python.org", "https://google.com", "http://example.com"]
por_protocolo = {}
for url in urls:
    if url.startswith("https"):
        por_protocolo.setdefault("https", []).append(url)
    else:
        por_protocolo.setdefault("http", []).append(url)
print(por_protocolo)

```

Explicación: Separamos las URLs según su protocolo.

Solución 18

```

pares = [("grande", "enorme"), ("rápido", "veloz"), ("grande", "gigante")]
sinonimos = {}
for palabra, sinonimo in pares:
    sinonimos.setdefault(palabra, []).append(sinonimo)
print(sinonimos) # {'grande': ['enorme', 'gigante'], 'rápido': ['veloz']}

```

Explicación: Cada palabra es clave, y su lista de sinónimos se va llenando.

Solución 19

```
aristas = [("A","B"), ("A","C"), ("B","C")]
grafo = {}
for origen, destino in aristas:
    grafo.setdefault(origen, []).append(destino)
    grafo.setdefault(destino, []).append(origen) # no dirigido
print(grafo)
```

Explicación: Para cada arista, agregamos ambos extremos mutuamente.

Solución 20

```
puntuaciones = {}
# Simulación de puntos de jugadores
secuencia = ["Ana", "Luis", "Ana", "Pedro", "Luis", "Ana"]
for jugador in secuencia:
    puntuaciones[jugador] = puntuaciones.setdefault(jugador, 0) + 1
print(puntuaciones) # {'Ana': 3, 'Luis': 2, 'Pedro': 1}
```

Explicación: Cada jugador se inicializa en 0 la primera vez y se incrementa.

Resumen del capítulo

En este capítulo aprendiste:

- `setdefault()` busca una clave y, si no existe, la crea con un valor por defecto.
- **Si la clave existe:** devuelve el valor actual y no modifica el diccionario.
- **Si la clave no existe:** la agrega con el valor indicado (o `None`) y lo devuelve.
- Es una **red de seguridad** que evita comprobar con `if` si la clave está.
- Ideal para contadores, agrupaciones, inicialización de configuraciones y grafos.
- A diferencia de la asignación directa, `setdefault()` no pisa valores existentes.

setdefault() es como una red de seguridad: “Si no está, ponelo; si ya está, no me rompas nada y decime qué tiene”. Es de los métodos más prácticos para ahorrarte un par de if.

Método update() – Actualizar Diccio- rios de Forma Masiva

¿Para que sirve update()?

Cuando estaba dando mis primeros pasos en Python, recuerdo que tenía dos listas de productos con precios y quería juntarlas en un solo diccionario. Al principio hacía un bucle `for` para ir agregando uno por uno, pero cuando se cruzaban productos repetidos, terminaba sobrescribiendo sin querer o dejando duplicados. Un compañero de la UBA Exactas me dijo: “Che, ¿no viste `update()`? Te fusiona todo en una sola línea y maneja las repeticiones como esperas”. Lo probé y me ahorré diez líneas de código.

El método `update()` es una herramienta clave para trabajar con diccionarios. En criollo, sirve para **agregar** y **modificar** varios elementos a la vez, partiendo de otro diccionario (o de pares clave-valor que le pases). Hace dos cosas al mismo tiempo: si una clave no existe, la crea; si ya existe, la actualiza con el nuevo valor. Un golazo.

```
+-----+
|
|  diccionario_original.update(otro_diccionario) |
|
|  Si la clave NO EXISTE -> AGREGA el nuevo par  |
|  Si la clave YA EXISTE -> SOBREScribe el valor |
|
|  ;Dos operaciones en una sola linea!           |
|
+-----+
```

Sintaxis

```
diccionario_original.update(otro_diccionario)
```

Podes pasarle:

- Un diccionario que ya hayas creado antes.
- Un diccionario definido directamente entre llaves dentro de los parentesis.

```
# Forma 1: con un diccionario existente
nuevos_datos = {"z": 4, "w": 5}
dic_original.update(nuevos_datos)
```

```
# Forma 2: definiendo el diccionario en el momento
dic_original.update({"z": 4, "w": 5})
```

Analogía: Imaginate que tienes una caja con herramientas (el diccionario original). De repente te traen una bolsa con más herramientas. `update()` vacía la bolsa dentro de la caja: si ya tenías un martillo, lo reemplaza por el nuevo; si trae una llave que no estaba, la agrega.

Comportamiento: Agregar y Actualizar

Agregar nuevos elementos

Si la clave NO existe en el diccionario original, `update()` la **agrega** con su valor.

```
# Diccionario original
dic = {"a": 1, "b": 2, "c": 3}
print("Original:", dic)

# Agregamos una clave que no existe
dic.update({"z": 4})
print("Después de agregar 'z':", dic)
```

```
# Agregamos varias claves que no existen
dic.update({"x": 10, "y": 11})
print("Después de agregar 'x' e 'y':", dic)
```

Salida:

```
Original: {'a': 1, 'b': 2, 'c': 3}
Después de agregar 'z': {'a': 1, 'b': 2, 'c': 3, 'z': 4}
Después de agregar 'x' e 'y': {'a': 1, 'b': 2, 'c': 3, 'z': 4, 'x': 10, 'y': 11}
```

Actualizar elementos existentes

Si la clave YA EXISTE, `update()` la **sobrescribe** con el nuevo valor.

```
# Diccionario original
dic = {"a": 1, "b": 2, "c": 3}
print("Original:", dic)

# Actualizamos una clave que ya existe
dic.update({"a": 99})
print("Después de actualizar 'a':", dic)

# Actualizamos varias claves existentes
dic.update({"b": 88, "c": 77})
print("Después de actualizar 'b' y 'c':", dic)
```

Salida:

```
Original: {'a': 1, 'b': 2, 'c': 3}
Después de actualizar 'a': {'a': 99, 'b': 2, 'c': 3}
Después de actualizar 'b' y 'c': {'a': 99, 'b': 88, 'c': 77}
```

Ejemplo completo

```
# Diccionario inicial
datos = {"a": 1, "b": 2, "c": 3}
print("=== DICCIONARIO ORIGINAL ===")
print(datos)

# Ejemplo 1: Agregar (clave no existe)
print("\n--- AGREGANDO 'z': 4 ---")
datos.update({"z": 4})
print(datos)

# Ejemplo 2: Actualizar (clave existe)
print("\n--- ACTUALIZANDO 'a': 6 ---")
datos.update({"a": 6})
print(datos)

# Ejemplo 3: Múltiples operaciones juntas
print("\n--- MÚLTIPLES: 'b': 10, 'c': 20, 'w': 30 ---")
datos.update({"b": 10, "c": 20, "w": 30})
print(datos)
```

Ejemplo práctico: Configuración de parámetros

En la facultad, cuando hacía experimentos, tenía un diccionario con parámetros base (temperatura, presión, etc.) y quería probar variantes sin perder los originales. Con `update()` lo hacía en un segundo.

```
parametros = {
    "temperatura": 25,
    "presion": 1013,
    "volumen": 10,
    "masa": 50,
    "densidad": 2.5
}

print("=== PARAMETROS BASE ===")
for clave, valor in parametros.items():
    print(f"{clave}: {valor}")

# Modificamos solo algunos
parametros.update({"temperatura": 30, "presion": 1020})
# Agregamos uno nuevo
parametros.update({"conductividad": 0.8})

print("\n=== PARAMETROS ACTUALIZADOS ===")
for clave, valor in parametros.items():
    print(f"{clave}: {valor}")
```

Ejemplo practico: Combinar dos diccionarios de ventas

```
enero = {"manzanas": 100, "peras": 80, "naranjas": 60}
febrero = {"peras": 90, "naranjas": 70, "uvas": 50}

print("Ventas enero:", enero)
print("Ventas febrero:", febrero)

# Fusionamos febrero dentro de enero
enero.update(febrero)
print("Ventas acumuladas:", enero)
```

Nota: Las claves que ya existían (*peras*, *naranjas*) se *sobrescribieron*. Las nuevas (*uvas*) se *agregaron*. El resto quedo intacto.

Ejemplo practico: Actualizacion masiva de notas

```
notas = {"Ana": 8, "Luis": 7, "Pedro": 9, "Marta": 6}
print("Notas originales:", notas)

# Despues del recuperatorio
notas.update({"Luis": 8, "Marta": 7, "Juan": 9})

print("Notas actualizadas:", notas)
```

Unir diccionarios sin modificar el original

Si quieres conservar el diccionario original, primero hace una copia.

```
dic1 = {"a": 1, "b": 2}
dic2 = {"c": 3, "d": 4, "a": 99}

union = dic1.copy()
union.update(dic2)

print("dic1 original:", dic1)
print("dic2:", dic2)
print("Union:", union)
```

Ejercicios propuestos

1. Crea un diccionario con 3 elementos. Usa `update()` para agregar un nuevo par clave-valor.
2. Crea un diccionario con 3 elementos. Usa `update()` para modificar el valor de una clave existente.

3. Crea un diccionario. Usa `update()` con un diccionario que tenga 2 claves nuevas y 1 clave existente. Muestra el resultado.
4. Crea dos diccionarios y combinalos usando `update()`. Muestra el diccionario resultante.
5. Crea un diccionario de configuracion. Usa `update()` para cambiar varios parametros a la vez.
6. Crea un diccionario de notas. Usa `update()` para actualizar las notas de 3 alumnos y agregar 1 alumno nuevo.
7. Crea un diccionario de inventario. Usa `update()` para agregar nuevos productos y actualizar cantidades de otros.
8. Crea un diccionario de empleados. Usa `update()` para modificar el salario de 2 empleados y agregar 2 nuevos.
9. ¿Que pasa si usas `update()` con un diccionario vacio? Probalo.
10. Crea un diccionario. Usa `update()` para agregar multiples claves definiendo un diccionario directamente dentro del metodo.
11. Tienes un diccionario `stock = {"lapiceras": 10, "cuadernos": 5}`. Usa `update()` para modificar el stock de "lapiceras" a 25 y agregar "gomas" con 15 unidades.
12. Dos diccionarios representan las compras de dos clientes distintos. Quieres unificar las compras en un solo diccionario, sumando las cantidades si un producto aparece en ambos. Escribe un programa que lo haga sin perder datos (pista: no uses solo `update()`, combina con `get` o `setdefault`).
13. ¿Se puede usar `update()` con un solo par clave-valor pasandolo como tupla o lista? Investiga y prueba con `mi_dic.update([(".", 1)])` (lista de tuplas) o `mi_dic.update((".", 1))` (tupla). Conta que sucede.
14. Escribe una funcion `combinar_diccionarios(d1, d2)` que devuelva un nuevo diccionario con la union de ambos, sin modificar los originales. Usa `update()` adentro.
15. Crea un diccionario `config = {"debug": False, "version": "0.5"}`. Luego defini otro diccionario `defaults = {"debug": True, "timeout": 30, "version": "1.0"}`. Quieres construir una configuracion final donde los valores de `config` tengan prioridad sobre los `defaults`. Usa `update()` de manera adecuada (primero copia los `defaults` y luego actualiza con los valores especificos).
16. Ingresa 3 palabras y sus longitudes en un diccionario. Luego, usando `update()`, agrega una cuarta palabra y modifica la longitud de una de las que ya estaban. Muestra el antes y el despues.
17. Simula un carrito de compras que se actualiza desde un formulario web. El carrito inicial es `{"mouse": 2, "teclado": 1}`. Llega un diccionario con la compra actualizada: `{"mouse": 3, "monitor": 1}`. Aplica `update()` y muestra el carrito resultante.
18. ¿Que diferencia hay entre `d1.update(d2)` y `d1 = d2`? Explica con un ejemplo. ¿Cual modifica el diccionario original y cual reemplaza la variable?

19. Escribi un programa que pida al usuario que ingrese 3 productos y sus precios en un diccionario. Despues, usando `update()`, aumenta un 10% a los precios de los productos que ya existen (crea un diccionario con los precios aumentados y actualiza). Mostra el resultado final.
20. (Desafio) Tienes un diccionario de materias con el profesor a cargo. Quieres actualizar varios profesores a la vez, pero solo si la materia existe; si no, no debe agregarse. Por ejemplo: `materias = {"Mate": "Garcia", "Fisica": "Lopez"}`. Si intentas `update({"Mate": "Rodriguez", "Quimica": "Perez"})`, solo debe cambiar Mate y no agregar Quimica. Implementa una funcion que haga esto usando `update()` y alguna verificacion.

Soluciones paso a paso

```
1. dic = {"a": 1, "b": 2, "c": 3}
   dic.update({"d": 4})
   print(dic) # {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Explicacion: La clave 'd' no existia, asi que se agrego al final.

```
2. dic = {"a": 1, "b": 2, "c": 3}
   dic.update({"b": 99})
   print(dic) # {'a': 1, 'b': 99, 'c': 3}
```

Explicacion: 'b' ya estaba, entonces su valor se actualiza.

```
3. dic = {"a": 1, "b": 2, "c": 3}
   dic.update({"b": 20, "d": 4, "e": 5})
   print(dic) # {'a': 1, 'b': 20, 'c': 3, 'd': 4, 'e': 5}
```

Explicacion: Mezcla de actualizacion y agregado.

```
4. dic1 = {"a": 1, "b": 2}
   dic2 = {"c": 3, "d": 4}
   dic1.update(dic2)
   print(dic1) # {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Explicacion: Fusiona dic2 dentro de dic1.

```
5. config = {"tema": "claro", "fuente": 12, "idioma": "es"}
   config.update({"tema": "oscuro", "fuente": 14})
   print(config) # {'tema': 'oscuro', 'fuente': 14, 'idioma': 'es'}
```

Explicacion: Actualiza las claves indicadas.

```
6. notas = {"Ana": 8, "Luis": 7, "Pedro": 9}
   notas.update({"Luis": 8, "Pedro": 10, "Marta": 9})
   print(notas) # {'Ana': 8, 'Luis': 8, 'Pedro': 10, 'Marta': 9}
```

Explicacion: Luis y Pedro se actualizan, Marta se agrega.

```
7. inventario = {"manzanas": 50, "peras": 30}
   inventario.update({"peras": 40, "naranjas": 25, "uvas": 15})
   print(inventario) # {'manzanas': 50, 'peras': 40, 'naranjas': 25, 'uvas': 15}
```

Explicacion: "peras" se actualiza, los demas se agregan.

```
8. empleados = {"Ana": 50000, "Luis": 60000}
   empleados.update({"Luis": 65000, "Pedro": 55000, "Marta": 70000})
   print(empleados) # {'Ana': 50000, 'Luis': 65000, 'Pedro': 55000, 'Marta': 70000}
```

Explicacion: Similar al anterior.

```
9. dic = {"a": 1, "b": 2}
   dic.update({})
   print(dic) # {'a': 1, 'b': 2} (sin cambios)
```

Explicacion: Un diccionario vacio no modifica nada.

```
10. dic = {"a": 1}
    dic.update({"b": 2, "c": 3, "d": 4})
    print(dic) # {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Explicacion: Se pueden pasar varios pares directamente.

```
11. stock = {"lapiceras": 10, "cuadernos": 5}
    stock.update({"lapiceras": 25, "gomas": 15})
    print(stock) # {'lapiceras': 25, 'cuadernos': 5, 'gomas': 15}
```

Explicacion: “lapiceras” se actualiza, “gomas” se agrega.

```
12. compra1 = {"pan": 3, "leche": 2}
    compra2 = {"leche": 1, "huevos": 6}
    for producto, cantidad in compra2.items():
        compra1[producto] = compra1.get(producto, 0) + cantidad
    print(compra1) # {'pan': 3, 'leche': 3, 'huevos': 6}
```

Explicacion: Usamos get() para sumar cantidades en lugar de sobrescribir.

```
13. d = {"x": 5}
    d.update([("a", 1), ("b", 2)]) # Funciona con lista de tuplas
    print(d) # {'x': 5, 'a': 1, 'b': 2}
    d.update(("c", 3)) # Esto dara error: ValueError: dictionary update sequence element # 0: La tupla debe ser de pares, por lo tanto no es valido pasar una tupla simple.
```

Explicacion: Se aceptan listas de tuplas, pero una tupla simple de dos elementos no es un iterable de pares; se necesita una tupla de tuplas.

```
14. def combinar_diccionarios(d1, d2):
    resultado = d1.copy()
    resultado.update(d2)
    return resultado

a = {1: "a", 2: "b"}
b = {2: "c", 3: "d"}
print(combinar_diccionarios(a, b)) # {1: 'a', 2: 'c', 3: 'd'}
print(a, b) # sin cambios
```

Explicacion: Copiamos el primero y luego actualizamos.

```
15. config = {"debug": False, "version": "0.5"}
    defaults = {"debug": True, "timeout": 30, "version": "1.0"}
    final = defaults.copy()
    final.update(config)
    print(final) # {'debug': False, 'timeout': 30, 'version': '0.5'}
```

Explicacion: Los valores de config tienen prioridad.

```
16. palabras = {}
    for i in range(3):
        p = input("Palabra: ")
        palabras[p] = len(p)
    print("Antes:", palabras)
```

```
palabras.update({"python": 6})
palabras.update({"a": 1}) # modifica si ya existia
print("Despues:", palabras)
```

Explicacion: Agregamos y modificamos usando update().

```
17. carrito = {"mouse": 2, "teclado": 1}
nuevo = {"mouse": 3, "monitor": 1}
carrito.update(nuevo)
print(carrito) # {'mouse': 3, 'teclado': 1, 'monitor': 1}
```

Explicacion: "mouse" se actualiza y "monitor" se agrega.

```
18. d1 = {"a": 1, "b": 2}
d2 = {"a": 100, "c": 3}
d1.update(d2) # Modifica d1 agregando/actualizando
print(d1) # {'a': 100, 'b': 2, 'c': 3}

d1 = {"a": 1, "b": 2}
d1 = d2 # Reemplaza la referencia, d1 ahora apunta al mismo objeto que d2
print(d1) # {'a': 100, 'c': 3} (d1 original se pierde)
```

Explicacion: update() modifica el contenido del diccionario existente; la asignacion = cambia la variable para que referencie a otro diccionario.

```
19. productos = {}
for i in range(3):
    nom = input(f"Producto {i+1}: ")
    precio = float(input("Precio: "))
    productos[nom] = precio

print("Original:", productos)

aumento = {prod: prec * 1.10 for prod, prec in productos.items()}
productos.update(aumento)
print("Con aumento 10%:", productos)
```

Explicacion: Creamos un diccionario con los precios incrementados y actualizamos.

```
20. def actualizar_solo_existentes(dic, cambios):
    # Filtramos las claves que ya existen en dic
    claves_existentes = {k: v for k, v in cambios.items() if k in dic}
    dic.update(claves_existentes)
```

```
materias = {"Mate": "Garcia", "Fisica": "Lopez"}
cambios = {"Mate": "Rodriguez", "Quimica": "Perez"}
actualizar_solo_existentes(materias, cambios)
print(materias) # {'Mate': 'Rodriguez', 'Fisica': 'Lopez'}
```

Explicacion: Creamos un diccionario intermedio solo con las claves que ya estaban, y luego aplicamos update().

Resumen del capítulo

En este capítulo aprendiste:

- `update()` actualiza un diccionario con otro: **agrega** las claves que no existen y **sobrescribe** las que ya están.
- Es una forma limpia de fusionar diccionarios o aplicar cambios masivos.
- Puedes pasarle un diccionario preexistente o definirlo directamente.
- Si quieres mantener el original, hace una copia antes.
- Combinado con `get()` o `setdefault()` puedes hacer operaciones mas complejas, como sumar valores.
- `update()` modifica el diccionario in-place, a diferencia de una asignación directa que cambia la referencia.

update() es como un asistente que recibe una lista de cambios: los que son nuevos los agrega, los que ya existen los actualiza. Todo al mismo tiempo.

Recorrer Diccionarios con el Ciclo for

¿Por que necesitamos recorrer diccionarios?

Cuando trabajas con diccionarios, muchas veces tenes que procesar **todos** los elementos que contienen. Por ejemplo, imaginate que tenes un diccionario con las masas atomicas de 100 elementos quimicos. No vas a escribir `dic["Hidrogeno"]`, `dic["Helio"]`, etc., uno por uno. ¡Para eso estan los bucles!

Como decia Nahuel en la facultad (UBA Exactas): "Recorrer un diccionario con un `for` es como tener un robot que mira cajon por cajon, te dice lo que hay adentro y hace algo con eso automaticamente."

```
+-----+
|
|  DICCIONARIO GRANDE
|  +-----+-----+-----+
|  | clave1   | clave2   | ... |
|  +-----+-----+-----+
|  | valor1   | valor2   | ... |
|  +-----+-----+-----+
|
|  for clave in diccionario:
|      # procesar cada elemento
|
+-----+
```

Alternativa 1: Recorrer solo las claves

En esta opcion, el ciclo `for` toma por defecto **cada una de las claves** del diccionario. Para obtener el valor, tenes que llamar al diccionario dentro del `print` (o hacer la operacion que necesites).

Ejemplo basico

```
# Diccionario de ejemplo
diccionario = {"a": 1, "b": 2, "c": 3}
print("=== RECORRIENDO SOLO CLAVES ===")

for clave in diccionario:
    valor = diccionario[clave]
    print(f"Clave: {clave} - Valor: {valor}")
```

Salida:

```
=== RECORRIENDO SOLO CLAVES ===
```

```
Clave: a - Valor: 1
```

```
Clave: b - Valor: 2
```

```
Clave: c - Valor: 3
```

Pros y contras

- **Pros:** Es la forma mas basica y directa. Facil de entender para alguien que recién empieza.
- **Contras:** Tienes que hacer un paso extra (la consulta `diccionario[clave]`) para ver el valor. Es menos legible que la otra opcion.

Ejemplo practico: Masas atomicas (con claves)

Imagina que tienes un diccionario con las masas atomicas de elementos quimicos.

```
# Diccionario de masas atomicas (valores aproximados)
```

```
masas = {
    "Hidrogeno": 1.008,
    "Helio": 4.0026,
    "Litio": 6.94,
    "Berilio": 9.012,
    "Boro": 10.81
}
```

```
print("=== MASA ATOMICA DE ELEMENTOS ===")
for elemento in masas:
    masa = masas[elemento]
    print(f"{elemento}: {masa} u")
```

Salida:

```
=== MASA ATOMICA DE ELEMENTOS ===
```

```
Hidrogeno: 1.008 u
```

```
Helio: 4.0026 u
```

```
Litio: 6.94 u
```

```
Berilio: 9.012 u
```

```
Boro: 10.81 u
```

Alternativa 2: Usar el metodo `.items()` (La mas elegante)

Esta es la opcion **recomendada** porque es mas “Pythonic” (asi se le dice a la forma elegante y natural de escribir codigo en Python).

El metodo `.items()` te devuelve **tanto la clave como el valor** al mismo tiempo, y el ciclo `for` los asigna a dos variables distintas en cada vuelta.

Ejemplo basico

```
print("=== RECORRIENDO CON .items() ===")
for clave, valor in diccionario.items():
    print(f"Clave: {clave} - Valor: {valor}")
```

Salida:

```
=== RECORRIENDO CON .items() ===
Clave: a - Valor: 1
Clave: b - Valor: 2
Clave: c - Valor: 3
```

Pros y contras

- **Pros:** Es mas legible, mas eficiente y demuestra que manejas bien el lenguaje. No necesitas hacer la consulta extra `diccionario[clave]`.
- **Contras:** Ninguna, ¡es la que deberias tratar de usar siempre!

Nota: *Regla de oro:* Cuando necesites recorrer un diccionario y necesitas tanto la clave como el valor, usa `.items()`. Tu codigo va a ser mas claro y facil de leer.

Ejemplo practico: Masa total de una muestra (con `.items()`)

```
# Diccionario de elementos y sus masas atomicas
elementos = {
    "Hidrogeno": 1.008,
    "Helio": 4.0026,
    "Litio": 6.94,
    "Berilio": 9.012,
    "Boro": 10.81
}

# Cantidad de cada elemento en la muestra (en moles)
cantidades = {
    "Hidrogeno": 2,
    "Helio": 1,
    "Litio": 0.5,
    "Berilio": 0,
    "Boro": 1.5
}

print("=== CALCULO DE MASA TOTAL ===")
masa_total = 0

for elemento, masa in elementos.items():
    cantidad = cantidades[elemento]
    masa_parcial = masa * cantidad
```



```

masa_total = masa_total + masa_parcial
if cantidad > 0:
    print(f"{elemento}: {cantidad} mol x {masa} u = {masa_parcial:.4f} u")

print(f"\nMasa total de la muestra: {masa_total:.4f} u")

Salida:
=== CALCULO DE MASA TOTAL ===
Hidrogeno: 2 mol x 1.008 u = 2.0160 u
Helio: 1 mol x 4.0026 u = 4.0026 u
Litio: 0.5 mol x 6.94 u = 3.4700 u
Boro: 1.5 mol x 10.81 u = 16.2150 u

Masa total de la muestra: 25.7036 u

```

Comparativa visual de los dos metodos

ALTERNATIVA	CODIGO
Recorrer claves (basico)	<pre> for clave in diccionario: valor = diccionario[clave] print(clave, valor) </pre>
Recorrer items (recomendado)	<pre> for clave, valor in diccionario.items(): print(clave, valor) </pre>

Nota: ¿Notas la diferencia? En el segundo caso, Python te da la clave Y el valor en la misma linea. No tienes que hacer la consulta extra. ¡Mucho mas limpio!

Ejemplo practico: Filtrado de elementos pesados

Imagina que tienes un diccionario con elementos quimicos y sus masas atomicas, y quieres filtrar solo los que pesan mas de 9 u.

```

masas = {
    "Hidrogeno": 1.008,
    "Helio": 4.0026,
    "Litio": 6.94,
    "Berilio": 9.012,
    "Boro": 10.81,
    "Carbono": 12.011,
    "Nitrogeno": 14.007,
    "Oxigeno": 15.999
}

print("=== ELEMENTOS CON MASA > 9 u ===")

```

```
for elemento, masa in masas.items():
    if masa > 9:
        print(f"{elemento}: {masa} u")
```

Salida:

```
=== ELEMENTOS CON MASA > 9 u ===
```

Berilio: 9.012 u

Boro: 10.81 u

Carbono: 12.011 u

Nitrogeno: 14.007 u

Oxigeno: 15.999 u

Ejemplo practico: Calcular el elemento mas pesado

```
masas = {
    "Hidrogeno": 1.008,
    "Helio": 4.0026,
    "Litio": 6.94,
    "Berilio": 9.012,
    "Boro": 10.81,
    "Carbono": 12.011
}
```

```
elemento_max = None
masa_max = 0
```

```
for elemento, masa in masas.items():
    if masa > masa_max:
        masa_max = masa
        elemento_max = elemento
```

```
print(f"El elemento mas pesado es {elemento_max} con {masa_max} u")
```

Salida:

El elemento mas pesado es Carbono con 12.011 u

¿Que pasa si solo quiero recorrer los valores?

Si solo te interesan los valores y no las claves, puedes usar `.values()`.

```
print("=== SOLO LOS VALORES (.values()) ===")
for valor in diccionario.values():
    print(f"Valor: {valor}")
```

Salida:

```
=== SOLO LOS VALORES (.values()) ===
```

Valor: 1

Valor: 2

Valor: 3

Ejercicios Propuestos

Ejercicio 1

Crea un diccionario con 3 países y sus capitales. Recorrela con un `for` usando `.items()` y muestra "La capital de [país] es [capital]".

Ejercicio 2

Crea un diccionario con 5 alumnos y sus notas. Usa `.items()` para mostrar los que aprobaron (nota ≥ 7).

Ejercicio 3

Crea un diccionario con 4 productos y sus precios. Usa `.items()` para calcular el precio total de todos los productos.

Ejercicio 4

Crea un diccionario con las edades de 5 personas. Usa `.items()` para encontrar la persona más joven.

Ejercicio 5

Crea un diccionario con 3 ciudades y su población. Usa `.items()` para mostrar las ciudades con más de 1 millón de habitantes.

Ejercicio 6

Crea un diccionario con 4 materias y sus notas. Usa `.items()` para calcular el promedio.

Ejercicio 7

Recorre un diccionario usando solo claves (sin `.items()`). Muestra clave y valor.

Ejercicio 8

Crea un diccionario de 6 alumnos con sus notas. Usa `.items()` para contar cuántos aprobaron (nota ≥ 7).

Ejercicio 9

Crea un diccionario de productos con precios. Usa `.items()` para mostrar los productos que cuestan más de \$1000.

Ejercicio 10

Crea un diccionario con 3 elementos. Recorrela usando `.values()` y suma los valores.

Ejercicio 11

Crea un diccionario de frutas con sus precios. Usa `.items()` para encontrar la fruta mas cara y la mas barata. Muestra ambos.

Ejercicio 12

Crea un diccionario que represente el stock de una tienda. Usa `.items()` para mostrar los productos que tienen menos de 10 unidades.

Ejercicio 13

Crea un diccionario con las temperaturas de una semana (lunes a viernes). Usa `.items()` para calcular la temperatura promedio y mostrar los dias que superaron los 25 grados.

Ejercicio 14

Crea un diccionario con los nombres de 4 empleados y su sueldo. Aplica un aumento del 15 % a todos los sueldos y muestra el nuevo diccionario, pero sin modificar el original (usa comprension o una copia).

Ejercicio 15

Dado un diccionario de cursos y la cantidad de inscriptos, usa `.items()` para encontrar el curso con mas inscriptos y el que tiene menos.

Ejercicio 16

Crea un diccionario con 5 palabras como claves y sus longitudes como valores. Pide al usuario una palabra; si la palabra existe, decile su longitud; si no, agregala al diccionario con su longitud y muestra el diccionario actualizado.

Ejercicio 17

Crea un diccionario que relacione el nombre de una persona con una lista de sus hobbies. Usa `.items()` para imprimir cada persona seguida de sus hobbies.

Ejercicio 18

Con un diccionario de notas de estudiantes (donde algunos valores pueden ser listas si hay varias notas), calcula el promedio de cada estudiante usando `.items()` y muestra los aprobados (promedio ≥ 7). Asumi que cada clave tiene una lista de notas.

Ejercicio 19

Tenes un diccionario con las ventas mensuales (mes: monto). Recorrela con `.items()` y genera un diccionario nuevo con los mismos meses pero con los montos incrementados en un 20 %, solo si el mes tiene ventas superiores a 1000; para los demas, que el incremento sea del 5 %.

Ejercicio 20

Crea un diccionario de configuracion con al menos 5 opciones. Recorrela con `.items()` para imprimir cada opcion en formato “opcion = valor”. Luego, pide al usuario una opcion y un nuevo valor; si la opcion existe, actualizala; si no, agregala. Muestra la configuracion final.

Soluciones Paso a Paso

Solucion 1

```
capitales = {"Argentina": "Bs As", "Brasil": "Brasilia", "Chile": "Santiago"}
for pais, capital in capitales.items():
    print(f"La capital de {pais} es {capital}")
```

Solucion 2

```
notas = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 6, "Juan": 10}
for alumno, nota in notas.items():
    if nota >= 7:
        print(f"{alumno} aprobo con {nota}")
```

Solucion 3

```
productos = {"pan": 70, "leche": 90, "huevos": 120, "queso": 200}
total = 0
for producto, precio in productos.items():
    total = total + precio
print(f"Total: ${total}")
```

Solucion 4

```
edades = {"Ana": 22, "Luis": 25, "Pedro": 21, "Marta": 24, "Juan": 23}
mas_joven = None
edad_min = float('inf')
for nombre, edad in edades.items():
    if edad < edad_min:
        edad_min = edad
        mas_joven = nombre
print(f"La persona mas joven es {mas_joven} con {edad_min} años")
```

Solucion 5

```
ciudades = {"Bs As": 15000000, "Cordoba": 1500000, "Rosario": 1200000}
for ciudad, poblacion in ciudades.items():
    if poblacion > 1000000:
        print(f"{ciudad} tiene mas de 1M de habitantes")
```

Solucion 6

```
materias = {"Mate": 8, "Fisica": 7, "Quimica": 9, "Historia": 6}
suma = 0
for materia, nota in materias.items():
    suma = suma + nota
promedio = suma / len(materias)
print(f"Promedio: {promedio:.2f}")
```

Solucion 7

```
dic = {"a": 1, "b": 2, "c": 3}
for clave in dic:
    valor = dic[clave]
    print(f"{clave}: {valor}")
```

Solucion 8

```
alumnos = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 6, "Juan": 10, "Lucia": 7}
aprobados = 0
for alumno, nota in alumnos.items():
    if nota >= 7:
        aprobados = aprobados + 1
print(f"Aprobaron {aprobados} alumnos")
```

Solucion 9

```
productos = {"mouse": 1500, "teclado": 2500, "pendrive": 800, "monitor": 50000}
for producto, precio in productos.items():
    if precio > 1000:
        print(f"{producto}: ${precio}")
```

Solucion 10

```
dic = {"a": 1, "b": 2, "c": 3}
suma = 0
for valor in dic.values():
    suma = suma + valor
print(f"Suma: {suma}")
```

Solucion 11

```
frutas = {"manzana": 3.5, "pera": 2.8, "uva": 4.2, "banana": 2.5}
precio_max = 0
precio_min = float('inf')
fruta_cara = ""
fruta_barata = ""
for fruta, precio in frutas.items():
    if precio > precio_max:
        precio_max = precio
        fruta_cara = fruta
    if precio < precio_min:
        precio_min = precio
        fruta_barata = fruta
print(f"Mas cara: {fruta_cara} (${precio_max})")
print(f"Mas barata: {fruta_barata} (${precio_min})")
```

Solucion 12

```
stock = {"lapiceras": 5, "cuadernos": 12, "gomas": 8, "reglas": 3}
for producto, cantidad in stock.items():
    if cantidad < 10:
        print(f"{producto} tiene solo {cantidad} unidades")
```

Solucion 13

```
temps = {"lunes": 22, "martes": 27, "miercoles": 30, "jueves": 24, "viernes": 26}
suma = 0
for dia, temp in temps.items():
    suma += temp
promedio = suma / len(temps)
print(f"Temperatura promedio: {promedio:.1f}")
print("Dias con mas de 25 grados:")
for dia, temp in temps.items():
    if temp > 25:
        print(f" {dia}: {temp}")
```

Solucion 14

```
suealdos = {"Ana": 50000, "Luis": 60000, "Pedro": 55000, "Marta": 70000}
nuevos = {}
for empleado, sueldo in suealdos.items():
    nuevos[empleado] = sueldo * 1.15
print("Suealdos originales:", suealdos)
print("Con aumento del 15%:", nuevos)
```

Solucion 15

```
cursos = {"Python": 35, "Java": 42, "HTML": 28, "CSS": 15}
max_ins = 0
min_ins = float('inf')
curso_max = ""
curso_min = ""
for curso, inscriptos in cursos.items():
    if inscriptos > max_ins:
        max_ins = inscriptos
        curso_max = curso
    if inscriptos < min_ins:
        min_ins = inscriptos
        curso_min = curso
print(f"Curso con mas inscriptos: {curso_max} ({max_ins})")
print(f"Curso con menos inscriptos: {curso_min} ({min_ins})")
```

Solucion 16

```
palabras = {"hola": 4, "python": 6, "codigo": 6, "sol": 3, "luna": 4}
```



```
consulta = input("Ingresa una palabra: ").lower()
if consulta in palabras:
    print(f"Longitud de '{consulta}': {palabras[consulta]}")
else:
    palabras[consulta] = len(consulta)
    print(f"Agregada '{consulta}' con longitud {palabras[consulta]}")
    print("Diccionario actualizado:", palabras)
```

Solucion 17

```
hobbies = {
    "Nahuel": ["leer", "programar"],
    "Ana": ["pintar", "correr"],
    "Luis": ["futbol", "musica", "viajar"]
}
for persona, lista in hobbies.items():
    print(f"{persona} le gusta: {' '.join(lista)}")
```

Solucion 18

```
notas = {"Ana": [8,7,9], "Luis": [6,5], "Pedro": [9,10,8], "Marta": [7,6,7]}
for alumno, lista_notas in notas.items():
    promedio = sum(lista_notas) / len(lista_notas)
    if promedio >= 7:
        print(f"{alumno}: promedio {promedio:.2f} - Aprobado")
    else:
        print(f"{alumno}: promedio {promedio:.2f} - Desaprobado")
```

Solucion 19

```
ventas = {"enero": 1200, "febrero": 800, "marzo": 1500, "abril": 950}
nuevas = {}
for mes, monto in ventas.items():
    if monto > 1000:
        nuevas[mes] = monto * 1.20
    else:
        nuevas[mes] = monto * 1.05
print("Ventas originales:", ventas)
print("Ventas actualizadas:", nuevas)
```

Solucion 20

```
config = {"debug": True, "version": "1.0", "timeout": 30, "host": "localhost", "puerto": 8080}
print("Configuracion actual:")
for clave, valor in config.items():
    print(f"{clave} = {valor}")

clave = input("Opcion a modificar/agregar: ")
valor_nuevo = input("Nuevo valor: ")
```

```
config[clave] = valor_nuevo
print("\nConfiguracion final:")
for clave, valor in config.items():
    print(f"{clave} = {valor}")
```

Resumen del Capitulo

En este capitulo aprendiste:

- Para recorrer un diccionario con `for`, tienes **dos formas principales**.
- **Alternativa 1 (basica)**: `for clave in diccionario`: Luego tienes que buscar el valor con `diccionario[clave]`.
- **Alternativa 2 (recomendada)**: `for clave, valor in diccionario.items()`: Te da CLAVE y VALOR al mismo tiempo. Es mas legible y “Pythonic”.
- Si solo necesitas los **valores**, usa `.values()`.
- Si solo necesitas las **claves**, usa `.keys()` o directamente `for clave in diccionario`.
- Recorrer diccionarios te permite procesar grandes volúmenes de datos en milisegundos: calcular totales, filtrar, encontrar maximos/minimos, etc.

Recorrer un diccionario con `for` es como tener un robot que revista cada cajon de un archivero y hace algo automaticamente con lo que encuentra. ¡No desperdicies tu tiempo escribiendo todo a mano!

Ejercicios Clave para Dominar los Diccionarios

Introduccion

Los diccionarios son una de las estructuras de datos mas poderosas de Python. Para dominarlos de una vez por todas, te propongo estos **10 ejercicios clave** que cubren las operaciones fundamentales: acceder, agregar, eliminar, listar, comprobar, recorrer, filtrar y actualizar.

Cuando empecé en la UBA Exactas, mi relacion con los diccionarios era de amor y odio. Me encantaba lo rapido que encontraba datos, pero siempre me olvidaba de usar `get()` en lugar de los corchetes y el programa se me caía con un `KeyError`. Hasta que un dia me senté y practique estos ejercicios una y otra vez. Desde entonces, los diccionarios son mis mejores amigos. Te juro que si los haces todos, te va a pasar lo mismo.

```
+-----+
|
|  DICcionario FRUTAS (para los ejercicios)
|
|  fruits = {
|      "manzanas": 10,
|      "peras": 15,
|      "naranjas": 20,
|      "frutillas": 12
|  }
|
+-----+
```

Ejercicio 1: Acceder a valores

Objetivo: Obtener cuantas manzanas hay en el diccionario `fruits`.

```
# Diccionario base
fruits = {
    "manzanas": 10,
    "peras": 15,
    "naranjas": 20,
    "frutillas": 12
}
```

Metodo 1 (recomendado): `fruits.get("manzanas")` – mas seguro, no da error si la clave no existe.

Metodo 2: `fruits["manzanas"]` – acceso directo con corchetes (riesgo de `KeyError`).

Solucion

```
# Metodo 1: con get() (mas seguro)
cantidad = fruits.get("manzanas")
print(f"Hay {cantidad} manzanas")
```

```
# Metodo 2: acceso directo
cantidad = fruits["manzanas"]
print(f"Hay {cantidad} manzanas")
```

Nota: Siempre que puedas, usa `get()` en lugar de los corchetes cuando no estes 100 % seguro de que la clave existe. Te ahorra dolores de cabeza.

Ejercicio 2: Agregar elementos

Objetivo: Sumar bananas, mangos y uvas al diccionario original.

Formas de hacerlo:

- **Forma directa:** `fruits["bananas"] = 5`
- **Con `setdefault()`:** `fruits.setdefault("mangos", 6)`
- **Con `update()`:** `fruits.update({"uvas": 3})`

Solucion

```
# Forma 1: asignacion directa
fruits["bananas"] = 5
```

```
# Forma 2: con setdefault()
fruits.setdefault("mangos", 6)
```

```
# Forma 3: con update()
fruits.update({"uvas": 3})
```

```
print(fruits)
```

Reflexion: Fijate que con `setdefault()` solo se agrega si la clave no existia. Si ya estaba, no la pisa. En cambio, la asignacion directa y `update()` si la sobrescriben.

Ejercicio 3: Eliminar elementos

Objetivo: Sacar las “peras” del diccionario.

Formas de hacerlo:

- **Con `del`:** `del fruits["peras"]`
- **Con `.pop()`:** `fruits.pop("peras")` (ademas devuelve el valor eliminado)

Solucion

```
# Forma 1: con del
del fruits["peras"]
print("Despues de del:", fruits)

# Forma 2: con pop() (recuperamos el valor eliminado)
# (reinicializamos el diccionario para probar)
fruits = {"manzanas": 10, "peras": 15, "naranjas": 20, "frutillas": 12}
valor_eliminado = fruits.pop("peras")
print(f"Valor eliminado: {valor_eliminado}")
print("Despues de pop():", fruits)
```

Ejercicio 4: Listar claves y valores por separado

Objetivo: Convertir las claves y los valores del diccionario en listas independientes.

Que necesitamos:

- Claves: `list(fruits.keys())`
- Valores: `list(fruits.values())`

Solucion

```
# Convertir claves a lista
claves = list(fruits.keys())
print("Claves:", claves)

# Convertir valores a lista
valores = list(fruits.values())
print("Valores:", valores)
```

Ejercicio 5: Comprobar existencia

Objetivo: Verificar si hay manzanas en el diccionario usando el operador `in` dentro de un `if`.

Solucion

```
if "manzanas" in fruits:
    print("True - Hay manzanas")
else:
    print("False - No hay manzanas")
```

Ejercicio 6: Recorrer el diccionario con `.items()`

Objetivo: Recorrer el diccionario y mostrar cada fruta con su cantidad.

Solucion

```
print("=== INVENTARIO DE FRUTAS ===")
for fruta, cantidad in fruits.items():
    print(f"{fruta}: {cantidad} unidades")
```

Ejercicio 7: Actualizar multiples valores con update()

Objetivo: Actualizar las cantidades de manzanas (a 25) y naranjas (a 30), y agregar “kiwis” con 8 unidades.

Solucion

```
fruits.update({
    "manzanas": 25,
    "naranjas": 30,
    "kiwis": 8
})

print("Diccionario actualizado:", fruits)
```

Ejercicio 8: Filtrar elementos con condicion

Objetivo: Mostrar solo las frutas que tienen mas de 10 unidades.

Solucion

```
print("=== FRUTAS CON STOCK > 10 ===")
for fruta, cantidad in fruits.items():
    if cantidad > 10:
        print(f"{fruta}: {cantidad} unidades")
```

Ejercicio 9: Calcular el total de unidades

Objetivo: Sumar todas las cantidades del diccionario.

Solucion

```
total = 0
for cantidad in fruits.values():
    total = total + cantidad

print(f"Total de unidades: {total}")

# Tambien puedes usar sum() directamente
total_rapido = sum(fruits.values())
print(f"Total con sum(): {total_rapido}")
```

Ejercicio 10: Diccionario de constantes universales

Objetivo: Crear un diccionario con constantes universales (como en Física) y luego acceder a cada una.

Como desafío extra, te propongo aplicar diccionarios para organizar constantes universales. Podes tener un diccionario `constantes` donde la clave sea el nombre (como "G" o "c") y el valor sea su magnitud.

Solucion

```
# Diccionario de constantes universales
constantes = {
    "G": 6.67430e-11,      # Constante gravitacional (m3 kg-1 s-2)
    "c": 299792458,       # Velocidad de la luz (m/s)
    "h": 6.62607015e-34,  # Constante de Planck (J·s)
    "k": 1.380649e-23,    # Constante de Boltzmann (J/K)
    "e": 1.602176634e-19, # Carga elemental (C)
    "R": 8.314462618,     # Constante de los gases ideales (J/(mol·K))
    "NA": 6.02214076e23   # Numero de Avogadro (mol-1)
}

print("=== CONSTANTES UNIVERSALES ===")
for nombre, valor in constantes.items():
    # Mostramos en notacion cientifica con 4 decimales
    print(f"{nombre} = {valor:.4e}")

# Acceder a una constante especifica
print(f"\nVelocidad de la luz (c): {constantes['c']} m/s")
print(f"Constante de Planck (h): {constantes.get('h')} J·s")
```


Ejercicio Integrador: Sistema de Inventario

Aca te propongo un ejercicio mas completo que combina TODO lo que aprendiste. Es un sistema de inventario que podrias usar en un almacen o en un laboratorio de la facu.

```
# =====
# SISTEMA DE INVENTARIO DE TIENDA
# =====

# Diccionario inicial (producto: stock)
inventario = {
    "manzanas": 50,
    "peras": 30,
    "naranjas": 40,
    "bananas": 25
}

print("=== SISTEMA DE INVENTARIO ===")
print("Inventario inicial:", inventario)

# 1. Acceder a un valor (cuantas manzanas hay?)
print(f"\n1. Manzanas disponibles: {inventario.get('manzanas', 0)}")

# 2. Agregar nuevos productos
inventario["uvas"] = 15
inventario.setdefault("kiwis", 10)
print(f"2. Despues de agregar: {inventario}")

# 3. Eliminar un producto (peras)
valor_eliminado = inventario.pop("peras")
print(f"3. Eliminamos 'peras' (habia {valor_eliminado} unidades)")

# 4. Actualizar multiples productos (update)
inventario.update({"manzanas": 60, "naranjas": 35, "frutillas": 20})
print(f"4. Despues de actualizar: {inventario}")

# 5. Verificar si existe un producto
producto = input("\n5. Que producto quieres consultar? ")
if producto in inventario:
    print(f"    ¡Si! Hay {inventario[producto]} unidades de {producto}")
else:
    print(f"    No tenemos {producto} en el inventario")

# 6. Listar productos (claves)
print(f"\n6. Productos en stock: {list(inventario.keys())}")

# 7. Calcular total de unidades
total = sum(inventario.values())
print(f"7. Total de unidades en inventario: {total}")
```

```
# 8. Filtrar productos con bajo stock (< 20)
print("\n8. Productos con stock bajo (< 20):")
for producto, stock in inventario.items():
    if stock < 20:
        print(f"    - {producto}: {stock} unidades (;reponer!)")

# 9. Mostrar todos los productos
print("\n9. INVENTARIO FINAL:")
for producto, stock in inventario.items():
    print(f"    {producto}: {stock} unidades")
```

Lista de Ejercicios Resumida

Nro	EJERCICIO
1	Acceder a valores (get / corchetes)
2	Agregar elementos (directo / setdefault / update)
3	Eliminar elementos (del / pop)
4	Listar claves y valores por separado
5	Comprobar existencia (in)
6	Recorrer diccionario con .items()
7	Actualizar multiples valores con update()
8	Filtrar elementos con condicion
9	Calcular total de valores (sum)
10	Crear diccionario de constantes universales

Resumen de Metodos Utiles

METODO	DESCRIPCION
.get(clave, def)	Accede al valor (None o default si no existe)
.setdefault(c,v)	Agrega clave con valor si no existe
.update(dic)	Actualiza o agrega multiples pares
.pop(clave)	Elimina y devuelve el valor
.keys()	Devuelve todas las claves
.values()	Devuelve todos los valores
.items()	Devuelve pares (clave, valor)
del dic[clave]	Elimina una clave (no devuelve nada)
clave in dic	Verifica si la clave existe (True/False)

Resumen del Capitulo

En este capitulo hiciste un repaso intensivo de las operaciones mas importantes con diccionarios:

- **Acceder:** con `get()` (seguro) o con corchetes (directo).
- **Agregar:** con asignacion directa, `setdefault()` o `update()`.
- **Eliminar:** con `del` o `pop()`.
- **Listar:** con `keys()` y `values()`.
- **Comprobar:** con el operador `in`.
- **Recorrer:** con `.items()` para obtener clave y valor juntos.
- **Actualizar:** con `update()` para cambios masivos.
- **Filtrar:** combinando `items()` con un `if`.
- **Calcular:** usando `sum()` sobre `values()`.

¡Practica estos ejercicios hasta que te salgan sin pensar! Los diccionarios son una herramienta fundamental que vas a usar en cada proyecto de Python, especialmente en la facultad para organizar datos de experimentos, constantes, configuraciones y mucho mas.

REPASO: Diccionarios y sus Métodos en Python

¿Que es un diccionario?

Cuando arranque a programar en la UBA Exactas, mi primer encuentro con los diccionarios fue raro. Ya venia usando listas para todo, pero un dia necesite guardar los precios de cien productos y encontrarlos al instante sin recorrer toda la lista. Ahi un ayudante me dijo: “Nahuel, usa un diccionario, es como un indice de libro: pones la etiqueta y vas directo al dato”. Y tenia razon. Desde ese dia, los diccionarios son mi herramienta preferida.

Un **diccionario** es una estructura de datos que guarda informacion en pares de **clave** y **valor**. A diferencia de las listas, que usan indices numericos (0, 1, 2...), los diccionarios usan claves que pueden ser texto, numeros, lo que vos quieras.

Analogia: Imagina un casillero de correo. Cada casillero tiene una etiqueta con un nombre (la clave) y adentro guarda cartas o paquetes (el valor). No necesitas saber en que posicion esta; con solo leer la etiqueta, lo encontras al toque.

```
+-----+
|
|  DICCIONARIO = { clave1: valor1,
|                  clave2: valor2,
|                  clave3: valor3 }
|
|  Las claves son UNICAS (no se repiten)
|  Los valores pueden ser cualquier cosa
|
+-----+
```

¿Como se crea un diccionario?

Se usan **llaves** {} y se separan las claves de los valores con **dos puntos** :. Cada par clave-valor se separa con comas.

```
# Diccionario vacio
diccionario_vacio = {}

# Diccionario con datos
persona = {
    "nombre": "Nahuel",
    "edad": 25,
```

```
    "ciudad": "Buenos Aires"  
}
```

```
print(persona)
```

Salida:

```
{'nombre': 'Nahuel', 'edad': 25, 'ciudad': 'Buenos Aires'}
```

Ejemplo practico: Bolsa de la compra

En mis primeras practicas, modele un carrito de compras con diccionarios. Cada producto era un diccionario con claves como 'nombre', 'precio', 'cantidad'.

```
# Creamos un diccionario que representa un producto (pan)
```

```
producto = {  
    'nombre': 'pan',  
    'precio': 70,  
    'cantidad': 2  
}
```

```
# Creamos otro producto (ajo)
```

```
producto1 = {  
    'nombre': 'ajo',  
    'precio': 701,  
    'cantidad': 2  
}
```

```
# Creamos otro producto (donas)
```

```
producto2 = {  
    'nombre': 'donas',  
    'precio': 704,  
    'cantidad': 2  
}
```

```
print("Producto 1:", producto)  
print("Producto 2:", producto1)  
print("Producto 3:", producto2)
```

Lista de diccionarios

Una de las estructuras mas utiles es tener una **lista de diccionarios**. Cada elemento de la lista es un diccionario que representa un objeto (un producto, una persona, etc.). Es como tener una base de datos chiquita.

```
# Creamos una lista vacia llamada "bolsa"  
bolsa = []
```

```
# Agregamos los productos a la lista
```

```
bolsa.append(producto)    # Posicion 0
bolsa.append(producto1)   # Posicion 1
bolsa.append(producto2)   # Posicion 2

print("Bolsa de compras:", bolsa)
```

Acceder a los valores de un diccionario

Para obtener un valor, usas el nombre del diccionario y, entre corchetes, la **clave**.

```
# Acceder a valores de un diccionario individual
print("Nombre del producto:", producto['nombre'])
print("Precio del producto:", producto['precio'])
print("Cantidad:", producto['cantidad'])
```

Acceder dentro de una lista de diccionarios

Si tienes una lista de diccionarios, primero accedes al índice de la lista y luego a la clave del diccionario. Es como un doble salto.

```
# Accedemos al nombre del producto en posicion 1 (ajo)
print(bolsa[1]['nombre'])    # "ajo"
print(bolsa[2]['precio'])    # 704 (precio de las donas)
```

Nota: *Diferencia clave entre listas y diccionarios:*

- **Listas:** se accede por *índice numerico* (0, 1, 2...)
- **Diccionarios:** se accede por *clave* ("nombre", .edad"...)

Recorrer diccionarios

Recorrer las claves

```
for clave in producto:
    print(f"Clave: {clave}")
```

Recorrer los valores

```
for valor in producto.values():
    print(f"Valor: {valor}")
```

Recorrer claves y valores juntos con .items()

Esta es la forma más pitónica y la que más vas a usar. Te da la clave y el valor al mismo tiempo.

```
for clave, valor in producto.items():
    print(f"{clave} -> {valor}")
```

Modificar un diccionario

Agregar o modificar un valor

La misma sintaxis sirve para las dos cosas. Si la clave existe, modifica; si no, agrega.

```
producto['marca'] = 'Bimbo' # Agrega una nueva clave 'marca'
producto['precio'] = 80     # Modifica el valor de 'precio'
print(producto)
```

Eliminar una clave con del

```
del producto['marca'] # Elimina la clave 'marca'
print(producto)
```

Eliminar una clave con pop()

pop() elimina la clave y te devuelve el valor que tenia, por si lo necesitas.

```
precio_eliminado = producto.pop('precio')
print(f"Precio eliminado: {precio_eliminado}")
print(producto)
```

Metodos utiles de diccionarios

METODO	DESCRIPCION
.keys()	Devuelve todas las claves del diccionario
.values()	Devuelve todos los valores del diccionario
.items()	Devuelve todos los pares (clave, valor)
.get(clave, def)	Devuelve el valor o None si no existe
.setdefault(c,v)	Agrega clave con valor si no existe
.pop(clave)	Elimina una clave y devuelve su valor
.update(otro)	Actualiza el diccionario con otro
.clear()	Vacia el diccionario

Ejemplos

```
producto = {'nombre': 'pan', 'precio': 70, 'cantidad': 2}
```

```
print("Claves:", producto.keys())
print("Valores:", producto.values())
print("Items:", producto.items())
```

get() es mas seguro que [] porque no da error si la clave no existe

```
print(producto.get('nombre')) # 'pan'
print(producto.get('marca', 'no existe')) # 'no existe'
```

```
# update() agrega o actualiza multiples claves
producto.update({'precio': 85, 'marca': 'Bimbo'})
print(producto)

# clear() vacia el diccionario
producto.clear()
print(producto) # {}
```

Ejemplo completo: Sistema de Login con Diccionario

Un sistema de login que usa un diccionario para guardar usuarios registrados. Lo arme en mis primeras practicas y me sirvio un monton.

```
# Diccionario que guarda usuarios registrados
usuarios_registrados = {}

while True:
    print("\n=== SISTEMA DE LOGIN ===")
    print("1. Registrarse")
    print("2. Iniciar sesion")
    print("3. Ver todos los usuarios")
    print("4. Salir")

    opcion = input("Elegi una opcion: ")

    if opcion == "1":
        usuario = input("Ingresa un nombre de usuario: ")
        if usuario in usuarios_registrados:
            print("Ese usuario ya existe. Elegi otro.")
        else:
            contrasena = input("Ingresa una contrasena: ")
            usuarios_registrados[usuario] = contrasena
            print(f"Usuario {usuario} registrado con exito!")

    elif opcion == "2":
        usuario = input("Usuario: ")
        contrasena = input("Contrasena: ")
        if usuario in usuarios_registrados and usuarios_registrados[usuario] == contrasena:
            print(f"Bienvenido {usuario}!")
        else:
            print("Usuario o contrasena incorrectos.")

    elif opcion == "3":
        if len(usuarios_registrados) == 0:
            print("No hay usuarios registrados.")
        else:
            print("\n--- USUARIOS REGISTRADOS ---")
            for user in usuarios_registrados:
```



```
print(f"- {user}")

elif opcion == "4":
    print("Hasta luego!")
    break
else:
    print("Opcion no valida.")
```

Ejemplo Integrador: Sistema de Inventario

Un ejercicio que combina todo lo que aprendiste sobre diccionarios. Es un sistema de inventario como el que podrias usar en un almacen.

```
inventario = {
    "manzanas": 50,
    "peras": 30,
    "naranjas": 40,
    "bananas": 25
}

print("=== SISTEMA DE INVENTARIO ===")
print("Inventario inicial:", inventario)

# 1. Acceder a un valor
print(f"\n1. Manzanas disponibles: {inventario.get('manzanas', 0)}")

# 2. Agregar nuevos productos
inventario["uvas"] = 15
inventario.setdefault("kiwis", 10)
print(f"2. Despues de agregar: {inventario}")

# 3. Eliminar un producto
valor_eliminado = inventario.pop("peras")
print(f"3. Eliminamos 'peras' (habia {valor_eliminado} unidades)")

# 4. Actualizar multiples productos
inventario.update({"manzanas": 60, "naranjas": 35, "frutillas": 20})
print(f"4. Despues de actualizar: {inventario}")

# 5. Verificar si existe un producto
producto = input("\n5. Que producto quieres consultar? ")
if producto in inventario:
    print(f"    ¡Si! Hay {inventario[producto]} unidades de {producto}")
else:
    print(f"    No tenemos {producto} en el inventario")

# 6. Listar productos
print(f"\n6. Productos en stock: {list(inventario.keys())}")
```

```
# 7. Calcular total de unidades
total = sum(inventario.values())
print(f"7. Total de unidades en inventario: {total}")

# 8. Filtrar productos con bajo stock
print("\n8. Productos con stock bajo (< 20):")
for producto, stock in inventario.items():
    if stock < 20:
        print(f"    - {producto}: {stock} unidades (;reponer!)")

# 9. Mostrar inventario final
print("\n9. INVENTARIO FINAL:")
for producto, stock in inventario.items():
    print(f"    {producto}: {stock} unidades")
```

Ejemplo: Carrito de Compras con Diccionesarios

Otro clasico que te va a servir para practicar. Un carrito de compras que usa una lista de diccionarios.

```
carrito = []

def mostrar_menu():
    print("\n=== CARRITO DE COMPRAS ===")
    print("1. Agregar producto")
    print("2. Ver carrito")
    print("3. Calcular total")
    print("4. Eliminar producto")
    print("5. Salir")

def agregar_producto():
    nombre = input("Nombre del producto: ")
    precio = float(input("Precio: $"))
    cantidad = int(input("Cantidad: "))
    producto = {"nombre": nombre, "precio": precio, "cantidad": cantidad}
    carrito.append(producto)
    print(f"'{nombre}' agregado al carrito.")

def ver_carrito():
    if len(carrito) == 0:
        print("El carrito esta vacio.")
        return
    print("\n--- CARRITO ---")
    for i, item in enumerate(carrito, 1):
        subtotal = item["precio"] * item["cantidad"]
        print(f"{i}. {item['nombre']} - ${item['precio']:.2f} x {item['cantidad']} = ${s
```

```
def calcular_total():
    if len(carrito) == 0:
        print("El carrito esta vacio.")
        return
    total = 0
    for item in carrito:
        total = total + (item["precio"] * item["cantidad"])
    print(f"Total a pagar: ${total:.2f}")

def eliminar_producto():
    if len(carrito) == 0:
        print("El carrito esta vacio.")
        return
    ver_carrito()
    try:
        indice = int(input("Numero del producto a eliminar: ")) - 1
        if 0 <= indice < len(carrito):
            eliminado = carrito.pop(indice)
            print(f"'{eliminado['nombre']}' eliminado del carrito.")
        else:
            print("Numero invalido.")
    except ValueError:
        print("Ingresa un numero valido.")

while True:
    mostrar_menu()
    opcion = input("Elegi una opcion: ")
    if opcion == "1":
        agregar_producto()
    elif opcion == "2":
        ver_carrito()
    elif opcion == "3":
        calcular_total()
    elif opcion == "4":
        eliminar_producto()
    elif opcion == "5":
        print("Gracias por su compra!")
        break
    else:
        print("Opcion no valida.")
```

Ejercicios Propuestos

Ejercicio 1

Crea un diccionario llamado **persona** con las claves: "nombre", "edad", "ciudad". Asigna los valores y mostralo.

Ejercicio 2

Accede al valor de la clave `edad` del diccionario del ejercicio anterior y mostralo.

Ejercicio 3

Agrega una nueva clave `profesion` al diccionario `persona` y asigne un valor.

Ejercicio 4

Modifica el valor de la clave `ciudad` del diccionario `persona`.

Ejercicio 5

Elimina la clave `edad` del diccionario usando `del`.

Ejercicio 6

Crea un diccionario con 3 productos (clave: nombre del producto, valor: precio). Muestra todos los productos con su precio.

Ejercicio 7

Recorre el diccionario del ejercicio anterior usando `.items()` y muestra cada producto.

Ejercicio 8

Crea una lista de 3 diccionarios, donde cada diccionario representa una persona con nombre, edad y ciudad. Luego, recorre la lista y muestra los datos de cada persona.

Ejercicio 9

Agrega un nuevo diccionario (persona) a la lista del ejercicio anterior.

Ejercicio 10

Usando el diccionario de productos del ejercicio 6, calcula el precio total de comprar 2 de cada producto.

Ejercicio 11

Crea un diccionario que guarde las notas de 3 materias (clave: materia, valor: nota). Luego calcula el promedio.

Ejercicio 12

Usando `.get()`, intenta acceder a una clave que no existe en un diccionario y muestra un mensaje personalizado.

Ejercicio 13

Crea un diccionario vacío. Usa un bucle para pedir al usuario que ingrese 3 nombres y edades, y guardalos en el diccionario.

Ejercicio 14

Crea un programa que permita al usuario buscar un producto por su nombre en un diccionario de precios.

Ejercicio 15

Usando `.keys()`, muestra todas las claves de un diccionario.

Ejercicio 16

Usando `.values()`, muestra todos los valores de un diccionario.

Ejercicio 17

Crea un diccionario con los meses del año (1: "Enero", 2: "Febrero", ...). Pide al usuario un número y muestra el mes correspondiente.

Ejercicio 18

Crea un diccionario que guarde el inventario de una tienda (producto: cantidad). Simula una venta: pide un producto y resta 1 de su cantidad.

Ejercicio 19

Toma dos diccionarios con datos de personas. Únelos usando `.update()`.

Ejercicio 20

Crea un programa completo de agenda telefónica usando un diccionario (nombre: número de teléfono). Debe permitir: agregar contacto, buscar contacto, eliminar contacto, mostrar todos.

Soluciones Paso a Paso

Solucion 1

```
persona = {  
    "nombre": "Nahuel",  
    "edad": 25,  
    "ciudad": "Buenos Aires"  
}  
print(persona)
```

Explicacion: Usamos llaves y dos puntos para crear el diccionario.

Solucion 2

```
print(persona["edad"])
```

Explicacion: Accedemos con corchetes y la clave exacta.

Solucion 3

```
persona["profesion"] = "programador"  
print(persona)
```

Explicacion: Al asignar a una clave nueva, Python la agrega automaticamente.

Solucion 4

```
persona["ciudad"] = "La Plata"  
print(persona)
```

Explicacion: Si la clave ya existe, se sobrescribe el valor.

Solucion 5

```
del persona["edad"]  
print(persona)
```

Explicacion: del elimina la clave y su valor.

Solucion 6

```
productos = {"pan": 70, "leche": 90, "huevos": 120}  
print(productos)
```

Explicacion: Las claves son los nombres, los valores los precios.

Solucion 7

```
for producto, precio in productos.items():  
    print(f"{producto}: ${precio}")
```

Explicacion: `.items()` nos da cada par en cada vuelta.

Solucion 8

```
personas = [  
    {"nombre": "Ana", "edad": 22, "ciudad": "CABA"},  
    {"nombre": "Luis", "edad": 25, "ciudad": "Cordoba"},  
    {"nombre": "Marta", "edad": 21, "ciudad": "Rosario"}  
]  
for p in personas:  
    print(f"{p['nombre']} tiene {p['edad']} años y vive en {p['ciudad']}")
```

Explicacion: Recorremos la lista y por cada diccionario accedemos a sus claves.

Solucion 9

```
personas.append({"nombre": "Pedro", "edad": 23, "ciudad": "Mendoza"})  
print(personas)
```

Explicacion: Agregamos un diccionario nuevo a la lista con `append()`.

Solucion 10

```
total = 0  
for producto, precio in productos.items():  
    total = total + (precio * 2)  
print(f"Total por 2 de cada producto: ${total}")
```

Explicacion: Multiplicamos cada precio por 2 y sumamos.

Solucion 11

```
notas = {"matematica": 8, "fisica": 7, "quimica": 9}  
promedio = sum(notas.values()) / len(notas)  
print(f"Promedio: {promedio:.2f}")
```

Explicacion: `sum()` suma todos los valores y `len()` cuenta las materias.

Solucion 12

```
valor = persona.get("altura", "La clave 'altura' no existe")  
print(valor)
```

Explicacion: `.get()` con segundo argumento evita el error y devuelve el mensaje.

Solucion 13

```
datos = {}
for i in range(3):
    nombre = input("Nombre: ")
    edad = input("Edad: ")
    datos[nombre] = edad
print(datos)
```

Explicacion: El usuario va llenando el diccionario; la clave es el nombre, el valor la edad.

Solucion 14

```
productos = {"pan": 70, "leche": 90, "huevos": 120}
buscar = input("Producto a buscar: ")
if buscar in productos:
    print(f"El precio de {buscar} es ${productos[buscar]}")
else:
    print("Producto no encontrado")
```

Explicacion: Verificamos pertenencia con `in` y luego accedemos.

Solucion 15

```
print(persona.keys())
```

Explicacion: Devuelve una vista con todas las claves.

Solucion 16

```
print(persona.values())
```

Explicacion: Similar pero con los valores.

Solucion 17

```
meses = {1:"Enero",2:"Febrero",3:"Marzo",4:"Abril",5:"Mayo",6:"Junio",
7:"Julio",8:"Agosto",9:"Septiembre",10:"Octubre",11:"Noviembre",12:"Diciembre"}
num = int(input("Numero de mes: "))
print(meses.get(num, "Numero invalido"))
```

Explicacion: Mapeamos numero a nombre y usamos `get()` para evitar errores.

Solucion 18

```
inventario = {"pan": 10, "leche": 5, "huevos": 12}
producto = input("Producto a vender: ")
if producto in inventario and inventario[producto] > 0:
    inventario[producto] -= 1
    print(f"Vendido. Quedan {inventario[producto]} unidades de {producto}")
else:
    print("Producto no disponible")
```


Explicacion: Verificamos disponibilidad y descontamos una unidad.

Solucion 19

```
persona1 = {"nombre": "Ana", "edad": 22}
persona2 = {"ciudad": "CABA", "profesion": "ingeniera"}
persona1.update(persona2)
print(persona1)
```

Explicacion: update() vuelca las claves de un diccionario en otro.

Solucion 20

```
agenda = {}
while True:
    print("\n--- AGENDA TELEFONICA ---")
    print("1. Agregar contacto")
    print("2. Buscar contacto")
    print("3. Eliminar contacto")
    print("4. Mostrar todos")
    print("5. Salir")
    op = input("Opcion: ")
    if op == "1":
        nombre = input("Nombre: ")
        tel = input("Telefono: ")
        agenda[nombre] = tel
        print(f"{nombre} agregado.")
    elif op == "2":
        nombre = input("Nombre a buscar: ")
        if nombre in agenda:
            print(f"{nombre}: {agenda[nombre]}")
        else:
            print("No encontrado.")
    elif op == "3":
        nombre = input("Nombre a eliminar: ")
        if nombre in agenda:
            del agenda[nombre]
            print(f"{nombre} eliminado.")
        else:
            print("No encontrado.")
    elif op == "4":
        if agenda:
            for n, t in agenda.items():
                print(f"{n}: {t}")
        else:
            print("Agenda vacia.")
    elif op == "5":
        print("Hasta luego!")
        break
```

```
else:  
    print("Opcion invalida.")
```

Explicacion: Menu clasico que utiliza todas las operaciones basicas de diccionarios.

Resumen del Capítulo

En este capítulo repasaste a fondo los diccionarios, una de las estructuras más importantes de Python:

- Un **diccionario** guarda información en pares **clave: valor**.
- Se crean con `{}` y se separan las claves de los valores con `:`.
- Para acceder a un valor usas `diccionario[clave]` o `.get()`.
- Las claves son **únicas**; si repetis una, se pisa el valor anterior.
- Puedes tener **listas de diccionarios**, ideal para representar colecciones de objetos.
- **Métodos fundamentales:**
 - `.keys()`: todas las claves.
 - `.values()`: todos los valores.
 - `.items()`: pares (clave, valor).
 - `.get(clave, default)`: acceso seguro.
 - `.setdefault(clave, valor)`: agrega si no existe.
 - `.pop(clave)`: elimina y devuelve el valor.
 - `.update(otro)`: actualiza con otro diccionario.
 - `.clear()`: vacía el diccionario.
- Los diccionarios son **mutables**: puedes modificarlos después de creados.
- Son ideales para representar objetos con múltiples atributos, bases de datos simples, configuraciones y mucho más.

Los diccionarios son como un placard con cajones etiquetados: guardas lo que quieras y lo encontras al instante. ¡Practica hasta que te salga sin pensar!

Random y Juegos: La Diversión de la Aleatoriedad

¿Qué es un módulo?

Antes de empezar con los juegos, tenemos que entender qué es un **módulo**.

En programación existen dos tipos de módulos:

- **Módulos nativos (o estándar)**: vienen con Python. No tenés que instalar nada extra. Ejemplos: `random`, `os`, `math`, `time`, etc.
- **Módulos externos**: hay que instalarlos con `pip` antes de usarlos. Ejemplos: `numpy`, `pandas`, `matplotlib`, etc.

En este capítulo vamos a usar el módulo `random`, que es **nativo** (no necesitás instalar nada).

```
+-----+
|
|  MÓDULOS NATIVOS: ya vienen con Python
|  MÓDULOS EXTERNOS: hay que instalarlos
|
|  import random # Importamos el módulo random
|
|  random crea números aleatorios usando la
|  hora exacta del sistema (hace operaciones
|  matemáticas y genera un número al azar)
|
+-----+
```

El módulo `random`: funciones principales

El módulo `random` nos permite generar números aleatorios y elegir elementos al azar.

```
import random
```

```
# random.randint(inicio, fin) → número entero aleatorio entre inicio y fin
numero = random.randint(1, 10) # Número entre 1 y 10 (incluye 1 y 10)
print(f"Número aleatorio: {numero}")
```

```
# random.choice(lista) → elige un elemento al azar de una lista
```

```
frutas = ["manzana", "pera", "uva", "sandia"]
fruta_aleatoria = random.choice(frutas)
print(f"Fruta elegida: {fruta_aleatoria}")
```

Juego 1: Adivinar el número (versión simple)

Este es el juego clásico: la computadora piensa un número y vos tenés que adivinarlo.

```
# =====
# JUEGO DE ADIVINANZA - VERSIÓN SIMPLE
# =====

import random

# Generamos un número secreto entre 1 y 5
numero_secreto = random.randint(1, 5)
print("Número aleatorio generado (solo para prueba):", numero_secreto)

print("\n¡Adivine el número secreto entre 1 y 5!")

contador = 0 # Contador de intentos

while contador < 3:
    intento = int(input("Ingrese su número: "))

    if intento == numero_secreto:
        print("¡Excelente! ¡Adivinaste el número!")
        break # Sale del bucle porque ya ganó
    else:
        print("Siga adivinando...")

    contador = contador + 1

if contador == 3 and intento != numero_secreto:
    print(f"¡Se acabaron los intentos! El número era {numero_secreto}.")
```

Juego 2: Adivinar el número (con pistas)

Esta versión es más avanzada: después de cada intento, el programa te da una pista ("más alto." "más bajo").

```
# =====
# JUEGO DE ADIVINANZA - CON PISTAS
# =====

import random
```

```

# random.randint(1, 5) genera un número entero ALEATORIO entre 1 y 5
numero_secreto = random.randint(1, 5)

print("\n=== ADIVINA EL NÚMERO SECRETO ===")
print("El número está entre 1 y 5. Tenés 3 intentos.")

contador = 0

while contador < 3:
    intento = int(input(f"\nIntento {contador + 1}: "))

    if intento == numero_secreto:
        print("¡Excelente! ¡Adivinaste el número!")
        break
    else:
        print("Incorrecto...")

    contador = contador + 1

# Damos pistas después de cada intento
if contador == 1:
    if intento < numero_secreto:
        print("Pista: Más alto")
    elif intento > numero_secreto:
        print("Pista: Más bajo")

elif contador == 2:
    if intento < numero_secreto:
        print("Pista: Más alto (último intento!)")
    elif intento > numero_secreto:
        print("Pista: Más bajo (último intento!)")

if contador == 3 and intento != numero_secreto:
    print(f"\n¡Se acabaron los intentos! El número era {numero_secreto}.")

```

Juego 3: Piedra, Papel o Tijera

El clásico juego de manos. En este caso, vos jugás contra la computadora.

Versión 1: Una sola partida (con errores)

Esta versión tiene un error: no usa las reglas correctas. La comparación `eleccion > CPU` no sirve para este juego.

```

# =====
# PIEDRA, PAPEL O TIJERA - VERSIÓN CON ERROR
# =====

```

```
import random

formas = ['Piedra', 'Papel', 'Tijera']
cpu = random.choice(formas) # La computadora elige una opción al azar

contador = 0

while contador < 3:
    jugador = input("\nPiedra, Papel o Tijera: ")

    # ¡ESTO ESTÁ MAL! No se puede comparar strings con > en este juego
    if jugador > cpu:
        print("Ganaste")
    elif jugador == cpu:
        print("Empate")
    else:
        print("Perdiste")

    contador = contador + 1
```

Versión 2: Una sola partida (con reglas correctas)

```
# =====
# PIEDRA, PAPEL O TIJERA - UNA PARTIDA (REGLAS CORRECTAS)
# =====

import random

formas = ['Piedra', 'Papel', 'Tijera']
cpu = random.choice(formas)

jugador = input("Piedra, Papel o Tijera: ")
print(f"La computadora eligió {cpu}")

# Reglas del juego
if jugador == cpu:
    print("Empate")
elif jugador == 'Piedra' and cpu == 'Tijera':
    print("Ganaste! Piedra aplasta Tijera")
elif jugador == 'Papel' and cpu == 'Piedra':
    print("Ganaste! Papel envuelve Piedra")
elif jugador == 'Tijera' and cpu == 'Papel':
    print("Ganaste! Tijera corta Papel")
else:
    print("Perdiste...")
```

Versión 3: Mejor de 3 rondas

```
# =====
# PIEDRA, PAPEL O TIJERA - MEJOR DE 3 RONDAS
# =====

import random

formas = ['Piedra', 'Papel', 'Tijera']

contador = 0
ganadas_jugador = 0
ganadas_cpu = 0

while contador < 3:
    print(f"\n--- RONDA {contador + 1} ---")

    cpu = random.choice(formas)
    jugador = input("Piedra, Papel o Tijera: ")
    print(f"Computadora eligió: {cpu}")

    if jugador == cpu:
        print("Empate")
    elif jugador == 'Piedra' and cpu == 'Tijera':
        print("Ganaste esta ronda!")
        ganadas_jugador = ganadas_jugador + 1
    elif jugador == 'Papel' and cpu == 'Piedra':
        print("Ganaste esta ronda!")
        ganadas_jugador = ganadas_jugador + 1
    elif jugador == 'Tijera' and cpu == 'Papel':
        print("Ganaste esta ronda!")
        ganadas_jugador = ganadas_jugador + 1
    else:
        print("Perdiste esta ronda!")
        ganadas_cpu = ganadas_cpu + 1

    contador = contador + 1

print("\n=== RESULTADO FINAL ===")
print(f"Jugador: {ganadas_jugador} - Computadora: {ganadas_cpu}")

if ganadas_jugador > ganadas_cpu:
    print("¡Ganaste el juego!")
elif ganadas_cpu > ganadas_jugador:
    print("¡Perdiste el juego!")
else:
    print("¡Empate general!")
```


Juego 4: Batalla por turnos (Jugador vs Monstruo)

Este juego simula un combate por turnos. Vos elegís entre atacar o defenderte.

```
# =====
# BATALLA POR TURNOS - JUGADOR VS MONSTRUO
# =====

import random

# Vidas iniciales
vida_jugador = 10
vida_monstruo = 20

print("=== BATALLA: JUGADOR VS MONSTRUO ===")
print(f"Vida del jugador: {vida_jugador}")
print(f"Vida del monstruo: {vida_monstruo}")

while vida_jugador > 0 and vida_monstruo > 0:

    print("\n--- NUEVO TURNO ---")

    # Se genera el daño aleatorio para este turno
    ataque_jugador = random.randint(0, 20)
    ataque_monstruo = random.randint(0, 40)

    print(f"Tu poder de ataque: {ataque_jugador}")
    print(f"Ataque del monstruo: {ataque_monstruo}")

    eleccion = input("¿Qué querés hacer? (atacar / defender): ").lower()

    if eleccion == 'atacar':
        vida_monstruo = vida_monstruo - ataque_jugador
        print(f"¡Atacaste! El monstruo ahora tiene {vida_monstruo} de vida.")
    elif eleccion == 'defender':
        # Te defendés y recibís la mitad del daño
        vida_jugador = vida_jugador - (ataque_monstruo // 2)
        print(f"Te defendiste. Recibiste {ataque_monstruo // 2} de daño.")
        print(f"Tu vida: {vida_jugador}")
    else:
        print("Opción inválida. El monstruo te ataca sin defensa!")
        vida_jugador = vida_jugador - ataque_monstruo

# Verificar si alguien murió
if vida_monstruo <= 0:
    print("\n¡VICTORIA! ¡Derrotaste al monstruo!")
elif vida_jugador <= 0:
    print("\nGAME OVER. El monstruo te derrotó.")
```

```
print("\n===== FIN DEL JUEGO =====")
```

Juego 5: Batalla avanzada (con funciones y curación)

Esta versión usa funciones para organizar mejor el código. Además, podés curarte durante la batalla.

```
# =====
# BATALLA AVANZADA - CON FUNCIONES Y CURACIÓN
# =====

import random

# =====
# FUNCIONES DEL JUEGO
# =====

def conseguir_ataque_jugador():
    """Genera un ataque aleatorio para el jugador (entre 0 y 30)"""
    return random.randint(0, 30)

def conseguir_ataque_monstruo():
    """Genera un ataque aleatorio para el monstruo (entre 0 y 90)"""
    return random.randint(0, 90)

def curar(vida_jugador):
    """Cura al jugador sumando 100 puntos de vida"""
    vida_jugador = vida_jugador + 100
    print(f"Usaste poción mágica curativa. Vida actual: {vida_jugador}")
    return vida_jugador

def defender(vida_jugador, ataque_monstruo):
    """El jugador se defiende y recibe la mitad del daño"""
    vida_jugador = vida_jugador - (ataque_monstruo // 2)
    print(f"Te defendiste. Recibiste {ataque_monstruo // 2} de daño. Vida: {vida_jugador}")
    return vida_jugador

def atacar(vida_monstruo, ataque_jugador):
    """El jugador ataca al monstruo"""
    vida_monstruo = vida_monstruo - ataque_jugador
    print(f"¡Atacaste! Daño: {ataque_jugador}. Vida del monstruo: {vida_monstruo}")
    return vida_monstruo

def vida_baja(vida_monstruo):
    """Verifica si el monstruo tiene menos de la mitad de su vida máxima"""
    vida_maxima = 1000
    if vida_monstruo < (vida_maxima / 2):
        print("¡El monstruo está enojado! Su ataque se duplica")
```

```
        return True
    return False

# =====
# JUEGO PRINCIPAL
# =====

vida_jugador = 1000
vida_monstruo = 1000
turno = 1

print("=== BATALLA ÉPICA: JUGADOR VS MONSTRUO ===")
print(f"Vida inicial del jugador: {vida_jugador}")
print(f"Vida inicial del monstruo: {vida_monstruo}")

while vida_jugador > 0 and vida_monstruo > 0:

    print(f"\n===== TURNO {turno} =====")
    print(f"Vida jugador: {vida_jugador}")
    print(f"Vida monstruo: {vida_monstruo}")

    # El jugador elige una acción
    eleccion = input("¿Qué querés hacer? (atacar / defender / curar): ").lower()

    # Generar ataques aleatorios
    ataque_jugador = conseguir_ataque_jugador()
    ataque_monstruo = conseguir_ataque_monstruo()

    # Si el monstruo tiene poca vida, su ataque se duplica
    if vida_baja(vida_monstruo):
        ataque_monstruo = ataque_monstruo * 2

    print(f"\nTu poder de ataque: {ataque_jugador}")
    print(f"Ataque del monstruo: {ataque_monstruo}")

    # Acción del jugador
    if eleccion == 'atacar':
        vida_monstruo = atacar(vida_monstruo, ataque_jugador)

    elif eleccion == 'defender':
        vida_jugador = defender(vida_jugador, ataque_monstruo)

    elif eleccion == 'curar':
        vida_jugador = curar(vida_jugador)

    else:
        print("Opción inválida. Perdés el turno y recibís el ataque completo!")
        vida_jugador = vida_jugador - ataque_monstruo
```

```

    print(f"Vida del jugador: {vida_jugador}")

# Verificar si alguien murió
if vida_monstruo <= 0:
    print("\n¡VICTORIA! ¡Derrotaste al monstruo!")
elif vida_jugador <= 0:
    print("\nGAME OVER. El monstruo te derrotó.")

turno = turno + 1

# Prevenir bucles infinitos
if turno > 50:
    print("¡Empate! Demasiados turnos.")
    break

print("\n===== FIN DEL JUEGO =====")

```

Juego 6: Simulador Pokémon (Pikachu vs Meatwho)

Una versión más temática con personajes de Pokémon. Pikachu (vos) se enfrenta a Meatwho (el enemigo).

```

# =====
# SIMULADOR POKÉMON - PIKACHU VS MEATWHO
# =====

import random

def ataque_pikachu():
    return random.randint(12, 15)

def ataque_meatwho():
    return random.randint(15, 20)

def curar_pikachu(vida):
    curacion = random.randint(10, 20)
    nueva_vida = vida + curacion
    if nueva_vida > 100:
        nueva_vida = 100
    print(f"Usaste kit médico. Curaste {curacion} PV. Vida: {nueva_vida}")
    return nueva_vida

def curar_meatwho(vida):
    curacion = random.randint(5, 10)
    nueva_vida = vida + curacion
    if nueva_vida > 100:
        nueva_vida = 100
    print(f"Meatwho se regenera {curacion} PV. Vida: {nueva_vida}")

```

```
    return nueva_vida

def vida_baja_meatwho(vida):
    return vida <= 20 # Cuando tiene menos del 20% de vida

# Vidas iniciales
vida_pikachu = 100
vida_meatwho = 100
turno = 1

print("=== BATALLA POKÉMON: PIKACHU VS MEATWHO ===")
print(f"Pikachu: {vida_pikachu} PV | Meatwho: {vida_meatwho} PV")

while vida_pikachu > 0 and vida_meatwho > 0:

    print(f"\n===== TURNO {turno} =====")
    print(f"Pikachu: {vida_pikachu} PV")
    print(f"Meatwho: {vida_meatwho} PV")

    # El jugador elige acción
    accion = input("¿Qué hacés? (atacar / curar): ").lower()

    # Calcular ataques
    atk_pikachu = ataque_pikachu()
    atk_meatwho = ataque_meatwho()

    # Verificar si Meatwho está enojado (vida baja)
    if vida_baja_meatwho(vida_meatwho):
        print("¡Meatwho está en apuros y se enoja! Su ataque se duplica")
        atk_meatwho = atk_meatwho * 2
        vida_meatwho = curar_meatwho(vida_meatwho) # Se regenera

    print(f"Tu ataque: {atk_pikachu} | Ataque de Meatwho: {atk_meatwho}")

    # Resolver turno
    if accion == 'atacar':
        vida_meatwho = vida_meatwho - atk_pikachu
        print(f"¡Atacaste! Meatwho ahora tiene {vida_meatwho} PV")
        if vida_meatwho > 0:
            vida_pikachu = vida_pikachu - atk_meatwho
            print(f"Meatwho te ataca! Pikachu tiene {vida_pikachu} PV")

    elif accion == 'curar':
        vida_pikachu = curar_pikachu(vida_pikachu)
        # Si te curás, Meatwho no ataca en ese turno

    else:
        print("¡Opción inválida! Perdés el turno y recibís el ataque")
```

```
vida_pikachu = vida_pikachu - atk_meatwho

# Verificar si alguien ganó
if vida_meatwho <= 0:
    print("\n ¡VICTORIA! ¡Derrotaste a Meatwho!")
elif vida_pikachu <= 0:
    print("\n GAME OVER. Meatwho te derrotó.")

turno = turno + 1
if turno > 50:
    print("\n¡Empate! Demasiados turnos.")
    break

print("\n===== FIN DE LA BATALLA =====")
```

Ejercicios Propuestos

Ejercicio 1

Creá un programa que genere un número aleatorio entre 1 y 10 y lo muestre por pantalla.

Ejercicio 2

Creá un programa que elija una letra al azar de la lista ['a', 'b', 'c', 'd', 'e'].

Ejercicio 3

Mejorá el juego de adivinanza para que el número secreto sea entre 1 y 20, con 5 intentos.

Ejercicio 4

Agregale un contador de puntos al juego de Piedra, Papel o Tijera (quien gane 3 rondas, gana el juego).

Ejercicio 5

Creá un juego de Cara o Cruz usando 'random.choice()'.

Ejercicio 6

Creá un juego de dados: que "tire" dos dados y muestre la suma.

Ejercicio 7

Creá un juego donde la computadora piense un número entre 1 y 100 y el jugador tenga que adivinarlo con pistas de "más alto." o "más bajo".

Ejercicio 8

Agregale un sistema de "vidas" al juego de Piedra, Papel o Tijera (3 vidas, cada derrota resta una vida).

Ejercicio 9

Creá un juego de "Adivinanza de números" donde el jugador tenga que adivinar un número de 3 cifras.

Ejercicio 10

Creá un juego de Ruleta" donde el jugador apueste un número y gane si sale su número.

Ejercicio 11

Creá un juego de "Piedra, Papel, Tijera, Lagarto, Spock" (versión extendida).

Ejercicio 12

Creá un juego de "Mayor o Menor" con cartas (generá un número entre 1 y 13, el jugador apuesta si la siguiente carta será mayor o menor).

Ejercicio 13

Creá un juego de Carrera" donde dos personajes tiran dados y avanza el que saca más alto.

Ejercicio 14

Creá un juego de "Adivinanza de palabras" (elegí una palabra de una lista y el jugador tiene que adivinarla letra por letra).

Ejercicio 15

Creá un juego de "Batalla Naval" simple (tablero de 3x3, la computadora esconde un barco, el jugador tiene que adivinar la posición).

Ejercicio 16

Creá un juego de "Memoria" donde se muestren 3 números al azar y el jugador tenga que repetirlos.

Ejercicio 17

Creá un juego de "Preguntados" simple (3 preguntas de opción múltiple, sumá puntos por cada acierto).

Ejercicio 18

Creá un juego de "Ahorcado" simple con una lista de palabras y dibujito ASCII.

Ejercicio 19

Creá un juego de "Tateti"(tres en línea) contra la computadora.

Ejercicio 20

Creá tu propio juego usando random. Puede ser de aventura, dado, cartas, o lo que se te ocurra.

Soluciones a los Ejercicios

Solución 1

```
import random
print(random.randint(1, 10))
```

Solución 2

```
import random
letras = ['a', 'b', 'c', 'd', 'e']
print(random.choice(letras))
```

Solución 3

```
import random
secreto = random.randint(1, 20)
intentos = 0
while intentos < 5:
    num = int(input("Adivina (1-20): "))
    if num == secreto:
        print("¡Ganaste!")
        break
    intentos += 1
if intentos == 5:
    print(f"Perdiste. Era {secreto}")
```

Solución 4

```
import random
puntos_jugador = 0
puntos_cpu = 0
while puntos_jugador < 3 and puntos_cpu < 3:
    cpu = random.choice(['Piedra', 'Papel', 'Tijera'])
    jugador = input("Piedra, Papel o Tijera: ")
    print(f"CPU: {cpu}")
    if jugador == cpu:
        print("Empate")
    elif (jugador == 'Piedra' and cpu == 'Tijera') or \
         (jugador == 'Papel' and cpu == 'Piedra') or \
         (jugador == 'Tijera' and cpu == 'Papel'):
        print("Ganaste!")
        puntos_jugador += 1
    else:
        print("Perdiste!")
        puntos_cpu += 1
    print(f"Marcador: {puntos_jugador} - {puntos_cpu}")
print("¡Ganaste el juego!" if puntos_jugador == 3 else "¡Perdiste el juego!")
```

Solución 5

```
import random
cara_o_cruz = random.choice(['Cara', 'Cruz'])
print(f"Salió: {cara_o_cruz}")
```

Solución 6

```
import random
dado1 = random.randint(1, 6)
dado2 = random.randint(1, 6)
print(f"Dado 1: {dado1}, Dado 2: {dado2}")
print(f"Suma: {dado1 + dado2}")
```

Solución 7

```
import random
secreto = random.randint(1, 100)
intentos = 0
while True:
    num = int(input("Adivina (1-100): "))
    intentos += 1
    if num == secreto:
        print(f"¡Ganaste en {intentos} intentos!")
        break
    elif num < secreto:
        print("Más alto")
    else:
        print("Más bajo")
```

Solución 8

```
import random
vidas = 3
while vidas > 0:
    cpu = random.choice(['Piedra', 'Papel', 'Tijera'])
    jugador = input("Piedra, Papel o Tijera: ")
    print(f"CPU: {cpu}")
    if jugador == cpu:
        print("Empate")
    elif (jugador == 'Piedra' and cpu == 'Tijera') or \
        (jugador == 'Papel' and cpu == 'Piedra') or \
        (jugador == 'Tijera' and cpu == 'Papel'):
        print("Ganaste!")
    else:
        vidas -= 1
        print(f"Perdiste! Te quedan {vidas} vidas")
print("Game Over!")
```

Solución 9

```
import random
secreto = random.randint(100, 999)
print("Adivina el número de 3 cifras")
intento = int(input("Número: "))
if intento == secreto:
    print("¡Ganaste!")
else:
    print(f"Perdiste. Era {secreto}")
```

Solución 10

```
import random
balance = 1000
while balance > 0:
    apuesta = int(input(f"Balance: ${balance}. Cuánto apostás? "))
    if apuesta > balance:
        print("No tenés suficiente saldo")
        continue
    numero = int(input("Elegí un número (1-6): "))
    resultado = random.randint(1, 6)
    print(f"Salió: {resultado}")
    if numero == resultado:
        balance += apuesta
        print(f"Ganaste! Balance: ${balance}")
    else:
        balance -= apuesta
        print(f"Perdiste! Balance: ${balance}")
print("Te quedaste sin dinero!")
```

Solución 11

```
import random
opciones = ['Piedra', 'Papel', 'Tijera', 'Lagarto', 'Spock']
reglas = {
    ('Piedra', 'Tijera'): 'aplasta',
    ('Piedra', 'Lagarto'): 'aplasta',
    ('Papel', 'Piedra'): 'cubre',
    ('Papel', 'Spock'): 'refuta',
    ('Tijera', 'Papel'): 'corta',
    ('Tijera', 'Lagarto'): 'decapita',
    ('Lagarto', 'Papel'): 'come',
    ('Lagarto', 'Spock'): 'envenena',
    ('Spock', 'Tijera'): 'rompe',
    ('Spock', 'Piedra'): 'vaporiza',
}
cpu = random.choice(opciones)
jugador = input(f"Elegí: {opciones}: ")
```

```
print(f"CPU: {cpu}")
if jugador == cpu:
    print("Empate")
elif (jugador, cpu) in reglas:
    print(f"Ganaste! {jugador} {reglas[(jugador, cpu)]} {cpu}")
elif (cpu, jugador) in reglas:
    print(f"Perdiste! {cpu} {reglas[(cpu, jugador)]} {jugador}")
else:
    print("No sé qué pasó...")
```

Solución 12

```
import random
carta1 = random.randint(1, 13)
print(f"Primera carta: {carta1}")
apuesta = input("La siguiente será mayor (M) o menor (m)? ")
carta2 = random.randint(1, 13)
print(f"Segunda carta: {carta2}")
if apuesta == 'M' and carta2 > carta1:
    print("Ganaste")
elif apuesta == 'm' and carta2 < carta1:
    print("Ganaste")
else:
    print("Perdiste")
```

Solución 13

```
import random
pos1 = 0
pos2 = 0
while pos1 < 10 and pos2 < 10:
    input("Presioná Enter para tirar dados...")
    avance1 = random.randint(1, 6)
    avance2 = random.randint(1, 6)
    pos1 += avance1
    pos2 += avance2
    print(f"Jugador 1 avanza {avance1} → posición {pos1}")
    print(f"Jugador 2 avanza {avance2} → posición {pos2}")
    print("-" * 20)
if pos1 >= 10:
    print("Ganó el Jugador 1!")
else:
    print("Ganó el Jugador 2!")
```

Solución 14

```
import random
palabras = ['python', 'programacion', 'computadora', 'algoritmo']
```

```
secreta = random.choice(palabras)
adivinadas = ['_'] * len(secreta)
intentos = 6
while intentos > 0 and '_' in adivinadas:
    print(' '.join(adivinadas))
    letra = input("Letra: ")
    if letra in secreta:
        for i, l in enumerate(secreta):
            if l == letra:
                adivinadas[i] = letra
        print("Bien!")
    else:
        intentos -= 1
        print(f"Mal! Te quedan {intentos} intentos")
if '_' not in adivinadas:
    print(f"Ganaste! La palabra era {secreta}")
else:
    print(f"Perdiste! La palabra era {secreta}")
```

Solución 15

```
import random
tablero = [[' ' for _ in range(3)] for _ in range(3)]
barco_fila = random.randint(0, 2)
barco_col = random.randint(0, 2)
intentos = 0
while True:
    print("  0 1 2")
    for i, fila in enumerate(tablero):
        print(f"{i} " + ' '.join(fila))
    fila = int(input("Fila: "))
    col = int(input("Columna: "))
    intentos += 1
    if fila == barco_fila and col == barco_col:
        print(f"Hundiste el barco en {intentos} intentos!")
        break
    else:
        print("Agua...")
        tablero[fila][col] = 'X'
```

Solución 16

```
import random
import time
numeros = [random.randint(1, 9) for _ in range(3)]
print("Memorizá estos números:")
print(' '.join(map(str, numeros)))
time.sleep(3)
```



```

0 |
  |
  |
=====''' , '''
+---+
|   |
0   |
|   |
  |
=====''' , '''
+---+
|   |
0   |
/|  |
  |
=====''' , '''
+---+
|   |
0   |
/|\ |
  |
=====''' , '''
+---+
|   |
0   |
/|\ |
/   |
  |
=====''' , '''
+---+
|   |
0   |
/|\ |
/ \ |
  |
=====''' ]

```

```

while intentos > 0 and '_' in adivinadas:
    print(ahorcado[6 - intentos])
    print(' '.join(adivinadas))
    letra = input("Letra: ")
    if letra in secreta:
        for i, l in enumerate(secreta):
            if l == letra:
                adivinadas[i] = letra

```

```

else:
    intentos -= 1
    print(f"Mal! Intentos: {intentos}")
if '_' not in adivinadas:
    print(f"Ganaste! Era {secreta}")
else:
    print(ahorcado[6])
    print(f"Perdiste! Era {secreta}")

```

Solución 19

```

import random
tablero = [[' ' for _ in range(3)] for _ in range(3)]
def mostrar():
    for fila in tablero:
        print(' '.join(fila))
        print('-' * 5)
def hay_ganador():
    for i in range(3):
        if tablero[i][0] == tablero[i][1] == tablero[i][2] != ' ':
            return True
        if tablero[0][i] == tablero[1][i] == tablero[2][i] != ' ':
            return True
    if tablero[0][0] == tablero[1][1] == tablero[2][2] != ' ':
        return True
    if tablero[0][2] == tablero[1][1] == tablero[2][0] != ' ':
        return True
    return False
jugador = 'X'
while True:
    mostrar()
    if jugador == 'X':
        fila = int(input("Fila: "))
        col = int(input("Columna: "))
    else:
        print("Turno de la computadora...")
        while True:
            fila = random.randint(0, 2)
            col = random.randint(0, 2)
            if tablero[fila][col] == ' ':
                break
    if tablero[fila][col] == ' ':
        tablero[fila][col] = jugador
        if hay_ganador():
            mostrar()
            print(f"Ganó {jugador}!")
            break
    jugador = 'O' if jugador == 'X' else 'X'

```



```
else:
    print("Posición ocupada")
```

Solución 20

```
import random
print("=== JUEGO DE AVENTURA ===")
print("Sos un aventurero buscando un tesoro escondido.")
vida = 10
while vida > 0:
    print(f"\nVida: {vida}")
    print("1. Ir al bosque")
    print("2. Ir a la cueva")
    print("3. Descansar")
    op = input("Elegí: ")
    if op == '1':
        evento = random.choice(['tesoro', 'monstruo', 'nada'])
        if evento == 'tesoro':
            print("Encontraste un tesoro! Ganaste!")
            break
        elif evento == 'monstruo':
            danio = random.randint(1, 5)
            vida -= danio
            print(f"Un monstruo te atacó! Perdiste {danio} de vida!")
        else:
            print("No encontraste nada...")
    elif op == '2':
        evento = random.choice(['pocion', 'monstruo', 'trampa'])
        if evento == 'pocion':
            vida += random.randint(3, 7)
            print(f"Encontraste una poción! +{vida - (vida - 3)} vida!")
        elif evento == 'monstruo':
            danio = random.randint(3, 8)
            vida -= danio
            print(f"Un monstruo te atacó! Perdiste {danio} de vida!")
        else:
            vida -= 2
            print("Caíste en una trampa! Perdiste 2 de vida!")
    elif op == '3':
        vida += 2
        print("Descansaste. Recuperaste 2 de vida!")
    else:
        print("Opción inválida")
if vida <= 0:
    print("Game Over! El aventurero murió...")
    break
```

Resumen del Capítulo

En este capítulo aprendiste:

- Los **módulos** son bibliotecas de código que podemos importar.
- **Módulos nativos** vienen con Python (como `random`).
- `random.randint(a, b)` genera un número entero aleatorio entre a y b.
- `random.choice(lista)` elige un elemento al azar de una lista.
- Los juegos son una excelente forma de practicar programación: usan variables, condicionales, bucles, funciones y aleatoriedad.
- La aleatoriedad hace que cada partida sea diferente.

La programación no tiene que ser aburrida. Con un poco de imaginación y el módulo `random`, podés crear tus propios juegos.

El Módulo String y un Generador de Contraseñas

¿Qué es el módulo string?

Cuando empecé a programar en la UBA Exactas, una de las primeras cosas que me pidieron fue validar si un texto tenía mayúsculas, números y símbolos. Al principio me armé unas cadenas enormes con todas las letras del abecedario a mano, y vivía olvidándome de alguna. Un día un ayudante me dijo: “Nahuel, ¿no conocés el módulo `string`? Ya tiene todo eso organizadito.” Y era verdad. Me cambió la vida.

El módulo `string` es una biblioteca que viene con Python (es nativa) y nos da acceso a **grupos de caracteres predefinidos**. Es como una biblioteca de letras, números y símbolos que Python ya tiene ordenados para vos.

```
+-----+
|
|  string.ascii_letters → todas las letras      |
|  (a-z y A-Z)                                           |
|
|  string.digits → todos los números (0-9)          |
|
|  string.punctuation → todos los símbolos         |
|  (!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~)          |
|
+-----+
```

Ejemplo de uso del módulo string

```
import string

# string.ascii_letters contiene TODAS las letras
print("Letras:", string.ascii_letters)
# Salida: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ

# string.digits contiene todos los números
print("Dígitos:", string.digits)
# Salida: 0123456789

# string.punctuation contiene todos los símbolos
print("Símbolos:", string.punctuation)
```

```
# Salida: !"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

Nota: *¿Por qué es útil? En lugar de escribir manualmente todas las letras del abecedario (mayúsculas y minúsculas) o todos los números, el módulo `string` ya los tiene listos para usar. Es como tener una “caja de herramientas” con todos los caracteres organizados.*

Generador de Contraseñas Aleatorias

Una de las aplicaciones más útiles del módulo `string` es crear un generador de contraseñas seguras. Combinás letras, números y símbolos para que la contraseña sea difícil de adivinar.

¿Cómo funciona?

1. Creamos una “pool” (pileta) de caracteres posibles: letras + números + símbolos.
2. Elegimos caracteres al azar de esa pool.
3. Repetimos el proceso tantas veces como la longitud de la contraseña que queramos.

```
import random
import string

def generar_contraseña(longitud):
    """
    Genera una contraseña aleatoria de la longitud especificada.

    Parámetros:
        longitud: número entero que indica cuántos caracteres tendrá

    Devuelve:
        Un string con la contraseña generada
    """
    # Paso 1: Crear la "pool" de caracteres (letras + números + símbolos)
    caracteres_disponibles = string.ascii_letters + string.digits + string.punctuation

    # Paso 2: Inicializar la contraseña vacía
    contraseña = ""

    # Paso 3: Elegir caracteres al azar, uno por uno
    for i in range(longitud):
        # random.choice() elige un carácter al azar de la pool
        caracter = random.choice(caracteres_disponibles)
        contraseña = contraseña + caracter # Agregamos el carácter

    # Paso 4: Devolver la contraseña generada
    return contraseña

# Ejemplo de uso
```

```
longitud = int(input("¿Cuántos caracteres quieres que tenga tu contraseña? "))
mi_contraseña = generar_contraseña(longitud)
print(f"Tu contraseña generada es: {mi_contraseña}")
```

Ejecución:

```
¿Cuántos caracteres quieres que tenga tu contraseña? 12
Tu contraseña generada es: aB3#kL9$mN2@
```

Nota: *Importante:* Esta es considerada la “opción más segura” porque no discrimina ningún tipo de carácter; todos tienen la misma probabilidad de salir (letras, números y símbolos).

Análisis del código típico

En muchos cursos vas a ver una función similar a esta:

```
def contra_gen(longitud):
    # Paso 1: Crear la pool de caracteres
    contra = string.ascii_letters + string.digits + string.punctuation

    # Paso 2: Contraseña vacía
    password = ""

    # Paso 3: Bucle infinito (se rompe con return)
    while True:
        # Construir la contraseña carácter por carácter
        for i in range(longitud):
            password += random.choice(contra)

        # Devolver la contraseña y salir
        return password
```

Nota: *Nota sobre el código:* El bucle `while True` en esta función es innecesario porque siempre se ejecuta una sola vez y luego sale con `return`. Podrías simplificarlo eliminando el `while True` y dejando solo el `for`.

• Funciones auxiliares que conviene repasar

En tus primeras prácticas seguro te encontraste con funciones como estas. Vale la pena recordar la diferencia entre las que devuelven algo y las que solo imprimen.

Función pura que suma dos números

```
def suma_entre_ellos(num1, num2):
    num3 = num1 + num2
    return num3
```

Función pura que multiplica dos números

```
def multiplicacion_entre_ellos(num7, num8):  
    num9 = num7 * num8  
    return num9
```

Función impura que determina si un número es par o impar

```
def es_o_impar(numero):  
    if numero % 2 == 0:  
        print('Es par')  
    else:  
        print('Es impar')
```

Función impura que dice si una lista tiene más de 6 elementos

```
def la_lista_es_mayor_a_6(lista):  
    if len(lista) > 6:  
        print('La lista es mayor a 6')  
    else:  
        print('La lista es menor a 6')
```

Función impura que agrega un elemento a una lista

```
def agregar_al_final_de_la_lista(lista, cosa_a_agregar):  
    lista.append(cosa_a_agregar)  
    print(lista)
```

Errores comunes que tenés que evitar

Error 1: Importar archivos que no existen

```
import clase1          # ERROR: 'clase1.py' no existe  
import funciones       # ERROR: 'funciones.py' no existe
```

Solución: Asegurate de que los archivos existan en la misma carpeta, o eliminá las importaciones si no las necesitás.

Error 2: Pasar parámetros incorrectos a las funciones

```
# ERROR: 'la_lista_es_mayor_a_6' espera una LISTA, pero recibe un STRING  
la_lista_es_mayor_a_6('Banana')
```

```
# ERROR: 'agregar_al_final_de_la_lista' espera DOS parámetros, recibe UNO  
agregar_al_final_de_la_lista('Ana')
```

Solución: Leé la definición de la función para saber qué parámetros espera y de qué tipo.

Error 3: Asignar el resultado de funciones impuras

```
prueba1 = es_o_impar(10)    # 'es_o_impar' imprime pero devuelve None
print(prueba1)              # Muestra None
```

Solución: Si una función no tiene `return`, devuelve `None`. Si necesitas guardar un valor, usá funciones puras con `return`.

Ejemplo completo: Programa de contraseñas con validación

Acá te muestro un programa más completo que genera contraseñas y verifica su fortaleza.

```
import random
import string

def generar_contraseña(longitud, usar_letras=True, usar_numeros=True, usar_simbolos=True):
    """Genera una contraseña según las opciones elegidas"""
    pool = ""
    if usar_letras:
        pool += string.ascii_letters
    if usar_numeros:
        pool += string.digits
    if usar_simbolos:
        pool += string.punctuation

    if not pool:
        return "Error: Debes seleccionar al menos un tipo de caracter"

    contraseña = ""
    for i in range(longitud):
        contraseña += random.choice(pool)
    return contraseña

def validar_fortaleza(contraseña):
    """Evalúa la fortaleza de una contraseña"""
    tiene_letras = any(c.isalpha() for c in contraseña)
    tiene_numeros = any(c.isdigit() for c in contraseña)
    tiene_simbolos = any(c in string.punctuation for c in contraseña)

    if len(contraseña) >= 12 and tiene_letras and tiene_numeros and tiene_simbolos:
        return "Fuerte"
    elif len(contraseña) >= 8 and tiene_letras and tiene_numeros:
        return "Media"
    else:
        return "Débil"

# Programa principal
```

```
print("=== GENERADOR DE CONTRASEÑAS ===")
long = int(input("Longitud de la contraseña: "))

usar_letras = input("¿Incluir letras? (s/n): ").lower() == 's'
usar_numeros = input("¿Incluir números? (s/n): ").lower() == 's'
usar_simbolos = input("¿Incluir símbolos? (s/n): ").lower() == 's'

contraseña = generar_contraseña(long, usar_letras, usar_numeros, usar_simbolos)
print(f"\nContraseña generada: {contraseña}")
print(f"Fortaleza: {validar_fortaleza(contraseña)}")
```

Ejercicios Propuestos

Ejercicio 1

Importa el módulo `string` y muestra en pantalla `string.ascii_letters`.

Ejercicio 2

Muestra en pantalla `string.digits`.

Ejercicio 3

Muestra en pantalla `string.punctuation`.

Ejercicio 4

Creá una función que genere una contraseña de 8 caracteres solo con letras (sin números ni símbolos).

Ejercicio 5

Creá una función que genere una contraseña de 10 caracteres solo con números.

Ejercicio 6

Creá una función que genere una contraseña de 12 caracteres con letras y números, pero sin símbolos.

Ejercicio 7

Creá una función que genere una contraseña de 15 caracteres con letras, números y símbolos.

Ejercicio 8

Modificá la función `generar_contraseña` para que reciba un parámetro `longitud` y devuelva una contraseña aleatoria.

Ejercicio 9

Agregale a la función `generar_contraseña` la opción de elegir si querés mayúsculas, minúsculas o ambas.

Ejercicio 10

Creá una función que valide si una contraseña tiene al menos 8 caracteres.

Ejercicio 11

Creá una función que valide si una contraseña tiene al menos un número.

Ejercicio 12

Creá una función que valide si una contraseña tiene al menos una letra mayúscula.

Ejercicio 13

Creá una función que valide si una contraseña tiene al menos un símbolo.

Ejercicio 14

Combiná las validaciones de los ejercicios 10-13 en una función `validar_contraseña_fuerte()`.

Ejercicio 15

Creá un programa que genere 5 contraseñas diferentes y las muestre en pantalla.

Ejercicio 16

Creá un programa que permita al usuario elegir la longitud y la cantidad de contraseñas a generar.

Ejercicio 17

Creá un programa que guarde las contraseñas generadas en una lista y luego las muestre numeradas.

Ejercicio 18

Creá una función que genere una contraseña que SÍ o SÍ tenga al menos un número y una letra mayúscula.

Ejercicio 19

Creá un programa que pida al usuario su nombre y genere una contraseña personalizada (ej: primeras 3 letras + números aleatorios).

Ejercicio 20

Creá un generador de contraseñas con menú que permita elegir entre diferentes niveles de seguridad (baja, media, alta).

Soluciones a los Ejercicios

Solución 1

```
import string
print(string.ascii_letters)
```

Explicación: Simplemente importás el módulo y accedés a la constante.

Solución 2

```
import string
print(string.digits)
```

Explicación: `string.digits` contiene "0123456789".

Solución 3

```
import string
print(string.punctuation)
```

Explicación: Te muestra todos los símbolos disponibles.

Solución 4

```
import random
import string

def generar_solo_letras():
    contraseña = ""
    for i in range(8):
        contraseña += random.choice(string.ascii_letters)
    return contraseña

print(generar_solo_letras())
```

Explicación: Elegimos 8 caracteres al azar solo de letras.

Solución 5

```
import random
import string

def generar_solo_numeros():
    contraseña = ""
    for i in range(10):
        contraseña += random.choice(string.digits)
    return contraseña

print(generar_solo_numeros())
```

Explicación: Similar, pero usando solo dígitos.

Solución 6

```
import random
import string

def generar_letras_y_numeros():
    pool = string.ascii_letters + string.digits
    contraseña = ""
    for i in range(12):
        contraseña += random.choice(pool)
    return contraseña

print(generar_letras_y_numeros())
```

Explicación: Combinamos letras y dígitos en la pool.

Solución 7

```
import random
import string

def generar_contraseña_completa():
    pool = string.ascii_letters + string.digits + string.punctuation
    contraseña = ""
    for i in range(15):
        contraseña += random.choice(pool)
    return contraseña

print(generar_contraseña_completa())
```

Explicación: Pool completa con los tres grupos.

Solución 8

```
import random
import string

def generar_contraseña(longitud):
    pool = string.ascii_letters + string.digits + string.punctuation
    contraseña = ""
    for i in range(longitud):
        contraseña += random.choice(pool)
    return contraseña

print(generar_contraseña(10))
```

Explicación: La función recibe la longitud como parámetro.

Solución 9

```
import random
import string

def generar_contraseña(longitud, mayusculas=True, minusculas=True):
    pool = ""
    if mayusculas:
        pool += string.ascii_uppercase
    if minusculas:
        pool += string.ascii_lowercase
    if not pool:
        pool = string.ascii_letters

    contraseña = ""
    for i in range(longitud):
        contraseña += random.choice(pool)
    return contraseña

print(generar_contraseña(10, mayusculas=True, minusculas=False))
```

Explicación: Con dos parámetros booleanos controlas qué incluye la pool.

Solución 10

```
def validar_longitud(contraseña):
    return len(contraseña) >= 8

print(validar_longitud("abc12345")) # True
print(validar_longitud("abc"))      # False
```

Explicación: len() te da la cantidad de caracteres.

Solución 11

```
def tiene_numero(contraseña):
    for c in contraseña:
        if c.isdigit():
            return True
    return False

print(tiene_numero("abc123")) # True
print(tiene_numero("abc"))    # False
```

Explicación: isdigit() verifica si un carácter es un dígito.

Solución 12

```
def tiene_mayuscula(contraseña):
    for c in contraseña:
```

```
        if c.isupper():
            return True
    return False

print(tiene_mayuscula("Hola")) # True
print(tiene_mayuscula("hola")) # False
```

Explicación: `isupper()` detecta mayúsculas.

Solución 13

```
import string

def tiene_simbolo(contraseña):
    for c in contraseña:
        if c in string.punctuation:
            return True
    return False

print(tiene_simbolo("hola!")) # True
print(tiene_simbolo("hola")) # False
```

Explicación: Verificamos si el carácter está dentro de `string.punctuation`.

Solución 14

```
import string

def validar_contraseña_fuerte(contraseña):
    if len(contraseña) < 8:
        return False
    tiene_num = False
    tiene_may = False
    tiene_sim = False
    for c in contraseña:
        if c.isdigit():
            tiene_num = True
        if c.isupper():
            tiene_may = True
        if c in string.punctuation:
            tiene_sim = True
    return tiene_num and tiene_may and tiene_sim

print(validar_contraseña_fuerte("Hola123!")) # True
print(validar_contraseña_fuerte("hola123")) # False
print(validar_contraseña_fuerte("Hola123")) # False
```

Explicación: Combinamos todas las validaciones en una sola función.

Solución 15

```
import random
import string

def generar_contraseña(longitud):
    pool = string.ascii_letters + string.digits + string.punctuation
    return ''.join(random.choice(pool) for _ in range(longitud))

for i in range(5):
    print(f"{i+1}: {generar_contraseña(10)}")
```

Explicación: Usamos una comprensión con join para generar y mostramos 5.

Solución 16

```
import random
import string

def generar_contraseña(longitud):
    pool = string.ascii_letters + string.digits + string.punctuation
    return ''.join(random.choice(pool) for _ in range(longitud))

long = int(input("Longitud: "))
cant = int(input("Cantidad: "))

for i in range(cant):
    print(f"{i+1}: {generar_contraseña(long)}")
```

Explicación: El usuario define ambos parámetros.

Solución 17

```
import random
import string

def generar_contraseña(longitud):
    pool = string.ascii_letters + string.digits + string.punctuation
    return ''.join(random.choice(pool) for _ in range(longitud))

contraseñas = []
for i in range(5):
    contraseñas.append(generar_contraseña(10))

for i, contra in enumerate(contraseñas, 1):
    print(f"{i}. {contra}")
```

Explicación: Guardamos en una lista y luego enumeramos.

Solución 18

```
import random
import string
```

```
def generar_contraseña_segura(longitud):
    # Aseguramos que tenga al menos un número y una mayúscula
    contraseña = random.choice(string.digits)
    contraseña += random.choice(string.ascii_uppercase)
    pool = string.ascii_letters + string.digits + string.punctuation
    for i in range(longitud - 2):
        contraseña += random.choice(pool)
    # Mezclamos para que no queden fijos al principio
    lista = list(contraseña)
    random.shuffle(lista)
    return ''.join(lista)
```

```
print(generar_contraseña_segura(10))
```

Explicación: Agregamos forzosamente un número y una mayúscula, completamos y mezclamos.

Solución 19

```
import random
import string
```

```
def generar_contraseña_personalizada(nombre):
    prefijo = nombre[:3].lower()
    numeros = ''.join(random.choice(string.digits) for _ in range(4))
    simbolos = ''.join(random.choice(string.punctuation) for _ in range(2))
    return prefijo + numeros + simbolos
```

```
nombre = input("Ingresa tu nombre: ")
print(f"Contraseña sugerida: {generar_contraseña_personalizada(nombre)}")
```

Explicación: Combinamos las primeras letras del nombre con dígitos y símbolos.

Solución 20

```
import random
import string
```

```
def generar_baja(longitud):
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(longitud))
```

```
def generar_media(longitud):
    pool = string.ascii_letters + string.digits
    return ''.join(random.choice(pool) for _ in range(longitud))
```



```
def generar_alta(longitud):
    pool = string.ascii_letters + string.digits + string.punctuation
    return ''.join(random.choice(pool) for _ in range(longitud))

while True:
    print("\n=== GENERADOR DE CONTRASEÑAS ===")
    print("1. Seguridad BAJA (solo minúsculas)")
    print("2. Seguridad MEDIA (letras y números)")
    print("3. Seguridad ALTA (letras, números y símbolos)")
    print("4. Salir")

    opcion = input("Elegí una opción: ")

    if opcion == "1":
        long = int(input("Longitud: "))
        print(f"Contraseña: {generar_baja(long)}")
    elif opcion == "2":
        long = int(input("Longitud: "))
        print(f"Contraseña: {generar_media(long)}")
    elif opcion == "3":
        long = int(input("Longitud: "))
        print(f"Contraseña: {generar_alta(long)}")
    elif opcion == "4":
        print("¡Hasta luego!")
        break
    else:
        print("Opción inválida")
```

Explicación: Menú interactivo que llama a diferentes funciones según el nivel de seguridad.

Resumen del Capítulo

En este capítulo aprendiste:

- El módulo `string` te da acceso a grupos de caracteres predefinidos:
 - `string.ascii_letters`: todas las letras (a-z y A-Z)
 - `string.digits`: todos los números (0-9)
 - `string.punctuation`: todos los símbolos (!@#\$ etc.)
- Con estos grupos podés crear **generadores de contraseñas** combinando caracteres al azar con `random.choice()`.
- Un generador de contraseñas típico:
 1. Define una “pool” de caracteres: letras + números + símbolos
 2. Usa un bucle `for` para elegir caracteres al azar
 3. Devuelve la contraseña generada
- También repasamos las **funciones puras e impuras**:
 - Puras: solo calculan y devuelven (usan `return`)
 - Impuras: tienen efectos secundarios (como `print()`)
- Es importante leer los errores para entender qué parámetros espera cada función.

El módulo `string` es como una caja de herramientas llena de letras, números y símbolos. Combinándolos con `random`, podés crear contraseñas seguras que protejan tu información.

El sistema operativo: cómo moverte entre archivos y carpetas con Python

¿Para qué necesitamos el módulo `os`?

Mirá, cuando yo arranqué a programar, mis programas eran como una burbuja. Todo pasaba adentro: entraban datos, hacían cuentas, mostraban resultados. Pero en cuanto quería tocar algo de afuera, no podía.

No podía leer un archivo que tenía en el escritorio. No podía saber en qué carpeta estaba mi programa. No podía crear una carpeta nueva desde el código.

¿Viste? Una limitación enorme.

Ahí apareció el módulo `os`. Es la puerta que conecta Python con el sistema operativo de tu computadora. Te paso a explicar qué permite hacer:

- Saber en qué carpeta está parado tu programa
- Leer archivos que tengas guardados
- Crear carpetas para organizar información
- Moverte entre carpetas como hacés con el `cd` de la terminal
- Renombrar o borrar archivos y carpetas

La idea central es esta: `os` es el traductor entre vos (a través de Python) y el sistema operativo. Windows, Linux, Mac... todos tienen sus propias reglas, pero `os` sabe cómo hablar con cada uno. En criollo, escribís el código una vez y funciona en todos lados.

Tip: `os` viene de "operating system" (sistema operativo). No importa si usás Windows, Linux o Mac. `os` entiende los tres.

Importar el módulo: el primer paso sin falta

Antes de usar cualquier función de `os`, hay que importarlo. Es obligatorio. No hay atajo.

```
>>> import os
```

Pensalo así: `import` es abrir la caja de herramientas. Si no abrís la caja, no podés agarrar el martillo. Una vez que lo hacés, tenés a disposición todas las funciones que vamos a ver.

Fijate que el `import` se escribe una sola vez, al principio del archivo. No lo pongas adentro de un bucle ni adentro de una función si no es necesario. Es como poner las herramientas sobre la mesa antes de empezar a trabajar.

Importante: `import os` va al principio del archivo, después de los comentarios iniciales. Es una buena práctica.

¿Dónde estoy parado? `os.getcwd()`

El primer problema que tuve fue no saber desde qué carpeta se estaba ejecutando mi programa. Porque si no sabés dónde estás, no podés agarrar archivos.

Imaginate que estás en un shopping grande y no sabés en qué piso estás. Necesitás un cartel que te lo diga. `getcwd()` es ese cartel.

```
>>> import os
>>> os.getcwd()
'/home/nahuel/Documentos/proyectos'
```

`getcwd()` significa "get current working directory". En criollo: "decime en qué carpeta estoy parado". Te devuelve la ruta completa desde la raíz de tu disco.

Probalo en tu máquina. Abrí Python, importá `os` y ejecutá `os.getcwd()`. Te va a mostrar una ruta distinta a la mía, obvio. Pero la idea es la misma.

Ejercicio: Probá `print(os.getcwd())` en tu computadora. Anotá la ruta que te muestra para acordarte.

¿Qué hay adentro? `os.listdir()`

Una vez que sabés dónde estás, lo natural es preguntar: "¿qué archivos y carpetas hay acá adentro?".

```
>>> import os
>>> os.listdir()
['Documentos', 'Descargas', 'Imágenes', 'notas.txt', 'script.py']
```

`listdir()` te devuelve una lista con todo lo que hay en la carpeta actual. Fijate que no distingue entre archivos y carpetas. Te da los nombres, y punto. Después vas a tener que preguntarle a otras funciones si cada cosa es archivo o carpeta.

Otra forma de pensarlo: `listdir()` es como abrir una carpeta en el explorador de archivos y mirar lo que hay adentro. Pero en lugar de ver íconos, ves una lista de texto.

Tip: `listdir()` solo muestra nombres. No te dice si es archivo o carpeta. Más adelante vamos a ver cómo diferenciarlos.

Ejemplo práctico: explorar una carpeta con un bucle

Acá te muestro cómo podés recorrer el contenido de una carpeta con un `for`. Es útil para cuando querés hacer algo con cada archivo.

```
>>> import os
>>> contenido = os.listdir()
>>> print("Hay", len(contenido), "elementos en esta carpeta")
>>> for elemento in contenido:
...     print("-", elemento)
```

Primero guardás la lista en una variable (`contenido`). Después imprimís cuántos elementos hay usando `len()`. Y finalmente recorrés cada elemento con un `for` y lo mostrás con un guión adelante.

Reflexión: ¿Qué crees que pasa si ejecutás `os.listdir()` en una carpeta vacía? ¿Qué tipo de dato te devuelve Python? Probá y después volvé acá.

Crear carpetas: `os.makedirs()`

A veces necesitás organizar tu trabajo y crear carpetas nuevas desde el código. Por ejemplo, si estás haciendo un programa que descarga archivos, podrías crear una carpeta "descargas" automáticamente. `makedirs()` te permite crear una o varias carpetas anidadas de una sola vez.

```
>>> import os
>>> os.makedirs("proyectos/python/mi_app")
```

Este comando hace lo siguiente:

- Crea la carpeta `proyectos`
- Dentro de `proyectos`, crea `python`
- Dentro de `python`, crea `mi_app`

Todo en un solo paso. Es como si hicieras `mkdir -p` en la terminal, si conocés ese comando.

Ojo con esto: Si alguna de las carpetas que querés crear ya existe, `makedirs()` va a dar un error. Para evitarlo, podés usar el parámetro `exist_ok=True`:

```
>>> os.makedirs("mi_carpeta", exist_ok=True)
```

Así Python no se queja si la carpeta ya está creada. Es una buena práctica usarlo siempre.

Moverse entre carpetas: `os.chdir()`

Una vez que creaste una carpeta, podés meterte adentro. `chdir()` hace exactamente eso: cambia el directorio de trabajo actual.

```
>>> import os
>>> os.chdir("proyectos")
>>> print(os.getcwd())
/home/nahuel/Documentos/proyectos
```

`chdir()` significa "change directory". Es el equivalente a escribir `cd proyectos` en la terminal. A partir de ese momento, todo lo que hagas (listar, crear archivos, etc.) va a pasar dentro de `proyectos`.

Tip: `os.chdir()` modifica la carpeta actual para el resto del programa. Si después querés volver a la carpeta anterior, tenés que guardar la ruta original antes de cambiarte.

Caminos (paths): por qué es importante `os.path.join()`

Acá tengo que contar algo que me pasó al principio y me hizo perder un par de horas. Windows usa la barra invertida (`\`) para separar carpetas. Linux y Mac usan la barra normal (`/`).

Si yo escribo en mi código:

```
ruta = "carpeta/archivo.txt"
```

Eso funciona en Linux y Mac, pero en Windows, no. Si escribo con barra invertida, en Linux no funciona.

¿La solución? Usar `os.path.join()`. Esta función se encarga de poner el separador correcto según el sistema operativo donde se esté ejecutando el código.

```
>>> import os
>>> ruta = os.path.join("carpeta", "subcarpeta", "archivo.txt")
>>> print(ruta)
carpeta/subcarpeta/archivo.txt    # En Linux o Mac
carpeta\subcarpeta\archivo.txt    # En Windows
```

Fijate que no importa si escribís todo el ruteado de una vez. `join()` recibe partes separadas y las une correctamente.

Regla de oro que aprendí después de varios errores: nunca escribas rutas con `/` o `\` directamente. Siempre usá `os.path.join()`. Te ahorrarás dolores de cabeza cuando compartas tu código con alguien que usa otro sistema operativo.

¿Existe? `os.path.exists()`

Antes de intentar abrir un archivo o meterte en una carpeta, está bueno preguntar si existe. `exists()` te dice eso: devuelve `True` si el archivo o carpeta existe, y `False` si no.

```
>>> import os
>>> os.path.exists("mi_archivo.txt")
False
>>> os.path.exists("proyectos")
True
```

Es como preguntar ¿he, ¿esto está acá o no?". Muy útil para evitar errores cuando tu programa espera un archivo que todavía no existe.

¿Qué es cada cosa? `isfile()` y `isdir()`

Cuando usás `listdir()`, obtenés una lista de nombres. Pero no sabés si cada nombre es un archivo o una carpeta. Para eso están estas dos funciones:

```
>>> import os
>>> for elemento in os.listdir():
...     if os.path.isfile(elemento):
...         print(elemento, "es un ARCHIVO")
...     elif os.path.isdir(elemento):
...         print(elemento, "es una CARPETA")
```

- `isfile()` pregunta: "¿esto es un archivo?"
- `isdir()` pregunta: "¿esto es una carpeta?"

En el ejemplo, recorreremos cada elemento de la carpeta actual, y para cada uno preguntamos si es archivo o carpeta. Después imprimimos el resultado.

Reflexión: ¿Qué te parece que devuelve `os.path.isdir(".")`? El punto representa la carpeta actual. ¿Y `os.path.isfile(".")`? Probá en tu consola si tenés dudas.

¿Cuánto pesa? `os.path.getsize()`

A veces necesitás saber el tamaño de un archivo. Por ejemplo, para mostrar cuánto espacio ocupa o para decidir si es muy grande y conviene procesarlo de otra forma.

```
>>> import os
>>> tamaño = os.path.getsize("foto.jpg")
>>> print(tamaño, "bytes")
245760 bytes
```

`getsize()` devuelve el tamaño en bytes. Eso está bien, pero a veces no es tan fácil de leer. Si querés mostrarlo en kilobytes o megabytes, podés hacer divisiones:

```
>>> tamaño_kb = tamaño / 1024
>>> tamaño_mb = tamaño_kb / 1024
>>> print(f"{tamaño_mb:.2f} MB")
0.23 MB
```

Eliminar carpetas: `os.rmdir()` y `os.removedirs()`

Para borrar carpetas vacías, usás `rmdir()`. Para borrar varias carpetas anidadas y vacías, usás `removedirs()`.

```
>>> os.rmdir("carpeta_vacia")           # Borra solo si está vacía
>>> os.removedirs("a/b/c")              # Borra a/b/c despues b despues a si estan vacias
```

Importante: `rmdir()` solo borra carpetas vacías. Si la carpeta tiene archivos adentro, no la va a borrar y te va a dar un error. Para borrar carpetas con contenido, necesitás otro módulo que se llama `shutil`. Lo vemos más adelante.

Recorrer carpetas completas: `os.walk()`

Esta función es un poco más avanzada, pero te la muestro porque es muy útil cuando querés explorar una carpeta y todo lo que hay adentro, incluyendo subcarpetas.

```
>>> import os
>>> for raiz, carpetas, archivos in os.walk("mi_proyecto"):
...     print("Estoy en:", raiz)
...     print("Carpetas:", carpetas)
...     print("Archivos:", archivos)
...     print("---")
```

`walk()` te devuelve tres cosas por cada carpeta que encuentra:

- `raiz`: la ruta de la carpeta actual
- `carpetas`: lista de subcarpetas dentro de `raiz`
- `archivos`: lista de archivos dentro de `raiz`

Es como tener un GPS que recorre TODO el árbol de carpetas sin que vos tengas que hacer nada. Muy útil para hacer búsquedas o respaldos.

Información del sistema: `os.name`, `os.environ`, `os.sep`

A veces necesitás saber en qué sistema operativo estás corriendo, o acceder a variables del entorno como el usuario o la ruta de sistema.


```
>>> import os
>>> os.name
'posix'    # En Linux/Mac. En Windows sería 'nt'

>>> os.sep
'/'        # En Linux/Mac. En Windows sería '\\'

>>> os.environ.get("USER")
'nahuel'

>>> os.environ.get("PATH")
'/usr/local/bin:/usr/bin:/bin'
```

- `os.name` te dice si estás en Linux/Mac (`'posix'`) o Windows (`'nt'`)
- `os.sep` te da el separador de rutas que usa tu sistema
- `os.environ` es un diccionario con todas las variables de entorno

Renombrar y eliminar archivos: `os.rename()` y `os.remove()`

Para cambiarle el nombre a un archivo o carpeta, usás `rename()`. Para borrar un archivo, usás `remove()`.

```
>>> os.rename("viejonombre.txt", "nuevonombre.txt")
>>> os.remove("archivo_que_no_quiero.txt")
```

`rename()` recibe dos argumentos: el nombre actual y el nombre nuevo. `remove()` borra el archivo para siempre (no va a la papelera de reciclaje, ojo con eso).

Precaución: No hay confirmación. Cuando borras con `remove()`, el archivo desaparece. Usalo con cuidado.

Tabla de referencia rápida

Acá te dejo una tablita con las funciones más usadas. Podés volver a esta página cuando necesites acordarte de algo.

Función	Qué hace
<code>os.getcwd()</code>	Muestra la ruta actual
<code>os.listdir()</code>	Lista el contenido de la carpeta
<code>os.chdir(ruta)</code>	Cambia a otra carpeta
<code>os.makedirs(ruta)</code>	Crea una o varias carpetas anidadas
<code>os.path.join(a,b,c)</code>	Une partes de ruta correctamente
<code>os.path.exists(ruta)</code>	Pregunta si existe
<code>os.path.isfile(ruta)</code>	Pregunta si es archivo
<code>os.path.isdir(ruta)</code>	Pregunta si es carpeta
<code>os.path.getsize(ruta)</code>	Devuelve el tamaño en bytes
<code>os.rmdir(ruta)</code>	Borra carpeta vacía
<code>os.rename(original, nuevo)</code>	Renombra archivo o carpeta
<code>os.remove(archivo)</code>	Borra un archivo
<code>os.walk(ruta)</code>	Recorre todo el árbol de carpetas
<code>os.name</code>	Sistema operativo ('posix' o 'nt')
<code>os.sep</code>	Separador de rutas
<code>os.environ</code>	Diccionario de variables de entorno

Ejercicios para practicar

Acá van unos ejercicios. Hacerlos de verdad, no solo los leas. Aprendés haciendo, no mirando.

Ejercicio 1: Explorador de carpetas

Escribí un programa que:

1. Muestre la ruta actual con `getcwd()`
2. Liste todo el contenido con `listdir()`
3. Cuente cuántos archivos y cuántas carpetas hay (usando `isfile()` e `isdir()`)

Ejercicio 2: Organizador automático

Escribí un programa que:

1. Cree las carpetas Imagenes, Documentos, Videos
2. Dentro de Documentos, cree Trabajo y Personal
3. Muestre la estructura completa usando `listdir()` sobre cada carpeta

Ejercicio 3: Informe de archivos

Escribí un programa que recorra la carpeta actual y muestre:

- Nombre de cada archivo
- Su tamaño en kilobytes
- Si es archivo o carpeta

(Pista: usá un bucle for y `os.path.getsize()`)

Ejercicio 4: Recorrido completo con walk

Usando `os.walk()`, escribí un programa que recorra una carpeta que elijas (por ejemplo, "Documentos") y muestre:

- La ruta de cada carpeta
- Cuántos archivos tiene cada carpeta

Resumen de lo que vimos

Dale, repasemos rápido lo importante:

- `import os` → trae todas las herramientas del sistema operativo
- `getcwd()` → te dice dónde estás parado
- `listdir()` → te dice qué hay en la carpeta actual
- `makedirs()` → crea carpetas, incluso varias al mismo tiempo
- `chdir()` → te mueve entre carpetas
- `os.path.join()` → construye rutas que funcionan en cualquier sistema operativo
- `exists()`, `isfile()`, `isdir()` → son preguntas que le hacés al sistema
- `getsize()` → te dice cuánto pesa un archivo
- `rmdir()` → borra carpetas vacías
- `rename()`, `remove()` → renombrar y borrar archivos
- `walk()` → recorre TODO el árbol de carpetas automáticamente
- `os.name`, `os.sep`, `os.environ` → información del sistema

¿Qué sigue ahora?

Ahora que sabés moverte por el sistema de archivos, el próximo paso lógico es aprender a leer y escribir archivos. Con `open()`, `read()` y `write()` vas a poder crear documentos, guardar información que no se pierda cuando cerrás el programa, y procesar archivos que ya tenías guardados.

Pero antes de eso, practicá lo que vimos acá. Movete entre carpetas, creá estructuras, listá contenido. Todo esto lo vas a usar todo el tiempo, en casi cualquier programa que hagas.

*El sistema de archivos es como el taller
del programador. Aprendé a moverte sin miedo.*

Mas herramientas del sistema operativo: rutas, procesos y comandos

Un poco de historia personal

Cuando empecé a programar, lo que más me costaba era entender cómo se relacionaba Python con el sistema operativo. Me acuerdo patente una noche que quise borrar un archivo temporal y terminé borrando una carpeta entera de fotos de mi viaje a Bariloche. Use `os.remove()` sin preguntar antes si el archivo existía... y pum, adiós recuerdos. Desde ese día le tengo un respeto enorme a las funciones que borran cosas.

Otra confusión enorme fueron las diferencias entre Windows y Linux. En mi casa tenía Windows, pero en la facultad usaban Linux. Me volvía loco con las barras: en Windows las rutas usan `\` y en Linux `/`. Mi primer programa que hacía `carpeta\archivo.txt` andaba joya en casa, pero en la facu rompía todo. Hasta que descubrí `os.path.join()` y mi vida cambió.

En este capítulo vamos a seguir explorando el módulo `os`, pero con más confianza. Ya aprendiste a moverte, crear carpetas y listar archivos. Ahora vamos a manipular rutas de forma inteligente, ejecutar comandos del sistema, y hasta crear procesos (si te animas).

Como siempre, vamos paso a paso, con ejemplos que puedes probar vos mismo en tu computadora. No importa si tenés Windows, Linux o Mac: te voy a mostrar las diferencias y cómo escribir código que funcione en todos lados.

Antes de arrancar: el import de siempre

No te olvides de importar el módulo `os`. Hacerlo cada vez que abras una terminal nueva.

```
>>> import os
```

Manipulando rutas como un profesional

Cuando trabajas con archivos, casi siempre necesitas sacar pedacitos de la ruta: el nombre del archivo, la carpeta donde está, la extensión, etc. Hacer eso a mano con `split()` es un dolor de cabeza y lleno de errores. Por suerte, `os.path` tiene funciones específicas para esto.

`os.path.basename()` : el nombre del archivo

Imaginate que tenés la ruta completa de un archivo y querés mostrarle al usuario solo el nombre, sin todo el quilombo de carpetas. `basename()` es justo para eso.

```
>>> ruta_completa = "/home/nahuel/documentos/carta.txt"
>>> os.path.basename(ruta_completa)
'carta.txt'
```

Es como cuando alguien te pregunta "de donde sos?" vos decis Rosario.^{en} lugar de .Argentina, Santa Fe, Rosario". Te quedas con lo ultimo.

Funciona igual en Windows:

```
>>> ruta_win = "C:\\Users\\nahuel\\foto.jpg"
>>> os.path.basename(ruta_win)
'foto.jpg'
```

os.path.dirname() : la carpeta que contiene al archivo

Si en lugar del nombre quieres la carpeta donde esta el archivo, usas `dirname()`.

```
>>> os.path.dirname("/home/nahuel/documentos/carta.txt")
'/home/nahuel/documentos'
```

Fijate que `basename()` y `dirname()` son complementarias. Si las juntas, recuperas la ruta original.

```
>>> ruta = "/home/nahuel/documentos/carta.txt"
>>> carpeta = os.path.dirname(ruta)
>>> nombre = os.path.basename(ruta)
>>> os.path.join(carpeta, nombre) == ruta
True
```

os.path.abspath() : de ruta relativa a absoluta

Cuando trabajas con rutas relativas (como "docs/nota.txt"), a veces necesitas la ruta absoluta completa, desde la raiz del disco. `abspath()` te da eso.

```
>>> os.path.abspath("docs/nota.txt")
'/home/nahuel/proyectos/docs/nota.txt'
```

Es como cuando alguien te dice .^a dos cuadras del obeliscoz vos lo convertis a Corrientes y 9 de Julio". Lo mismo pero con archivos.

os.path.splitext() : separar nombre y extension

¿Necesitas saber si un archivo es .txt, .jpg o .py? `splitext()` divide la ruta en dos partes: lo que esta antes del ultimo punto, y lo que esta despues (incluyendo el punto).

```
>>> nombre, extension = os.path.splitext("vacaciones.jpg")
>>> print(f"Nombre: {nombre}, Extension: {extension}")
Nombre: vacaciones, Extension: .jpg
```

Ideal para filtrar archivos por tipo. Por ejemplo, para listar solo los .py de una carpeta:

```
>>> for archivo in os.listdir("."):
...     if os.path.isfile(archivo) and os.path.splitext(archivo)[1] == ".py":
...         print(archivo)
```

os.path.join() : el pegamento universal de rutas

Ya lo vimos en el capitulo anterior, pero vale la pena repetirlo: `join()` une partes de una ruta usando el separador correcto de tu sistema operativo.

```
>>> carpeta = "documentos"
>>> archivo = "nota.txt"
>>> ruta = os.path.join(carpeta, archivo)
>>> print(ruta)
documentos/nota.txt    # en Linux/Mac
# documentos\textbackslash nota.txt en Windows
```

Con `join()` te olvidas de si tenes que poner `/` o `\`. Python lo resuelve por vos.

Reflexion: ¿Te das cuenta de que con solo estas cuatro funciones (`basename`, `dirname`, `splitext`, `join`) puedes hacer el 90 % de lo que necesitas con rutas? No hace falta que te aprendas de memoria todas las funciones de `os.path`. Con estas estas mas que cubierto para empezar.

Creando una sola carpeta: os.mkdir()

Ya viste `makedirs()` que crea carpetas y todas las carpetas padres que faltan. Pero si ya sabes que la carpeta padre existe, alcanza con `mkdir()`.

```
>>> os.mkdir("nueva_carpeta")
```

Si la carpeta ya existe, te va a dar un error. Por eso, la buena practica es preguntar antes con `os.path.exists()`.

```
>>> if not os.path.exists("nueva_carpeta"):
...     os.mkdir("nueva_carpeta")
... else:
...     print("La carpeta ya existe")
```

Anécdota: Una vez, en un apuro, puse `os.mkdir("backup")` sin verificar si existia. El programa crasheo porque la carpeta ya estaba ahi desde una ejecucion anterior. Desde ese dia, siempre, **siempre** pregunto antes de crear algo.

Fecha de modificacion: os.path.getmtime()

A veces necesitas saber hace cuanto tiempo se modifico un archivo. Por ejemplo, para hacer backups solo de los archivos cambiados en el ultimo dia.

`getmtime()` devuelve un numero enorme (timestamp) que representa la cantidad de segundos desde el 1 de enero de 1970.

```
>>> timestamp = os.path.getmtime("archivo.txt")
>>> print(timestamp)
1698765432.123
```

Ese numero no es muy amigable para un humano. Por suerte, el modulo `datetime` lo puede convertir a una fecha linda.

```
>>> import datetime
>>> fecha = datetime.datetime.fromtimestamp(timestamp)
>>> print(fecha)
2023-10-31 15:30:32.123000
```

Si quieres saber si un archivo se modifico hace menos de 24 horas:

```
>>> ahora = datetime.datetime.now()
>>> hace_un_dia = ahora - datetime.timedelta(days=1)
>>> fecha_mod = datetime.datetime.fromtimestamp(os.path.getmtime("archivo.txt"))
>>> if fecha_mod > hace_un_dia:
...     print("El archivo se modifico recientemente")
```

Informacion del sistema y variables de entorno

Tu programa a veces necesita saber en que entorno se esta ejecutando. ¿Es Windows o Linux? ¿Cuantos nucleos tiene el procesador? ¿Quien es el usuario actual?

os.name : el nombre del sistema operativo

Ya lo vimos, pero repasemos:

```
>>> os.name
'posix'    # Linux o Mac
# o 'nt' en Windows
```

Muy util para escribir codigo que se comporte distinto segun el sistema.

os.sep : el separador de rutas

```
>>> os.sep
'/'        # en Linux/Mac
# '\\\' en Windows
```

Variables de entorno: os.getenv() y os.environ

Las variables de entorno son como "preferencias globales" del sistema. Por ejemplo, HOME (en Linux/Mac) o USERPROFILE (en Windows) es la carpeta personal del usuario. PATH es la lista de carpetas donde el sistema busca programas.

Para obtener una variable especifica, usa `getenv()`. Si la variable no existe, devuelve `None` (o un valor por defecto que le pases).

```
>>> os.getenv("HOME")
'/home/nahuel'
>>> os.getenv("USER", "invitado") # si no existe USER, devuelve "invitado"
'nahuel'
```

Si quieres todas las variables, `os.environ` es un diccionario.

```
>>> for clave, valor in os.environ.items():
...     print(f"{clave} = {valor}")
```

Ejemplo practico: obtener la carpeta personal del usuario sin importar el sistema operativo.

```
>>> home = os.getenv("HOME") or os.getenv("USERPROFILE")
>>> print(f"Tu carpeta personal es: {home}")
```

Otros datos utiles: `os.cpu_count()`, `os.curdir`, `os.pardir`

- `os.cpu_count()` te dice cuantos nucleos tiene tu procesador. Sirve para decidir cuantos procesos en paralelo puedes lanzar.
- `os.curdir` es siempre `'.'` (el directorio actual).
- `os.pardir` es siempre `'..'` (el directorio padre).

```
>>> os.cpu_count()
8
>>> os.curdir
'.'
>>> os.pardir
'..'
```

Borrar archivos y renombrar (con cuidado)

Ya vimos `rename()` y `remove()` en el capitulo anterior, pero quiero hacer hincapie en la seguridad.

`remove()` borra un archivo para siempre. No va a la papelera de reciclaje. No te pide confirmacion. Es definitivo.

```
>>> os.remove("archivo_importante.txt")
```

Regla de oro: antes de borrar, preguntale al usuario o al menos comprueba que el archivo existe y que no es algo critico.

```
>>> archivo = "borrame.txt"
>>> if os.path.exists(archivo):
...     respuesta = input(f"Seguro que quieres borrar {archivo}? (s/n): ")
...     if respuesta.lower() == "s":
...         os.remove(archivo)
...         print("Borrado")
...     else:
...         print("No se borro")
... else:
...     print("El archivo no existe")
```

Anécdota dolorosa: Una vez hice un script que limpiaba archivos temporales. Por un error, la variable que tenia la ruta estaba vacia, y termine borrando toda la carpeta actual. Perdi horas de trabajo. Desde ese dia, siempre agrego un `print()` antes de borrar para ver que voy a borrar realmente.

Ejecutar comandos del sistema: `os.system()`

A veces quieres hacer algo que ya tiene un comando listo en la terminal: limpiar la pantalla, mostrar el calendario, abrir un programa, etc.

`system()` te permite ejecutar cualquier comando como si estuvieras escribiendo en la terminal.

```
>>> os.system("echo Hola desde Python")
Hola desde Python
0
```

El numero 0 que devuelve significa que el comando se ejecuto bien (en Linux/Mac, 0 es exito; otros numeros indican error).

Un ejemplo practico: limpiar la pantalla segun el sistema operativo.

```
>>> if os.name == 'nt':
...     os.system("cls")
... else:
...     os.system("clear")
```

Cuidado: `system()` puede ser peligroso si ejecutas comandos que vienen del usuario. Por ejemplo, si el usuario escribe `rm -rf /` (borrar todo en Linux) y vos lo ejecutas con `system()`, despedite de tu sistema. Si necesitas mas control, hay modulos mas seguros como `subprocess`, pero eso lo veremos mas adelante.

Recorrido eficiente con `os.scandir()`

Ya conoces `listdir()`, que te devuelve una lista con los nombres de todo lo que hay en una carpeta. El problema es que si despues quieres saber si cada cosa es un archivo o una carpeta, `listdir()` obliga a hacer una llamada extra al sistema por cada elemento. Eso es lento cuando tenes muchas carpetas o archivos.

`scandir()` es mas eficiente porque devuelve objetos que ya contienen esa informacion (y otros metadatos) sin necesidad de preguntar de nuevo.

```
>>> with os.scandir(".") as entradas:
...     for entrada in entradas:
...         if entrada.is_file():
...             print(f"Archivo: {entrada.name}")
...         elif entrada.is_dir():
...             print(f"Carpeta: {entrada.name}")
```

Ademas, puedes obtener el tamaño, la fecha de modificacion, etc., sin hacer consultas extra.

```
>>> entrada.stat().st_size    # tamaño en bytes
>>> entrada.stat().st_mtime   # timestamp de modificacion
```

Recomendacion: cuando tengas que recorrer muchas carpetas (cientos o miles de archivos), usa `scandir()` en lugar de `listdir()`. Es notablemente mas rapido.

Generar bytes aleatorios seguros: `os.urandom()`

Si alguna vez necesitas generar una clave secreta, un token para restablecer contraseña, o cualquier dato aleatorio que tenga que ser seguro (que no se pueda adivinar), no uses `random.randint()`. Eso es para juegos y simulaciones, no para seguridad.

Usa `urandom()`, que genera bytes realmente aleatorios, aptos para criptografía.

```
>>> token = os.urandom(16)    # 16 bytes = 128 bits
>>> token
b'\x9f\x8c\x12\x3a\x7f\xd1\xe4\x5c\x9a\x2b\xf3\x8d\x4e\xa6\xc7\x1b'
```

Para que sea legible (por ejemplo para mostrarlo en un enlace o guardarlo en un archivo), puedes convertirlo a hexadecimal:

```
>>> token.hex()
'9f8c123a7fd1e45c9a2bf38d4ea6c71b'
```

Reflexion: Cuando firmes a alguien en tu web, genera un token de al menos 32 bytes (64 caracteres hexadecimales) con `urandom()`. Nunca uses `random.randint()` para eso, porque es predecible.

Funciones avanzadas: procesos (principalmente Unix)

Esto es para los que quieren ir un poco mas alla. Si estas en Windows, algunas funciones no funcionan (como `fork()`). Pero esta bueno saber que existen, porque las vas a encontrar en codigo de otras personas.

`os.getpid()` : el ID del proceso actual

Cada vez que ejecutas un programa, el sistema operativo le asigna un numero unico (PID, Process ID). Puedes saber el PID de tu programa asi:

```
>>> print("Mi PID es:", os.getpid())
Mi PID es: 12345
```

Sirve para, por ejemplo, crear archivos temporales con el PID en el nombre y evitar colisiones.

`os.fork()` : crear un proceso hijo (solo Unix)

`fork()` crea una copia exacta del programa actual. A partir de ahi, tienes dos procesos ejecutandose al mismo tiempo: el "padre" (el original) y el "hijo" (la copia). En el hijo, `fork()` devuelve 0; en el padre, devuelve el PID del hijo.

```
>>> pid = os.fork()
>>> if pid == 0:
...     print("Soy el hijo")
... else:
...     print("Soy el padre, mi hijo tiene PID", pid)
```

Advertencia: Esto es muy avanzado y puede ser confuso. No te preocupes si no lo entiendes del todo. Mas adelante, cuando veas el modulo `multiprocessing`, vas a tener formas mas amigables de hacer lo mismo sin meterte en `fork()`.

os.kill() : enviar señales a procesos (Unix)

Podes mandarle una señal a otro proceso. Por ejemplo, para terminarlo:

```
>>> os.kill(pid, 9)    # 9 es la señal SIGKILL (mata el proceso)
```

Tambien podes enviar la señal 15 (SIGTERM) para pedirle que termine de forma ordenada.

os.execv() : reemplazar el programa actual (Unix)

`execv()` reemplaza el programa que se esta ejecutando por otro. Es como si Python se transformara en otro programa.

```
>>> os.execv("/bin/ls", ["ls", "-l"])
```

Despues de esa linea, el programa Python desaparece y se ejecuta `ls -l`. No se vuelve atras. No hay vuelta atras.

os.chmod() : cambiar permisos (Unix)

En Linux/Mac, los archivos tienen permisos de lectura, escritura y ejecucion para el dueño, el grupo y otros. Con `chmod()` podes cambiarlos.

```
>>> os.chmod("mi_script.py", 0o755)
```

El 0o755 es un numero en octal (base 8) que representa los permisos: dueño puede leer, escribir y ejecutar; grupo y otros solo leer y ejecutar. En Windows no funciona igual (los permisos son muy distintos).

Proyecto integrador: un explorador de archivos simple

Vamos a juntar varias de estas funciones para hacer un mini explorador de archivos desde la terminal. Podes navegar por carpetas, ver contenido y saber que es archivo y que es carpeta.

```
import os

def explorador():
    ruta = os.getcwd()
    while True:
        print(f"\n[DIR] Directorio actual: {ruta}")
        print("Contenido:")
        with os.scandir(ruta) as entradas:
            for entrada in entradas:
                tipo = "[FILE]" if entrada.is_file() else "[DIR]"
                print(f"    {tipo} {entrada.name}")
        opcion = input("\n[1] Entrar a carpeta  [2] Subir  [3] Salir: ")
        if opcion == "1":
            nombre = input("Nombre de la carpeta: ")
            nueva = os.path.join(ruta, nombre)
```

```

        if os.path.isdir(nueva):
            ruta = nueva
        else:
            print("No es una carpeta valida.")
    elif opcion == "2":
        ruta = os.path.dirname(ruta)
    elif opcion == "3":
        print("Hasta luego!")
        break
    else:
        print("Opcion invalida")

if __name__ == "__main__":
    explorador()

```

Mejora propuesta: modifica el explorador para que muestre tambien el tamaño de los archivos (usa `entrada.stat().st_size`) y la fecha de modificacion formateada.

Tabla de referencia de funciones nuevas

Funcion	Que hace
<code>os.path.basename(ruta)</code>	Ultimo componente de la ruta (nombre del archivo)
<code>os.path.dirname(ruta)</code>	Carpeta padre
<code>os.path.abspath(ruta)</code>	Convierte a ruta absoluta
<code>os.path.splitext(ruta)</code>	Separa nombre y extension
<code>os.path.join(p1, p2, ...)</code>	Une partes con el separador correcto
<code>os.mkdir(carpeta)</code>	Crea una sola carpeta
<code>os.path.getmtime(ruta)</code>	Fecha de modificacion (timestamp)
<code>os.getenv(var)</code>	Valor de una variable de entorno
<code>os.environ</code>	Diccionario de todas las variables de entorno
<code>os.cpu_count()</code>	Numero de nucleos del procesador
<code>os.getcwd()</code>	'.' (directorio actual)
<code>os.pardir</code>	'..' (directorio padre)
<code>os.system(comando)</code>	Ejecuta un comando en la terminal
<code>os.scandir(ruta)</code>	Iterador eficiente con metadatos
<code>os.urandom(n)</code>	n bytes aleatorios seguros
<code>os.getpid()</code>	ID del proceso actual
<code>os.chmod(ruta, modo)</code>	Cambia permisos (Unix)
<code>os.fork()</code>	Crea un proceso hijo (Unix)
<code>os.kill(pid, senal)</code>	Envia una señal a un proceso (Unix)
<code>os.execv(programa, args)</code>	Reemplaza el proceso actual (Unix)

Ejercicios resueltos (20 ejercicios paso a paso)

Aca te dejo 20 ejercicios para que practiques. Cada ejercicio tiene su solucion explicada paso a paso. Intentá resolverlos por tu cuenta antes de mirar la solucion.

Ejercicio 1

Enunciado: Escribi una funcion que reciba una ruta y devuelva el nombre del archivo sin la extension (por ejemplo, para "foto.jpg" debe devolver "foto").

Solucion:

```
def nombre_sin_extension(ruta):
    # Primero obtenemos el nombre base (podria venir con carpetas)
    base = os.path.basename(ruta)
    # Luego separamos nombre y extension
    nombre, ext = os.path.splitext(base)
    return nombre

# Prueba:
print(nombre_sin_extension("/home/user/docs/foto.jpg")) # foto
```

Explicacion: Usamos `basename()` para quedarnos solo con el nombre del archivo (sin las carpetas). Luego `splitext()` divide en dos partes justo en el ultimo punto. La primera parte es lo que queremos.

Ejercicio 2

Enunciado: Mostra el PID del proceso actual y la cantidad de nucleos de la CPU.

Solucion:

```
import os
print(f"PID: {os.getpid()}")
print(f"Nucleos: {os.cpu_count()}")
```

Explicacion: `getpid()` devuelve el identificador del proceso actual. `cpu_count()` devuelve la cantidad de nucleos logicos (incluyendo hyperthreading).

Ejercicio 3

Enunciado: Crea una carpeta llamada "mis_archivos" si no existe.

Solucion:

```
carpeta = "mis_archivos"
if not os.path.exists(carpeta):
    os.mkdir(carpeta)
    print("Carpeta creada")
else:
    print("Ya existe")
```

Explicacion: Primero verificamos si la carpeta ya existe con `exists()`. Si no existe, la creamos con `mkdir()`.

Ejercicio 4

Enunciado: Recorre la carpeta actual y muestra solo los archivos `.txt`. Usa `scandir()`.

Solucion:

```
with os.scandir(".") as entradas:
    for entrada in entradas:
        if entrada.is_file() and entrada.name.endswith(".txt"):
            print(entrada.name)
```

Explicacion: `scandir()` nos da objetos `DirEntry` que tienen metodo `is_file()` y el atributo `name`. Filtramos los que terminan con `".txt"`.

Ejercicio 5

Enunciado: Muestra la fecha de ultima modificacion del archivo `"datos.txt"` en formato legible (aaaa-mm-dd hh:mm:ss).

Solucion:

```
import datetime
timestamp = os.path.getmtime("datos.txt")
fecha = datetime.datetime.fromtimestamp(timestamp)
print(fecha.strftime("%Y-%m-%d %H:%M:%S"))
```

Explicacion: `getmtime()` devuelve un timestamp. `fromtimestamp()` lo convierte en objeto `datetime`. Luego `strftime()` formatea como queremos.

Ejercicio 6

Enunciado: Obtene el valor de la variable de entorno `PATH` y muestra cada carpeta en una linea aparte.

Solucion:

```
path = os.getenv("PATH")
if path:
    for carpeta in path.split(os.pathsep):
        print(carpeta)
```

Explicacion: `os.pathsep` es `':'` en Linux/Mac y `';'` en Windows. Con `split()` separamos la cadena en partes.

Ejercicio 7

Enunciado: Escribe un programa que limpie la pantalla sin importar el sistema operativo.

Solucion:

```
if os.name == 'nt':
    os.system("cls")
else:
    os.system("clear")
```

Explicacion: En Windows el comando es `cls`, en Linux/Mac es `clear`.

Ejercicio 8

Enunciado: Genera un token aleatorio de 32 bytes y mostralo en hexadecimal.

Solucion:

```
token = os.urandom(32)
print(token.hex())
```

Explicacion: `urandom(32)` devuelve 32 bytes aleatorios. `hex()` los convierte a una cadena hexadecimal (64 caracteres).

Ejercicio 9

Enunciado: Renombra el archivo "viejo.txt" a "nuevo.txt" si existe.

Solucion:

```
if os.path.exists("viejo.txt"):
    os.rename("viejo.txt", "nuevo.txt")
    print("Renombrado")
else:
    print("El archivo no existe")
```

Explicacion: `rename()` cambia el nombre. Siempre verificar que el origen existe antes.

Ejercicio 10

Enunciado: Usando `scandir()`, calcula el tamaño total (en bytes) de todos los archivos de la carpeta actual.

Solucion:

```
total = 0
with os.scandir(".") as entradas:
    for entrada in entradas:
        if entrada.is_file():
            total += entrada.stat().st_size
print(f"Tamaño total: {total} bytes")
```

Explicacion: Cada `DirEntry` tiene un metodo `stat()` que devuelve un objeto con atributos como `st_size` (tamaño en bytes). Sumamos todos.

Ejercicio 11

Enunciado: Convierte la ruta relativa "docs/foto.png" a absoluta.

Solucion:

```
ruta_abs = os.path.abspath("docs/foto.png")
print(ruta_abs)
```

Explicacion: `abspath()` toma una ruta (relativa o absoluta) y devuelve su version absoluta desde la raiz del sistema de archivos.

Ejercicio 12

Enunciado: Obtiene la carpeta padre de `./a/b/c/archivo.txt`.

Solucion:

```
ruta = "a/b/c/archivo.txt"
padre = os.path.dirname(ruta)
print(padre) # a/b/c
```

Explicacion: `dirname()` devuelve todo lo que esta antes de la ultima barra.

Ejercicio 13

Enunciado: Borra el archivo `temp.txt` solo si el usuario confirma escribiendo `"SI"`.

Solucion:

```
if os.path.exists("temp.txt"):
    resp = input("Escribi SI para borrar: ")
    if resp == "SI":
        os.remove("temp.txt")
        print("Borrado")
    else:
        print("No se borro")
else:
    print("No existe")
```

Explicacion: Pedimos confirmacion explicita con una palabra clave. Nunca borres sin preguntar.

Ejercicio 14

Enunciado: Muestra cuantos archivos y cuantas carpetas hay en la carpeta actual usando `scandir()`.

Solucion:

```
archivos = 0
carpetas = 0
with os.scandir(".") as entradas:
    for entrada in entradas:
        if entrada.is_file():
            archivos += 1
        elif entrada.is_dir():
            carpetas += 1
print(f"Archivos: {archivos}, Carpetas: {carpetas}")
```

Explicacion: Recorremos con `scandir()` y contamos segun `is_file()` y `is_dir()`.

Ejercicio 15

Enunciado: Escribi una funcion que reciba una carpeta y devuelva una lista con los nombres de todos los archivos .py que contiene (sin la extension).

Solucion:

```
def listar_py(carpeta):
    resultado = []
    with os.scandir(carpeta) as entradas:
        for entrada in entradas:
            if entrada.is_file() and entrada.name.endswith(".py"):
                nombre, ext = os.path.splitext(entrada.name)
                resultado.append(nombre)
    return resultado
```

Explicacion: Filtramos por extension .py y luego usamos `splitext()` para quedarnos solo con el nombre.

Ejercicio 16

Enunciado: Usa `os.system()` para mostrar el calendario del mes actual. En Linux/Mac el comando es `cal`; en Windows podes usar `get-date` (PowerShell) o instalar `cal`. Hacelo portable.

Solucion:

```
if os.name == 'nt':
    os.system("powershell -Command \"Get-Date -Format 'MMMM yyyy'; cal\"")
else:
    os.system("cal")
```

Explicacion: Detectamos el SO y ejecutamos el comando apropiado. En Windows llamamos a PowerShell.

Ejercicio 17

Enunciado: Crea la siguiente estructura de carpetas: "proyecto/src", "proyecto/docs", "proyecto/tests" usando una sola linea.

Solucion:

```
os.makedirs("proyecto/src", exist_ok=True)
os.makedirs("proyecto/docs", exist_ok=True)
os.makedirs("proyecto/tests", exist_ok=True)
```

O bien:

```
for carpeta in ["src", "docs", "tests"]:
    os.makedirs(os.path.join("proyecto", carpeta), exist_ok=True)
```

Explicacion: `makedirs()` con `exist_ok=True` no da error si ya existe. Creamos cada subcarpeta dentro de "proyecto".

Ejercicio 18

Enunciado: Escribi un programa que muestre el tamaño de cada archivo en la carpeta actual, ordenado de menor a mayor.

Solucion:

```
archivos = []
with os.scandir(".") as entradas:
    for entrada in entradas:
        if entrada.is_file():
            archivos.append((entrada.name, entrada.stat().st_size))
archivos.sort(key=lambda x: x[1]) # ordena por tamaño
for nombre, tamaño in archivos:
    print(f"{nombre}: {tamaño} bytes")
```

Explicacion: Guardamos tuplas (nombre, tamaño) en una lista, luego ordenamos con `sort()` usando el segundo elemento de la tupla como clave.

Ejercicio 19

Enunciado: Crea un archivo temporal con un nombre unico (por ejemplo "temp_12345.txt") que incluya el PID y la fecha actual en el nombre.

Solucion:

```
import datetime
pid = os.getpid()
fecha = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
nombre = f"temp_{pid}_{fecha}.txt"
with open(nombre, "w") as f:
    f.write("contenido temporal")
print(f"Creado {nombre}")
```

Explicacion: Usamos el PID y una marca de tiempo para asegurar unicidad. Luego creamos el archivo con `open()`.

Ejercicio 20

Enunciado: (Desafio) Implementa una funcion recursiva que recorra una carpeta y todas sus subcarpetas, y devuelva una lista con las rutas de todos los archivos .jpg encontrados.

Solucion:

```
def buscar_jpg(ruta):
    jpgs = []
    with os.scandir(ruta) as entradas:
        for entrada in entradas:
            ruta_completa = os.path.join(ruta, entrada.name)
            if entrada.is_file() and entrada.name.lower().endswith(".jpg"):
                jpgs.append(ruta_completa)
            elif entrada.is_dir():
                jpgs.extend(buscar_jpg(ruta_completa))
    return jpgs
```

```
# Uso:
imagenes = buscar_jpg(".")
for img in imagenes:
    print(img)
```

Explicacion: Funcion recursiva: para cada entrada, si es archivo .jpg lo agregamos; si es carpeta, llamamos a la misma funcion dentro de esa carpeta y agregamos sus resultados con `extend()`.

Resumen de lo que vimos

Dale, repasemos rapido lo importante:

- `basename()`, `dirname()`, `splitext()`, `abspath()` y `join()` te ayudan a manipular rutas sin ensuciarte las manos con barras o backslashes.
- `mkdir()` crea una carpeta; `makedirs()` crea varias de una vez.
- `getmtime()` te da la fecha de modificacion de un archivo (en formato timestamp).
- Podes leer variables de entorno con `getenv()` y el diccionario `environ`.
- `system()` ejecuta comandos de la terminal, pero usalo con cuidado.
- `scandir()` es mas eficiente que `listdir()` cuando necesitas saber si algo es archivo o carpeta.
- `urandom()` es la forma segura de generar bytes aleatorios (para claves, tokens, etc.).
- Las funciones de procesos (`fork()`, `kill()`, `execv()`) son avanzadas y tipicas de Unix. No te asustes si no las usas al principio.

Que sigue?

Ya manejas el sistema de archivos como un profesional. Podes crear, borrar, renombrar, listar, y hasta ejecutar comandos desde Python. El proximo paso logico es aprender a leer y escribir archivos de texto, CSV, JSON, etc. Ahora que sabes moverte por el sistema, leer y escribir archivos va a ser pan comido.

Pero antes de seguir, practica con estos ejercicios. Hacele modificaciones al explorador de archivos, agregale opciones para borrar o renombrar desde el mismo programa. Rompe cosas (en una carpeta de prueba, no en tu escritorio). La mejor forma de aprender es equivocandote.

Conocer el sistema operativo desde adentro te da un poder enorme. Pero con grandes poderes... ya sabes, anda con cuidado al borrar.

Sistema Operativo - Parte III: Depuración, mejora de código y proyectos reales

Querido lector, vamos a ensuciarnos las manos

Hasta ahora vimos muchas funciones del módulo `os`: como movernos entre carpetas, crear directorios, renombrar archivos, ejecutar comandos... Pero hay una cosa que no te voy a mentir: al principio, tu código va a tener errores. Y no está mal. Lo malo es no saber como encontrar esos errores.

En este capítulo vamos a hacer algo distinto. No voy a darte un código perfecto y lindo. Te voy a mostrar un código REAL que encuentre (y lo escribí yo mismo cuando estaba aprendiendo). Tiene errores, tiene cosas mejorables, y vamos a depurarlo juntos.

La idea es esta: agarrar un programa que YA ESTÁ ESCRITO (pero medio mal), entender por qué anda mal o lento, y mejorarlo paso a paso. Así se aprende de verdad.

El código que vamos a depurar

Mira este código. Es un programa que intenta encontrar una frase mezclando letras al azar. Como un mono con una máquina de escribir, pero más lento.

```
import random
import time
import os

letras = list("nqtuesoroeoaytldsb")
objetivo = "esotuboque doler stanly"
intentos = 0

while True:
    combinacion = ""
    random.shuffle(letras)
    combinacion = "".join(letras)
    intentos += 1
    os.system("clear")
    print("=" * 50)
    print(" BUSCANDO LA FRASE ")
    print("=" * 50)
    print(f"\nIntentos: {intentos}\n")
    for letra in combinacion:
```

```
print(letra, end="", flush=True)
print("\n")
if combinacion == objetivo:
    print("FRASE ENCONTRADA!")
    break
```

Ves algo raro? Funcionara? Probemos a pensar juntos.

El primer problema: cuantas letras hay?

Fijate que la lista `letras` tiene esta pinta: `"nqtuesoroeoaytldsb"`. Contemos: cuantas letras hay? (Podes contar con los dedos como hacia yo).

La frase objetivo es `."esotubokedolerstanly"`. Tambien contemos esas.

En criollo: si tenes menos letras en el mazo que las que necesitas para la frase, NUNCA vas a poder formarla. Es como querer armar un rompecabezas de 100 piezas con solo 80. No hay chance.

Error 1: Cantidad de letras incorrecta. En este codigo faltan letras.

El segundo problema: la indentacion

Mira el `for letra in combinacion:`. Dentro de ese bucle, hay un `if combinacion == objetivo:`. Pero fijate que la condicion se evalua **dentro del bucle** que imprime letra por letra. O sea, por cada letra que imprime, pregunta si ya encontro la frase. Eso no tiene sentido. Deberia preguntar UNA SOLA VEZ por intento, no 21 veces por intento.

Error 2: La condicion de salida esta mal indentada.

El tercer problema: cuando se corta el while?

El `while True` nunca termina. El `break` esta dentro del `if` que esta dentro del `for`. Pero si nunca se cumple la condicion, el programa sigue para siempre. Y si se cumple, corta el `for`, pero el `while` sigue. Otra vez, mal.

Error 3: El break no sale del bucle principal.

El cuarto problema: el programa duerme mas de lo necesario

Hay un `time.sleep(0.1)` al final que no tiene sentido. Ademas, dentro del `for` hay un `print(letra, end=, flush=True)` y despues un `print("")` que hace saltos de linea raros. El codigo imprime cosas que no aportan.

Error 4: Eficiencia y legibilidad.

Vamos a arreglarlo paso a paso

Ahora que sabemos los problemas, encaremoslos como quien arregla una bicicleta: una cosa por vez.

Paso 1: Asegurar la misma cantidad de letras

Primero, aseguremonos de que la lista de letras tenga las mismas letras que la frase objetivo, aunque esten desordenadas.

```
import random
import os

# La frase que queremos formar
objetivo = "esotubo quedo doler stanly"
# Sacamos los espacios para trabajar sin ellos
objetivo_sin_espacios = objetivo.replace(" ", "")

# Creamos una lista con las letras de la frase (desordenada)
letras = list(objetivo_sin_espacios)
# Mezclamos para que no este ordenada al principio
random.shuffle(letras)

print("Letras disponibles:", letras)
print("Letras necesarias:", len(letras))
print("Frase objetivo:", objetivo_sin_espacios)
```

Reflexion: Ahora si, la cantidad de letras es la misma que necesitamos. El mono tiene todas las piezas del rompecabezas.

Paso 2: Corregir la indentacion y el bucle

La condicion de victoria debe evaluarse UNA VEZ por intento, no dentro del `for`.

```
intentos = 0

while True:
    random.shuffle(letras)
    combinacion = "".join(letras)
    intentos += 1

    # Limpiar pantalla (compatible con Windows y Linux/Mac)
    os.system("cls" if os.name == "nt" else "clear")

    print("=" * 50)
    print(f" INTENTO NRO {intentos}")
    print("=" * 50)
    print(f"Frase actual: {combinacion}")

    # ACA ESTA LA CONDICION BIEN UBICADA!
    if combinacion == objetivo_sin_espacios:
        print("\nFRASE ENCONTRADA!")
        print(f"Logrado en {intentos} intentos")
        break
```

Paso 3: Agregar un limite de intentos

Esto es importante. Si la frase tiene muchas letras, las combinaciones posibles son miles de millones. Un programa podria tardar años. Vamos a poner un limite razonable.

```
LIMITE_INTENTOS = 1000000

while intentos < LIMITE_INTENTOS:
    random.shuffle(letras)
    combinacion = "".join(letras)
    intentos += 1

    if combinacion == objetivo_sin_espacios:
        print(f"\nENCONTRADO en {intentos} intentos!")
        break

    if intentos % 10000 == 0:
        print(f"Intentos hasta ahora: {intentos}")
else:
    print(f"\nNo se encontro despues de {LIMITE_INTENTOS} intentos")
    print("La fuerza bruta a veces no alcanza")
```

Paso 4: Version final mejorada

Aca te dejo el codigo completo, limpio y funcionando:

```
import random
import os
import sys

def limpiar_pantalla():
    """Limpia la pantalla segun el sistema operativo"""
    os.system("cls" if os.name == "nt" else "clear")

def buscar_frase_por_fuerza_bruta(frase, limite=1000000):
    """
    Busca una frase mezclando letras al azar.
    Frase: string con la frase objetivo
    limite: cantidad maxima de intentos
    """
    # Sacamos los espacios para trabajar mas facil
    frase_original = frase
    frase_sin_espacios = frase.replace(" ", "")

    # Creamos el mazo de letras
    letras = list(frase_sin_espacios)
    random.shuffle(letras)

    intentos = 0
```

```
print("=" * 60)
print(" BUSCADOR DE FRASES POR FUERZA BRUTA ")
print("=" * 60)
print(f"Buscando: {frase_original}")
print(f"Letras disponibles: {len(letras)}")
print(f"Maximo de intentos: {limite}")
print("-" * 60)

while intentos < limite:
    # Mezclar y formar combinacion
    random.shuffle(letras)
    combinacion = "".join(letras)
    intentos += 1

    # Mostrar progreso cada cierto tiempo
    if intentos % 10000 == 0:
        limpiar_pantalla()
        print(f"Intentos realizados: {intentos}")
        print(f"Ultima combinacion: {combinacion[:30]}...")

    # Verificar si encontramos la frase
    if combinacion == frase_sin_espacios:
        limpiar_pantalla()
        print("\n" + "=" * 20)
        print("FRASE ENCONTRADA!")
        print("=" * 20)
        print(f"Frase: {frase_original}")
        print(f"Intentos necesarios: {intentos}")
        print(f"Combinacion final: {combinacion}")
        return True

    # Si llegamos aca, no se encontro
    print("\n" + "=" * 20)
    print("No se encontro la frase dentro del limite de intentos")
    print("=" * 20)
    return False

# Programa principal
if __name__ == "__main__":
    # Podemos probar con frases cortas primero
    buscar_frase_por_fuerza_bruta("hola mundo", limite=500000)
```


El código con importaciones externas (y como manejar errores)

En el código original había importaciones de un módulo llamado `funciones`. Esto es interesante: muestra como separar el código en archivos distintos.

Pero hay un problema: si el archivo `funciones.py` no existe, el programa se rompe. Vamos a ver como manejarlo con `try/except`.

```
import random
import os

# Intentamos importar funciones externas
try:
    import funciones
    TIENE_FUNCIONES = True
    print("Modulo 'funciones' encontrado. Funciones adicionales disponibles.")
except ImportError:
    TIENE_FUNCIONES = False
    print("Modulo 'funciones' NO encontrado. Solo funciones basicas.")

def buscar_frase(frase, limite=500000):
    # ... (mismo código de antes)

    if TIENE_FUNCIONES:
        funciones.registrar_intento(frase, intentos)

    return encontrado
```

Tabla de referencia: manejo de errores comunes con os

Error	Causa	Solucion
<code>FileNotFoundError</code>	El archivo no existe	Usar <code>os.path.exists()</code> antes
<code>OSError</code>	Permisos insuficientes	Verificar permisos con <code>os.access()</code>
<code>FileExistsError</code>	La carpeta ya existe	Usar <code>exist_ok=True</code> en <code>makedirs</code>
<code>NotADirectoryError</code>	Se esperaba carpeta y es archivo	Verificar con <code>os.path.isdir()</code>

20 ejercicios propuestos para practicar con os

Ejercicios nivel 1 (faciles)

Ejercicio 1: Escribe un programa que muestre la ruta completa del directorio actual y luego liste su contenido.

Ejercicio 2: Crea un programa que cree una carpeta llamada `mis_archivos` y dentro de ella, un archivo `README.txt` con el texto `"Este es mi proyecto"`.

Ejercicio 3: Hace un programa que pregunte al usuario un nombre de archivo y luego diga si existe o no.

Ejercicio 4: Muestra el tamaño en bytes de un archivo que elija el usuario (si existe).

Ejercicio 5: Crea un programa que renombre el archivo `viejo.txt` a `nuevo.txt` (manejando el error si no existe).

Ejercicios nivel 2 (intermedios)

Ejercicio 6: Escribe un programa que recorra la carpeta actual y muestre separados: primero todos los archivos, luego todas las carpetas.

Ejercicio 7: Crea una función que reciba una carpeta y devuelva cuantos archivos `.py` hay dentro.

Ejercicio 8: Hace un programa que cree la estructura: `proyecto/src`, `proyecto/docs`, `proyecto/tests`.

Ejercicio 9: Muestra las variables de entorno `PATH` y `HOME` (o `USERPROFILE` en Windows).

Ejercicio 10: Escribe un programa que limpie la pantalla usando `os.system()` según el sistema operativo.

Ejercicios nivel 3 (avanzados)

Ejercicio 11: Usando `os.walk()`, recorre una carpeta y muestra todos los archivos que pesen más de 1 MB.

Ejercicio 12: Crea un programa que ordene los archivos de una carpeta por fecha de modificación (del más nuevo al más viejo).

Ejercicio 13: Hace un programa que copie todos los archivos `.txt` de una carpeta a otra (usando `shutil.copy()`).

Ejercicio 14: Escribe un programa que elimine todas las carpetas vacías dentro de una carpeta dada.

Ejercicio 15: Usando `scandir()`, muestra el tamaño total de todos los archivos de una carpeta.

Ejercicios nivel 4 (proyectos)

Ejercicio 16: Crea un "buscador de archivos" que reciba un nombre parcial y muestre todas las rutas donde aparece.

Ejercicio 17: Hace un programa que organice los archivos de una carpeta por extensión (mover `.jpg` a `Imágenes/`, `.pdf` a `Documentos/`, etc.).

Ejercicio 18: Escribe un "limpiador de temporales" que borre archivos con más de 30 días sin modificarse.

Ejercicio 19: Crea un programa que genere un informe en un archivo `informe.txt` con la lista de todos los archivos y carpetas del sistema (empezando desde una raíz).

Ejercicio 20: Hace un "explorador de archivos interactivo" con menú: 1) Listar, 2) Entrar a carpeta, 3) Subir, 4) Ver tamaño de archivo, 5) Salir.

Soluciones explicadas paso a paso

Solucion Ejercicio 1

```
import os
```

```
# Mostrar ruta actual
ruta_actual = os.getcwd()
print("Directorio actual:", ruta_actual)

# Mostrar contenido
contenido = os.listdir()
print("\nContenido:")
for elemento in contenido:
    # Usamos un operador ternario para mostrar icono segun tipo
    icono = "ARCHIVO" if os.path.isfile(elemento) else "CARPETA"
    print(f" [{icono}] {elemento}")
```

Explicacion: `getcwd()` obtiene la ruta actual. `listdir()` devuelve una lista. Luego recorreremos cada elemento y preguntamos si es archivo o carpeta con `isfile()` para mostrar un icono distinto.

Solucion Ejercicio 2

```
import os

# Crear carpeta
nombre_carpeta = "mis_archivos"
os.makedirs(nombre_carpeta, exist_ok=True)

# Crear archivo dentro de la carpeta
ruta_archivo = os.path.join(nombre_carpeta, "README.txt")
with open(ruta_archivo, "w") as archivo:
    archivo.write("Este es mi proyecto")

print(f"Carpeta '{nombre_carpeta}' y archivo creados")
```

Explicacion: `makedirs(..., exist_ok=True)` crea la carpeta sin errores si ya existe. `os.path.join()` arma la ruta completa. Con `open()` en modo escritura ("w") creamos el archivo.

Solucion Ejercicio 3

```
import os

nombre = input("Ingrese el nombre del archivo: ")

if os.path.exists(nombre):
    print(f"El archivo '{nombre}' EXISTE")
    if os.path.isfile(nombre):
        print("Es un archivo")
    elif os.path.isdir(nombre):
        print("Es una carpeta")
else:
    print(f"El archivo '{nombre}' NO EXISTE")
```

Explicacion: `exists()` es la funcion principal. Luego, si existe, diferenciamos si es archivo o carpeta con `isfile()` e `isdir()`.

Solucion Ejercicio 4

```
import os

nombre = input("Ingrese el nombre del archivo: ")

if os.path.exists(nombre) and os.path.isfile(nombre):
    tamano = os.path.getsize(nombre)
    print(f"Tamano: {tamano} bytes")
    print(f"Tamano: {tamano / 1024:.2f} KB")
    print(f"Tamano: {tamano / (1024*1024):.2f} MB")
else:
    print("El archivo no existe o no es valido")
```

Explicacion: `getsize()` devuelve bytes. Para mostrarlo mas amigable, dividimos por 1024 para KB y por 1024*1024 para MB.

Solucion Ejercicio 5

```
import os

viejo = "viejo.txt"
nuevo = "nuevo.txt"

try:
    os.rename(viejo, nuevo)
    print(f"Renombrado: {viejo} -> {nuevo}")
except FileNotFoundError:
    print(f"El archivo '{viejo}' no existe")
except PermissionError:
    print(f"No tenes permisos para renombrar")
except Exception as e:
    print(f"Error inesperado: {e}")
```

Explicacion: Usamos `try/except` para capturar errores especificos. `FileNotFoundError` si no existe, `PermissionError` si no tenemos permisos.

Solucion Ejercicio 6

```
import os

archivos = []
carpetas = []

for elemento in os.listdir():
    if os.path.isfile(elemento):
```

```
        archivos.append(elemento)
    elif os.path.isdir(elemento):
        carpetas.append(elemento)

print("ARCHIVOS:")
for a in archivos:
    print(f" - {a}")

print("\nCARPETAS:")
for c in carpetas:
    print(f" - {c}")
```

Explicacion: Creamos dos listas vacias. Recorremos el contenido y segun el tipo, agregamos a una lista u otra. Al final mostramos cada lista por separado.

Solucion Ejercicio 7

```
import os

def contar_py(carpeta):
    cantidad = 0
    try:
        for elemento in os.listdir(carpeta):
            ruta_completa = os.path.join(carpeta, elemento)
            if os.path.isfile(ruta_completa) and elemento.endswith(".py"):
                cantidad += 1
    except FileNotFoundError:
        print(f"La carpeta '{carpeta}' no existe")
        return -1
    return cantidad

# Prueba
carpeta = input("Carpeta a analizar: ")
resultado = contar_py(carpeta)
if resultado >= 0:
    print(f"Archivos .py encontrados: {resultado}")
```

Explicacion: `endswith(".py")` verifica la extension. `os.path.join()` arma la ruta completa para las verificaciones.

Solucion Ejercicio 8

```
import os

base = "proyecto"
subcarpetas = ["src", "docs", "tests"]

# Crear carpeta principal
os.makedirs(base, exist_ok=True)
```

```
# Crear subcarpetas
for sub in subcarpetas:
    ruta = os.path.join(base, sub)
    os.makedirs(ruta, exist_ok=True)
    print(f"Creada: {ruta}")

# Verificar
print("\nEstructura creada:")
for raiz, dirs, archivos in os.walk(base):
    nivel = raiz.count(os.sep)
    indentacion = " " * nivel
    print(f"{indentacion}[{os.path.basename(raiz)}]")
```

Explicacion: `os.walk()` recorre toda la estructura. `count(os.sep)` calcula la profundidad para indentar.

Solucion Ejercicio 9

```
import os

# Variables segun sistema operativo
if os.name == "nt": # Windows
    path = os.getenv("PATH")
    home = os.getenv("USERPROFILE")
else: # Linux/Mac
    path = os.getenv("PATH")
    home = os.getenv("HOME")

print("VARIABLES DE ENTORNO")
print("=" * 40)
print(f"PATH: {path[:100]}...") # Mostramos solo el principio
print(f"HOME/USERPROFILE: {home}")

# Mostrar todas las variables (opcional)
print("\nTodas las variables:")
for clave, valor in list(os.environ.items())[:5]:
    print(f" {clave} = {valor[:50]}...")
```

Explicacion: `os.getenv()` obtiene variables especificas. `os.environ` es un diccionario con todas. Usamos `[:100]` para no saturar la pantalla.

Solucion Ejercicio 10

```
import os

def limpiar_pantalla():
    """Limpia la pantalla de forma compatible"""
    os.system("cls" if os.name == "nt" else "clear")
```

```
def pausar():
    """Pausa la ejecucion hasta que el usuario presione Enter"""
    input("\nPresiona Enter para continuar...")

# Demostracion
print("Este texto va a desaparecer...")
pausar()
limpiar_pantalla()
print("Pantalla limpiada!")
print("Ahora aparece este mensaje.")
```

Explicacion: Usamos un operador ternario: `cls if os.name == "nt" else clear`. Si es Windows ('nt'), usa `cls`; si no (Linux/Mac), usa `clear`.

Solucion Ejercicio 11

```
import os

def buscar_grandes(ruta_inicial, limite_mb=1):
    limite_bytes = limite_mb * 1024 * 1024
    grandes = []

    for raiz, carpetas, archivos in os.walk(ruta_inicial):
        for archivo in archivos:
            ruta_completa = os.path.join(raiz, archivo)
            try:
                tamano = os.path.getsize(ruta_completa)
                if tamano > limite_bytes:
                    grandes.append((ruta_completa, tamano))
            except (OSError, PermissionError):
                # Ignoramos archivos que no podemos leer
                pass

    return grandes

# Usar la funcion
ruta = input("Ruta a analizar: ")
if os.path.exists(ruta):
    grandes = buscar_grandes(ruta, 1)
    print(f"\nArchivos de mas de 1 MB encontrados: {len(grandes)}")
    for archivo, tamano in grandes[:10]: # Mostramos hasta 10
        print(f" - {archivo} ({tamano / (1024*1024):.2f} MB)")
else:
    print("Ruta no existe")
```

Explicacion: `os.walk()` recorre recursivamente. Por cada archivo, obtenemos su tamaño con `getsize()` y lo comparamos con el límite. Manejamos errores de permisos con `try/except`.

Solucion Ejercicio 12

```
import os
import time

def archivos_por_fecha(carpeta):
    archivos_con_fecha = []

    for elemento in os.listdir(carpeta):
        ruta = os.path.join(carpeta, elemento)
        if os.path.isfile(ruta):
            mtime = os.path.getmtime(ruta)
            archivos_con_fecha.append((elemento, mtime, ruta))

    # Ordenar por fecha (mas reciente primero)
    archivos_con_fecha.sort(key=lambda x: x[1], reverse=True)
    return archivos_con_fecha

# Usar
carpeta = input("Carpeta a analizar: ")
if os.path.exists(carpeta):
    archivos = archivos_por_fecha(carpeta)
    print("\nArchivos ordenados por fecha (mas nuevo primero):")
    for i, (nombre, fecha, ruta) in enumerate(archivos[:10]):
        fecha_legible = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(fecha))
        print(f"{i+1}. {nombre} - {fecha_legible}")
else:
    print("Carpeta no existe")
```

Explicacion: `getmtime()` devuelve timestamp. Guardamos tuplas (nombre, fecha, ruta). Usamos `sort()` con una funcion lambda para ordenar por fecha descendente. `time.strftime()` convierte a formato legible.

Solucion Ejercicio 13

```
import os
import shutil

def copiar_txt(origen, destino):
    # Crear carpeta destino si no existe
    os.makedirs(destino, exist_ok=True)

    copiados = 0
    errores = 0

    for elemento in os.listdir(origen):
        ruta_origen = os.path.join(origen, elemento)
        if os.path.isfile(ruta_origen) and elemento.endswith(".txt"):
            ruta_destino = os.path.join(destino, elemento)
```



```
try:
    shutil.copy2(ruta_origen, ruta_destino)
    print(f"Copiado: {elemento}")
    copiados += 1
except Exception as e:
    print(f"Error copiando {elemento}: {e}")
    errores += 1

print(f"\nResumen: {copiados} copiados, {errores} errores")
return copiados

# Usar
carpeta_origen = input("Carpeta origen: ")
carpeta_destino = input("Carpeta destino: ")

if os.path.exists(carpeta_origen):
    copiar_txt(carpeta_origen, carpeta_destino)
else:
    print("Carpeta origen no existe")
```

Explicacion: `shutil.copy2()` copia archivos preservando metadatos. Verificamos que sea archivo con `isfile()` y extension `.txt` con `endswith()`.

Solucion Ejercicio 14

```
import os

def eliminar_carpetas_vacias(ruta):
    eliminadas = 0
    errores = 0

    # Primero recorremos de adentro hacia afuera con walk
    for raiz, carpetas, archivos in os.walk(ruta, topdown=False):
        if not archivos and not carpetas:
            # La carpeta esta vacia
            try:
                os.rmdir(raiz)
                print(f"Eliminada carpeta vacia: {raiz}")
                eliminadas += 1
            except OSError as e:
                print(f"No se pudo eliminar {raiz}: {e}")
                errores += 1

    print(f"\nResumen: {eliminadas} carpetas vacias eliminadas")
    return eliminadas

# Usar con cuidado!
ruta = input("Ruta a limpiar: ")
if os.path.exists(ruta) and os.path.isdir(ruta):
```

```
confirmar = input(f"Eliminar carpetas vacias en {ruta}? (s/n): ")
if confirmar.lower() == "s":
    eliminar_carpetas_vacias(ruta)
else:
    print("Operacion cancelada")
else:
    print("Ruta no valida")
```

Explicacion: `topdown=False` en `walk()` hace que recorra de adentro hacia afuera, así podemos eliminar carpetas hijas antes que las padres. Verificamos que no haya archivos ni carpetas para saber si esta vacia.

Solucion Ejercicio 15

```
import os

def calcular_tamano_total(carpetas):
    tamaño_total = 0
    cantidad_archivos = 0

    for raiz, carpetas, archivos in os.walk(carpetas):
        for archivo in archivos:
            ruta = os.path.join(raiz, archivo)
            try:
                tamaño_total += os.path.getsize(ruta)
                cantidad_archivos += 1
            except (OSError, PermissionError):
                # Ignoramos archivos que no podemos leer
                pass

    return tamaño_total, cantidad_archivos

def formato_tamano(bytes):
    """Convierte bytes a formato legible (KB, MB, GB)"""
    for unidad in ['B', 'KB', 'MB', 'GB', 'TB']:
        if bytes < 1024.0:
            return f"{bytes:.2f} {unidad}"
        bytes /= 1024.0
    return f"{bytes:.2f} PB"

# Usar
carpetas = input("Carpetas a analizar: ")
if os.path.exists(carpetas):
    tamaño, cantidad = calcular_tamano_total(carpetas)
    print(f"\nESTADÍSTICAS DE '{carpetas}'")
    print("=" * 40)
    print(f"Cantidad de archivos: {cantidad}")
    print(f"Tamaño total: {formato_tamano(tamaño)}")
else:
```

```
print("Carpeta no existe")
```

Explicacion: Recorremos con `walk()`, sumamos tamanios con `getsize()`. La funcion `formato_tamano()` convierte bytes a KB, MB, GB automaticamente.

Solucion Ejercicio 16

```
import os

def buscar_archivo(ruta_inicial, nombre_parcial):
    resultados = []

    for raiz, carpetas, archivos in os.walk(ruta_inicial):
        for archivo in archivos:
            if nombre_parcial.lower() in archivo.lower():
                ruta_completa = os.path.join(raiz, archivo)
                tamano = os.path.getsize(ruta_completa)
                resultados.append({
                    'nombre': archivo,
                    'ruta': ruta_completa,
                    'tamano': tamano
                })

    return resultados

# Usar
busqueda = input("Buscar archivo (nombre parcial): ")
ruta = input("Ruta donde buscar: ")

if os.path.exists(ruta):
    print(f"\nBuscando '{busqueda}' en {ruta}...")
    resultados = buscar_archivo(ruta, busqueda)

    if resultados:
        print(f"\nEncontrados {len(resultados)} archivos:")
        for i, r in enumerate(resultados[:20]):
            tamano_kb = r['tamano'] / 1024
            print(f"{i+1}. [*] {r['nombre']}")
            print(f"    Ruta: {r['ruta']}")
            print(f"    Tamano: {tamano_kb:.2f} KB")
            print()
    else:
        print("No se encontraron archivos")
else:
    print("La ruta no existe")
```

Explicacion: Recorremos recursivamente con `walk()`. Usamos `in` para verificar si el nombre parcial esta contenido. Convertimos a minusculas con `lower()` para que la busqueda no distinga mayusculas.

Solucion Ejercicio 17

```
import os
import shutil

def organizar_por_extension(carpeta):
    # Mapeo de extensiones a carpetas de destino
    extensiones = {
        '.jpg': 'Imágenes',
        '.jpeg': 'Imágenes',
        '.png': 'Imágenes',
        '.gif': 'Imágenes',
        '.pdf': 'Documentos',
        '.doc': 'Documentos',
        '.docx': 'Documentos',
        '.txt': 'Textos',
        '.py': 'Python',
        '.js': 'JavaScript',
        '.html': 'Web',
        '.css': 'Web',
    }

    movidos = 0

    for elemento in os.listdir(carpeta):
        ruta_origen = os.path.join(carpeta, elemento)
        if os.path.isfile(ruta_origen):
            nombre, ext = os.path.splitext(elemento)
            ext = ext.lower()

            if ext in extensiones:
                carpeta_destino = os.path.join(carpeta, extensiones[ext])
                os.makedirs(carpeta_destino, exist_ok=True)
                ruta_destino = os.path.join(carpeta_destino, elemento)

                # Si ya existe, agregamos un numero
                if os.path.exists(ruta_destino):
                    contador = 1
                    while True:
                        nuevo_nombre = f"{nombre}_{contador}{ext}"
                        ruta_destino = os.path.join(carpeta_destino, nuevo_nombre)
                        if not os.path.exists(ruta_destino):
                            break
                    contador += 1

                shutil.move(ruta_origen, ruta_destino)
                print(f"Movido: {elemento} -> {extensiones[ext]}/")
                movidos += 1
```

```
print(f"\nOrganizacion completada: {movidos} archivos movidos")

# Usar (con cuidado)
carpeta = input("Carpeta a organizar: ")
if os.path.exists(carpeta):
    confirmar = input(f"Organizar {carpeta} por extension? (s/n): ")
    if confirmar.lower() == "s":
        organizar_por_extension(carpeta)
    else:
        print("Operacion cancelada")
else:
    print("Carpeta no existe")
```

Explicacion: Usamos un diccionario que mapea extensiones a nombres de carpetas. `splitext()` separa nombre y extension. Manejamos conflictos de nombres agregando un numero. `shutil.move()` mueve archivos entre carpetas.

Solucion Ejercicio 18

```
import os
import time

def limpiar_temporales(ruta, dias=30):
    """
    Elimina archivos con mas de 'dias' sin modificarse
    """
    ahora = time.time()
    limite = ahora - (dias * 24 * 60 * 60) # segundos en 'dias'

    eliminados = 0
    errores = 0

    for raiz, carpetas, archivos in os.walk(ruta):
        for archivo in archivos:
            ruta_completa = os.path.join(raiz, archivo)
            try:
                mtime = os.path.getmtime(ruta_completa)
                if mtime < limite:
                    os.remove(ruta_completa)
                    eliminados += 1
                    print(f"Eliminado: {ruta_completa}")
            except PermissionError:
                print(f"Sin permisos: {ruta_completa}")
                errores += 1
            except Exception as e:
                print(f"Error con {ruta_completa}: {e}")
                errores += 1

    print(f"\nResumen: {eliminados} eliminados, {errores} errores")
```

```

    return eliminados

# Usar con MUCHO CUIDADO
print("LIMPIADOR DE ARCHIVOS TEMPORALES")
print("Este programa elimina archivos viejos. No hay vuelta atras!")
print()

ruta = input("Ruta a limpiar: ")
if os.path.exists(ruta):
    try:
        dias = int(input("Eliminar archivos con mas de (dias): "))
        if dias <= 0:
            print("Cantidad de dias invalida")
        else:
            confirmar = input(f"Eliminar archivos con más de {dias} dias en {ruta}? (s/n)")
            if confirmar.lower() == "s":
                limpiar_temporales(ruta, dias)
            else:
                print("Operacion cancelada")
    except ValueError:
        print("Ingrese un numero valido")
else:
    print("Ruta no existe")

```

Explicacion: Calculamos el timestamp limite: `ahora - (dias * 86400)`. Comparamos con `getmtime()` de cada archivo. Si es mas viejo, lo eliminamos con `remove()`. Manejamos permisos con `try/except`.

Solucion Ejercicio 19

```

import os
import datetime

def generar_informe(ruta_inicial, archivo_salida="informe.txt"):
    """
    Genera un informe de todos los archivos y carpetas
    """
    total_archivos = 0
    total_carpetas = 0
    total_bytes = 0

    with open(archivo_salida, "w", encoding="utf-8") as informe:
        informe.write("=" * 60 + "\n")
        informe.write("INFORME DEL SISTEMA DE ARCHIVOS\n")
        informe.write(f"Fecha: {datetime.datetime.now()}\n")
        informe.write(f"Ruta inicial: {ruta_inicial}\n")
        informe.write("=" * 60 + "\n\n")

    for raiz, carpetas, archivos in os.walk(ruta_inicial):

```

```
# Escribir la carpeta actual
nivel = raiz.count(os.sep)
indentacion = "  " * nivel
informe.write(f"{indentacion}[{os.path.basename(raiz)}]/\n")
total_carpetas += 1

# Escribir archivos dentro de la carpeta
for archivo in archivos:
    ruta_completa = os.path.join(raiz, archivo)
    try:
        tamano = os.path.getsize(ruta_completa)
        mtime = os.path.getmtime(ruta_completa)
        fecha = datetime.datetime.fromtimestamp(mtime).strftime("%Y-%m-%d %H:%M:%S")

        informe.write(f"{indentacion} - {archivo} ({tamano} bytes) - Mod: {fecha}\n")
        total_archivos += 1
        total_bytes += tamano
    except:
        informe.write(f"{indentacion} ? {archivo} (no se pudo leer)\n")

return total_archivos, total_carpetas, total_bytes

# Usar
ruta = input("Ruta a analizar: ")
if os.path.exists(ruta):
    print("Generando informe... (puede tomar tiempo)")
    archivos, carpetas, bytes_total = generar_informe(ruta, "informe.txt")

    print(f"\nInforme guardado en 'informe.txt'")
    print(f"Estadísticas:")
    print(f"  Archivos: {archivos}")
    print(f"  Carpetas: {carpetas}")
    print(f"  Tamaño total: {bytes_total / (1024*1024):.2f} MB")
else:
    print("Ruta no existe")
```

Explicación: Recorremos con `walk()`. Por cada carpeta, escribimos su nombre indentado según profundidad. Por cada archivo, escribimos nombre, tamaño y fecha de modificación. Todo se guarda en un archivo de texto.

Solucion Ejercicio 20

```
import os

def limpiar_pantalla():
    os.system("cls" if os.name == "nt" else "clear")

def mostrar_menu():
    print("\n" + "=" * 50)
```

```

print(" EXPLORADOR DE ARCHIVOS INTERACTIVO ")
print("=" * 50)
print(" 1. Listar contenido de carpeta actual")
print(" 2. Entrar a una carpeta")
print(" 3. Subir al directorio padre")
print(" 4. Ver tamaño de un archivo")
print(" 5. Informacion de la carpeta actual")
print(" 6. Eliminar archivo (con confirmacion)")
print(" 7. Crear carpeta nueva")
print(" 8. Salir")
print("=" * 50)

def listar_contenido(ruta):
    print(f"\nContenido de: {ruta}")
    print("-" * 40)

    archivos = []
    carpetas = []

    try:
        for elemento in os.listdir(ruta):
            ruta_elemento = os.path.join(ruta, elemento)
            if os.path.isfile(ruta_elemento):
                archivos.append(elemento)
            else:
                carpetas.append(elemento)

        # Mostrar carpetas primero
        if carpetas:
            print("\nCARPETAS:")
            for c in sorted(carpetas):
                print(f"    [{c}]")

        if archivos:
            print("\nARCHIVOS:")
            for a in sorted(archivos):
                print(f"    {a}")

        if not carpetas and not archivos:
            print("    (La carpeta esta vacia)")
    except PermissionError:
        print("No tienes permisos para leer esta carpeta")

def entrar_carpeta(ruta):
    nombre = input("Nombre de la carpeta: ")
    nueva_ruta = os.path.join(ruta, nombre)
    if os.path.isdir(nueva_ruta):
        return nueva_ruta

```



```
else:
    print("No existe esa carpeta")
    return ruta

def ver_tamano(ruta):
    nombre = input("Nombre del archivo: ")
    ruta_archivo = os.path.join(ruta, nombre)
    if os.path.isfile(ruta_archivo):
        tamano = os.path.getsize(ruta_archivo)
        print(f"\n'{nombre}' pesa:")
        print(f"    {tamano} bytes")
        print(f"    {tamano / 1024:.2f} KB")
        print(f"    {tamano / (1024*1024):.2f} MB")
    else:
        print("No existe ese archivo")

def info_carpeta(ruta):
    archivos = 0
    carpetas = 0
    tamano_total = 0

    try:
        for elemento in os.listdir(ruta):
            ruta_elem = os.path.join(ruta, elemento)
            if os.path.isfile(ruta_elem):
                archivos += 1
                tamano_total += os.path.getsize(ruta_elem)
            else:
                carpetas += 1

        print(f"\nINFORMACION DE '{ruta}'")
        print("-" * 40)
        print(f"Archivos: {archivos}")
        print(f"Carpetas: {carpetas}")
        print(f"Tamano total: {tamano_total / (1024*1024):.2f} MB")
        print(f"Ruta completa: {os.path.abspath(ruta)}")
    except PermissionError:
        print("No se pudo leer la carpeta")

def eliminar_archivo(ruta):
    nombre = input("Nombre del archivo a eliminar: ")
    ruta_archivo = os.path.join(ruta, nombre)
    if os.path.isfile(ruta_archivo):
        tamano = os.path.getsize(ruta_archivo)
        print(f"Vas a eliminar '{nombre}' ({tamano} bytes)")
        confirmar = input("Estas seguro? (s/n): ")
        if confirmar.lower() == "s":
            os.remove(ruta_archivo)
```

```
        print("Archivo eliminado")
    else:
        print("Operacion cancelada")
else:
    print("No existe ese archivo")

def crear_carpeta(ruta):
    nombre = input("Nombre de la nueva carpeta: ")
    nueva_ruta = os.path.join(ruta, nombre)
    try:
        os.makedirs(nueva_ruta, exist_ok=False)
        print(f"Carpeta '{nombre}' creada")
    except FileExistsError:
        print("Ya existe una carpeta con ese nombre")
    except PermissionError:
        print("No tenes permisos para crear aqui")

def explorador():
    ruta_actual = os.getcwd()

    while True:
        limpiar_pantalla()
        print(f"\nDirectorio actual: {ruta_actual}")
        mostrar_menu()

        opcion = input("\nElije una opcion: ")

        if opcion == "1":
            listar_contenido(ruta_actual)
            input("\nPresiona Enter para continuar...")

        elif opcion == "2":
            ruta_actual = entrar_carpeta(ruta_actual)

        elif opcion == "3":
            ruta_actual = os.path.dirname(ruta_actual)
            print(f"Subiste a: {ruta_actual}")
            input("Presiona Enter para continuar...")

        elif opcion == "4":
            ver_tamano(ruta_actual)
            input("Presiona Enter para continuar...")

        elif opcion == "5":
            info_carpeta(ruta_actual)
            input("Presiona Enter para continuar...")

        elif opcion == "6":
```

```
        eliminar_archivo(ruta_actual)
        input("Presiona Enter para continuar...")

    elif opcion == "7":
        crear_carpeta(ruta_actual)
        input("Presiona Enter para continuar...")

    elif opcion == "8":
        print("\nHasta luego! Gracias por explorar.")
        break

    else:
        print("Opcion invalida")
        input("Presiona Enter para continuar...")

# Ejecutar el explorador
if __name__ == "__main__":
    explorador()
```

Explicacion: Este es un explorador de archivos completo. Usa todas las funciones que vimos:

- `getcwd()` y `chdir()` para moverse
- `listdir()` para listar contenido
- `isfile()` e `isdir()` para clasificar
- `getsize()` para tamanios
- `makedirs()` para crear carpetas
- `remove()` para borrar (con confirmacion)
- `system()` para limpiar pantalla

Resumen de lo que vimos en este capitulo

Dale, repasemos rapido lo importante que aprendimos aca:

- **Depurar codigo** es una habilidad tan importante como escribir codigo nuevo. No te frustres con los errores, son oportunidades de aprender.
- **La indentacion correcta** es crucial. Una condicion mal puesta puede hacer que tu programa nunca termine o que termine en el momento equivocado.
- **Siempre verifica las cantidades.** Si tu codigo no funciona, empieza por contar: cuantos elementos tenes? cuantos necesitas?
- **Los bucles infinitos** son peligrosos. Siempre pone un limite de intentos o una condicion de salida clara.

- Usa `try/except` para manejar errores predecibles: archivos que no existen, permisos insuficientes, etc.
- La **compatibilidad entre sistemas operativos** se logra con `os.name` y `os.sep`. No asumas que tu código corre en Windows si lo probaste solo en Linux.

Que sigue ahora?

Ya dominas el módulo `os`. Podes:

- Moverte entre carpetas como un pez en el agua
- Crear y eliminar archivos y directorios
- Recorrer arboles completos con `walk()`
- Ejecutar comandos del sistema
- Y lo mas importante: depurar código que no funciona

El proximo capitulo vamos a ver **el módulo `shutil`**, que es como `os` pero con esteroides: permite copiar arboles enteros de carpetas, mover cosas pesadas, y hacer backups completos.

Pero antes, practica con los 20 ejercicios. Hacerlos de verdad, no solo los leas. Cada error que cometas te va a enseñar algo nuevo.

*El mejor programador no es el que
nunca comete errores, es el que aprende algo de cada error que comete.*

Depuracion y Errores parte I: Como encontrar y corregir problemas en tu codigo

Querido lector: los errores son normales

Mira, cuando yo empecé a programar, creía que los buenos programadores escribían código perfecto a la primera. Que equivocado estaba! Recuerdo una madrugada en la facultad (UBA Exactas) peleando con un maldito `SyntaxError` que resultó ser un paréntesis mal cerrado. Estuve una hora mirando la pantalla, maldiciendo en silencio, hasta que un compañero se acercó y me dijo: “Nahuel, es un cierre de paréntesis, no la muerte de un familiar”. Me reí y aprendí la lección.

Los errores no son fracasos. Son **señales de tránsito** que te indican por donde no ir. Cada error que ves y entiendes te hace mejor programador. Te lo digo por experiencia: después de años programando, sigo viendo errores todos los días. La diferencia es que ahora se leerlos.

En este capítulo vamos a aprender a:

- Identificar los tipos de error más comunes
- Entender que nos dice cada mensaje de error
- Corregir los errores paso a paso
- Desarrollar una mentalidad de “detective” para depurar código

La buena noticia: Python es muy amigable. Cuando algo sale mal, te muestra un mensaje que incluye:

- El **tipo de error** (que salió mal)
- La **línea donde ocurrió** (donde buscar)
- Una **descripción** (por qué ocurrió)

Aprender a leer estos mensajes es la habilidad más importante que vas a desarrollar.

```
+-----+
|
|  TIPOS DE ERRORES MAS COMUNES:
|
|  1. SyntaxError      -> Error de sintaxis
|
```

	2. NameError	-> Variable no definida	
	3. TypeError	-> Tipo de dato incorrecto	
	4. IndexError	-> Indice fuera de rango	
	5. KeyError	-> Clave no existe en dict	
	6. ValueError	-> Valor incorrecto	
	7. ZeroDivisionError	-> Division por cero	
	8. IndentationError	-> Indentacion incorrecta	
+-----+			

SyntaxError: El error de ortografia de Python

El **SyntaxError** ocurre cuando escribes codigo que no sigue las reglas del lenguaje. Es como tener una oracion mal escrita: las palabras estan, pero el orden o la puntuacion no es correcta.

Falta de dos puntos (:)

El primer error que tuve muchas veces fue olvidar los dos puntos despues de if, for o while.

```
# CODIGO CON ERROR
if edad > 18
    print("Mayor de edad")
```

Cuando ejecutas esto, Python te muestra:

```
Error:
File "archivo.py", line 1
    if edad > 18
        ^
SyntaxError: expected ':'
```

Que nos dice el mensaje? La flecha ^ marca el lugar exacto donde Python se confundio. Y el texto dice que esperaba dos puntos.

Correccion:

```
if edad > 18:
    print("Mayor de edad")
```

Falta cerrar parentesis o comillas

Este error me paso incontables veces. Abres un parentesis o una comilla y te olvidas de cerrarlo.

```
print("Hola mundo)
nombre = input("Ingrese nombre: ")
```

```
Error:
SyntaxError: unterminated string literal
```

Correccion: Cada vez que abres un parentesis, una comilla o un corchete, tenes que cerrarlo.

```
print("Hola mundo")
nombre = input("Ingrese nombre: ")
```

Usar = en lugar de == en condiciones

Este es clasico. En Python, = significa “asignar valor”, mientras que == significa “comparar si son iguales”.

```
if numero = 5:
    print("Es cinco")
```

Error:
SyntaxError: invalid syntax

Correccion:

```
if numero == 5:
    print("Es cinco")
```

Operadores mal escritos (=> en lugar de >=)

A veces escribimos los operadores en el orden incorrecto.

```
if nota => 7:
    print("Aprobado")
```

Correccion: El orden correcto es >=.

```
if nota >= 7:
    print("Aprobado")
```

NameError: Quien es ese tal variable?

El **NameError** aparece cuando intentas usar una variable que no existe. Puede ser porque nunca la definiste, o porque escribiste mal el nombre.

```
nombre = "Juan"
print(nombre)    # Esto funciona: muestra "Juan"
print(nombres)   # ERROR: 'nombres' no esta definida
```

Error:
NameError: name 'nombres' is not defined. Did you mean 'nombre'?

Viste que bueno? Python incluso te sugiere la correccion (“Did you mean ‘nombre’?”). En mis epocas esto no existia, asi que aprovechalo.

Correccion: Revisa el nombre de la variable y asegurate de que este definida antes de usarla.

```
nombre = "Juan"
print(nombre)
```

Ejemplo real de un error que tuve

Una vez escribi `imput` en lugar de `input` y pase quince minutos buscando el error. Mira:

```
cantidad = imput("Cuántas notas quieres ingresar? ")
```

Python me dijo:

```
NameError: name 'imput' is not defined
```

Cuando vi la “i” de mas, me quise pegar. Desde ese día, reviso siempre esa palabra.

Correccion:

```
cantidad = input("Cuántas notas quieres ingresar? ")
```

TypeError: Tipos incompatibles

El **TypeError** ocurre cuando intentas hacer una operación con tipos de datos que no se llevan bien.

Sumar string con entero

Este es el mas comun cuando empiezas.

```
numero = input("Ingrese un numero: ") # input() siempre devuelve un string
resultado = numero + 5 # ERROR: no se puede sumar string con entero
```

Error:

```
TypeError: can only concatenate str (not "int") to str
```

El mensaje dice: “solo puedes concatenar strings con strings”. O sea, el + para strings significa “pegar”, no “sumar”.

Correccion: Convierte el string a numero con `int()` o `float()`.

```
numero = int(input("Ingrese un numero: "))
resultado = numero + 5
```

Llamar a una funcion sin parentesis

A veces escribimos el nombre de la funcion pero nos olvidamos los parentesis.

```
lista = [1, 2, 3]
lista.append # ERROR: falta ()
```

Correccion: Los parentesis son los que ejecutan la funcion.

```
lista = [1, 2, 3]
lista.append(4)
```


IndexError: Te saliste del limite

El **IndexError** aparece cuando intentas acceder a una posicion de una lista o string que no existe.

```
frutas = ["manzana", "banana", "pera"]  
print(frutas[3]) # ERROR: solo hay indices 0, 1 y 2
```

Error:

IndexError: list index out of range

Pensalo asi: una lista de 3 elementos tiene indices 0, 1 y 2. El indice 3 seria el “cuarto” elemento, pero no existe.

Correccion: Verifica que el indice este entre 0 y `len(lista)-1`.

```
frutas = ["manzana", "banana", "pera"]  
for i in range(len(frutas)):  
    print(frutas[i]) # i sera 0, 1, 2
```

KeyError: La clave no esta en el diccionario

El **KeyError** aparece cuando buscas una clave en un diccionario que no existe.

```
persona = {"nombre": "Juan", "edad": 25}  
print(persona["profesion"]) # ERROR: la clave 'profesion' no existe
```

Error:

KeyError: 'profesion'

Correccion: Usa el metodo `.get()` que devuelve `None` (o un valor por defecto) si la clave no existe.

```
persona = {"nombre": "Juan", "edad": 25}  
profesion = persona.get("profesion", "No especificada")  
print(profesion) # "No especificada"
```

ValueError: El valor no es valido

El **ValueError** ocurre cuando una funcion recibe un valor del tipo correcto pero que no tiene sentido para esa operacion.

```
numero = int(input("Ingrese un numero: "))  
# Si el usuario ingresa "hola" en lugar de un numero
```

Error:

ValueError: invalid literal for int() with base 10: 'hola'

El mensaje dice: “no puedo convertir ‘hola’ a numero entero”.

Correccion: Usa un bloque `try-except` para manejar el error sin que el programa se rompa.

```
try:
    numero = int(input("Ingrese un numero: "))
    print(f"Ingresaste {numero}")
except ValueError:
    print("Error: Debes ingresar un numero valido")
```

ZeroDivisionError: Dividir por cero

El **ZeroDivisionError** es exactamente lo que parece: intentaste dividir algo entre cero.

```
resultado = 10 / 0
```

```
Error:
ZeroDivisionError: division by zero
```

En matematica, dividir por cero no esta definido. Python tampoco puede.

Correccion: Verifica que el divisor no sea cero antes de dividir.

```
divisor = 0
if divisor != 0:
    resultado = 10 / divisor
else:
    print("No se puede dividir por cero")
```

IndentationError: La indentacion esta mal

Python usa la indentacion (espacios o tabs) para definir que codigo esta adentro de un bloque. Si la indentacion es incorrecta, Python no puede entender tu codigo.

```
if x > 0:
print("Positivo") # ERROR: falta indentacion
```

```
Error:
IndentationError: expected an indented block
```

Correccion: Usa indentacion consistente. Lo mejor es usar 4 espacios.

```
if x > 0:
    print("Positivo")
```

Nota: *Ojo con mezclar tabs y espacios. A veces copias codigo de internet que usa tabs y tu editor usa espacios. Python se vuelve loco. Siempre configura tu editor para que convierta los tabs en 4 espacios.*

Errores logicos: el programa corre pero hace cosas raras

Estos son los errores mas dificiles de encontrar. Python no muestra ningun mensaje porque sintacticamente el codigo esta bien. El problema es que no hace lo que vos queres que haga.

Confundir suma con concatenacion

```
edad = input("Edad: ") # input devuelve string
print("El anio que viene tendras", edad + 1)
```

Si el usuario ingresa 25, esto mostraria: "El anio que viene tendras 251" (pega "25" con "1").

Correccion: Convierte a entero primero.

```
edad = int(input("Edad: "))
print("El anio que viene tendras", edad + 1)
```

Bucle infinito

```
contador = 0
while contador < 5:
    print(contador)
    # Falta actualizar contador
```

Este bucle nunca termina porque contador siempre vale 0.

Correccion:

```
contador = 0
while contador < 5:
    print(contador)
    contador = contador + 1
```

Modificar una lista mientras se recorre

```
nombres = ["Ana", "Luis", "Ana", "Pedro"]
for nombre in nombres:
    if nombre == "Ana":
        nombres.remove(nombre) # ERROR: modifica la lista durante iteracion
```

Esto puede hacer que Python saltee elementos.

Correccion: Recorre una copia de la lista.

```
nombres = ["Ana", "Luis", "Ana", "Pedro"]
for nombre in nombres.copy():
    if nombre == "Ana":
        nombres.remove(nombre)
print(nombres) # ["Luis", "Pedro"]
```

20 Ejercicios de Deteccion de Errores

Vamos a practicar. En cada ejercicio, tenes que:

1. Identificar que tipo de error es
2. Explicar por que ocurre
3. Escribir el codigo corregido

Ejercicio 1

```
edad = input("Ingrese su edad: ")
if edad >= 18:
    print("Mayor de edad")
```

Solucion:

```
# Tipo de error: TypeError
# Causa: input() devuelve string, no se puede comparar con numero
# Correccion:
```

```
edad = int(input("Ingrese su edad: "))
if edad >= 18:
    print("Mayor de edad")
```

Ejercicio 2

```
numero = int(input("Ingrese un numero: "))
if numero % 2 = 0:
    print("Es par")
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: se usa = (asignacion) en lugar de == (comparacion)
# Correccion:
```

```
numero = int(input("Ingrese un numero: "))
if numero % 2 == 0:
    print("Es par")
```

Ejercicio 3

```
nota = int(input("Ingrese nota: "))
if nota => 7:
    print("Aprobado")
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: el operador => no existe, debe ser >=
# Correccion:
```

```
nota = int(input("Ingrese nota: "))
if nota >= 7:
    print("Aprobado")
```

Ejercicio 4

```
a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))
if a > b:
    print("El mayor es", a)
elif a < b:
    print("El mayor es", b)
else
    print("Son iguales")
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: falta : despues del else
# Correccion:
```

```
a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))
if a > b:
    print("El mayor es", a)
elif a < b:
    print("El mayor es", b)
else:
    print("Son iguales")
```

Ejercicio 5

```
numero = input("Ingrese un numero: ")
if numero > 0:
    print("Positivo")
else:
    print("Negativo o cero")
```

Solucion:

```
# Tipo de error: TypeError
# Causa: input() devuelve string, no se puede comparar con numero
# Correccion:
```

```
numero = int(input("Ingrese un numero: "))
if numero > 0:
    print("Positivo")
else:
    print("Negativo o cero")
```

Ejercicio 6

```
letra = input("Ingrese una letra: ")
if letra == 'a' or 'e' or 'i' or 'o' or 'u':
```

```
    print("Es vocal")
else:
    print("Es consonante")
```

Solucion:

```
# Tipo de error: Error logico
# Causa: la condicion esta mal escrita, 'or' necesita condiciones completas
# Correccion (opcion 1):

letra = input("Ingrese una letra: ").lower()
if letra == 'a' or letra == 'e' or letra == 'i' or letra == 'o' or letra == 'u':
    print("Es vocal")
else:
    print("Es consonante")

# Correccion (opcion 2, mas limpia):

letra = input("Ingrese una letra: ").lower()
if letra in "aeiou":
    print("Es vocal")
else:
    print("Es consonante")
```

Ejercicio 7

```
suelo = float(input("Ingrese su sueldo: "))
if sueldo => 1000:
    print("Sueldo alto")
elif sueldo => 500:
    print("Sueldo medio")
else:
    print("Sueldo bajo")
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: el operador => no existe
# Correccion:

suelo = float(input("Ingrese su sueldo: "))
if sueldo >= 1000:
    print("Sueldo alto")
elif sueldo >= 500:
    print("Sueldo medio")
else:
    print("Sueldo bajo")
```

Ejercicio 8

```
num = int(input("Ingrese un numero: "))
if num > 0 and < 10:
    print("Esta entre 1 y 9")
else:
    print("No esta en el rango")
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: falta repetir la variable en la segunda condicion
# Correccion:
```

```
num = int(input("Ingrese un numero: "))
if num > 0 and num < 10:
    print("Esta entre 1 y 9")
else:
    print("No esta en el rango")
```

Ejercicio 9

```
clave = input("Ingrese clave: ")
if clave == 1234:
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

Solucion:

```
# Tipo de error: TypeError
# Causa: input() devuelve string, se compara con numero
# Correccion:
```

```
clave = input("Ingrese clave: ")
if clave == "1234":
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

Ejercicio 10

```
promedio = int(input("Ingrese su promedio: "))
if promedio >= 6:
    print("Aprobado")
else:
    print("Desaprobado")
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: falta : despues del if
# Correccion:

promedio = int(input("Ingrese su promedio: "))
if promedio >= 6:
    print("Aprobado")
else:
    print("Desaprobado")
```

Ejercicio 11

```
lista = [1, 2, 3]
print(lista[5])
```

Solucion:

```
# Tipo de error: IndexError
# Causa: el indice 5 no existe en una lista de 3 elementos (indices 0,1,2)
# Correccion:

lista = [1, 2, 3]
print(lista[2]) # 3, el ultimo elemento
```

Ejercicio 12

```
def suma(a b)
    return a + b
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: falta coma entre parametros y falta : despues de la funcion
# Correccion:

def suma(a, b):
    return a + b
```

Ejercicio 13

```
for i in range(5)
    print(i)
```

Solucion:

```
# Tipo de error: SyntaxError
# Causa: falta : despues del for
# Correccion:

for i in range(5):
    print(i)
```


Ejercicio 14

```
dic = {"a": 1, "b": 2}
print(dic["c"])
```

Solucion:

```
# Tipo de error: KeyError
# Causa: la clave "c" no existe en el diccionario
# Correccion (opcion 1):
```

```
dic = {"a": 1, "b": 2}
if "c" in dic:
    print(dic["c"])
else:
    print("Clave no existe")
```

```
# Correccion (opcion 2, mas elegante):
print(dic.get("c", "No existe"))
```

Ejercicio 15

```
resultado = "10" + 5
```

Solucion:

```
# Tipo de error: TypeError
# Causa: no se puede sumar string con entero
# Correccion (opcion 1):
resultado = int("10") + 5 # 15

# Correccion (opcion 2):
resultado = "10" + str(5) # "105" (concatenacion)
```

Ejercicio 16

```
x = 10
while x > 0:
    print(x)
```

Solucion:

```
# Tipo de error: Error logico (bucle infinito)
# Causa: la variable x nunca cambia, siempre es 10
# Correccion:
```

```
x = 10
while x > 0:
    print(x)
    x = x - 1 # o x -= 1
```

Ejercicio 17

```
if x > 0:  
print("Positivo")
```

Solucion:

```
# Tipo de error: IndentationError  
# Causa: falta indentar el print dentro del if  
# Correccion:
```

```
if x > 0:  
    print("Positivo")
```

Ejercicio 18

```
edad = 20  
if edad >= 18:  
    print("Mayor de edad")  
elif edad >= 60:  
    print("Adulto mayor")  
else:  
    print("Menor de edad")
```

Solucion:

```
# Tipo de error: Error logico  
# Causa: el orden de las condiciones esta mal. Si edad es 65,  
# primero entra en edad >= 18 y nunca llega a edad >= 60  
# Correccion:
```

```
edad = 20  
if edad >= 60:  
    print("Adulto mayor")  
elif edad >= 18:  
    print("Mayor de edad")  
else:  
    print("Menor de edad")
```

Ejercicio 19

```
numeros = [1, 2, 3, 4, 5]  
for i in range(1, len(numeros)):  
    print(numeros[i])
```

Solucion:

```
# Tipo de error: Error logico  
# Causa: range(1, len(numeros)) empieza en indice 1, se salta el 0  
# Correccion (si quieres todos los elementos):
```

```
numeros = [1, 2, 3, 4, 5]
for i in range(len(numeros)):
    print(numeros[i])

# 0 mejor aun:
for num in numeros:
    print(num)
```

Ejercicio 20

```
def calcular_promedio(notas):
    total = 0
    for nota in notas:
        total = total + nota
    return total / len(notas)

print(calcular_promedio([]))
```

Solucion:

```
# Tipo de error: ZeroDivisionError
# Causa: la lista vacia tiene len() = 0, no se puede dividir
# Correccion:
```

```
def calcular_promedio(notas):
    if len(notas) == 0:
        return 0 # o podrias devolver None
    total = 0
    for nota in notas:
        total = total + nota
    return total / len(notas)

print(calcular_promedio([])) # 0
```

Tabla Resumen de Errores Comunes

ERROR	CAUSA	SOLUCION
SyntaxError	Falta :, faltan parentesis, comillas sin cerrar	Revisar sintaxis basica
NameError	Variable no definida o mal escrita	Definir variable o corregir nombre
TypeError	Operacion con tipo de dato incorrecto	Convertir tipos con int(), str(), etc.
IndexError	Indice fuera del rango de la lista	Verificar con len()
KeyError	Clave no existe en diccionario	Usar .get() o in
ValueError	Valor incorrecto para la funcion	Validar entrada con try/except
ZeroDivisionError	Division por cero	Verificar que divisor != 0
IndentationError	Indentacion incorrecta	Usar 4 espacios consistentes

Consejos para Depurar como un Profesional

Despues de anos programando, estos son los consejos que mas me ayudaron:

1. **LeÃ el mensaje de error completo.** No te quedes solo con la primera linea. Python te dice exactamente que paso y en que linea.
2. **Usa `print()` para ver valores.** Si no sabes que valor tiene una variable, imprimila. Es la forma mas simple y efectiva de entender que esta pasando.
3. **Proba el codigo por partes.** Si tienes una funcion larga, comenta partes y ejecuta paso a paso para aislar el error.
4. **Usa un linter.** Herramientas como `pylint` o `flake8` encuentran errores antes de que ejecutes el codigo. En la facultad me salvo las papas mas de una vez.
5. **Explica el problema en voz alta.** A veces, al explicarle el error a alguien (o a un patito de goma), la solucion aparece magicamente. Se llama "rubber duck debugging".
6. **Toma un descanso.** Cuando llevas una hora mirando el mismo error sin solucion, alejate. Vuelve con ojos frescos y lo vas a ver.
7. **Usa un depurador (debugger).** El IDE de Python (como VSCode o PyCharm) tiene herramientas para ejecutar el codigo linea por linea y ver el valor de las variables en cada paso.

Reflexion final

Querido lector, depurar codigo es como ser detective. Cada error es una pista. Aprende a leer los mensajes de error, no les tengas miedo. Son tus aliados, no tus enemigos.

La proxima vez que veas un error, respira hondo, lee el mensaje, fijate la linea que indica, y piensa: "que le falta a esta linea?". Te prometo que con practica, vas a encontrar los errores cada vez mas rapido.

Los programadores no escriben codigo perfecto. Los programadores escriben codigo, encuentran errores, los corrigen, y aprenden. Ese es el ciclo.

Importacion de modulos y funciones: como organizar y reutilizar tu codigo

El problema que todos tuvimos al principio

Mira, querido lector, cuando yo empecé a programar, escribía todo en un solo archivo. Tenía cientos de líneas mezcladas: funciones, variables, el menú principal, todo junto. El resultado? Un quilombo. Cada vez que quería encontrar algo, perdía cinco minutos scrolleando.

Y lo peor no era eso. Lo peor era que si quería usar las mismas funciones en otro programa, tenía que copiar y pegar el código. Y si encontraba un error, lo corregía en un lugar... pero en el otro seguía roto.

Ahi entendí la importancia de **modularizar**, de separar el código en partes que se puedan reutilizar.

En este capítulo vamos a aprender a:

- Crear nuestros propios **modulos** (archivos .py con funciones)
- Importar modulos completos o solo funciones especificas
- Usar un modulo “Biblioteca” con operaciones matematicas basicas
- Construir un programa completo de gestion de peliculas

Y al final, 20 ejercicios para que practiques hasta que te salga sin pensar.

Analogia del dia: Un modulo es como una caja de herramientas. Vos no tenes un martillo, una llave y un destornillador sueltos por ahi. Los guardas en una caja. Cuando necesitas el martillo, abris la caja y lo sacas. Eso es un modulo.

Creando nuestro propio modulo: la biblioteca matematica

Vamos a empezar por algo sencillo. Supongamos que tenemos varias funciones que hacen operaciones matematicas: sumar, restar, multiplicar, dividir, ver si un numero es par o impar, comparar dos numeros, calcular promedios.

En lugar de escribir esas funciones en cada programa, las ponemos en un archivo separado llamado `Biblioteca.py`. Ese archivo va a tener este contenido:

```
# =====  
# BIBLIOTECA DE FUNCIONES MATEMATICAS  
# Archivo: Biblioteca.py
```

```
# =====

def suma_entre_ellos(a, b):
    """Suma dos numeros y muestra el resultado"""
    resultado = a + b
    print(f"La suma de {a} y {b} es {resultado}")

def resta_entre_ellos(a, b):
    """Resta dos numeros y muestra el resultado"""
    resultado = a - b
    print(f"La resta de {a} y {b} es {resultado}")

def multiplicacion_entre_ellos(a, b):
    """Multiplica dos numeros y muestra el resultado"""
    resultado = a * b
    print(f"La multiplicacion de {a} y {b} es {resultado}")

def dividir_entre_ellos(a, b):
    """Divide dos numeros y muestra el resultado"""
    if b != 0:
        resultado = a / b
        print(f"La division de {a} y {b} es {resultado}")
    else:
        print("Error: No se puede dividir por cero")

def es_o_impar(n):
    """Determina si un numero es par o impar"""
    if n % 2 == 0:
        print(f"{n} es un numero par")
    else:
        print(f"{n} es un numero impar")

def que_numero_es_mayor(a, b):
    """Compara dos numeros y dice cual es mayor"""
    if a > b:
        print(f"{a} es mayor que {b}")
    elif b > a:
        print(f"{b} es mayor que {a}")
    else:
        print(f"{a} y {b} son iguales")

def promedio(a, b):
    """Calcula el promedio de dos numeros"""
    resultado = (a + b) / 2
    print(f"El promedio de {a} y {b} es {resultado}")
```

Ahora, cuando queramos usar estas funciones en otro programa, solo tenemos que importarlas. Y aca es donde aparecen las dos formas principales.

Forma 1: Importar el modulo completo (import modulo)

La primera forma es la mas simple: importas todo el modulo. Pero para usar una funcion, tenes que poner el nombre del modulo seguido de un punto y el nombre de la funcion.

```
import Biblioteca

a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))

Biblioteca.suma_entre_ellos(a, b)
Biblioteca.es_o_impar(a)
```

Ventaja: Es muy claro saber de donde viene cada funcion (de la `Biblioteca`). **Desventaja:** Tenes que escribir `Biblioteca.` cada vez.

Analogia: Es como decir “caja de herramientas.martillo()” en lugar de solo “martillo()”. Esta bien si tenes muchas cajas diferentes, pero puede ser tedioso.

Forma 2: Importar funciones especificas (from modulo import funcion)

Si solo vas a usar algunas funciones de un modulo, podes importarlas directamente. Asi las llamas sin poner el nombre del modulo.

```
from Biblioteca import suma_entre_ellos, es_o_impar, promedio

a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))

suma_entre_ellos(a, b)    # No hace falta poner "Biblioteca."
es_o_impar(a)
promedio(a, b)
```

Ventaja: El codigo queda mas limpio y corto. **Desventaja:** Si tenes funciones con el mismo nombre de diferentes modulos, puede haber conflicto.

Importante: Tambien podes importar todas las funciones de una vez con `from modulo import *`. Pero no es recomendable porque puede generar confusiones. Mejor ser explicito.

Forma 3: Importar con un alias (import modulo as alias)

Si el nombre del modulo es muy largo o queres abreviarlo, podes darle un alias.

```
import Biblioteca as bib

a = int(input("Primer numero: "))
b = int(input("Segundo numero: "))

bib.suma_entre_ellos(a, b)
bib.es_o_impar(a)
```

Es un punto medio entre las dos formas anteriores. Muy comun en modulos con nombres largos (como `matplotlib.pyplot` al que muchos le ponen `plt`).

¿Cual forma usar?

Aca va mi regla personal despues de anos programando:

- Si el modulo es estandar y lo usas mucho (`os`, `math`, `random`) -> `import modulo` (o alias si es largo).
- Si solo necesitas 2 o 3 funciones de un modulo grande -> `from modulo import funcion1, funcion2`.
- Para tus propios modulos pequenos -> cualquiera de las dos, segun tu gusto.

Ejemplo completo: calculadora con modulo personalizado

Vamos a armar un programa que pida dos numeros al usuario y use todas las funciones de nuestra Biblioteca.

```
# =====
# PROGRAMA PRINCIPAL
# Archivo: calculadora.py
# =====

# Importo las funciones que voy a usar
from Biblioteca import (
    suma_entre_ellos,
    resta_entre_ellos,
    multiplicacion_entre_ellos,
    dividir_entre_ellos,
    es_o_impar,
    que_numero_es_mayor,
    promedio
)

# Entrada de datos
print("=== CALCULADORA BASICA ===\n")
a = int(input("Introduzca el primer numero: "))
b = int(input("Introduzca el segundo numero: "))
print()
```



```
# Operaciones con los dos numeros
que_numero_es_mayor(a, b)
suma_entre_ellos(a, b)
resta_entre_ellos(a, b)
multiplicacion_entre_ellos(a, b)
dividir_entre_ellos(a, b)
promedio(a, b)

# Paridad de cada numero
es_o_impar(a)
es_o_impar(b)

print("\nGracias por usar la calculadora!")
```

Ejercicio rapido: Modifica el programa para que pida los numeros adentro de un bucle, y termine solo cuando el usuario escriba "salir".

Proyecto completo: Gestor de peliculas

Ahora vamos a hacer algo mas interesante. Un programa que permita gestionar una lista de peliculas. Vamos a construir paso a paso.

Paso 1: La lista inicial

Primero, creamos la lista con algunas peliculas de ejemplo.

```
lista_de_peliculas = ['Flash', 'Spider-man', 'Superman', 'Batman']
```

Paso 2: Funcion para agregar peliculas

Necesitamos agregar peliculas al final de la lista.

```
def agregar_pelicula(lista, pelicula_nueva):
    """Agrega una nueva pelicula al final de la lista"""
    lista.append(pelicula_nueva)
    print(f"{pelicula_nueva}' fue agregada correctamente")
```

`append()` es el metodo que agrega un elemento al final de una lista. Fijate que la lista se modifica directamente (no hace falta devolverla).

Paso 3: Funcion para contar peliculas

Saber cuantas peliculas tenemos es util para mostrar estadisticas.

```
def cantidad_peliculas(lista):
    """Devuelve la cantidad de peliculas en la lista"""
    return len(lista)
```

`len()` es una funcion nativa de Python que cuenta los elementos de una lista. Usamos `return` para devolver ese numero.

Paso 4: Funcion para mostrar todas las peliculas

Para listar todo el contenido.

```
def mostrar_pelicula(lista):
    """Muestra todas las peliculas, una por linea"""
    if not lista:
        print("No hay peliculas en la lista")
    else:
        print("\n=== LISTA DE PELICULAS ===")
        for i, elemento in enumerate(lista):
            print(f"{i+1}. {elemento}")
```

`enumerate()` nos da el indice y el valor. Asi mostramos un numero de orden junto a cada pelicula.

Paso 5: Funcion para buscar peliculas

El usuario ingresa un nombre y le decimos si esta y en que posicion.

```
def buscar_pelicula(lista, nombre_de_pelicula):
    """Busca una pelicula por nombre y muestra su posicion"""
    if nombre_de_pelicula in lista:
        posicion = lista.index(nombre_de_pelicula)
        print(f"'{nombre_de_pelicula}' esta en la posicion {posicion}")
    else:
        print(f"'{nombre_de_pelicula}' no esta en la lista")
```

`in` verifica si un elemento existe en la lista. `index()` devuelve la posicion (empezando desde 0).

Paso 6: El menu principal con bucle infinito

Finalmente, armamos el menu que se repite hasta que el usuario quiera salir.

```
def main():
    lista_de_peliculas = ['Flash', 'Spider-man', 'Superman', 'Batman']

    while True:
        print("\n" + "=" * 10)
        print("MENU DE SELECTOR DE PELICULAS")
        print("=" * 10)
        print("1. AGREGAR PELICULA")
        print("2. CANTIDAD DE PELICULAS")
        print("3. MOSTRAR PELICULAS")
        print("4. BUSCAR PELICULA")
        print("5. SALIR")
        print("-" * 10)

        opcion = input("Indique la opcion: ")
```

```
if opcion == "1":
    pelicula = input("Escribir pelicula: ")
    agregar_pelicula(lista_de_peliculas, pelicula)

elif opcion == "2":
    total = cantidad_peliculas(lista_de_peliculas)
    print(f"Hay {total} peliculas en la lista")

elif opcion == "3":
    mostrar_pelicula(lista_de_peliculas)

elif opcion == "4":
    titulo = input("Nombre de la pelicula a buscar: ")
    buscar_pelicula(lista_de_peliculas, titulo)

elif opcion == "5":
    print("Selecciono salir. Adios!")
    break

else:
    print("Opcion no valida. Ingrese un numero del 1 al 5")

if __name__ == "__main__":
    main()
```

El `if __name__ == "__main__":` es una buena practica. Significa: “si este archivo se ejecuta directamente (no se importa), entonces ejecuta `main()`”.

Ejercicio rapido: Agrega una opcion “6” para eliminar una pelicula de la lista, usando `remove()`.

Ejercicios Propuestos

Ejercicio 1

Crea el archivo `operaciones.py` con las funciones `sumar(a,b)`, `restar(a,b)`, `multiplicar(a,b)` y `dividir(a,b)` que retornen los resultados. Luego importalo desde otro archivo y usalas.

Ejercicio 2

Modifica el archivo `operaciones.py` del ejercicio anterior para que `dividir(a,b)` devuelva un mensaje de error si `b == 0` en lugar de lanzar una excepcion.

Ejercicio 3

Crea un modulo llamado `validaciones.py` con funciones `es_mayor_de_edad(edad)` y `es_par(numero)` que devuelvan `True` o `False`. Probalo importando las funciones.

Ejercicio 4

En el gestor de peliculas, agrega una funcion `eliminar_pelicula(lista, titulo)` que use `remove()`. Agregala al menu.

Ejercicio 5

Modifica el gestor de peliculas para que, al listar, muestre tambien el indice desde 1.

Ejercicio 6

Crea un modulo `convertor.py` con funciones `cm_a_pulgadas(cm)` y `pulgadas_a_cm(pulgadas)`. Despues hace un programa que pregunte al usuario que conversion quiere hacer.

Ejercicio 7

Importa la funcion `randint` de `random` y crea un programa que simule el lanzamiento de un dado de 6 caras 10 veces, mostrando los resultados.

Ejercicio 8

Importa `choice` de `random` y hace que el programa elija aleatoriamente una pelicula de la lista del gestor.

Ejercicio 9

Crea un modulo `temperaturas.py` con funciones `celsius_a_fahrenheit(c)` y `fahrenheit_a_celsius`. Hace un programa que las use en un menu.

Ejercicio 10

Usa la funcion `strftime` de `time` para mostrar la fecha y hora actuales formateadas.

Ejercicio 11

Crea una funcion `filtrar_mayores(edades, limite)` que reciba una lista de edades y un limite, y devuelva una nueva lista con las edades mayores al limite. Guardala en un modulo.

Ejercicio 12

Importa la funcion `sqrt` (raiz cuadrada) de `math` y pide un numero al usuario para calcular su raiz.

Ejercicio 13

Crea un modulo `geometria.py` con funciones para calcular area y perimetro de un rectangulo y un circulo (con `pi` de `math`). Hace un programa que las pruebe.

Ejercicio 14

En el gestor de películas, agrega la opción de “película aleatoria” usando `random.choice` y muestra el título elegido.

Ejercicio 15

Modifica el gestor para que la lista de películas se cargue desde un archivo `peliculas.txt` al iniciar (si existe) y se guarde al salir.

Ejercicio 16

Crea un archivo `utilidades.py` que contenga la función `es_palindromo(palabra)` y luego usala en otro programa para verificar palabras ingresadas por el usuario.

Ejercicio 17

Hace un programa que pida un número y muestre su factorial. La función factorial debe estar en un módulo aparte.

Ejercicio 18

Crea una función `generar_contrasena(longitud)` en un módulo `seguridad.py` usando `random` y `string`. Después usala desde un programa principal.

Ejercicio 19

Importa `datetime` y crea una función que reciba una fecha de nacimiento y calcule la edad actual. Próbala.

Ejercicio 20

Disena un pequeño sistema de gestión de contactos (nombre, teléfono) usando diccionarios. Cada función (agregar, buscar, eliminar, listar) debe estar en un módulo separado `contactos.py`. El programa principal debe importar y usar esas funciones.

Soluciones a los Ejercicios

Solucion 1

Archivo operaciones.py:

```
def sumar(a, b):  
    return a + b
```

```
def restar(a, b):  
    return a - b
```

```
def multiplicar(a, b):  
    return a * b
```

```
def dividir(a, b):  
    return a / b
```

Programa principal:

```
import operaciones
```

```
x = int(input("Numero 1: "))  
y = int(input("Numero 2: "))  
print(f"Suma: {operaciones.sumar(x, y)}")  
print(f"Division: {operaciones.dividir(x, y)}")
```

Solucion 2

En operaciones.py, modificar dividir:

```
def dividir(a, b):  
    if b == 0:  
        return "Error: division por cero"  
    return a / b
```

Solucion 3

Archivo validaciones.py:

```
def es_mayor_de_edad(edad):  
    return edad >= 18
```

```
def es_par(numero):  
    return numero % 2 == 0
```

Programa:

```
from validaciones import es_mayor_de_edad, es_par
```

```
print(es_mayor_de_edad(20)) # True  
print(es_par(5))           # False
```

Solucion 4

Agregar la funcion en el gestor:

```
def eliminar_pelicula(lista, titulo):
    if titulo in lista:
        lista.remove(titulo)
        print(f"'{titulo}' eliminada")
    else:
        print("No existe esa pelicula")
```

Y en el menu, agregar el elif opcion == "6": correspondiente.

Solucion 5

Ya lo hicimos con enumerate y i+1.

Solucion 6

Archivo conversor.py:

```
def cm_a_pulgadas(cm):
    return cm / 2.54

def pulgadas_a_cm(pulgadas):
    return pulgadas * 2.54
```

Programa:

```
import conversor

op = input("Convertir 1) cm a pulgadas 2) pulgadas a cm: ")
val = float(input("Valor: "))
if op == "1":
    print(conversor.cm_a_pulgadas(val))
else:
    print(conversor.pulgadas_a_cm(val))
```

Solucion 7

```
from random import randint

for i in range(10):
    print(randint(1, 6))
```

Solucion 8

```
from random import choice

peliculas = ['Flash', 'Spider-man', 'Superman', 'Batman']
print("Pelicula aleatoria:", choice(peliculas))
```

Solucion 9

Archivo temperaturas.py:

```
def celsius_a_fahrenheit(c):  
    return c * 9/5 + 32  
  
def fahrenheit_a_celsius(f):  
    return (f - 32) * 5/9
```

Y el programa similar al conversor del ejercicio 6.

Solucion 10

```
from time import strftime  
  
print("Fecha y hora:", strftime("%d/%m/%Y %H:%M:%S"))
```

Solucion 11

Modulo filtros.py:

```
def filtrar_mayores(edades, limite):  
    resultado = []  
    for edad in edades:  
        if edad > limite:  
            resultado.append(edad)  
    return resultado
```

Solucion 12

```
from math import sqrt  
  
num = float(input("Numero: "))  
print("Raiz cuadrada:", sqrt(num))
```

Solucion 13

Archivo geometria.py:

```
from math import pi  
  
def area_rectangulo(base, altura):  
    return base * altura  
  
def perimetro_rectangulo(base, altura):  
    return 2 * (base + altura)  
  
def area_circulo(radio):  
    return pi * radio ** 2
```



```
def perimetro_circulo(radio):
    return 2 * pi * radio
```

Probar con imports.

Solucion 14

Agregar al menu:

```
elif opcion == "6":
    from random import choice
    print("Película al azar:", choice(lista_de_peliculas))
```

Solucion 15

Al inicio del programa:

```
import os
archivo = "peliculas.txt"
if os.path.exists(archivo):
    with open(archivo, "r") as f:
        lista_de_peliculas = [line.strip() for line in f]
else:
    lista_de_peliculas = []
```

Al salir (opcion 5), antes del break:

```
with open(archivo, "w") as f:
    for peli in lista_de_peliculas:
        f.write(peli + "\n")
```

Solucion 16

Archivo utilidades.py:

```
def es_palindromo(palabra):
    palabra = palabra.lower()
    return palabra == palabra[::-1]
```

Programa:

```
from utilidades import es_palindromo
p = input("Palabra: ")
print("Es palindromo?", es_palindromo(p))
```

Solucion 17

Archivo factorial.py:

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)
```

Programa:

```
import factorial  
print(factorial.factorial(5))
```

Solucion 18

Archivo seguridad.py:

```
import random  
import string  
  
def generar_contrasena(longitud):  
    caracteres = string.ascii_letters + string.digits + string.punctuation  
    return ''.join(random.choice(caracteres) for _ in range(longitud))
```

Luego importar y usar.

Solucion 19

```
from datetime import date  
  
def calcular_edad(fecha_nac):  
    hoy = date.today()  
    edad = hoy.year - fecha_nac.year - ((hoy.month, hoy.day) < (fecha_nac.month, fecha_nac.day))  
    return edad  
  
# Ejemplo de uso  
nacimiento = date(1995, 5, 17)  
print(calcular_edad(nacimiento))
```

Solucion 20

Archivo contactos.py:

```
def agregar_contacto(agenda, nombre, telefono):  
    agenda[nombre] = telefono  
  
def buscar_contacto(agenda, nombre):  
    return agenda.get(nombre, "No encontrado")  
  
def eliminar_contacto(agenda, nombre):
```

```
    if nombre in agenda:
        del agenda[nombre]
        return True
    return False

def listar_contactos(agenda):
    for nombre, tel in agenda.items():
        print(f"{nombre}: {tel}")
```

Programa principal: un menu con while que llame a estas funciones.

Resumen del capitulo

En este capitulo aprendiste:

- **Modulos:** archivos `.py` que contienen funciones, variables y clases para reutilizar.
- **Importar modulos:** tres formas:
 - `import modulo` (usar con prefijo)
 - `from modulo import funcion` (sin prefijo)
 - `import modulo as alias`
- **Organizar el codigo** en modulos lo vuelve mas limpio, mantenible y reusable.
- **Aplicacion practica:** desde una biblioteca matematica hasta un gestor de peliculas completo.

Importar un modulo es como abrir la caja de herramientas correcta. Saber cual usar y como usarla te ahorra tiempo y te hace mas eficiente.

Try y Except: Como hacer que tu programa no se rompa nunca mas

Querido lector: todos los programas se rompen

Mira, cuando yo empecé a programar, escribía mis códigos pensando que los usuarios iban a hacer exactamente lo que yo esperaba. Que ingenuo!

Ponia `input(Ingresa un numero: )` y confiaba ciegamente en que el usuario iba a escribir un numero. Adivina que pasaba cuando alguien escribía “hola”? Mi programa se rompía. Se cerraba. Mostraba un error feo y se iba. Suena conocido?

Eso es porque los **errores en tiempo de ejecucion** son inevitables. No importa que tan bueno sea tu código: el usuario siempre va a hacer algo que no esperas. En la UBA Exactas, una profesora me dijo una vez: “Nahuel, programa como si el usuario estuviera empenado en romper tu programa, porque lo va a hacer”. Y tenía razón.

La buena noticia es que Python tiene una forma de manejar esto. Se llama **manejo de excepciones** y te permite decirle a tu programa: “Mira, si pasa este error, no te rompas, hace esto otro”.

En este capítulo vamos a aprender a usar `try`, `except`, `else` y `finally` para hacer programas robustos que no se caen ante el primer problema.

La idea central: Las excepciones son eventos (errores) que ocurren durante la ejecución y rompen el flujo normal. El manejo de excepciones te permite anticipar estos problemas y responder de forma controlada. Como un airbag en un auto: no evita el choque, pero te salva del desastre.

El primer problema: que pasa si el usuario escribe cualquier cosa?

Pongamos un ejemplo simple. Pedimos un numero y dividimos 50 por ese numero.

```
numero = int(input("Introduce un numero entero: "))
resultado = 50 // numero
print(f"Resultado: {resultado}")
```

Proba esto. Si ingresas un numero valido, funciona. Pero:

- Si el usuario escribe “hola” ->se rompe con un `ValueError`
- Si el usuario escribe 0 ->se rompe con un `ZeroDivisionError`

El programa se cierra y el usuario ve un mensaje de error feo. Eso no es profesional.

Reflexion: Alguna vez usaste un programa que se cerro de golpe cuando hiciste algo mal? Te gusto? Seguro que no. Nuestros programas tienen que ser amables con el usuario, aunque este se equivoque.

La solucion: try y except

La forma de manejar esto es con un bloque `try` y uno o varios `except`.

```
try:
    numero = int(input("Introduce un numero entero: "))
    resultado = 50 // numero
    print(f"Resultado: {resultado}")
except ValueError:
    print("Debes ingresar un numero entero")
except ZeroDivisionError:
    print("No puedes dividir por cero")
```

Viste que cambio? Ahora, si el usuario escribe "hola", no se rompe. En lugar de eso, mostramos un mensaje amable. Y el programa termina de forma ordenada.

Como funciona exactamente?

Pensalo como un sistema de seguridad:

- El bloque `try` es la "zona de riesgo". Ahi pones el codigo que podria fallar.
- Cada `except` es un "escudo" para un tipo especifico de error.
- Si el error coincide con alguno de los `except`, se ejecuta ese bloque y el programa no se rompe.

Analogia: Es como un paracaidas. Mientras volas (el `try`), sabes que si algo sale mal, tenes un respaldo (el `except`) que te permite aterrizar suavemente en lugar de estrellarte.

Atrapar el mensaje de error con as

A veces quieres mostrar el mensaje original de Python para dar mas informacion.

```
try:
    numero = int(input("Introduce un numero: "))
    resultado = 50 // numero
    print(f"Resultado: {resultado}")
except ValueError as error:
    print(f"Error de valor: {error}")
except ZeroDivisionError as error:
    print(f"Error matematico: {error}")
```

La palabra `as` nos permite guardar el objeto de la excepcion en una variable. Asi podemos mostrar el mensaje original que Python hubiera mostrado, pero con nuestro propio texto.

La excepcion generica: Exception

Que pasa si no sabemos todos los errores que pueden ocurrir? O si queremos un “atajo” para cualquier error imprevisto? Ahi entra **Exception** (con mayuscula, cuidado).

```
try:
    numero = int(input("Introduce un numero: "))
    resultado = 50 // numero
    print(f"Resultado: {resultado}")
except ValueError:
    print("Error: no ingresaste un numero entero")
except ZeroDivisionError:
    print("Error: no se puede dividir por cero")
except Exception as e:
    print(f"Error inesperado: {e}")
```

Nota: Consejo importante: Pone **Exception** al final, despues de las excepciones especificas. Porque los **except** se evaluan en orden, y **Exception** captura **TODO**. Si la pones primera, nunca van a llegar a las otras.

Es como tener un salvavidas: lo usas cuando todo lo demas fallo.

El bloque else: cuando todo sale bien

A veces quieres ejecutar codigo SOLAMENTE si no hubo ningun error. Para eso existe **else**.

```
try:
    numero = int(input("Introduce un numero: "))
    resultado = 50 // numero
except ValueError:
    print("No ingresaste un numero entero")
except ZeroDivisionError:
    print("No se puede dividir por cero")
else:
    print(f"Operacion exitosa! El resultado es {resultado}")
    print("Gracias por usar el programa")
```

La diferencia? Si pones el **print** dentro del **try**, se ejecuta igual si hay error. No, porque si hay error el **try** se corta. Pero el **else** es mas limpio: separa lo que es “codigo que puede fallar” de lo que es “codigo que corre si no falla”.

Analogia: El **else** es como la recompensa. Si pasaste la zona de riesgo sin problemas, te llevas un premio. Es como cruzar un puente con una red de seguridad abajo. Si cruzaste bien, al llegar al otro lado te dan un cafe.

El bloque finally: pase lo que pase, esto se ejecuta

Este es mi favorito. **finally** se ejecuta SIEMPRE. Haya ocurrido un error o no. Haya entrado en un **except** o en el **else**.

```

try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
    print(contenido)
except FileNotFoundError:
    print("El archivo no existe")
finally:
    print("Cerrando archivo...")
    if 'archivo' in locals():
        archivo.close()
    print("Fin del programa")

```

Para que sirve finally?

- Cerrar archivos (como en el ejemplo)
- Liberar recursos de memoria
- Cerrar conexiones a bases de datos
- Mostrar un mensaje de “Fin del programa” pase lo que pase

Analogia: El finally es la salida de emergencia que siempre se usa, pase lo que pase en el edificio. Aunque no haya emergencia, siempre la ves al salir.

Todos los bloques juntos: el combo completo

Mira este ejemplo que usa try, except (multiples), else y finally:

```

def dividir_seguro():
    try:
        num1 = float(input("Primer numero: "))
        num2 = float(input("Segundo numero: "))
        resultado = num1 / num2
    except ValueError:
        print("Error: Debes ingresar numeros validos")
    except ZeroDivisionError:
        print("Error: No se puede dividir por cero")
    except Exception as e:
        print(f"Error inesperado: {e}")
    else:
        print(f"La division es: {resultado}")
        print("Operacion exitosa!")
    finally:
        print("--- Fin del intento de division ---")

dividir_seguro()

```

Proba este codigo varias veces:

1. Con numeros validos (ej: 10 y 2) -> ves el `else`
2. Con texto en lugar de numeros -> ves el `ValueError`
3. Con segundo numero = 0 -> ves el `ZeroDivisionError`
4. Siempre ves el `finally` al final

Errores comunes al usar try/except

Poner except muy amplio al principio

```
# MAL: Exception captura todo, los otros except nunca se ejecutan
try:
    numero = int(input("Numero: "))
    resultado = 10 / numero
except Exception:
    print("Algo salio mal")
except ZeroDivisionError: # NUNCA se ejecuta
    print("No se puede dividir por cero")
```

Correccion: Las excepciones especificas van PRIMERO.

```
# BIEN
try:
    numero = int(input("Numero: "))
    resultado = 10 / numero
except ZeroDivisionError:
    print("No se puede dividir por cero")
except Exception:
    print("Algo salio mal")
```

No guardar el error para depurar

```
# MAL - perdes informacion
try:
    archivo = open("datos.txt")
except FileNotFoundError:
    print("Error") # que error? por que?
```

Correccion: Usa `as` para mostrar el mensaje original.

```
# BIEN
try:
    archivo = open("datos.txt")
except FileNotFoundError as error:
    print(f"No se pudo abrir el archivo: {error}")
```

No usar finally para limpiar recursos

```
# MAL - el archivo podria quedar abierto
try:
    archivo = open("datos.txt", "w")
    archivo.write("Hola")
except Exception:
    print("Error")
```

Correccion: Usa finally.

```
# BIEN
archivo = None
try:
    archivo = open("datos.txt", "w")
    archivo.write("Hola")
except Exception as e:
    print(f"Error: {e}")
finally:
    if archivo:
        archivo.close()
    print("Archivo cerrado")
```

Tabla de excepciones mas comunes

Excepcion	Cuando ocurre?
ValueError	Convertir "hola" a numero
ZeroDivisionError	Dividir por cero
IndexError	Acceder a lista[10] con lista de 3 elementos
KeyError	Buscar dict[clave] que no existe
FileNotFoundError	Abrir un archivo que no existe
TypeError	Sumar "2" + 2
NameError	Usar variable no definida
Exception	Cualquier otra excepcion (el comodin)

20 Ejercicios de Manejo de Excepciones

Ahora te toca a vos. No leas las soluciones hasta que hayas intentado cada uno. La practica con try/except es lo que te va a salvar de dolores de cabeza futuros.

Ejercicio 1

Enunciado: Escribe un programa que pida un numero entero y muestre su doble. Si el usuario ingresa algo que no es un numero, mostrar "Error: Debes ingresar un numero entero".

Solucion paso a paso:

```
try:
    numero = int(input("Ingrese un numero entero: "))
```

```
print(f"El doble es: {numero * 2}")
except ValueError:
    print("Error: Debes ingresar un numero entero")
```

Ejercicio 2

Enunciado: Escribe un programa que pida dos numeros y muestre el resultado de la division. Manejar la division por cero.

Solucion paso a paso:

```
try:
    num1 = float(input("Primer numero: "))
    num2 = float(input("Segundo numero: "))
    resultado = num1 / num2
    print(f"Resultado: {resultado}")
except ZeroDivisionError:
    print("Error: No se puede dividir por cero")
except ValueError:
    print("Error: Debes ingresar numeros validos")
```

Ejercicio 3

Enunciado: Escribe una funcion que reciba una lista y un indice, y devuelva el elemento. Si el indice no existe, devolver "Indice fuera de rango".

Solucion paso a paso:

```
def obtener_elemento(lista, indice):
    try:
        return lista[indice]
    except IndexError:
        return "Indice fuera de rango"

mi_lista = [10, 20, 30, 40]
print(obtener_elemento(mi_lista, 2))    # 30
print(obtener_elemento(mi_lista, 10))   # Indice fuera de rango
```

Ejercicio 4

Enunciado: Escribe un programa que intente abrir un archivo "config.txt". Si no existe, que muestre "El archivo no existe" y lo cree vacio.

Solucion paso a paso:

```
try:
    with open("config.txt", "r") as archivo:
        contenido = archivo.read()
        print("Contenido del archivo:")
        print(contenido)
except FileNotFoundError:
    print("El archivo no existe. Creandolo...")
    with open("config.txt", "w") as archivo:
```

```

archivo.write("")
print("Archivo creado exitosamente")

```

Ejercicio 5

Enunciado: Escribe un programa que pida numeros hasta que el usuario ingrese “salir”. Si ingresa algo que no es numero, mostrar “No es un numero valido, intenta de nuevo”.

Solucion paso a paso:

```

while True:
    entrada = input("Ingrese un numero (o 'salir' para terminar): ")
    if entrada.lower() == "salir":
        print("Hasta luego!")
        break
    try:
        numero = float(entrada)
        print(f"Ingresaste: {numero}")
    except ValueError:
        print("No es un numero valido, intenta de nuevo")

```

Ejercicio 6

Enunciado: Escribe una funcion que reciba un diccionario y una clave, y devuelva el valor. Si la clave no existe, devolver “Clave no encontrada”.

Solucion paso a paso:

```

def obtener_valor(diccionario, clave):
    try:
        return diccionario[clave]
    except KeyError:
        return "Clave no encontrada"

persona = {"nombre": "Juan", "edad": 30}
print(obtener_valor(persona, "nombre"))      # Juan
print(obtener_valor(persona, "profesion"))    # Clave no encontrada

```

Ejercicio 7

Enunciado: Escribe un programa que convierta una lista de strings a enteros. Si algun elemento no se puede convertir, saltarlo y mostrar un mensaje.

Solucion paso a paso:

```

def convertir_lista(lista_strings):
    enteros = []
    for elemento in lista_strings:
        try:
            numero = int(elemento)
            enteros.append(numero)
        except ValueError:
            print(f"'{elemento}' no es un numero, se ignora")

```

```
return enteros
```

```
datos = ["10", "20", "hola", "30", "mundo", "40"]
resultado = convertir_lista(datos)
print(f"Enteros convertidos: {resultado}")
```

Ejercicio 8

Enunciado: Escribe un programa que pida al usuario un indice y muestre el elemento de una lista fija. Manejar ValueError e IndexError.

Solucion paso a paso:

```
lista = ["manzana", "banana", "pera", "naranja", "uva"]

try:
    indice = int(input(f"Ingrese un indice (0-{len(lista)-1}): "))
    print(f"Elemento: {lista[indice]}")
except ValueError:
    print("Error: Debes ingresar un numero entero")
except IndexError:
    print(f"Error: El indice debe estar entre 0 y {len(lista)-1}")
```

Ejercicio 9

Enunciado: Escribe una funcion que pida un numero entero al usuario y no se detenga hasta que ingrese uno valido.

Solucion paso a paso:

```
def pedir_entero(mensaje):
    while True:
        try:
            numero = int(input(mensaje))
            return numero
        except ValueError:
            print("Error: Debes ingresar un numero entero")

edad = pedir_entero("Ingrese su edad: ")
print(f"Edad ingresada: {edad}")
```

Ejercicio 10

Enunciado: Escribe un programa que pida dos numeros y los divida. Si el usuario ingresa texto o divide por cero, que vuelva a pedir los numeros.

Solucion paso a paso:

```
def division_con_reintentos():
    while True:
        try:
            num1 = float(input("Primer numero: "))
            num2 = float(input("Segundo numero: "))
```

```

        resultado = num1 / num2
        print(f"Resultado: {resultado}")
        break
    except ValueError:
        print("Error: Debes ingresar numeros")
    except ZeroDivisionError:
        print("Error: No se puede dividir por cero")

```

division_con_reintentos()

Ejercicio 11

Enunciado: Escribe una funcion que reciba una lista mixta (con numeros y texto) y devuelva la suma de los numeros. Los elementos que no sean numeros deben ser ignorados.

Solucion paso a paso:

```

def suma_segura(lista_mixta):
    suma = 0
    ignorados = 0
    for elemento in lista_mixta:
        try:
            suma += float(elemento)
        except (ValueError, TypeError):
            ignorados += 1
    return suma, ignorados

datos = [10, "20", "hola", 30, None, "40", "mundo"]
total, ignorados = suma_segura(datos)
print(f"Suma: {total}, Elementos ignorados: {ignorados}")

```

Ejercicio 12

Enunciado: Escribe un programa que intente acceder a una variable que no esta definida y capture el NameError.

Solucion paso a paso:

```

try:
    print(variable_no_definida)
except NameError as error:
    print(f"Error: La variable no esta definida - {error}")

```

Ejercicio 13

Enunciado: Escribe una funcion que divida dos numeros pero que devuelva None si ocurre cualquier error, sin mostrar mensajes.

Solucion paso a paso:

```

def division_silenciosa(a, b):
    try:
        return a / b
    
```

```
except:
    return None

print(division_silenciosa(10, 2))    # 5.0
print(division_silenciosa(10, 0))    # None
print(division_silenciosa("10", 2)) # None
```

Ejercicio 14

Enunciado: Escribe un programa que abra un archivo y use finally para asegurarse de que se cierra, incluso si hay un error.

Solucion paso a paso:

```
archivo = None
try:
    archivo = open("mi_archivo.txt", "r")
    contenido = archivo.read()
    print(contenido)
except FileNotFoundError:
    print("El archivo no existe")
finally:
    if archivo:
        archivo.close()
        print("Archivo cerrado correctamente")
```

Ejercicio 15

Enunciado: Escribe una funcion que convierta un string a entero, pero si falla, que devuelva 0 en lugar de romperse.

Solucion paso a paso:

```
def entero_seguro(texto):
    try:
        return int(texto)
    except ValueError:
        return 0

print(entero_seguro("123"))    # 123
print(entero_seguro("hola"))   # 0
print(entero_seguro("45.6"))   # 0 (porque no es entero)
```

Ejercicio 16

Enunciado: Escribe un programa que pida al usuario un numero y calcule su raiz cuadrada. Manejar ValueError si ingresa un numero negativo y ValueError si ingresa texto.

Solucion paso a paso:

```
import math

try:
```

```
numero = float(input("Ingrese un numero positivo: "))
if numero < 0:
    raise ValueError("No se puede calcular raiz de un numero negativo")
resultado = math.sqrt(numero)
print(f"La raiz cuadrada es: {resultado}")
except ValueError as e:
    print(f"Error: {e}")
```

Ejercicio 17

Enunciado: Escribe una funcion que reciba una lista de indices y una lista principal, y devuelva una lista con los elementos. Si un indice no existe, poner None en su lugar.

Solucion paso a paso:

```
def obtener_elementos_seguros(lista_principal, indices):
    resultados = []
    for i in indices:
        try:
            resultados.append(lista_principal[i])
        except IndexError:
            resultados.append(None)
    return resultados
```

```
colores = ["rojo", "verde", "azul"]
indices = [0, 1, 5, 2, 10]
print(obtener_elementos_seguros(colores, indices))
# ['rojo', 'verde', None, 'azul', None]
```

Ejercicio 18

Enunciado: Escribe un programa que use try-except-else-finally para mostrar un mensaje diferente en cada bloque.

Solucion paso a paso:

```
try:
    num = int(input("Ingrese un numero: "))
    resultado = 100 / num
except ValueError:
    print("Bloque except: No ingresaste un numero")
except ZeroDivisionError:
    print("Bloque except: No se puede dividir por cero")
else:
    print(f"Bloque else: El resultado es {resultado}")
finally:
    print("Bloque finally: Esto se ejecuta siempre")
```

Ejercicio 19

Enunciado: Escribe una funcion que intente convertir una lista de strings a floats. Si hay error en algun elemento, registrar el error pero seguir con los demas.

Solucion paso a paso:

```
def convertir_floats(lista_strings):
    resultado = []
    errores = []
    for i, elemento in enumerate(lista_strings):
        try:
            resultado.append(float(elemento))
        except ValueError as e:
            errores.append((i, elemento, str(e)))
    return resultado, errores
```

```
datos = ["10.5", "20", "hola", "30.7", "mundo"]
floats, errores = convertir_floats(datos)
print(f"Convertidos: {floats}")
print(f"Errores: {errores}")
```

Ejercicio 20

Enunciado: Escribe un programa simulando un cajero automatico que maneje saldo insuficiente como una excepcion personalizada.

Solucion paso a paso:

```
class SaldoInsuficienteError(Exception):
    pass

def retirar(saldo, monto):
    if monto > saldo:
        raise SaldoInsuficienteError(f"Saldo insuficiente. Saldo: {saldo}, Monto: {monto}")
    return saldo - monto

saldo_actual = 1000
try:
    monto = float(input("Monto a retirar: "))
    nuevo_saldo = retirar(saldo_actual, monto)
    print(f"Retiro exitoso. Nuevo saldo: {nuevo_saldo}")
except SaldoInsuficienteError as e:
    print(f"Error: {e}")
except ValueError:
    print("Error: Ingrese un monto valido")
```

Resumen del capitulo

Dale, repasemos rapido lo importante:

- **try** -> bloque donde pones codigo que puede fallar
- **except** -> bloque que se ejecuta si ocurre un error
- **as** -> guarda el mensaje de error original

- **Exception** -> atrapa cualquier error no capturado antes
- **else** -> se ejecuta SOLAMENTE si no hubo errores
- **finally** -> se ejecuta SIEMPRE, haya o no error

La estructura completa es:

```
try:
    # codigo que puede fallar
except TipoError as variable:
    # que hacer si pasa ese error
except Exception as e:
    # que hacer con cualquier otro error
else:
    # si todo salio bien
finally:
    # siempre, pase lo que pase
```

Reflexion final

Querido lector, aprender a manejar excepciones es lo que separa a un programa fragil (que se rompe con cualquier cosa) de un programa robusto y profesional.

No se trata de escribir codigo perfecto. Se trata de anticipar que el usuario, el sistema o el destino van a hacer algo que no esperas. Y cuando eso pase, tu programa no se va a caer. Va a responder con gracia.

Empeza incorporando `try/except` en tus `input()` cuando pidas numeros. Despues, usalo cada vez que abras un archivo. Despues, cuando accedas a diccionarios o listas.

Con practica, vas a hacerlo automatico. Y tus programas van a ser mucho mejores.

El codigo robusto no es el que nunca falla. Es el que, cuando falla, sabe como recuperarse.

PyInstaller: Como convertir tu script Python en un ejecutable

Querido lector: tus programas pueden vivir fuera de Python

Mira, cuando yo empecé a programar, creía que para ejecutar un programa Python tenías que tener Python instalado. Y tenía razón... pero hay una trampa.

Imaginate que le quieres pasar un programa a tu amigo que no sabe nada de programación. O a tu abuela. O a un cliente. No puedes pedirles que instalen Python, que abran la terminal, que ejecuten `python mi_programa.py`. No funciona así.

La solución es crear un **ejecutable**. Un archivo que se abre haciendo doble click, como cualquier programa de la computadora. En Windows es un `.exe`. En Linux o Mac es un binario. Pero la idea es la misma: un archivo autosuficiente que contiene todo lo necesario para correr tu programa.

En este capítulo vamos a aprender a usar **PyInstaller**, la herramienta que hace toda la magia.

La idea central: PyInstaller empaqueta el intérprete de Python, tu script y todas las librerías que usa dentro de un solo archivo. El usuario final solo ve tu programa, nada de tecnicismos.

El problema invisible: la terminal se cierra

Antes de compilar, tengo que contarte algo que me pasó y me volvió loco.

Hice mi primer ejecutable. Todo perfecto. Abro el archivo... y la ventana aparece y desaparece en una fracción de segundo. No vi nada. No sabía si funcionó o no.

Resulta que los programas de consola terminan su ejecución y la ventana se cierra. Si tu programa hace todo rápido, el usuario no ve nada.

La solución: agregar una pausa al final.

```
# Todo tu código del programa aca
# ...

# Al final, esto mantiene la ventana abierta
input("Presiona Enter para salir...")
```

Esa línea espera a que el usuario presione una tecla. La ventana se queda abierta hasta que lo hace. Magia simple pero fundamental.

Ejemplo completo con pausa

```
# mi_programa.py
nombre = input("Como te llamas? ")
print(f"Hola {nombre}!")
print("Bienvenido a mi programa")

# Esta linea es la clave
input("Presiona Enter para salir...")
```

Que es PyInstaller y para que sirve?

PyInstaller es una herramienta que lee tu script, encuentra todas las librerías que usa (hasta las mas escondidas), y las empaqueta junto con una copia del interprete de Python.

El resultado es un solo archivo que:

- No necesita Python instalado para correr
- Incluye todas las dependencias
- Se puede ejecutar con doble click
- Se puede compartir sin explicaciones tecnicas

Hay otras herramientas (como cx_Freeze o Nuitka), pero PyInstaller es la mas popular y facil de usar.

Guia para Windows (paso a paso)

Paso 1: Abrir la terminal (cmd)

Apreta las teclas Windows + R. Se abre una ventana “Ejecutar”. Escribi cmd y presiona Enter.

Paso 2: Verificar que pip esta instalado

En la terminal, escribi:

```
pip --version
```

Si ves algo como pip 24.0 from ..., estas listo. Si dice “no se reconoce como comando”, tenes que instalar pip.

Paso 3: Instalar pip (si no lo tenes)

Si no tenes pip, descarga el script get-pip.py desde la pagina oficial de Python. Luego, en la terminal:

```
python get-pip.py
```

Paso 4: Instalar PyInstaller

Una vez que pip funciona, instalas PyInstaller:

```
pip install pyinstaller
```

Espera a que termine. Aparecera algo como "Successfully installed".

Paso 5: Navegar a la carpeta de tu script

Usa el comando `cd` (change directory) para ir a donde guardaste tu script. Por ejemplo, si esta en el Escritorio:

```
cd Desktop
```

Si esta en una carpeta especifica:

```
cd C:\Users\TuUsuario\Documents\mi_proyecto
```

Paso 6: Compilar tu programa

Ejecuta el comando magico:

```
pyinstaller --onefile mi_programa.py
```

El `-onefile` es importante; le dice a PyInstaller que cree UN SOLO archivo ejecutable. Sin esa opcion, crearia una carpeta con muchos archivos.

Paso 7: Encontrar tu ejecutable

PyInstaller crea varias carpetas:

- `build` â archivos temporales (los puedes borrar)
- `dist` â **Â¡ACA ESTA TU EJECUTABLE!**
- `__pycache__` â cache de Python (lo puedes borrar)
- `mi_programa.spec` â archivo de configuracion (se puede borrar)

Entra a la carpeta `dist`. Ahi esta `mi_programa.exe`. Ese es tu programa. Puedes copiarlo a cualquier lado. Puedes mandarselo a tus amigos.

Paso 8: Probar tu ejecutable

Hace doble click en `mi_programa.exe`. Si agregaste la linea `input("Presiona Enter...")` al final, la ventana se va a quedar abierta hasta que presiones una tecla.

Errores comunes en Windows

Error: 'pyinstaller' no se reconoce

Solucion: PyInstaller no esta en el PATH. Proba con:

```
python -m PyInstaller --onefile mi_programa.py
```

Error: No se encuentra el archivo

Solucion: Asegurate de estar en la carpeta correcta con `cd`. Usa `dir` para ver los archivos de la carpeta actual.

Guia para Linux (Ubuntu) y Mac

El proceso es muy parecido, solo cambian algunos comandos.

Paso 1: Abrir la terminal

En Ubuntu: `Ctrl + Alt + T`

En Mac: `Cmd + Espacio`, escribi "Terminal".

Paso 2: Instalar pip (si no lo tenes)

En Ubuntu:

```
sudo apt-get install python3-pip
```

En Mac:

```
python3 -m ensurepip
```

Paso 3: Instalar PyInstaller

```
pip install pyinstaller
```

O en algunos sistemas:

```
pip3 install pyinstaller
```

Paso 4: Navegar a tu script

Usa `cd` para ir a la carpeta:

```
cd ~/Documentos/mi_proyecto
```

Paso 5: Compilar

```
pyinstaller --onefile mi_programa.py
```

Paso 6: Ejecutar el binario

En Linux y Mac, no puedes hacer doble click como en Windows. Tienes que ejecutarlo desde la terminal:

```
cd dist
./mi_programa
```

El `./` es importante: le dice a la terminal "busca el archivo en la carpeta actual".

Dar permisos de ejecucion (si es necesario)

Si el archivo no se ejecuta, puede que necesites darle permisos:

```
chmod +x mi_programa
```

Errores comunes en Linux/Mac

Error: Permission denied

Solucion: Usa `sudo` para instalar:

```
sudo pip install pyinstaller
```

Error: command not found: pyinstaller

Solucion: Usa `python3 -m PyInstaller` en lugar de `pyinstaller`.

Opciones utiles de PyInstaller

Opcion	Que hace
<code>-onefile</code>	Crea un solo archivo ejecutable
<code>-windowed</code>	No abre terminal (para programas con interfaz grafica)
<code>-name "MiApp"</code>	Cambia el nombre del ejecutable
<code>-icon "logo.ico"</code>	Agrega un icono al ejecutable (Windows)
<code>-add-data carpeta</code>	Incluye archivos extra (imagenes, datos)
<code>-hidden-import modulo</code>	Incluye modulos que PyInstaller no detecta solo
<code>-noconsole</code>	Igual que <code>-windowed</code> (oculta la terminal)

Ejemplo con varias opciones

```
pyinstaller --onefile --windowed --name "MiCalculadora" --icon "calc.ico" calculadora.py
```

Incluir archivos adicionales (imagenes, datos)

Si tu programa usa imagenes, archivos de texto o datos externos, PyInstaller no los incluye automaticamente. Tienes que decirle.

Opcion 1: Usar -add-data

```
# Windows
pyinstaller --onefile --add-data "imagenes;imagenes" mi_programa.py

# Linux/Mac
pyinstaller --onefile --add-data "imagenes:imagenes" mi_programa.py
```

Opcion 2: Carpeta "data" en el spec file

Para casos mas complicados, creas un archivo .spec y lo modificas. Pero eso es mas avanzado. Por ahora, -add-data alcanza.

Como acceder a los archivos desde el codigo

Cuando compilas, los archivos se mueven. Usa esta funcion para encontrarlos sin importar si estas en desarrollo o compilado:

```
import sys
import os

def resource_path(relative_path):
    try:
        base_path = sys._MEIPASS
    except Exception:
        base_path = os.path.abspath(".")
    return os.path.join(base_path, relative_path)

# Para usar imagenes
ruta_imagen = resource_path("imagenes/foto.jpg")
```

sys._MEIPASS es una variable especial que PyInstaller crea para decirte donde estan los archivos extra.

Proyecto completo: crear un ejecutable profesional

Vamos a hacer un programa completo y luego compilarlo.

Paso 1: El programa (calculadora.py)

```
import sys
import os

def resource_path(relative_path):
    try:
        base_path = sys._MEIPASS
    except Exception:
        base_path = os.path.abspath(".")
    return os.path.join(base_path, relative_path)
```



```
def main():
    print("=== MI CALCULADORA ===")
    print("1. Sumar")
    print("2. Restar")
    print("3. Multiplicar")
    print("4. Dividir")

    opcion = input("Elige una opcion: ")
    num1 = float(input("Primer numero: "))
    num2 = float(input("Segundo numero: "))

    if opcion == "1":
        print(f"Resultado: {num1 + num2}")
    elif opcion == "2":
        print(f"Resultado: {num1 - num2}")
    elif opcion == "3":
        print(f"Resultado: {num1 * num2}")
    elif opcion == "4":
        if num2 != 0:
            print(f"Resultado: {num1 / num2}")
        else:
            print("Error: No se puede dividir por cero")
    else:
        print("Opcion invalida")

    input("Presiona Enter para salir...")

if __name__ == "__main__":
    main()
```

Paso 2: Compilar

```
pyinstaller --onefile --name "Calculadora" calculadora.py
```

Paso 3: Probar

Entra a la carpeta dist y ejecuta Calculadora.exe (Windows) o ./Calculadora (Linux/Mac).

20 Ejercicios de PyInstaller

Ejercicio 1

Enunciado: Crea un programa que salude al usuario y espere a que presione Enter antes de cerrarse.

Solucion paso a paso:

```
# saludar.py
nombre = input("Como te llamas? ")
print(f"Hola {nombre}!")
input("Presiona Enter para salir...")
```

```
# Compilar con:
# pyinstaller --onefile saludar.py
```

Ejercicio 2

Enunciado: Compila un programa que sume dos numeros y agregale un icono (busca un .ico en internet).

Solucion paso a paso:

```
# suma.py
num1 = float(input("Primer numero: "))
num2 = float(input("Segundo numero: "))
print(f"Suma: {num1 + num2}")
input("Presiona Enter para salir...")

# Compilar con icono:
# pyinstaller --onefile --icon "icono.ico" suma.py
```

Ejercicio 3

Enunciado: Crea un programa de lista de tareas y compilalo con el nombre "MisTareas".

Solucion paso a paso:

```
# tareas.py
tareas = []

while True:
    print("\n--- LISTA DE TAREAS ---")
    print("1. Agregar tarea")
    print("2. Ver tareas")
    print("3. Salir")

    op = input("Opcion: ")
    if op == "1":
        tarea = input("Tarea: ")
        tareas.append(tarea)
        print("Tarea agregada")
    elif op == "2":
        for i, t in enumerate(tareas):
            print(f"{i+1}. {t}")
    elif op == "3":
        break
    input("Presiona Enter para continuar...")
```

```
# Compilar con:
# pyinstaller --onefile --name "MisTareas" tareas.py
```

Ejercicio 4

Enunciado: Compila un programa de calculadora de IMC.

Solucion paso a paso:

```
# imc.py
peso = float(input("Peso (kg): "))
altura = float(input("Altura (m): "))
imc = peso / (altura ** 2)
print(f"Tu IMC es: {imc:.2f}")

if imc < 18.5:
    print("Bajo peso")
elif imc < 25:
    print("Normal")
elif imc < 30:
    print("Sobrepeso")
else:
    print("Obesidad")

input("Presiona Enter para salir...")

# Compilar con:
# pyinstaller --onefile --name "CalculadoraIMC" imc.py
```

Ejercicio 5

Enunciado: Compila un programa de juego de adivinanza.

Solucion paso a paso:

```
# adivina.py
import random

secreto = random.randint(1, 100)
intentos = 0

print("Adivina el numero entre 1 y 100")

while True:
    try:
        guess = int(input("Tu numero: "))
        intentos += 1
        if guess < secreto:
            print("Mas alto")
        elif guess > secreto:
            print("Mas bajo")
```

```

        else:
            print(f"Correcto! Lo lograste en {intentos} intentos")
            break
    except ValueError:
        print("Ingresa un numero valido")

input("Presiona Enter para salir...")

# Compilar con:
# pyinstaller --onefile --name "Adivina" adivina.py

```

Ejercicio 6

Enunciado: Compila un programa sin consola (-windowed) que muestre un mensaje usando tkinter.

Solucion paso a paso:

```

# ventana.py
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.withdraw()
messagebox.showinfo("Mensaje", "Hola desde mi ejecutable!")

# Compilar sin consola:
# pyinstaller --onefile --windowed ventana.py

```

Ejercicio 7

Enunciado: Compila un programa que incluya un archivo .txt con datos usando -add-data.

Solucion paso a paso:

```

# lector_datos.py
import sys
import os

def resource_path(relative_path):
    try:
        base_path = sys._MEIPASS
    except Exception:
        base_path = os.path.abspath(".")
    return os.path.join(base_path, relative_path)

ruta_datos = resource_path("datos.txt")
with open(ruta_datos, "r") as f:
    print(f.read())

input("Presiona Enter para salir...")

```

```
# Crear datos.txt con contenido
# Compilar:
# pyinstaller --onefile --add-data "datos.txt;." lector_datos.py
# En Linux/Mac usar "datos.txt:."
```

Ejercicio 8

Enunciado: Compila un programa que use varias imagenes y empaquetarlas juntas.

Solucion paso a paso:

```
# mostrar_imagen.py
import sys
import os
from PIL import Image

def resource_path(relative_path):
    try:
        base_path = sys._MEIPASS
    except Exception:
        base_path = os.path.abspath(".")
    return os.path.join(base_path, relative_path)

ruta_img = resource_path("imagenes/foto.jpg")
img = Image.open(ruta_img)
img.show()

input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile --add-data "imagenes;imagenes" mostrar_imagen.py
```

Ejercicio 9

Enunciado: Compila un programa que genere un numero aleatorio y lo guarde en un archivo.

Solucion paso a paso:

```
# guardar_numero.py
import random
import os

numero = random.randint(1, 1000)
with open("resultado.txt", "w") as f:
    f.write(f"Numero generado: {numero}")

print(f"Numero {numero} guardado en resultado.txt")
input("Presiona Enter para salir...")
```

```
# Compilar:
# pyinstaller --onefile --name "Generador" guardar_numero.py
```

Ejercicio 10

Enunciado: Compila un programa que lea la hora actual y muestre si es mañana, tarde o noche.

Solucion paso a paso:

```
# hora.py
import datetime

ahora = datetime.datetime.now()
hora = ahora.hour

if hora < 12:
    print("Buenos días!")
elif hora < 18:
    print("Buenas tardes!")
else:
    print("Buenas noches!")

print(f"Son las {hora}:{ahora.minute:02d}")
input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile --name "Saludo" hora.py
```

Ejercicio 11

Enunciado: Compila un programa de agenda de contactos simple.

Solucion paso a paso:

```
# agenda.py
agenda = {}

while True:
    print("\n--- AGENDA ---")
    print("1. Agregar contacto")
    print("2. Buscar contacto")
    print("3. Ver todos")
    print("4. Salir")

    op = input("Opcion: ")
    if op == "1":
        nombre = input("Nombre: ")
        telefono = input("Telefono: ")
        agenda[nombre] = telefono
        print("Contacto agregado")
```

```
elif op == "2":
    nombre = input("Nombre a buscar: ")
    if nombre in agenda:
        print(f"{nombre}: {agenda[nombre]}")
    else:
        print("No existe")
elif op == "3":
    for n, t in agenda.items():
        print(f"{n}: {t}")
elif op == "4":
    break
input("Presiona Enter...")
```

Compilar:

pyinstaller --onefile --name "MiAgenda" agenda.py

Ejercicio 12

Enunciado: Compila un programa que convierta divisas (dolar a peso, euro a dolar, etc.).

Solucion paso a paso:

```
# conversor.py
print("=== CONVERSION DE DIVISAS ===")
print("1. USD a ARS")
print("2. ARS a USD")
print("3. EUR a USD")

op = input("Opcion: ")
monto = float(input("Monto: "))

if op == "1":
    resultado = monto * 1200
    print(f"{monto} USD = {resultado} ARS")
elif op == "2":
    resultado = monto / 1200
    print(f"{monto} ARS = {resultado:.2f} USD")
elif op == "3":
    resultado = monto * 1.08
    print(f"{monto} EUR = {resultado:.2f} USD")

input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile conversor.py
```

Ejercicio 13

Enunciado: Compila un programa que simule un lanzamiento de dados y guarde el historial.

Solucion paso a paso:

```
# dados.py
import random
historial = []

while True:
    print("\n1. Lanzar dado")
    print("2. Ver historial")
    print("3. Salir")
    op = input("Opcion: ")

    if op == "1":
        dado = random.randint(1, 6)
        historial.append(dado)
        print(f"Resultado: {dado}")
    elif op == "2":
        print(f"Historial: {historial}")
        if historial:
            print(f"Promedio: {sum(historial)/len(historial):.2f}")
        else:
            print("Sin datos")
    elif op == "3":
        break
    input("Presiona Enter...")

# Compilar:
# pyinstaller --onefile dados.py
```

Ejercicio 14

Enunciado: Compila un programa que pida una contraseña y solo permita acceso con la correcta (tres intentos).

Solucion paso a paso:

```
# login.py
clave_correcta = "python123"
intentos = 0

while intentos < 3:
    clave = input("Ingrese la contraseña: ")
    if clave == clave_correcta:
        print("Acceso concedido!")
        print("Bienvenido al sistema")
        break
    else:
        intentos += 1
        print(f"Contraseña incorrecta. Intentos restantes: {3 - intentos}")
else:
    print("Demasiados intentos. Acceso denegado")
```



```
input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile --name "LoginSeguro" login.py
```

Ejercicio 15

Enunciado: Compila un programa que muestre informacion del sistema operativo y del hardware.

Solucion paso a paso:

```
# info_sistema.py
import os
import platform
import sys

print("=== INFO DEL SISTEMA ===")
print(f"Sistema: {platform.system()}")
print(f"Version: {platform.version()}")
print(f"Procesador: {platform.processor()}")
print(f"Python: {sys.version}")
print(f"Arquitectura: {platform.machine()}")

input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile info_sistema.py
```

Ejercicio 16

Enunciado: Compila un programa que genere una contraseña aleatoria de 12 caracteres.

Solucion paso a paso:

```
# password_generator.py
import random
import string

longitud = 12
caracteres = string.ascii_letters + string.digits + string.punctuation
password = ''.join(random.choice(caracteres) for _ in range(longitud))

print(f"Contraseña generada: {password}")
input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile --name "GeneradorPass" password_generator.py
```

Ejercicio 17

Enunciado: Compila un programa que cuente cuantas veces aparece una letra en una frase ingresada por el usuario.

Solucion paso a paso:

```
# contador.py
frase = input("Ingrese una frase: ")
letra = input("Letra a contar: ")

if len(letra) == 1:
    cantidad = frase.lower().count(letra.lower())
    print(f"La letra '{letra}' aparece {cantidad} veces")
else:
    print("Ingresa una sola letra")

input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile contador.py
```

Ejercicio 18

Enunciado: Compila un programa que ordene una lista de numeros ingresados por el usuario.

Solucion paso a paso:

```
# ordenar.py
numeros = input("Ingrese numeros separados por espacios: ")
lista = [int(x) for x in numeros.split()]
lista.sort()
print(f"Numeros ordenados: {lista}")
input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile --name "Ordenador" ordenar.py
```

Ejercicio 19

Enunciado: Compila un programa que calcule el factorial de un numero (usando recursion).

Solucion paso a paso:

```
# factorial.py
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

try:
    num = int(input("Numero: "))
```

```
if num < 0:
    print("No existe factorial de numeros negativos")
else:
    print(f"{num}! = {factorial(num)}")
except ValueError:
    print("Ingresa un numero entero")

input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile factorial.py
```

Ejercicio 20

Enunciado: Compila un programa completo estilo "juego de aventura" con opciones y diferentes caminos.

Solucion paso a paso:

```
# aventura.py
def historia():
    print("=== AVENTURA EN EL BOSQUE ===")
    print("Estas perdido en un bosque oscuro.")
    print("1. Ir al norte")
    print("2. Ir al sur")
    print("3. Quedarte quieto")

    op = input("Que haces? ")
    if op == "1":
        print("Encuentras un rio. Tomas agua y sobrevives. GANASTE!")
    elif op == "2":
        print("Te encuentras con un lobo. PERDISTE!")
    elif op == "3":
        print("Te quedaste esperando y te encontraron. GANASTE!")
    else:
        print("Opcion invalida. PERDISTE!")

historia()
input("Presiona Enter para salir...")

# Compilar:
# pyinstaller --onefile --name "Aventura" aventura.py
```

Resumen del capitulo

En este capitulo aprendiste:

- Agregar una **pausa** al final con `input("Presiona Enter...")` para que la ventana no se cierre sola.

- **PyInstaller** convierte un script Python en un ejecutable independiente.
- `-onefile` genera un solo archivo (recomendado).
- `-windowed` oculta la terminal para aplicaciones con interfaz grafica.
- `-icon` agrega un icono personalizado (en Windows).
- `-add-data` incluye archivos extra (imagenes, datos).
- La funcion `resource_path` ayuda a encontrar esos archivos tanto en desarrollo como en el ejecutable.

Comandos basicos:

```
# Windows, Linux, Mac
pip install pyinstaller
pyinstaller --onefile mi_programa.py

# Con icono (Windows)
pyinstaller --onefile --icon icono.ico mi_programa.py

# Sin consola (GUI)
pyinstaller --onefile --windowed mi_programa.py

# Con archivos adicionales
pyinstaller --onefile --add-data "carpeta;carpeta" mi_programa.py
```

Reflexion final

Querido lector, poder crear un ejecutable es un antes y un despues. Pasas de ser alguien que escribe codigo a alguien que **CREA PROGRAMAS** que cualquier persona puede usar.

No necesitas que tu amigo instale Python. No necesitas explicar que es una terminal. Solo le pasas un archivo y hace doble click.

Es magia. Pero es magia real que vos podes hacer.

Compila tus programas, compartilos, mostralos. Que la gente vea lo que podes crear.

Tu codigo puede vivir fuera de tu editor. Compila tus programas y compartilos con el mundo.

Introduccion

En mis primeros meses como programador, yo tenía el problema de que sabía las piezas, pero no sabía como armarlas. Me pasaba como con un juego de Lego: tenía todas las piezas esparcidas, pero me faltaba la idea de que construir. Por eso en este capítulo vamos a hacer exactamente eso: construir proyectos completos, paso a paso, desde cero.

- Variables y tipos de datos
- Condicionales (if, elif, else)
- Bucles (while, for)
- Listas y sus metodos
- Diccionarios
- Funciones (puras e impuras)
- Modulo random
- Manejo de errores con try/except

PROYECTOS INTEGRADORES

1. Sistema de Gestion de Alumnos y Notas
2. Carrito de Compras
3. Cajero Automatico
4. Calculadora con Menu
5. Juego del Ahorcado
6. Piedra, Papel o Tijera

```
| 7. Generador de Contraseñas |
|                               |
+-----+

```

Proyecto 1: Sistema de Gestion de Alumnos y Notas

Este sistema te permite administrar alumnos y sus notas. Lo use mucho cuando daba clases particulares. Al principio lo tenia todo mezclado, hasta que aprendi a separar en funciones.

Codigo completo del sistema

```
# =====
# SISTEMA DE GESTION DE ALUMNOS Y NOTAS
# =====

# --- Funcion: mostrar_menu ---
def mostrar_menu():
    print("==== ESCUELA =====")
    print("1. Ver alumnos")
    print("2. Agregar alumno")
    print("3. Agregar nota")
    print("4. Ver promedios")
    print("5. Eliminar alumno")
    print("6. Alumno con mayor promedio")
    print("7. Modificar nota")
    print("8. Alumno con peor promedio")
    print("9. Salir")

# --- Funcion: mostrar_alumnos ---
def mostrar_alumnos(alumnos):
    print("\n--- LISTA DE ALUMNOS ---")
    for i in range(len(alumnos)):
        print(f"{i + 1}. {alumnos[i]}")
    print("-----")

# --- Funcion: agregar_alumno ---
def agregar_alumno(alumnos, notas):
    nombre = input("Ingrese el nombre del nuevo alumno: ")
    if nombre in alumnos:
        print(f"El alumno '{nombre}' ya existe.")
    else:
        alumnos.append(nombre)
        notas[nombre] = []
        print(f"Alumno '{nombre}' agregado exitosamente.")

# --- Funcion: agregar_nota ---
def agregar_nota(alumnos, notas):
    nombre = input("Ingrese el nombre del alumno: ")

```

```

if nombre in alumnos:
    try:
        nota = int(input(f"Ingrese la nota para '{nombre}': "))
        if 0 <= nota <= 10:
            notas[nombre].append(nota)
            print(f"Nota {nota} agregada a '{nombre}'.")
        else:
            print("La nota debe estar entre 0 y 10.")
    except ValueError:
        print("Entrada invalida. Ingrese un numero entero.")
else:
    print(f"El alumno '{nombre}' no existe.")

# --- Funcion: mostrar_promedios ---
def mostrar_promedios(notas):
    print("\n--- PROMEDIOS DE ALUMNOS ---")
    for alumno in notas:
        if len(notas[alumno]) > 0:
            suma_notas = sum(notas[alumno])
            promedio = suma_notas / len(notas[alumno])
            print(f"{alumno}: Promedio = {promedio:.2f}")
        else:
            print(f"{alumno}: Sin notas registradas.")
    print("-----")

# --- Funcion: eliminar_alumnos ---
def eliminar_alumnos(alumnos, notas):
    nombre = input("Ingrese el nombre del alumno a eliminar: ")
    if nombre in alumnos:
        alumnos.remove(nombre)
        if nombre in notas:
            del notas[nombre]
            print(f"Alumno '{nombre}' eliminado exitosamente.")
    else:
        print(f"El alumno '{nombre}' no se encuentra.")

# --- Funcion: mayor_promedio ---
def mayor_promedio(notas):
    mayor_promedio = -1.0
    alumno_mayor = None
    for alumno in notas:
        if len(notas[alumno]) == 0:
            promedio_actual = 0
        else:
            promedio_actual = sum(notas[alumno]) / len(notas[alumno])
        if promedio_actual > mayor_promedio:
            mayor_promedio = promedio_actual
            alumno_mayor = alumno

```

```

    if alumno_mayor is not None:
        return f"Mejor alumno: {alumno_mayor} con promedio {mayor_promedio:.2f}"
    else:
        return "No hay alumnos con notas."

# --- Funcion: peor_promedio ---
def peor_promedio(notas):
    peor_promedio = 11.0
    alumno_peor = None
    for alumno in notas:
        if len(notas[alumno]) == 0:
            promedio_actual = 0
        else:
            promedio_actual = sum(notas[alumno]) / len(notas[alumno])
            if promedio_actual < peor_promedio:
                peor_promedio = promedio_actual
                alumno_peor = alumno
    if alumno_peor is not None:
        return f"Peor alumno: {alumno_peor} con promedio {peor_promedio:.2f}"
    else:
        return "No hay alumnos con notas."

# --- Funcion: modificar_nota ---
def modificar_nota(notas):
    nombre = input("Ingrese el nombre del alumno: ")
    if nombre in notas:
        if not notas[nombre]:
            print(f"'{nombre}' no tiene notas para modificar.")
            return
        print(f"Notas actuales de {nombre}: {notas[nombre]}")
        try:
            posicion = int(input("Posicion de la nota a modificar (0, 1, 2...): "))
            if 0 <= posicion < len(notas[nombre]):
                nueva_nota = int(input("Nueva nota: "))
                if 0 <= nueva_nota <= 10:
                    notas[nombre][posicion] = nueva_nota
                    print(f"Nota modificada. Notas actuales: {notas[nombre]}")
                else:
                    print("La nota debe estar entre 0 y 10.")
            else:
                print("Posicion invalida.")
        except ValueError:
            print("Entrada invalida.")
    else:
        print(f"El alumno '{nombre}' no existe.")

# --- PROGRAMA PRINCIPAL ---
alumnos = ["Ana", "Luis", "Marta"]

```



```

notas = {
    "Ana": [8, 9],
    "Luis": [10, 7],
    "Marta": [6, 8]
}

opcion = 0

while opcion != 9:
    mostrar_menu()
    try:
        opcion = int(input("Seleccione una opcion: "))
    except ValueError:
        print("Entrada invalida. Ingrese un numero.")
        continue

    if opcion == 1:
        mostrar_alumnos(alumnos)
    elif opcion == 2:
        agregar_alumno(alumnos, notas)
    elif opcion == 3:
        agregar_nota(alumnos, notas)
    elif opcion == 4:
        mostrar_promedios(notas)
    elif opcion == 5:
        eliminar_alumnos(alumnos, notas)
    elif opcion == 6:
        print(mayor_promedio(notas))
    elif opcion == 7:
        modificar_nota(notas)
    elif opcion == 8:
        print(peor_promedio(notas))
    elif opcion == 9:
        print("Cerrando sistema. Hasta luego!")
    else:
        print("Opcion invalida.")

```

Explicacion paso a paso

Cuando yo empecé a hacer este sistema, primero lo tenía todo en el bucle principal. Era un desastre. Después aprendí a separar cada acción en una función. Fíjate:

- **Lista y diccionario:** `alumnos` es una lista con los nombres. `notas` es un diccionario donde cada clave es un alumno y el valor es otra lista con sus notas. Es como un fichero de cartón: cada alumno tiene su carpeta con sus calificaciones.
- **Validaciones:** antes de agregar una nota, pregunto si el alumno existe. También controló que la nota esté entre 0 y 10.

- **Manejo de errores:** si el usuario ingresa letras donde espero un numero, el programa no se rompe gracias a `try/except`.
- **Promedios:** sumo todas las notas y divido por la cantidad. Si no tiene notas, muestro un mensaje especial.

Proyecto 2: Carrito de Compras

Este proyecto me salvo cuando quise simular un ecommerce para un trabajo practico. La clave esta en usar una lista de diccionarios, porque cada producto tiene varias propiedades.

Codigo completo del carrito

```
# =====
# CARRITO DE COMPRAS
# =====

def mostrar_menu():
    print("\n=== CARRITO DE COMPRAS ===")
    print("1. Agregar producto")
    print("2. Ver carrito")
    print("3. Calcular total")
    print("4. Eliminar producto")
    print("5. Salir")

def agregar_producto(carrito):
    nombre = input("Nombre del producto: ")
    try:
        precio = float(input("Precio: $"))
        cantidad = int(input("Cantidad: "))
        producto = {
            "nombre": nombre,
            "precio": precio,
            "cantidad": cantidad
        }
        carrito.append(producto)
        print(f'{nombre}' agregado al carrito.")
    except ValueError:
        print("Error: Ingrese valores numericos validos.")

def ver_carrito(carrito):
    if len(carrito) == 0:
        print("El carrito esta vacio.")
        return
    print("\n--- CARRITO ---")
    for i, item in enumerate(carrito, 1):
        subtotal = item["precio"] * item["cantidad"]
        print(f"{i}. {item['nombre']} - ${item['precio']:.2f} x {item['cantidad']} = ${s
```

```
def calcular_total(carrito):
    if len(carrito) == 0:
        print("El carrito esta vacio.")
        return 0
    total = 0
    for item in carrito:
        total = total + (item["precio"] * item["cantidad"])
    print(f"Total a pagar: ${total:.2f}")
    return total

def eliminar_producto(carrito):
    if len(carrito) == 0:
        print("El carrito esta vacio.")
        return
    ver_carrito(carrito)
    try:
        indice = int(input("Numero del producto a eliminar: ")) - 1
        if 0 <= indice < len(carrito):
            eliminado = carrito.pop(indice)
            print(f"'{eliminado['nombre']}' eliminado del carrito.")
        else:
            print("Numero invalido.")
    except ValueError:
        print("Ingrese un numero valido.")

# PROGRAMA PRINCIPAL
carrito = []
while True:
    mostrar_menu()
    opcion = input("Elija una opcion: ")
    if opcion == "1":
        agregar_producto(carrito)
    elif opcion == "2":
        ver_carrito(carrito)
    elif opcion == "3":
        calcular_total(carrito)
    elif opcion == "4":
        eliminar_producto(carrito)
    elif opcion == "5":
        print("Gracias por su compra!")
        break
    else:
        print("Opcion no valida.")
```

Explicacion

Cada producto es un diccionario con tres campos: nombre, precio y cantidad. El carrito es una lista que los contiene. Cuando agrego un producto, creo ese diccionario y lo agrego con `append()`. Para eliminar, uso `pop()` con el indice (restando 1 porque el usuario ve numeros desde 1).

Proyecto 3: Cajero Automatico

Un clasico para practicar variables globales y operaciones bancarias. Al principio me costaba entender `global`, pero es simple: cuando modificas una variable fuera de la funcion, tenes que avisar.

Codigo completo del cajero

```
# =====
# CAJERO AUTOMATICO
# =====

saldo = 5000

def mostrar_saldo():
    print(f"Su saldo actual es: ${saldo}")

def depositar():
    global saldo
    try:
        monto = float(input("Monto a depositar: $"))
        if monto > 0:
            saldo += monto
            print(f"Deposito exitoso. Nuevo saldo: ${saldo:.2f}")
        else:
            print("El monto debe ser positivo.")
    except ValueError:
        print("Ingrese un monto valido.")

def retirar():
    global saldo
    try:
        monto = float(input("Monto a retirar: $"))
        if monto > 0 and monto <= saldo:
            saldo -= monto
            print(f"Retiro exitoso. Nuevo saldo: ${saldo:.2f}")
        elif monto > saldo:
            print("Saldo insuficiente.")
        else:
            print("El monto debe ser positivo.")
    except ValueError:
```

```

        print("Ingrese un monto valido.")

# PROGRAMA PRINCIPAL
while True:
    print("\n=== CAJERO AUTOMATICO ===")
    print("1. Ver saldo")
    print("2. Depositar")
    print("3. Retirar")
    print("4. Salir")

    opcion = input("Elija una opcion: ")

    if opcion == "1":
        mostrar_saldo()
    elif opcion == "2":
        depositar()
    elif opcion == "3":
        retirar()
    elif opcion == "4":
        print("Gracias por usar el cajero. Hasta luego!")
        break
    else:
        print("Opcion invalida.")

```

Proyecto 4: Calculadora con Menu

Una calculadora simple pero con funciones bien separadas. Ideal para entender como pasar parametros y devolver resultados.

Codigo completo

```

# =====
# CALCULADORA CON MENU
# =====

def suma(a, b):
    return a + b

def resta(a, b):
    return a - b

def multiplicacion(a, b):
    return a * b

def division(a, b):
    if b != 0:
        return a / b
    else:

```

```

        return "Error: Division por cero"

while True:
    print("\n=== CALCULADORA ===")
    print("1. Suma")
    print("2. Resta")
    print("3. Multiplicacion")
    print("4. Division")
    print("5. Salir")

    try:
        opcion = int(input("Elija una opcion: "))
    except ValueError:
        print("Ingrese un numero valido.")
        continue

    if opcion == 5:
        print("Gracias por usar la calculadora!")
        break

    if opcion < 1 or opcion > 4:
        print("Opcion invalida.")
        continue

    try:
        num1 = float(input("Primer numero: "))
        num2 = float(input("Segundo numero: "))
    except ValueError:
        print("Ingrese numeros validos.")
        continue

    if opcion == 1:
        print(f"{num1} + {num2} = {suma(num1, num2)}")
    elif opcion == 2:
        print(f"{num1} - {num2} = {resta(num1, num2)}")
    elif opcion == 3:
        print(f"{num1} x {num2} = {multiplicacion(num1, num2)}")
    elif opcion == 4:
        resultado = division(num1, num2)
        print(f"{num1} / {num2} = {resultado}")

```

Proyecto 5: Juego del Ahorcado

El clasico. La parte mas divertida es dibujar el ahorcado con strings multilinea. Cuando lo hice por primera vez, me confundi con los espacios, pero despues le agarre la mano.

Codigo completo

```
# =====
# JUEGO DEL AHORCADO
# =====

import random

def elige_palabra():
    palabras = ["python", "programacion", "computadora", "algoritmo",
                "desarrollo", "codigo", "variable", "funcion", "lista",
                "diccionario"]
    return random.choice(palabras)

def jugar():
    palabra_secreta = elige_palabra()
    letras_adivinadas = []
    intentos = 6

    # Representacion del ahorcado
    ahorcado = [
        '''
        +---+
        |
        |
        |
        |
        =====''',
        '''
        +---+
        |
        |
        |
        |
        =====''',
        '''
        +---+
        |
        |
        |
        |
        =====''',
        '''
        +---+
        |
        0
        |
        |
        |
        =====''',
        '''
        +---+
        |
        0
        |
        |
        |
        ====='''
    ]
```

```

    |   |
    |   |
    |   |
=====',',
',,
    +---+
    |   |
    0   |
    /|   |
    |   |
    |   |
=====',',
',,
    +---+
    |   |
    0   |
    /|\  |
    |   |
    |   |
=====',',
',,
    +---+
    |   |
    0   |
    /|\  |
    /|\  |
    |   |
    |   |
=====',',
',,
    +---+
    |   |
    0   |
    /|\  |
    /|\  |
    |   |
    |   |
=====',',
],

```

```

palabra_mostrada = ["_"] * len(palabra_secreta)

```

```

print("=== BIENVENIDO AL AHORCADO ===")

```

```

print("Tendrás 6 intentos para adivinar la palabra.")

```

```

while intentos > 0:

```

```

    print(ahorcado[6 - intentos])

```

```

    print("Palabra: " + " ".join(palabra_mostrada))

```

```

    print(f"Intentos restantes: {intentos}")

```

```

    print(f"Letras usadas: {'', '}.join(letras_adivinadas) if letras_adivinadas else ''

```



```

letra = input("Ingrese una letra: ").lower()

if len(letra) != 1 or not letra.isalpha():
    print("Ingrese una sola letra valida.")
    continue

if letra in letras_adivinadas:
    print("Ya ingresaste esa letra. Probá con otra.")
    continue

letras_adivinadas.append(letra)

if letra in palabra_secreta:
    print("Bien! La letra está en la palabra.")
    for i, caracter in enumerate(palabra_secreta):
        if caracter == letra:
            palabra_mostrada[i] = letra
else:
    print("Mal! La letra no está en la palabra.")
    intentos -= 1

if "_" not in palabra_mostrada:
    print(ahorcado[6 - intentos])
    print(f"Ganaste! La palabra era: {palabra_secreta}")
    break

if "_" in palabra_mostrada:
    print(ahorcado[6])
    print(f"Perdiste! La palabra era: {palabra_secreta}")

# PROGRAMA PRINCIPAL
jugar()

```

Proyecto 6: Piedra, Papel o Tijera

Un jueguito simple pero con varias variantes. Le agregue la opcion de "mejor de 3 rondas" para practicar contadores.

Codigo completo

```

# =====
# PIEDRA, PAPEL O TIJERA
# =====

import random

def jugar_ronda(jugador, computadora):

```

```

if jugador == computadora:
    return "Empate"
elif (jugador == "piedra" and computadora == "tijera") or \
     (jugador == "papel" and computadora == "piedra") or \
     (jugador == "tijera" and computadora == "papel"):
    return "Ganaste"
else:
    return "Perdiste"

def mostrar_menu():
    print("\n=== PIEDRA, PAPEL O TIJERA ===")
    print("1. Jugar una partida")
    print("2. Mejor de 3 rondas")
    print("3. Salir")

def jugar_partida():
    opciones = ["piedra", "papel", "tijera"]
    jugador = input("Elegi (piedra, papel, tijera): ").lower()
    if jugador not in opciones:
        print("Opcion invalida.")
        return
    computadora = random.choice(opciones)
    print(f"Computadora eligi: {computadora}")
    resultado = jugar_ronda(jugador, computadora)
    print(f"Resultado: {resultado}")

def jugar_mejor_de_3():
    opciones = ["piedra", "papel", "tijera"]
    puntos_jugador = 0
    puntos_computadora = 0
    rondas = 0

    while puntos_jugador < 3 and puntos_computadora < 3:
        rondas += 1
        print(f"\n--- RONDA {rondas} ---")
        print(f"Marcador: Jugador {puntos_jugador} - Computadora {puntos_computadora}")

        jugador = input("Elegi (piedra, papel, tijera): ").lower()
        if jugador not in opciones:
            print("Opcion invalida. Perdís la ronda.")
            puntos_computadora += 1
            continue

        computadora = random.choice(opciones)
        print(f"Computadora eligi: {computadora}")
        resultado = jugar_ronda(jugador, computadora)

        if resultado == "Ganaste":

```

```

        puntos_jugador += 1
        print("Ganaste la ronda!")
    elif resultado == "Perdiste":
        puntos_computadora += 1
        print("Perdiste la ronda!")
    else:
        print("Empate en la ronda.")

print(f"\n--- RESULTADO FINAL ---")
print(f"Marcador: Jugador {puntos_jugador} - Computadora {puntos_computadora}")
if puntos_jugador == 3:
    print("Ganaste el juego!")
else:
    print("Perdiste el juego!")

# PROGRAMA PRINCIPAL
while True:
    mostrar_menu()
    try:
        opcion = int(input("Elegí una opción: "))
    except ValueError:
        print("Ingrese un numero valido.")
        continue

    if opcion == 1:
        jugar_partida()
    elif opcion == 2:
        jugar_mejor_de_3()
    elif opcion == 3:
        print("Gracias por jugar!")
        break
    else:
        print("Opcion invalida.")

```

Proyecto 7: Generador de Contraseñas

Una herramienta super util. Aprendi a usar el modulo **string** que tiene constantes como **ascii_letters**, **digits**, **punctuation**. Es como tener una caja llena de letras, numeros y simbolos listos para usar.

Codigo completo

```

# =====
# GENERADOR DE CONTRASENAS
# =====

import random
import string

```

```
def generar_contrasena(longitud, usar_mayusculas=True, usar_minusculas=True,
                        usar_numeros=True, usar_simbolos=True):
    """Genera una contrasena segun los parametros elegidos."""
    pool = ""
    if usar_mayusculas:
        pool += string.ascii_uppercase
    if usar_minusculas:
        pool += string.ascii_lowercase
    if usar_numeros:
        pool += string.digits
    if usar_simbolos:
        pool += string.punctuation

    if not pool:
        return "Error: Seleccione al menos un tipo de caracter."

    contrasena = ""
    for i in range(longitud):
        contrasena += random.choice(pool)
    return contrasena

def validar_fortaleza(contrasena):
    """Evalua la fortaleza de una contrasena."""
    tiene_mayus = any(c.isupper() for c in contrasena)
    tiene_minus = any(c.islower() for c in contrasena)
    tiene_num = any(c.isdigit() for c in contrasena)
    tiene_sim = any(c in string.punctuation for c in contrasena)

    if len(contrasena) >= 12 and tiene_mayus and tiene_minus and tiene_num and tiene_sim:
        return "Fuerte"
    elif len(contrasena) >= 8 and ((tiene_mayus or tiene_minus) and tiene_num):
        return "Media"
    else:
        return "Debil"

# PROGRAMA PRINCIPAL
print("=== GENERADOR DE CONTRASENAS ===")
print()

while True:
    print("\n1. Generar contrasena")
    print("2. Salir")

    try:
        opcion = int(input("Opcion: "))
    except ValueError:
        print("Ingrese un numero valido.")
```

```

        continue

    if opcion == 2:
        print("Hasta luego!")
        break

    if opcion != 1:
        print("Opcion invalida.")
        continue

    try:
        longitud = int(input("Longitud de la contraseña: "))
        if longitud < 4:
            print("La longitud debe ser al menos 4 caracteres.")
            continue
    except ValueError:
        print("Ingrese un numero valido.")
        continue

    print("\nTipos de caracteres a incluir:")
    usar_mayus = input("Incluir mayusculas? (s/n): ").lower() == 's'
    usar_minus = input("Incluir minusculas? (s/n): ").lower() == 's'
    usar_nums = input("Incluir numeros? (s/n): ").lower() == 's'
    usar_sims = input("Incluir simbolos? (s/n): ").lower() == 's'

    contraseña = generar_contraseña(longitud, usar_mayus, usar_minus, usar_nums, usar_sims)
    fortaleza = validar_fortaleza(contraseña)

    print(f"\nContraseña generada: {contraseña}")
    print(f"Fortaleza: {fortaleza}")

```

20 Ejercicios Propuestos con Soluciones Explicadas Paso a Paso

Ahora que viste los proyectos completos, te propongo 20 ejercicios para que los resuelvas vos solo. Si te trabas, aca estan las soluciones explicadas. Recuerda que programar se aprende haciendo, no solo mirando.

Ejercicio 1

Agrega al sistema de alumnos la opcion de modificar el nombre de un alumno. (Actualizar tambien la clave en el diccionario de notas).

Solucion:

```

def modificar_nombre(alumnos, notas):
    viejo = input("Nombre actual del alumno: ")
    if viejo in alumnos:
        nuevo = input("Nuevo nombre: ")

```

```

    if nuevo in alumnos:
        print("Ya existe un alumno con ese nombre.")
        return
    idx = alumnos.index(viejo)
    alumnos[idx] = nuevo
    notas[nuevo] = notas.pop(viejo)
    print(f"Nombre cambiado de '{viejo}' a '{nuevo}'")
else:
    print("Alumno no encontrado.")

```

Explicacion: `index()` encuentra la posicion en la lista. `pop()` elimina la entrada del diccionario y la devuelve, así la reasignamos con la nueva clave.

Ejercicio 2

Agrega al carrito de compras un descuento del 10 % si el total supera los \$10000.

Solucion:

```

def calcular_total(carrito):
    if not carrito:
        print("Carrito vacio")
        return 0
    total = sum(item["precio"] * item["cantidad"] for item in carrito)
    if total > 10000:
        descuento = total * 0.10
        total -= descuento
        print(f"Descuento aplicado: ${descuento:.2f}")
    print(f"Total a pagar: ${total:.2f}")
    return total

```

Explicacion: Usamos una *comprension de generador* para sumar los subtotales. Luego aplicamos el 10 % si corresponde.

Ejercicio 3

Agrega al cajero automatico un historial de movimientos (depositos y retiros).

Solucion:

```

historial = []
def registrar_movimiento(tipo, monto):
    historial.append((tipo, monto, saldo))
def mostrar_historial():
    print("\n--- HISTORIAL ---")
    for mov in historial:
        print(f"{mov[0]}: ${mov[1]} - Saldo: ${mov[2]}")

```

Explicacion: Guardamos tuplas con tipo, monto y saldo despues del movimiento. Luego mostramos.

Ejercicio 4

Crea una version del juego del ahorcado con una lista de palabras predefinida de al menos 20 palabras.

Solucion: (Ya tiene 10, solo hay que extender la lista en `elige_palabra()`)

Ejercicio 5

Agrega al generador de contraseñas la opción de guardar las contraseñas generadas en un archivo de texto.

Solucion:

```
def guardar_contraseña(contraseña, archivo="contraseñas.txt"):
    with open(archivo, "a") as f:
        f.write(contraseña + "\n")
    print("Contraseña guardada.")
```

Explicacion: Abrimos en modo `.append("a")` para agregar al final sin borrar las anteriores.

Ejercicio 6

Crea un juego de "Preguntados" con 5 preguntas de opción múltiple. Suma puntos por cada acierto.

Solucion:

```
preguntas = [
    {"pregunta": "¿Color del cielo?", "opciones": ["rojo", "verde", "azul"], "respuesta": "verde"},
    ...
]
puntaje = 0
for i, p in enumerate(preguntas, 1):
    print(f"{i}. {p['pregunta']}")
    for idx, opt in enumerate(p['opciones'], 1):
        print(f"{idx}. {opt}")
    resp = input("Tu respuesta: ")
    if p['opciones'][int(resp)-1] == p['respuesta']:
        puntaje += 1
print(f"Puntaje: {puntaje}/{len(preguntas)}")
```

Ejercicio 7

Crea un sistema de agenda telefónica que permita guardar nombres, teléfonos y emails.

Solucion:

```
agenda = {}
def agregar_contacto():
    nombre = input("Nombre: ")
    telefono = input("Telefono: ")
    email = input("Email: ")
    agenda[nombre] = {"telefono": telefono, "email": email}
```

Ejercicio 8

Agrega al sistema de alumnos la funcion de exportar los datos (alumnos y notas) a un archivo CSV.

Solucion:

```
import csv
def exportar_csv(alumnos, notas):
    with open("alumnos.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Alumno", "Notas"])
        for a in alumnos:
            writer.writerow([a, "|".join(map(str, notas[a]))])
```

Ejercicio 9

Crea un juego de "Memoria" que muestre 3 numeros al azar y el jugador tenga que repetirlos.

Solucion:

```
import time, random
nums = [random.randint(1,9) for _ in range(3)]
print("Memoriza:", nums)
time.sleep(2)
print("\n"*50)
intento = list(map(int, input("Repetir los numeros separados por espacio: ").split()))
if intento == nums:
    print("Correcto!")
else:
    print(f"Fallaste. Era {nums}")
```

Ejercicio 10

Crea un programa que simule un juego de dados con dos jugadores. Cada jugador tira dos dados y gana el que obtenga mayor suma. Juegan 5 rondas.

Solucion:

```
import random
puntajes = [0,0]
for r in range(5):
    suma1 = random.randint(1,6)+random.randint(1,6)
    suma2 = random.randint(1,6)+random.randint(1,6)
    print(f"Ronda {r+1}: Jug1 {suma1} - Jug2 {suma2}")
    if suma1>suma2:
        puntajes[0]+=1
    elif suma2>suma1:
        puntajes[1]+=1
print(f"Final: Jug1 {puntajes[0]} - Jug2 {puntajes[1]}")
```


Ejercicio 11

Agrega validacion al generador de contraseñas para que no tenga dos caracteres iguales consecutivos.

Solucion:

```
def generar_contraseña(...):
    ...
    contraseña = ""
    ultimo = ""
    for _ in range(longitud):
        nuevo = random.choice(pool)
        while nuevo == ultimo:
            nuevo = random.choice(pool)
        contraseña += nuevo
        ultimo = nuevo
    return contraseña
```

Ejercicio 12

En el sistema de alumnos, permite buscar un alumno por nombre y mostrar sus notas y promedio.

Solucion:

```
def buscar_alumno(notas):
    nombre = input("Nombre: ")
    if nombre in notas:
        print(f"Notas: {notas[nombre]}")
        if notas[nombre]:
            print(f"Promedio: {sum(notas[nombre])/len(notas[nombre]):.2f}")
    else:
        print("No existe")
```

Ejercicio 13

Agrega al carrito la opción de aplicar un cupón de descuento (código "AHORRO10" da 10 % de descuento).

Solucion:

```
def aplicar_cupon(total):
    codigo = input("Codigo de cupon: ")
    if codigo == "AHORRO10":
        return total * 0.9
    return total
```

Ejercicio 14

Crea un programa que genere un número aleatorio entre 1 y 100 y el jugador tenga que adivinarlo. Pistas de "mas alto." "mas bajo".

Solucion:

```
secreto = random.randint(1,100)
intentos=0
while True:
    guess = int(input("Adivina: "))
    intentos+=1
    if guess == secreto:
        print(f"Acertaste en {intentos} intentos")
        break
    elif guess < secreto:
        print("Mas alto")
    else:
        print("Mas bajo")
```

Ejercicio 15

Modifica el cajero para que no permita retirar mas de 5000 por operacion.

Solucion:

```
if monto > 5000:
    print("Maximo por retiro $5000")
elif monto > saldo:
    print("Saldo insuficiente")
else:
    saldo -= monto
```

Ejercicio 16

En el juego del ahorcado, agrega la opcion de que el jugador pueda ingresar la palabra completa.

Solucion: (Dentro del bucle)

```
if len(letra) > 1:
    if letra == palabra_secreta:
        palabra_mostrada = list(palabra_secreta)
        break
    else:
        intentos -= 1
        print("Palabra incorrecta")
        continue
```

Ejercicio 17

Crea una funcion que reciba una lista de precios y devuelva el precio con IVA (21 %).

Solucion:

```
def aplicar_iva(precios):
    return [p * 1.21 for p in precios]
```

Ejercicio 18

En el sistema de alumnos, permite agregar varias notas de una vez separadas por comas.

Solucion:

```
notas_str = input("Notas separadas por coma: ")
lista_notas = [int(n) for n in notas_str.split(",") if 0<=int(n)<=10]
notas[nombre].extend(lista_notas)
```

Ejercicio 19

Agrega un temporizador al juego de preguntados: cada pregunta tiene 10 segundos para responder.

Solucion: (Usando signal o thread, pero una version simple con time)

```
import time
inicio = time.time()
respuesta = input("Respuesta: ")
if time.time() - inicio > 10:
    print("Tiempo agotado")
else:
    # evaluar respuesta
```

Ejercicio 20

Crea un programa que lea un archivo de texto y cuente la cantidad de palabras.

Solucion:

```
with open("texto.txt", "r") as f:
    contenido = f.read()
    palabras = contenido.split()
    print(f"Palabras: {len(palabras)}")
```

Resumen del Capitulo

Dale, repasemos lo importante que aprendiste en este capitulo:

- **Integrar conceptos:** combinar listas, diccionarios, bucles y funciones en programas reales.
- **Modularizar:** dividir el codigo en funciones pequenas, cada una con una responsabilidad clara.
- **Manejar errores:** usar try-except para que el programa no explote por una entrada incorrecta.
- **Validar datos:** preguntar siempre si el alumno existe, si la nota es valida, si el monto es positivo.
- **Leer y escribir archivos:** exportar datos a CSV, guardar contraseñas, etc.

- **Dar feedback al usuario:** mostrar mensajes claros de éxito o error.

La mejor manera de aprender a programar es construyendo proyectos reales. Cada uno de estos proyectos te enseña una habilidad nueva que vas a poder aplicar en tus propios desarrollos. No tengas miedo de modificar, romper y volver a armar. Así es como se aprende de verdad.

Otra forma de resolver el juego del Ahorcado

Querido lector: recordas el Ahorcado?

Seguro alguna vez jugaste al Ahorcado. Uno elige una palabra secreta, el otro tiene que adivinarla letra por letra. Por cada error, se dibuja una parte del cuerpo. Si completas el dibujo antes de adivinar, perdes.

Cuando yo empecé a programar, el Ahorcado fue uno de mis primeros proyectos. Pero lo hice mal. Todo mezclado, sin funciones, sin estructura. Funcionaba, sí. Pero cuando quería cambiarlo o agregarle algo nuevo, me quería cortar los pelos.

Por eso hoy te voy a mostrar **otra forma de resolverlo**. Una forma ordenada, con funciones bien separadas, variables que tienen sentido, y lógica clara. No es la única forma, pero es una que me salvo muchas veces.

Vamos por partes.

Que necesitamos para armar el juego?

Antes de escribir una sola línea de código, pensemos. Un Ahorcado necesita:

- Una **palabra secreta** que el jugador tenga que adivinar.
- Un lugar donde guardar las **letras que ya acerto**.
- Un contador de **fallos** (o vidas).
- Una forma de saber si ya **gano** (adivino todas las letras).
- Una forma de saber si ya **perdio** (llego al máximo de fallos).

Y todo eso tiene que estar ordenado, no todo revuelto en un solo bloque gigante.

Importar lo necesario

Lo único que necesitamos de Python por ahora es el módulo **random**, para elegir la palabra secreta al azar.

```
>>> import random
```

Funcion 1: elegir una palabra secreta

El primer problema que tuve fue como elegir la palabra. Al principio la ponía fija en el código. Un embole. Después aprendí a usar una lista y `random.choice()`.

```
def elegir_palabra():
    palabras = [
        'entropia', 'algoritmo', 'clementina',
        'criollo', 'pabellon', 'sinergia',
        'binario', 'vortice', 'matriz', 'zenit'
    ]
    return random.choice(palabras)
```

`elegir_palabra()` no recibe nada, y devuelve una palabra al azar. Simple, no?

Reflexion: Por que crees que es mejor tener la lista de palabras adentro de la función y no afuera? Que pasaria si quisiéramos cambiar la lista sin tocar el resto del código?

Funcion 2: ya gano el jugador?

Necesitamos una función que responda una pregunta muy sencilla: **El jugador ya adivino todas las letras?**

Para eso, recibimos la palabra secreta y la lista de letras que ya acertó. Recorremos cada letra de la palabra secreta. Si alguna letra no está en la lista de acertadas, todavía no gana. Si todas están, gana!

```
def verificar_ganaste(palabra_secreta, letras_acertadas):
    for letra in palabra_secreta:
        if letra not in letras_acertadas:
            return False
    return True
```

Fíjate que esta función no imprime nada. Solo devuelve `True` o `False`. Eso es una buena práctica: las funciones hacen una sola cosa y no mezclan lógica con impresión.

Funcion 3: mostrar el progreso (pero bien hecho)

Una de las cosas más lindas del Ahorcado es ir viendo como se va revelando la palabra. Por ejemplo, si la palabra es "python" y el jugador acertó 'p', 't' y 'n', debería ver algo como:

```
p _ t _ _ n
```

Vamos a hacer una función que haga eso.

```
def mostrar_progreso(palabra_secreta, letras_acertadas):
    resultado = ""
    for letra in palabra_secreta:
```

```
    if letra in letras_acertadas:
        resultado += letra
    else:
        resultado += "_"
    return resultado
```

No imprime nada. Solo construye el string y lo devuelve. Despues, en el juego principal, hacemos `print()` con lo que nos devuelve.

Funcion 4: validar la entrada del jugador

Otro problema clasico: el jugador puede meter cualquier cosa. Numeros, simbolos, frases enteras. Tenemos que asegurarnos de que solo acepte una letra (una sola, y que sea letra).

```
def validar_entrada(texto):
    if len(texto) != 1:
        return False, "Ingresaste mas de una letra"
    if not texto.isalpha():
        return False, "Solo se permiten letras (a-z)"
    return True, ""
```

Esta funcion devuelve dos cosas: un booleano que dice si es valido, y un mensaje de error. Despues, en el juego principal, manejamos el error sin desordenar la logica.

La logica principal: la funcion jugar()

Ahora viene la parte mas importante. Aca vamos a juntar todo lo que construimos.

```
def jugar():
    palabra_secreta = elegir_palabra()
    letras_acertadas = []
    letras_intentadas = []
    fallos = 0
    max_fallos = 6

    print("Bienvenido al Ahorcado!")
    print(f"La palabra tiene {len(palabra_secreta)} letras.")

    while fallos < max_fallos:
        # Mostrar progreso actual
        progreso = mostrar_progreso(palabra_secreta, letras_acertadas)
        print(f"\nPalabra: {progreso}")
        print(f"Letras intentadas: {letras_intentadas}")
        print(f"Fallos: {fallos}/{max_fallos}")

        # Pedir letra al jugador
        entrada = input("Ingresa una letra: ").lower()
```

```
# Validar entrada
valida, mensaje = validar_entrada(entrada)
if not valida:
    print(mensaje)
    continue

letra = entrada

# Evitar letras repetidas
if letra in letras_acertadas or letra in letras_intentadas:
    print("Ya intentaste esa letra. Prueba con otra.")
    continue

# Verificar si esta en la palabra
if letra in palabra_secreta:
    letras_acertadas.append(letra)
    print("Bien! Esa letra esta.")

# Verificar si gano
if verificar_ganaste(palabra_secreta, letras_acertadas):
    print(f"\nGANASTE! La palabra era: {palabra_secreta}")
    break
else:
    letras_intentadas.append(letra)
    fallos += 1
    print(f"Fallaste. Te quedan {max_fallos - fallos} intentos.")

if fallos == max_fallos:
    print(f"\nPERDISTE. La palabra era: {palabra_secreta}")
```

Que hace cada parte del bucle principal?

- `mostrar_progreso()` muestra la palabra con guiones bajos.
- `validar_entrada()` asegura que el jugador no meta cualquier cosa.
- Si la letra ya se intento (sea acertada o no), avisamos y pedimos otra.
- Si acierta, la agregamos a `letras_acertadas`.
- Si falla, la agregamos a `letras_intentadas` y aumentamos fallos.
- Si `verificar_ganaste()` da `True`, mostramos el mensaje de victoria y salimos.

El programa completo, ordenado y listo para jugar

Aca te dejo el programa entero, con todo junto y listo para correr.

```
import random
```



```
def elegir_palabra():
    palabras = [
        'entropia', 'algoritmo', 'clementina',
        'criollo', 'pabellon', 'sinergia',
        'binario', 'vortice', 'matriz', 'zenit'
    ]
    return random.choice(palabras)

def verificar_ganaste(palabra_secreta, letras_acertadas):
    for letra in palabra_secreta:
        if letra not in letras_acertadas:
            return False
    return True

def mostrar_progreso(palabra_secreta, letras_acertadas):
    resultado = ""
    for letra in palabra_secreta:
        if letra in letras_acertadas:
            resultado += letra
        else:
            resultado += "_"
    return resultado

def validar_entrada(texto):
    if len(texto) != 1:
        return False, "Ingresaste mas de una letra"
    if not texto.isalpha():
        return False, "Solo se permiten letras (a-z)"
    return True, ""

def jugar():
    palabra_secreta = elegir_palabra()
    letras_acertadas = []
    letras_intentadas = []
    fallos = 0
    max_fallos = 6

    print("Bienvenido al Ahorcado!")
    print(f"La palabra tiene {len(palabra_secreta)} letras.")

    while fallos < max_fallos:
        progreso = mostrar_progreso(palabra_secreta, letras_acertadas)
        print(f"\nPalabra: {progreso}")
        print(f"Letras intentadas: {letras_intentadas}")
        print(f"Fallos: {fallos}/{max_fallos}")

        entrada = input("Ingresa una letra: ").lower()
```

```
valida, mensaje = validar_entrada(entrada)
if not valida:
    print(mensaje)
    continue

letra = entrada

if letra in letras_acertadas or letra in letras_intentadas:
    print("Ya intentaste esa letra. Prueba con otra.")
    continue

if letra in palabra_secreta:
    letras_acertadas.append(letra)
    print("Bien! Esa letra esta.")

    if verificar_ganaste(palabra_secreta, letras_acertadas):
        print(f"\nGANASTE! La palabra era: {palabra_secreta}")
        break
else:
    letras_intentadas.append(letra)
    fallos += 1
    print(f"Fallaste. Te quedan {max_fallos - fallos} intentos.")

if fallos == max_fallos:
    print(f"\nPERDISTE. La palabra era: {palabra_secreta}")

if __name__ == "__main__":
    jugar()
```

Que diferencias hay con la version anterior?

Dale, repasemos las mejoras:

- **Funciones bien definidas:** cada una hace una sola cosa. Podes probar cada funcion por separado.
- **Variables claras:** fallos en lugar de cont, max_fallos en lugar de un numero magico.
- **No hay listas de palabras mostradas al jugador:** eso era un error grave, porque el jugador podia ver las posibles palabras.
- **Validacion separada:** la entrada del usuario se valida en su propia funcion.
- **No hay codigo repetido:** cada bloque hace algo distinto.
- **Facil de modificar:** quieres cambiar el maximo de fallos? Cambias max_fallos. Queres agregar palabras? Modificas la lista en elegir_palabra().

Ejercicios para practicar (20 ejercicios con soluciones)

Ejercicio 1: Mostrar el dibujo del ahorcado

Agrega una función que muestre el dibujo del ahorcado según la cantidad de fallos. Usa una lista de strings con los estados.

Solución:

```
def mostrar_ahorcado(fallos):
    estados = [
        """
        -----
        |       |
        |       |
        |       |
        |       |
        |       |
        -----
        """,
        """
        -----
        |       |
        0       |
        |       |
        |       |
        |       |
        -----
        """,
        # ... (completa los 7 estados)
    ]
    print(estados[fallos])
```

Explicación paso a paso: Definimos una lista `estados` con 7 strings (desde 0 fallos hasta 6). Cada string dibuja una parte del cuerpo. En la función `jugar()`, después de cada fallo llamamos a `mostrar_ahorcado(fallos)`.

Ejercicio 2: Permitir acentos sin acento

Modifica el programa para que si la palabra tiene acentos (ej: `árbol`), el jugador pueda escribir la vocal sin acento y se considere correcto.

Solución:

```
import unicodedata

def normalizar(letra):
    letra = unicodedata.normalize('NFD', letra)[0]
    return letra

def jugar():
    # ... dentro del bucle
```

```

letra = normalizar(entrada)
palabra_secreta_norm = normalizar(palabra_secreta)
if letra in palabra_secreta_norm:
    # ...

```

Explicacion: `unicodedata.normalize('NFD', ...)` separa la vocal del acento, y `[0]` toma solo la vocal base. Así "Ãa" se convierte en `.a`.

Ejercicio 3: Agregar categorías

Modifica `elegir_palabra()` para que reciba una categoría (animales, frutas, etc.) y elija de esa lista. Mostrar la categoría al jugador al principio.

Solucion:

```

def elegir_palabra(categoria):
    palabras = {
        'animales': ['perro', 'gato', 'elefante', 'jirafa'],
        'frutas': ['manzana', 'pera', 'uva', 'naranja'],
        'python': ['lista', 'tupla', 'diccionario', 'funcion']
    }
    return random.choice(palabras[categoria])

# En jugar():
categoria = input("Elegi categoria (animales/frutas/python): ")
palabra_secreta = elegir_palabra(categoria)
print(f"Categoria: {categoria}")

```

Explicacion: Usamos un diccionario que asigna cada categoría a una lista de palabras. El usuario elige y se pasa a la función.

Ejercicio 4: Llevar puntaje

Lleva un registro de cuántas partidas gana y pierde el jugador. Muestra el puntaje después de cada partida y pregunta si quiere volver a jugar.

Solucion:

```

def jugar():
    ganadas = 0
    perdidas = 0
    while True:
        # ... codigo del juego
        if fallos == max_fallos:
            perdidas += 1
        else:
            ganadas += 1
        print(f"Puntaje: Ganadas {ganadas} - Perdidas {perdidas}")
        otra = input("Jugar de nuevo? (s/n): ")
        if otra.lower() != 's':
            break

```

Explicacion: Variables `ganadas` y `perdidas` fuera del bucle principal. Al terminar cada partida, actualizamos y mostramos. Preguntamos si quiere seguir.

Ejercicio 5: Limitar intentos de letras repetidas

Evita que el jugador ingrese la misma letra repetidamente (ya lo hace) pero agrega un mensaje mas claro y un contador de intentos desperdiciados.

Solucion: (Ya implementado) Se puede agregar un contador de intentos invalidos:

```
intentos_invalidos = 0
# cuando se repite letra:
intentos_invalidos += 1
print(f"Ya intentaste esa letra (invalidos: {intentos_invalidos})")
```

Ejercicio 6: Mostrar solo las letras no acertadas en el listado

En lugar de mostrar todas las letras intentadas, muestra solo las que fallaste (las no acertadas).

Solucion:

```
# En lugar de letras_intentadas (que son los fallos), ya las tenemos
print(f"Fallos: {letras_intentadas}")
```

Ejercicio 7: Agregar pista

Cada 3 fallos, muestra una pista: la primera letra de la palabra o su longitud.

Solucion:

```
if fallos > 0 and fallos % 3 == 0:
    print(f"Pista: la palabra empieza con '{palabra_secreta[0]}')")
```

Ejercicio 8: Guardar historial de partidas en un archivo

Cada vez que termina una partida, guarda en un archivo `historial.txt` la fecha, palabra, resultado y cantidad de fallos.

Solucion:

```
import datetime
with open("historial.txt", "a") as f:
    f.write(f"{datetime.datetime.now()} - {palabra_secreta} - {'Gano' if gano else 'Perdo'}
```

Ejercicio 9: Permitir jugar con dos jugadores

Un jugador ingresa la palabra secreta (sin que el otro vea), y el otro adivina. Usa `getpass` o limpieza de pantalla.

Solucion:

```
import os
os.system("cls" if os.name == "nt" else "clear")
palabra_secreta = input("Jugador 1, ingresa la palabra secreta: ").lower()
os.system("cls" if os.name == "nt" else "clear")
# Luego el juego normal con esa palabra
```

Ejercicio 10: Agregar nivel de dificultad

Facil: 8 fallos, palabras cortas. Dificil: 4 fallos, palabras largas.

Solucion:

```
dificultad = input("Dificultad (facil/normal/dificil): ")
if dificultad == "facil":
    max_fallos = 8
    # elegir palabras de longitud <=5
elif dificultad == "dificil":
    max_fallos = 4
    # palabras largas
else:
    max_fallos = 6
```

Ejercicio 11: Mostrar el tiempo transcurrido

Mide el tiempo desde que empezo la partida hasta que termina, y muestralo.

Solucion:

```
import time
inicio = time.time()
# ... juego
fin = time.time()
print(f"Tiempo: {fin - inicio:.2f} segundos")
```

Ejercicio 12: Permitir adivinar la palabra completa

Si el jugador ingresa mas de una letra, consideralo como intento de palabra completa.

Solucion:

```
if len(entrada) > 1:
    if entrada == palabra_secreta:
        # gana
    else:
        fallos += 1
        print("Palabra incorrecta")
        continue
```

Ejercicio 13: Mostrar estadísticas de las letras

Al final, muestra la frecuencia de cada letra en la palabra secreta.

Solucion:

```
from collections import Counter
frecuencias = Counter(palabra_secreta)
print("Frecuencia de letras:", frecuencias)
```

Ejercicio 14: Modo vs computadora (la computadora elige la palabra mas dificil)

La computadora elige la palabra que maximice la cantidad de letras diferentes.

Solucion:

```
def elegir_palabra_dificil():
    palabras = [...]
    return max(palabras, key=lambda p: len(set(p)))
```

Ejercicio 15: Agregar sonidos (usando beep)

Cada vez que el jugador falla, emite un pitido.

Solucion:

```
import winsound # Windows
# o
print('\a') # en Linux/Mac
```

Ejercicio 16: Colores en la terminal

Pinta las letras acertadas de verde y los fallos de rojo.

Solucion:

```
# Usar codigos ANSI
VERDE = '\033[92m'
ROJO = '\033[91m'
RESET = '\033[0m'
print(f"{VERDE}{letra}{RESET}")
```

Ejercicio 17: Guardar la partida y retomarla

Permite guardar el estado (palabra, letras acertadas, fallos) en un archivo y cargarlo despues.

Solucion:

```
import json
def guardar_partida(palabra, acertadas, fallos):
    with open('save.json', 'w') as f:
        json.dump({'palabra': palabra, 'acertadas': acertadas, 'fallos': fallos}, f)
def cargar_partida():
    with open('save.json') as f:
        return json.load(f)
```

Ejercicio 18: Limitar el tiempo por turno

Cada turno tiene 10 segundos para ingresar una letra; si no, se considera fallo.

Solucion:

```
import signal
def timeout_handler(signum, frame):
    raise TimeoutError
signal.signal(signal.SIGALRM, timeout_handler)
signal.alarm(10)
try:
    entrada = input("Letra: ")
    signal.alarm(0)
except TimeoutError:
    print("Tiempo agotado! Fallaste.")
    fallos += 1
```

Ejercicio 19: Soporte para mayusculas y minusculas sin distincion

Ya lo tenemos con `.lower()`.

Ejercicio 20: Interfaz grafica simple con turtle

Dibuja el ahorcado con turtle en lugar de texto.

Solucion: (Esquema)

```
import turtle
def dibujar_ahorcado(fallos):
    # dibujar horca, cabeza, cuerpo, etc segun fallos
    pass
```

Reflexion final

Mira, querido lector, el Ahorcado es un jueguito sencillo, pero la forma en que lo resuelve te ensena muchisimo sobre como organizar tu codigo. Las funciones no son un adorno. Son la diferencia entre un programa que entiendes y puedes modificar, y un choclazo que funciona de pedo.

La proxima vez que te enfrentes a un problema, piensa: puedo dividirlo en pedacitos mas chicos? Cada pedacito puede tener su propia funcion? Si la respuesta es si, vas por buen camino.

El buen codigo no es el que funciona.

Es el que puedes entender y mejorar seis meses despues.

Tuplas: Las listas que no cambian

Querido lector: imagínate una lista con candado

Cuando yo empecé a programar, usaba listas para todo. Guardaba nombres, números, lo que sea. Pero un día me pasó algo: en medio del programa, sin querer, modifiqué una lista que no debía tocar. Y todo se rompió.

Ahí entendí que a veces necesitas una estructura que se comporte como una lista, pero que sea **immutable**, que no se pueda modificar una vez creada. En la vida real pensalo como una caja fuerte: una vez que cerraste la combinación, no puedes cambiar lo que hay adentro (a menos que la rompas).

En Python, esa estructura se llama **tupla**. Se parece mucho a una lista, pero con una diferencia enorme: no se puede modificar después de crearla.

Que es una tupla y como se crea

Una tupla es una colección ordenada de elementos, igual que una lista. Se crea con parentesis '()' en lugar de corchetes '[]'.

```
# Lista (mutable)
mi_lista = [1, 2, 3]

# Tupla (immutable)
mi_tupla = (1, 2, 3)
```

Fíjate que la sintaxis es casi idéntica, solo cambian los parentesis por corchetes.

Tupla de un solo elemento (el truco del diablo)

Cuando quieras una tupla con un solo elemento, tienes que poner una coma después del elemento. Si no, Python no entiende que es una tupla.

```
>>> tupla_un_elemento = (5,)
>>> type(tupla_un_elemento)
<class 'tuple'>
>>> no_es_tupla = (5)
>>> type(no_es_tupla)
<class 'int'>
```

Esa coma es la que le dice a Python: “esto es una tupla, no un parentesis de agrupación”. Me costó un par de errores entenderlo, pero ahora no se me olvida más.

Tupla vacia

Si quieres una tupla sin nada adentro:

```
tupla_vacia = ()
```

Sin parentesis (empaquetado de tuplas)

Python tambien te permite crear tuplas sin usar parentesis. Solo separas los valores con comas.

```
>>> colores = "rojo", "verde", "azul"
>>> type(colores)
<class 'tuple'>
>>> print(colores)
('rojo', 'verde', 'azul')
```

Esto se llama “tuple packing” (empaquetado de tuplas). Es muy util cuando quieres devolver multiples valores de una funcion.

Acceder a los elementos (igual que en las listas)

Para acceder a un elemento de una tupla, usas los mismos corchetes y los mismos indices (empiezan en 0).

```
>>> frutas = ("manzana", "pera", "uva")
>>> print(frutas[0])
manzana
>>> print(frutas[-1])
uva
```

Rebanadas (slicing) tambien funcionan

```
>>> numeros = (0, 1, 2, 3, 4, 5)
>>> print(numeros[1:4])
(1, 2, 3)
>>> print(numeros[:2])
(0, 1)
```

La gran diferencia: no se puede modificar

Aca esta la cuestion. Si intentas cambiar un elemento, agregar uno nuevo o eliminar alguno, Python te va a dar un error.

```
>>> frutas = ("manzana", "pera", "uva")
>>> frutas[0] = "naranja"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Tampoco puedes usar 'append()', 'remove()', 'pop()' o 'insert()'.

```
>>> frutas.append("kiwi")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'tuple' object has no attribute 'append'
```

Entonces, para que sirve una tupla?

Buena pregunta. La inmutabilidad no es una limitación, es una **característica de seguridad**. Usa tuplas cuando:

- Tengas datos que no deben cambiar (días de la semana, coordenadas, constantes matemáticas).
- Quieras usar la estructura como clave de un diccionario (las listas no pueden ser claves, las tuplas sí).
- Quieras devolver múltiples valores de una función.
- Quieras asegurarte de que otra parte del programa no modifique tus datos.

Ejemplo: días de la semana como tupla

```
dias_semana = ("lunes", "martes", "miercoles", "jueves", "viernes", "sabado", "domingo")
print(f"El primer día es {dias_semana[0]}")
print(f"El último día es {dias_semana[-1]}")
```

Si alguien intenta hacer 'dias_semana[0] = "otrodia"', el programa se romperá y te avisará del error. Es una forma

Operaciones permitidas con tuplas

Aunque no puedes modificar una tupla, sí puedes hacer muchas otras cosas.

Concatenar tuplas (crear una nueva)

El operador '+' une dos tuplas en una nueva.

```
>>> a = (1, 2)
>>> b = (3, 4)
>>> c = a + b
>>> print(c)
(1, 2, 3, 4)
```

Repetir tuplas con *

```
>>> repite = ("ja",) * 3
>>> print(repite)
('ja', 'ja', 'ja')
```

Preguntar si un elemento existe con 'in'

```
>>> colores = ("rojo", "verde", "azul")
>>> print("rojo" in colores)
True
>>> print("amarillo" in colores)
False
```

Longitud de una tupla con 'len()'

```
>>> print(len(colores))
3
```

Recorrer una tupla con 'for'

```
for color in colores:
    print(color)
```

Empaquetado y desempaquetado de tuplas

Esto es una de las cosas mas lindas de Python.

Empaquetado (crear tupla sin parentesis)

Ya lo vimos: asignas varios valores separados por comas y Python los empaqueta en una tupla.

```
>>> coordenadas = 10, 20
>>> print(coordenadas)
(10, 20)
```

Desempaquetado (asignar a varias variables)

Haces lo inverso: una tupla se “desarma” en varias variables.

```
>>> punto = (5, 7)
>>> x, y = punto
>>> print(x)
5
>>> print(y)
7
```

Esto es super util para intercambiar valores sin usar una variable temporal.

```
>>> a = 10
>>> b = 20
>>> a, b = b, a
>>> print(a, b)
20 10
```

Desempaquetado con '*' (para el resto)

Si una tupla tiene muchos elementos y solo te interesan los primeros, puedes usar '*' para agarrar el resto.

```
>>> numeros = (1, 2, 3, 4, 5)
>>> primero, *resto = numeros
>>> print(primero)
1
>>> print(resto)
[2, 3, 4, 5] # Ojo: resto es una lista, no una tupla
```

Desempaquetado en funciones

Cuando una función devuelve múltiples valores, en realidad está devolviendo una tupla. Y vos la puedes desempaquetar directamente.

```
def dividir_con_resto(dividendo, divisor):
    cociente = dividendo // divisor
    resto = dividendo % divisor
    return cociente, resto
```

```
c, r = dividir_con_resto(10, 3)
print(f"Cociente: {c}, Resto: {r}")
```

Tuplas vs listas: cuando usar cada una

Característica	Lista	Tupla
Mutable	Si	No
Puede cambiar tamaño	Si	No
Puede ser clave de diccionario	No	Si
Velocidad	Más lenta	Más rápida
Memoria	Más memoria	Menos memoria
Sintaxis	'[1, 2, 3]'	'(1, 2, 3)'

Metodos de las tuplas (son solo dos)

Como las tuplas no se pueden modificar, tienen muy pocos métodos.

- 'count(valor)': cuenta cuantas veces aparece un valor.
- 'index(valor)': devuelve la posición de la primera aparición.

```
>>> numeros = (1, 2, 2, 3, 2, 4)
>>> print(numeros.count(2))
3
>>> print(numeros.index(3))
3
```

Ejemplos practicos

Coordenadas de un punto

```
punto = (3, 5)
x, y = punto
print(f"Punto en x={x}, y={y}")
```

Fechas como tuplas (anio, mes, dia)

```
fecha_nacimiento = (1990, 5, 15)
anio, mes, dia = fecha_nacimiento
print(f"Naci el {dia}/{mes}/{anio}")
```

RGB de un color

```
rojo = (255, 0, 0)
verde = (0, 255, 0)
azul = (0, 0, 255)

def mezclar(color1, color2):
    return (color1[0] + color2[0], color1[1] + color2[1], color1[2] + color2[2])

print(mezclar(rojo, verde))
```

20 ejercicios con soluciones paso a paso

Ejercicio 1

Crear una tupla con los numeros del 1 al 5 y mostrarla.

Solucion:

```
numeros = (1, 2, 3, 4, 5)
print(numeros)
```

Explicacion: Se usan parentesis para crear la tupla.

Ejercicio 2

Acceder al tercer elemento de la tupla '(10, 20, 30, 40, 50)'.

Solucion:

```
tupla = (10, 20, 30, 40, 50)
print(tupla[2]) # 30 (indice 2 es el tercero)
```

Explicacion: Los indices empiezan en 0: 0->10, 1->20, 2->30.

Ejercicio 3

Intentar modificar el primer elemento de una tupla y ver que error da.

Solucion:

```
tupla = (1, 2, 3)
try:
    tupla[0] = 99
except TypeError as e:
    print(f"Error: {e}")
# Esto muestra: 'tuple' object does not support item assignment
```

Explicacion: Las tuplas son inmutables, no se puede asignar.

Ejercicio 4

Concatenar dos tuplas: '(1, 2)' y '(3, 4)'.

Solucion:

```
t1 = (1, 2)
t2 = (3, 4)
t3 = t1 + t2
print(t3) # (1, 2, 3, 4)
```

Explicacion: El operador '+' une tuplas y devuelve una nueva.

Ejercicio 5

Repetir la tupla '("hola",)' tres veces.

Solucion:

```
tupla = ("hola",) * 3
print(tupla) # ('hola', 'hola', 'hola')
```

Explicacion: El operador '*' repite la tupla. La coma es necesaria para que sea tupla.

Ejercicio 6

Verificar si el numero 3 esta en la tupla '(1, 2, 3, 4, 5)'.

Solucion:

```
numeros = (1, 2, 3, 4, 5)
if 3 in numeros:
    print("Si esta")
else:
    print("No esta")
```

Explicacion: 'in' verifica pertenencia.

Ejercicio 7

Obtener la longitud de la tupla `('a', 'b', 'c', 'd')`.

Solucion:

```
tupla = ("a", "b", "c", "d")
print(len(tupla)) # 4
```

Explicacion: `len()` devuelve la cantidad de elementos.

Ejercicio 8

Recorrer la tupla `(5, 10, 15, 20)` con un bucle for e imprimir cada elemento.

Solucion:

```
numeros = (5, 10, 15, 20)
for n in numeros:
    print(n)
```

Explicacion: Se itera sobre cada elemento de la tupla.

Ejercicio 9

Crear una tupla empaquetada sin parentesis con los valores `"perro", "gato", "raton"`.

»**Solucion:**

```
»animales = "perro", "gato", "raton"
»print(animales)
»print(type(animales))
```

»**Explicacion:** Las comas crean la tupla (tuple packing).

»Ejercicio 10

»Desempaquetar la tupla `(100, 200, 300)` en tres variables y mostrarlas.

»**Solucion:**

```
»tupla = (100, 200, 300)
»a, b, c = tupla
»print(a, b, c)
```

»**Explicacion:** El desempaquetado asigna cada elemento a una variable.

»Ejercicio 11

»Intercambiar los valores de dos variables usando tuplas.

»**Solucion:**

```
»x = 5
»y = 10
»x, y = y, x
»print(x, y) # 10 5
```

»**Explicacion:** Python crea una tupla `(y, x)` y luego desempaqueta.

»Ejercicio 12

»Crear una tupla de un solo elemento con el numero 42 (correctamente).

»Solucion:

```
»tupla = (42,)
»print(tupla)
»print(type(tupla))
```

»Explicacion: La coma es obligatoria. '(42)' es un entero.

»Ejercicio 13

»Contar cuantas veces aparece el numero 2 en la tupla '(1, 2, 2, 3, 2, 4)'.

»Solucion:

```
»numeros = (1, 2, 2, 3, 2, 4)
»print(numeros.count(2)) # 3
```

»Explicacion: 'count()' cuenta ocurrencias.

»Ejercicio 14

»Encontrar el indice de la primera aparicion de "verde" en la tupla '(rojo', "verde", "azul)".

»Solucion:

```
»colores = ("rojo", "verde", "azul")
»print(colores.index("verde")) # 1
```

»Explicacion: 'index()' devuelve la posicion (0-based).

»Ejercicio 15

»Usando desempaquetado con '*', obtener el primer elemento y el resto de la tupla '(1, 2, 3, 4, 5)'.

»Solucion:

```
»numeros = (1, 2, 3, 4, 5)
»primero, *resto = numeros
»print(primero) # 1
»print(resto) # [2, 3, 4, 5]
```

»Explicacion: '*resto' captura los elementos restantes como lista.

»Ejercicio 16

»Crear una funcion que reciba dos numeros y devuelva su suma y su producto como una tupla.

»Solucion:

```
»def sumar_y_multiplicar(a, b):  
»    return a + b, a * b  
»  
»s, p = sumar_y_multiplicar(4, 5)  
»print(f"Suma: {s}, Producto: {p}")
```

»**Explicacion:** La funcion devuelve una tupla que se desempaqueta.

»Ejercicio 17

»Convertir una lista '[1, 2, 3]' en una tupla.

»**Solucion:**

```
»lista = [1, 2, 3]  
»tupla = tuple(lista)  
»print(tupla)
```

»**Explicacion:** 'tuple()' convierte cualquier iterable en tupla.

»Ejercicio 18

»Convertir una tupla '(4, 5, 6)' en una lista.

»**Solucion:**

```
»tupla = (4, 5, 6)  
»lista = list(tupla)  
»print(lista)
```

»**Explicacion:** 'list()' convierte la tupla en lista mutable.

»Ejercicio 19

»Usar una tupla como clave en un diccionario (las listas no pueden).

»**Solucion:**

```
»coordenadas = {(10, 20): "punto A", (30, 40): "punto B"}  
»print(coordenadas[(10, 20)])
```

»**Explicacion:** Las tuplas son hashables, las listas no.

»Ejercicio 20

»Dada la tupla '(.a", "b", ç", "d", .e)', obtener una nueva tupla con los elementos en posiciones pares (indice 0, 2, 4).

»**Solucion:**

```
»original = ("a", "b", "c", "d", "e")  
»nueva = original[::2]  
»print(nueva) # ('a', 'c', 'e')
```

»**Explicacion:** El slicing '[::2]' toma elementos cada 2 posiciones.

» Resumen de lo que vimos

» Dale, repasemos rapido lo importante:

- » ■ Una **tupla** es una coleccion ordenada e **immutable** (no se puede modificar).
- » ■ Se crea con parentesis '()' o sin ellos (separando valores por comas).
- » ■ Para una tupla de un solo elemento, hay que poner una coma: '(42,)'
- » ■ Se accede a los elementos igual que en las listas: 'tupla[0]'.
- » ■ No se puede modificar, agregar ni eliminar elementos (immutable).
- » ■ Se pueden concatenar ('+'), repetir ('*'), usar 'in', 'len()', recorrer con 'for'.
- » ■ Metodos: 'count()' e 'index()'.
- » ■ **Empaquetado**: crear tupla asignando valores separados por comas.
- » ■ **Desempaquetado**: asignar los elementos de una tupla a varias variables.
- » ■ Sirven como clave de diccionario (las listas no).

» Reflexion final

» Querido lector, las tuplas son como las listas pero con una promesa: no van a cambiar. Cuando veas una tupla en un codigo, puedes confiar en que su contenido se mantendra igual. Eso te ahorra dolores de cabeza.

» Usa listas cuando necesites flexibilidad. Usa tuplas cuando necesites seguridad y velocidad. Las dos son amigas, solo que una es mas “fiestera” y la otra mas “responsable”.

» _____ *La inmutabilidad no es una limitacion, es una garantia.*

»Sets (conjuntos): Cuando los elementos no se repiten

»Querido lector: imaginate una bolsa donde todo es unico

»Cuando yo empecé a programar, usaba listas para todo. Pero un día necesite guardar una lista de nombres, sin repetir ninguno. Podia hacerlo con una lista y cada vez que iba a agregar, revisar si ya existia. Un quilombo.

»Ahi descubri los **sets** (conjuntos en español). Un set es como una bolsa donde no pueden haber elementos repetidos. Además, no tienen orden. Si metes varias cosas, Python se encarga de que cada una aparezca una sola vez.

»En la vida real pensalo como un conjunto de personas en una fiesta: no importa el orden, y no puede haber dos veces la misma persona.

»Que es un set y como se crea

»Un set es una coleccion **desordenada** de elementos **unicos** (sin repeticiones). Se crea con llaves “ o con la funcion ‘set()’.

```
»# Set con elementos
»frutas = {"manzana", "pera", "uva", "manzana"} # la segunda "manzana" se ignora
»print(frutas) # {'pera', 'uva', 'manzana'} (el orden puede variar)
»
»# Set vacio (ojo: {} es un diccionario vacio)
»set_vacio = set()
```

»No se puede usar “ vacio para set

»Un error comun: usar “ para un set vacio. Eso crea un diccionario vacio, no un set.

```
»>>> equivocado = {}
»>>> type(equivocado)
»<class 'dict'>
»>>> correcto = set()
»>>> type(correcto)
»<class 'set'>
```

» Convertir una lista a set (para eliminar duplicados)

» Una de las aplicaciones mas utiles: pasar una lista con repeticiones a un set, y se quedan solo los elementos unicos.

```
>>> lista_con_repetidos = [1, 2, 2, 3, 3, 3, 4]
>>> conjunto = set(lista_con_repetidos)
>>> print(conjunto)
{1, 2, 3, 4}
```

» Características principales

- » **No tiene orden:** no puedes acceder por indice ('conjunto[0]' da error).
- » **No permite duplicados:** si agregas un elemento que ya existe, no pasa nada.
- » **Elementos inmutables:** puedes guardar numeros, strings, tuplas, pero no listas ni diccionarios.
- » **Es mutable:** puedes agregar y eliminar elementos.

» Operaciones basicas con sets

» Agregar elementos con 'add()'

```
>>> colores = {"rojo", "verde"}
>>> colores.add("azul")
>>> print(colores)
{'rojo', 'verde', 'azul'}
>>> colores.add("rojo") # ya existe, no pasa nada
>>> print(colores)
{'rojo', 'verde', 'azul'}
```

» Eliminar elementos con 'remove()' y 'discard()'

» 'remove()' da error si el elemento no existe. 'discard()' no da error.

```
>>> colores.remove("verde")
>>> print(colores)
{'rojo', 'azul'}
>>> colores.discard("amarillo") # no existe, pero no da error
>>> colores.remove("amarillo") # esto si da KeyError
```

» Eliminar un elemento al azar con 'pop()'

» Como los sets no tienen orden, 'pop()' elimina y devuelve un elemento arbitrario.

```
>>> conjunto = {1, 2, 3}
>>> elemento = conjunto.pop()
>>> print(elemento) # puede ser 1, 2 o 3
>>> print(conjunto)
```

» Eliminar todos los elementos con 'clear()'

```
»>>> colores.clear()
»>>> print(colores)
»set()
```

» Verificar si un elemento existe con 'in'

```
»>>> frutas = {"manzana", "pera"}
»>>> print("manzana" in frutas)
»True
»>>> print("banana" in frutas)
»False
```

» Recorrer un set con 'for'

```
»for fruta in frutas:
»    print(fruta)
```

» Operaciones de conjuntos (matematicas)

»Esta es la parte mas poderosa. Los sets soportan operaciones como union, interseccion, diferencia, etc.

» Union ('|' o 'union()')

»Todos los elementos de ambos conjuntos, sin repetir.

```
»>>> A = {1, 2, 3}
»>>> B = {3, 4, 5}
»>>> union = A | B
»>>> print(union)
»{1, 2, 3, 4, 5}
»>>> # o tambien: A.union(B)
```

» Interseccion ('&' o 'intersection()')

»Elementos que estan en ambos conjuntos.

```
»>>> interseccion = A & B
»>>> print(interseccion)
»{3}
»>>> # o tambien: A.intersection(B)
```

» Diferencia ('-' o 'difference()')

»Elementos que estan en A pero no en B.

```
»»» diferencia = A - B
»»» print(diferencia)
»{1, 2}
»»» # o tambien: A.difference(B)
```

» Diferencia simetrica ('o' *symmetric_difference()*)

» Elementos que estan en A o en B, pero no en ambos.

```
»»» dif_simetrica = A ^ B
»»» print(dif_simetrica)
»{1, 2, 4, 5}
```

» Subconjunto y superconjunto

```
»»» C = {1, 2}
»»» print(C.issubset(A)) # True, C esta dentro de A
»»» print(A.issuperset(C)) # True, A contiene a C
```

» Ejemplos practicos

» Eliminar duplicados de una lista

```
»lista = [1, 2, 2, 3, 4, 4, 4, 5]
»unicos = list(set(lista))
»print(unicos) # [1, 2, 3, 4, 5] (el orden puede variar)
```

» Encontrar elementos comunes entre dos listas

```
»lista1 = [1, 2, 3, 4, 5]
»lista2 = [4, 5, 6, 7, 8]
»comunes = set(lista1) & set(lista2)
»print(comunes) # {4, 5}
```

» Elementos que estan en una lista pero no en otra

```
»solo_en_lista1 = set(lista1) - set(lista2)
»print(solo_en_lista1) # {1, 2, 3}
```

» Contar palabras unicas en una frase

```
»frase = "hola mundo hola python mundo"
»palabras = frase.split()
»unicas = set(palabras)
»print(f"Total de palabras: {len(palabras)}")
»print(f"Palabras unicas: {len(unicas)}")
»print(f"Unicas: {unicas}")
```

»Frozenset: la version inmutable de set

»Si necesitas un conjunto que no se pueda modificar (para usarlo como clave de diccionario, por ejemplo), existe 'frozenset()'.

```
>>> conjunto = frozenset([1, 2, 3])
>>> conjunto.add(4) # Esto da error
AttributeError: 'frozenset' object has no attribute 'add'
```

»20 ejercicios con soluciones paso a paso

»Ejercicio 1

»Crear un set con los numeros 1, 2, 3, 4, 5 y mostrarlo.

»Solucion:

```
>numeros = {1, 2, 3, 4, 5}
>print(numeros)
```

»Explicacion: Los sets se crean con llaves. Al imprimirlo, el orden puede variar.

»Ejercicio 2

»Crear un set vacio correctamente.

»Solucion:

```
>conjunto_vacio = set()
>print(type(conjunto_vacio))
```

»Explicacion: " crea un diccionario vacio. Para set vacio se usa 'set()'.

»Ejercicio 3

»Agregar el elemento "pera" al set 'frutas = "manzana", "banana"
»'.

»Solucion:

```
>frutas = {"manzana", "banana"}
>frutas.add("pera")
>print(frutas)
```

»Explicacion: 'add()' agrega un elemento al set.

»Ejercicio 4

»Eliminar el elemento "banana" del set anterior usando 'remove()'.

»Solucion:

```
>frutas.remove("banana")
>print(frutas)
```

»Explicacion: 'remove()' elimina el elemento si existe; si no, da error.

»Ejercicio 5

»Usar `'discard()'` para eliminar un elemento que no existe (sin que de error).

»Solucion:

```
»frutas.discard("sandia") # no existe, pero no da error
»print(frutas)
```

»Explicacion: `'discard()'` es mas seguro que `'remove()'` cuando no estas seguro de la existencia.

»Ejercicio 6

»Verificar si "manzana" esta en el set.

»Solucion:

```
»if "manzana" in frutas:
»    print("Si esta")
»else:
»    print("No esta")
```

»Explicacion: El operador `'in'` funciona muy rapido en sets.

»Ejercicio 7

»Recorrer el set `'10, 20, 30, 40, 50'` con un bucle `for` e imprimir cada elemento.

»Solucion:

```
»numeros = {10, 20, 30, 40, 50}
»for n in numeros:
»    print(n)
```

»Explicacion: Se itera sobre cada elemento del set (el orden no es fijo).

»Ejercicio 8

»Convertir una lista con elementos repetidos `'[1, 1, 2, 2, 3, 4]'` en un set para eliminar duplicados.

»Solucion:

```
»lista = [1, 1, 2, 2, 3, 4]
»conjunto = set(lista)
»print(conjunto) # {1, 2, 3, 4}
```

»Explicacion: El constructor `'set()'` elimina automaticamente los duplicados.

»Ejercicio 9

»Dados dos sets `'A = 1, 2, 3'` y `'B = 3, 4, 5'`, calcular la union.

»Solucion:

```
»A = {1, 2, 3}
»B = {3, 4, 5}
»union = A | B
»print(union) # {1, 2, 3, 4, 5}
```

»Explicacion: El operador `'|'` une los conjuntos sin repetir elementos.

»Ejercicio 10

»Calcular la interseccion de A y B.

»Solucion:

```
»interseccion = A & B
»print(interseccion) # {3}
```

»Explicacion: El operador “&” devuelve los elementos comunes.

»Ejercicio 11

»Calcular la diferencia A - B.

»Solucion:

```
»diferencia = A - B
»print(diferencia) # {1, 2}
```

»Explicacion: ‘A - B’ devuelve los elementos que estan en A pero no en B.

»Ejercicio 12

»Calcular la diferencia simetrica $A \oplus B$.

»Solucion:

```
»dif_simetrica = A ^ B
»print(dif_simetrica) # {1, 2, 4, 5}
```

»Explicacion: “^” devuelve los elementos que estan en uno u otro, pero no en ambos.

»Ejercicio 13

»Verificar si ‘{1, 2}’ es subconjunto de ‘{1, 2, 3}’.

»Solucion:

```
»C = {1, 2}
»D = {1, 2, 3}
»print(C.issubset(D)) # True
```

»Explicacion: ‘issubset()’ devuelve True si todos los elementos de C estan en D.

»Ejercicio 14

»Eliminar todos los elementos de un set con ‘clear()’.

»Solucion:

```
»conjunto = {1, 2, 3}
»conjunto.clear()
»print(conjunto) # set()
```

»Explicacion: ‘clear()’ vacia el set.

»Ejercicio 15

»Crear un frozenset a partir de una lista `[1, 2, 3]`.

»Solucion:

```
»lista = [1, 2, 3]
»fs = frozenset(lista)
»print(fs)
```

»Explicacion: `'frozenset()'` crea una version inmutable de un set.

»Ejercicio 16

»Contar cuantas palabras distintas hay en la frase `.'el gato y el perro son amigos del gato'.`

»Solucion:

```
»frase = "el gato y el perro son amigos del gato"
»palabras = frase.split()
»unicas = set(palabras)
»print(f"Palabras distintas: {len(unicas)}")
»print(unicas)
```

»Explicacion: Dividimos la frase en palabras con `'split()'`, convertimos a set para eliminar repeticiones, y contamos.

»Ejercicio 17

»Encontrar los elementos que estan en `'1, 2, 3, 4'` pero no en `'3, 4, 5, 6'`.

»Solucion:

```
»A = {1, 2, 3, 4}
»B = {3, 4, 5, 6}
»print(A - B) # {1, 2}
```

»Explicacion: La diferencia da los elementos que estan solo en A.

»Ejercicio 18

»Dadas dos listas, mostrar los elementos que aparecen en ambas (sin repetir).

»Solucion:

```
»lista1 = [1, 2, 3, 4]
»lista2 = [3, 4, 5, 6]
»comunes = set(lista1) & set(lista2)
»print(comunes) # {3, 4}
```

»Explicacion: Convertimos a sets y aplicamos interseccion.

»Ejercicio 19

»Usar 'pop()' para eliminar un elemento arbitrario de un set y mostrarlo.

»Solucion:

```
»conjunto = {10, 20, 30}
»eliminado = conjunto.pop()
»print(f"Elemento eliminado: {eliminado}")
»print(f"Set restante: {conjunto}")
```

»Explicacion: 'pop()' elimina y devuelve un elemento aleatorio (porque el set no tiene orden).

»Ejercicio 20

»Verificar si dos sets son disjuntos (no tienen elementos en comun).

»Solucion:

```
»A = {1, 2, 3}
»B = {4, 5, 6}
»C = {3, 7, 8}
»print(A.isdisjoint(B)) # True
»print(A.isdisjoint(C)) # False
```

»Explicacion: 'isdisjoint()' devuelve True si la interseccion es vacia.

»Resumen de lo que vimos

»Dale, repasemos:

- »■ Un **set** es una coleccion desordenada de elementos unicos.
- »■ Se crea con "" o con 'set()' (vacio). Cuidado: "" es diccionario.
- »■ No se puede acceder por indice (no tiene orden).
- »■ Metodos: 'add()', 'remove()', 'discard()', 'pop()', 'clear()'.
- »■ Operaciones de conjuntos: union ('|'), interseccion ("&"), diferencia ('-'), diferencia simetrica ('^').
- »■ Muy util para eliminar duplicados y para hacer operaciones entre colecciones.
- »■ 'frozenset' es la version inmutable (para usar como clave de diccionario).

»Reflexion final

»Los sets son como una caja magica donde cada cosa solo puede estar una vez. Si necesitas saber que elementos son comunes entre dos listas, o cuales estan en una pero no en otra, los sets te van a salvar horas de codigo. No te olvides de ellos: son una herramienta fundamental para cualquier programador.

»_____ En un conjunto, lo importante no es
el orden, sino la pertenencia.

» Comprension de listas, diccionarios y sets:Codigo mas elegante y rapido

» Querido lector: cuando el bucle for se vuelve una linea

» Cuando yo empecé a programar, para crear una lista con los cuadrados de los números del 1 al 10, escribía algo como:

```
»cuadrados = []
»for i in range(1, 11):
»    cuadrados.append(i ** 2)
```

» Funciona, pero es un montón de líneas para algo tan simple. Un día un compañero de la facultad me mostró algo que me voló la cabeza: la **comprension de listas**.

» Con comprension de listas, lo mismo se escribe en una sola línea:

```
»cuadrados = [i ** 2 for i in range(1, 11)]
```

» Es más corto, más rápido, y una vez que le agarras la mano, más fácil de leer.

» En este capítulo vamos a aprender a usar comprension de listas, de diccionarios y de sets. Te prometo que después de esto, vas a querer usarlas todo el tiempo.

» Comprension de listas (list comprehensions)

» Sintaxis básica

» La fórmula mágica es:

```
»[nueva_lista] = [expresion for elemento in iterable]
```

» Dale, desglosemos:

» ■ ‘expresion’ es lo que se va a guardar en la nueva lista.

» ■ ‘elemento’ es la variable temporal que toma cada valor del iterable.

» ■ ‘iterable’ puede ser un rango, una lista, un string, etc.

»Ejemplo basico: cuadrados

```

>>>> # Sin comprension
>>>> cuadrados = []
>>>> for i in range(1, 6):
>...     cuadrados.append(i ** 2)
>...
>>>> print(cuadrados)
>[1, 4, 9, 16, 25]
>
>>>> # Con comprension
>>>> cuadrados = [i ** 2 for i in range(1, 6)]
>>>> print(cuadrados)
>[1, 4, 9, 16, 25]

```

»Ejemplo: convertir strings a mayusculas

```

>>>> nombres = ["ana", "luis", "pedro"]
>>>> mayusculas = [nombre.upper() for nombre in nombres]
>>>> print(mayusculas)
>['ANA', 'LUIS', 'PEDRO']

```

»Con condicion (if)

»Podes agregar un 'if' al final para filtrar.

```

>>>> # Solo numeros pares
>>>> pares = [i for i in range(1, 11) if i % 2 == 0]
>>>> print(pares)
>[2, 4, 6, 8, 10]

```

»Con if-else (operador ternario)

»Si necesitas un 'if-else' dentro de la expresion, se escribe de otra forma: primero el 'if-else' y despues el 'for'.

```

>>>> # Marcar como 'par' o 'impar'
>>>> etiquetas = ["par" if i % 2 == 0 else "impar" for i in range(1, 6)]
>>>> print(etiquetas)
>['impar', 'par', 'impar', 'par', 'impar']

```

»Comprension anidada (for dentro de for)

»A veces necesitas combinar dos listas. Es como el producto cartesiano.

```

>>>> colores = ["rojo", "verde"]
>>>> tamanios = ["chico", "grande"]
>>>> combinaciones = [(color, tam) for color in colores for tam in tamanios]
>>>> print(combinaciones)
>[('rojo', 'chico'), ('rojo', 'grande'), ('verde', 'chico'), ('verde', 'grande')]

```

»El orden de los 'for' es el mismo que si anidaras los bucles:

```
>combinaciones = []
>for color in colores:
>    for tam in tamanios:
>        combinaciones.append((color, tam))
```

»Compresion de sets (set comprehensions)

»Es casi lo mismo que las listas, pero en lugar de corchetes usas llaves. La diferencia es que los sets no tienen orden y no guardan duplicados.

```
>>> cuadrados_set = {i ** 2 for i in range(1, 6)}
>>> print(cuadrados_set)
>{16, 1, 4, 9, 25}
```

»Ejemplo: vocales unicas en una frase

```
>>> frase = "hola mundo como estas"
>>> vocales = {letra for letra in frase if letra in "aeiou"}
>>> print(vocales)
>{'a', 'e', 'o', 'u'}
```

»Compresion de diccionarios (dict comprehensions)

»Tambien se puede crear diccionarios con una sintaxis similar, usando 'clave: valor'.

```
>>> # Diccionario con numeros y sus cuadrados
>>> cuadrados_dict = {i: i ** 2 for i in range(1, 6)}
>>> print(cuadrados_dict)
>{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

»Ejemplo: invertir claves y valores

»Si tienes un diccionario y quieres intercambiar las claves con los valores (siempre que los valores sean unicos, porque las claves no se repiten).

```
>>> original = {"a": 1, "b": 2, "c": 3}
>>> invertido = {valor: clave for clave, valor in original.items()}
>>> print(invertido)
>{1: 'a', 2: 'b', 3: 'c'}
```

»Ejemplo: filtrar elementos de un diccionario

```
>>> notas = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 6}
>>> aprobados = {nombre: nota for nombre, nota in notas.items() if nota >= 7}
>>> print(aprobados)
>{'Ana': 8, 'Pedro': 9}
```

» Usos practicos y ejemplos reales

» Aplanar una matriz (lista de listas)

```

>>> matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> aplanada = [num for fila in matriz for num in fila]
>>> print(aplanada)
>>> [1, 2, 3, 4, 5, 6, 7, 8, 9]

```

» Filtrar y transformar al mismo tiempo

```

>>> numeros = [1, 2, 3, 4, 5, 6]
>>> # Doble de los numeros pares
>>> resultado = [n * 2 for n in numeros if n % 2 == 0]
>>> print(resultado)
>>> [4, 8, 12]

```

» Procesar lineas de un archivo (simulado)

```

>>> lineas = ["1,Ana,8", "2,Luis,7", "3,Pedro,9"]
>>> datos = [linea.split(",") for linea in lineas]
>>> print(datos)
>>> [['1', 'Ana', '8'], ['2', 'Luis', '7'], ['3', 'Pedro', '9']]

```

» Extraer una columna de una matriz

```

>>> matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> columna_1 = [fila[1] for fila in matriz]
>>> print(columna_1)
>>> [2, 5, 8]

```

» 20 ejercicios para practicar (con soluciones explicadas paso a paso)

» Ahora te toca a vos. Intentá resolver cada ejercicio antes de mirar la solución.

» Ejercicio 1

» Crear una lista con los numeros del 1 al 10 usando comprension.

» Solucion:

```

>>> numeros = [i for i in range(1, 11)]
>>> print(numeros) # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```

» Explicacion: 'range(1,11)' genera los numeros del 1 al 10. La comprension itera sobre cada 'i' y lo guarda tal cual.

»Ejercicio 2

»Crear una lista con los cuadrados de los numeros del 1 al 10.

»Solucion:

```
»cuadrados = [i ** 2 for i in range(1, 11)]
»print(cuadrados) # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

»Explicacion: La expresion 'i ** 2' calcula el cuadrado.

»Ejercicio 3

»Crear una lista solo con los numeros pares del 1 al 20.

»Solucion:

```
»pares = [i for i in range(1, 21) if i % 2 == 0]
»print(pares) # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

»Explicacion: El 'if i

»Ejercicio 4

»De una lista de palabras, crear una nueva lista con las palabras en mayusculas.

»Solucion:

```
»palabras = ["hola", "mundo", "python"]
»mayus = [p.upper() for p in palabras]
»print(mayus) # ['HOLA', 'MUNDO', 'PYTHON']
```

»Explicacion: Aplicamos el metodo 'upper()' a cada palabra.

»Ejercicio 5

»De una lista de numeros, crear una lista con el string "par.º impar"segun corresponda.

»Solucion:

```
»numeros = [1, 2, 3, 4, 5]
»etiquetas = ["par" if n % 2 == 0 else "impar" for n in numeros]
»print(etiquetas) # ['impar', 'par', 'impar', 'par', 'impar']
```

»Explicacion: Usamos el operador ternario dentro de la expresion.

»Ejercicio 6

»Crear un set con los cuadrados de los numeros del 1 al 5 (compresion de sets).

»Solucion:

```
»cuadrados_set = {i ** 2 for i in range(1, 6)}
»print(cuadrados_set) # {1, 4, 9, 16, 25}
```

»Explicacion: Llaves en lugar de corchetes. No importa el orden.

»Ejercicio 7

»Crear un diccionario que mapee numeros del 1 al 5 a su cubo.

»Solucion:

```
»cubos = {i: i ** 3 for i in range(1, 6)}
»print(cubos) # {1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
```

»Explicacion: La clave es 'i', el valor es 'i ** 3'.

»Ejercicio 8

»Dado un diccionario 'a': 1, 'b': 2, 'c': 3, crear otro intercambiando claves y valores.

»Solucion:

```
»original = {"a": 1, "b": 2, "c": 3}
»invertido = {v: k for k, v in original.items()}
»print(invertido) # {1: 'a', 2: 'b', 3: 'c'}
```

»Explicacion: 'original.items()' devuelve pares '(clave, valor)'. Los intercambiamos.

»Ejercicio 9

»De una lista de numeros, crear una lista con los numeros mayores a 5.

»Solucion:

```
»numeros = [2, 5, 7, 1, 8, 3, 10]
»mayores = [n for n in numeros if n > 5]
»print(mayores) # [7, 8, 10]
```

»Explicacion: Filtramos con 'if n > 5'.

»Ejercicio 10

»Dada una lista de strings, crear una lista con las longitudes de cada string.

»Solucion:

```
»palabras = ["python", "es", "genial"]
»longitudes = [len(p) for p in palabras]
»print(longitudes) # [6, 2, 6]
```

»Explicacion: 'len(p)' da la cantidad de caracteres.

»Ejercicio 11

»Aplanar la lista '[[1,2],[3,4],[5,6]]' en una sola lista.

»Solucion:

```
»matriz = [[1, 2], [3, 4], [5, 6]]
»aplanada = [num for fila in matriz for num in fila]
»print(aplanada) # [1, 2, 3, 4, 5, 6]
```

»Explicacion: Doble 'for': primero recorre cada fila, luego cada numero de la fila.

»Ejercicio 12

»De una frase, crear una lista con las palabras que tengan mas de 3 letras.

»Solucion:

```
»frase = "hola mundo python es genial"
»palabras = frase.split()
»largas = [p for p in palabras if len(p) > 3]
»print(largas) # ['hola', 'mundo', 'python', 'genial']
```

»Explicacion: 'split()' separa por espacios. Filtramos por longitud.

»Ejercicio 13

»Crear una lista con los primeros caracteres de cada palabra de una lista.

»Solucion:

```
»palabras = ["manzana", "pera", "uva"]
»primeras = [p[0] for p in palabras]
»print(primeras) # ['m', 'p', 'u']
```

»Explicacion: 'p[0]' es el primer caracter.

»Ejercicio 14

»De un diccionario de notas, crear un nuevo diccionario solo con los aprobados (nota >= 7).

»Solucion:

```
»notas = {"Ana": 8, "Luis": 5, "Pedro": 9, "Marta": 6}
»aprobados = {nombre: nota for nombre, nota in notas.items() if nota >= 7}
»print(aprobados) # {'Ana': 8, 'Pedro': 9}
```

»Explicacion: Filtramos los items con 'if nota >= 7'.

»Ejercicio 15

»Crear una lista con los numeros del 1 al 10, pero sustituyendo los multiplos de 3 por "Fizz".

»Solucion:

```
»resultado = ["Fizz" if i % 3 == 0 else i for i in range(1, 11)]
»print(resultado) # [1, 2, 'Fizz', 4, 5, 'Fizz', 7, 8, 'Fizz', 10]
```

»Explicacion: Operador ternario: si es multiplo de 3 va "Fizz", sino el numero.

»Ejercicio 16

»Dada una lista de numeros, crear una lista con el doble de cada numero, pero solo si el numero es positivo.

»Solucion:

```
»numeros = [1, -2, 3, -4, 5]
»dobles_positivos = [n * 2 for n in numeros if n > 0]
»print(dobles_positivos) # [2, 6, 10]
```

»Explicacion: Filtramos 'n > 0' y despues multiplicamos.

»Ejercicio 17

»Crear un set con las vocales que aparecen en una frase.

»Solucion:

```
»frase = "aprendiendo python"
»vocales = {letra for letra in frase if letra in "aeiou"}
»print(vocales) # {'a', 'e', 'i', 'o'}
```

»Explicacion: Set comprehension. Si una vocal se repite, en el set aparece una sola vez.

»Ejercicio 18

»Dada una lista de tuplas '(nombre, edad)', crear un diccionario nombre: edad.

»Solucion:

```
»personas = [("Ana", 22), ("Luis", 25), ("Pedro", 21)]
»diccionario = {nombre: edad for nombre, edad in personas}
»print(diccionario) # {'Ana': 22, 'Luis': 25, 'Pedro': 21}
```

»Explicacion: Desempaquetamos cada tupla en 'nombre' y 'edad'.

»Ejercicio 19

»Crear una lista con la raiz cuadrada de los numeros del 1 al 10 (usando math.sqrt).

»Solucion:

```
»import math
»raices = [math.sqrt(i) for i in range(1, 11)]
»print(raices) # [1.0, 1.414..., 1.732..., ...]
```

»Explicacion: 'math.sqrt(i)' calcula la raiz cuadrada.

»Ejercicio 20

»Dada una lista de numeros, crear una lista que contenga el numero y su cuadrado en una tupla.

»Solucion:

```
»numeros = [1, 2, 3, 4, 5]
»pares_cuadrado = [(n, n ** 2) for n in numeros]
»print(pares_cuadrado) # [(1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

»Explicacion: Cada elemento es una tupla '(n, n**2)'.

»Tabla de referencia rapida

Tipo	Sintaxis
» Lista	'[exp for elemento in iterable if cond]'
Set	'exp for elemento in iterable if cond'
Diccionario	'clave: valor for elemento in iterable if cond'

» Resumen de lo que vimos

» Dale, repasemos:

- » ■ La **comprension de listas** permite crear listas en una sola linea: `[expr for elemento in iterable]`.
- » ■ Se puede agregar un `'if'` para filtrar: `[expr for elemento in iterable if condicion]`.
- » ■ Se puede usar un `'if-else'` dentro de la expresion: `[valor_si_true if condicion else valor_si_false for elemento in iterable]`.
- » ■ La **comprension de sets** es similar con llaves: `{expr for elemento in iterable}`. Elimina duplicados.
- » ■ La **comprension de diccionarios** usa `'clave: valor'`: `{clave: expr : valor_expr for elemento in iterable}`.
- » ■ Son mas rapidas y legibles que los bucles tradicionales, pero no abuses: si la comprension es muy larga (mas de 2 o 3 `'for'`), usa un bucle normal para que otros entiendan tu codigo.

» Reflexion final

» Mirãa, la comprension de listas es como pasar de andar en bicicleta a manejar un auto. No vas a querer volver atras. Pero ojo: con poder viene responsabilidad. No hagas comprensiones de tres lineas que nadie entienda. A veces, un bucle simple es mas claro.

» Recordá que el codigo se lee mas veces de las que se escribe. Prioriza la claridad por sobre la coolness".

» Practica estos ejercicios, modificalos, rompelos. Proba hacer comprensiones con strings, con rangos, con archivos. Vas a ver que rapido se vuelve natural.

» _____ *Escribe codigo que los humanos puedan entender. La maquina lo entendera igual.*

»Modulo datetime: Fechas y horas

»Querido lector: el tiempo no es tan facil de manejar

»Cuando yo empece a programar, pense que manejar fechas era tan simple como usar strings. "2024-05-20", listo. Pero despues me choque con la realidad: restar dos fechas, sumar dias, saber que dia de la semana fue hace un mes... con strings es un infierno. Me acuerdo una noche que queria calcular cuantos dias faltaban para Navidad y termine con una funcion gigante llena de bugs.

»Por suerte, Python tiene el modulo `datetime`. Es una de esas joyitas que vienen incluidas y te resuelven la vida. En este capitulo te voy a mostrar como usarlo con ejemplos que podes probar vos mismo.

»Que es datetime y como importarlo

»El modulo `datetime` viene con Python (no hace falta instalar nada). Lo importas asi:

```
»import datetime
```

»Pero la forma mas comun es importar solo las clases que necesitas:

```
»from datetime import datetime, date, time, timedelta
```

»La clase `date` (solo fecha, sin hora)

»Crear una fecha

```
• >>>> from datetime import date
>>>> # Fecha actual
>>>> hoy = date.today()
>>>> print(hoy)
>2024-05-20
>
>>>> # Fecha especifica (anio, mes, dia)
>>>> mi_cumple = date(1990, 5, 15)
>>>> print(mi_cumple)
>1990-05-15
```

» Acceder a sus componentes

```
»»» print(mi_cumple.year)
»1990
»»» print(mi_cumple.month)
»5
»»» print(mi_cumple.day)
»15
```

» Dia de la semana

»weekday() devuelve 0 para lunes, 1 para martes... 6 para domingo. isoweekday() devuelve 1 para lunes, 7 para domingo.

```
»»» print(mi_cumple.weekday())    # 1 (martes si 1990-05-15 fue martes)
»1
»»» print(mi_cumple.isoweekday())
»2
```

» Formato a string

```
»»» print(mi_cumple.strftime("%d/%m/%Y"))
»15/05/1990
»»» print(mi_cumple.strftime("%A, %d de %B de %Y"))
»Tuesday, 15 de May de 1990
```

» La clase time (solo hora, sin fecha)

```
»»» from datetime import time
»»» hora = time(14, 30, 15) # 14:30:15
»»» print(hora)
»14:30:15
»»» print(hora.hour)
»14
»»» print(hora.minute)
»30
»»» print(hora.second)
»15
```

» La clase datetime (fecha + hora)

»Esta es la mas completa.

» Fecha y hora actual

```
»»» from datetime import datetime
»»» ahora = datetime.now()
»»» print(ahora)
»2024-05-20 15:30:45.123456
```

» Crear una fecha y hora específica

```
>>> fecha_especifica = datetime(2024, 12, 25, 20, 30, 0)
>>> print(fecha_especifica)
»2024-12-25 20:30:00
```

» Obtener solo fecha o solo hora

```
>>> solo_fecha = ahora.date()
>>> solo_hora = ahora.time()
```

» Formato personalizado

```
>>> print(ahora.strftime("%Y-%m-%d %H:%M:%S"))
»2024-05-20 15:30:45
>>> print(ahora.strftime("%d/%m/%Y"))
»20/05/2024
>>> print(ahora.strftime("%A %d de %B"))
»Monday 20 de May
```

» La clase timedelta (diferencias de tiempo)

»timedelta representa una diferencia de tiempo. Sirve para sumar o restar días, horas, minutos, etc.

```
>>> from datetime import timedelta
>>> hoy = date.today()
>>> manana = hoy + timedelta(days=1)
>>> ayer = hoy - timedelta(days=1)
>>> print(f"Hoy: {hoy}")
»Hoy: 2024-05-20
>>> print(f"Manana: {manana}")
»Manana: 2024-05-21
>>> print(f"Ayer: {ayer}")
»Ayer: 2024-05-19
```

» Calcular edad (en días)

```
>>> nacimiento = date(1990, 5, 15)
>>> hoy = date.today()
>>> dias_vividos = hoy - nacimiento
>>> print(f"Dias vividos: {dias_vividos.days}")
»Dias vividos: 12400 (aproximadamente)
```

» Diferencia entre dos datetimes

```
>>> inicio = datetime(2024, 1, 1, 0, 0, 0)
>>> fin = datetime(2024, 1, 10, 12, 30, 0)
```



```

>>> diferencia = fin - inicio
>>> print(diferencia)
>9 days, 12:30:00
>>> print(diferencia.days)
>9
>>> print(diferencia.seconds)
>45000 # 12*3600 + 30*60
>>> print(diferencia.total_seconds())
>815400.0 # 9*86400 + 45000

```

»Parsear strings a fechas

»A veces tienes una fecha en formato string y la quieres convertir a un objeto `date` o `datetime`. Para eso usas `strptime` (string parse time).

```

>>> fecha_str = "15/05/1990"
>>> fecha = datetime.strptime(fecha_str, "%d/%m/%Y")
>>> print(fecha)
>1990-05-15 00:00:00

```

»Tabla de codigos de formato comunes

Codigo	Significado
%Y	Año con 4 dígitos (2024)
%y	Año con 2 dígitos (24)
%m	Mes con 2 dígitos (01-12)
%B	Nombre del mes completo (January)
%b	Nombre del mes abreviado (Jan)
%d	Día del mes con 2 dígitos (01-31)
%A	Día de la semana completo (Monday)
%a	Día de la semana abreviado (Mon)
%H	Hora en formato 24h (00-23)
%I	Hora en formato 12h (01-12)
%M	Minutos (00-59)
%S	Segundos (00-59)
%p	AM o PM

»Ejemplos practicos

»Calcular días hasta tu cumpleaños

```

>from datetime import date
>
>hoy = date.today()
>proximo_cumple = date(hoy.year, 5, 15)
>if proximo_cumple < hoy:
>    proximo_cumple = date(hoy.year + 1, 5, 15)

```

```
»
»dias_faltantes = (proximo_cumple - hoy).days
»print(f"Faltan {dias_faltantes} dias para tu cumpleaños")
```

»Saber si un año es bisiesto

```
»def es_bisiesto(año):
»    return (año % 4 == 0 and año % 100 != 0) or (año % 400 == 0)
```

»Listar los próximos 7 días

```
»hoy = date.today()
»for i in range(7):
»    día = hoy + timedelta(days=i)
»    print(día.strftime("%A %d/%m/%Y"))
```

»20 ejercicios con soluciones paso a paso

»Ahora te toca a vos. Intentá resolverlos antes de mirar la solución.

»Ejercicio 1

»Mostrar la fecha actual.

»Solución:

```
»from datetime import date
»print(date.today())
```

»Explicación: `date.today()` devuelve la fecha actual del sistema.

»Ejercicio 2

»Mostrar la fecha y hora actual.

»Solución:

```
»from datetime import datetime
»print(datetime.now())
```

»Explicación: `datetime.now()` incluye fecha, hora, minutos, segundos y microsegundos.

»Ejercicio 3

»Crear una fecha para el 25 de diciembre de 2025.

»Solución:

```
»from datetime import date
»navidad = date(2025, 12, 25)
»print(navidad)
```

»Explicación: `date(año, mes, día)` crea el objeto.

»Ejercicio 4

»Obtener el año, mes y día de la fecha actual por separado.

»Solucion:

```
»hoy = date.today()
»print(f"Año: {hoy.year}")
»print(f"Mes: {hoy.month}")
»print(f"Día: {hoy.day}")
```

»Explicacion: Los atributos `year`, `month`, `day` son enteros.

»Ejercicio 5

»Mostrar la fecha de mañana.

»Solucion:

```
»from datetime import date, timedelta
»manana = date.today() + timedelta(days=1)
»print(manana)
```

»Explicacion: `timedelta(days=1)` representa un día.

»Ejercicio 6

»Mostrar la fecha de ayer.

»Solucion:

```
»ayer = date.today() - timedelta(days=1)
»print(ayer)
```

»Ejercicio 7

»Calcular cuantos días faltan para fin de año (31 de diciembre).

»Solucion:

```
»hoy = date.today()
»fin_año = date(hoy.year, 12, 31)
»dias = (fin_año - hoy).days
»print(f"Faltan {dias} días para fin de año")
```

»Explicacion: La resta de dos `date` devuelve un `timedelta`. El atributo `days` da el número de días.

»Ejercicio 8

»Mostrar la hora actual (solo hora, minutos y segundos).

»Solucion:

```
»from datetime import datetime
»ahora = datetime.now()
»print(f"{ahora.hour}:{ahora.minute}:{ahora.second}")
```

»Ejercicio 9

»Crear una hora especifica (18:30:00).

»Solucion:

```
»from datetime import time
»hora = time(18, 30, 0)
»print(hora)
```

»Ejercicio 10

»Sumar 10 dias a la fecha actual.

»Solucion:

```
»from datetime import date, timedelta
»fecha = date.today() + timedelta(days=10)
»print(fecha)
```

»Ejercicio 11

»Calcular la edad en años (suponiendo nacimiento el 15/05/1990).

»Solucion:

```
»from datetime import date
»nacimiento = date(1990, 5, 15)
»hoy = date.today()
»edad = hoy.year - nacimiento.year
»if hoy.month < nacimiento.month or (hoy.month == nacimiento.month and hoy.day < nacimiento.day):
»    edad -= 1
»print(f"Edad: {edad} años")
```

»**Explicacion:** Primero restamos los años, luego ajustamos si todavía no cumplió años este año.

»Ejercicio 12

»Convertir el string "20/05/2024." a un objeto date.

»Solucion:

```
»from datetime import datetime
»fecha_str = "20/05/2024"
»fecha = datetime.strptime(fecha_str, "%d/%m/%Y")
»print(fecha)
```

»**Explicacion:** strptime parsea según el formato dado. El resultado es datetime.

»Ejercicio 13

»Mostrar la fecha actual en formato "DD/MM/YYYY".

»Solucion:

```
»from datetime import date
»print(date.today().strftime("%d/%m/%Y"))
```

»Ejercicio 14

»Mostrar el día de la semana de una fecha (ej: "lunes").

»Solucion:

```
»hoy = date.today()
»print(hoy.strftime("%A"))
```

»Ejercicio 15

»Crear un datetime con fecha 2024-12-25 20:30:00.

»Solucion:

```
»from datetime import datetime
»fecha_hora = datetime(2024, 12, 25, 20, 30, 0)
»print(fecha_hora)
```

»Ejercicio 16

»Calcular la diferencia en días entre dos fechas: 2024-01-01 y 2024-12-31.

»Solucion:

```
»from datetime import date
»inicio = date(2024, 1, 1)
»fin = date(2024, 12, 31)
»diferencia = fin - inicio
»print(diferencia.days) # 365
```

»Ejercicio 17

»Sumar 2 horas y 30 minutos a la hora actual.

»Solucion:

```
»from datetime import datetime, timedelta
»ahora = datetime.now()
»mas_tarde = ahora + timedelta(hours=2, minutes=30)
»print(mas_tarde)
```

»Ejercicio 18

»Verificar si un año es bisiesto (funcion).

»Solucion:

```
»def es_bisiesto(anio):
»    return (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0)
»
»print(es_bisiesto(2024)) # True
»print(es_bisiesto(2023)) # False
```

»Ejercicio 19

»Mostrar los nombres de los meses del año.

»Solucion:

```
»import calendar
»for i in range(1, 13):
»    print(calendar.month_name[i])
```

»Explicacion: El modulo calendar tiene listas con nombres de meses y dias.

»Ejercicio 20

»Calcular cuantos dias han pasado desde tu nacimiento hasta hoy.

»Solucion:

```
»nacimiento = date(1990, 5, 15)
»hoy = date.today()
»dias = (hoy - nacimiento).days
»print(f"Dias vividos: {dias}")
```

»Resumen de lo que vimos

»Dale, repasemos:

- »■ **date**: solo fecha (año, mes, día).
- »■ **time**: solo hora (hora, minuto, segundo, microsegundo).
- »■ **datetime**: fecha + hora.
- »■ **timedelta**: diferencia de tiempo (para sumar o restar).
- »■ **strftime**: formatear fecha a string.
- »■ **strptime**: parsear string a fecha.
- »■ Siempre atento a los formatos: %Y año con 4 dígitos, %m mes, %d día, etc.

»Reflexion final

»Mirando, el tiempo es complicado: años bisiestos, zonas horarias, cambios de hora. Por suerte, **datetime** se encarga de casi todo. Nunca intentes manejar fechas con strings o haciendo cuentas manuales. Dejalo en manos de Python. Con las herramientas de este modulo vas a poder hacer calendarios, recordatorios, analisis de datos temporales y mucho mas.

»Seguía practicando con los ejercicios. Probando hacer tu propia funcion que te diga cuantos dias faltan para tu cumpleaños. O un programa que te recuerde eventos importantes. Es la mejor forma de aprender.

»_____ El tiempo es relativo, pero datetime es absoluto.

»Modulo math: Funciones matematicas

»Querido lector: la matematica esta a un import de distancia

»Cuando empecé a programar, necesitaba calcular raíces cuadradas. Mi primera reacción fue: “bueno, elevo a la 0.5”. Y funcionaba. Pero después necesité seno, coseno, logaritmos, factorial... y ahí me di cuenta que no podía inventar todo. Me puse a buscar en internet y encontré el modulo `math`.

»Python tiene el modulo `math`. Es una biblioteca de funciones matemáticas que te ahorra tener que implementar vos mismo cosas como el valor de π , la exponencial, o funciones trigonométricas. En este capítulo te voy a mostrar todo lo que puedes hacer con `math`, y además te voy a dar ejemplos pensados para físicos, matemáticos y programadores científicos.

»Importar math

»Como siempre, primero lo importas:

```
»import math
```

»Si solo quieres algunas funciones (para no escribir `math.` todo el tiempo):

```
»from math import sqrt, pi, sin, cos, tan
```

»Constantes matematicas

»Estas son constantes universales que usamos todo el tiempo.

»Numero pi

```
»>>> import math
»>>> print(math.pi)
»3.141592653589793
```

»Numero e (base del logaritmo natural)

```
»>>> print(math.e)
»2.718281828459045
```

» Infinito

» A veces necesitas representar un numero infinitamente grande.

```
»>>> print(math.inf)
»inf
```

» NaN (Not a Number)

» Representa un resultado indefinido, como 0/0.

```
»>>> print(math.nan)
»nan
```

» Funciones basicas

» Redondeo: ceil y floor

» **Ceil** redondea hacia arriba (al entero mayor o igual). **Floor** redondea hacia abajo.

```
»>>> print(math.ceil(4.2))
»5
»>>> print(math.floor(4.9))
»4
```

» Truncar (quitar decimales)

» `trunc()` es como `int()` para numeros positivos: saca los decimales sin redondear.

```
»>>> print(math.trunc(4.9))
»4
```

» Valor absoluto flotante

» Ya existe `abs()` nativo, pero `fabs()` devuelve siempre float.

```
»>>> print(math.fabs(-5.5))
»5.5
```

» Factorial

» El factorial de un numero entero no negativo. Muy usado en combinatoria y probabilidad.

```
»>>> print(math.factorial(5))
»120
```

» Raiz cuadrada

```
»>>> print(math.sqrt(16))
»4.0
```


»Potencia (mas precisa que **)

»pow() devuelve float y maneja mejor casos extremos.

```
»»» print(math.pow(2, 3))
»8.0
```

»Exponencial (e^x)

```
»»» print(math.exp(1))
»2.718281828459045
```

»Logaritmos

»Logaritmo natural (base e), base 10, base 2, o base arbitraria.

```
»»» print(math.log(100, 10))    # log base 10 de 100
»2.0
»»» print(math.log(100))        # log natural (base e)
»4.605170185988092
»»» print(math.log10(100))      # log base 10
»2.0
»»» print(math.log2(8))         # log base 2
»3.0
```

»Funciones trigonometricas

»Ojo: todas trabajan en radianes, no en grados. Es una confusion clasica que tuve al principio.

```
»»» angulo_grados = 90
»»» angulo_radianes = math.radians(angulo_grados)
»»» print(math.sin(angulo_radianes))
»1.0
»»» print(math.cos(angulo_radianes))
»6.123233995736766e-17    # practicamente 0
»»» print(math.tan(angulo_radianes))
»1.633123935319537e+16    # muy grande (infinito)
```

»Conversion entre radianes y grados

```
»»» rad = math.pi
»»» grados = math.degrees(rad)
»»» print(grados)
»180.0
»»» rad2 = math.radians(180)
»»» print(rad2)
»3.141592653589793
```

»Funciones trigonometricas inversas

```
»>>> print(math.asin(1))
»1.5707963267948966 # pi/2
»>>> print(math.acos(1))
»0.0
»>>> print(math.atan(1))
»0.7853981633974483 # pi/4
```

»Funciones de redondeo adicionales

»Parte entera y fraccionaria

```
»>>> x = 3.75
»>>> entero, fraccion = math.modf(x)
»>>> print(entero)
»3.0
»>>> print(fraccion)
»0.75
```

»Copiar signo

»Toma el valor absoluto del primer numero y le pone el signo del segundo.

```
»>>> print(math.copysign(5, -2))
»-5.0
```

»Distancia euclidiana (norma)

»hypot(x, y) calcula $\sqrt{x^2 + y^2}$, util para la hipotenusa o distancia en 2D.

```
»>>> print(math.hypot(3, 4))
»5.0
```

»Funciones de combinacion y permutacion

»Combinaciones (nCr)

»nk, muy usado en probabilidad y estadística.

```
»>>> print(math.comb(5, 2))
»10
```

»Permutaciones (nPr)

» $\frac{n!}{(n-k)!}$

```
»>>> print(math.perm(5, 2))
»20
```

» Verificar si un numero es cercano a otro

» Los numeros flotantes tienen errores de precision. `isclose()` te dice si dos numeros son “aproximadamente iguales” dentro de una tolerancia.

```
>>>> print(math.isclose(0.1 + 0.2, 0.3))
>True
```

» Aplicaciones para fisicos, matematicos y programadores

» Vamos a ver ejemplos concretos donde `math` brilla.

» Area y perimetro de un circulo

» Un clasico de la geometria.

```
>import math
>radio = 5
>area = math.pi * radio ** 2
>perimetro = 2 * math.pi * radio
>print(f"Area: {area:.2f}, Perimetro: {perimetro:.2f}")
```

» Resolver ecuacion cuadratica

» Para fisicos y matematicos: raices de $ax^2 + bx + c = 0$.

```
>def resolver_cuadratica(a, b, c):
>    discriminante = b**2 - 4*a*c
>    if discriminante < 0:
>        return None # raices complejas (necesitas cmath para eso)
>    raiz = math.sqrt(discriminante)
>    x1 = (-b + raiz) / (2*a)
>    x2 = (-b - raiz) / (2*a)
>    return x1, x2
>
>print(resolver_cuadratica(1, -3, 2)) # (2.0, 1.0)
```

» Calcular la hipotenusa (teorema de Pitagoras)

» Muy usado en graficos, navegacion y fisica.

```
>cateto1 = 3
>cateto2 = 4
>hipotenusa = math.hypot(cateto1, cateto2)
>print(hipotenusa) # 5.0
```

»Movimiento parabolico (tiro oblicuo)

»Para fisicos: calcular el alcance de un proyectil.

```
>def alcance(velocidad_inicial, angulo_grados):  
>    g = 9.8  
>    ang_rad = math.radians(angulo_grados)  
>    return (velocidad_inicial**2 * math.sin(2 * ang_rad)) / g  
>  
>print(alcance(20, 45)) # unos 40.8 metros
```

»Conversiones angulares entre grados y radianes

»Util en cualquier programa de simulacion.

```
>ang_rad = math.radians(180)  
>print(ang_rad) # 3.14159  
>ang_grad = math.degrees(math.pi)  
>print(ang_grad) # 180.0
```

»20 ejercicios con soluciones paso a paso

»Ahora te toca a vos. Intentá resolverlos antes de mirar la solucion.

»Ejercicio 1

»Mostrar el valor de pi.

»Solucion:

```
>import math  
>print(math.pi)
```

»Explicacion: `math.pi` es una constante que contiene el valor de π con alta precision.

»Ejercicio 2

»Mostrar el valor de e.

»Solucion:

```
>import math  
>print(math.e)
```

»Explicacion: `math.e` es la base del logaritmo natural.

»Ejercicio 3

»Calcular la raiz cuadrada de 25.

»Solucion:

```
>import math  
>print(math.sqrt(25)) # 5.0
```

»Explicacion: `sqrt()` devuelve la raiz cuadrada como float.

»Ejercicio 4

»Redondear 5.3 hacia arriba.

»Solucion:

```
»import math
»print(math.ceil(5.3)) # 6
```

»Explicacion: `ceil()` redondea al entero mayor o igual.

»Ejercicio 5

»Redondear 5.8 hacia abajo.

»Solucion:

```
»print(math.floor(5.8)) # 5
```

»Explicacion: `floor()` redondea al entero menor o igual.

»Ejercicio 6

»Calcular el factorial de 6.

»Solucion:

```
»print(math.factorial(6)) # 720
```

»Explicacion: $6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720$.

»Ejercicio 7

»Calcular 2 elevado a la 10.

»Solucion:

```
»print(math.pow(2, 10)) # 1024.0
```

»Explicacion: `pow(x, y)` es equivalente a x^y , devuelve float.

»Ejercicio 8

»Calcular el seno de 90 grados.

»Solucion:

```
»import math
»rad = math.radians(90)
»print(math.sin(rad)) # 1.0
```

»Explicacion: Primero convertimos grados a radianes con `radians()`, luego aplicamos `sin()`.

»Ejercicio 9

»Calcular el coseno de 0 grados.

»Solucion:

```
»print(math.cos(0)) # 1.0
```

»Explicacion: `cos(0)` en radianes (0 rad) es 1.

»Ejercicio 10

»Calcular el logaritmo natural de e.

»Solucion:

```
»print(math.log(math.e)) # 1.0
```

»Explicacion: $\ln(e) = 1$.

»Ejercicio 11

»Calcular $\log_{10}(1000)$.

»Solucion:

```
»print(math.log10(1000)) # 3.0
```

»Explicacion: $10^3 = 1000$.

»Ejercicio 12

»Calcular la hipotenusa de un triangulo con catetos 6 y 8.

»Solucion:

```
»print(math.hypot(6, 8)) # 10.0
```

»Explicacion: $\sqrt{6^2 + 8^2} = \sqrt{36 + 64} = 10$.

»Ejercicio 13

»Calcular el area de un circulo de radio 7.

»Solucion:

```
»import math
»radio = 7
»area = math.pi * radio ** 2
»print(f"{area:.2f}")
```

»Explicacion: $\text{Area} = \pi r^2$. Redondeamos a dos decimales.

»Ejercicio 14

»Convertir 180 grados a radianes.

»Solucion:

```
»print(math.radians(180)) # 3.141592653589793
```

»Explicacion: $180 \text{ grados} = \pi \text{ radianes}$.

»Ejercicio 15

»Convertir π radianes a grados.

»Solucion:

```
»print(math.degrees(math.pi)) # 180.0
```

»Explicacion: $\pi \text{ rad} = 180 \text{ grados}$.

»Ejercicio 16

»Calcular la parte entera y fraccionaria de 5.67.

»Solucion:

```
»entero, fraccion = math.modf(5.67)
»print(f"Entero: {entero}, Fraccion: {fraccion}")
```

»Explicacion: `modf()` devuelve una tupla (parte fraccionaria, parte entera) como floats.

»Ejercicio 17

»Calcular 52 (combinaciones de 5 elementos tomados de a 2).

»Solucion:

```
»print(math.comb(5, 2)) # 10
```

»Explicacion: Formula $\frac{5!}{2!(5-2)!} = 10$.

»Ejercicio 18

»Calcular permutaciones de 5 elementos tomados de a 2.

»Solucion:

```
»print(math.perm(5, 2)) # 20
```

»Explicacion: $P(5, 2) = \frac{5!}{(5-2)!} = 5 \cdot 4 = 20$.

»Ejercicio 19

»Calcular la distancia entre dos puntos (2,3) y (5,7) usando `hypot`.

»Solucion:

```
»distancia = math.hypot(5-2, 7-3)
»print(distancia) # 5.0
```

»Explicacion: Diferencia en x = 3, en y = 4, distancia = 5.

»Ejercicio 20

»Verificar si $0,1 + 0,2$ es aproximadamente igual a 0,3 (evitando el error de precision).

»Solucion:

```
»print(math.isclose(0.1 + 0.2, 0.3)) # True
```

»Explicacion: Por la precision de punto flotante, $0,1 + 0,2$ no es exactamente 0,3. `isclose()` usa una tolerancia para decidir.

» Tabla de referencia rapida

Funcion	Que hace
<code>math.pi</code>	π (3.14159...)
<code>math.e</code>	e (2.71828...)
<code>math.sqrt(x)</code>	Raiz cuadrada de x
<code>math.pow(x, y)</code>	x^y (float)
<code>math.exp(x)</code>	e^x
<code>math.log(x)</code>	$\ln x$
<code>math.log10(x)</code>	$\log_{10} x$
<code>math.ceil(x)</code>	Redondeo hacia arriba
<code>math.floor(x)</code>	Redondeo hacia abajo
» <code>math.trunc(x)</code>	Trunca decimales
<code>math.modf(x)</code>	Parte entera y fraccionaria
<code>math.sin(x)</code> , <code>cos</code> , <code>tan</code>	Seno, coseno, tangente (x en radianes)
<code>math.asin(x)</code> , <code>acos</code> , <code>atan</code>	Inversas (arcoseno, etc.)
<code>math.radians(g)</code>	Grados a radianes
<code>math.degrees(r)</code>	Radianes a grados
<code>math.factorial(n)</code>	$n!$
<code>math.comb(n, k)</code>	nk
<code>math.perm(n, k)</code>	$P(n, k)$
<code>math.hypot(x, y)</code>	$\sqrt{x^2 + y^2}$
<code>math.isclose(a, b)</code>	Compara con tolerancia

» Resumen de lo que vimos

» Dale, repasemos lo importante:

- » ■ `math` trae constantes como π y e .
- » ■ Funciones de redondeo: `ceil()`, `floor()`, `trunc()`, `modf()`.
- » ■ Potencias y logaritmos: `sqrt()`, `pow()`, `exp()`, `log()`, `log10()`, `log2()`.
- » ■ Trigonometria: `sin()`, `cos()`, `tan()`, y sus inversas, mas `radians()` y `degrees()`.
- » ■ Combinaciones y factorial: `factorial()`, `comb()`, `perm()`.
- » ■ Geometria: `hypot()` para la hipotenusa.
- » ■ Comparacion segura: `isclose()`.

» Reflexion final

» Mirá, el modulo `math` es como tener una calculadora cientifica dentro de Python. No intentes implementar vos mismo una raiz cuadrada o un seno. Usa `math`. Es mas rapido, mas preciso y mas legible.

»Si sos fisico, matematico o programador cientifico, este modulo va a ser uno de tus mejores amigos. Desde calcular trayectorias hasta hacer analisis estadistico, `math` te cubre las necesidades basicas. Y cuando necesites algo mas avanzado (matrices, algebra lineal, etc.), ahi ya vas a pasar a `numpy`. Pero `math` es la base.

»SeguÃn practicando con los ejercicios. ProbÃ hacer tus propias funciones que calculen el area de un triangulo, o la distancia entre dos puntos en 3D (usando `math.sqrt` y sumas de cuadrados). No tengas miedo de experimentar.

»_____ *La matematica es la base de la
programacion. math es el puente.*

»Modulo re: Expresiones regulares (búsqueda avanzada en texto)

»Querido lector: cuando buscar texto se vuelve poderoso

»Imaginate que tienes un archivo con cien mil líneas y necesitas encontrar todos los correos electrónicos, o todas las fechas, o todas las palabras que empiezan con mayúscula. Hacerlo con 'in' o con 'find()' sería un suicidio. Te lo digo por experiencia: una vez perdí horas intentando validar emails con un montón de condiciones, y al final el código era un desastre.

»Ahí aparecen las **expresiones regulares** (regex). Son un lenguaje para describir patrones de texto. Una vez que las aprendes, puedes buscar, validar y extraer información de strings de una manera increíblemente poderosa. Es como tener un superpoder para procesar texto.

»En Python, el módulo 're' se encarga de todo esto.

»Importar re y primeros pasos

```
>>>> import re
```

»Buscar si un patrón aparece (search)

»'search()' busca la primera aparición del patrón en el string.

```
>>>> texto = "Mi correo es nahuel@ejemplo.com"
>>>> patron = r"@"
>>>> if re.search(patron, texto):
>...     print("Encontre una arroba")
>...
>Encontre una arroba
```

»La 'r' antes del string es para raw string", hace que las contrabarras no se interpreten como escapes. Siempre usa 'r' para expresiones regulares. Es un error común olvidarse de la 'r' y que las '.' se conviertan en algo raro.

»Verificar si todo el string coincide con el patrón (match)

»'match()' verifica desde el principio del string.

```
>>>> texto = "Python es genial"
>>>> if re.match(r"Python", texto):
```

```
»... print("Empieza con Python")
»...
»Empieza con Python
```

»Metacaracteres basicos

»Aca van los simbolos que usas para construir patrones. Son la gramatica de las regex.

Caracter	Significado
‘.’	Cualquier caracter (excepto salto de linea)
‘\d’	Digito (0-9)
‘\D’	No digito
‘\w’	Letra, digito o guion bajo (alfanumerico + _)
‘\W’	No alfanumerico
‘\s’	Espacio, tabulacion, salto de linea
‘\S’	No espacio
‘^’	Inicio de string
‘\$’	Fin de string

»Ejemplos

```
»>>> texto = "Hola 123"
»>>> print(re.search(r"\d", texto)) # encuentra el '1'
»<re.Match object; span=(5, 6), match='1'>
»>>> print(re.search(r"\D", texto)) # encuentra la 'H'
»<re.Match object; span=(0, 1), match='H'>
»>>> print(re.search(r"\w", texto)) # encuentra la 'H'
»<re.Match object; span=(0, 1), match='H'>
»>>> print(re.search(r"\s", texto)) # encuentra el espacio
»<re.Match object; span=(4, 5), match=' '>
```

»Cuantificadores (repeticiones)

»Estos te dicen cuantas veces se repite un caracter o grupo.

Cuantificador	Significado
‘*’	0 o mas veces
‘+’	1 o mas veces
‘?’	0 o 1 vez
‘n’	Exactamente n veces
‘n,’	n o mas veces
‘n,m’	entre n y m veces

»Ejemplos

```
»>>> texto = "aaa a aaa"
»>>> print(re.findall(r"a+", texto))
```

```
»['aaa', 'a', 'aaa']
»>>> print(re.findall(r"a{2,}", texto))
»['aaa', 'aaa']
```

» Clases de caracteres (corchetes)

»Dentro de '['' puedes especificar un conjunto de caracteres permitidos.

```
»>>> texto = "Python3.10"
»>>> print(re.findall(r"[aeiou]", texto))
»['o', 'o']
»>>> print(re.findall(r"[0-9]", texto))
»['3', '1', '0']
»>>> print(re.findall(r"[a-zA-Z]", texto))
»['P', 'y', 't', 'h', 'o', 'n']
»>>> print(re.findall(r"[^aeiou]", texto))
»['P', 'y', 't', 'h', 'n', '3', '.', '1', '0']
```

» Agrupacion y captura (parentesis)

»Los parentesis '('' agrupan y capturan^{el} texto. Muy util para extraer partes especificas.

```
»>>> texto = "nombre: Juan, edad: 25"
»>>> patron = r"nombre: (\w+), edad: (\d+)"
»>>> resultado = re.search(patron, texto)
»>>> if resultado:
»...     print(resultado.group(1)) # Juan
»...     print(resultado.group(2)) # 25
»...
»Juan
»25
```

» Grupos con nombre

»Podes ponerles nombre para que el codigo sea mas claro.

```
»>>> patron = r"nombre: (?P<nom>\w+), edad: (?P<ed>\d+)"
»>>> resultado = re.search(patron, texto)
»>>> print(resultado.group("nom"))
»Juan
»>>> print(resultado.group("ed"))
»25
```

» Funciones principales de re

» search() - buscar primera coincidencia

»Devuelve un objeto match o None. Sirve para saber si algo aparece.

» **match()** - buscar desde el inicio

» Parecido a search, pero solo al principio del string.

» **findall()** - lista con todas las coincidencias

```
»>>> texto = "manzana 10, pera 20, uva 30"
»>>> numeros = re.findall(r"\d+", texto)
»>>> print(numeros)
»['10', '20', '30']
```

» **finditer()** - iterador de coincidencias (mas eficiente)

» Si tienes muchas coincidencias, 'finditer()' es mejor porque no crea una lista gigante.

```
»>>> for match in re.finditer(r"\d+", texto):
»...     print(match.group(), match.span())
»...
»10 (8, 10)
»20 (16, 18)
»30 (23, 25)
```

» **sub()** - reemplazar coincidencias

```
»>>> texto = "La fecha es 2024-05-20"
»>>> modificado = re.sub(r"\d{4}-\d{2}-\d{2}", "DD/MM/AAAA", texto)
»>>> print(modificado)
»La fecha es DD/MM/AAAA
```

» **split()** - dividir por un patron

```
»>>> texto = "uno,dos,tres,cuatro"
»>>> partes = re.split(r",", texto)
»>>> print(partes)
»['uno', 'dos', 'tres', 'cuatro']
```

» **Compilacion de patrones (para mayor velocidad)**

» Si vas a usar el mismo patron muchas veces (por ejemplo dentro de un bucle), compilarlo mejora el rendimiento.

```
»>>> patron_compilado = re.compile(r"\d+")
»>>> textos = ["123", "abc45", "xyz"]
»>>> for t in textos:
»...     if patron_compilado.search(t):
»...         print(f"{t} tiene numeros")
»...
»123 tiene numeros
»abc45 tiene numeros
```

» Ejemplos practicos

» Validar email simple

» Un email tipico tiene: letras, punto, guion, arroba, dominio, etc.

```
>def es_email_valido(email):
>    patron = r"^[w\.-]+@[w\.-]+\.\w+$"
>    return re.match(patron, email) is not None
>
>print(es_email_valido("user@example.com")) # True
>print(es_email_valido("user@example"))     # False
```

» Extraer todas las URLs de un texto

```
>texto = "Visita https://python.org o http://ejemplo.com"
>urls = re.findall(r"https?:\/\/[w\.-]+", texto)
>print(urls) # ['https://python.org', 'http://ejemplo.com']
```

» Validar telefono argentino (11 1234-5678)

```
>def validar_telefono(tel):
>    patron = r"^11 \d{4}-\d{4}$"
>    return re.match(patron, tel) is not None
>
>print(validar_telefono("11 1234-5678")) # True
>print(validar_telefono("11 123-4567"))  # False
```

» 20 ejercicios con soluciones paso a paso

» Ahora te toca a vos. Resolve cada ejercicio antes de mirar la solucion.

» Ejercicio 1

» Verificar si un string contiene al menos un numero.

» Solucion:

```
>import re
>def tiene_numero(s):
>    return re.search(r"\d", s) is not None
>
>print(tiene_numero("hola")) # False
>print(tiene_numero("hola123")) # True
```

» Explicacion: ‘.’ busca cualquier digito. ‘search()’ devuelve ‘None’ si no encuentra.

»Ejercicio 2

»Extraer todos los numeros de un string.

»Solucion:

```
>texto = "Hay 10 manzanas, 20 peras y 30 uvas"
>numeros = re.findall(r"\d+", texto)
>print(numeros) # ['10', '20', '30']
```

»Explicacion: '+' significa uno o mas digitos. 'findall()' devuelve lista.

»Ejercicio 3

»Verificar si un string es un email valido (version simple).

»Solucion:

```
>def es_email(email):
>    patron = r"^[a-zA-Z0-9_]+@[a-zA-Z0-9_]+\.[a-zA-Z]+$"
>    return bool(re.match(patron, email))
>
>print(es_email("test@example.com")) # True
>print(es_email("test@example")) # False
```

»Explicacion: '^y' aseguran que todo el string coincida. '[a-zA-Z0-9_]+' son letras, digitos, punto o guion.

»Ejercicio 4

»Reemplazar todos los numeros de un string por "".

»Solucion:

```
>texto = "Mi edad es 25, vivo en la casa 10"
>resultado = re.sub(r"\d", "", texto)
>print(resultado) # Mi edad es ##, vivo en la casa ##
```

»Explicacion: 'sub()' reemplaza cada digito (uno por uno) por "". Si usas '+' reemplazaria cada numero entero.

»Ejercicio 5

»Extraer todas las palabras que empiezan con mayuscula.

»Solucion:

```
>texto = "Hola mundo, Python es Genial"
>mayusculas = re.findall(r"\b[A-Z][a-z]*\b", texto)
>print(mayusculas) # ['Hola', 'Python', 'Genial']
```

»Explicacion: '_' es limite de palabra. '[A-Z]' una mayuscula, '[a-z]*' cero o mas minusculas.

» Ejercicio 6

» Verificar si un string comienza con "Python".

» **Solucion:**

```
» texto = "Python es genial"
» if re.match(r"Python", texto):
»     print("Empieza con Python")
```

» **Explicacion:** 'match()' verifica desde el inicio. No hace falta *'porque match ya ancla al principio.'*

» Ejercicio 7

» Verificar si un string termina con "fin".

» **Solucion:**

```
» texto = "El cuento tiene un fin"
» if re.search(r"fin$", texto):
»     print("Termina con 'fin'")
```

» **Explicacion:** *"significa" fin de string*. 'search()' puede encontrar en cualquier posicion.

» Ejercicio 8

» Extraer las fechas en formato DD/MM/AAAA de un texto.

» **Solucion:**

```
» texto = "Naci el 15/05/1990, y mi hermano el 20/12/1995"
» fechas = re.findall(r"\d{2}/\d{2}/\d{4}", texto)
» print(fechas) # ['15/05/1990', '20/12/1995']
```

» **Explicacion:** '2' exactamente dos digitos. La barra '/' se usa literal.

» Ejercicio 9

» Validar un telefono fijo argentino (11 1234-5678).

» **Solucion:**

```
» patron = r"^11 \d{4}-\d{4}$"
» print(bool(re.match(patron, "11 1234-5678"))) # True
» print(bool(re.match(patron, "11 123-4567"))) # False
```

» **Explicacion:** Espacio literal, luego 4 digitos, guion, 4 digitos.

» Ejercicio 10

» Extraer las palabras que tienen exactamente 5 letras.

» **Solucion:**


```

»texto = "Python es un lenguaje genial y poderoso"
»cinco = re.findall(r"\b\w{5}\b", texto)
»print(cinco) # ['lenguaje']? Revisar: 'Python' tiene 6, 'lenguaje' tiene 8, 'genial' t
»# En este texto, ninguna palabra tiene 5 letras. Usemos otro: "Hola mundo python"
»texto2 = "Hola mundo python"
»cinco2 = re.findall(r"\b\w{5}\b", texto2)
»print(cinco2) # ['mundo']

```

»**Explicacion:** ‘_’ limite, ‘5’ exactamente 5 caracteres alfanumericos.

»Ejercicio 11

»Dividir un string por espacios o comas.

»**Solucion:**

```

»texto = "manzana, pera, uva, naranja"
»partes = re.split(r"[, ]+", texto)
»print(partes) # ['manzana', 'pera', 'uva', 'naranja']

```

»**Explicacion:** ‘[,]+’ significa uno o mas comas o espacios”. El ‘+’ evita elementos vacios.

»Ejercicio 12

»Contar cuantas veces aparece la palabra "Python"(sin importar mayusculas).

»**Solucion:**

```

»texto = "Python y python son lo mismo, PYTHON"
»matches = re.findall(r"python", texto, re.IGNORECASE)
»print(len(matches)) # 3

```

»**Explicacion:** La bandera ‘re.IGNORECASE’ hace que la busqueda no distinga mayusculas.

»Ejercicio 13

»Extraer el dominio de un email (lo que va despues de @).

»**Solucion:**

```

»email = "nahuel@ejemplo.com"
»dominio = re.search(r"@([\w\.-]+)", email)
»if dominio:
»    print(dominio.group(1)) # ejemplo.com

```

»**Explicacion:** El parentesis captura lo que sigue a ‘@’. ‘[]+’ caracteres validos en un dominio.

»Ejercicio 14

»Validar que una contrasena tenga al menos 8 caracteres, una mayuscula y un numero.

»**Solucion:**

```
>def validar_password(pwd):
>    if len(pwd) < 8:
>        return False
>    if not re.search(r"[A-Z]", pwd):
>        return False
>    if not re.search(r"\d", pwd):
>        return False
>    return True
>
>print(validar_password("Hola1234")) # True
>print(validar_password("hola1234")) # False (sin mayuscula)
```

»**Explicacion:** Usamos tres condiciones separadas. No hace falta una unica regex gigante.

»Ejercicio 15

»Eliminar todos los caracteres que no sean letras ni numeros.

»**Solucion:**

```
>texto = "Hola! @mundo# 123"
>limpio = re.sub(r"[^a-zA-Z0-9]", "", texto)
>print(limpio) # Holamundo123
```

»**Explicacion:** '[^a-zA-Z0-9]' significa "cualquier caracter que NO sea letra mayuscula, minuscula o numero".

»Ejercicio 16

»Extraer las horas (HH:MM) de un texto.

»**Solucion:**

```
>texto = "La reunion es a las 14:30 y otra a las 18:45"
>horas = re.findall(r"\d{2}:\d{2}", texto)
>print(horas) # ['14:30', '18:45']
```

»**Explicacion:** Dos digitos, dos puntos, dos digitos.

»Ejercicio 17

»Reemplazar todas las vocales por "*".

»**Solucion:**

```
>texto = "Hola mundo"
>resultado = re.sub(r"[aeiouAEIOU]", "*", texto)
>print(resultado) # H*1* m*nd*
```

»**Explicacion:** La clase '[aeiouAEIOU]' incluye todas las vocales.

»Ejercicio 18

»Verificar si un string es un numero entero (puede tener signo).

»Solucion:

```
>def es_entero(s):
>    return bool(re.match(r"^-?\d+$", s))
>
>print(es_entero("123"))    # True
>print(es_entero("-45"))    # True
>print(es_entero("12.3"))   # False
```

»Explicacion: ‘-’ significa "guion opcional". ‘+’ uno o mas digitos. ‘\$’ aseguran toda la cadena.

»Ejercicio 19

»Extraer todas las palabras que contengan la letra ‘a’.

»Solucion:

```
>texto = "manzana, pera, banana, kiwi"
>palabras_con_a = re.findall(r"\b\w*a\w*\b", texto)
>print(palabras_con_a)    # ['manzana', 'pera', 'banana']
```

»Explicacion: ‘*a*’ significa cero o mas alfanumericos, luego una ‘a’, luego cero o mas alfanumericos”. Los ‘_’ aseguran palabras completas.

»Ejercicio 20

»Usar ‘finditer()’ para mostrar posicion y contenido de cada numero en un string.

»Solucion:

```
>texto = "Hay 10 gatos, 5 perros y 3 peces"
>for match in re.finditer(r"\d+", texto):
>    print(f"Numero {match.group()} en posicion {match.start()}")
```

»Explicacion: ‘finditer()’ devuelve un iterador de objetos match. Cada match tiene ‘group()’ (el texto) y ‘start()’ (indice inicial).

»Tabla de referencia rapida

Funcion	Que hace
‘re.search(patron, texto)’	Busca primera coincidencia en cualquier parte
‘re.match(patron, texto)’	Busca solo al principio del texto
‘re.findall(patron, texto)’	Devuelve lista de todas las coincidencias
» ‘re.finditer(patron, texto)’	Devuelve iterador de objetos match
‘re.sub(patron, reemplazo, texto)’	Reemplaza todas las coincidencias
‘re.split(patron, texto)’	Divide el texto por el patron
‘re.compile(patron)’	Compila el patron para reutilizarlo

»Metacaracteres y cuantificadores

Patron	Significado
'.'	Cualquier caracter (excepto salto de linea)
'\d'	Digito (0-9)
'\w'	Letra, digito o guion bajo
'\s'	Espacio, tab, salto de linea
'^'	Inicio de string
'\$'	Fin de string
'*'	0 o mas
'+'	1 o mas
'?'	0 o 1
'n'	Exactamente n
'n,''	n o mas
'n,m'	entre n y m
'[abc]'	Un caracter de a, b o c
'[abc]'	Un caracter que no sea a, b o c
'(ab)'	Grupo (captura)
'(?:ab)'	Grupo no capturante
'(?P<name>...)'	Grupo con nombre

»Resumen de lo que vimos

»Dale, repasemos:

- »■ Las expresiones regulares describen patrones de texto.
- »■ Metacaracteres: '.', '\d', '\w', '\s', '^', '\$'.
- »■ Cuantificadores: '*', '+', '?', 'n', 'n,m'.
- »■ Clases: '[aeiou]', '[0-9]', '[...]'.
- »■ Agrupacion con '()' y grupos con nombre.
- »■ Funciones: 'search()', 'match()', 'findall()', 'finditer()', 'sub()', 'split()', 'compile()'.
- »■ Siempre usa raw strings: 'r"patron"'.
 - »■ Las banderas como 're.IGNORECASE' cambian el comportamiento.

»Reflexion final

»Las expresiones regulares son como un cuchillo suizo para texto. Aprendelas poco a poco, empieza con patrones simples, y vas a ver como transforman la forma en que procesas informacion. Pero ojo: un mal regex puede ser lento o difícil de leer. No abuses. A veces un 'split()' o 'find()' simple es mejor que una regex complicada.

»Seguñ practicando con los ejercicios. Proba validar tus propios datos, como numeros de telefono, patentes, codigos postales. Cuanto mas las uses, mas naturales se van a volver.

» _____ *El texto es caos. Las regex son
orden.*

Manuel Villafañe - Python paso

»Modulo pathlib: La forma moderna de manejar rutas

»Querido lector: olvidate de os.path

»Cuando yo empecé a programar, para manejar rutas usaba `os.path.join()`, `os.path.dirname()`, `os.path.basename()`, etc. Funcionaba, pero era engorroso. Además, todo eran strings. Cada vez que quería obtener el nombre del archivo tenía que acordarme de la función exacta, y cuando cambiaba de sistema operativo las barras me volvían loco.

»Un día descubrí `pathlib` y fue amor a primera vista. Te explico: `pathlib` trata las rutas como **objetos**, no como strings. Tiene métodos súper intuitivos y funciona en Windows, Linux y Mac sin que te preocupes por las barras. Es como pasar de usar un mapa de papel a usar el GPS.

»En este capítulo te voy a mostrar todo lo que puedes hacer con `pathlib`. Y como siempre, al final tienes 20 ejercicios para practicar.

»Importar y crear rutas

»Lo primero es importar la clase `Path`:

```
»»» from pathlib import Path
```

»Crear un objeto Path

»Para crear una ruta, simplemente llamas a `Path()` con los componentes:

```
»»» # Ruta actual
»»» actual = Path.cwd()
»»» print(actual)
»/home/usuario/proyecto
»
»»» # Ruta a un archivo (podes pasar varios argumentos)
»»» archivo = Path("mi_carpeta", "subcarpeta", "archivo.txt")
»»» print(archivo)
»mi_carpeta/subcarpeta/archivo.txt
```

» Usar el operador / para concatenar

» La magia de 'pathlib' es que puedes usar el slash '/' para unir rutas. Es mucho más legible que 'os.path.join()'.

```

>>> base = Path("/home/usuario")
>>> completa = base / "documentos" / "nota.txt"
>>> print(completa)
>/home/usuario/documentos/nota.txt

```

» Propiedades útiles

» Una vez que tienes un objeto 'Path', puedes acceder a sus partes con propiedades muy claras:

```

>>> ruta = Path("/home/usuario/documentos/archivo.txt")
>>> print(ruta.name)           # archivo.txt
>archivo.txt
>>> print(ruta.stem)           # archivo (sin extension)
>archivo
>>> print(ruta.suffix)         # .txt
> .txt
>>> print(ruta.suffixes)       # para archivos con doble extension
> ['.txt']
>>> print(ruta.parent)         # /home/usuario/documentos
>/home/usuario/documentos
>>> print(ruta.parents[0])     # lo mismo
>/home/usuario/documentos
>>> print(ruta.parents[1])     # /home/usuario
>/home/usuario
>>> print(ruta.anchor)         # la raíz del sistema ( / en Linux, C:\ en Windows)
>/

```

» Verificar existencia y tipo

» Antes de hacer algo con un archivo o carpeta, siempre es bueno preguntar si existe y que es. Con 'pathlib' es muy sencillo:

```

>>> ruta = Path("archivo.txt")
>>> print(ruta.exists())       # True o False
>True
>>> print(ruta.is_file())      # True si es archivo
>True
>>> print(ruta.is_dir())       # True si es carpeta
>False

```

»Operaciones con archivos

»Leer y escribir texto

»‘pathlib’ tiene metodos directos para leer y escribir texto. No hace falta usar ‘open()’ si solo quieres todo el contenido de una vez.

```
»>>> archivo = Path("datos.txt")
»>>> # Escribir
»>>> archivo.write_text("Hola mundo", encoding="utf-8")
»11
»>>> # Leer
»>>> contenido = archivo.read_text(encoding="utf-8")
»>>> print(contenido)
»Hola mundo
```

»Leer lineas

»Si necesitas las lineas como lista, podes usar ‘splitlines()’:

```
»>>> lineas = archivo.read_text(encoding="utf-8").splitlines()
»>>> print(lineas)
»['Hola mundo']
```

»Copiar y mover

»‘pathlib’ no tiene metodos directos de copiar o mover, pero se combina muy bien con el modulo ‘shutil’. Te muestro:

```
»>>> import shutil
»>>> origen = Path("datos.txt")
»>>> destino = Path("backup/datos.txt")
»>>> shutil.copy(origen, destino)
```

»Eliminar archivos y carpetas

```
»>>> archivo = Path("temporal.txt")
»>>> archivo.unlink()          # elimina archivo
»>>> carpeta = Path("vacía")
»>>> carpeta.rmdir()           # elimina carpeta VACIA (si tiene algo, da error)
```


»Recorrer directorios

»Listar contenido simple

»'iterdir()' te da un iterador con todos los elementos de una carpeta:

```

>>> carpeta = Path(".")
>>> for item in carpeta.iterdir():
>...     if item.is_file():
>...         print(f"Archivo: {item.name}")
>...     elif item.is_dir():
>...         print(f"Carpeta: {item.name}")
>...
>Archivo: script.py
>Carpeta: docs
>Archivo: readme.md

```

»Buscar archivos con glob (comodines)

»'glob()' te permite usar comodines como '*' y '?'. Es como buscar archivos en la terminal.

```

>>> # Todos los archivos .txt de la carpeta actual
>>> for archivo in Path(".").glob("*.txt"):
>...     print(archivo)
>...
>notas.txt
>lista.txt
>
>>> # Recursivo: busca en todas las subcarpetas
>>> for archivo in Path(".").glob("**/*.py"):
>...     print(archivo)
>...
>script.py
>src/main.py
>tests/test.py

```

»Usar rglob (recursivo directo)

»'rglob()' es un atajo para 'glob("**/*")'. Es mas comodo.

```

>>> for archivo in Path(".").rglob("*.py"):
>...     print(archivo)
>...
>script.py
>src/main.py
>tests/test.py

```

» Crear carpetas

» `mkdir()` te permite crear carpetas. Con `parents=True` crea toda la estructura de una vez, y con `exist_ok=True` no da error si ya existe.

```
>>> # Crear una carpeta simple
>>> Path("nueva_carpeta").mkdir(exist_ok=True)
>>> # Crear estructura anidada de una sola vez
>>> Path("a/b/c").mkdir(parents=True, exist_ok=True)
```

» Rutas absolutas y relativas

» A veces necesitas la version absoluta de una ruta, o extraer una parte relativa a otra ruta.

```
>>> ruta_rel = Path("documentos/archivo.txt")
>>> print(ruta_rel.absolute())
>/home/usuario/proyecto/documentos/archivo.txt
>
>>> ruta_abs = Path("/home/usuario/archivo.txt")
>>> print(ruta_abs.relative_to("/home"))
>usuario/archivo.txt
```

» Comparacion con os.path

» Para que veas lo mas limpio que es `pathlib`, aca tenes una tabla comparativa:

os.path	pathlib
<code>'os.path.join(a,b)'</code>	<code>'Path(a) / b'</code>
<code>'os.path.basename(p)'</code>	<code>'Path(p).name'</code>
<code>'os.path.dirname(p)'</code>	<code>'Path(p).parent'</code>
» <code>'os.path.splitext(p)'</code>	<code>'Path(p).stem' y '.suffix'</code>
<code>'os.path.exists(p)'</code>	<code>'Path(p).exists()'</code>
<code>'os.path.isfile(p)'</code>	<code>'Path(p).is_file()'</code>
<code>'os.path.isdir(p)'</code>	<code>'Path(p).is_dir()'</code>
<code>'os.walk()'</code>	<code>'Path(p).rglob("*")'</code>

» Ejemplos practicos

» Organizar archivos por extension

» Este programa toma todos los archivos de la carpeta actual y los mueve a subcarpetas segun su extension (ej: `'txt'` a carpeta `'txt'`, `'jpg'` a `'jpg'`).

```

»from pathlib import Path
»import shutil
»
»carpeta = Path(".")
»for archivo in carpeta.glob("*"):
»    if archivo.is_file():
»        ext = archivo.suffix[1:] # sacamos el punto
»        if ext: # si tiene extension
»            destino = carpeta / ext
»            destino.mkdir(exist_ok=True)
»            shutil.move(str(archivo), str(destino / archivo.name))
»            print(f"Movido {archivo.name} a {ext}/")

```

»Buscar archivos grandes

»Funcion que recorre una carpeta recursivamente y muestra los archivos que pesan mas de cierto limite.

```

»def buscar_grandes(carpeta, limite_mb=1):
»    limite = limite_mb * 1024 * 1024
»    for archivo in Path(carpeta).rglob("*"):
»        if archivo.is_file() and archivo.stat().st_size > limite:
»            tamaño_mb = archivo.stat().st_size / (1024*1024)
»            print(f"{archivo} - {tamaño_mb:.2f} MB")
»
»# Ejemplo: buscar archivos mayores a 5 MB en la carpeta actual
»buscar_grandes(".", 5)

```

»20 ejercicios con soluciones paso a paso

»Ahora te toca a vos. Hace cada ejercicio por tu cuenta antes de mirar la solucion. Vas a ver que 'pathlib' es un placer de usar.

»Ejercicio 1

»Crear un objeto Path para la ruta actual y mostrarla.

»Solucion:

```

»from pathlib import Path
»actual = Path.cwd()
»print(actual)

```

»**Explicacion:** 'Path.cwd()' es el equivalente a 'os.getcwd()'. Devuelve un objeto 'Path' con la ruta del directorio actual.

»Ejercicio 2

»Concatenar rutas usando el operador `'/'` para llegar a `carpeta/archivo.txt`.

»Solucion:

```
»ruta = Path("carpeta") / "archivo.txt"
»print(ruta)
```

»Explicacion: El operador `'/'` une componentes de ruta. Es mucho mas legible que `'os.path.join()'`.

»Ejercicio 3

»Obtener el nombre, el nombre sin extension y la extension de `"documento.pdf"`.

»Solucion:

```
»p = Path("documento.pdf")
»print(p.name)      # documento.pdf
»print(p.stem)      # documento
»print(p.suffix)    # .pdf
```

»Explicacion: `'name'` da el nombre completo, `'stem'` quita la extension, `'suffix'` da la extension (incluyendo el punto).

»Ejercicio 4

»Verificar si un archivo existe (pedir nombre al usuario).

»Solucion:

```
»nombre = input("Nombre del archivo: ")
»if Path(nombre).exists():
»    print("El archivo existe")
»else:
»    print("No existe")
```

»Explicacion: `'Path(nombre).exists()'` devuelve `'True'` si el archivo o carpeta existe en el sistema.

»Ejercicio 5

»Crear una carpeta `"mis_datos"` (si ya existe, no dar error).

»Solucion:

```
»Path("mis_datos").mkdir(exist_ok=True)
```

»Explicacion: `'exist_ok = True'` evita el error `'FileExistsError'` si la carpeta ya esta creada.

»Ejercicio 6

»Escribir "Hola Python." en un archivo "saludo.txt" luego leerlo.

»Solucion:

```
»archivo = Path("saludo.txt")
»archivo.write_text("Hola Python", encoding="utf-8")
»contenido = archivo.read_text(encoding="utf-8")
»print(contenido)
```

»Explicacion: `write_text()` 'escribetodoelstringenelarchivo(sobrescribe)'. `read_text()` 'leetodoelcontenido'.

»Ejercicio 7

»Listar todos los archivos '.py' de la carpeta actual.

»Solucion:

```
»for archivo in Path(".").glob("*.py"):
»    print(archivo.name)
```

»Explicacion: `glob("*.py")` busca archivos que terminen en '.py' en la carpeta actual (no subcarpetas).

»Ejercicio 8

»Listar recursivamente todos los archivos '.txt'.

»Solucion:

```
»for archivo in Path(".").rglob("*.txt"):
»    print(archivo)
```

»Explicacion: `rglob("*.txt")` es equivalente a `glob("**/*.txt")`: busca en todas las subcarpetas.

»Ejercicio 9

»Obtener el directorio padre de una ruta y el padre del padre.

»Solucion:

```
»ruta = Path("a/b/c/archivo.txt")
»print(ruta.parent)      # a/b/c
»print(ruta.parents[1])  # a/b
```

»Explicacion: `.parent` da el directorio que contiene al archivo. `.parents` es una secuencia con todos los ancestros.

»Ejercicio 10

»Verificar si una ruta es un archivo o una carpeta.

»Solucion:

```
»p = Path("algo")
»if p.is_file():
»    print("Es un archivo")
»elif p.is_dir():
»    print("Es una carpeta")
»else:
»    print("No existe")
```

»Explicacion: `is_file()` y `is_dir()` devuelven `True` solo si el elemento existe y es del tipo indicado.

»Ejercicio 11

»Crear la estructura de carpetas "proyecto/src", "proyecto/docs" usando pathlib.

»Solucion:

```
»Path("proyecto/src").mkdir(parents=True, exist_ok=True)
»Path("proyecto/docs").mkdir(parents=True, exist_ok=True)
```

»Explicacion: `parents=True` crea las carpetas intermedias (en este caso 'proyecto'). `exist_ok = True` evita errores si ya existen.

»Ejercicio 12

»Obtener la ruta absoluta de un archivo relativo.

»Solucion:

```
»relativo = Path("archivo.txt")
»absoluto = relativo.absolute()
»print(absoluto)
```

»Explicacion: `absolute()` devuelve la ruta completa desde la raiz del sistema de archivos.

»Ejercicio 13

»Obtener el tamaño en bytes de un archivo.

»Solucion:

```
»archivo = Path("datos.txt")
»if archivo.exists():
»    print(archivo.stat().st_size, "bytes")
```

»Explicacion: `stat()` devuelve un objeto con informacion del archivo. `st_size` es el tamaño en bytes.

»Ejercicio 14

»Renombrar un archivo de "viejo.txt" a "nuevo.txt".

»Solucion:

```
»Path("viejo.txt").rename("nuevo.txt")
```

»**Explicacion:** 'rename()' cambia el nombre del archivo. Tambien se puede usar para moverlo a otra carpeta.

»Ejercicio 15

»Eliminar un archivo (con verificacion previa).

»Solucion:

```
»archivo = Path("temporal.txt")
»if archivo.exists():
»    archivo.unlink()
»    print("Archivo eliminado")
»else:
»    print("No existe")
```

»**Explicacion:** 'unlink()' borra el archivo. Es buena practica verificar que existe antes para evitar errores.

»Ejercicio 16

»Contar cuantos archivos hay en la carpeta actual (sin subcarpetas).

»Solucion:

```
»contador = sum(1 for item in Path(".").iterdir() if item.is_file())
»print(f"Archivos: {contador}")
```

»**Explicacion:** 'iterdir()' da todos los elementos. Filtramos con 'is_file()' y sumamos.

»Ejercicio 17

»Contar cuantos archivos '.py' hay recursivamente.

»Solucion:

```
»contador = sum(1 for _ in Path(".").rglob("*.py"))
»print(f"Archivos .py: {contador}")
```

»**Explicacion:** 'rglob("*.py")' genera todos los archivos que coinciden, incluyendo subcarpetas. 'sum(1 for _ in ...)' cuenta eficientemente sin crear una lista.

»Ejercicio 18

»Obtener la fecha de ultima modificacion de un archivo.

»Solucion:

```
»import time
»archivo = Path("datos.txt")
»if archivo.exists():
»    mtime = archivo.stat().st_mtime
»    print("Ultima modificacion:", time.ctime(mtime))
```

»Explicacion: `'st_mtime'` es el timestamp de la ultima modificacion. `'time.ctime()'` lo convierte a un string legible.

»Ejercicio 19

»Leer un archivo y mostrar las lineas que contienen la palabra `.error`.

»Solucion:

```
»archivo = Path("log.txt")
»if archivo.exists():
»    for linea in archivo.read_text(encoding="utf-8").splitlines():
»        if "error" in linea.lower():
»            print(linea)
```

»Explicacion: Leemos todo el texto, lo dividimos en lineas con `'splitlines()'`, y filtramos las que contienen `.error`.

»Ejercicio 20

»Crear un programa que recorra la carpeta actual y muestre el tamaño total de todos los archivos.

»Solucion:

```
»total = 0
»for archivo in Path(".").rglob("*"):
»    if archivo.is_file():
»        total += archivo.stat().st_size
»print(f"Tamaño total: {total} bytes")
»print(f"{total / 1024 / 1024:.2f} MB")
```

»Explicacion: Recorremos recursivamente todos los archivos, sumamos sus tamaños, y mostramos en bytes y megabytes.

» Tabla de referencia rapida

Operacion	pathlib
Ruta actual	<code>'Path.cwd()'</code>
Concatenar	<code>'Path(a) / b / c'</code>
Nombre del archivo	<code>'p.name'</code>
Nombre sin extension	<code>'p.stem'</code>
Extension	<code>'p.suffix'</code>
Carpeta padre	<code>'p.parent'</code>
Verificar existencia	<code>'p.exists()'</code>
Es archivo?	<code>'p.is_file()'</code>
Es carpeta?	<code>'p.is_dir()'</code>
Crear carpeta	<code>'p.mkdir(exist_ok = True)'</code>
Eliminar archivo	<code>'p.unlink()'</code>
Leer texto	<code>'p.read_text(encoding = 'utf-8')'</code>
Escribir texto	<code>'p.write_text(texto, encoding = 'utf-8')'</code>
Listar carpeta	<code>'p.iterdir()'</code>
Buscar archivos	<code>'p.glob("*.py")'</code>
Buscar recursivo	<code>'p.rglob("*.py")'</code>
Tamano del archivo	<code>'p.stat().st_size'</code>

» Resumen de lo que vimos

» Dale, repasemos lo importante:

- »• 'pathlib' maneja rutas como ****objetos 'Path'**, no como strings.**
- »• Para concatenar se usa el operador `'/'` (mucho mas limpio que `'os.path.join()'`).
- »• Propiedades utiles: `'name'`, `'stem'`, `'suffix'`, `'parent'`, `'parents'`.
- »• Metodos de verificacion: `'exists()'`, `'is_file()'`, `'is_dir()'`.
- »• Crear carpetas con `'mkdir(parents=True, exist_ok = True)'`.
- »• Leer y escribir archivos de texto directamente con `'read_text()'` y `'write_text()'`.
- »• Recorrer directorios con `'iterdir()'`, `'glob()'` y `'rglob()'`.
- »• 'pathlib' es ****multiplataforma****: funciona igual en Windows, Linux y Mac.

» Reflexion final

» Mirá, 'pathlib' es una de esas cosas que cuando las descubris no entiendes como viviste sin ellas. Las rutas como objetos son mucho mas claras, menos propensas a errores, y el codigo queda mucho mas legible. Ademas, te olvidas de si la barra es `'/'` o `'\'`, porque Python se encarga de todo.

»Mi recomendacion es que a partir de ahora, uses 'pathlib' para cualquier proyecto nuevo. Los modulos viejos como 'os.path' van a seguir funcionando, pero 'pathlib' es el futuro.

»SeguÃn practicando con los ejercicios. Modificalos, prueba cosas distintas, rompe y arregla. Es la unica forma de aprender de verdad.

»_____ *Las rutas son caminos. pathlib es el
GPS.*

»Modulo shutil: Copiar, mover y borrar archivos/carpetas como un pro

»Querido lector: os no alcanza para todo

»Con el modulo 'os' podemos crear, renombrar, borrar archivos y carpetas vacias. Pero ¿y si queremos copiar un archivo? ¿lo mover un arbol entero de carpetas? ¿lo borrar una carpeta llena de archivos sin tener que vaciarla primero? Yo recuerdo una vez que quise copiar una carpeta con cientos de fotos y termine escribiendo un script de 50 lineas con 'os.listdir()' y 'shutil.copy()' dentro de un bucle. Un desastre.

»Despu s descubri el modulo 'shutil'. Es como el hermano mayor de 'os', especializado en operaciones de alto nivel con archivos y directorios. En criollo: 'shutil' es una navaja suiza para manejar archivos y carpetas enteras de una sola vez.

»En este capitulo te voy a mostrar todo lo que puedes hacer con 'shutil'. Y como siempre, al final tenes 20 ejercicios para practicar.

»Importar shutil

»Como cualquier modulo, lo importas asi:

```
»>>> import shutil
```

»Copiar archivos

»El primer problema que tuve fue copiar un archivo. Con 'os' tenias que abrir el origen en modo lectura, crear el destino en modo escritura, leer bloque por bloque... un bodrio. Con 'shutil' es una linea.

»copy() - copia contenido y permisos

»'shutil.copy()' copia el contenido y los permisos del archivo (modo, etc.).

```
»>>> shutil.copy("origen.txt", "destino.txt")
»'destino.txt'
```

»Si el destino es una carpeta, copia el archivo con el mismo nombre dentro de ella:

```
»»» shutil.copy("origen.txt", "carpeta_destino/")
»'carpeta_destino/origen.txt'
```

»copy2() - copia tambien metadatos (fecha de modificacion, etc.)

»'copy2()' es como 'copy()' pero ademas preserva la fecha de ultima modificacion y otros metadatos.

```
»»» shutil.copy2("origen.txt", "destino_metadata.txt")
»'destino_metadata.txt'
```

»copyfile() - solo el contenido (mas rapido)

»Si solo te importa el contenido y no los permisos ni metadatos, 'copyfile()' es mas rapida.

```
»»» shutil.copyfile("origen.txt", "destino_simple.txt")
»'destino_simple.txt'
```

»Mover archivos y carpetas

»Mover es como copiar y luego borrar el original. 'shutil.move()' lo hace por vos.

```
»»» shutil.move("origen.txt", "nueva_ubicacion/origen.txt")
»'nueva_ubicacion/origen.txt'
»
»»» # Mover a una carpeta (conserva el nombre)
»»» shutil.move("origen.txt", "nueva_carpeta/")
»'nueva_carpeta/origen.txt'
»
»»» # Tambien mueve carpetas enteras
»»» shutil.move("carpeta_vieja", "carpeta_nueva/")
»'carpeta_nueva/carpeta_vieja'
```

»Copiar carpetas enteras

»Aca esta la magia: 'copytree()' copia una carpeta y todo su contenido (archivos, subcarpetas, etc.) de forma recursiva.

```
»»» shutil.copytree("carpeta_origen", "carpeta_destino")
»'carpeta_destino'
```

»Ignorar ciertos archivos

»Podes decirle que ignore archivos o carpetas que coincidan con un patron.

```
»>>> shutil.copytree("origen", "destino",
»...                 ignore=shutil.ignore_patterns("*.tmp", "*.log", "__pycache__")
»'destino')
```

»Eliminar carpetas con contenido

»Con 'os.rmdir()' solo podes borrar carpetas vacias. Para borrar una carpeta llena de cosas, necesitas 'shutil.rmtree()'.

```
»>>> shutil.rmtree("carpeta_a_borrar")
```

»**¡Cuidado!** 'rmtree()' no va a la papelera de reciclaje. Se borra para siempre. No hay confirmacion. Usalo con mucho respeto.

»Manejar errores al borrar

»Podes ignorar errores o pasar una funcion para manejarlos.

```
»>>> shutil.rmtree("carpeta", ignore_errors=True)
»
»>>> def manejar_error(func, path, exc_info):
»...     print(f"Error en {path}: {exc_info}")
»>>> shutil.rmtree("carpeta", onerror=manejar_error)
```

»Otras utilidades muy utiles

»disk_usage() – espacioendisco

»Te dice cuanto espacio total, usado y libre hay en una particion.

```
»>>> uso = shutil.disk_usage("/")
»>>> print(f"Total: {uso.total / 1024**3:.2f} GB")
»Total: 500.00 GB
»>>> print(f"Usado: {uso.used / 1024**3:.2f} GB")
»Usado: 250.00 GB
»>>> print(f"Libre: {uso.free / 1024**3:.2f} GB")
»Libre: 250.00 GB
```

» **which()** - encontrar la ruta de un ejecutable

» Es como el comando 'which' en Linux o 'where' en Windows. Busca un ejecutable en el PATH del sistema.

```
»>>> python_path = shutil.which("python")
»>>> print(python_path)
»/usr/bin/python
```

» **make_archive()** – crear un archivo zip/tar

» Comprime una carpeta entera en un solo archivo.

```
»>>> shutil.make_archive("mi_backup", "zip", "carpeta_a_comprimir")
»'mi_backup.zip'
```

» Los formatos soportados dependen del sistema, pero normalmente 'zip', 'tar', 'gztar', 'bztar', 'xztar'.

» **unpack_archive()** – descomprimir

» La operacion inversa.

```
»>>> shutil.unpack_archive("mi_backup.zip", "carpeta_destino")
```

» **get_archive_formats()** – listar formatos soportados

```
»>>> print(shutil.get_archive_formats())
»[('bztar', "bzip2'ed tar-file"), ('gztar', "gzip'ed tar-file"), ...]
```

» **Ejemplos practicos**

» **Backup automatico con fecha**

» Una de las cosas que mas uso: backup de una carpeta con la fecha actual.

```
»import shutil
»from datetime import datetime
»
»origen = "/home/usuario/documentos"
»fecha = datetime.now().strftime("%Y%m%d_%H%M%S")
»nombre_backup = f"backup_{fecha}"
»shutil.make_archive(nombre_backup, "zip", origen)
»print(f"Backup creado: {nombre_backup}.zip")
```

»Sincronizar dos carpetas (copiar solo lo que falta)

»A partir de Python 3.8, 'copytree()' tiene 'dirs_exist_ok = True' que permite copiar una carpeta sobre otra ya existente.

```
»def sincronizar(origen, destino):  
»    shutil.copytree(origen, destino, dirs_exist_ok=True)
```

»Organizar archivos por extension (version con shutil)

»Ya viste una version con 'pathlib' y 'shutil.move'. Aca va una completa:

```
»from pathlib import Path  
»import shutil  
»  
»carpeta = Path(".")  
»for archivo in carpeta.glob("*"):  
»    if archivo.is_file():  
»        ext = archivo.suffix[1:] # sin el punto  
»        if ext:  
»            destino = carpeta / ext  
»            destino.mkdir(exist_ok=True)  
»            shutil.move(str(archivo), str(destino / archivo.name))  
»            print(f"Movido {archivo.name} a {ext}/")
```

»20 ejercicios con soluciones paso a paso

»Ahora te toca a vos. Hace cada ejercicio por tu cuenta antes de mirar la solucion.

»Ejercicio 1

»Copiar un archivo "a.txt" a "b.txt".

»Solucion:

```
»import shutil  
»shutil.copy("a.txt", "b.txt")
```

»Explicacion: 'shutil.copy()' copia el contenido y los permisos basicos.

»Ejercicio 2

»Copiar un archivo a una carpeta (conservando el nombre).

»Solucion:

```
»shutil.copy("archivo.txt", "copia/")
```

»Explicacion: Si el destino es una carpeta, 'copy()' coloca el archivo dentro con el mismo nombre.

»Ejercicio 3

»Mover un archivo de “datos.txt” a “backup/datos.txt”.

»Solucion:

```
»shutil.move("datos.txt", "backup/datos.txt")
```

»Explicacion: ‘move()’ cambia la ubicacion del archivo.

»Ejercicio 4

»Copiar una carpeta entera “proyecto” a “proyecto_backup”.

»Solucion:

```
»shutil.copytree("proyecto", "proyecto_backup")
```

»Explicacion: ‘copytree()’ copia recursivamente todo el contenido.

»Ejercicio 5

»Eliminar una carpeta llena (sin vaciarla primero).

»Solucion:

```
»shutil.rmtree("carpeta_vieja")
```

»Explicacion: ‘rmtree()’ borra la carpeta y todo su contenido. Usar con cuidado.

»Ejercicio 6

»Crear un archivo zip de la carpeta “documentos”.

»Solucion:

```
»shutil.make_archive("documentos_backup", "zip", "documentos")
```

»Explicacion: El primer argumento es el nombre sin extension. El segundo es el formato.

»Ejercicio 7

»Mostrar el espacio libre en disco de la unidad actual (en GB).

»Solucion:

```
»uso = shutil.disk_usage(".")
»print(f"Libre: {uso.free / 1024**3:.2f} GB")
```

»Explicacion: ‘disk_usage()’ devuelve una tupla con total, usado y libre en bytes.

»Ejercicio 8

»Encontrar la ruta del ejecutable “python”.

»Solucion:

```
»ruta = shutil.which("python")
»print(ruta)
```

»Explicacion: ‘which()’ busca en el PATH del sistema.

»Ejercicio 9

»Copiar un archivo preservando metadatos (fecha de modificacion, permisos).

»Solucion:

```
»shutil.copy2("original.txt", "copia_metadata.txt")
```

»Explicacion: ‘copy2()’ es como ‘copy()’ pero ademas copia los metadatos.

»Ejercicio 10

»Mover una carpeta entera a otra ubicacion.

»Solucion:

```
»shutil.move("carpeta_origen", "destino/carpeta")
```

»Explicacion: ‘move()’ funciona tanto para archivos como para carpetas.

»Ejercicio 11

»Al copiar una carpeta, ignorar todos los archivos ‘.tmp’.

»Solucion:

```
»shutil.copytree("origen", "destino", ignore=shutil.ignore_patterns("*.tmp"))
```

»Explicacion: ‘ignore_patterns()’ crea una funcion que ignora los patrones especificados.

»Ejercicio 12

»Descomprimir un archivo zip.

»Solucion:

```
»shutil.unpack_archive("backup.zip", "extraccion")
```

»Explicacion: ‘unpack_archive()’ detecta automaticamente el formato.

»Ejercicio 13

»Crear un backup de la carpeta actual con fecha en el nombre.

»Solucion:

```
»from datetime import datetime
»fecha = datetime.now().strftime("%Y%m%d")
»shutil.make_archive(f"backup_{fecha}", "zip", ".")
```

»**Explicacion:** La fecha en formato YYYYMMDD evita problemas con caracteres especiales.

»Ejercicio 14

»Usar 'copyfile()' para copiar solo el contenido (sin permisos).

»Solucion:

```
»shutil.copyfile("a.txt", "b.txt")
```

»**Explicacion:** 'copyfile()' es mas rapida pero solo copia el contenido, no permisos ni metadatos.

»Ejercicio 15

»Mover todos los archivos '.txt' de una carpeta a otra.

»Solucion:

```
»from pathlib import Path
»import shutil
»
»destino = Path("txts")
»destino.mkdir(exist_ok=True)
»for archivo in Path(".").glob("*.txt"):
»    shutil.move(str(archivo), str(destino / archivo.name))
```

»**Explicacion:** Usamos 'pathlib' para encontrar los '.txt' y 'shutil.move' para moverlos.

»Ejercicio 16

»Copiar una carpeta, y si ya existe, sobrescribir usando 'dirs_exist_ok=True'.

»Solucion:

```
»shutil.copytree("origen", "destino", dirs_exist_ok=True)
```

»**Explicacion:** A partir de Python 3.8, 'dirs_exist_ok = True' permite copiar sobre una carpeta existente.

»Ejercicio 17

»Eliminar una carpeta y si hay error, ignorarlo.

»Solucion:

```
»shutil.rmtree("carpeta", ignore_errors=True)
```

»Explicacion: `ignore_errors = True` hace que no se detenga si hay problemas de permisos, etc.

»Ejercicio 18

»Crear un archivo tar (en lugar de zip).

»Solucion:

```
»shutil.make_archive("backup", "tar", "carpeta")
```

»Explicacion: Cambiando `"zip"` por `"tar"` obtienes un archivo tar sin comprimir.

»Ejercicio 19

»Obtener la lista de formatos de archivo soportados para compresion.

»Solucion:

```
»print(shutil.get_archive_formats())
```

»Explicacion: Devuelve una lista de tuplas con el nombre del formato y una descripcion.

»Ejercicio 20

»Realizar una copia de seguridad completa de una carpeta, y luego comprimirla en un zip usando un directorio temporal.

»Solucion:

```
»import shutil, tempfile, os
»with tempfile.TemporaryDirectory() as tmp:
»    shutil.copytree("mi_carpeta", os.path.join(tmp, "copia"))
»    shutil.make_archive("backup_final", "zip", tmp)
»print("Backup creado")
```

»Explicacion: Creamos un directorio temporal, copiamos la carpeta ahi, y luego comprimimos ese directorio temporal.

»»»Tabla de referencia rapida

Funcion	Que hace
'shutil.copy(src, dst)'	Copia archivo (contenido + permisos)
'shutil.copy2(src, dst)'	Copia archivo + metadatos (fechas, etc.)
'shutil.copyfile(src, dst)'	Copia solo el contenido (mas rapido)
'shutil.move(src, dst)'	Mueve archivo o carpeta
'shutil.copytree(src, dst)'	Copia carpeta recursivamente
'shutil.rmtree(path)'	Elimina carpeta con todo su contenido (PELIGROSO)
'shutil.make_archive(base, format, root)'	Crea archivo zip/tar de una carpeta
'shutil.unpack_archive(archive, extract_dir)'	Descomprime un archivo
'shutil.disk_usage(path)'	Espacio en disco (total, usado, libre)
'shutil.which(cmd)'	Ruta de un ejecutable en el PATH
'shutil.ignore_patterns(*patterns)'	Para usar con 'copytree', ignora archivos

»»»Resumen de lo que vimos

»»»Dale, repasemos:

- 'shutil.copy()', 'shutil.copy2()', 'shutil.copyfile()': distintas formas de copiar archivos.
- 'shutil.move()': mover (cortar y pegar) archivos o carpetas.
- 'shutil.copytree()': copiar carpetas enteras de forma recursiva.
- 'shutil.rmtree()': eliminar carpetas con todo su contenido (cuidado).
- 'shutil.make_archive()' y 'shutil.unpack_archive()': *comprimir y descomprimir*.
- 'shutil.disk_usage()': *saber cuanto espacio libre te queda*.
- 'shutil.which()': encontrar donde esta un programa.

»»»Reflexion final

»»»MirÃ¡, 'shutil' es la navaja suiza para operaciones con archivos. Copiar, mover, comprimir, descomprimir, todo con una linea. Te va a ahorrar muchisimo tiempo y lineas de codigo. Pero ojo con '.rmtree()': no va a la papelera, no pregunta confirmacion. Es definitivo. Usalo con respeto.

»»»A partir de ahora, cuando necesites copiar o mover algo, piensa primero en 'shutil'. Vas a ver que rapido te olvidas de hacerlo a mano.

»»»SeguÃ¡ practicando con los ejercicios. Proba hacer backups automaticos, o clonar proyectos completos. La mejor forma de aprender es usarlo en proyectos reales.

»»» _____ Con grandes poderes vienen grandes responsabilidades.

»'''Modulo collections: Counter, defaultdict, deque

»'''Querido lector: las estructuras de datos avanzadas

»'''Cuando yo empecé a programar, siempre usaba las mismas herramientas: listas, diccionarios, y algún que otro conjunto. Hasta que un día me tocó procesar un archivo enorme con miles de palabras y necesitaba saber cuáles eran las más frecuentes. Hacerlo a mano con diccionarios normales funcionaba, pero era mucho código.

»'''Después descubrí el módulo `collections`. Tiene tres clases que te cambian la vida: **Counter**, **defaultdict** y **deque**. Son como versiones mejoradas de las estructuras básicas, pero con superpoderes.

»'''En este capítulo te voy a contar para qué sirve cada una, con ejemplos que puedes probar vos mismo. Y al final, como siempre, 20 ejercicios para que practiques.

»'''Importar collections

»'''Para usar estas herramientas, solo tenés que importarlas. No hace falta instalar nada, vienen con Python.

```
»'''>>> from collections import Counter, defaultdict, deque
```

»'''Counter: contador de frecuencias

»'''Imaginate que tenés una lista y querés saber cuántas veces aparece cada elemento. Con un diccionario normal harías:

```
»'''>>> lista = ["a", "b", "a", "c", "b", "a"]
»'''>>> contador = {}
»'''>>> for elem in lista:
»'''...     contador[elem] = contador.get(elem, 0) + 1
»'''...
»'''>>> print(contador)
»'''{'a': 3, 'b': 2, 'c': 1}
```

»'''Funciona, pero es mucho código. **Counter** hace todo eso en una sola línea.

»»» Crear un Counter

```
»»»>>> from collections import Counter
»»»>>> contador = Counter(["a", "b", "a", "c", "b", "a"])
»»»>>> print(contador)
»»»Counter({'a': 3, 'b': 2, 'c': 1})
```

»»» También puedes crear un Counter a partir de un string (cuenta letras):

```
»»»>>> contador = Counter("abracadabra")
»»»>>> print(contador)
»»»Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
```

»»» Metodos utiles de Counter

```
»»»>>> # Los mas comunes
»»»>>> contador.most_common(2)
»»»[('a', 5), ('b', 2)]
»»»
»»»>>> # Sumar dos counters
»»»>>> c1 = Counter(a=3, b=1)
»»»>>> c2 = Counter(a=1, b=2)
»»»>>> print(c1 + c2)
»»»Counter({'a': 4, 'b': 3})
»»»
»»»>>> # Restar (solo positivos)
»»»>>> print(c1 - c2)
»»»Counter({'a': 2})
»»»
»»»>>> # Total de elementos
»»»>>> sum(contador.values())
»»»11
```

»»» Ejemplo practico: contar palabras en un texto

```
»»»>>> texto = "hola mundo hola python mundo hola"
»»»>>> palabras = texto.split()
»»»>>> contador = Counter(palabras)
»»»>>> print(contador.most_common())
»»»[('hola', 3), ('mundo', 2), ('python', 1)]
```

»»» defaultdict: diccionario con valor por defecto

»»» ¿Te paso alguna vez que intentaste acceder a una clave de un diccionario y Python te tiro un `KeyError`? Con `defaultdict` eso no pasa. Si la clave no existe, la crea con un valor por defecto.

»»» Crear defaultdict

»»» Le tienes que pasar una función que devuelva el valor por defecto. Las más comunes son `int` (devuelve 0), `list` (devuelve lista vacía), `set` (devuelve set vacío).

```
»»»>>> from collections import defaultdict
»»»
»»»>>> # Con int: valor por defecto 0
»»»>>> dd_int = defaultdict(int)
»»»>>> dd_int["a"] += 1
»»»>>> print(dd_int["a"])
»»»1
»»»>>> print(dd_int["b"])    # no existe, pero devuelve 0
»»»0
»»»
»»»>>> # Con list: valor por defecto []
»»»>>> dd_list = defaultdict(list)
»»»>>> dd_list["frutas"].append("manzana")
»»»>>> print(dd_list["frutas"])
»»»['manzana']
»»»
»»»>>> # Con set: valor por defecto set()
»»»>>> dd_set = defaultdict(set)
»»»>>> dd_set["numeros"].add(1)
»»»>>> print(dd_set["numeros"])
»»»{1}
```

»»» Ejemplo práctico: agrupar elementos

»»» Imagínate que tienes una lista de palabras y quieres agruparlas por su primera letra.

```
»»»>>> palabras = ["auto", "avion", "casa", "arbol", "coche"]
»»»>>> agrupado = defaultdict(list)
»»»>>> for p in palabras:
»»»...     agrupado[p[0]].append(p)
»»»...
»»»>>> print(dict(agrupado))
»»»{'a': ['auto', 'avion', 'arbol'], 'c': ['casa', 'coche']}
```

»»» deque: cola de dos extremos

»»» Una **deque** (double-ended queue) es como una lista, pero optimizada para agregar o quitar elementos por ambos extremos. Si necesitas una cola (FIFO) o una pila (LIFO), **deque** es mucho más rápida que una lista.

»»»Crear deque

```
»»»>>> from collections import deque
»»»>>> d = deque([1, 2, 3])
»»»>>> print(d)
»»»deque([1, 2, 3])
```

»»»Operaciones basicas

```
»»»>>> # Agregar al final (como append de lista)
»»»>>> d.append(4)
»»»>>> print(d)
»»»deque([1, 2, 3, 4])
»»»
»»»>>> # Agregar al principio
»»»>>> d.appendleft(0)
»»»>>> print(d)
»»»deque([0, 1, 2, 3, 4])
»»»
»»»>>> # Quitar del final
»»»>>> ultimo = d.pop()
»»»>>> print(ultimo)
»»»4
»»»>>> print(d)
»»»deque([0, 1, 2, 3])
»»»
»»»>>> # Quitar del principio
»»»>>> primero = d.popleft()
»»»>>> print(primero)
»»»0
»»»>>> print(d)
»»»deque([1, 2, 3])
```

»»»Operaciones adicionales

```
»»»>>> # Extender por la derecha (agregar varios)
»»»>>> d.extend([4, 5])
»»»>>> print(d)
»»»deque([1, 2, 3, 4, 5])
»»»
»»»>>> # Extender por la izquierda (cuidado: el orden se invierte)
»»»>>> d.extendleft([-2, -1])
»»»>>> print(d)
»»»deque([-1, -2, 1, 2, 3, 4, 5])
»»»
»»»>>> # Rotar: mueve los ultimos N al principio
»»»>>> d.rotate(2)
```



```

»'»»» print(d)    # [4, 5, -1, -2, 1, 2, 3] (los ultimos dos van al frente)
»'»»» deque([4, 5, -1, -2, 1, 2, 3])
»'»'
»'»»» # Rotar a la izquierda con numero negativo
»'»»» d.rotate(-2)
»'»»» print(d)    # vuelve a como estaba
»'»»» deque([-1, -2, 1, 2, 3, 4, 5])

```

»'»'Limitar el tamaño máximo

»'»'Si creas una deque con maxlen, cuando supera esa longitud, se descartan los elementos del extremo opuesto por donde agregas.

```

»'»»» d_lim = deque(maxlen=3)
»'»»» for i in range(5):
»'»...     d_lim.append(i)
»'»...     print(d_lim)
»'»...
»'»»» deque([0], maxlen=3)
»'»»» deque([0, 1], maxlen=3)
»'»»» deque([0, 1, 2], maxlen=3)
»'»»» deque([1, 2, 3], maxlen=3)    # se fue el 0
»'»»» deque([2, 3, 4], maxlen=3)    # se fue el 1

```

»'»'Ejemplo practico: historial de ultimas N acciones

```

»'»»» historial = deque(maxlen=5)
»'»»» for accion in range(10):
»'»...     historial.append(f"accion_{accion}")
»'»...
»'»»» print(list(historial))
»'»'['accion_5', 'accion_6', 'accion_7', 'accion_8', 'accion_9']

```

»'»'Ejemplos practicos combinados

»'»'Contar frecuencias de letras con defaultdict

```

»'»»» texto = "abracadabra"
»'»»» frecuencias = defaultdict(int)
»'»»» for letra in texto:
»'»...     frecuencias[letra] += 1
»'»...
»'»»» print(dict(frecuencias))
»'»'{'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}

```

»»Cola FIFO con deque

```
»»»>>> cola = deque()
»»»>>> cola.append("tarea1")
»»»>>> cola.append("tarea2")
»»»>>> cola.append("tarea3")
»»»>>> while cola:
»»...     print(cola.popleft())
»»...
»»tarea1
»»tarea2
»»tarea3
```

»»Pila LIFO con deque

```
»»»>>> pila = deque()
»»»>>> pila.append("libro1")
»»»>>> pila.append("libro2")
»»»>>> print(pila.pop())
»»libro2
»»»>>> print(pila.pop())
»»libro1
```

»»20 ejercicios con soluciones paso a paso

»»Ahora te toca a vos. Intentá resolverlos antes de mirar la solución.

»»Ejercicio 1

»»Usar Counter para contar las vocales en la palabra "programacion".

»»Solucion:

```
»»»from collections import Counter
»»»texto = "programacion"
»»»vocales = [c for c in texto if c in "aeiou"]
»»»contador = Counter(vocales)
»»»print(contador)
```

»»Explicacion: Filtramos las letras que son vocales con una comprensión de listas, luego creamos el Counter.

»»Ejercicio 2

»»Encontrar la palabra mas comun en la frase .^{el} perro y el gato son amigos del perro".

»»Solucion:

```
»'''frase = "el perro y el gato son amigos del perro"
»'''palabras = frase.split()
»'''contador = Counter(palabras)
»'''mas_comun = contador.most_common(1)
»'''print(mas_comun[0][0]) # 'el' o 'perro' (el que tenga mas)
```

»'''**Explicacion:** Dividimos la frase en palabras con `split()`, contamos con `Counter`, y `most_common(1)` devuelve una lista con la tupla (palabra, frecuencia).

»'''Ejercicio 3

»'''Usar `defaultdict` para agrupar numeros pares e impares de la lista [1,2,3,4,5,6].

»'''**Solucion:**

```
»'''from collections import defaultdict
»'''numeros = [1,2,3,4,5,6]
»'''agrupado = defaultdict(list)
»'''for n in numeros:
»'''    if n % 2 == 0:
»'''        agrupado["pares"].append(n)
»'''    else:
»'''        agrupado["impares"].append(n)
»'''print(dict(agrupado))
```

»'''**Explicacion:** Usamos una lista como valor por defecto. Para cada numero, lo agregamos a la clave correspondiente.

»'''Ejercicio 4

»'''Crear un deque con los numeros del 1 al 5 y agregar un 0 al principio.

»'''**Solucion:**

```
»'''from collections import deque
»'''d = deque([1,2,3,4,5])
»'''d.appendleft(0)
»'''print(d)
```

»'''**Explicacion:** `appendleft()` agrega al extremo izquierdo.

»'''Ejercicio 5

»'''Usar deque para implementar una cola que procese los elementos `.a`, `"b"`, `c`.^{en} orden.

»'''**Solucion:**

```
»'''cola = deque(["a","b","c"])
»'''while cola:
»'''    print(cola.popleft())
```

»'''**Explicacion:** `popleft()` saca del principio, manteniendo el orden FIFO.

»»Ejercicio 6

»»Usar Counter para contar las letras de un string "python".

»»Solucion:

```
»»texto = "python"
»»contador = Counter(texto)
»»print(contador)
```

»»Ejercicio 7

»»Usar defaultdict para crear un diccionario donde las claves son letras y los valores son listas de posiciones donde aparecen en "hola".

»»Solucion:

```
»»texto = "hola"
»»posiciones = defaultdict(list)
»»for i, letra in enumerate(texto):
»»    posiciones[letra].append(i)
»»print(dict(posiciones))
```

»»**Explicacion:** enumerate() devuelve (indice, letra). Agregamos el indice a la lista de la letra.

»»Ejercicio 8

»»Crear un deque con maxlen=3 y agregar los numeros del 0 al 4. Mostrar el resultado final.

»»Solucion:

```
»»d = deque(maxlen=3)
»»for i in range(5):
»»    d.append(i)
»»print(d) # deque([2, 3, 4], maxlen=3)
```

»»Ejercicio 9

»»Usar Counter para encontrar los 2 numeros mas frecuentes de [1,1,2,2,2,3,3,3,3].

»»Solucion:

```
»»numeros = [1,1,2,2,2,3,3,3,3]
»»contador = Counter(numeros)
»»print(contador.most_common(2))
```

»»Ejercicio 10

»»Usar deque para rotar una lista [1,2,3,4,5] dos posiciones a la derecha.

»»Solucion:

```
»»d = deque([1,2,3,4,5])
»»d.rotate(2)
»»print(list(d))  # [4,5,1,2,3]
```

»»Ejercicio 11

»»Sumar dos Counters: c1 = Counter(a=3, b=1) y c2 = Counter(a=1, b=2).

»»Solucion:

```
»»c1 = Counter(a=3, b=1)
»»c2 = Counter(a=1, b=2)
»»print(c1 + c2)  # Counter({'a':4, 'b':3})
```

»»Ejercicio 12

»»Usar defaultdict para contar frecuencias de [1,2,2,3,3,3] sin usar Counter.

»»Solucion:

```
»»lista = [1,2,2,3,3,3]
»»freq = defaultdict(int)
»»for x in lista:
»»    freq[x] += 1
»»print(dict(freq))
```

»»Ejercicio 13

»»Simular un buffer circular (ultimos 5 elementos) con deque(maxlen=5). Agregar numeros del 0 al 9 y mostrar en cada paso.

»»Solucion:

```
»»buffer = deque(maxlen=5)
»»for i in range(10):
»»    buffer.append(i)
»»    print(buffer)
```

»»Ejercicio 14

»»Extraer los dos elementos mas comunes de Counter(.^abracadabra") con sus frecuencias.

»»Solucion:

```
»»contador = Counter("abracadabra")
»»for elemento, frecuencia in contador.most_common(2):
»»    print(f"{elemento}: {frecuencia}")
```

»»Ejercicio 15

»»Implementar una pila (LIFO) usando deque y probar apilando y desapilando 1,2.

»»Solucion:

```
»»pila = deque()
»»pila.append(1)
»»pila.append(2)
»»print(pila.pop()) # 2
»»print(pila.pop()) # 1
```

»»Ejercicio 16

»»Convertir Counter(a=3, b=2) a un diccionario normal.

»»Solucion:

```
»»c = Counter(a=3, b=2)
»»d = dict(c)
»»print(d)
```

»»Ejercicio 17

• »»Usar defaultdict(list) para agrupar palabras [.^auto",.^avion",çasa",.^arbol",çoche"] por su primera letra.

»»Solucion:

```
»»palabras = ["auto","avion","casa","arbol","coche"]
»»agrupado = defaultdict(list)
»»for p in palabras:
»»    agrupado[p[0]].append(p)
»»print(dict(agrupado))
```

»»Ejercicio 18

»»Usar deque para implementar una cola donde una tarea urgente se puede agregar al principio.

»»Solucion:

```
»»cola = deque()
»»cola.append("tarea normal")
»»cola.appendleft("tarea urgente")
»»print(cola.popleft()) # urgente
»»print(cola.popleft()) # normal
```

»»Ejercicio 19

»»Encontrar la letra que mas aparece en el texto "hola mundo hola"(ignorando espacios).

»»Solucion:

```
»»texto = "hola mundo hola"
»»sin_espacios = texto.replace(" ", "")
»»contador = Counter(sin_espacios)
»»letra, freq = contador.most_common(1)[0]
»»print(f"Letra: {letra}, Frecuencia: {freq}")
```

»»Ejercicio 20

»»Simular un historial de navegacion con deque de maximo 5 paginas. Visitar "google", "python", "github", luego volver atras (pop) y mostrar historial.

»»Solucion:

```
»»historial = deque(maxlen=5)
»»def visitar(pagina):
»»    historial.append(pagina)
»»    print(f"Visitando: {pagina}")
»»
»»def atras():
»»    if historial:
»»        print(f"Volviendo atras desde {historial.pop()}")
»»
»»visitar("google.com")
»»visitar("python.org")
»»visitar("github.com")
»»atras()
»»print(historial)
```

»»»Tabla de referencia rapida

Clase	Principales operaciones
Counter	<code>most_common(n)</code> , <code>suma/resta</code> , <code>values()</code>
defaultdict	<code>defaultdict(int)</code> , <code>defaultdict(list)</code> , <code>defaultdict(set)</code>
deque	<code>appendleft()</code> , <code>popleft()</code> , <code>rotate()</code> , <code>maxlen</code>

»»»Resumen de lo que vimos

»»»Dale, repasemos:

- **Counter**: ideal para contar frecuencias. Con `most_common()` obtienes los mas repetidos.
- **defaultdict**: evita el `KeyError`. Muy util para agrupar o para acumular valores.
- **deque**: lista de dos extremos. Perfecta para colas (FIFO) y pilas (LIFO). Con `maxlen` puedes limitar el tamaño.

»»»Estas herramientas no las vas a usar todos los días, pero cuando las necesitas, te ahorran muchísimo código y dolores de cabeza.

»»»Reflexion final

»»»El módulo `collections` es como una caja de herramientas especializadas. No siempre necesitas un martillo, pero cuando tienes un clavo, es mejor que usar una piedra. Aprende estas tres clases: `Counter`, `defaultdict` y `deque`. Te van a sacar de apuros muchas veces.

»»»Seguía practicando con los ejercicios. Modifica los ejemplos, prueba con tus propios datos. La única forma de internalizarlos es usarlos.

»»»_____ *La herramienta correcta para el trabajo correcto.*

»»»Modulo itertools: Iteradores avanzados

»»»Querido lector: cuando los bucles normales no alcanzan

»»»¿Te paso alguna vez que necesitaste combinar varias listas de todas las formas posibles? ¿O generar todas las permutaciones de un conjunto? A mi si. Recuerdo una vez que queria probar todas las combinaciones de colores y talles para un sistema de ropa, y termine escribiendo cuatro bucles anidados. Un desastre.

»»»Despu s descubri el modulo `itertools`. Es como tener una navaja suiza para iteradores. Te permite hacer cosas increibles con muy poco codigo, de manera eficiente y elegante.

»»»En este capitulo te voy a mostrar las funciones mas utiles de `itertools`. No te asustes si al principio parecen abstractas: con los ejemplos y ejercicios vas a ver que son mas sencillas de lo que parecen.

»»»Importar itertools

```
»»»>>> import itertools
```

»»»Iteradores infinitos

»»»Estos iteradores nunca terminan. Por eso tenes que combinarlos con un `break` o usarlos dentro de un bucle con limite.

»»»`count(start, step)` - cuenta desde start hasta infinito

»»»El primer problema que tuve fue generar numeros sin tener que hacer un bucle `while` manual. `count()` es como un `range()` sin fin.

```
»»»>>> for i in itertools.count(5, 2):
»»»...     if i > 20:
»»»...         break
»»»...     print(i)
»»»...
```

```
»'5
»'7
»'9
»'11
»'13
»'15
»'17
»'19
```

»'cycle(iterable) - repite el iterable infinitamente

»'Cycle es como poner una lista en un loop eterno.

```
»'»» colores = itertools.cycle(["rojo", "verde", "azul"])
»'»» for i in range(7):
»'...     print(next(colores))
»'...
»'rojo
»'verde
»'azul
»'rojo
»'verde
»'azul
»'rojo
```

»'repeat(element, times) - repite un elemento N veces (o infinito)

»'Sirve para generar el mismo valor una y otra vez.

```
»'»» for x in itertools.repeat("hola", 3):
»'...     print(x)
»'...
»'hola
»'hola
»'hola
```

»'Iteradores combinatorios

»'Aca esta lo mas jugoso: combinaciones, permutaciones y producto cartesiano.

»'product(*iterables, repeat=1) - producto cartesiano

»'Si tienes dos listas y quieres todas las combinaciones posibles, `product()` es tu función. Es como un bucle for anidado pero en una línea.

```

>>> colores = ["rojo", "verde"]
>>> tamanios = ["chico", "grande"]
>>> for combinacion in itertools.product(colores, tamanios):
>>>     print(combinacion)
>>> ...
>>> ('rojo', 'chico')
>>> ('rojo', 'grande')
>>> ('verde', 'chico')
>>> ('verde', 'grande')

```

»»» **permutations(iterable, r) - permutaciones (el orden importa)**

»»» Si el orden importa, usas permutaciones. Por ejemplo, las contraseñas de dos dígitos con los números 1,2,3.

```

>>> for p in itertools.permutations([1,2,3], 2):
>>>     print(p)
>>> ...
>>> (1, 2)
>>> (1, 3)
>>> (2, 1)
>>> (2, 3)
>>> (3, 1)
>>> (3, 2)

```

»»» **combinations(iterable, r) - combinaciones (el orden no importa)**

»»» Si el orden no importa, usas combinaciones. Por ejemplo, los equipos de 2 personas que puedes formar con 3 personas.

```

>>> for c in itertools.combinations([1,2,3], 2):
>>>     print(c)
>>> ...
>>> (1, 2)
>>> (1, 3)
>>> (2, 3)

```

»»» **combinations_{with replacement} — combinaciones con repetición**

»»» Y si además puedes repetir elementos, esta es la variante.

```

>>> for c in itertools.combinations_with_replacement([1,2,3], 2):
>>>     print(c)
>>> ...
>>> (1, 1)
>>> (1, 2)

```

```

»'(1, 3)
»'(2, 2)
»'(2, 3)
»'(3, 3)

```

»'Iteradores de agrupacion y filtrado

»'Estos te ayudan a unir, filtrar o transformar iteradores.

»'chain(*iterables) - concatenar iterables

»'Es como pegar varias listas con +, pero sin crear una nueva lista (mas eficiente).

```

»'>>> resultado = list(itertools.chain([1,2], [3,4], [5,6]))
»'>>> print(resultado)
»'[1, 2, 3, 4, 5, 6]

```

»'zip_longest(*iterables, fillvalue = None) - zip con relleno

»'zip() se corta en el iterable mas corto. zip_longest() completa con un valor.

```

»'>>> a = [1,2]
»'>>> b = [3,4,5]
»'>>> for x,y in itertools.zip_longest(a, b, fillvalue=0):
»'...     print(x, y)
»'...
»'1 3
»'2 4
»'0 5

```

»'compress(data, selectors) - filtrar por selector booleano

»'Te quedas solo con los elementos de data donde el selector correspondiente es True.

```

»'>>> datos = ["a","b","c","d"]
»'>>> selectores = [True, False, True, False]
»'>>> resultado = list(itertools.compress(datos, selectores))
»'>>> print(resultado)
»'['a', 'c']

```

»»filterfalse(predicate, iterable) - lo contrario a filter

»»filter() devuelve los que cumplen la condicion; filterfalse() los que no.

```
»»>>> numeros = [1,2,3,4,5]
»»>>> impares = list(itertools.filterfalse(lambda x: x%2==0, numeros))
»»>>> print(impares)
»»[1, 3, 5]
```

»»dropwhile(predicate, iterable) - descarta mientras se cumple

»»Descarta elementos desde el principio mientras se cumpla la condicion. En cuanto falla, devuelve todo lo que queda (inclusive el que fallo).

```
»»>>> numeros = [1,2,3,4,1,2]
»»>>> resultado = list(itertools.dropwhile(lambda x: x<3, numeros))
»»>>> print(resultado)
»»[3, 4, 1, 2]
```

»»takewhile(predicate, iterable) - toma mientras se cumple

»»Al revés que dropwhile: toma elementos mientras se cumple la condicion, y corta en cuanto falla.

```
»»>>> resultado = list(itertools.takewhile(lambda x: x<3, numeros))
»»>>> print(resultado)
»»[1, 2]
```

»»Otros iteradores utiles

• »»islice(iterable, start, stop, step) - slice para iteradores

»»Así como list[start:stop:step] funciona con listas, islice() funciona con cualquier iterador.

```
»»>>> numeros = range(10)
»»>>> parte = list(itertools.islice(numeros, 2, 8, 2))
»»>>> print(parte)
»»[2, 4, 6]
```

»»starmap(function, iterable) - map con argumentos en tuplas

»»map() aplica una funcion a un solo argumento. starmap() aplica una funcion que espera varios argumentos, que vienen en tuplas.

```
»»>>> puntos = [(2,3), (4,5), (6,7)]
»»>>> sumas = list(itertools.starmap(lambda x,y: x+y, puntos))
»»>>> print(sumas)
»» [5, 9, 13]
```

»»tee(iterable, n) - crear n copias de un iterador

»»A veces necesitas recorrer el mismo iterador varias veces. tee() te da n copias independientes.

```
»»>>> original = [1,2,3]
»»>>> copia1, copia2 = itertools.tee(original, 2)
»»>>> print(list(copia1))
»» [1, 2, 3]
»»>>> print(list(copia2))
»» [1, 2, 3]
```

»»Ejemplos practicos

»»Generar todas las combinaciones de un menu

»»Imagina que tienes un menu de opciones y quieres mostrar todas las combinaciones posibles (sin repetir).

```
»»opciones = ["sumar", "restar", "multiplicar"]
»»for r in range(1, len(opciones)+1):
»»    for comb in itertools.combinations(opciones, r):
»»        print(comb)
```

»»Simular un reloj digital (horas, minutos, segundos)

»»Con product() puedes generar todas las combinaciones de horas, minutos y segundos.

```
»»horas = range(24)
»»minutos = range(60)
»»segundos = range(60)
»»for h, m, s in itertools.product(horas, minutos, segundos):
»»    print(f"{h:02d}:{m:02d}:{s:02d}")
»»    # cuidado: esto es infinito (24*60*60 = 86400 combinaciones)
»»    if h == 0 and m == 0 and s == 5:
»»        break # solo mostramos los primeros 6 segundos
```

»»20 ejercicios con soluciones paso a paso

»»Ahora te toca a vos. Hace cada ejercicio, trata de resolverlo solo, y despues mira la solucion.

»»Ejercicio 1

»»Usar `count` para generar los numeros pares desde 0 hasta 20.

»»**Solucion:**

```
»»import itertools
»»for i in itertools.count(0, 2):
»»    if i > 20:
»»        break
»»    print(i)
```

»»**Explicacion:** `count(0,2)` genera 0,2,4,... hasta que cortamos con el `if`.

»»Ejercicio 2

»»Usar `cycle` para iterar sobre una lista de 3 colores 5 veces.

»»**Solucion:**

```
»»colores = itertools.cycle(["rojo","verde","azul"])
»»for i in range(5):
»»    print(next(colores))
```

»»**Explicacion:** `cycle` genera una secuencia infinita. Con `next()` obtenemos el siguiente elemento.

»»Ejercicio 3

»»Usar `repeat` para imprimir "Python"4 veces.

»»**Solucion:**

```
»»for _ in itertools.repeat("Python", 4):
»»    print(_)
```

»»**Explicacion:** `repeat("Python",4)` genera cuatro veces el string.

»»Ejercicio 4

»»Generar el producto cartesiano de `[1,2]` y `['a','b']`.

»»**Solucion:**

```
»»list(itertools.product([1,2], ['a','b']))
```

»»**Explicacion:** Devuelve las 4 combinaciones: (1,'a'), (1,'b'), (2,'a'), (2,'b').

»»Ejercicio 5

»»Generar todas las permutaciones de [1,2,3] de tamaño 2.

»»Solucion:

```
list(itertools.permutations([1,2,3], 2))
```

»»Explicacion: Hay 6 permutaciones (3x2).

»»Ejercicio 6

»»Generar todas las combinaciones de [1,2,3] de tamaño 2.

»»Solucion:

```
list(itertools.combinations([1,2,3], 2))
```

»»Explicacion: Hay 3 combinaciones: (1,2),(1,3),(2,3).

»»Ejercicio 7

»»Concatenar dos listas [1,2] y [3,4] con chain.

»»Solucion:

```
list(itertools.chain([1,2], [3,4])) # [1,2,3,4]
```

»»Explicacion: Une los iterables en uno solo.

»»Ejercicio 8

»»Usar zip_longest para combinar [1,2] y [3,4,5] rellenando con 0.

»»Solucion:

```
list(itertools.zip_longest([1,2], [3,4,5], fillvalue=0))
```

»»Explicacion: Los pares son (1,3),(2,4),(0,5).

»»Ejercicio 9

»»Filtrar una lista con compress: datos=['a','b','c'], selectores=[True,False,True].

»»Solucion:

```
list(itertools.compress(['a','b','c'], [True,False,True])) # ['a','c']
```

»»Explicacion: Solo se quedan los elementos donde el selector es True.

»»Ejercicio 10

»»Usar `dropwhile` para descartar los primeros numeros menores a 3 de `[1,2,3,4,1,2]`.

»»**Solucion:**

```
»»list(itertools.dropwhile(lambda x: x<3, [1,2,3,4,1,2]))
```

»»**Explicacion:** Descarta el 1 y 2, cuando llega a 3 empieza a devolver todo (incluido el 1 y 2 finales).

»»Ejercicio 11

»»Usar `takewhile` para tomar mientras el numero sea menor a 3 de la misma lista.

»»**Solucion:**

```
»»list(itertools.takewhile(lambda x: x<3, [1,2,3,4,1,2]))
```

»»**Explicacion:** Toma 1 y 2, al llegar a 3 corta, no toma el resto.

»»Ejercicio 12

»»Usar `islice` para obtener los elementos en posiciones pares de `range(10)` (0,2,4,6,8).

»»**Solucion:**

```
»»list(itertools.islice(range(10), 0, 10, 2)) # [0,2,4,6,8]
```

»»**Explicacion:** Sintaxis similar a `[start:stop:step]`.

»»Ejercicio 13

»»Usar `starmap` para sumar los pares (1,2),(3,4).

»»**Solucion:**

```
»»list(itertools.starmap(lambda x,y: x+y, [(1,2),(3,4)])) # [3,7]
```

»»**Explicacion:** Toma cada tupla y desempaqueta sus elementos como argumentos de la funcion.

»»Ejercicio 14

»»Generar todas las combinaciones con repeticion de `[1,2]` de tamaño 2.

»»**Solucion:**

```
»»list(itertools.combinations_with_replacement([1,2], 2))
```

»»**Explicacion:** Devuelve (1,1),(1,2),(2,2).

»»Ejercicio 15

»»Usar `filterfalse` para obtener los numeros impares de `[1,2,3,4,5]`.

»»**Solucion:**

```
»»list(itertools.filterfalse(lambda x: x%2==0, [1,2,3,4,5]))
```

»»**Explicacion:** `filterfalse` es lo opuesto a `filter`: devuelve los que no cumplen la condicion.

»»Ejercicio 16

»»Crear dos copias de un iterador con `tee` a partir de `[1,2,3]`.

»»**Solucion:**

```
»»it = iter([1,2,3])
»»a, b = itertools.tee(it, 2)
»»print(list(a), list(b))
```

»»**Explicacion:** `tee` hace dos iteradores independientes que recorren la misma secuencia.

»»Ejercicio 17

»»Generar todas las combinaciones de 3 letras del alfabeto (usando las primeras 5 letras: abcde).

»»**Solucion:**

```
»»list(itertools.combinations("abcde", 3))
```

»»**Explicacion:** El string `"abcde"` se comporta como una secuencia de caracteres.

»»Ejercicio 18

»»Simular un ciclo de semaforo con `cycle`: rojo, amarillo, verde, repetir 3 veces.

»»**Solucion:**

```
»»semaforo = itertools.cycle(["rojo","amarillo","verde"])
»»for _ in range(9):
»»    print(next(semaforo))
```

»»**Explicacion:** 3 ciclos completos = 9 elementos.

»»Ejercicio 19

»»Crear un contador que empiece en 10 y vaya de a 3 hasta 30 (inclusive) usando `count`.

»»Solucion:

```

»»for i in itertools.count(10, 3):
»»    if i > 30:
»»        break
»»    print(i)

```

»»Explicacion: Se imprime 10,13,16,19,22,25,28.

»»Ejercicio 20

»»Calcular el producto cartesiano de una lista consigo misma (`repeat=2`) usando `product`.

»»Solucion:

```

»»list(itertools.product([1,2], repeat=2)) # (1,1),(1,2),(2,1),(2,2)

```

»»Explicacion: `repeat=2` es equivalente a `product([1,2], [1,2])`.

»»Resumen de lo que vimos

»»Dale, repasemos:

- **Iteradores infinitos:** `count()`, `cycle()`, `repeat()`. Cuidado con usarlos sin un limite.
- **Combinatorios:** `product()` (producto cartesiano), `permutations()` (orden importa), `combinations()` (orden no importa), `combinations_with_replacement()` (con repeticion).
- **Agrupacion y filtrado:** `chain()` (concatena), `zip_longest()` (zip con relleno), `compress()` (filtro por selector), `filterfalse()` (opuesto a `filter`), `dropwhile()` y `takewhile()` (filtro por condicion al principio).
- **Otros:** `islice()` (slice para iteradores), `starmap()` (map con desempaqueado), `tee()` (copias de iterador).

»»Reflexion final

»»MirÃ¡, `itertools` es un modulo que al principio parece abstracto, pero una vez que lo dominas, te permite escribir codigo muy expresivo y eficiente. No trates de memorizar todo; solo recuerda que existe y cuando te enfrentes a un problema de iteracion, revisa si `itertools` tiene una solucion. La mayoría de las veces, la tiene.

»»SeguÃ¡ practicando con los ejercicios. Proba crear tus propias combinaciones, o usa `cycle` para hacer un cronometro. Cuanto mas los uses, mas naturales se van a volver.

»'''_____ *Iterar es humano, iterar
eficientemente es divino.*

Manuel Villafañe - Python paso a paso

»»»Archivos CSV: Leer y escribir datos tabulares

»»»Querido lector: las tablas mas simples del mundo

»»»Cuando empecé a trabajar con datos, me encontré con archivos que tenían filas y columnas. Excel está bien, pero a veces solo quieres un archivo de texto simple que cualquier programa pueda leer. Ahí aparecen los archivos CSV.

»»»CSV significa “Comma Separated Values” (valores separados por comas). Es un formato de texto donde cada línea es una fila y los valores se separan con comas (u otro separador). Es como una tabla de Excel pero en texto plano.

»»»¿Que es un archivo CSV?

»»»Un archivo CSV típico se ve así:

```
»»»nombre,edad,ciudad
»»»Ana,25,Buenos Aires
»»»Luis,30,Cordoba
»»»Pedro,22,Rosario
```

»»»La primera línea suele ser el encabezado (los nombres de las columnas). Las líneas siguientes son los datos.

»»»Leer CSV con el modulo csv

»»»Python tiene un modulo csv que facilita todo esto. No tienes que instalar nada, ya viene con Python.

»»»Importar csv

```
»»»>>> import csv
```

»»Leer archivo CSV simple

»»El primer problema que tuve fue leer un CSV sin saber las columnas. Para eso usamos `csv.reader()`. Lee cada fila como una lista.

```
»»>>> import csv
»»>>> with open("datos.csv", newline='', encoding='utf-8') as archivo:
»»...     lector = csv.reader(archivo)
»»...     for fila in lector:
»»...         print(fila)
»»...
»»['nombre', 'edad', 'ciudad']
»»['Ana', '25', 'Buenos Aires']
»»['Luis', '30', 'Cordoba']
»»['Pedro', '22', 'Rosario']
```

»»**Importante:** Usamos `newline=''` para que los saltos de línea se manejen bien en todos los sistemas operativos (Windows, Linux, Mac). Y `encoding='utf-8'` para que los acentos y la Ñ se lean correctamente.

»»Leer con encabezados (DictReader)

»»Si tu archivo tiene encabezados, puedes leerlo como diccionarios. Es mucho mas comodo porque accedes a los valores por el nombre de la columna.

```
»»>>> with open("datos.csv", newline='', encoding='utf-8') as archivo:
»»...     lector = csv.DictReader(archivo)
»»...     for fila in lector:
»»...         print(fila['nombre'], fila['edad'], fila['ciudad'])
»»...
»»Ana 25 Buenos Aires
»»Luis 30 Cordoba
»»Pedro 22 Rosario
```

»»Especificar otro separador (punto y coma, etc.)

»»A veces los archivos CSV usan punto y coma en lugar de coma, especialmente en paises donde la coma es separador decimal. No te asustes, `csv.reader()` te deja cambiar el delimitador.

```
»»>>> with open("datos_ps.csv", newline='', encoding='utf-8') as archivo:
»»...     lector = csv.reader(archivo, delimiter=';')
»»...     for fila in lector:
»»...         print(fila)
```

»»Escribir archivos CSV

»»Escribir es igual de facil.

»»Escribir con csv.writer

»»Si tenes una lista de listas, podés escribirlas directamente.

```

>>> datos = [
>>>     ["nombre", "edad", "ciudad"],
>>>     ["Ana", 25, "Buenos Aires"],
>>>     ["Luis", 30, "Cordoba"],
>>>     ["Pedro", 22, "Rosario"]
>>> ]
>>> with open("salida.csv", "w", newline='', encoding='utf-8') as archivo:
>>>     escritor = csv.writer(archivo)
>>>     escritor.writerows(datos)

```

»»Escribir con DictWriter (con encabezados)

»»Si tenes una lista de diccionarios, DictWriter es tu amigo.

```

>>> datos = [
>>>     {"nombre": "Ana", "edad": 25, "ciudad": "Buenos Aires"},
>>>     {"nombre": "Luis", "edad": 30, "ciudad": "Cordoba"},
>>>     {"nombre": "Pedro", "edad": 22, "ciudad": "Rosario"}
>>> ]
>>> with open("salida_dict.csv", "w", newline='', encoding='utf-8') as archivo:
>>>     campos = ["nombre", "edad", "ciudad"]
>>>     escritor = csv.DictWriter(archivo, fieldnames=campos)
>>>     escritor.writeheader() # escribe la primera fila con los nombres
>>>     escritor.writerows(datos)

```

»»Manejo de problemas comunes

»»Valores con comas dentro (escapado)

»»¿Qué pasa si un valor tiene una coma? Por ejemplo: "Libro, el". CSV lo resuelve envolviendo ese valor entre comillas dobles. El modulo csv hace esto automaticamente.

```

>>> datos = [ ["nombre", "descripcion"], ["Libro, el", "Un libro interesante"] ]
>>> with open("escapado.csv", "w", newline='', encoding='utf-8') as f:
>>>     writer = csv.writer(f)
>>>     writer.writerows(datos)

```

»»Si abris escapado.csv con un editor, vas a ver:

```

>>> nombre,descripcion
>>> "Libro, el",Un libro interesante

```

»»Archivos con encoding diferente

»»Siempre especifica `encoding='utf-8'` para evitar problemas con acentos y caracteres especiales. Si recibis un archivo con otro encoding (por ejemplo `latin-1`), cambialo:

```
»»with open("archivo_latin.csv", newline='', encoding='latin-1') as f:
»»    lector = csv.reader(f)
»»    ...
```

»»Ejemplos practicos

»»Leer CSV y calcular promedios

»»Imaginate que tenes un archivo `notas.csv` con las columnas `nombre` y `nota`. Queres calcular el promedio.

```
»»>>> import csv
»»>>> with open("notas.csv", newline='', encoding='utf-8') as f:
»»...     lector = csv.DictReader(f)
»»...     suma = 0
»»...     contador = 0
»»...     for fila in lector:
»»...         suma += float(fila["nota"])
»»...         contador += 1
»»...     promedio = suma / contador if contador > 0 else 0
»»...     print(f"Promedio: {promedio:.2f}")
```

»»Filtrar datos y guardar en otro CSV

»»Queres quedarte solo con las personas mayores de 25 años.

```
»»>>> with open("datos.csv", "r", encoding='utf-8') as entrada:
»»...     lector = csv.DictReader(entrada)
»»...     with open("filtrados.csv", "w", newline='', encoding='utf-8') as salida:
»»...         campos = ["nombre", "edad", "ciudad"]
»»...         escritor = csv.DictWriter(salida, fieldnames=campos)
»»...         escritor.writeheader()
»»...         for fila in lector:
»»...             if int(fila["edad"]) > 25:
»»...                 escritor.writerow(fila)
```

»»20 ejercicios para practicar (con soluciones explicadas paso a paso)

»»Ahora te toca a vos. Cada ejercicio tiene su solucion abajo. Intentã resolverlo solo antes de mirar.

»»Ejercicio 1

»»Leer un archivo CSV simple (sin encabezados) y mostrar cada fila.

»»Solucion:

```
»»import csv
»»with open("datos.csv", newline='', encoding='utf-8') as f:
»»    lector = csv.reader(f)
»»    for fila in lector:
»»        print(fila)
```

»»**Explicacion:** Abrimos el archivo con `newline=''` y `encoding='utf-8'`. `csv.reader()` devuelve un iterador que entrega cada fila como lista de strings.

»»Ejercicio 2

»»Leer un archivo CSV con encabezados y mostrar la columna "nombre".

»»Solucion:

```
»»with open("datos.csv", newline='', encoding='utf-8') as f:
»»    lector = csv.DictReader(f)
»»    for fila in lector:
»»        print(fila["nombre"])
```

»»**Explicacion:** `DictReader` usa la primera fila como nombres de las columnas. Cada fila es un diccionario donde la clave es el nombre de la columna.

»»Ejercicio 3

»»Crear un archivo CSV con 3 filas de datos (nombre, edad, ciudad).

»»Solucion:

```
»»datos = [{"nombre","edad","ciudad"},
»»          ["Ana","25","Buenos Aires"],
»»          ["Luis","30","Cordoba"]]
»»with open("nuevo.csv", "w", newline='', encoding='utf-8') as f:
»»    writer = csv.writer(f)
»»    writer.writerows(datos)
```

»»**Explicacion:** `writer.writerows()` escribe todas las filas de una sola vez. Si quisieras una por una, usas `writer.writerow()`.

»»Ejercicio 4

»»Agregar una fila al final de un CSV existente.

»»Solucion:

```
»»with open("nuevo.csv", "a", newline='', encoding='utf-8') as f:
»»    writer = csv.writer(f)
»»    writer.writerow(["Pedro", "22", "Rosario"])
```

»»**Explicacion:** Abrimos en modo `.a` (append) para agregar sin borrar lo que ya habia.

»»Ejercicio 5

»»Leer un CSV y contar cuantas filas tiene (sin contar encabezado).

»»Solucion:

```
»»with open("datos.csv", newline='', encoding='utf-8') as f:
»»    lector = csv.DictReader(f)
»»    contador = sum(1 for _ in lector)
»»    print(f"Filas de datos: {contador}")
```

»»**Explicacion:** `sum(1 for _ in lector)` es una forma compacta de contar elementos de un iterable sin consumir mucha memoria.

»»Ejercicio 6

»»Leer un CSV y calcular el promedio de edades.

»»Solucion:

```
»»with open("datos.csv", newline='', encoding='utf-8') as f:
»»    lector = csv.DictReader(f)
»»    edades = [int(fila["edad"]) for fila in lector]
»»    promedio = sum(edades) / len(edades) if edades else 0
»»    print(f"Promedio de edad: {promedio:.2f}")
```

»»**Explicacion:** Guardamos todas las edades en una lista (convertidas a entero), luego sumamos y dividimos.

»»Ejercicio 7

»»Escribir un CSV usando DictWriter.

»»Solucion:

```

»'''datos = [{"nombre":"Ana","edad":25},{ "nombre":"Luis","edad":30}]
»'''with open("dict.csv", "w", newline='', encoding='utf-8') as f:
»'''    campos = ["nombre","edad"]
»'''    writer = csv.DictWriter(f, fieldnames=campos)
»'''    writer.writeheader()
»'''    writer.writerows(datos)

```

»'''**Explicacion:** fieldnames define el orden de las columnas. writeheader() escribe la fila de encabezados.

»'''Ejercicio 8

»'''Leer un CSV con separador punto y coma.

»'''**Solucion:**

```

»'''with open("datos_ps.csv", newline='', encoding='utf-8') as f:
»'''    lector = csv.reader(f, delimiter=',')
»'''    for fila in lector:
»'''        print(fila)

```

»'''**Explicacion:** El parametro delimiter cambia el caracter separador.

»'''Ejercicio 9

»'''Filtrar filas de un CSV donde la edad sea mayor a 25.

»'''**Solucion:**

```

»'''with open("datos.csv", "r", encoding='utf-8') as f_in:
»'''    lector = csv.DictReader(f_in)
»'''    with open("mayores_25.csv", "w", newline='', encoding='utf-8') as f_out:
»'''        writer = csv.DictWriter(f_out, fieldnames=lector.fieldnames)
»'''        writer.writeheader()
»'''        for fila in lector:
»'''            if int(fila["edad"]) > 25:
»'''                writer.writerow(fila)

```

»'''**Explicacion:** lector.fieldnames nos da los nombres de las columnas del archivo original.

»'''Ejercicio 10

»'''Leer un CSV y mostrar solo las primeras 5 filas (incluyendo encabezado).

»'''**Solucion:**

```
»'''with open("datos.csv", newline='', encoding='utf-8') as f:
»'''    lector = csv.reader(f)
»'''    for i, fila in enumerate(lector):
»'''        if i >= 5:
»'''            break
»'''        print(fila)
```

»'''**Explicacion:** enumerate() nos da el indice de cada fila. Cortamos cuando llegamos a 5.

»'''Ejercicio 11

»'''Contar cuantas veces aparece un valor en una columna (ej: ciudad "Buenos Aires").

»'''**Solucion:**

```
»'''with open("datos.csv", newline='', encoding='utf-8') as f:
»'''    lector = csv.DictReader(f)
»'''    cont = sum(1 for fila in lector if fila["ciudad"] == "Buenos Aires")
»'''    print(f"Buenos Aires aparece {cont} veces")
```

»'''**Explicacion:** Generador con condicional.

»'''Ejercicio 12

»'''Ordenar las filas de un CSV por edad (crear un nuevo CSV ordenado).

»'''**Solucion:**

```
»'''with open("datos.csv", "r", encoding='utf-8') as f:
»'''    lector = csv.DictReader(f)
»'''    filas = list(lector)
»'''    filas.sort(key=lambda x: int(x["edad"]))
»'''    with open("ordenado.csv", "w", newline='', encoding='utf-8') as f:
»'''        writer = csv.DictWriter(f, fieldnames=lector.fieldnames)
»'''        writer.writeheader()
»'''        writer.writerows(filas)
```

»'''**Explicacion:** Cargamos todas las filas en memoria, las ordenamos con sort() y una funcion lambda que extrae la edad como entero, luego escribimos.

»'''Ejercicio 13

»'''Leer un CSV sin encabezados y agregar encabezados al escribir un nuevo archivo.

»'''**Solucion:**

```

»»»with open("sin_encabezados.csv", "r", encoding='utf-8') as f_in:
»»»    lector = csv.reader(f_in)
»»»    with open("con_encabezados.csv", "w", newline='', encoding='utf-8') as f_out:
»»»        writer = csv.writer(f_out)
»»»        writer.writerow(["nombre","edad","ciudad"])
»»»        writer.writerows(lector)

```

»»»**Explicacion:** Escribimos primero la fila de encabezados, luego escribimos todas las filas originales.

»»»Ejercicio 14

»»»Crear un CSV a partir de una lista de diccionarios.

»»»**Solucion:**

```

»»»lista = [{"producto":"manzana","precio":100},
»»»          {"producto":"pera","precio":150}]
»»»with open("productos.csv", "w", newline='', encoding='utf-8') as f:
»»»    writer = csv.DictWriter(f, fieldnames=["producto","precio"])
»»»    writer.writeheader()
»»»    writer.writerows(lista)

```

»»»**Explicacion:** Similar al ejercicio 7.

»»»Ejercicio 15

»»»Leer un CSV y guardar solo una columna en otro archivo (lista).

»»»**Solucion:**

```

»»»with open("datos.csv", "r", encoding='utf-8') as f_in:
»»»    lector = csv.DictReader(f_in)
»»»    nombres = [fila["nombre"] for fila in lector]
»»»with open("nombres.txt", "w", encoding='utf-8') as f_out:
»»»    f_out.write("\n".join(nombres))

```

»»»**Explicacion:** Extraemos la columna “nombre” a una lista, luego la escribimos linea por linea en un archivo de texto.

»»»Ejercicio 16

»»»Actualizar un valor en un CSV (cambiar edad de Ana a 26).

»»»**Solucion:**

```

»'''with open("datos.csv", "r", encoding='utf-8') as f:
»'''    lector = csv.DictReader(f)
»'''    filas = list(lector)
»'''for fila in filas:
»'''    if fila["nombre"] == "Ana":
»'''        fila["edad"] = "26"
»'''with open("datos.csv", "w", newline='', encoding='utf-8') as f:
»'''    writer = csv.DictWriter(f, fieldnames=lector.fieldnames)
»'''    writer.writeheader()
»'''    writer.writerows(filas)

```

»'''**Explicacion:** Leemos todas las filas, modificamos la que corresponde, luego escribimos todo de vuelta (sobrescribiendo).

»'''Ejercicio 17

»'''Leer un CSV y eliminar filas duplicadas (basado en nombre).

»'''**Solucion:**

```

»'''vistos = set()
»'''with open("datos.csv", "r", encoding='utf-8') as f_in:
»'''    lector = csv.DictReader(f_in)
»'''    filas_unicas = []
»'''    for fila in lector:
»'''        if fila["nombre"] not in vistos:
»'''            vistos.add(fila["nombre"])
»'''            filas_unicas.append(fila)
»'''with open("sin_duplicados.csv", "w", newline='', encoding='utf-8') as f_out:
»'''    writer = csv.DictWriter(f_out, fieldnames=lector.fieldnames)
»'''    writer.writeheader()
»'''    writer.writerows(filas_unicas)

```

»'''**Explicacion:** Usamos un conjunto (**set**) para recordar los nombres que ya vimos. Si un nombre no esta en el conjunto, lo agregamos y guardamos la fila.

»'''Ejercicio 18

»'''Leer un CSV y calcular el producto mas caro (archivo de productos con columnas "producto" y "precio").

»'''**Solucion:**

```

»'''with open("productos.csv", newline='', encoding='utf-8') as f:
»'''    lector = csv.DictReader(f)
»'''    max_precio = -1
»'''    max_producto = ""
»'''    for fila in lector:

```

```

»'''      precio = float(fila["precio"])
»'''      if precio > max_precio:
»'''          max_precio = precio
»'''          max_producto = fila["producto"]
»'''      print(f"Producto mas caro: {max_producto} - ${max_precio}")

```

»'''**Explicacion:** Recorremos fila por fila, actualizando el maximo.

»'''Ejercicio 19

»'''Convertir un CSV a formato JSON (usando el modulo `json` que veremos despues, pero te lo muestro).

»'''**Solucion:**

```

»'''import json
»'''with open("datos.csv", "r", encoding='utf-8') as f:
»'''    lector = csv.DictReader(f)
»'''    datos = list(lector)
»'''with open("datos.json", "w", encoding='utf-8') as f:
»'''    json.dump(datos, f, indent=2, ensure_ascii=False)

```

»'''**Explicacion:** `DictReader` ya nos da una lista de diccionarios. `json.dump()` guarda esa estructura en un archivo JSON.

»'''Ejercicio 20

»'''Leer un CSV y generar un resumen: cantidad de filas, columnas, y primeros 3 registros.

»'''**Solucion:**

```

»'''with open("datos.csv", "r", encoding='utf-8') as f:
»'''    lector = csv.reader(f)
»'''    encabezados = next(lector)    # la primera fila
»'''    filas = list(lector)         # el resto
»'''print(f"Columnas: {encabezados}")
»'''print(f"Cantidad de filas de datos: {len(filas)}")
»'''print("Primeras 3 filas:")
»'''for i in range(min(3, len(filas))):
»'''    print(filas[i])

```

»'''**Explicacion:** `next(lector)` avanza al primer elemento del iterador (los encabezados). El resto se guarda en una lista con `list(lector)`.

»»Resumen de lo que vimos

»»Dale, repasemos rapido lo importante:

- **csv.reader()**: para leer CSV como listas. Ideal cuando no tenes encabezados.
- **csv.DictReader()**: para leer CSV como diccionarios. Los encabezados se convierten en claves.
- **csv.writer()**: para escribir listas en un CSV.
- **csv.DictWriter()**: para escribir diccionarios, con **writeheader()** para los nombres de columnas.
- **Parametros clave**: **newline=""** y **encoding='utf-8'** siempre.
- **delimiter**: para cambiar el separador (coma, punto y coma, tab, etc.).
- **Modos de apertura**: **r** para leer, **w** para escribir (sobrescribe), **a** para agregar.

»»Reflexion final

»»Los archivos CSV son como las hojas de calculo del programador. Son simples, portables, y cualquier herramienta los entiende: Excel, Google Sheets, bases de datos, R, Matlab, y por supuesto Python.

»»Dominar el modulo `csv` te va a servir para intercambiar datos con el mundo real. Vas a poder leer planillas que te mandan por mail, exportar resultados para que los analicen en Excel, y procesar grandes cantidades de datos tabulares sin complicaciones.

»»Recordá siempre usar **newline=""** y **encoding='utf-8'**. Eso te va a ahorrar dolores de cabeza con los saltos de linea y los acentos.

»»SeguÍ practica con los ejercicios. Hace tus propios archivos CSV, probá con distintos separadores, agregale valores con comas dentro. La unica forma de aprender es equivocandote y arreglando.

»» _____ *Los datos en columnas, el poder en tus manos.*

»'»Archivos JSON: El formato de intercambio de datos

»'»Querido lector: el lenguaje comun de las APIs

»'»Cuando yo arranque a programar, me costo un monton entender como hacian las aplicaciones para hablar entre ellas. ¿Como hace un programa en Python para mandarle datos a una pagina web? ¿Como guardo la configuracion de mi juego para que la proxima vez que lo abra recuerde el nivel y el puntaje?

»'»Un dia descubri JSON. Fue como encontrar un idioma que todos entienden. JSON es un formato de texto super simple que se parece mucho a los diccionarios y listas de Python. Por eso a nosotros, los pythonistas, nos resulta familiar.

»'»¿Que es JSON?

»'»En criollo, JSON es una forma de escribir datos que sea facil de leer para un humano y facil de parsear para una computadora. Viene de "JavaScript Object Notation", pero hoy se usa en casi todos los lenguajes.

»'»Una analogia: imaginate que tenes que anotar los datos de una persona en un papel. Podes escribirlo asi:

```
»'»nombre: Nahuel
»'»edad: 25
»'»ciudad: Buenos Aires
```

»'»JSON es parecido, pero con reglas mas estrictas. Se ve asi:

```
»'»{
»'»  "nombre": "Nahuel",
»'»  "edad": 25,
»'»  "ciudad": "Buenos Aires",
»'»  "hobbies": ["programar", "leer", "futbol"]
»'»}
```

»'»Las claves van entre comillas dobles, los strings tambien, los numeros sin comillas. Las listas entre corchetes. Los objetos (diccionarios) entre llaves. Sencillo, ¿no?

»»»El modulo json de Python

»»»Python tiene un modulo llamado `json` que viene con la instalacion. No tenes que instalar nada extra. Solo lo importas:

```
»»»>>> import json
```

»»»Con este modulo podés hacer dos cosas fundamentales:

- **Serializar:** convertir un diccionario o lista de Python a un string JSON.
- **Deserializar:** convertir un string JSON a un diccionario o lista de Python.

»»»Además, podés leer y escribir directamente en archivos.

»»»Convertir dict a JSON: serializar

»»»`json.dumps()` - a string

»»»La función `json.dumps()` recibe un objeto de Python (normalmente un diccionario o una lista) y devuelve un string con formato JSON.

```
»»»>>> import json
»»»>>> datos = {
»»»...     "nombre": "Nahuel",
»»»...     "edad": 25,
»»»...     "ciudad": "Buenos Aires"
»»»... }
»»»>>> json_string = json.dumps(datos)
»»»>>> print(json_string)
»»»{"nombre": "Nahuel", "edad": 25, "ciudad": "Buenos Aires"}
```

»»»Fijate que el resultado es un string. Las comillas simples de Python se transformaron en comillas dobles (como pide JSON). Los booleanos `True` y `False` se convierten en `true` y `false`. El `None` pasa a ser `null`.

»»»Dale formato: indent

»»»Por defecto, `dumps()` te da todo en una línea. Si querés que sea más legible (por ejemplo, para que un humano lo lea), usa el parametro `indent`.

```
»»»>>> print(json.dumps(datos, indent=2))
»»»{
»»»  "nombre": "Nahuel",
»»»  "edad": 25,
»»»  "ciudad": "Buenos Aires"
»»»}
```

»»»Con `indent=2` le decís que ponga dos espacios por nivel de anidación. Es mucho más fácil de leer.

»»» Guardar en archivo: `json.dump()`

»»» Si quieres guardar directamente en un archivo (sin pasar por un string), usas `json.dump()`. El primer argumento son los datos, el segundo es el archivo (abierto en modo escritura).

```
»»»>>> with open("datos.json", "w", encoding='utf-8') as archivo:
»»»...     json.dump(datos, archivo, indent=2, ensure_ascii=False)
```

»»» El parametro `ensure_ascii=False` lo explico mas adelante. Es para que los acentos se vean bien.

»»» Convertir JSON a dict: deserializar

»»» `json.loads()` - desde string

»»» Si tienes un string con formato JSON, lo conviertes a un diccionario o lista de Python con `json.loads()`.

```
»»»>>> json_string = '{"nombre": "Nahuel", "edad": 25}'
»»»>>> datos = json.loads(json_string)
»»»>>> print(datos["nombre"])
»»»Nahuel
»»»>>> print(type(datos))
»»»<class 'dict'>
```

»»» Es magico: de un texto estructurado recuperas tus datos.

»»» Leer desde archivo: `json.load()`

»»» Si tienes un archivo JSON, lo abris y usas `json.load()`.

```
»»»>>> with open("datos.json", "r", encoding='utf-8') as archivo:
»»»...     datos = json.load(archivo)
»»»...     print(datos)
»»»{'nombre': 'Nahuel', 'edad': 25, 'ciudad': 'Buenos Aires'}
```

»»» Tipos de datos compatibles

»»» MirÃ¡ esta tabla de equivalencias:

Python	JSON
dict	objeto (llaves con comillas dobles)
list, tuple	array (corchetes)
str	string (comillas dobles)
int, float	number
True	true
False	false
None	null

»»» ¿Que no se puede convertir directamente?

- Tuplas: se convierten a listas. Cuando leas, vas a obtener listas.
- Conjuntos (set): no existen en JSON. Tendrias que convertirlos a listas.
- Fechas (datetime): no estan soportadas. Tienes que convertirlas a string (por ejemplo con `.isoformat()`).
- Bytes: no. Puedes convertir a base64 o a string.

»»» Ejemplo con fecha:

```

>>> from datetime import datetime
>>> datos = {"fecha": datetime.now().isoformat()}
>>> print(json.dumps(datos, indent=2))
>>> {
>>>   "fecha": "2025-01-20T15:30:00.123456"
>>> }

```

»»» Problemas con acentos y caracteres especiales

»»» Por defecto, `json.dumps()` escapa los caracteres no ASCII. O sea, convierte la Ñ en `\u00f1`. Eso es feo de leer. Para evitarlo, usas `ensure_ascii=False`.

```

>>> datos = {"nombre": "Na\u0301hu\u0301"}
>>> print(json.dumps(datos))
>>> {"nombre": "Na\\u0301hu\\u0301"}
>>> print(json.dumps(datos, ensure_ascii=False))
>>> {"nombre": "Nahuel"}

```

»»» Cuando guardes en un archivo, asegurate de abrirlo con `encoding='utf-8'`.

»»Ejemplos practicos

»»Guardar configuracion de una app

```
»»config = {
»»    "tema": "oscuro",
»»    "idioma": "espanol",
»»    "notificaciones": True,
»»    "volumen": 0.8
»»}
»»with open("config.json", "w", encoding='utf-8') as f:
»»    json.dump(config, f, indent=2, ensure_ascii=False)
```

»»Cargar configuracion al iniciar (con valores por defecto)

```
»»try:
»»    with open("config.json", "r", encoding='utf-8') as f:
»»        config = json.load(f)
»»except FileNotFoundError:
»»    config = {"tema": "claro", "idioma": "ingles"}
```

»»Simular respuesta de una API

»»Muchas APIs devuelven JSON. Podes simularlo asi:

```
»»respuesta_api = '{"usuario": "nahuel", "puntos": 100, "activo": true}'
»»datos = json.loads(respuesta_api)
»»print(f"Bienvenido {datos['usuario']}, tenes {datos['puntos']} puntos.")
```

»»20 ejercicios para practicar (con soluciones explicadas paso a paso)

»»Ahora te toca a vos. Cada ejercicio tiene su solucion abajo. Intentã resolverlo solo antes de mirar.

»»Ejercicio 1

»»Converti el siguiente diccionario a un string JSON: {".a": 1, "b": 2}.

»»Solucion:

```
»»import json
»»dic = {"a": 1, "b": 2}
»»json_str = json.dumps(dic)
»»print(json_str) # {"a": 1, "b": 2}
```

»»»**Explicacion:** La funcion `dumps()` recibe el diccionario y devuelve un string con formato JSON. Las comillas simples se transforman en dobles.

»»»Ejercicio 2

»»»Dado el string JSON `'"nombre": "Ana", "edad": 25'`, convertilo a diccionario Python y mostrÃ¡ el valor de "nombre".

»»»**Solucion:**

```

»»»json_str = '{"nombre": "Ana", "edad": 25}'
»»»dic = json.loads(json_str)
»»»print(dic["nombre"]) # Ana

```

»»»**Explicacion:** `loads()` parsea el string y devuelve un diccionario. Despues accedemos a la clave como siempre.

»»»Ejercicio 3

»»»Guarda el diccionario `{"producto": "mouse", "precio": 1500}` en un archivo llamado `producto.json` con formato legible (`indent=2`).

»»»**Solucion:**

```

»»»datos = {"producto": "mouse", "precio": 1500}
»»»with open("producto.json", "w", encoding='utf-8') as f:
»»»    json.dump(datos, f, indent=2)

```

»»»**Explicacion:** Usamos `dump()` (sin la s) porque escribimos directamente al archivo. El parametro `indent=2` hace que el archivo sea legible.

»»»Ejercicio 4

»»»LeÃ¡ el archivo `producto.json` que creaste en el ejercicio anterior y mostrÃ¡ su contenido en pantalla.

»»»**Solucion:**

```

»»»with open("producto.json", "r", encoding='utf-8') as f:
»»»    datos = json.load(f)
»»»print(datos)

```

»»»**Explicacion:** `json.load()` lee el archivo y lo parsea automaticamente. El resultado es un diccionario de Python.

»»Ejercicio 5

»»Cargá el archivo `producto.json`, agregale una clave `"stock"` con valor 10, y volvé a guardarlo.

»»Solucion:

```

»»with open("producto.json", "r", encoding='utf-8') as f:
»»    datos = json.load(f)
»»datos["stock"] = 10
»»with open("producto.json", "w", encoding='utf-8') as f:
»»    json.dump(datos, f, indent=2)

```

»»**Explicacion:** Primero leemos, modificamos el diccionario en memoria, y luego escribimos de nuevo. Así se actualiza el archivo.

»»Ejercicio 6

»»Creá un string JSON con formato legible (`indent=4`) del diccionario `{"nombre": "Python", "version": 3}`.

»»Solucion:

```

»»datos = {"nombre": "Python", "version": 3}
»»json_str = json.dumps(datos, indent=4)
»»print(json_str)

```

»»**Explicacion:** El valor de `indent` controla la cantidad de espacios. Con 4 es más espaciado.

»»Ejercicio 7

»»Convertí la lista de diccionarios `[{"nombre": "Ana"`

`», "nombre": "Luis"`

`»}]` a JSON.

»»Solucion:

```

»»lista = [{"nombre": "Ana"}, {"nombre": "Luis"}]
»»json_str = json.dumps(lista, indent=2)
»»print(json_str)

```

»»**Explicacion:** JSON soporta listas de objetos. `dumps()` funciona igual con listas que con diccionarios.

»»Ejercicio 8

»»LeÃl un archivo JSON que contiene acentos (por ejemplo, "nombre": "Na0301hu0301") y mostralo correctamente (sin escapes).

»»Solucion:

```

»»with open("datos.json", "r", encoding='utf-8') as f:
»»    datos = json.load(f)
»»print(datos) # los acentos se ven bien

```

»»Explicacion: json.load() ya decodifica los escapes. Lo importante es abrir el archivo con encoding='utf-8'.

»»Ejercicio 9

»»EscribÃn un archivo JSON que contenga la palabra "Na0301hu0301"(Nahuel con acento) pero sin escapes, es decir, que el archivo guarde directamente "Nahuel".

»»Solucion:

```

»»datos = {"nombre": "Nahuel"}
»»with open("nombre.json", "w", encoding='utf-8') as f:
»»    json.dump(datos, f, ensure_ascii=False, indent=2)

```

»»Explicacion: ensure_ascii=False le dice a Python que no escape los caracteres no ASCII. Asi se guarda el texto legible.

»»Ejercicio 10

»»CreÃa un JSON que contenga una lista y un diccionario anidados. Por ejemplo: {."alumnos": [."na", "Luis"], "promedios": {."na": 8, "Luis": 7}}.

»»Solucion:

```

»»datos = {
»»    "alumnos": ["Ana", "Luis"],
»»    "promedios": {"Ana": 8, "Luis": 7}
»»}
»»print(json.dumps(datos, indent=2))

```

»»Explicacion: La estructura anidada se conserva perfectamente en JSON. Es muy comun en APIs.

»»»Ejercicio 11

»»»LeÃel el archivo del ejercicio anterior y mostrÃel el promedio de Ana.

»»»Solucion:

```

»»»with open("datos.json", "r", encoding='utf-8') as f:
»»»    datos = json.load(f)
»»»print(datos["promedios"]["Ana"])

```

»»»**Explicacion:** Accedes a los valores anidados igual que con cualquier diccionario de Python.

»»»Ejercicio 12

»»»SerializÃel un diccionario que contenga una fecha actual (usa `datetime.now()`) y guardalo en JSON. Pista: convertÃel la fecha a string con `.isoformat()`.

»»»Solucion:

```

»»»from datetime import datetime
»»»datos = {"fecha": datetime.now().isoformat()}
»»»json_str = json.dumps(datos, indent=2)
»»»print(json_str)

```

»»»**Explicacion:** Como JSON no entiende fechas, las guardamos como string en formato ISO. Despues al leer, podemos convertirlas de nuevo con `datetime.fromisoformat()`.

»»»Ejercicio 13

»»»EscribÃel una funcion `guardar_json(datos, nombre_archivo)` que guarde cualquier diccionario en un archivo JSON con indentacion.

»»»Solucion:

```

»»»def guardar_json(datos, nombre_archivo):
»»»    with open(nombre_archivo, "w", encoding='utf-8') as f:
»»»        json.dump(datos, f, indent=2, ensure_ascii=False)
»»»
»»»# Prueba
»»»guardar_json({"clave": "valor"}, "test.json")

```

»»»**Explicacion:** Encapsulamos la logica en una funcion reutilizable.

»»Ejercicio 14

»»LeÃl un archivo JSON que contiene una lista (por ejemplo, [1, 2, 3]) y recorrÃl sus elementos mostrÃndolos uno por uno.

»»Solucion:

```

»»with open("lista.json", "r", encoding='utf-8') as f:
»»    lista = json.load(f)
»»for elemento in lista:
»»    print(elemento)

```

»»Explicacion: load() devuelve una lista de Python. La recorres con un for normal.

»»Ejercicio 15

»»TenÃl dos archivos JSON: a.json con {"x": 1} y b.json con {"z": 2}. CombinÃllos en un solo diccionario y guardalo como combinado.json.

»»Solucion:

```

»»with open("a.json", "r", encoding='utf-8') as f1:
»»    datos1 = json.load(f1)
»»with open("b.json", "r", encoding='utf-8') as f2:
»»    datos2 = json.load(f2)
»»datos1.update(datos2) # fusiona los diccionarios
»»with open("combinado.json", "w", encoding='utf-8') as f:
»»    json.dump(datos1, f, indent=2)

```

»»Explicacion: update() agrega o reemplaza las claves del segundo diccionario en el primero.

»»Ejercicio 16

»»ValidÃa si el string '{"a":1}' es un JSON valido. HacÃl una funcion que devuelva True o False (capturando la excepcion).

»»Solucion:

```

»»def es_json_valido(texto):
»»    try:
»»        json.loads(texto)
»»        return True
»»    except json.JSONDecodeError:
»»        return False
»»
»»print(es_json_valido('{"a":1}')) # True
»»print(es_json_valido('{a:1}'))  # False (las claves deben ir entre comillas)

```

»»Explicacion: Intentamos parsear. Si tira JSONDecodeError, no es valido.

»»Ejercicio 17

»»CargÃ un archivo JSON y elimina una clave (por ejemplo, `clave_a_eliminar`) y guardÃ el resultado.

»»**Solucion:**

```

»»with open("datos.json", "r", encoding='utf-8') as f:
»»    datos = json.load(f)
»»datos.pop("clave_a_eliminar", None) # None evita error si no existe
»»with open("datos.json", "w", encoding='utf-8') as f:
»»    json.dump(datos, f, indent=2)

```

»»**Explicacion:** `pop()` elimina la clave. El segundo argumento evita que tire error si no existe.

»»Ejercicio 18

»»ConvertÃ un archivo CSV (con cabecera) a JSON. SuponÃ que el CSV tiene columnas "nombre" y "edad". UsÃ el modulo `csv`.

»»**Solucion:**

```

»»import csv, json
»»with open("datos.csv", "r", encoding='utf-8') as f_csv:
»»    lector = csv.DictReader(f_csv)
»»    datos = list(lector) # cada fila es un diccionario
»»with open("datos.json", "w", encoding='utf-8') as f_json:
»»    json.dump(datos, f_json, indent=2, ensure_ascii=False)

```

»»**Explicacion:** `csv.DictReader` lee cada fila como un diccionario. Lo convertimos a lista y luego a JSON.

»»Ejercicio 19

»»ConvertÃ un archivo JSON (que contiene una lista de objetos) a CSV. UsÃ el modulo `csv`.

»»**Solucion:**

```

»»with open("datos.json", "r", encoding='utf-8') as f:
»»    datos = json.load(f) # debe ser una lista de diccionarios
»»if datos:
»»    campos = datos[0].keys()
»»    with open("salida.csv", "w", newline='', encoding='utf-8') as f:
»»        writer = csv.DictWriter(f, fieldnames=campos)
»»        writer.writeheader()
»»        writer.writerows(datos)

```

»»**Explicacion:** Tomamos las claves del primer diccionario como cabeceras. Luego escribimos fila por fila.

»»Ejercicio 20

»»SimulÃ¡ una respuesta de una API que devuelve este JSON: `'{"status": ".ok", "data": [1,2,3]}'`. Parsealo y si `status` es `.ok`, mostrÃ¡ los datos.

»»Solucion:

```
»»respuesta = '{"status": "ok", "data": [1,2,3]}'
»»datos = json.loads(respuesta)
»»if datos["status"] == "ok":
»»    print("Datos recibidos:", datos["data"])
```

»»**Explicacion:** Es exactamente lo que pasa cuando trabajas con APIs reales. Parseas el JSON y actÃ¡as segun el contenido.

»»Resumen de lo que vimos

»»Dale, repasemos rapido lo importante:

- `json.dumps()`: de diccionario a string JSON. Usa `indent` para formato.
- `json.dump()`: de diccionario a archivo JSON.
- `json.loads()`: de string JSON a diccionario.
- `json.load()`: de archivo JSON a diccionario.
- `ensure_ascii=False`: para que no escape los acentos y la Ã.
- **Fechas**: no son soportadas directamente; convertilas a string con `.isoformat()`.
- **Tuplas y sets**: se convierten a listas.
- **Errores comunes**: olvidarse de las comillas dobles en las claves, o usar comillas simples. JSON exige dobles.

»»Reflexion final

»»MirÃ¡, JSON esta por todos lados. Cada vez que una app se comunica con un servidor, cada vez que guardas las preferencias de un programa, cada vez que intercambias datos entre Python y JavaScript, ahi esta JSON. Aprender a usarlo te va a abrir muchas puertas.

»»Es sencillo, liviano, legible. No tiene las complicaciones del XML. Es el rey de las APIs modernas.

»»Ahora que sabes como manejarlo, podes hacer cosas como:

- Guardar el estado de un juego.
- Consumir APIs de clima, noticias, etc.

- Compartir datos entre aplicaciones.
- Hacer backups de configuraciones.

»»»SeguÃn practicando con los ejercicios. No te quedes solo con leer las soluciones. EscribÃn el codigo, rompelos, arreglalo. Asi se aprende.

»»»_____ *JSON: el pegamento que une a las*
aplicaciones.

»»»Entornos virtuales (venv): Por que y como aislar tus proyectos

»»»Querido lector: el problema de las dependencias

»»»Cuando yo arranque a programar, me paso algo que te va a sonar familiar. Tenia un proyecto que usaba una libreria llamada 'requests' en su version 1.0. Todo andaba joya. Despues empeece otro proyecto que necesitaba la version 2.0 de 'requests'. Instale la nueva version y ¡pum! el primer proyecto se rompio. ¿La razon? Los cambios entre versiones no eran compatibles.

»»»Tambien me paso de instalar tantas librerias "por si acaso" que al final no sabia cuales eran realmente necesarias. Cuando queria compartir mi codigo con un amigo, el tenia que adivinar que instalar.

»»»La solucion se llama **entorno virtual**. En criollo: un entorno virtual es una carpeta aislada que tiene su propia instalacion de Python y sus propias librerias. Cada proyecto tiene su propio "mundito" sin que se peleen entre si.

»»»¿Que es un entorno virtual?

»»»Una analogia: imaginate que vivis en una casa compartida. Cada persona tiene su propia habitacion. En tu habitacion podes poner tu musica, tus libros, tus cosas. No molesta a los demas. Los entornos virtuales son como esas habitaciones: cada proyecto tiene la suya, con sus propias librerias y hasta su propia version de Python.

»»»La estructura tipica es:

```
»»»proyecto_A/  
»»»    venv/           # entorno virtual de A  
»»»    requirements.txt  
»»»proyecto_B/  
»»»    venv/           # entorno virtual de B  
»»»    requirements.txt
```

»»»Crear un entorno virtual (venv)

»»»Desde Python 3.3, el modulo `venv` viene incluido. No hace falta instalar nada extra.

»»»Comando basico

»»»Abre una terminal en la carpeta de tu proyecto y ejecuta:

```
»»»python -m venv venv
```

»»»Este comando crea una carpeta llamada **venv** (podes usar cualquier nombre, pero **venv** o **.venv** son los mas comunes). Dentro de esa carpeta esta todo: el interprete de Python, pip, y el espacio para instalar librerias.

»»»Si tenes varias versiones de Python

»»»En algunos sistemas (especialmente Linux), el comando **python** puede apuntar a Python 2. Para asegurarte de usar Python 3, usa:

```
»»»python3 -m venv venv
```

»»»Activar el entorno

»»»Aca viene la parte que mas confusion me genero al principio: activar el entorno es diferente en Windows y en Linux/Mac.

»»»En Linux o Mac:

```
»»»source venv/bin/activate
```

»»»En Windows (cmd o PowerShell):

```
»»»venv\Scripts\activate
```

»»»Verificar que se activo correctamente

»»»Cuando el entorno esta activado, el prompt de la terminal cambia y muestra el nombre del entorno entre parentesis:

```
»»»(venv) usuario@computadora:~/mi_proyecto$
```

»»»Ademas, si ejecutas:

```
»»»which python    # Linux/Mac
»»»# o
»»»where python    # Windows
```

»»»La ruta debe apuntar a la carpeta **venv** (por ejemplo, **/home/usuario/mi_proyecto/venv/bin/python**)

»»»Instalar librerías en el entorno

»»»Una vez activado, usas `pip` como siempre. Pero ahora las librerías se instalan dentro de la carpeta `venv`, no en el sistema global.

```
»»»pip install requests
»»»pip install pandas
```

»»»Ver que librerías estan instaladas

```
»»»pip list
```

»»»Guardar las dependencias (requirements.txt)

»»»Esto es clave. El archivo `requirements.txt` lista todas las librerías y sus versiones exactas. Asi cualquiera puede recrear tu entorno.

```
»»»pip freeze > requirements.txt
```

»»»El archivo generado se ve algo asi:

```
»»»certifi==2024.2.2
»»»charset-normalizer==3.3.2
»»»idna==3.6
»»»requests==2.31.0
»»»urllib3==2.2.1
```

»»»Instalar desde requirements.txt

»»»Si alguien clona tu proyecto, solo necesita:

```
»»»pip install -r requirements.txt
```

»»»Desactivar el entorno

»»»Cuando terminas de trabajar, salis del entorno con:

```
»»»deactivate
```

»»»El prompt vuelve a la normalidad.

»»»Ejemplo completo paso a paso

»»»Vamos a hacer un ejemplo real desde cero.

»»»Paso 1: Crear carpeta del proyecto y entorno

```
»»»mkdir mi_proyecto
»»»cd mi_proyecto
»»»python -m venv venv
```

»»»Paso 2: Activar entorno

```
»»»source venv/bin/activate # Linux/Mac
»»»# o venv\Scripts\activate en Windows
```

»»»Paso 3: Instalar librerías

```
»»»pip install requests
»»»pip install beautifulsoup4
```

»»»Paso 4: Escribir un script simple

»»»Crea un archivo script.py:

```
»»»import requests
»»»from bs4 import BeautifulSoup
»»»
»»»print("Entorno funcionando")
»»»print(f"Requests version: {requests.__version__}")
```

»»»Paso 5: Guardar dependencias

```
»»»pip freeze > requirements.txt
```

»»»Paso 6: Desactivar entorno

```
»»»deactivate
```

»»»Paso 7: Recrear el entorno en otra maquina (simulado)

»»»Si alguien mas quisiera correr tu proyecto:

```
»»»git clone <tu_repo>
»»»cd mi_proyecto
»»»python -m venv venv
»»»source venv/bin/activate
»»»pip install -r requirements.txt
»»»python script.py
```

»»» Usar entornos virtuales en VS Code

»»» Si usas VS Code (lo recomiendo), el editor detecta automaticamente el entorno virtual. Solo tenes que abrir la carpeta del proyecto y, si hay una carpeta `venv`, VS Code te va a preguntar si quieres usarla como interprete.

»»» En la parte inferior derecha, podes ver y cambiar el interprete. Selecciona `venv/bin/python` (Linux/Mac) o `venv.exe` (Windows).

»»» Buenas practicas

»»» Con los años, aprendi estas reglas:

- **Nunca instales librerias globalmente** (a menos que sea pip mismo). Cada proyecto merece su propio entorno.
- **No subas la carpeta venv al repositorio.** Es enorme y varia segun el sistema operativo. En su lugar, subi `requirements.txt`.
- **Siempre inclui un requirements.txt actualizado.** Es la receta para cocinar tu entorno.
- **Nombra el entorno venv o .venv.** Es una convencion que muchos editores y herramientas reconocen.
- **Activa el entorno antes de instalar cualquier cosa.** Es facil olvidarse y terminar instalando globalmente.

»»».gitignore para entornos virtuales

»»» Si usas Git, agrega estas lineas a tu archivo `.gitignore`:

```

»»» # Python
»»» venv/
»»» env/
»»» .env/
»»» __pycache__/_
»»» *.pyc
»»» *.pyo
»»» *.so
    
```

»»» 20 ejercicios para practicar (con soluciones explicadas paso a paso)

»»» Ahora te toca a vos. Hace cada ejercicio en tu propia maquina. No hay mejor manera de aprender que haciendo.

»»»Ejercicio 1

»»»Crear un entorno virtual llamado “mi_entorno”.

»»»Solucion:

```
»»»python -m venv mi_entorno
```

»»»**Explicacion:** El comando `python -m venv` crea la carpeta con toda la estructura.

»»»Ejercicio 2

»»»Activar el entorno recién creado (en Linux/Mac).

»»»Solucion:

```
»»»source mi_entorno/bin/activate
```

»»»**Explicacion:** `source` ejecuta el script de activación en el shell actual.

»»»Ejercicio 3

»»»Activar el entorno en Windows (cmd).

»»»Solucion:

```
»»»mi_entorno\Scripts\activate
```

»»»**Explicacion:** En Windows el script está en `Scripts` en lugar de `bin`.

»»»Ejercicio 4

»»»Verificar que el entorno está activado.

»»»Solucion:

```
»»»# Mirar el prompt: debe aparecer (mi_entorno)
»»»# O ejecutar: which python (Linux/Mac) o where python (Windows)
»»»# La ruta debe apuntar a mi_entorno/bin/python
```

»»»Ejercicio 5

»»»Instalar la librería “requests” dentro del entorno activado.

»»»Solucion:

```
»»»pip install requests
```

»»»**Explicacion:** `pip` instala dentro del entorno activo, no globalmente.

»»Ejercicio 6

»»Listar las librerías instaladas en el entorno.

»»Solucion:

```
»»pip list
```

»»Explicacion: `pip list` muestra todas las librerías del entorno actual.

»»Ejercicio 7

»»Generar un archivo `requirements.txt` con las dependencias actuales.

»»Solucion:

```
»»pip freeze > requirements.txt
```

»»Explicacion: `pip freeze` muestra las versiones exactas; `>` dirige la salida al archivo.

»»Ejercicio 8

»»Desactivar el entorno.

»»Solucion:

```
»»deactivate
```

»»Explicacion: El comando `deactivate` sale del entorno.

»»Ejercicio 9

»»Crear un nuevo entorno, instalar `pandas`, y generar `requirements.txt`.

»»Solucion:

```
»»python -m venv pandas_env
»»source pandas_env/bin/activate # o .\pandas_env\Scripts\activate
»»pip install pandas
»»pip freeze > requirements.txt
»»deactivate
```

»»Ejercicio 10

»»Clonar un proyecto que tiene `requirements.txt` y recrear su entorno.

»»Solucion:

```
»»git clone <url_del_proyecto>
»»cd proyecto
»»python -m venv venv
»»source venv/bin/activate
»»pip install -r requirements.txt
```

»»Ejercicio 11

»»Verificar la version de Python dentro del entorno.

»»Solucion:

```
»»python --version
```

»»Ejercicio 12

»»Eliminar un entorno virtual (borrar la carpeta).

»»Solucion:

```
»»# Linux/Mac
»»rm -rf mi_entorno
»»
»»# Windows (cmd)
»»rmdir /s mi_entorno
```

»»Ejercicio 13

»»Ejecutar un script de Python dentro del entorno sin activarlo manualmente (usando la ruta completa).

»»Solucion:

```
»»# Linux/Mac
»»./venv/bin/python script.py
»»
»»# Windows
»».\venv\Scripts\python.exe script.py
```

»»Ejercicio 14

»»Instalar una version especifica de una libreria (ej: requests==2.25.1).

»»Solucion:

```
»»pip install requests==2.25.1
```

»»Ejercicio 15

»»Actualizar pip dentro del entorno.

»»Solucion:

```
»»pip install --upgrade pip
```

»»»Ejercicio 16

»»»Crear un entorno que use Python 3.8 en especifico (suponiendo que lo tenes instalado).

»»»Solucion:

```
»»python3.8 -m venv venv38
```

»»»Ejercicio 17

»»»Mostrar la ruta del interprete actual (para verificar que estas en el entorno).

»»»Solucion:

```
»»»# Linux/Mac
»»»which python
»»»
»»»# Windows
»»»where python
```

»»»Ejercicio 18

»»»Crear un alias en tu shell para activar el entorno rapidamente (opcional). Escribi el comando para agregar al .bashrc.

»»»Solucion:

```
»»»echo "alias activate='source venv/bin/activate'" >> ~/.bashrc
»»»source ~/.bashrc
```

»»»Ejercicio 19

»»»Exportar solo las librerias principales (sin las dependencias internas como pkg-resources) usando grep.

»»»Solucion:

```
»»»pip freeze | grep -v "pkg-resources" > requirements.txt
```

»»»Ejercicio 20

»»»Crear un script bash que automaticamente cree el entorno, active, instale dependencias y ejecute el programa principal.

»»»Solucion:

```

»'''#!/bin/bash
»'''# setup.sh
»'''python -m venv venv
»'''source venv/bin/activate
»'''pip install -r requirements.txt
»'''python main.py
»'''deactivate
    
```

»'''Darle permisos de ejecucion: `chmod +x setup.sh`.

»'''Tabla de referencia rapida

Accion	Comando
Crear entorno	<code>python -m venv nombre</code>
Activar (Linux/Mac)	<code>source nombre/bin/activate</code>
Activar (Windows)	<code>nombre\Scripts\activate</code>
»''' Instalar paquete	<code>pip install paquete</code>
Listar paquetes	<code>pip list</code>
Guardar dependencias	<code>pip freeze >requirements.txt</code>
Instalar desde archivo	<code>pip install -r requirements.txt</code>
Desactivar	<code>deactivate</code>

»'''Resumen de lo que vimos

»'''Dale, repasemos lo importante:

- Un **entorno virtual** aísla las dependencias de cada proyecto.
- Se crea con `python -m venv nombre_del_entorno`.
- Se **activa** con `source nombre/bin/activate` (Linux/Mac) o `nombre\Scripts\activate` (Windows).
- Una vez activado, `pip` instala dentro del entorno.
- `pip freeze >requirements.txt` guarda la lista de dependencias.
- Con `pip install -r requirements.txt` cualquiera puede recrear el entorno.
- Para salir usas `deactivate`.
- **Buena practica:** nunca subas la carpeta `venv` al repositorio, solo `requirements.txt`.

»»»Reflexion final

»»»MirÃa, al principio los entornos virtuales parecen un tramite molesto. Uno dice "Ã¶para que tanto quilombo si puedo instalar todo global?"Pero despues de que se te rompe un proyecto por actualizar una libreria, o cuando quieres compartir tu codigo y nadie sabe que instalar, te das cuenta de que son indispensables.

»»»Hoy en dia, cada proyecto que empiezo, lo primero que hago es crear su entorno. Es como lavarse los dientes: al principio cuesta crear el habito, pero despues no podes vivir sin eso.

»»»Hacete un favor: empieza a usarlos desde ahora. Tu yo del futuro te lo va a agradecer.

»»»_____ *Cada proyecto, su propio mundo.*

Cada mundo, sus propias reglas.

Manejo de excepciones avanzado - Crear tus propias excepciones

Querido lector: cuando las excepciones de Python no son suficientes

Python tiene muchas excepciones integradas: 'ValueError', 'TypeError', 'IndexError', 'FileNotFoundError', etc. Son geniales para errores generales. Pero a veces quieres que tu programa lance un error mas especifico de tu dominio. Algo que le diga al que usa tu codigo exactamente que paso, con un nombre que tenga sentido en tu problema.

Por ejemplo, si estas haciendo un sistema de cuentas bancarias, podrias tener una excepcion 'SaldoInsuficienteError'. En un juego, 'SinVidasError'. En un validador de formularios, 'EmailInvalidoError'. La buena noticia es que crear tus propias excepciones es facil y hace tu codigo mas claro, mas legible y mas facil de depurar.

Como crear una excepcion personalizada

La receta es simple: creas una clase que herede de 'Exception' (o de alguna otra excepcion mas especifica). No necesita metodos, solo el 'pass'. Pero puedes personalizarla cuanto quieras.

```
>>> class MiErrorPersonalizado(Exception):
...     pass
...
>>> raise MiErrorPersonalizado("Algo salio mal")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
__main__.MiErrorPersonalizado: Algo salio mal
```

Ejemplo practico: SaldoInsuficienteError

Vamos a armar un mini sistema bancario para verlo en accion.

```
class SaldoInsuficienteError(Exception):
    """Se lanza cuando el saldo es insuficiente para un retiro"""
    pass

def retirar(saldo, monto):
    if monto > saldo:
        raise SaldoInsuficienteError(f"No hay suficiente saldo. Saldo: {saldo}, Monto: {monto}")
```

```
return saldo - monto
```

```
try:
    nuevo_saldo = retirar(100, 150)
except SaldoInsuficienteError as e:
    print(f"Error: {e}")
```

Fijate que ‘raise’ lanza la excepcion con un mensaje personalizado. Despues la capturas como cualquier otra excepcion.

Agregar informacion adicional a tu excepcion

A veces no basta con un mensaje de texto. Queres guardar datos como el saldo actual, el monto intentado, etc. Para eso defines el metodo ‘__init__’.

```
class SaldoInsuficienteError(Exception):
    def __init__(self, saldo, monto):
        self.saldo = saldo
        self.monto = monto
        super().__init__(f"Saldo insuficiente: saldo ${saldo}, intento de retiro ${monto}")

def retirar(saldo, monto):
    if monto > saldo:
        raise SaldoInsuficienteError(saldo, monto)
    return saldo - monto
```

```
try:
    retirar(100, 150)
except SaldoInsuficienteError as e:
    print(f"Error: {e}")
    print(f"Saldo actual: {e.saldo}")
    print(f"Monto solicitado: {e.monto}")
```

Ahora en el bloque ‘except’ podes acceder a ‘e.saldo’ y ‘e.monto’ para tomar decisiones o loguear informacion extra.

Crear una jerarquia de excepciones

En proyectos grandes, conviene tener una excepcion base para todo un modulo, y luego excepciones mas especificas que hereden de ella.

```
class ErrorBanco(Exception):
    """Excepcion base para el modulo bancario"""
    pass

class SaldoInsuficienteError(ErrorBanco):
    pass
```

```
class CuentaBloqueadaError(ErrorBanco):  
    pass
```

```
class MontoInvalidoError(ErrorBanco):  
    pass
```

Ventajas: - Podes capturar 'ErrorBanco' para atrapar cualquier error de ese modulo. - Podes capturar una especifica si quieres manejar cada caso distinto.

Excepciones en contextos reales

Validacion de formularios

Imagina que tenes un formulario de registro y quieres validar la edad.

```
class ValidacionError(Exception):  
    pass
```

```
class EdadInvalidaError(ValidacionError):  
    def __init__(self, edad):  
        self.edad = edad  
        super().__init__(f"Edad invalida: {edad}. Debe ser entre 0 y 150.")
```

```
def validar_edad(edad):  
    if not isinstance(edad, int):  
        raise ValidacionError("La edad debe ser un numero entero")  
    if edad < 0 or edad > 150:  
        raise EdadInvalidaError(edad)  
    return True
```

```
try:  
    validar_edad(-5)  
except EdadInvalidaError as e:  
    print(e)          # Edad invalida: -5. Debe ser entre 0 y 150.  
    print(e.edad)     # -5  
except ValidacionError as e:  
    print(f"Error generico: {e}")
```

Juego: sin vidas

En un juego, cuando te quedas sin vidas, quieres lanzar una excepcion que detenga la partida.

```
class SinVidasError(Exception):  
    pass
```

```
class Jugador:  
    def __init__(self, vidas=3):  
        self.vidas = vidas
```

```

def perder_vida(self):
    self.vidas -= 1
    if self.vidas < 0:
        raise SinVidasError("Game Over - No te quedan vidas")
    print(f"Te quedan {self.vidas} vidas")

j = Jugador(1)
try:
    j.perder_vida()    # queda en 0
    j.perder_vida()    # esto lanza SinVidasError
except SinVidasError as e:
    print(e)

```

Usar 'raise ... from' para encadenar excepciones

A veces, mientras manejas un error, quieres lanzar otro error que explique mejor el contexto. Con 'raise ... from ...' encadenas las excepciones.

```

def leer_entero(archivo):
    try:
        with open(archivo, "r") as f:
            return int(f.read().strip())
    except ValueError as e:
        raise ValueError(f"El archivo {archivo} no contiene un entero valido") from e
    except FileNotFoundError as e:
        raise FileNotFoundError(f"No se encontro el archivo {archivo}") from e

try:
    leer_entero("datos.txt")
except ValueError as e:
    print(e)
    print(f"Causa original: {e.__cause__}")

```

Buenas practicas

- Hereda siempre de 'Exception', no de 'BaseException' (salvo que sepas muy bien que haces).
- Documenta tus excepciones con un docstring.
- Dale nombres claros que terminen en 'Error' (por convencion).
- No abuses: si una excepcion se usa una sola vez en un lugar muy especifico, quizas no vale la pena.
- En una libreria, crea una excepcion base (ej: 'MiLibreriaError') y que todas hereden de ella.

20 ejercicios con soluciones paso a paso

Ahora te toca a vos. Practica creando tus propias excepciones.

Ejercicio 1

Crear una excepcion 'ValorNegativoError'.

Solucion:

```
class ValorNegativoError(Exception):  
    pass
```

Explicacion: Es una clase vacia que hereda de 'Exception'.

Ejercicio 2

Crear una funcion que reciba un numero y lance 'ValorNegativoError' si es negativo.

Solucion:

```
def validar_positivo(n):  
    if n < 0:  
        raise ValorNegativoError(f"Valor negativo no permitido: {n}")  
    return n
```

Explicacion: Usamos 'raise' con el mensaje.

Ejercicio 3

Crear excepcion 'NumeroParError' y lanzarla si un numero es par.

Solucion:

```
class NumeroParError(Exception):  
    pass  
  
def solo_impares(n):  
    if n % 2 == 0:  
        raise NumeroParError(f"Numero par no permitido: {n}")
```

Ejercicio 4

Crear excepcion 'LongitudInvalidaError' para validar que un string tenga una longitud entre min y max.

Solucion:

```
class LongitudInvalidaError(Exception):  
    pass  
  
def validar_longitud(texto, min_len, max_len):  
    if not (min_len <= len(texto) <= max_len):  
        raise LongitudInvalidaError(f"Longitud {len(texto)} fuera de rango [{min_len}, {max_len}]")
```

Ejercicio 5

Crear excepcion 'EmailInvalidoError' y usarla para validar que un email tenga '@' y un punto.

Solucion:

```
class EmailInvalidoError(Exception):
    pass

def validar_email(email):
    if "@" not in email or "." not in email:
        raise EmailInvalidoError(f"Email invalido: {email}")
```

Ejercicio 6

Crear clase 'CuentaBancaria' con metodo 'retirar' que lance 'SaldoInsuficienteError' si el monto supera el saldo.

Solucion:

```
class SaldoInsuficienteError(Exception):
    pass

class CuentaBancaria:
    def __init__(self, saldo):
        self.saldo = saldo

    def retirar(self, monto):
        if monto > self.saldo:
            raise SaldoInsuficienteError(f"Saldo ${self.saldo}, intento ${monto}")
        self.saldo -= monto
```

Ejercicio 7

Crear excepcion 'ProductoNoEncontradoError' para un inventario simple.

Solucion:

```
class ProductoNoEncontradoError(Exception):
    pass

inventario = {"manzana": 10, "pera": 5}

def comprar(producto, cantidad):
    if producto not in inventario:
        raise ProductoNoEncontradoError(f"Producto '{producto}' no existe")
    # resto de la logica...
```

Ejercicio 8

Crear excepcion 'StockInsuficienteError' para el mismo inventario.

Solucion:

```

class StockInsuficienteError(Exception):
    pass

def comprar(producto, cantidad):
    if producto not in inventario:
        raise ProductoNoEncontradoError(f"Producto '{producto}' no existe")
    if inventario[producto] < cantidad:
        raise StockInsuficienteError(f"Stock insuficiente de '{producto}': solo hay {inventario[producto]}")
    inventario[producto] -= cantidad

```

Ejercicio 9

Crear una excepcion que herede de 'ValueError' llamada 'RangoError'.

Solucion:

```

class RangoError(ValueError):
    pass

```

Explicacion: Asi, 'RangoError' se comporta como un 'ValueError' pero con un nombre mas especifico.

Ejercicio 10

Crear una jerarquia: 'ErrorArchivo' (base), 'ArchivoNoEncontradoError', 'ArchivoCorruptoError'.

Solucion:

```

class ErrorArchivo(Exception):
    pass

class ArchivoNoEncontradoError(ErrorArchivo):
    pass

class ArchivoCorruptoError(ErrorArchivo):
    pass

```

Ejercicio 11

Crear excepcion 'DivisionPorCeroPersonalizada' y una funcion que la lance.

Solucion:

```

class DivisionPorCeroPersonalizada(Exception):
    pass

def dividir(a, b):
    if b == 0:
        raise DivisionPorCeroPersonalizada("No se puede dividir por cero")
    return a / b

```

Ejercicio 12

Crear excepcion 'EdadInvalidaError' que guarde la edad invalida como atributo.

Solucion:

```
class EdadInvalidaError(Exception):
    def __init__(self, edad):
        self.edad = edad
        super().__init__(f"Edad invalida: {edad}. Debe estar entre 0 y 150.")
```

Ejercicio 13

En un juego, crear excepcion 'GameOverError' y lanzarla cuando las vidas lleguen a 0.

Solucion:

```
class GameOverError(Exception):
    pass

class Jugador:
    def __init__(self, vidas=3):
        self.vidas = vidas

    def recibir_danio(self):
        self.vidas -= 1
        if self.vidas <= 0:
            raise GameOverError("Game Over - No te quedan vidas")
```

Ejercicio 14

Capturar una excepcion personalizada y mostrar su mensaje.

Solucion:

```
try:
    validar_positivo(-5)
except ValorNegativoError as e:
    print(f"Capturado: {e}")
```

Ejercicio 15

Crear excepcion 'ContrasenaDebilError' para validar que una contrasena tenga al menos 8 caracteres.

Solucion:

```
class ContrasenaDebilError(Exception):
    pass

def validar_contrasena(pwd):
    if len(pwd) < 8:
        raise ContrasenaDebilError("La contrasena debe tener al menos 8 caracteres")
```


Ejercicio 16

Usar 'raise ... from' para encadenar una excepcion de conversion de entero.

Solucion:

```
try:
    valor = int("no es numero")
except ValueError as e:
    raise ValueError("No se pudo convertir el texto a entero") from e
```

Ejercicio 17

Crear excepcion 'ErrorAPI' que guarde un codigo de error y un mensaje.

Solucion:

```
class ErrorAPI(Exception):
    def __init__(self, codigo, mensaje):
        self.codigo = codigo
        super().__init__(f"Error {codigo}: {mensaje}")
```

Ejercicio 18

Usar 'else' y 'finally' con excepciones personalizadas.

Solucion:

```
try:
    resultado = dividir(10, 2)
except DivisionPorCeroPersonalizada as e:
    print(f"Error: {e}")
else:
    print(f"Division exitosa: {resultado}")
finally:
    print("Esto se ejecuta siempre")
```

Ejercicio 19

Crear excepcion 'TemperaturaExtremaError' y lanzarla si temperatura >40 o <0.

Solucion:

```
class TemperaturaExtremaError(Exception):
    pass

def verificar_temperatura(temp):
    if temp > 40 or temp < 0:
        raise TemperaturaExtremaError(f"Temperatura extrema: {temp}°C")
```

Ejercicio 20

Crear una clase 'Validador' con metodos que lancen excepciones personalizadas: 'CampoVacioError', 'FormatoInvalidoError'.

Solucion:

```

class ValidacionError(Exception):
    pass

class CampoVacioError(ValidacionError):
    pass

class FormatoInvalidoError(ValidacionError):
    pass

class Validador:
    @staticmethod
    def no_vacio(texto):
        if not texto or not texto.strip():
            raise CampoVacioError("El campo no puede estar vacio")

    @staticmethod
    def email_valido(email):
        if "@" not in email or "." not in email:
            raise FormatoInvalidoError(f"Email invalido: {email}")

# Uso
try:
    Validador.no_vacio("")
except CampoVacioError as e:
    print(e)

```

Resumen de lo que vimos

Dale, repasemos:

- Las excepciones personalizadas se crean heredando de 'Exception'.
- Se lanzan con 'raise' y se capturan con 'try/except'.
- Podes agregar atributos y personalizar el '__init__' para guardar informacion adicional.
- Podes crear jerarquias con una excepcion base y derivadas.
- 'raise ... from' encadena excepciones para no perder la causa original.
- Usa nombres claros que terminen en 'Error' y documentalas.

Reflexion final

Crear tus propias excepciones es una senal de que estas pensando en serio en la calidad de tu codigo. Hace que los errores sean mas faciles de entender y manejar. En lugar de lanzar un 'ValueError' generico, puedes lanzar un 'SaldoInsuficienteError' que dice exactamente que paso. Los que usen tu codigo (incluyendote a vos mismo en el futuro) te lo van a agradecer.

No tengas miedo de usarlas. Es facil y te da un control mucho mas fino.

*Tus errores, tus reglas. Tus
excepciones, tu control.*

Depurador (pdb) y logging: Deja de usar print para siempre

Querido lector: el pecado original de todo principiante

Cuando empecé a programar, mi forma favorita de saber que pasaba adentro de mi código era poner `print()` por todos lados. Ponia prints antes de una función, después de una función, dentro de los bucles, en los condicionales. Mi código terminaba lleno de `print("llegue hasta aca")` y `print(variable)`. Después, cuando el programa andaba, tenía que borrar todos esos prints o comentarlos. Y si aparecía un error nuevo, volvía a poner prints. Un círculo vicioso.

Un día un compañero más experimentado me mostró el depurador (debugger) y el módulo `logging`. Fue como pasar de andar a los empujones a manejar un auto. El depurador te permite ejecutar el código línea por línea y ver el estado de las variables en tiempo real. El `logging` te permite dejar mensajes permanentes que no ensucian la salida normal y que se pueden activar o desactivar según el nivel de gravedad.

En este capítulo te voy a enseñar a usar estas dos herramientas. Son profesionales y te van a ahorrar muchísimas horas de frustración.

El depurador pdb (Python Debugger)

El depurador viene con Python, no hace falta instalar nada. Se llama `pdb`. Tienes dos formas de usarlo.

Forma 1: ejecutar tu script con pdb desde la terminal

```
python -m pdb mi_script.py
```

Esto arranca el programa ya dentro del depurador. El programa se va a ejecutar pero se va a detener en la primera línea, esperando tus comandos.

Forma 2: poner un punto de interrupción en el código (mucho más común)

Dentro de tu código, en el lugar donde quieres que se detenga, pones:

```
breakpoint() # Python 3.7 y superiores
```

Si tienes una versión más vieja de Python, puedes usar:

```
import pdb; pdb.set_trace()
```

Cuando el programa llegue a esa linea, se va a pausar y vas a caer en la terminal del depurador.

Los comandos basicos de pdb

Una vez que estas dentro de pdb, ves un prompt (Pdb). Ahi podes escribir estos comandos:

Comando	Que hace
n (next)	Ejecuta la linea actual y pasa a la siguiente (sin entrar en funciones)
s (step)	Ejecuta la linea actual, y si es una funcion, entra dentro de ella
c (continue)	Sigue ejecutando hasta el proximo breakpoint o hasta el final
q (quit)	Sale del depurador y termina el programa
p variable	Imprime el valor de una variable (ej: p x)
pp variable	Imprime con formato (pretty print) para listas y diccionarios grandes
l (list)	Muestra el codigo alrededor de la linea actual
b numero	Pone un breakpoint en esa linea (ej: b 15)
b	Lista todos los breakpoints activos
cl	Borra todos los breakpoints
w (where)	Muestra la pila de llamadas (donde estas dentro de las funciones)
u (up)	Sube un nivel en la pila de llamadas
d (down)	Baja un nivel en la pila de llamadas

Ejemplo practico de depuracion

Mira este codigo con un bug:

```
def dividir_lista(numeros, divisor):
    resultado = []
    for i in range(len(numeros)):
        resultado.append(numeros[i] / divisor)
    return resultado

datos = [10, 20, 30]
divisor = 0
breakpoint() # aca se detiene
final = dividir_lista(datos, divisor)
print(final)
```

Cuando ejecutes esto, se va a detener justo antes de llamar a `dividir_lista()`. En el prompt (Pdb) podes hacer:

- p datos -> ves la lista.
- p divisor -> ves que es 0. Ahi esta el problema!
- n -> avanzas a la siguiente linea.
- s -> entras dentro de la funcion.
- Una vez dentro, p divisor sigue siendo 0.
- Podes seguir linea por linea con n y ver cuando se produce la division por cero.

Asi se encuentra el error al toque.

El modulo logging: mensajes profesionales

Cuando tu programa ya anda y lo usas en serio, no puedes andar poniendo `print()` por todos lados. Los prints se mezclan con la salida normal del programa y no puedes distinguir entre un mensaje de informacion, una advertencia o un error. Ademas, si quieres guardar los mensajes en un archivo, con prints tendrías que reescribir todo.

Para eso existe el modulo `logging`. Es flexible, potente y viene con Python.

Niveles de logging (de menos a mas grave)

```
import logging
```

```
logging.debug("Detalle minucioso para depurar")
logging.info("Informacion general del flujo normal")
logging.warning("Algo anda mal pero el programa sigue")
logging.error("Ocurrio un error, algo fallo")
logging.critical("Error gravicimo, el programa no puede continuar")
```

Por defecto, `logging` solo muestra los mensajes de nivel `WARNING` o superior. Si quieres ver los de `DEBUG` e `INFO`, tenes que configurarlo.

Configuracion basica

```
import logging
```

```
logging.basicConfig(level=logging.DEBUG,
                    format='%(asctime)s - %(levelname)s - %(message)s')
```

```
logging.debug("Este se ve ahora")
logging.info("Este tambien")
```

`basicConfig()` te permite definir: - `level`: el minimo nivel que se muestra. - `format`: como se formatea cada mensaje. - `filename`: si quieres guardar en un archivo en lugar de la consola.

Guardar logs en un archivo

```
logging.basicConfig(filename='mi_aplicacion.log', level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
logging.info("Este mensaje va al archivo")
```

Formato: los codigos mas utiles

<code>%(asctime)s</code>	Fecha y hora
<code>%(levelname)s</code>	Nivel (DEBUG, INFO, WARNING, etc.)
<code>%(message)s</code>	El mensaje que pasaste
<code>%(name)s</code>	Nombre del logger
<code>%(funcName)s</code>	Nombre de la funcion desde donde se llama
<code>%(lineno)d</code>	Numero de linea

Ejemplo practico

```
import logging

logging.basicConfig(level=logging.INFO,
                    format='%(levelname)s: %(message)s')

def dividir(a, b):
    logging.info(f"Dividiendo {a} / {b}")
    if b == 0:
        logging.error("Intento de division por cero")
        return None
    resultado = a / b
    logging.debug(f"Resultado: {resultado}") # no se ve porque level=INFO
    return resultado

dividir(10, 2)
dividir(10, 0)
```

La salida va a mostrar:

```
INFO: Dividiendo 10 / 2
INFO: Dividiendo 10 / 0
ERROR: Intento de division por cero
```

Logging de excepciones (muy util)

Si atrapas una excepcion, puedes guardar toda la traza completa con `logging.exception()`.

```
try:
    1 / 0
except ZeroDivisionError:
    logging.exception("Esto es lo que paso")
```

Esto va a guardar el mensaje y la traza completa de la excepcion.

20 ejercicios con soluciones paso a paso

Ahora te toca a vos. Ejecuta cada ejercicio, prueba los comandos y fijate que pasa.

Ejercicio 1

Usar `breakpoint()` en un script simple para detener la ejecucion.

Solucion:

```
def prueba():
    x = 10
    breakpoint()
    y = x * 2
```

```
print(y)
```

```
prueba()
```

Explicacion: Al ejecutar, el programa se detiene en la linea del `breakpoint()`. Desde ahi podes inspeccionar las variables.

Ejercicio 2

Desde pdb, imprimir el valor de una variable (ej: `x`).

Solucion:

```
# Cuando el programa este detenido en pdb, escribir:  
(Pdb) p x
```

Ejercicio 3

Usar `n` (`next`) para ejecutar linea por linea sin entrar en funciones.

Solucion:

```
(Pdb) n
```

Se ejecuta la linea actual y se pasa a la siguiente. Si la linea llama a una funcion, `n` la ejecuta entera y sigue.

Ejercicio 4

Usar `s` (`step`) para entrar dentro de una funcion llamada desde el breakpoint.

Solucion:

```
def interna():  
    a = 1  
  
def principal():  
    breakpoint()  
    interna()  
  
principal()  
# Cuando llegue a interna(), escribir 's'
```

Con `s` entras dentro de la funcion `interna()` y podes depurarla linea por linea.

Ejercicio 5

Configurar logging basico con nivel INFO.

Solucion:

```
import logging  
logging.basicConfig(level=logging.INFO)  
logging.info("Mensaje de prueba")
```

Solo se muestra INFO y superiores. DEBUG no se muestra.

Ejercicio 6

Configurar logging para mostrar timestamp en cada mensaje.

Solucion:

```
logging.basicConfig(format='%(asctime)s - %(message)s', level=logging.INFO)
logging.info("Hola")
```

Ejercicio 7

Guardar logs en un archivo `mi_app.log`.

Solucion:

```
logging.basicConfig(filename='mi_app.log', level=logging.DEBUG)
logging.debug("Mensaje en archivo")
```

Ahora el archivo `mi_app.log` contiene el mensaje.

Ejercicio 8

Usar los cinco niveles de logging: debug, info, warning, error, critical.

Solucion:

```
logging.debug("debug")
logging.info("info")
logging.warning("warning")
logging.error("error")
logging.critical("critical")
```

Ejercicio 9

Configurar logging para que solo muestre warnings y superiores, ignorando debug e info.

Solucion:

```
logging.basicConfig(level=logging.WARNING)
logging.info("esto no se ve")
logging.warning("esto se ve")
```

Ejercicio 10

En un bucle, agregar logging para ver el progreso.

Solucion:

```
for i in range(5):
    logging.info(f"Procesando indice {i}")
```

Ejercicio 11

Usar `logging.exception()` dentro de un bloque `except` para guardar la traza completa de la excepcion.

Solucion:

```
try:
    1 / 0
except ZeroDivisionError:
    logging.exception("Error capturado")
```

Esto guarda el mensaje y la traza (las lineas donde ocurrio el error).

Ejercicio 12

Crear un logger personalizado con nombre en lugar de usar la funcion raiz.

Solucion:

```
logger = logging.getLogger("mi_logger")
logger.setLevel(logging.DEBUG)
logger.info("Mensaje desde logger personalizado")
```

Esto permite tener multiples loggers con diferentes configuraciones.

Ejercicio 13

Agregar un handler para enviar logs a consola y otro para enviar a archivo al mismo tiempo.

Solucion:

```
logger = logging.getLogger()
logger.setLevel(logging.DEBUG)

fh = logging.FileHandler("app.log")
ch = logging.StreamHandler()

logger.addHandler(fh)
logger.addHandler(ch)

logger.info("Se ve en consola y en archivo")
```

Ejercicio 14

En pdb, listar el codigo alrededor de la linea actual con el comando `l`.

Solucion:

```
(Pdb) l
```

Muestra unas 10 lineas alrededor de donde estas parado.

Ejercicio 15

En pdb, establecer un breakpoint en una linea especifica (ej: linea 5) usando `b 5`.

Solucion:

```
(Pdb) b 5
```

```
(Pdb) c
```

El programa corre hasta la linea 5 y se detiene ahi.

Ejercicio 16

En pdb, ver la pila de llamadas (donde estas dentro de las funciones) con `w`.

Solucion:

```
(Pdb) w
```

Ejercicio 17

Crear un log con formato que incluya el nombre de la funcion desde la que se llama.

Solucion:

```
logging.basicConfig(format='%(funcName)s - %(message)s', level=logging.INFO)
def test():
    logging.info("dentro de test")
test()
```

Ejercicio 18

Usar `pp` en pdb para imprimir una estructura grande (como un diccionario) con formato legible.

Solucion:

```
(Pdb) pp {"a": [1,2,3,4,5,6,7,8,9]}
```

Ejercicio 19

Simular un programa con error y usar pdb para depurar. Escribi una funcion que divida elementos de una lista por un divisor, pero pone un breakpoint antes de la division.

Solucion:

```
def dividir_lista(lista, divisor):
    resultado = []
    for i, elem in enumerate(lista):
        breakpoint()
        resultado.append(elem / divisor)
    return resultado
```

```
dividir_lista([10,20,30], 0)
```

Cuando se detenga, usa `p divisor` para ver el divisor, `p elem` para ver el elemento, y anda linea por linea con `n`.

Ejercicio 20

Configurar logging para que el formato incluya el nivel, la hora, el mensaje y el nombre del modulo.

Solucion:

```
logging.basicConfig(format='%(asctime)s - %(levelname)s - %(name)s - %(message)s',  
                    level=logging.INFO)  
logging.info("Mensaje con formato completo")
```

Resumen de lo que vimos

Dale, repasemos rapido:

- **pdb**: depurador paso a paso. Agregas `breakpoint()` en el codigo y ejecutas con `python script.py`. Comandos clave: `n` (next), `s` (step), `c` (continue), `p` variable, `pp` variable, `l`, `b`, `w`.
- **logging**: sistema de mensajes por niveles. Configuras una vez y usas `logging.debug()`, `logging.info()`, etc. Podes guardar en archivo y controlar que se muestra.
- Nunca uses `print` para depurar en programas serios. Usa logging con diferentes niveles.
- La combinacion de `pdb` (para encontrar el error) y logging (para dejar trazabilidad) es profesional.

Reflexion final

Dejar atras los `print()` es un signo de madurez como programador. El depurador te permite entender que esta pasando adentro de tu codigo como una radiografia. El logging te permite tener un registro de lo que paso, incluso despues de que el programa termino. Inverti tiempo en aprender `pdb`: te va a ahorrar horas de frustracion.

Y recuerda: cuando veas un error, no empieces a poner prints en cualquier lado. Pone un `breakpoint()` justo antes del lugar sospechoso y camina linea por linea con `n` y `s`. Es como tener rayos X para tu codigo.

Depurar es entender. Loggear es

recordar.

Punto de entrada de scripts: `if __name__ == "__main__"` y argumentos por terminal

Querido lector: dos problemas que todo novato tiene

Cuando empecé a programar, me pasaron dos cosas que me volvieron loco.

La primera: escribía un archivo con funciones, lo importaba desde otro script, y al importar se me ejecutaba todo el código que no quería que se ejecutara. Por ejemplo, tenía un módulo que definía una función y adentro tenía un `print("Probando...")`. Cuando lo importaba, veía ese `print` que no tenía nada que ver. Un desastre.

La segunda: quería pasarle argumentos a mi script desde la terminal, algo como `python script.py -nombre Nahuel`. No tenía idea de cómo hacerlo. Terminaba usando `input()` para pedir datos, pero eso no es lo mismo que pasar argumentos al ejecutar.

En este capítulo vamos a resolver ambos problemas con dos herramientas esenciales: `if __name__ == "__main__"` y `sys.argv`. Son fáciles, pero te cambian la vida.

» Cuando ejecutas un script directamente, Python asigna a la variable especial `__name__` el valor `"__main__"`. Cuando importas ese módulo desde otro lado, `__name__` toma el nombre del módulo (el nombre del archivo sin el `.py`).

» Puedes aprovechar esto para que cierto código solo se ejecute cuando corres el script directamente, y no cuando alguien lo importa.

»»Ejemplo básico

»» Imaginate que tienes un archivo `matematicas.py`:

```
>>># matematicas.py
>>>def suma(a, b):
>>>    return a + b
>>>
>>>def resta(a, b):
>>>    return a - b
>>>
>>>if __name__ == "__main__":
>>>    # Este bloque solo se ejecuta si corro "python matematicas.py"
>>>    print("Probando funciones...")
>>>    print(f"5 + 3 = {suma(5, 3)}")
>>>    print(f"5 - 3 = {resta(5, 3)}")
```

»Si ejecutas `'python matematicas.py'`, vas a ver los mensajes de prueba. Si en otro archivo haces `'import matematicas'`, solo se importan las funciones, pero no se ejecutan las pruebas. Eso es exactamente lo que queremos.

»Estructura típica de un script profesional

»Ya vas a ver que casi todos los scripts bien hechos tienen esta forma:

```
#!/usr/bin/env python3
import sys
import math
'''
def funcion_principal():
    # ... codigo principal ...
    pass
def otra_funcion():
    pass
if __name__ == "__main__":
    funcion_principal()
```

»Así puedes importar las funciones sin que se ejecute nada automáticamente, y cuando quieras correr el programa desde la terminal, invocas la función `'main()'`.

»Analogía: la puerta giratoria

»En criollo: `'__name__'` es como una puerta giratoria. Cuando entras por el frente (ejecutas el script directamente), la puerta te lleva a `'__main__'`. Cuando entras por un costado (importas el módulo), la puerta te lleva al nombre del archivo. El bloque `'if __name__ == "__main__":'` es el cartel que dice "solo para los que entran por el frente".

»Argumentos de línea de comandos con `sys.argv`

»El módulo `'sys'` (que ya usamos para `'exit()'`) tiene una lista llamada `'argv'` (argument vector). Ahí se guardan los argumentos que pasas al ejecutar el script.

»Que contiene `sys.argv`

»El primer elemento (`'sys.argv[0]'`) es siempre el nombre del script. Los siguientes son los argumentos que escribiste separados por espacios.

»Probemos:

```
#!/usr/bin/env python3
import sys
print(sys.argv)
```

»Si ejecutas `'python argv_demo.py hola mundo 123'`, la salida es: `['argv_demo.py', 'hola', 'mundo', '123']`

»Fíjate que todos los argumentos llegan como strings, incluso los números.

»»»Acceder a los argumentos

»»»Vamos a hacer un script que salude a la persona que le pasemos como primer argumento.

```
»»»import sys
»»»
»»»if len(sys.argv) < 2:
»»»    print("Uso: python script.py <nombre>")
»»»    sys.exit(1)
»»»
»»»nombre = sys.argv[1]
»»»print(f"Hola {nombre}!")
```

»»»Si ejecutas `'python saludo.py Nahuel'`, te dice `'Hola Nahuel'`. Si no le pasas argumento, te muestra el mensaje de uso y sale con `'sys.exit(1)'` (el 1 indica error).

»»»Convertir argumentos numericos

»»»Como todo es string, si necesitas numeros tenes que convertirlos.

```
»»»import sys
»»»
»»»if len(sys.argv) < 3:
»»»    print("Uso: python calculadora.py <a> <b>")
»»»    sys.exit(1)
»»»
»»»a = float(sys.argv[1])
»»»b = float(sys.argv[2])
»»»print(f"Suma: {a + b}")
```

»»»Ejemplo: calculadora por terminal completa

»»»Ahora que ya sabes, puedes hacer una calculadora que se usa asi: `'python calculadora.py suma 5 3'` y te devuelve 8.

```
»»»import sys
»»»
»»»def suma(a, b): return a + b
»»»def resta(a, b): return a - b
»»»def multi(a, b): return a * b
»»»def division(a, b): return a / b if b != 0 else "Error: division por cero"
»»»
»»»def mostrar_ayuda():
»»»    print("Calculadora por terminal")
»»»    print("Uso: python calculadora.py <operacion> <a> <b>")
»»»    print("Operaciones: suma, resta, multi, div")
»»»    print("Ejemplo: python calculadora.py suma 5 3")
»»»
»»»def main():
»»»    if len(sys.argv) != 4:
```

```

>>>     mostrar_ayuda()
>>>     sys.exit(1)
>>>
>>>     operacion = sys.argv[1].lower()
>>>     try:
>>>         a = float(sys.argv[2])
>>>         b = float(sys.argv[3])
>>>     except ValueError:
>>>         print("Error: los argumentos deben ser numeros")
>>>         sys.exit(1)
>>>
>>>     if operacion == "suma":
>>>         resultado = suma(a, b)
>>>     elif operacion == "resta":
>>>         resultado = resta(a, b)
>>>     elif operacion == "multi":
>>>         resultado = multi(a, b)
>>>     elif operacion == "div":
>>>         resultado = division(a, b)
>>>     else:
>>>         print("Operacion no reconocida")
>>>         mostrar_ayuda()
>>>         sys.exit(1)
>>>
>>>     print(f"Resultado: {resultado}")
>>>
>>> if __name__ == "__main__":
>>>     main()

```

»Este script ya es un programa serio: tiene ayuda, manejo de errores y el bloque `if __name__ == "__main__":` bien puesto.

»Combinando todo: scripts reutilizables y ejecutables

»La magia de `if __name__ == "__main__":` es que puedes tener un archivo que sirva tanto como librería (para importar funciones) como programa ejecutable. Eso es muy común en paquetes de Python.

»Ejemplo: un modulo con funciones y una interfaz por terminal

```

>>> # saludos.py
>>> def saludar(nombre):
>>>     return f"Hola {nombre}"
>>>
>>> def despedir(nombre):
>>>     return f"Chau {nombre}"
>>>
>>> def main():

```



```

>>> import sys
>>> if len(sys.argv) < 3:
>>>     print("Uso: python saludos.py <saludar|despedir> <nombre>")
>>>     sys.exit(1)
>>> accion = sys.argv[1]
>>> nombre = sys.argv[2]
>>> if accion == "saludar":
>>>     print(saludar(nombre))
>>> elif accion == "despedir":
>>>     print(despedir(nombre))
>>> else:
>>>     print("Accion no reconocida")
>>>
>>> if __name__ == "__main__":
>>>     main()
    
```

»»» Ahora puedes correrlo desde terminal: `python saludos.py saludar Ana`. Y tambien puedes importarlo desde otro script: `from saludos import saludar`.

»»» 20 ejercicios con soluciones paso a paso

»»» Ahora te toca a vos. Resuelve cada ejercicio. Las soluciones estan abajo.

»»» Ejercicio 1

»»» Crear un script que imprima "Hola mundo" solo si se ejecuta directamente, no cuando se importa.

»»» Solucion:

```

>>> if __name__ == "__main__":
>>>     print("Hola mundo")
    
```

»»» Explicacion: El bloque se ejecuta solo cuando `'__name__'` es `'__main__'`.

»»» Ejercicio 2

»»» Crear una funcion `saludar(nombre)` y en el bloque principal llamarla con "Nahuel".

»»» Solucion:

```

>>> def saludar(nombre):
>>>     print(f"Hola {nombre}")
>>>
>>> if __name__ == "__main__":
>>>     saludar("Nahuel")
    
```

»»» Ejercicio 3

»»» Usar `'sys.argv'` para imprimir el nombre del script (el primer argumento).

»»» Solucion:

```
»'''import sys
»'''print(sys.argv[0])
```

»'''Ejercicio 4

»'''Usar `'sys.argv'` para imprimir cuantos argumentos se pasaron (sin contar el nombre del script).

»'''Solucion:

```
»'''import sys
»'''print(f"Argumentos: {len(sys.argv) - 1}")
```

»'''Ejercicio 5

»'''Si no se pasa al menos un argumento, mostrar "Uso: script.py <nombre>" y salir con error; sino saludar al nombre.

»'''Solucion:

```
»'''import sys
»'''if len(sys.argv) < 2:
»'''    print("Uso: script.py <nombre>")
»'''    sys.exit(1)
»'''print(f"Hola {sys.argv[1]}")
```

»'''Ejercicio 6

»'''Convertir el primer argumento a entero y mostrar su doble. Si no hay argumento o no es numero, mostrar error.

»'''Solucion:

```
»'''import sys
»'''if len(sys.argv) < 2:
»'''    print("Uso: script.py <numero>")
»'''    sys.exit(1)
»'''try:
»'''    n = int(sys.argv[1])
»'''    print(f"Doble: {n * 2}")
»'''except ValueError:
»'''    print("Error: argumento debe ser entero")
»'''    sys.exit(1)
```

»'''Ejercicio 7

»'''Sumar dos numeros pasados como argumentos. Si no son dos argumentos, mostrar ayuda.

»'''Solucion:

```
»'''import sys
»'''if len(sys.argv) < 3:
»'''    print("Uso: script.py <a> <b>")
»'''    sys.exit(1)
```

```
»'a = float(sys.argv[1])
»'b = float(sys.argv[2])
»'print(f"Suma: {a + b}")
```

»'Ejercicio 8

»'Crear un modulo 'operaciones.py' con funciones suma, resta, multiplicacion, division.
Usar 'if `__name__ == "__main__"` para probarlas con valores fijos.

»'Solucion:

```
»'def suma(a,b): return a+b
»'def resta(a,b): return a-b
»'def multi(a,b): return a*b
»'def division(a,b): return a/b if b!=0 else "Error division por cero"
»'
»'if __name__ == "__main__":
»'    print(suma(5,3))
»'    print(resta(5,3))
»'    print(multi(5,3))
»'    print(division(5,3))
```

»'Ejercicio 9

»'Importar el modulo anterior desde otro script y usar la funcion suma.

»'Solucion:

```
»'import operaciones
»'print(operaciones.suma(10,5))
```

»'Ejercicio 10

»'Crear un script que reciba argumentos con banderas: '-n' para nombre y '-e' para edad.
Ejemplo: 'python datos.py -n Ana -e 25'.

»'Solucion:

```
»'import sys
»'nombre = None
»'edad = None
»'i = 1
»'while i < len(sys.argv):
»'    if sys.argv[i] == "-n" and i+1 < len(sys.argv):
»'        nombre = sys.argv[i+1]
»'        i += 2
»'    elif sys.argv[i] == "-e" and i+1 < len(sys.argv):
»'        edad = sys.argv[i+1]
»'        i += 2
»'    else:
»'        i += 1
»'if nombre and edad:
»'    print(f"{nombre} tiene {edad} anos")
```

```
»'''else:
»'''    print("Uso: script.py -n nombre -e edad")
```

»'''Ejercicio 11

»'''Usar `'sys.exit(1)'` para indicar error si faltan argumentos.

»'''Solucion:

```
»'''import sys
»'''if len(sys.argv) < 2:
»'''    print("Falta argumento")
»'''    sys.exit(1)
»'''print("OK")
```

»'''Ejercicio 12

»'''En una calculadora por argumentos, manejar division por cero.

»'''Solucion:

```
»'''import sys
»'''if len(sys.argv) != 3:
»'''    sys.exit(1)
»'''a = float(sys.argv[1])
»'''b = float(sys.argv[2])
»'''if b == 0:
»'''    print("Error: division por cero")
»'''    sys.exit(1)
»'''print(a / b)
```

»'''Ejercicio 13

»'''Crear un script que muestre ayuda si se pasa `'-h'` o `'--help'` como argumento.

»'''Solucion:

```
»'''import sys
»'''if "-h" in sys.argv or "--help" in sys.argv:
»'''    print("Ayuda: este script no hace mucho, pero acepta la opcion -h")
»'''    sys.exit(0)
»'''print("Ejecutando script normal...")
```

»'''Ejercicio 14

»'''En un modulo, poner pruebas unitarias dentro de `'if __name__ == "__main__":'`.
Escribir una funcion que multiplique varios numeros y probarla.

»'''Solucion:

```
»'''def multiplicar_todo(*args):
»'''    res = 1
»'''    for n in args:
»'''        res *= n
```

```
»»»     return res
»»»
»»»if __name__ == "__main__":
»»»     assert multiplicar_todo(2,3) == 6
»»»     assert multiplicar_todo(2,3,4) == 24
»»»     print("Todas las pruebas pasaron")
```

»»»Ejercicio 15

»»»Leer un archivo cuyo nombre se pasa como argumento y mostrar su contenido.

»»»Solucion:

```
»»»import sys
»»»if len(sys.argv) < 2:
»»»     print("Uso: script.py <archivo>")
»»»     sys.exit(1)
»»»with open(sys.argv[1], "r", encoding='utf-8') as f:
»»»     print(f.read())
```

»»»Ejercicio 16

»»»Crear un script que reciba ‘-suma a b’ y muestre el resultado.

»»»Solucion:

```
»»»import sys
»»»if len(sys.argv) != 4 or sys.argv[1] != "--suma":
»»»     print("Uso: script.py --suma a b")
»»»     sys.exit(1)
»»»print(float(sys.argv[2]) + float(sys.argv[3]))
```

»»»Ejercicio 17

»»»Estructura típica: un módulo con varias funciones y una función ‘main()’ llamada desde
‘if `__name__ == "__main__"`.’

»»»Solucion:

```
»»»def func1():
»»»     print("func1")
»»»
»»»def func2():
»»»     print("func2")
»»»
»»»def main():
»»»     print("Ejecutando main")
»»»     func1()
»»»     func2()
»»»
»»»if __name__ == "__main__":
»»»     main()
```

»»Ejercicio 18

»»Mostrar todos los argumentos pasados al script, numerados.

»»Solucion:

```
»»import sys
»»for i, arg in enumerate(sys.argv):
»»    print(f"{i}: {arg}")
```

»»Ejercicio 19

»»Simular el comando 'head': mostrar las primeras N líneas de un archivo. El primer argumento es el archivo, el segundo opcional es la cantidad de líneas (por defecto 10).

»»Solucion:

```
»»import sys
»»if len(sys.argv) < 2:
»»    print("Uso: script.py <archivo> [n]")
»»    sys.exit(1)
»»archivo = sys.argv[1]
»»n = int(sys.argv[2]) if len(sys.argv) > 2 else 10
»»with open(archivo, "r", encoding='utf-8') as f:
»»    for i, linea in enumerate(f):
»»        if i >= n:
»»            break
»»    print(linea, end="")
```

»»Ejercicio 20

»»Script que recibe 'help' o '-h' (muestra ayuda) o ningun argumento (muestra "Hola"). Si recibe cualquier otra cosa, muestra error.

»»Solucion:

```
»»import sys
»»if len(sys.argv) == 1:
»»    print("Hola")
»»elif len(sys.argv) == 2 and sys.argv[1] in ("-h", "--help"):
»»    print("Ayuda: este script saluda o muestra ayuda")
»»else:
»»    print("Error: argumentos no reconocidos. Use -h o --help")
»»    sys.exit(1)
```

»»Resumen de lo que vimos

»»Dale, repasemos:

- La variable '`__name__`' vale '`"__main__"`' cuando ejecutas el script directamente. Cuando importas, vale el nombre del modulo.

- El bloque `if __name__ == "__main__":` sirve para que cierto código solo se ejecute al correr el script, no al importarlo.
- `sys.argv` es una lista con los argumentos de la terminal. `sys.argv[0]` es el nombre del script. Los siguientes son los argumentos como strings.
- `sys.exit(código)` termina el programa. `sys.exit(0)` es éxito, cualquier otro número (como 1) indica error.
- Convertir los argumentos numéricos con `int()` o `float()`.

»»»Reflexion final

»»»Querido lector, estos dos conceptos son los que separan a un script “casero” de un programa “profesional”. Aprenderlos bien y usarlos siempre. Con ellos puedes crear herramientas que se integran con la terminal, que otros pueden reutilizar sin miedo, y que tienen una interfaz clara. No hay excusa para no poner `if __name__ == "__main__"` en todos tus scripts a partir de ahora.

»»»Además, `sys.argv` te abre la puerta a escribir tus propios comandos estilo terminal. Hace un script que reciba `-help` y muestre una linda ayuda. Convertir tus calculadoras en comandos. Es más divertido que usar `input()`.

»»»_____ *Un script que se puede
importar es un script que se puede reutilizar.*

»'''PEP 8 y Docstrings: Escribe código lindo y documentado

»'''Querido lector: el código se lee mas veces de las que se escribe

»'''Cuando empecé a programar, me importaba una sola cosa: que el código funcionara. Nada más. Usaba nombres de variables como 'a', 'b', 'c', 'temp', 'x1', 'x2'. Todo en una sola línea. Sin comentarios. Sin explicaciones. Y funcionaba... hasta que volvía a ese código tres meses después y no entendía nada. Me pasaba horas tratando de descifrar que hacía cada cosa. Era como leer un mensaje encriptado que yo mismo había escrito.

»'''Ahí entendí algo fundamental: el código no solo es para la computadora. Es para los humanos (incluyéndote a vos en el futuro). Por eso existen las convenciones de estilo y la documentación.

»'''Python tiene su propio manual de estilo: se llama **PEP 8** (Python Enhancement Proposal 8). Es un documento que explica cómo escribir código Python limpio, consistente y legible. Además, tenemos los **docstrings**, que son comentarios especiales para documentar que hace cada función, clase o módulo.

»'''En este capítulo te voy a enseñar las reglas más importantes de PEP 8 y cómo escribir buenos docstrings. Al final, como siempre, 20 ejercicios para practicar.

»'''Que es PEP 8 y por que importa

»'''PEP 8 es el manual de estilo oficial de Python. No es obligatorio, pero es la convención que sigue toda la comunidad. Si seguís PEP 8, otros programadores van a poder leer tu código sin esfuerzo. Y vos también vas a poder leer código ajeno más fácilmente.

»'''En criollo: es como ponerse de acuerdo en qué lado de la ruta se maneja. Si cada uno maneja por donde quiere, chocamos. Lo mismo con el código.

»'''Las reglas mas importantes de PEP 8

»'''Vamos a ver las que más vas a usar. Son fáciles de recordar.

»»»Nombres de variables y funciones: snake_case

»»»Todas en minúsculas, con guiones bajos separando palabras.

```
»»»# Correcto
»»»mi_variable = 10
»»»nombre_usuario = "Nahuel"
»»»saludar_usuario = True
»»»
»»»# Incorrecto (camelCase o mezclas raras)
»»»miVariable = 10
»»»nombreUsuario = "Nahuel"
»»»saludarusuario = True
```

»»»Nombres de clases: CamelCase

»»»Cada palabra empieza con mayúscula, sin guiones bajos.

```
»»»# Correcto
»»»class MiPrimeraClase:
»»»    pass
»»»
»»»class UsuarioRegistrado:
»»»    pass
»»»
»»»# Incorrecto
»»»class mi_primera_clase:
»»»    pass
»»»
»»»class usuarioregistrado:
»»»    pass
```

»»»Constantes: MAYUSCULAS con guion bajo

»»»Para valores que no cambian.

```
»»»PI = 3.14159
»»»MAXIMO_INTENTOS = 3
»»»COLORES_PERMITIDOS = ["rojo", "verde", "azul"]
```

»»»Indentacion: 4 espacios, nunca tabs

»»»Python usa indentacion para definir bloques. PEP 8 dice: 4 espacios por nivel. Configura tu editor para que convierta tabs a espacios.

```

»'''# Correcto
»'''if True:
»'''    print("Hola")
»'''    print("Mundo")
»'''
»'''# Incorrecto (tabulador o mezcla)
»'''if True:
»'''    print("Hola")
»'''        print("Mundo")

```

»'''Longitud de linea maxima: 79 caracteres

»'''No hagas lineas interminables. Si una linea es muy larga, partila.

```

»'''# Correcto
»'''resultado = funcion_muy_larga(primer_parametro, segundo_parametro,
»'''                                tercer_parametro, cuarto_parametro)
»'''
»'''# Tambien se puede con parentesis implicitos
»'''resultado = (
»'''    primer_parametro + segundo_parametro +
»'''    tercer_parametro + cuarto_parametro
»''')
»'''
»'''# Incorrecto (linea muy larga)
»'''resultado = funcion_muy_larga(primer_parametro, segundo_parametro, tercer_parametro,
»'''                                cuarto_parametro, quinto_parametro, sexto_parametro,
»'''                                septimo_parametro, octavo_parametro, noveno_parametro,
»'''                                decimo_parametro)

```

»'''Espacios alrededor de operadores

»'''Para que el codigo respire.

```

»'''# Correcto
»'''x = y + z
»'''x += 1
»'''lista = [1, 2, 3]
»'''
»'''# Incorrecto
»'''x=y+z
»'''x+=1
»'''lista = [1,2,3]

```

»'''Lineas en blanco

»'''Separan las partes del codigo.

```

»'''# Dos lineas en blanco entre funciones de nivel superior
»'''def funcion1():
»'''    pass
»'''
»'''
»'''def funcion2():
»'''    pass
»'''
»'''
»'''# Una linea en blanco entre metodos de una clase
»'''class Ejemplo:
»'''    def metodo1(self):
»'''        pass
»'''
»'''    def metodo2(self):
»'''        pass

```

»'''Importaciones: orden y agrupacion

»'''Siempre al principio del archivo, agrupadas en tres bloques:

»'''1. Modulos de la biblioteca estandar de Python. 2. Modulos de terceros (los que instalaste con pip). 3. Modulos locales (los que escribiste vos).

```

»'''# Correcto
»'''import os
»'''import sys
»'''
»'''import requests
»'''from flask import Flask
»'''
»'''from mi_modulo import mi_funcion
»'''from . import otro_modulo
»'''
»'''# Incorrecto (mezclado)
»'''import os
»'''import requests
»'''import sys
»'''from mi_modulo import mi_funcion

```

»'''Comillas: consistencia

»'''Podes usar comillas simples o dobles, pero se constante. PEP 8 recomienda usar las mismas que usa el resto del proyecto.

```

»'''# Consistente (comillas simples)
»'''nombre = 'Nahuel'

```

```

>>>print('Hola')
>>>
>>># Consistente (comillas dobles)
>>>nombre = "Nahuel"
>>>print("Hola")

```

»»»Docstrings: documentar tu codigo

»»»Un docstring es un string que aparece justo despues de la definicion de una funcion, clase o modulo. Se escribe con triple comillas (simples o dobles). No es un comentario (que empieza con “), es una documentacion que Python guarda y puedes consultar con ‘help()’.

»»»Docstring de funcion - el caso mas comun

```

>>>def sumar(a, b):
>>>    """Suma dos numeros y devuelve el resultado."""
>>>    return a + b

```

»»»Si la funcion es mas compleja, puedes poner varias lineas:

```

>>>def dividir(dividendo, divisor):
>>>    """
>>>    Divide un numero por otro.
>>>
>>>    Args:
>>>        dividendo (float): Numero a dividir.
>>>        divisor (float): Numero que divide.
>>>
>>>    Returns:
>>>        float: Resultado de la division, o None si divisor es cero.
>>>    """
>>>    if divisor == 0:
>>>        return None
>>>    return dividendo / divisor

```

»»»Docstring de clase

```

>>>class Persona:
>>>    """
>>>    Representa a una persona con nombre y edad.
>>>
>>>    Attributes:
>>>        nombre (str): El nombre de la persona.
>>>        edad (int): La edad de la persona.

```

```

>>> """
>>> def __init__(self, nombre, edad):
>>>     self.nombre = nombre
>>>     self.edad = edad
>>>
>>> def saludar(self):
>>>     """Devuelve un saludo de la persona."""
>>>     return f"Hola, me llamo {self.nombre}"

```

»»»Docstring de modulo

»»»Al principio del archivo '.py':

```

>>>"""
>>>Modulo para operaciones matematicas basicas.
>>>
>>>Este modulo provee funciones de suma, resta, multiplicacion y division.
>>>Tambien incluye una funcion para calcular potencias.
>>>
>>>Example:
>>>    >>> from mate import suma
>>>    >>> suma(5, 3)
>>>    8
>>>"""

```

»»»Ver el docstring con help()

»»»Una vez que documentaste, cualquiera puede verlo:

```

>>>>> help(sumar)
>>>Help on function sumar in module __main__:
>>>sumar(a, b)
>>>    Suma dos numeros y devuelve el resultado.

```

»»»Tambien puedes acceder al docstring como atributo:

```

>>>>> print(sumar.__doc__)
>>>Suma dos numeros y devuelve el resultado.

```

»»»Ejemplo completo con PEP 8 y docstrings

»»»Aca te dejo una calculadora completa, escrita con estilo PEP 8 y bien documentada.

```

>>>#!/usr/bin/env python3
>>>"""
>>>Calculadora basica en Python.
>>>
>>>Este modulo implementa operaciones aritmeticas basicas (suma, resta,
>>>multiplicacion, division) y puede ejecutarse desde la terminal.
>>>"""
>>>
>>>import sys
>>>
>>>def suma(a, b):
>>>    """Devuelve la suma de a y b."""
>>>    return a + b
>>>
>>>def resta(a, b):
>>>    """Devuelve la resta de a y b."""
>>>    return a - b
>>>
>>>def multiplicacion(a, b):
>>>    """Devuelve el producto de a y b."""
>>>    return a * b
>>>
>>>def division(a, b):
>>>    """
>>>    Devuelve la division de a entre b.
>>>
>>>    Args:
>>>        a (float): Dividendo.
>>>        b (float): Divisor.
>>>
>>>    Returns:
>>>        float: Cociente, o None si b es 0.
>>>    """
>>>    if b == 0:
>>>        return None
>>>    return a / b
>>>
>>>def main():
>>>    """Funcion principal que ejecuta la calculadora desde terminal."""
>>>    if len(sys.argv) != 4:
>>>        print("Uso: python calculadora.py <op> <a> <b>")
>>>        print("Operaciones: suma, resta, multi, div")
>>>        sys.exit(1)
>>>
>>>    op = sys.argv[1]
>>>    try:
>>>        a = float(sys.argv[2])
>>>        b = float(sys.argv[3])

```

```

»»» except ValueError:
»»»     print("Error: Los argumentos deben ser numeros.")
»»»     sys.exit(1)
»»»
»»» if op == "suma":
»»»     print(suma(a, b))
»»» elif op == "resta":
»»»     print(resta(a, b))
»»» elif op == "multi":
»»»     print(multiplicacion(a, b))
»»» elif op == "div":
»»»     resultado = division(a, b)
»»»     if resultado is None:
»»»         print("Error: Division por cero.")
»»»     else:
»»»         print(resultado)
»»» else:
»»»     print(f"Operacion '{op}' no reconocida.")
»»»
»»» if __name__ == "__main__":
»»»     main()

```

»»» Herramientas para verificar PEP 8

»»» No tienes que memorizar todas las reglas. Puedes usar herramientas automáticas que te dicen que está mal.

»»» pycodestyle (antes llamado pep8)

»»» Instálalo con 'pip install pycodestyle'. Después corrélo sobre tu script:

```
»»» pycodestyle mi_script.py
```

»»» Te va a mostrar las líneas que violan PEP 8.

»»» pylint (más completo)

»»» Pylint no solo revisa estilo, también busca errores lógicos, variables no usadas, etc.

```

»»» pip install pylint
»»» pylint mi_script.py

```

»'flake8 (combina varias herramientas)

»'Flake8 combina pycodestyle, pyflakes y mccabe. Es muy popular.

```
»'pip install flake8
»'flake8 mi_script.py
```

»'20 ejercicios con soluciones paso a paso

»'Ahora te toca a vos. Practica corrigiendo código y escribiendo docstrings.

»'Ejercicio 1

»'Corregir el nombre de variable: 'miVariable' según PEP 8.

»'Solucion:

```
»'mi_variable
```

»'Explicacion: Variables en snake_case: minúsculas con guiones bajos.

»'Ejercicio 2

»'Corregir el nombre de clase: 'miClase'.

»'Solucion:

```
»'MiClase
```

»'Explicacion: Clases en CamelCase: primera letra mayúscula, sin guiones bajos.

»'Ejercicio 3

»'Corregir la indentación en: 'if True:("Hola")'

»'Solucion:

```
»'if True:
»'    print("Hola")
```

»'Explicacion: El bloque del if debe tener 4 espacios de indentación.

»'Ejercicio 4

»'Agregar espacios alrededor de operadores: 'x=y+z'

»'Solucion:

```
»'x = y + z
```


»»Ejercicio 5

»»Escribir un docstring simple para la funcion 'es_par(n)'.

»»Solucion:

```
»»def es_par(n):
»»    """Devuelve True si n es par, False si es impar."""
»»    return n % 2 == 0
```

»»Ejercicio 6

»»Escribir un docstring mas completo para 'area_circulo(radio)', con Args y Returns.

»»Solucion:

```
»»def area_circulo(radio):
»»    """
»»    Calcula el area de un circulo.
»»
»»    Args:
»»        radio (float): Radio del circulo.
»»
»»    Returns:
»»        float: Area del circulo.
»»    """
»»    import math
»»    return math.pi * radio ** 2
```

»»Ejercicio 7

»»Agregar docstring a la clase 'Auto'.

»»Solucion:

```
»»class Auto:
»»    """Representa un automovil con marca y modelo."""
»»    def __init__(self, marca, modelo):
»»        self.marca = marca
»»        self.modelo = modelo
```

»»Ejercicio 8

»»Ordenar correctamente las importaciones: 'import sys', 'from math import pi', 'import os'.

»»Solucion:

```
»»import os
»»import sys
»»from math import pi
```

»»»**Explicacion:** Primero modulos estandar, luego de terceros (aunque pi es estandar). En este caso, orden alfabetico dentro de cada grupo.

»»»Ejercicio 9

»»»Escribir una constante para el numero PI siguiendo PEP 8.

»»»**Solucion:**

```
»»»PI = 3.14159265359
```

»»»Ejercicio 10

»»»Partir la siguiente linea larga en varias: 'resultado = funcion(param1, param2, param3, param4, param5)'

»»»**Solucion:**

```
»»»resultado = funcion(param1, param2,  
»»»                    param3, param4,  
»»»                    param5)
```

»»»Ejercicio 11

»»»Usar 'help()' sobre una funcion que tiene docstring.

»»»**Solucion:**

```
»»»help(mi_funcion)
```

»»»Ejercicio 12

»»»Agregar docstring de modulo a un archivo (al principio).

»»»**Solucion:**

```
»»» """  
»»» Modulo para procesar datos de clientes.  
»»»  
»»» Este modulo contiene funciones para validar emails, formatear nombres  
»»» y generar reportes basicos.  
»»» """
```

»»»Ejercicio 13

»»»Corregir nombre de funcion: 'calcularPromedio'.

»»»**Solucion:**

```
»»»calcular_promedio
```

»»Ejercicio 14

»»Agregar espacios despues de las comas: '[1,2,3]'.

»»Solucion:

```
»» [1, 2, 3]
```

»»Ejercicio 15

»»Usar 'pylint' para verificar un script. Escribi el comando.

»»Solucion:

```
»» pylint mi_script.py
```

»»Ejercicio 16

»»Escribir un docstring que incluya un ejemplo de uso (formato doctest).

»»Solucion:

```
»»def multiplicar(a, b):
»»    """
»»    Multiplica dos numeros.
»»
»»    Example:
»»        >>> multiplicar(3, 4)
»»        12
»»    """
»»    return a * b
```

»»Ejercicio 17

»»Corregir la indentacion en un bloque if anidado (mostrar como deberia verse).

»»Solucion:

```
»»if a > 0:
»»    if b > 0:
»»        print("ambos positivos")
```

»»Ejercicio 18

»»Mostrar como poner dos lineas en blanco entre dos funciones.

»»Solucion:

```

»'''def funcion1():
»'''    pass
»'''
»'''
»'''def funcion2():
»'''    pass

```

»'''Ejercicio 19

»'''Usar 'flake8' para verificar estilo. Escribi el comando.

»'''Solucion:

```

»'''flake8 mi_script.py

```

»'''Ejercicio 20

»'''Escribir un script completo (breve) con: docstring de modulo, al menos una funcion con docstring, y bloque 'if `__name__ == "__main__"`: *Todo con estilo PEP8.*

»'''Solucion:

```

»'''#!/usr/bin/env python3
»'''
»'''Utilidades de texto.
»'''
»'''Este modulo provee funciones para manipular strings.
»'''
»'''
»'''def invertir(texto):
»'''    """Devuelve el texto invertido."""
»'''    return texto[::-1]
»'''
»'''def es_palindromo(texto):
»'''    """
»'''    Verifica si un texto es palindromo.
»'''
»'''    Args:
»'''        texto (str): Texto a verificar.
»'''
»'''    Returns:
»'''        bool: True si es palindromo, False si no.
»'''    """
»'''    limpio = texto.lower().replace(" ", "")
»'''    return limpio == limpio[::-1]
»'''
»'''if __name__ == "__main__":
»'''    print(es_palindromo("anita lava la tina"))

```

»»Resumen de lo que vimos

»»Dale, repasemos:

- **PEP 8:** manual de estilo de Python. No es obligatorio, pero es la convencion.
- **Nombres:** 'snake_case' para variables y funciones; 'CamelCase' para clases; 'MAYUSCULAS' para constantes.
- **Indentacion:** 4 espacios, nunca tabs.
- **Lines:** maximo 79 caracteres.
- **Espacios:** alrededor de operadores, despues de comas.
- **Imports:** agrupados y ordenados.
- **Docstrings:** documentacion con triple comillas. Se ven con 'help()'.
- **Herramientas:** 'pycodestyle', 'pylint', 'flake8' te ayudan a verificar PEP 8.

»»Reflexion final

»»Escribir codigo legible y documentado no es una perdida de tiempo. Es respeto por vos mismo (cuando lo leas en el futuro) y por los que trabajen con vos. El codigo limpio es una virtud que se nota a simple vista.

»»Empeza a aplicar PEP 8 desde ahora. En cada script que escribas, segui las reglas. Te van a parecer molestas al principio, pero despues no vas a poder escribir codigo desordenado. Es como ordenar tu escritorio: cuesta un poco al principio, pero despues te sentis mejor.

»»Y los docstrings... Âausamelos! Nada mas triste que una funcion sin documentacion. Escribi siempre al menos una linea explicando que hace. Tu yo del futuro te lo va a agradecer.

»» _____ *Escribe tu codigo como si quien lo
fuera a mantener fuera un psicopata que sabe donde vivis.*

»»»Modulo copy - Copia superficial y profunda (y algo mas de sys)

»»»Querido lector: el error de la lista que no queria copiarse

»»»Cuando empecé a programar, intente copiar una lista haciendo `lista2 = lista1`. Después modifique `lista2` y, para mi sorpresa, `lista1` también se modificó. Pense que Python estaba loco. Me pase una noche entera tratando de entender por que mis datos se corrompian. Resulta que el error era mio.

»»»`lista2 = lista1` no crea una copia nueva. Hace que las dos variables apunten al MISMO objeto en memoria. Es como tener dos llaveros que abren la misma puerta: si cambias algo adentro, ambos lo ven.

»»»En este capitulo vamos a aprender a hacer ****copias de verdad****, y la diferencia entre copia superficial (shallow copy) y copia profunda (deep copy). Además, vamos a repasar algunas funciones del modulo `'sys'` que todavía no vimos.

»»»El problema: asignacion no es copia

»»»Para que lo veas con tus propios ojos:

```
»»»>>> lista1 = [1, 2, [3, 4]]
»»»>>> lista2 = lista1
»»»>>> lista2[0] = 99
»»»>>> print(lista1)
»»»[99, 2, [3, 4]] # <-- se modifiko!
```

»»»¿Que paso? `'lista2'` no es una copia, es un alias. Ambas variables apuntan al mismo objeto. Para crear una copia independiente, necesitas hacerlo explícitamente.

»»»Copia superficial (shallow copy)

»»»Una copia superficial crea un nuevo objeto (una nueva lista, nuevo diccionario, etc.), pero los elementos internos (como listas anidadas) siguen siendo los mismos objetos compartidos. Es decir, copia solo el primer nivel.

»»»Formas de hacer copia superficial en listas

```

>>>import copy
>>>
>>>lista1 = [1, 2, [3, 4]]
>>>
>>># Metodo 1: slice
>>>lista2 = lista1[:]
>>>
>>># Metodo 2: list()
>>>lista2 = list(lista1)
>>>
>>># Metodo 3: copy() de la lista (Python 3.3+)
>>>lista2 = lista1.copy()
>>>
>>># Metodo 4: copy.copy() del modulo copy
>>>lista2 = copy.copy(lista1)

```

»»»El problema con listas anidadas (la trampa)

»»»Fijate que pasa si tenes una lista dentro de otra:

```

>>>>> lista1 = [1, 2, [3, 4]]
>>>>> lista2 = lista1.copy() # copia superficial
>>>>> lista2[2][0] = 99
>>>>> print(lista1)
>>> [1, 2, [99, 4]] # <-- el interno se modifiko!

```

»»»¿Por que? Porque la copia superficial copia la lista exterior, pero la lista interior sigue siendo el mismo objeto compartido. Para copiar todo, necesitas una copia profunda.

»»»Copia profunda (deep copy)

»»»La copia profunda crea un nuevo objeto y tambien copia recursivamente todos los objetos internos. No comparten nada. Es como una fotocopia de una fotocopia: todo es nuevo.

```

>>>>> import copy
>>>>> lista1 = [1, 2, [3, 4]]
>>>>> lista2 = copy.deepcopy(lista1)
>>>>> lista2[2][0] = 99
>>>>> print(lista1)
>>> [1, 2, [3, 4]] # <-- no se modifiko
>>>>> print(lista2)
>>> [1, 2, [99, 4]]

```

»»»Usa 'deepcopy()' cuando tengas estructuras anidadas (listas de listas, diccionarios con listas, objetos que contienen otros objetos, etc.).

»»» Cuando usar cada una

- Usa **copia superficial** si tu estructura es plana (solo un nivel) y no necesitas independencia total de los objetos internos.
- Usa **copia profunda** cuando tengas anidamiento y quieras que la copia sea completamente independiente.

»»» Otras utilidades del modulo sys (lo que falta)

»»» Ya vimos `sys.argv` y `sys.exit()`. Ahora te muestro algunas funciones mas que te van a servir.

»»» `sys.path` - las rutas donde Python busca modulos

»»» Cuando haces `import algo`, Python busca en las carpetas que estan en `sys.path`. Es una lista de strings.

```
»»»>>> import sys
»»»>>> print(sys.path)
»»» ['', '/usr/lib/python3.10', '/usr/lib/python3.10/lib-dynload', ...]
```

»»» Si necesitas que Python encuentre un modulo en una carpeta personal, puedes agregarla:

```
»»»>>> sys.path.append("/home/nahuel/mis_modulos")
```

»»» `sys.stdin`, `sys.stdout`, `sys.stderr`

»»» Estos son los flujos de entrada, salida y error. Por defecto: - `sys.stdin` es el teclado (lo que lee `input()`). - `sys.stdout` es la pantalla (donde va `print()`). - `sys.stderr` tambien es la pantalla, pero separado para errores.

»»» Puedes usarlos directamente:

```
»»»>>> sys.stdout.write("Hola mundo\n")
»»» Hola mundo
»»»>>> sys.stderr.write("Esto es un error\n")
»»» Esto es un error
```

»»» Leer desde `sys.stdin` te sirve para procesar datos que vienen de un pipe (por ejemplo, `cat archivo.txt | python script.py`):

```
»»»>>> for linea in sys.stdin:
»»» ...     print(f"Lei: {linea.strip()}")
```


»»»sys.version y sys.platform

»»»Saber que version de Python y que sistema operativo estas usando puede ser util para compatibilidad.

```
»»»>>> print(sys.version)
»»»3.10.12 (main, ...)
»»»>>> print(sys.platform)
»»»linux # en Windows seria 'win32', en Mac 'darwin'
```

»»»sys.getrecursionlimit() y sys.setrecursionlimit()

»»»Python tiene un limite de recursividad (cuantas veces se puede llamar a si misma una funcion). Podes verlo y cambiarlo (con cuidado).

```
»»»>>> print(sys.getrecursionlimit())
»»»1000
»»»>>> sys.setrecursionlimit(2000)
```

»»»Ejemplos practicos

»»»Copia de diccionarios anidados

```
»»»import copy
»»»original = {"a": 1, "b": [2, 3]}
»»»copia_superficial = original.copy()
»»»copia_profunda = copy.deepcopy(original)
»»»
»»»copia_superficial["b"][0] = 99
»»»print(original["b"]) # [99, 3] (se modifiko)
»»»
»»»copia_profunda["b"][0] = 88
»»»print(original["b"]) # sigue [99, 3] (no se modifica)
```

»»»Script que lee desde la entrada estandar

»»»Este script funciona como 'grep' simple: filtra lineas que contienen una palabra.

```
»»»import sys
»»»
»»»if len(sys.argv) < 2:
»»»    print("Uso: python filtro.py <palabra>")
»»»    sys.exit(1)
»»»
»»»palabra = sys.argv[1]
```

```

»'''for linea in sys.stdin:
»'''    if palabra in linea:
»'''        sys.stdout.write(linea)

```

»'''Lo usas asi: 'cat archivo.txt | python filtro.py python'.

»'''20 ejercicios con soluciones paso a paso

»'''Ejercicio 1

»'''Demostrar que asignar una lista a otra variable no crea una copia.

»'''Solucion:

```

»'''a = [1,2,3]
»'''b = a
»'''b[0] = 99
»'''print(a)  # [99,2,3]

```

»'''Explicacion: 'b' es una referencia al mismo objeto que 'a'. Modificar 'b' modifica 'a'.

»'''Ejercicio 2

»'''Crear una copia superficial de una lista usando '[':']'.

»'''Solucion:

```

»'''original = [1,2,3]
»'''copia = original[:]
»'''copia[0] = 99
»'''print(original)  # [1,2,3] (sin cambios)
»'''print(copia)     # [99,2,3]

```

»'''Explicacion: El slice '[':']' crea una nueva lista con los mismos elementos.

»'''Ejercicio 3

»'''Crear una copia superficial con 'list()'.

»'''Solucion:

```

»'''copia = list(original)

```

»'''Ejercicio 4

»'''Crear una copia superficial con '.copy()'.

»'''Solucion:

```

»'''copia = original.copy()

```

»»Ejercicio 5

»»Usar 'copy.copy()' del modulo copy.

»»Solucion:

```
»»import copy
»»copia = copy.copy(original)
```

»»Ejercicio 6

»»Demostrar que la copia superficial no es suficiente para listas anidadas.

»»Solucion:

```
»»original = [1, [2,3]]
»»copia = original.copy()
»»copia[1][0] = 99
»»print(original) # [1, [99,3]] (se modifiko el interno)
```

»»Explicacion: La lista interior sigue siendo el mismo objeto compartido.

»»Ejercicio 7

»»Solucionar el problema anterior usando 'deepcopy'.

»»Solucion:

```
»»import copy
»»original = [1, [2,3]]
»»copia = copy.deepcopy(original)
»»copia[1][0] = 99
»»print(original) # [1, [2,3]] (sin cambios)
```

»»Ejercicio 8

»»Crear un diccionario anidado y aplicar 'deepcopy'.

»»Solucion:

```
»»original = {"a": {"b": 1}}
»»copia = copy.deepcopy(original)
»»copia["a"]["b"] = 2
»»print(original["a"]["b"]) # 1
»»print(copia["a"]["b"]) # 2
```

»»Ejercicio 9

»»Mostrar el contenido de 'sys.path'.

»»Solucion:

```
»»import sys
»»print(sys.path)
```

»»Ejercicio 10

»»Agregar una carpeta a 'sys.path'.

»»Solucion:

```
»»sys.path.append("/tmp/mis_modulos")
```

»»Ejercicio 11

»»Escribir "Hola" usando 'sys.stdout.write()' en lugar de 'print()'.

»»Solucion:

```
»»import sys
»»sys.stdout.write("Hola\n")
```

»»Ejercicio 12

»»Escribir un mensaje de error por 'sys.stderr'.

»»Solucion:

```
»»sys.stderr.write("Error: Algo salio mal\n")
```

»»Ejercicio 13

»»Leer una linea desde 'sys.stdin' (sin usar 'input()').

»»Solucion:

```
»»linea = sys.stdin.readline().strip()
```

»»Ejercicio 14

»»Mostrar la version de Python.

»»Solucion:

```
»»print(sys.version)
```

»»Ejercicio 15

»»Mostrar el nombre del sistema operativo.

»»Solucion:

```
»»print(sys.platform)
```

»»Ejercicio 16

»»Obtener el limite actual de recursion.

»»Solucion:

```
»»print(sys.getrecursionlimit())
```

»»Ejercicio 17

»»Aumentar el limite de recursion a 5000.

»»Solucion:

```
»»sys.setrecursionlimit(5000)
```

»»Ejercicio 18

»»Crear un script que lea todo 'sys.stdin' y lo imprima linea por linea.

»»Solucion:

```
»»import sys
»»for linea in sys.stdin:
»»    print(linea, end="")
```

»»Ejercicio 19

»»Usar 'sys.exit(1)' para terminar el programa con codigo de error.

»»Solucion:

```
»»if not archivo_existe:
»»    sys.exit(1)
```

»»Ejercicio 20

»»Demostrar que 'copy.deepcopy' funciona con objetos de clases personalizadas.

»»Solucion:

```

»»import copy
»»class Persona:
»»    def __init__(self, nombre):
»»        self.nombre = nombre
»»
»»p1 = Persona("Nahuel")
»»p2 = copy.deepcopy(p1)
»»p2.nombre = "Ana"
»»print(p1.nombre)  # Nahuel
»»print(p2.nombre)  # Ana

```

»»Explicacion: 'deepcopy' crea una instancia nueva e independiente.

»»Resumen de lo que vimos

»»Dale, repasemos:

- La asignacion '=' no crea una copia, solo una nueva referencia al mismo objeto.
- Copia superficial (shallow): se crea un nuevo contenedor, pero los elementos internos se comparten. Formas: '[:]', 'list()', '.copy()', 'copy.copy()'.
- Copia profunda (deep): se copia todo recursivamente. 'copy.deepcopy()'. Esencial para estructuras anidadas.
- 'sys.path': lista de rutas donde Python busca modulos. Se puede modificar.
- 'sys.stdin', 'sys.stdout', 'sys.stderr': flujos de entrada, salida y error.
- 'sys.version', 'sys.platform': informacion del entorno.
- 'sys.getrecursionlimit()' / 'setrecursionlimit()': control de recursividad.

»»Reflexion final

»»Entender la diferencia entre asignacion, copia superficial y copia profunda te ahorrara horas de debugging. Es un concepto sutil pero fundamental. Y conocer 'sys' te da el control sobre el entorno de ejecucion. Son herramientas que parecen chiquitas pero marcan la diferencia entre un script casero y un programa robusto.

»»RecordÃ¡: si modificar una lista copia te modifica la original, es porque hiciste una asignacion directa. Si te pasa con listas anidadas, es porque usaste copia superficial y necesitas 'deepcopy'.

»»_____ Copia con conciencia, no por referencia.

»'''Recursividad: Cuando una funcion se llama a si misma

»'''Querido lector: el error que me hizo entender la recursividad

»'''Cuando yo empecé a programar, me costó un montón entender la recursividad. La primera vez que vi una función que se llamaba a sí misma, pensé: "esto es un loop infinito, se va a trabar". Y tuve razón... hasta que le agregué el caso base.

»'''Una analogía que me ayudó: imagínate dos espejos enfrentados. Cada uno refleja al otro, y esa imagen se repite infinitamente. Eso es la recursividad sin caso base. Ahora imagínate que pones una mano entre los espejos: la imagen se repite pero cada vez más chica hasta que desaparece. Ese "desaparece" es el caso base.

»'''En este capítulo te voy a enseñar a usar la recursividad para resolver problemas que tienen una estructura naturalmente repetitiva. Vamos paso a paso, con ejemplos y muchos ejercicios.

»'''Que es la recursividad y por que usarla

»'''Una función recursiva es una función que se llama a sí misma. Si no tiene una condición que la detenga, se llama infinitamente y Python te tira un error 'RecursionError' (porque protege la memoria).

»'''Partes de una función recursiva

»'''Toda función recursiva necesita:

- **Caso base:** la condición que detiene la recursión. Es el caso más simple, que se resuelve sin llamar a la función de nuevo.
- **Caso recursivo:** la función se llama a sí misma con un argumento "más chico." "más cerca" del caso base.

```
»'''def funcion_recursiva(parametro):
»'''    # Caso base
»'''    if condicion_de_parada:
```

```

>>>         return valor_base
>>>     # Caso recursivo
>>>     else:
>>>         return alguna_operacion(funcion_recursiva(parametro_modificado))

```

»»» Analogía con las muñecas rusas (matrioshkas)

»»» Cada muñeca contiene otra mas chica adentro. Abris una, ves otra, la abris, y asi hasta llegar a la mas chica que ya no tiene nada adentro. El caso base es la muñeca mas chica. Cada muñeca que abris es una llamada recursiva.

»»» Primer ejemplo: sumar numeros del 1 al N

»»» El primer problema que resolví con recursion fue sumar los numeros del 1 al N. Con un 'for' es facil, pero con recursion se ve la estructura.

```

>>>def suma_hasta(n):
>>>    # Caso base: si n es 1, la suma es 1
>>>    if n == 1:
>>>        return 1
>>>    # Caso recursivo: n + suma de los anteriores
>>>    else:
>>>        return n + suma_hasta(n - 1)
>>>
>>>>> print(suma_hasta(5))
>>>15    # 5+4+3+2+1

```

»»» La recursion va de mayor a menor: 'suma_hasta(5)' llama a 'suma_hasta(4)', que llama a 'suma_hasta(3)', has...

»»» Segundo ejemplo: cuenta regresiva con suma

»»» Aca la funcion imprime los numeros y devuelve la suma total.

```

>>>def cuenta_y_suma(n):
>>>    if n == 0:
>>>        return 0
>>>    else:
>>>        print(n)
>>>        return n + cuenta_y_suma(n - 1)
>>>
>>>>> total = cuenta_y_suma(5)
>>>5
>>>4
>>>3

```



```

»'2
»'1
»'>>> print(total)
»'15

```

»'Tercer ejemplo: contar letras de una palabra

»'La recursion tambien sirve para recorrer strings.

```

»'def contar_letras(palabra):
»'    # Caso base: string vacio
»'    if palabra == "":
»'        return 0
»'    # Caso recursivo: 1 + contar el resto
»'    else:
»'        return 1 + contar_letras(palabra[1:])
»'
»'>>> print(contar_letras("Hola"))
»'4

```

»'palabra[1:] es un slice que devuelve la palabra sin la primera letra. Asi vamos reduciendo el problema.

»'Cuarto ejemplo: factorial

»'El factorial es el ejemplo clasico de recursion en cualquier libro.

```

»'def factorial(n):
»'    # Caso base: factorial de 0 o 1 es 1
»'    if n <= 1:
»'        return 1
»'    # Caso recursivo: n * factorial(n-1)
»'    else:
»'        return n * factorial(n - 1)
»'
»'>>> print(factorial(5))
»'120

```

»'Quinto ejemplo: potencia

»'Calcular 'base ^{exponente} de forma recursiva.

```

»'''def potencia(base, exponente):
»'''    if exponente == 0:
»'''        return 1
»'''    else:
»'''        return base * potencia(base, exponente - 1)
»'''
»'''>>> print(potencia(2, 4))
»'''16
    
```

»'''Sexto ejemplo: fibonacci (pero con cuidado)

»'''La serie de Fibonacci es hermosa pero la recursion clasica es muy ineficiente porque recalcula los mismos valores muchas veces.

```

»'''def fibonacci(n):
»'''    if n == 0:
»'''        return 0
»'''    if n == 1:
»'''        return 1
»'''    return fibonacci(n - 1) + fibonacci(n - 2)
»'''
»'''>>> print(fibonacci(10))
»'''55
    
```

»'''**Advertencia:** esta funcion hace muchas llamadas. Para 'fibonacci(30)' ya es lenta. Para 'fibonacci(40)' puede tardar minutos. Mas adelante te muestro como optimizarla con memoizacion, pero por ahora sirve para entender la recursion.

»'''Tabla comparativa: recursion vs iteracion (for/while)

Caracteristica	Iteracion (for/while)	Recursion
Direccion	De menor a mayor	De mayor a menor
Memoria	Poca (una variable)	Mucha (pila de llamadas)
Claridad	Simple y directo	Elegante pero complejo
Riesgo	Bucle infinito	RecursionError (maxima profundidad)
Velocidad	Rapida	Mas lenta (sobrecarga de llamadas)

»'''20 ejercicios de recursion con soluciones paso a paso

»'''Ahora te toca a vos. Resuelve cada ejercicio antes de mirar la solucion.

»»Ejercicio 1: suma de digitos de un numero

»»Escribe una funcion recursiva que sume los digitos de un numero entero positivo. Ejemplo: 'suma_digitos(123) = 1 + 2 + 3 = 6'.

»»Solucion:

```

def suma_digitos(n):
    if n < 10:
        return n
    else:
        return n % 10 + suma_digitos(n // 10)

print(suma_digitos(123)) # 6

```

»»**Explicacion:** El caso base es cuando el numero tiene un solo digito. En el recursivo, tomo el ultimo digito ('n

»»Ejercicio 2: invertir una cadena

»»Escribe una funcion recursiva que devuelva una cadena invertida. Ejemplo: 'invertir("hola") -> .^loh.

»»Solucion:

```

def invertir(cadena):
    if cadena == "":
        return ""
    else:
        return invertir(cadena[1:]) + cadena[0]

print(invertir("hola")) # aloh

```

»»**Explicacion:** Tomo el primer caracter y lo pongo al final del resultado de invertir el resto.

»»Ejercicio 3: contar vocales en una cadena

»»Escribe una funcion recursiva que cuente cuantas vocales (a,e,i,o,u) tiene una cadena (sin distinguir mayusculas/minusculas).

»»Solucion:

```

def contar_vocales(cadena):
    if cadena == "":
        return 0
    primera = cadena[0].lower()
    es_vocal = primera in "aeiou"
    return (1 if es_vocal else 0) + contar_vocales(cadena[1:])

print(contar_vocales("Hola mundo")) # 4

```

»»»**Explicacion:** Verifico si la primera letra es vocal, sumo 1 o 0, y luego cuento las vocales del resto.

»»»Ejercicio 4: potencia recursiva optimizada (exponente par)

»»»La potencia que vimos antes hace exponente pasos. Optimizala para que cuando el exponente sea par, haga $(base^{exponente/2})^2$.

»»»**Solucion:**

```

def potencia_rapida(base, exp):
    if exp == 0:
        return 1
    if exp % 2 == 0:
        mitad = potencia_rapida(base, exp // 2)
        return mitad * mitad
    else:
        return base * potencia_rapida(base, exp - 1)

print(potencia_rapida(2, 10)) # 1024

```

»»»**Explicacion:** Esta es la recursion usada en algoritmos eficientes. Para exponente par, divido el problema a la mitad.

»»»Ejercicio 5: maximo elemento de una lista

»»»Encuentra el valor maximo en una lista de numeros usando recursion.

»»»**Solucion:**

```

def maximo_lista(lista):
    if len(lista) == 1:
        return lista[0]
    else:
        max_resto = maximo_lista(lista[1:])
        return lista[0] if lista[0] > max_resto else max_resto

print(maximo_lista([3, 7, 2, 9, 5])) # 9

```

»»»**Explicacion:** Comparo el primer elemento con el maximo del resto.

»»»Ejercicio 6: palindromo

»»»Determina si una cadena es palindromo (se lee igual al derecho y al revés) usando recursion. Ejemplo: "neuquen.es" palindromo, "hola"no.

»»»**Solucion:**

```

»»»def es_palindromo(cadena):
»»»    cadena = cadena.lower().replace(" ", "")
»»»    if len(cadena) <= 1:
»»»        return True
»»»    if cadena[0] != cadena[-1]:
»»»        return False
»»»    return es_palindromo(cadena[1:-1])
»»»
»»»print(es_palindromo("neuquen"))    # True
»»»print(es_palindromo("anita lava la tina"))    # True

```

»»»**Explicacion:** Comparo el primer y ultimo caracter. Si son iguales, llamo a la funcion con la cadena sin esos dos.

»»»Ejercicio 7: producto de dos numeros usando solo sumas

»»»Calcula 'a * b' mediante sumas recursivas (sin usar el operador '*').

»»»**Solucion:**

```

»»»def producto(a, b):
»»»    if b == 0:
»»»        return 0
»»»    if b > 0:
»»»        return a + producto(a, b - 1)
»»»    else:
»»»        return -producto(a, -b)    # para b negativo
»»»
»»»print(producto(5, 3))    # 15
»»»print(producto(5, -3))    # -15

```

»»»**Explicacion:** Multiplicar es sumar 'a' repetido 'b' veces.

»»»Ejercicio 8: division entera usando restas

»»»Calcula el cociente de la division entera 'a // b' usando restas recursivas.

»»»**Solucion:**

```

»»»def division_entera(a, b):
»»»    if a < b:
»»»        return 0
»»»    return 1 + division_entera(a - b, b)
»»»
»»»print(division_entera(10, 3))    # 3

```

»»»**Explicacion:** Resto 'b' hasta que 'a' sea menor que 'b', contando cuantas veces puedo restar.

»»Ejercicio 9: convertir numero a binario (string)

»»Devuelve la representacion binaria de un entero positivo.

»»Solucion:

```

def a_binario(n):
    if n == 0:
        return "0"
    if n == 1:
        return "1"
    return a_binario(n // 2) + str(n % 2)

print(a_binario(13)) # "1101"

```

»»Explicacion: Voy dividiendo por 2 y concatenando el resto.

»»Ejercicio 10: buscar elemento en lista (busqueda lineal recursiva)

»»Devuelve el indice de la primera aparicion de un elemento, o -1 si no esta.

»»Solucion:

```

def buscar(lista, x, indice=0):
    if indice >= len(lista):
        return -1
    if lista[indice] == x:
        return indice
    return buscar(lista, x, indice + 1)

print(buscar([5,2,8,1], 8)) # 2
print(buscar([5,2,8,1], 9)) # -1

```

»»Explicacion: Recorro la lista con un indice que aumenta en cada llamada.

»»Ejercicio 11: torres de Hanoi (contar movimientos)

»»Calcula cuantos movimientos se necesitan para mover N discos de una torre a otra. La formula es $2^n - 1$.

»»Solucion:

```

def hanoi_movimientos(n):
    if n == 1:
        return 1
    return 2 * hanoi_movimientos(n - 1) + 1

print(hanoi_movimientos(3)) # 7

```

»»Explicacion: Para mover N discos, primero muevo N-1 a la torre auxiliar (movimientos), luego muevo el mas grande (1), luego muevo los N-1 a la torre destino (movimientos).

»»Ejercicio 12: mostrar el proceso de las torres de Hanoi

»»Imprime los pasos para resolver las torres de Hanoi.

»»Solucion:

```

def hanoi_pasos(n, origen, destino, auxiliar):
    if n == 1:
        print(f"Mover disco 1 de {origen} a {destino}")
    else:
        hanoi_pasos(n-1, origen, auxiliar, destino)
        print(f"Mover disco {n} de {origen} a {destino}")
        hanoi_pasos(n-1, auxiliar, destino, origen)
hanoi_pasos(3, "A", "C", "B")

```

»»Explicacion: Los parametros indican de donde a donde y cual es la torre auxiliar.

»»Ejercicio 13: calculadora recursiva de expresiones simples

»»Dada una lista con numeros y operadores (ej: '[1, '+', 2, '*', 3]'), evaluar la expresion respetando la prioridad de operadores. (Version simplificada sin parentesis). Usa recursion para separar por el operador de menor prioridad.

»»Solucion:

```

def evaluar(expr):
    if len(expr) == 1:
        return expr[0]
    # buscar primer + o - (menor prioridad)
    for i, token in enumerate(expr):
        if token in ('+', '-'):
            izquierda = evaluar(expr[:i])
            derecha = evaluar(expr[i+1:])
            if token == '+':
                return izquierda + derecha
            else:
                return izquierda - derecha
    # si no hay + o -, buscar * o /
    for i, token in enumerate(expr):
        if token in ('*', '/'):
            izquierda = evaluar(expr[:i])
            derecha = evaluar(expr[i+1:])
            if token == '*':
                return izquierda * derecha
            else:
                return izquierda / derecha

```

»»Explicacion: Es un ejemplo avanzado que muestra como se construye un parser recursivo.

»»Ejercicio 14: generar todas las combinaciones de una lista

»»Dada una lista, devuelve todas las sublistas posibles (incluyendo vacia). Ejemplo: 'combinaciones([1,2]) ->[[], [1], [2], [1,2]]'.

»»Solucion:

```

def combinaciones(lista):
    if not lista:
        return [[]]
    sin_primer = combinaciones(lista[1:])
    con_primer = [[lista[0]] + c for c in sin_primer]
    return sin_primer + con_primer

print(combinaciones([1,2]))

```

»»Explicacion: O tomo el primer elemento o no lo tomo, y combino con las combinaciones del resto.

»»Ejercicio 15: permutaciones de una cadena

»»Genera todas las permutaciones de una cadena. Ejemplo: 'permutaciones(.ab") ->[.ab", "ba"]'.

»»Solucion:

```

def permutaciones(cadena):
    if len(cadena) <= 1:
        return [cadena]
    res = []
    for i, letra in enumerate(cadena):
        resto = cadena[:i] + cadena[i+1:]
        for p in permutaciones(resto):
            res.append(letra + p)
    return res

print(permutaciones("abc"))

```

»»Explicacion: Tomo cada letra como primera y permuto el resto.

»»Ejercicio 16: busqueda binaria recursiva

»»Dada una lista ordenada y un elemento, devuelve el indice donde se encuentra (o -1). Usa recursion.

»»Solucion:

```

def busqueda_binaria(lista, x, inicio=0, fin=None):
    if fin is None:

```



```

»»»         fin = len(lista) - 1
»»»     if inicio > fin:
»»»         return -1
»»»     medio = (inicio + fin) // 2
»»»     if lista[medio] == x:
»»»         return medio
»»»     elif lista[medio] < x:
»»»         return busqueda_binaria(lista, x, medio+1, fin)
»»»     else:
»»»         return busqueda_binaria(lista, x, inicio, medio-1)
»»»
»»» print(busqueda_binaria([1,3,5,7,9], 5)) # 2

```

»»»Ejercicio 17: copia profunda de listas anidadas

»»»Implementa una funcion recursiva que copie una lista que puede contener listas anidadas (como 'copy.deepcopy').

»»»Solucion:

```

»»»def copia_profunda(obj):
»»»    if not isinstance(obj, list):
»»»        return obj
»»»    return [copia_profunda(elem) for elem in obj]
»»»
»»»original = [1, [2, [3, 4]], 5]
»»»copia = copia_profunda(original)
»»»copia[1][0] = 99
»»»print(original) # [1, [2, [3, 4]], 5] (no se modifiko)
»»»print(copia)   # [1, [99, [3, 4]], 5]

```

»»»Ejercicio 18: calcular el n-esimo numero de la serie de Lucas

»»»La serie de Lucas es como Fibonacci pero empieza con L0=2, L1=1, y $L_n = L_{n-1} + L_{n-2}$. Hazla recursiva.

»»»Solucion:

```

»»»def lucas(n):
»»»    if n == 0:
»»»        return 2
»»»    if n == 1:
»»»        return 1
»»»    return lucas(n-1) + lucas(n-2)
»»»
»»»print(lucas(5)) # 11

```

»»Ejercicio 19: longitud de una lista (sin usar len)

»»Cuenta la cantidad de elementos de una lista usando recursion.

»»Solucion:

```

def longitud(lista):
    if not lista:
        return 0
    return 1 + longitud(lista[1:])

print(longitud([1,2,3,4])) # 4

```

»»Ejercicio 20: aplanar lista anidada

»»Convierte una lista que puede tener listas anidadas en una lista plana. Ej: '[1, [2, [3, 4]] -> [1,2,3,4]'.

»»Solucion:

```

def aplanar(lista):
    resultado = []
    for elem in lista:
        if isinstance(elem, list):
            resultado.extend(aplanar(elem))
        else:
            resultado.append(elem)
    return resultado

print(aplanar([1, [2, [3, 4]], 5])) # [1,2,3,4,5]

```

»»**Explicacion:** Recorro cada elemento; si es lista, llamo recursivamente y extendiendo; si no, lo agrego.

»»Resumen de lo que vimos

»»Dale, repasemos:

- La recursion es una funcion que se llama a si misma.
- Toda funcion recursiva debe tener un **caso base** que la detiene.
- El **caso recursivo** reduce el problema y vuelve a llamar a la funcion.
- Sirve para problemas con estructura recursiva natural: arboles, fractales, torres de Hanoi, etc.
- Es menos eficiente que la iteracion para problemas simples, pero mas elegante para problemas complejos.
- Cuidado con la profundidad maxima de recursion (por defecto 1000, puedes cambiarla con 'sys.setrecursionlimit').

»'''Reflexion final

»'''Mira, la recursion no es algo que uses todos los dias. Pero entenderla te cambia la forma de pensar los problemas. Cuando te enfrentes a un problema que se pueda dividir en subproblemas del mismo tipo, la recursion es tu amiga.

»'''No te obsesiones con usarla para todo. Un 'for' bien hecho suele ser mas claro y rapido. Pero saber recursion te hace mejor programador.

»'''Sigue practicando con los ejercicios. Prueba modificar los ejemplos, rompelos, arregalos. Es la unica manera de aprender.

»'''_____ *La recursion es como mirarte en un espejo frente a otro espejo: al principio parece infinito, hasta que encuentras el borde.*

»'''Buenas practicas y como seguir aprendiendo - Tu proximo paso

»'''Querido lector: esto no es un adios

»'''Cuando yo empecé a programar, terminé el primer tutorial y me quede mirando la pantalla sin saber que hacer. Sabia lo que era un 'if', un 'for', una lista... pero no tenia idea de como armar un proyecto real. Me faltaban las buenas practicas y el norte para seguir.

»'''Este ultimo capitulo es justo eso: un mapa para que no te pierdas despues de cerrar este libro. Vamos a repasar las buenas practicas que deberias adoptar desde ahora, los caminos que puedes tomar, los recursos para seguir aprendiendo y, como siempre, ejercicios para que no te duermas.

»'''Buenas practicas (el resumen final que me hubiera gustado tener)

»'''Escribe codigo legible antes que codigo inteligente

»'''El primer error que cometí fue querer hacer codigo "inteligente": lineas de una sola con expresiones complejas. Mi yo del futuro no entendia nada.

```
»'''# Mal: "inteligente" pero ilegible
»'''r = lambda x: x**2
»'''
»'''# Bien: claro y simple
»'''def cuadrado(x):
»'''    return x ** 2
```

»'''Usa nombres que tengan sentido

»'''Tu codigo se lee mas veces de las que se escribe. Los nombres de variables son como los nombres de las personas: te dicen quien es.

```
»'''# Mal
»'''d = 5
```

```

»'t = "Nahuel"
»'
»'# Bien
»'dias_restantes = 5
»'nombre_usuario = "Nahuel"

```

»'Comenta el "por que", no el "que"

»'Los comentarios que dicen lo obvio ensucian. Comenta la razon de una decision.

```

»'# Mal: explica lo obvio
»'# Suma 1 al contador
»'contador += 1
»'
»'# Bien: explica por que
»'# Reintentamos la conexion despues de un timeout
»'contador_reintentos += 1

```

»'Funciones pequenas y con una sola responsabilidad

»'Una funcion debe hacer una cosa y hacerla bien. Si el nombre tiene un "z", es demasiado.

```

»'# Mal: una funcion que hace todo
»'def procesar_datos(datos):
»'    # limpia, procesa, guarda, notifica
»'
»'# Bien: separado
»'def limpiar_datos(datos): pass
»'def procesar_datos(datos): pass
»'def guardar_resultado(datos): pass
»'def notificar(): pass

```

»'No repitas codigo (DRY - Don't Repeat Yourself)

»'Si estas copiando y pegando el mismo codigo en varios lugares, es hora de hacer una funcion. La analogia: no tengas 20 velas si podes tener una que enciendas 20 veces.

»'Maneja errores explicitamente

»'except: a secas atrapa todo, incluso errores que no esperabas. Es como ponerse una venda en los ojos. Siempre especifica que error capturas.

```

»'# Mal
»'try:
»'    algo()

```

```

>>>except:
>>>    pass
>>>
>>># Bien
>>>try:
>>>    algo()
>>>except ValueError:
>>>    print("Error especifico")
>>>except ZeroDivisionError:
>>>    print("Division por cero")
    
```

»»»Usa entornos virtuales para cada proyecto

»»»Nunca instales librerias globalmente sin necesidad. Cada proyecto merece su propio cuarto limpio". Aprendelo desde ahora.

»»»Escribe pruebas (tests)

»»»Aunque sean simples, proba tu codigo. Un 'assert' te puede salvar de una pesadilla.

```

>>>def test_suma():
>>>    assert suma(2, 3) == 5
    
```

»»»VERSIONA TU CODIGO CON GIT

»»»Aprendelo. Usalo. Todos los dias. No es negociable. Empeza con 'git init', 'git add', 'git commit' y subi tus proyectos a GitHub. Es tu portfolio.

»»»Que aprender ahora (los caminos que se abren)

»»»Depende de lo que te guste. Te cuento las rutas mas comunes.

»»»Si te gusta la ciencia de datos / inteligencia artificial

- **NumPy**: arrays y operaciones matriciales. Es la base de todo.
- **Pandas**: DataFrames (como Excel pero en Python).
- **Matplotlib y Seaborn**: graficos.
- **Scikit-learn**: machine learning (regresion, clasificacion).
- **Jupyter Notebooks**: para explorar datos interactivamente.

»»»Si te gusta desarrollo web

- **Flask** (microframework) o **Django** (full stack).
- HTML, CSS, JavaScript basico.
- Bases de datos (SQLite, PostgreSQL).
- APIs con FastAPI.

»»»Si te gusta automatizacion y scripting

- Automatizar tareas con 'os', 'shutil', 'subprocess'.
- Web scraping con 'requests' y 'BeautifulSoup'.
- Selenium para automatizar navegadores.
- 'schedule' para tareas programadas.

»»»Si te gustan los videojuegos

- Pygame.
- Arcade (alternativa moderna).
- Conceptos de game loops, sprites, colisiones.

»»»Temas transversales (para cualquier camino)

»»»Esto es lo que diferencia a un programador amateur de uno profesional:

- Git y GitHub.
- Pruebas unitarias ('unittest', 'pytest').
- 'logging' (en lugar de 'print').
- 'argparse' para argumentos de linea de comandos.
- Decoradores y context managers.

»»»Recursos para seguir aprendiendo

»»»Documentacion oficial (siempre la fuente principal)

»»»<https://docs.python.org/3/>

»»»Tutoriales y cursos (gratis y en espanol)

- Real Python (en ingles, excelente).
- freeCodeCamp (Python para principiantes).
- W3Schools Python.
- Programiz.

»»»Canales de YouTube (en espanol)

- **Pildoras Informaticas**: curso de Python muy completo.
- **HolaMundo**: proyectos practicos.
- **MoureDev**: retos semanales.

»»»Practicar con ejercicios

- Exercism.org (mentores humanos).
- Codewars (katas por niveles).
- HackerRank.
- LeetCode (para entrevistas tecnicas).

»»»Librerias populares

»» <https://pypi.org/>

»»»Comunidad

- Stack Overflow (pregunta y responde).
- Reddit: r/learnpython, r/Python.
- Discord de Python en espanol.

»»»Mantenerte actualizado

»»»Python cambia. Cada anio sale una version nueva con mejoras. Segu las novedades:

- Python.org news.
- PEPs (Python Enhancement Proposals).
- Blogs: Real Python, PyCoder's Weekly.

»»»20 ejercicios para seguir practicando (con soluciones paso a paso)

»»»Ahora te toca a vos. Estos ejercicios son para que apliques las buenas practicas.

»»»Ejercicio 1

»»»Refactoriza este codigo para que use nombres de variables claros: 'a = "Nahuel"; b = 25; print(a, b)'

»»»Solucion:

```
»»»nombre = "Nahuel"
»»»edad = 25
»»»print(nombre, edad)
```

»»»Explicacion: 'a' y 'b' no dicen nada. 'nombre' y 'edad' son autoexplicativos.

»»»Ejercicio 2

»»»Extrae una funcion de este codigo repetido (calcula el area de un circulo dos veces):

»»»Donde seguir despues de este libro - Recursos, bibliografia y el camino que viene

»»»Querido lector, esto no es un adios

»»»Llegaste hasta aca. Ya leiste (y programaste) un monton. Tienes una base solida en Python. Pero el aprendizaje no termina. De hecho, recién empieza.

»»»En este anteultimo capitulo quiero compartirte recursos especialmente pensados para quienes estudian o trabajan en ciencias exactas y naturales: fisica, matematica, biologia, geologia y todo el mundo de la facultad. Pero tambien para cualquier persona que quiera seguir creciendo.

»»»Vamos a ver:

- Donde buscar documentacion oficial y de confianza.
- Libros para cada area cientifica.
- Cursos gratis y de pago.
- Canales de YouTube que te acompanan sin aburrirte.
- Comunidades para preguntar sin miedo.
- Herramientas que uso todos los dias.
- Librerias especificas para ciencias.
- Plataformas para practicar.
- Un plan de estudio para no perderte despues de este libro.

»»»Dale, empecemos.

»»»Documentacion oficial: tu mejor amiga

»»»Cuando yo empecé, la documentacion oficial me daba miedo. Mucho texto, muchos terminos raros. Pero la verdad es que ahí está todo. Es como el manual de instrucciones de un auto: no lo lees entero, pero cuando tienes una duda especifica, lo abris y encontras la respuesta.

»»Python.org (docs.python.org/3/)

»»<https://docs.python.org/3/>

»»Es la fuente de la verdad. Tiene:

- Tutorial paso a paso (ideal para repasar algun tema concreto).
- Referencia de la biblioteca estandar (todo lo que viene con Python sin instalar nada).
- Guias de instalacion para Windows, Linux y Mac.

»»**Consejo que me salvo:** usa el buscador de la pagina. No leas de arriba a abajo. Busca 'list.append', 'os.getcwd', 'try except', y te lleva directo al grano.

»»PEPs (Python Enhancement Proposals)

»»Los PEPs son las propuestas de mejora de Python. Los mas importantes:

- PEP 8 – Guia de estilo: <https://peps.python.org/pep-0008/>
- PEP 20 – El Zen de Python: <https://peps.python.org/pep-0020/>

»»El Zen de Python son 19 frases que resumen la filosofia del lenguaje. Una dice: *"Bello es mejor que feo. Explicito es mejor que implicito. Simple es mejor que complejo."* Vale la pena leerlo completo al menos una vez.

»»PyPI (Python Package Index)

»»<https://pypi.org/>

»»Ahi estan todas las librerias que puedes instalar con 'pip'. Si necesitas algo, buscalo aca primero. Cada paquete tiene su documentacion, ejemplos y la cantidad de descargas (asi sabes si es popular y confiable).

»»Librerias especificas para ciencias (el tesoro que no sabias que necesitabas)

»»Python tiene un ecosistema enorme para ciencias. Te voy a listar las mas importantes por disciplina.

»»»Para matematicas y fisica

```
»»»pip install numpy sympy scipy matplotlib
```

- **NumPy**: arrays multidimensionales y algebra lineal. Es la base de casi todo.
- **SymPy**: matematicas simbolicas. Podes derivar, integrar, resolver ecuaciones exactas.
- **SciPy**: funciones de optimizacion, integracion numerica, transformadas de Fourier, estadistica, etc.
- **Matplotlib**: graficos de todo tipo.

»»»Para biologia y bioinformatica

```
»»»pip install biopython pandas scipy matplotlib seaborn
```

- **Biopython**: leer formatos de archivos biologicos (FASTA, GenBank, PDB), hacer BLAST local, trabajar con secuencias.
- **pandas**: para manejar tablas de datos (por ejemplo, tablas de expresion genica).
- **scikit-bio**: herramientas adicionales para bioinformatica.

»»»Para geologia y geociencias

```
»»»pip install geopandas rasterio folium mapclassify
```

- **GeoPandas**: trabajar con datos geoespaciales (mapas, shapefiles).
- **Rasterio**: leer y escribir archivos raster (imagenes satelitales, modelos de elevacion).
- **Folium**: mapas interactivos en notebooks.

»»»Para quimica

```
»»»pip install rdkit-pypi chempy ase
```

- **RDKit**: quimica computacional, moléculas, fingerprints.
- **chempy**: herramientas quimicas basicas (estequiometria, ecuaciones).
- **ASE**: Atomic Simulation Environment (materiales, DFT).

»»»Para ciencia de datos y estadistica (todas las disciplinas)

```
»»»pip install pandas seaborn scikit-learn statsmodels
```

- **pandas:** DataFrames, la navaja suiza de datos tabulares.
- **Seaborn:** graficos estadisticos hermosos y faciles.
- **scikit-learn:** machine learning (clasificacion, regresion, clustering).
- **statsmodels:** modelos estadisticos (regresion lineal, ANOVA, series temporales).

»»»Ejecutar codigo interactivo: Jupyter

»»»Para explorar datos y hacer graficos al vuelo, nada mejor que Jupyter:

```
»»»pip install jupyterlab
»»»jupyter lab
```

»»»Abre un navegador y puedes mezclar codigo, texto, graficos. Es lo que usan los cientificos de datos hoy.

»»»Libros que me hubiera gustado tener al lado

»»»Cuando empecé, no sabía que libro comprar. Algunos eran demasiado teóricos, otros muy viejos. Aquí te dejo los que con los años demostraron ser los mejores.

»»»Para arrancar de cero (sin dolor de cabeza)

- **“Automate the Boring Stuff with Python” de Al Sweigart.** Disponible gratis en <https://automatetheboringstuff.com/> Es el libro que recomiendo siempre. Aprendes haciendo cosas útiles: renombrar archivos en masa, extraer datos de un PDF, mandar mails automáticos. No necesitas ser matemático ni ingeniero.
- **“Python Crash Course” de Eric Matthes.** La primera parte es lo mismo que viste en este manual pero más resumido. La segunda parte tiene tres proyectos grandes: un juego (Pygame), visualización de datos (matplotlib) y una aplicación web (Django). Muy práctico.

»»»Para científicos (matemática, física, ingeniería)

- **“A Primer on Scientific Programming with Python” de Hans Petter Langtangen.** Es un ladrillo, pero tiene TODO lo que necesitas para modelado numérico y simulaciones.
- **“Python for Scientists” de John M. Stewart.** Cortito, va al grano. Cubre NumPy, SciPy, Matplotlib, y aplicaciones a ecuaciones diferenciales.

- **“Effective Computation in Physics”** de **Anthony Scopatz y Kathryn D. Huff**. Para fisicos que quieren escribir codigo serio. Cubre desde shell hasta git y testing.

»»»Para biologia/bioinformatica

- **“Bioinformatics Programming Using Python”** de **Mitchell L. Model**. Excelente para aprender a procesar formatos geneticos y usar Biopython.
- **“Python for the Life Sciences”** de **Alexander Lancaster y Gordon Webster**. Mas moderno, con ejemplos reales de biologia.

»»»Para geologia y geociencias

- **“Geoprocessing with Python”** de **Chris Garrard**. Usa GeoPandas, Shapely, Fiona. Ideal para GIS.
- **“Python for Geospatial Data Analysis”** de **Bonny P. McClain**. Cubre raster, vector, mapas interactivos.

»»»Cuando ya te sientas comodo y quieras profundizar

- **“Fluent Python”** de **Luciano Ramalho**. Este es el libro que separa al que “sabe Python” del que “entiende Python”. Cubre decoradores, generadores, context managers, metaclasses. Pero no lo leas ahora. Guardalo para dentro de unos meses.
- **“Python Cookbook”** de **David Beazley y Brian Jones**. Es un libro de recetas: tenes un problema concreto (ej: leer un archivo enorme, trabajar con fechas, hacer scraping), y te da una solucion explicada. Ideal para tener abierto mientras programas.

»»»Libros en espanol (para los que les cuesta el ingles)

- **“Piensa en Python”** de **Allen Downey**. Traducccion de “Think Python”. Es el libro que usan en muchas universidades. Muy didactico y con muchos ejercicios. Descargable gratis desde la web del autor.
- **“Python a fondo”** de **J. Chazarra y F. Garcia**. Mas tecnico, orientado a programadores que ya saben otros lenguajes. Pero si venis de este manual, lo puedes leer sin problemas.

»»»Cursos online: gratis y de pago (y cuales valen la pena)

»»»Cuando yo empecé, me anote en tres cursos pagos que eran una porqueria. No cometes ese error. Aca van los que realmente sirven.

»»»Gratuitos (y excelentes)

- **Python for Everybody (Coursera, Universidad de Michigan).** <https://www.coursera.org/specializations/python> El profesor es Charles Severance ("Dr. Chuck"). Explica como si estuvieras tomando un cafe con el. Podes auditar todo gratis (sin certificado).
- **CS50's Introduction to Programming with Python (edX, Harvard).** <https://www.edx.org/course/cs50s-introduction-to-programming-with-python> Impartido por David Malan. Es exigente, pero salis sabiendo de verdad.
- **freeCodeCamp - Scientific Computing with Python.** <https://www.freecodecamp.org/learn/scientific-computing-with-python/> Son 400 horas de contenido con proyectos incluidos. Nada de vueltas: escribes codigo desde el minuto uno.

»»»De pago (solo si quieres certificado o seguimiento)

- **100 Days of Code: The Complete Python Pro Bootcamp (Udemy).** Es largo (100 dias), pero muy completo. Espera a que este en oferta (cada dos semanas baja a 12-15 dolares).
- **Google IT Automation with Python Professional Certificate (Coursera).** Si te interesa la automatizacion y la administracion de sistemas, es perfecto. Incluye manejo de archivos, expresiones regulares, APIs, etc.

»»»Canales de YouTube: aprender viendo codigo en accion

»»»Mira, hay muchisimo contenido basura en YouTube. Pero estos canales son oro puro.

»»»En espanol

- **Pildoras Informaticas:** <https://www.youtube.com/@pildorasinformaticas> Mas de 100 videos, desde cero hasta temas avanzados. Habla claro, pausado, y no asume que ya sabes nada.
- **HolaMundo:** <https://www.youtube.com/@HolaMundo> Proyectos practicos, consejos y tutoriales modernos. Muy entretenido, no te aburres.
- **MoureDev:** <https://www.youtube.com/@MoureDev> Brais es un crack. Tiene un "retos de programacion" semanal que te obliga a pensar. Y cursos completos gratuitos.

»»»En ingles (aprovecha para practicar el idioma)

- **Corey Schafer:** <https://www.youtube.com/@coreyms> Es el mejor canal de Python en ingles. Explica decoradores, generadores, `if name == "main"` *de una manera que se entiende a la primera.*

- **Sentdex:** <https://www.youtube.com/@sentdex> Proyectos reales: machine learning, juegos, finanzas, hacking etico. Sube contenido muy seguido.
- **ArjanCodes:** <https://www.youtube.com/@ArjanCodes> Se enfoca en codigo limpio, patrones de diseno y buenas practicas. Ideal para cuando quieras que tu codigo sea profesional.

»»Para científicos

- **Python Programmer:** <https://www.youtube.com/@pythonprogrammer> Data science y machine learning con ejemplos practicos.
- **Sentdex (nuevamente):** tiene series especificas de NumPy, pandas, y matplotlib.
- **Coding with Ryan:** ejemplos de fisica computacional (simulaciones, caida libre, pendulo).

»»Comunidades: donde preguntar sin miedo a quedar como un tonto

»»Cuando yo empecé, tenía miedo de preguntar boludeces. Después aprendí que todos pasamos por eso. En estas comunidades la gente ayuda de verdad.

- **Python Argentina:** <https://www.python.org.ar/> Tienen grupos de Telegram y Slack, y hacen reuniones virtuales. Son muy piolas.
- **r/learnpython (Reddit):** <https://www.reddit.com/r/learnpython/> Miles de principiantes y expertos respondiendo preguntas a diario. No importa si tu pregunta es muy basica, siempre alguien responde con buena onda.
- **Python Discord:** <https://pythondiscord.com/> La comunidad mas grande y activa. Tienen canales por nivel (desde “principiantes” hasta “avanzados”). Además, hay canales tematicos (web, datos, juegos).
- **Stack Overflow:** <https://stackoverflow.com/questions/tagged/python> Es la base de conocimiento definitiva. Antes de preguntar, busca porque seguro alguien ya tuvo tu misma duda. Y si preguntas, muestra lo que intentaste.

»»Comunidades científicas

- **Biopython mailing list:** https://biopython.org/wiki/Mailing_lists Para preguntas sobre Bioinformatica.
- **GeoPython:** grupo en Telegram y conferencia anual.
- **PyData:** grupos locales para ciencia de datos.

»»Herramientas esenciales para el dia a dia

»»No te voy a mentir: al principio usaba el bloc de notas para escribir codigo. No lo hagas. Estas herramientas te hacen la vida mucho mas facil.

»»Editores de codigo (IDEs)

- **VS Code:** <https://code.visualstudio.com/> Es el mas popular hoy. Liviano, gratuito, con extensiones para Python, depuracion, Jupyter y Git. Instala las extensiones "Python" (de Microsoft) y "Pylance".
- **PyCharm Community:** <https://www.jetbrains.com/pycharm/> Mas pesado pero mucho mas completo. Te ayuda a seguir PEP 8, tiene refactorizaciones automaticas y un depurador visual excelente. La version Community es gratis.
- **Jupyter Notebook / JupyterLab:** <https://jupyter.org/> Ideal para explorar datos, hacer pruebas rapidas y compartir codigo explicado. Muy usado en ciencia de datos.

»»Manejo de entornos y paquetes (esto si que es importante)

- **venv** (viene con Python): crea entornos virtuales.

```
»»python -m venv mi_entorno
»»source mi_entorno/bin/activate    # Linux/Mac
»»mi_entorno\Scripts\activate      # Windows
```

- **pip:** instala paquetes.

```
»»pip install requests
»»pip freeze > requirements.txt    # guarda lo que tenes instalado
»»pip install -r requirements.txt  # instala todo de una
```

- **conda/miniconda:** muy usado en ciencia de datos. Maneja entornos y paquetes no Python (como bibliotecas C). Ideal para instalar NumPy, SciPy, etc.

»»Linters y formateadores (para tener codigo lindo sin pensar)

»»Configura tu editor para que ejecute esto cada vez que guardas:

- **black:** formatea automaticamente tu codigo siguiendo PEP 8.
- **isort:** ordena las importaciones.
- **flake8:** detecta errores de estilo y logica.

»»Ejemplo de como instalarlos y usarlos:

```
»'pip install black isort flake8
»'black mi_script.py
»'isort mi_script.py
»'flake8 mi_script.py
```

»'Plataformas para practicar (convierten el aprendizaje en un juego)

»'Cuando yo empecé, hacía ejercicios de un libro y nada más. Me aburría. Estas plataformas te dan desafíos con puntos, rankings y soluciones. Es adictivo.

- **Exercism:** <https://exercism.org/tracks/python> Recibes ejercicios y mentores humanos corrigen tu código. La retroalimentación es increíble. Gratis.
- **Codewars:** <https://www.codewars.com/> Desafíos llamados “katas”. Subes de rango al resolverlos. Muy adictivo. Empezas con katas de nivel 8 (el más fácil) y andas subiendo.
- **HackerRank:** <https://www.hackerrank.com/domains/python> Ideal para preparar entrevistas técnicas. Tiene desde sintaxis hasta algoritmos complejos.
- **LeetCode:** <https://leetcode.com/> Es el sitio favorito para preparar entrevistas en Google, Amazon, etc. Los problemas son más difíciles, pero te enseñan a pensar.
- **Project Euler:** <https://projecteuler.net/> Si te gusta la matemática, esto es para vos. Problemas que combinan mates y programación. Resuelves el primero y no puedes parar.

»'Podcasts, blogs y newsletters (para mantenerte al día sin esfuerzo)

»'Puedes aprender mientras viajas o haces ejercicio.

»'Podcasts en español

- **Python en Español:** <https://podcast.jaimejmr.com/> Entrevistas a desarrolladores y novedades del mundo Python.

»'Newsletters (te llegan por mail una vez por semana)

- **PyCoder's Weekly:** <https://pycoders.com/> Resumen con los artículos, proyectos y debates más relevantes. Lo leo todos los viernes.
- **Real Python Newsletter:** <https://realpython.com/newsletter/> Tutoriales nuevos y consejos cada semana.

»»Blogs recomendados

- **Real Python:** <https://realpython.com/> Articulos muy bien escritos, con ejemplos de codigo y explicaciones paso a paso.
- **Planet Python:** <https://planetpython.org/> Un agregador de blogs. Ahi encuentras de todo.
- **NumPy blog:** novedades sobre computacion numerica.

»»Plan de estudio sugerido (despues de este manual)

»»No quiero que termines el libro y te quedes sin saber que hacer. Aca va un camino posible.

»»Semana 1-2: Refuerzo

- Vuelve a hacer todos los ejercicios de este libro, pero sin mirar las soluciones.
- Elegi un proyecto pequeno de los integradores (agenda, carrito, ahorcado) y extendelo. Por ejemplo, agregale persistencia con archivos JSON.

»»Semana 3-4: Herramientas profesionales

- Aprende Git y GitHub (busca el tutorial de “HolaMundo” en YouTube). Crea un repositorio y sube todos tus codigos. Ese es tu portfolio.
- Instala VS Code, configura black, isort y flake8.
- Aprende a crear entornos virtuales y a usar ‘requirements.txt’.

»»Semana 5-8: Elige un camino cientifico

- **Fisica/Matematica:** Instala NumPy, SciPy, SymPy. Hace un script que resuelva una ecuacion diferencial o calcule la trayectoria de un proyectil con viento.
- **Bioinformatica:** Instala Biopython. Descarga una secuencia de la base de datos NCBI y calcula su complemento reverso.
- **Geologia:** Instala GeoPandas. Carga un shapefile y plotea un mapa basico.
- **Ciencia de datos:** Kaggle. Elegi un dataset y hace un analisis exploratorio con pandas y seaborn.

»»Siempre: practica en Codewars o Exercism

»»Todos los dias 15 minutos. No importa el nivel, importa la constancia.

»”Ejercicios finales (20 para que no te duermas)

»”Ahora que tenes el mapa, pone manos a la obra. Aca van 20 tareas concretas.

1. Busca la documentacion oficial de `os.walk()`, lee el ejemplo y escribe un script que recorra todas las carpetas de tu computadora y cuente cuantos archivos `.py` hay.
2. Entra a Codewars, resuelve la kata “Even or Odd” (nivel 8) y despues la kata “Find the smallest integer in the array”.
3. Crea un entorno virtual nuevo, instala ‘requests’ y escribe un script que consulte la API de <https://jsonplaceholder.typicode.com/posts> y muestre los titulos de los primeros 5 posts.
4. Elige un canal de YouTube de los que te recomiende, mira un video y despues escribe un resumen de lo que aprendiste (en un archivo `.txt`).
5. Busca en Stack Overflow alguna pregunta sobre Python que puedas responder. Intenta ayudar a alguien. Vas a ver que explicar tambien ensena.
6. Instala NumPy y crea un array de 10 numeros aleatorios. Calcula su media, desviacion estandar y la suma acumulada.
7. Instala SymPy y resuelve la integral de `x**2 * sin(x)`.
8. Instala matplotlib y grafica la funcion `sin(x)` para `x` entre 0 y `4*pi`.
9. Descarga un archivo CSV de ejemplo (ej: de Kaggle) y cargalo con pandas. Muestra las primeras 5 filas y la informacion general.
10. Usando GeoPandas, carga un mapa de tu pais (podes descargar shapefiles gratis) y plotealo.
11. Con Biopython, lee una secuencia FASTA de ejemplo e imprime su longitud y el porcentaje de GC.
12. Crea un script que resuelva ecuaciones cuadraticas. Usa manejo de excepciones para raices complejas.
13. Escribe una funcion que genere una matriz identidad de tamano `N` usando NumPy.
14. Hace una simulacion simple de un pendulo con `matplotlib.animation`.
15. Instala ‘pytest’ y escribe pruebas para tu funcion de suma y resta.
16. Configura ‘black’ y ‘flake8’ en tu editor. Toma un script viejo y formatealo.
17. Crea un repositorio en GitHub y subi todos los ejercicios del curso.
18. Seguí a “Dr. Chuck” en Coursera y mira el primer video de Python for Everybody.
19. Unite al Discord de Python y saluda en el canal ‘introductions’.
20. Elegi un proyecto de tu disciplina (ej: procesar datos sismicos, analizar secuencias de proteinas, calcular orbitas) y empieza a escribirlo. No importa si es chico, terminalo.

»»Resumen de lo que vimos (y de lo que te espera)

»»Dale, repasemos rapido lo importante:

- La documentacion oficial (docs.python.org) es tu mejor amiga.
- Librerias cientificas: NumPy, SciPy, SymPy, Biopython, GeoPandas.
- Libros: “Automate the Boring Stuff” para empezar, “Fluent Python” para profundizar.
- Cursos gratis: Python for Everybody (Coursera) y CS50 de Harvard.
- Canales de YouTube: Pildoras Informaticas (espanol) y Corey Schafer (ingles).
- Comunidades: r/learnpython y Python Discord.
- Herramientas: VS Code + black + isort + flake8 + Jupyter.
- Practica: Codewars, Exercism, HackerRank.
- Plan de estudio: refuerzo, herramientas, elige un camino científico, constancia.

»»Tabla de referencia rapida de comandos utiles

Que quiero hacer	Comando
Crear entorno virtual	<code>python -m venv mi_env</code>
Activar entorno (Windows)	<code>mi_env\Scripts\activate</code>
Activar entorno (Linux/Mac)	<code>source mi_env/bin/activate</code>
Instalar un paquete	<code>pip install nombre_paquete</code>
»» Guardar dependencias	<code>pip freeze >requirements.txt</code>
Instalar desde requirements	<code>pip install -r requirements.txt</code>
Formatear con black	<code>black script.py</code>
Ver errores de estilo	<code>flake8 script.py</code>
Ejecutar pruebas con pytest	<code>pytest</code>
Ver documentacion de una funcion	<code>python -m pydoc nombre_funcion</code>

»»Una ultima reflexion

»»Mira, escribir este libro fue un viaje. Y vos leerlo y programar tambien. Lo que aprendiste aca es solo el comienzo.

»»**No intentes aprender todo de una vez.** Elige un proyecto chico, terminalo, y despues pasa al siguiente. Un proyecto terminado vale mas que diez empezados.

»»**Comparti lo que haces.** Mostrale a un amigo, subilo a GitHub, escribe un post en LinkedIn. No importa si es una boludez, el acto de mostrarlo te obliga a mejorarlo.

»»**No te compares con otros.** En redes sociales vas a ver gente que hace cosas increíbles. Ellos tambien empezaron con un `'print("Hola mundo")'`.

»””**Disfruta el proceso.** Programar es difícil, pero también es muy divertido. Cuando algo finalmente funciona después de horas de frustración, esa sensación no tiene precio.

»””_____ “Nunca pares de
programar. Nunca pares de aprender. Y cada vez que puedas, ayuda a alguien que está donde vos
estabas ayer.”

»””Gracias por leer. Ahora anda, abre tu editor, y escribe código. El mundo necesita lo que vos
sabes hacer.

»»»Tablas de Referencia: Todo lo que aprendiste en un solo lugar

»»»Querido lector, esto es para que no te pierdas

»»»Cuando yo empecé a programar, pasaba más tiempo buscando en Google que escribiendo código. Me olvidaba si el método para agregar a una lista era `add()` o `append()`. No recordaba la sintaxis de un `try-except`. Tenía que volver a leer el manual cada dos por tres.

»»»Por eso armé este capítulo: una colección de tablas con lo más importante de cada tema. Es como tener un acordeón donde está todo resumido. Lo puedes usar para repasar rápido, para buscar una sintaxis que se te escapó, o para tener a mano mientras programas.

»»»Vamos por partes. Use las referencias que aparecen en el índice de tu libro y las complete con todo lo que vimos.

»»»Variables y Tipos de Datos

Tipo	Ejemplo	Que guarda
<code>'int'</code>	<code>'edad = 25'</code>	Números enteros
<code>'float'</code>	<code>'pi = 3.1416'</code>	Números decimales
<code>'str'</code>	<code>'nombre = "Nahuel"</code>	Texto (cadenas)
<code>'bool'</code>	<code>'es_mayor = True'</code>	Verdadero/Falso
<code>'list'</code>	<code>'[1, 2, 3]'</code>	Secuencia mutable
<code>'tuple'</code>	<code>'(1, 2, 3)'</code>	Secuencia inmutable
<code>'dict'</code>	<code>'{a: 1}'</code>	Pares clave-valor
<code>'set'</code>	<code>'1, 2, 3'</code>	Colección sin orden y sin repetidos

»»»**Reflexion:** Cuando no sabes que tipo es una variable, usa `'type(variable)'`.

»»Operadores Aritmeticos

Operador	Significado	Ejemplo
'+'	Suma	'5 + 3' = 8
'-'	Resta	'5 - 3' = 2
'*'	Multiplicacion	'5 * 3' = 15
'/'	Division (float)	'5 / 2' = 2.5
'//'	Division entera	'5 // 2' = 2
'**'	Exponente	'5 ** 2' = 25

»»Operadores de Comparacion y Logicos

Operador	Significado
'=='	Igual que
'!='	Distinto que
'<', '>', '<=', '>='	Menor, mayor, menor o igual, mayor o igual
'and'	Y logico (ambos True)
'or'	O logico (al menos uno True)
'not'	Negacion (invierte)

»»Estructuras Condicionales

```

>>> if condicion:
>>>     # codigo
>>>
>>> if condicion:
>>>     # codigo
>>> else:
>>>     # codigo
>>>
>>> if cond1:
>>>     # codigo
>>> elif cond2:
>>>     # codigo
>>> else:
>>>     # codigo
    
```

»»Bucles

»»Bucle 'while'

```

>>> while condicion:
>>>     # codigo
>>>     # cuidado con el loop infinito
    
```


»»Bucle 'for'

```

>>>for elemento in iterable:
>>>    # codigo
>>>
>>>for i in range(10):      # 0..9
>>>for i in range(2, 10):   # 2..9
>>>for i in range(0, 10, 2):# 0,2,4,6,8
    
```

»»'break' y 'continue'

```

>>>while True:
>>>    if cond:
>>>        break      # sale del bucle
>>>    if otra_cond:
>>>        continue   # salta al proximo ciclo
    
```

»»Listas: Metodos Esenciales

Metodo	Que hace
'lista.append(x)'	Agrega x al final
'lista.insert(i, x)'	Inserta x en la posicion i
'lista.extend(iterable)'	Agrega todos los elementos del iterable
'lista.pop(i)'	Elimina y devuelve el elemento en i (default ultimo)
'lista.remove(x)'	Elimina la primera aparicion de x
'lista.clear()'	Vacía la lista
'lista.sort()'	Ordena la lista (modifica)
'sorted(lista)'	Devuelve una nueva lista ordenada
'lista.reverse()'	Invierte la lista (modifica)
'lista.index(x)'	Devuelve posicion de la primera aparicion de x
'lista.count(x)'	Cuenta cuantas veces aparece x
'x in lista'	Pregunta si x pertenece a la lista

»»»Diccionarios: Metodos Esenciales

Metodo	Que hace
»»» 'dict[clave] = valor'	Asigna o actualiza
'dict.get(clave, default)'	Devuelve valor o default si no existe
'dict.setdefault(clave, default)'	Como get pero si no existe la crea con default
'dict.update(otro_dict)'	Actualiza con pares de otro diccionario
'dict.pop(clave)'	Elimina y devuelve el valor
'dict.popitem()'	Elimina y devuelve el ultimo par (Python 3.7+)
'dict.keys()'	Vista de claves
'dict.values()'	Vista de valores
'dict.items()'	Vista de pares (clave, valor)
'clave in dict'	Pregunta si existe la clave

»»»Strings: Metodos que usas todo el tiempo

Metodo	Que hace
»»» 's.lower()' / 's.upper()'	Minusculas / mayusculas
's.capitalize()'	Primera letra mayuscula
's.title()'	Primera letra de cada palabra mayuscula
's.strip()' / 'lstrip()' / 'rstrip()'	Elimina espacios (o caracteres) al inicio/fin
»»» 's.replace(viejo, nuevo)'	Reemplaza subcadenas
's.split(separador)'	Divide en lista por separador
'separador.join(lista)'	Une lista con separador
's.find(sub)'	Posicion de sub (o -1)
's.count(sub)'	Cuantas apariciones de sub
's.startswith(prefix)' / 's.endswith(suffix)'	Verifica comienzo / final
's.isdigit()' / 's.isalpha()' / 's.isalnum()'	Verifica tipo de caracteres

»»»Tuplas (inmutables)

- Creacion: 'tupla = (1, 2, 3)' o 'tupla = 1, 2, 3'
- Tupla de un elemento: '(42,)' (la coma es obligatoria)
- Acceso: 'tupla[0]' igual que listas
- No se puede modificar: no 'append', no 'remove', no asignacion
- Metodos: 'count()', 'index()'
- Empaquetado y desempaquetado: 'a, b = (1, 2)'
- Sirven como claves de diccionario (las listas no)

»»Sets (conjuntos)

Operacion	Sintaxis
Union	'A B' o 'A.union(B)'
Interseccion	'A
B' o 'A.intersection(B)'	
Diferencia	'A - B' o 'A.difference(B)'
Diferencia simetrica	'A ^ B' o 'A.symmetric_difference(B)'
Subconjunto	'A <= B' o 'A.issubset(B)'
Agregar elemento	's.add(x)'
Eliminar (si existe)	's.discard(x)'
Eliminar (error si no existe)	's.remove(x)'
Eliminar y devolver uno	's.pop()'
Vaciar	's.clear()'

»»Comprension de listas, diccionarios y sets

```

>>># Listas
>>>[n**2 for n in range(10) if n % 2 == 0]
>>>
>>># Sets
>>>{letra for letra in "python" if letra in "aeiou"}
>>>
>>># Diccionarios
>>>{n: n**2 for n in range(5)}

```

»»Funciones

```

>>>def mi_funcion(param1, param2=default):
>>>    """Docstring: explica que hace"""
>>>    # codigo
>>>    return resultado
>>>
>>># Llamada
>>>resultado = mi_funcion(arg1, arg2)
>>>
>>># Argumentos variables
>>>def funcion(*args, **kwargs):
>>>    pass

```

»»»Modulo 'os' (lo mas usado)

Funcion	Que hace
'os.getcwd()'	Directorio actual
'os.chdir(ruta)'	Cambia de directorio
'os.listdir(ruta)'	Lista contenido
'os.mkdir(carpetas)'	Crea una carpeta
'os.makedirs(carpetas, exist_ok = True)'	Crea carpetas intermedias
'os.remove(archivo)'	Borra archivo
'os.rmdir(carpetas)'	Borra carpeta vacia
'os.rename(orig, dest)'	Renombra o mueve
'os.path.exists(ruta)'	Verifica existencia
'os.path.isfile(ruta)'	Es archivo?
'os.path.isdir(ruta)'	Es carpeta?
'os.path.join(a, b)'	Une rutas correctamente
'os.path.basename(ruta)'	Nombre del archivo
'os.path.dirname(ruta)'	Directorio padre
'os.path.splitext(ruta)'	Separa nombre y extension

»»»Modulo 'sys'

Atributo / Funcion	Que hace
'sys.argv'	Lista de argumentos de linea de comandos
'sys.exit(codigo)'	Termina el programa (0 exito)
'sys.path'	Lista de rutas de importacion
'sys.stdin'	Entrada estandar (para pipes)
'sys.stdout'	Salida estandar
'sys.stderr'	Salida de error
'sys.version'	Version de Python
'sys.platform'	Sistema operativo

»»»Modulo 'math'

Funcion / Constante	Que hace
'math.pi', 'math.e'	Constantes
'math.sqrt(x)'	Raiz cuadrada
'math.pow(x, y)'	Potencia (float)
'math.exp(x)'	e**x
'math.log(x, base)'	Logaritmo
'math.log10(x)', 'log2(x)'	Log base 10 y 2
'math.sin(x)', 'cos(x)', 'tan(x)'	Trigonometricas (radianes)
'math.radians(grad)'	Grados a radianes
'math.degrees(rad)'	Radianes a grados
'math.ceil(x)', 'floor(x)'	Redondeo arriba/abajo
'math.factorial(n)'	Factorial
'math.comb(n, k)', 'perm(n, k)'	Combinaciones, permutaciones
'math.hypot(x, y)'	Hipotenusa (distancia euclidiana)
'math.isclose(a, b)'	Compara con tolerancia

»»»Modulo 'datetime'

```

>>>from datetime import date, time, datetime, timedelta
>>>
>>>hoy = date.today()
>>>ahora = datetime.now()
>>>
>>># crear
>>>fecha = date(2024, 12, 25)
>>>momento = datetime(2024, 12, 25, 20, 30)
>>>
>>># sumar/restar dias
>>>manana = hoy + timedelta(days=1)
>>>
>>># diferencia
>>>diferencia = fecha2 - fecha1
>>>
>>># formatos
>>>fecha.strftime("%d/%m/%Y")
>>>datetime.strptime("25/12/2024", "%d/%m/%Y")

```

»»»Modulo 're' (expresiones regulares)

```

>>>import re
>>>
>>># buscar primera coincidencia
>>>re.search(r"\d+", texto)

```

```

>>>
>>># encontrar todas
>>>re.findall(r"\d+", texto)
>>>
>>># reemplazar
>>>re.sub(r"\d", "#", texto)
>>>
>>># compilar
>>>patron = re.compile(r"\d+")
>>>patron.findall(texto)
>>>
>>># metacaracteres: \d \w \s . + * ? {n,m} ^ $ [aeiou] (grupo)
>>># banderas: re.IGNORECASE

```

»»»Modulo 'json'

```

>>>import json
>>>
>>># dict -> string
>>>json_str = json.dumps(datos, indent=2, ensure_ascii=False)
>>>
>>># dict -> archivo
>>>with open("datos.json", "w") as f:
>>>    json.dump(datos, f, indent=2)
>>>
>>># string -> dict
>>>datos = json.loads(json_str)
>>>
>>># archivo -> dict
>>>with open("datos.json") as f:
>>>    datos = json.load(f)

```

»»»Modulo 'csv'

```

>>>import csv
>>>
>>># leer como listas
>>>with open("arch.csv", newline='') as f:
>>>    reader = csv.reader(f)
>>>    for fila in reader: ...
>>>
>>># leer como diccionarios (primera fila encabezados)
>>>with open("arch.csv", newline='') as f:
>>>    reader = csv.DictReader(f)
>>>    for fila in reader: print(fila["nombre"])
>>>

```

```

>>># escribir listas
>>>with open("salida.csv", "w", newline='') as f:
>>>    writer = csv.writer(f)
>>>    writer.writerows(datos)
>>>
>>># escribir diccionarios
>>>writer = csv.DictWriter(f, fieldnames=["nombre","edad"])
>>>writer.writeheader()
>>>writer.writerows(lista_dict)

```

»»»Modulo 'pathlib' (moderno)

```

>>>from pathlib import Path
>>>
>>>ruta = Path("/home/usuario") / "documentos" / "archivo.txt"
>>>
>>># propiedades
>>>ruta.name, ruta.stem, ruta.suffix, ruta.parent
>>>
>>># existencia
>>>ruta.exists(), ruta.is_file(), ruta.is_dir()
>>>
>>># leer/escribir texto
>>>ruta.write_text("contenido", encoding="utf-8")
>>>contenido = ruta.read_text(encoding="utf-8")
>>>
>>># listar
>>>for archivo in Path(".").iterdir(): ...
>>>for py in Path(".").glob("*.py"): ...
>>>for todo in Path(".").rglob("*"): ...
>>>
>>># crear carpeta
>>>Path("nueva").mkdir(exist_ok=True)
>>>Path("a/b/c").mkdir(parents=True, exist_ok=True)
>>>
>>># eliminar
>>>archivo.unlink()
>>>carpeta.rmdir() # solo vacia

```

»»»Modulo 'shutil' (alto nivel)

```

>>>import shutil
>>>
>>># copiar archivos
>>>shutil.copy(origen, destino)
>>>shutil.copy2(origen, destino) # preserva metadatos

```

```

>>>
>>># copiar carpetas
>>>shutil.copytree(origen, destino, dirs_exist_ok=True)
>>>
>>># mover
>>>shutil.move(origen, destino)
>>>
>>># borrar carpetas con contenido (CUIDADO)
>>>shutil.rmtree(carpetas)
>>>
>>># comprimir
>>>shutil.make_archive("backup", "zip", "carpetas")
>>>
>>># descomprimir
>>>shutil.unpack_archive("backup.zip", "extraccion")
>>>
>>># espacio en disco
>>>uso = shutil.disk_usage("/")

```

»»»Modulo 'copy': copia superficial vs profunda

```

>>>import copy
>>>
>>># asignacion: NO copia, solo referencia
>>>lista2 = lista1
>>>
>>># copia superficial
>>>copiasuperficial = lista1.copy()
>>>copiasuperficial = copy.copy(lista1)
>>>copiasuperficial = lista1[:]
>>>
>>># copia profunda (para anidados)
>>>copiaprofunda = copy.deepcopy(lista1)

```

»»»Modulo 'random' (lo basico)

```

>>>import random
>>>
>>>random.random()          # float 0..1
>>>random.randint(a, b)      # entero a..b inclusive
>>>random.choice(lista)      # elige un elemento
>>>random.shuffle(lista)     # desordena la lista (modifica)
>>>random.sample(poblacion, k) # k elementos sin repeticion

```


»»»Modulo 'collections'

```

>>>from collections import Counter, defaultdict, deque
>>>
>>># Counter
>>>contador = Counter(lista)
>>>mas_comunes = contador.most_common(3)
>>>
>>># defaultdict
>>>dd = defaultdict(int)    # valor por defecto 0
>>>dd = defaultdict(list)   # valor por defecto []
>>>
>>># deque (cola rapida)
>>>cola = deque([1,2,3])
>>>cola.appendleft(0)
>>>cola.popleft()
>>>cola.rotate(2)
>>>cola_max = deque(maxlen=5)
    
```

»»»Manejo de Excepciones

```

>>>try:
>>>    # codigo que puede dar error
>>>except TypeError as e:
>>>    # manejar error especifico
>>>except (OtroError, TercerError):
>>>    # varios
>>>except Exception as e:
>>>    # cualquier otro error
>>>else:
>>>    # se ejecuta si no hubo excepcion
>>>finally:
>>>    # siempre se ejecuta (limpiar recursos)
>>>
>>># lanzar excepcion
>>>raise ValueError("mensaje")
>>>
>>># excepcion personalizada
>>>class MiError(Exception):
>>>    pass
    
```

»»»Logging (para programas serios)

```

>>>import logging
>>>
    
```

```

>>> logging.basicConfig(level=logging.INFO,
>>>                        format='%(asctime)s - %(levelname)s - %(message)s',
>>>                        filename='app.log')
>>>
>>> logging.debug("detalle")
>>> logging.info("info general")
>>> logging.warning("alerta")
>>> logging.error("error")
>>> logging.critical("critico")
>>>
>>> # dentro de except: guarda la traza
>>> try:
>>>     1/0
>>> except:
>>>     logging.exception("division por cero")
    
```

»»» pdb - depurador

```

>>> # poner en el codigo
>>> breakpoint() # Python 3.7+
>>>
>>> # Comandos en (Pdb):
>>> n # next (siguiente linea, sin entrar a funciones)
>>> s # step (entra a funciones)
>>> c # continue (sigue hasta proximo breakpoint)
>>> p variable # imprime valor
>>> pp variable # pretty print
>>> l # listar codigo alrededor
>>> b 10 # breakpoint en linea 10
>>> q # quit
    
```

»»» PEP 8 - Reglas Clave

- Variables y funciones: 'snake_case'
- Clases: 'CamelCase'
- Constantes: 'MAYUSCULAS'
- Indentacion: 4 espacios (nunca tabs)
- Lineas: maximo 79 caracteres
- Espacios alrededor de operadores: 'x = y + z'
- Dos lineas en blanco entre funciones de nivel superior
- Imports: estandar, terceros, locales (orden alfabetico)

»»»Docstrings

```

>>>def funcion(a, b):
>>>    """Resumen de una linea."""
>>>    return a + b
>>>
>>>def funcion_compleja(x, y):
>>>    """
>>>    Descripcion mas larga.
>>>
>>>    Args:
>>>        x (int): primer numero
>>>        y (int): segundo numero
>>>
>>>    Returns:
>>>        int: suma de x e y
>>>
>>>    Example:
>>>        >>> funcion_compleja(2,3)
>>>        5
>>>    """
>>>    return x + y
    
```

»»»Punto de entrada y argumentos de terminal

```

>>>import sys
>>>
>>>def main():
>>>    if len(sys.argv) > 1:
>>>        print(f"Hola {sys.argv[1]}")
>>>    else:
>>>        print("Uso: python script.py <nombre>")
>>>        sys.exit(1)
>>>
>>>if __name__ == "__main__":
>>>    main()
    
```

»»»Entornos virtuales (venv)

```

>>># crear
>>>python -m venv mi_entorno
>>>
>>># activar (Linux/Mac)
>>>source mi_entorno/bin/activate
>>>
    
```

```

»'''# activar (Windows)
»'''mi_entorno\Scripts\activate
»'''
»'''# instalar paquetes
»'''pip install requests
»'''
»'''# guardar dependencias
»'''pip freeze > requirements.txt
»'''
»'''# recrear entorno
»'''pip install -r requirements.txt
»'''
»'''# desactivar
»'''deactivate
    
```

»'''PyInstaller - crear ejecutables

```

»'''# instalacion
»'''pip install pyinstaller
»'''
»'''# comando basico
»'''pyinstaller mi_script.py
»'''
»'''# un solo archivo
»'''pyinstaller --onefile mi_script.py
»'''
»'''# sin consola (aplicacion grafica)
»'''pyinstaller --windowed mi_script.py
»'''
»'''# con icono
»'''pyinstaller --onefile --icon=mi_icono.ico mi_script.py
    
```

»'''Errores Comunes (y como evitarlos)

Error	Solucion
'NameError'	Revisa que la variable este definida antes de usarla
'TypeError'	Verifica los tipos de datos (no sumes string con int)
'IndexError'	Asegurate que el indice este dentro del rango de la lista
'KeyError'	Usa 'dict.get(clave, default)' o pregunta 'if clave in dict'
'AttributeError'	Confirma que el objeto tenga el metodo que intentas usar
'FileNotFoundError'	Verifica la ruta del archivo y los permisos
'ZeroDivisionError'	Controla que el divisor no sea cero antes de dividir

»»»Proyectos Integradores (recordatorio)

»»»Algunos de los proyectos que hiciste o puedes hacer:

- Sistema de gestion de alumnos y notas
- Carrito de compras
- Cajero automatico
- Calculadora con menu
- Juego del Ahorcado
- Piedra, papel o tijera
- Generador de contraseñas
- Explorador de archivos
- Agenda telefonica

»»»Un resumen personal (y una historia)

»»»Mira, yo arranque con `'print("Hola mundo")'` y no tenia idea de lo que era una lista. Me pasaba horas mirando la pantalla sin saber por que mi `'for'` no funcionaba. Pero fui anotando en un cuaderno las cosas basicas: los metodos de listas, como se hacia un `'try-except'`, que hacia `'sys.argv'`.

»»»Este capitulo es ese cuaderno. No pretendas memorizarlo todo. Usalo como referencia cada vez que te trabes. Con el tiempo, las cosas se vuelven automaticas.

»»»Ejercicios finales para que pruebes lo que sabes

1. Sin mirar las tablas, escribi los metodos de listas que recuerdes.
2. Escribe una funcion que reciba una lista de numeros y devuelva una nueva lista solo con los pares (usando comprension).
3. Usa `'pathlib'` para listar todos los archivos `'txt'` de tu carpeta actual y mostrar su tamano.
4. Crea un diccionario con tres personas y sus edades. Luego usa `'items()'` para imprimir "Nombre tiene X anos".
5. Escribe un bloque `'try-except'` que capture `'ZeroDivisionError'` y muestre "No se puede dividir por cero".
6. Usa `'sys.argv'` para hacer un script que reciba dos numeros y muestre su suma.

»»Una ultima reflexion

»»Aprendiste desde variables hasta crear ejecutables. Eso es un monton. Ahora el secreto esta en practicar todos los dias, aunque sean 15 minutos. Anda a Codewars, resolve problemas de Project Euler, o hace tu propio proyecto.

»»No te compares con nadie. Cada quien tiene su ritmo. Lo importante es que disfrutes el proceso.

»»Y recuerda: si te trabas, las tablas estan aca. Abri esta pagina, busca lo que necesitas, y segui adelante.

»» _____ *El codigo se escribe linea por linea.*

El conocimiento se construye concepto por concepto.


```
git add .
npm run dev
```

```
> git add .
> demands mannalis
> synbrlats use meta
```

SOBRE ESTE CUADERNO

Este no es un libro de teoría para leer pasivamente.

Es un cuaderno de trabajo para aprender haciendo.

Cada ejercicio está pensado para que lo resuelvas.

Cada error, para que aprendas de él.

```
> def greet(name);
print("BUILDING
MODULES...")
> ptt
> mp/WHILE
cy/commeth
print("moà mundo!")
```

PRÓXIMOS PASOS

Cuando termines este cuaderno:

1. Habrás escrito tu primer programa.
2. Entenderás variables, bucles y funciones.
3. Estarás listo para seguir con automatización, análisis de datos o desarrollo web.

Para seguir practicando, podés descargar el libro **PYTHON HANDS ON** por Rivero y Villafañe.

CONTACTO

¿Preguntas, sugerencias o encontraste un error?

Instagram: [clasesconnahu](#)

```
if value == 5:
> value = "HULE"
> else:
> echo "BUILDING MODULES..."
> docker compose up -d
> docker compose up -d
> python cmamo compose
> curl https://api.data.io
```